The Production-Ready Engineer

Volume 2: Build, Ship and Cloud

THE PRINT SET

The Production-Ready Engineer is published in six volumes, cut at Part boundaries so that each one stands on its own and binds cleanly. Page numbers run continuously through the set, and the index in Volume 6 covers all of it.

1	**Foundations** Base Camp and Part I: DevOps, Linux, Networking, Git	pp. 1–734
2	**Build, Ship and Cloud** **Parts II and III: Docker, CI/CD, AWS, Terraform**	**pp. 735–1544**
3	**Orchestrate, See and Secure** Parts IV and V: Kubernetes, Helm, Observability, DevSecOps	pp. 1545–2318
4	**Under the Hood** Part VI: Advanced Networking, Databases and Distributed Systems	pp. 2319–2814
5	**Reliability and Platforms at Scale** Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering	pp. 2815–3548
6	**Senior Altitude and the Summit** Parts IX and X, Appendices and Index: Architecture, Cost, the Nine Projects	pp. 3549–3972

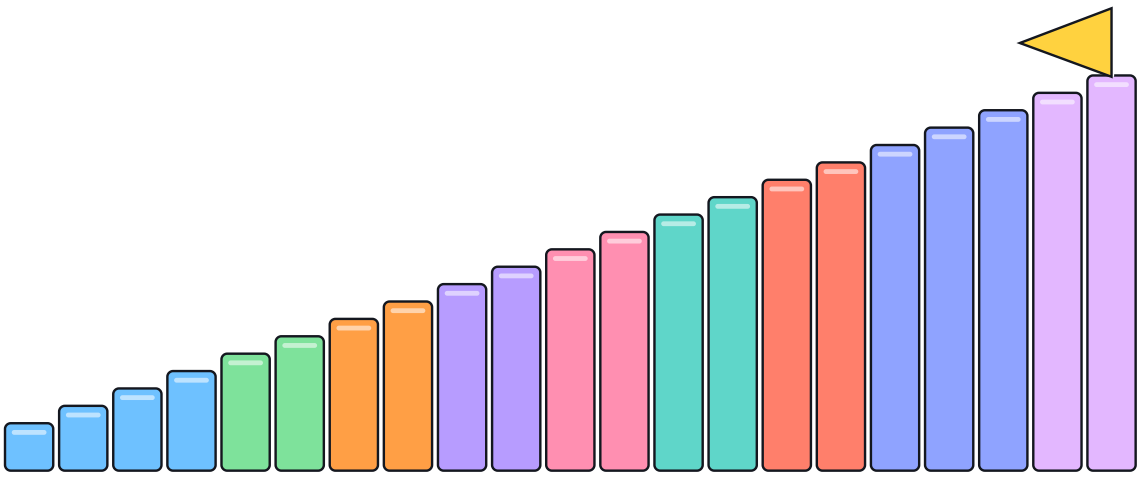
THE COMPLETE DEVOPS ROADMAP

The Production-Ready Engineer

The complete DevOps roadmap, from your first Linux command to senior architect

Volume 2 of 6: Build, Ship and Cloud

Ubong Eduok



github.com/mascot200/devops-roadmap

The Production-Ready Engineer
The complete DevOps roadmap, from your first Linux command to senior architect
Volume 2 of 6: Build, Ship and Cloud

Copyright © 2026 Ubong Eduok.

This book is released under the MIT License; the full text is printed in Appendix D, in Volume 6. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept.

Everything here is provided "as is", without warranty of any kind. Run the labs in sandboxes. Read the cost warnings before using a real cloud account, and finish each lab with its teardown steps. Commands that change systems can cause damage or charges; you are responsible for where you run them.

Product and company names are trademarks of their respective owners and are used here only to identify and teach their technology. Their use implies no affiliation or endorsement. Tool icons in the author portrait are from Devicon (MIT licence).

Tools change quickly. Command output in this book is illustrative, and version numbers will age. When in doubt, check the official documentation named in each chapter's further reading.

Source: github.com/mascot200/devops-roadmap, typeset from commit faf55fa.
First edition, 2026.

Do not aim to become the person who knows 50 DevOps tools.

Aim to become the engineer who can take a software application and independently make it production-ready.

Secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

How to Read This Book

The anatomy of a chapter

All 248 topic chapters follow the same eleven sections, which fall into three movements. Appendix A, in Volume 6, prints the template that every chapter was written against.

UNDERSTAND	1	**What it is**	A plain-English explanation, with every acronym defined on first use.
	2	**Why it matters**	What breaks, costs money, or wakes someone at 3 a.m. when this is misunderstood.
	3	**Real-world system**	How the topic shows up in a named, publicly documented system.
	4	**Diagram**	The real mechanism: a flow, a sequence, an architecture or a state machine.
	5	**How it works**	Key concepts, comparison tables and the commands that matter.
DO	6	**Step-by-step guide**	A lab you can type in a sandbox, with what to look for at each step.
	7	**Real-world scenario**	A short story under pressure: symptom, investigation, fix, lesson.
	8	**Mistakes and practices**	Two short lists: what not to do, and what to do.
PROVE	9	**Tasks**	Hands-on work, easiest first, each with a "done when" check. The last says Prove it.
	10	**Self-check**	Questions a senior engineer would expect you to answer without notes.
	11	**Further reading**	Official documentation and well-known books.

The eleven sections of every topic chapter

Each level opens with an overview of its goal, recommended time and topics, and closes with a **capstone project** marked with a star, which pulls all of that level's topics together.

Conventions used in print

- **Cross-references.** Where the repository has a hyperlink, this book prints a small tag after the words: Docker Compose §4.8 means Level 4, topic 8. Running heads carry the same numbers, so you can flip straight to it. In a PDF the tags are clickable when the target is in the same volume.
- **Check-boxes.** Every task has a box to tick with a pencil. Where the text mentions changing `- [ ]` to `- [x]`, that is the same idea for readers who keep their progress in a fork of the repository.

- **Answers.** On GitHub the self-check answers are folded away. In print they sit in a shaded box straight after the questions, so cover it with a card and answer first.
- **Code.** Long lines wrap, and the continuation is indented. Type a wrapped line as one line. Output is illustrative and yours may differ in the details.
- **Diagrams.** A few wide diagrams are printed sideways. Turn the book clockwise to read them.
- **Lab notes.** Chapters start on a right-hand page, and any left-hand page this leaves empty is ruled for your notes. Use them. For each task, write what you ran and what you saw. Explaining output is the skill.

Stay safe, and keep it cheap

Use disposable sandboxes: local virtual machines, Docker, kind or minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps. Never practise on a system you cannot afford to lose.

The Whole Climb on One Page



PART I: THE GROUND BENEATH EVERYTHING				
0	What DevOps Actually Is	1 week (reading + mapping)	13 topics	35
1	Linux Fundamentals	2-4 weeks	13 topics	213
2	Networking	2-4 weeks	11 topics	397
3	Git & GitHub	2-3 weeks	13 topics	563
PART II: BUILD IT, SHIP IT				
4	Docker	2-3 weeks	14 topics	737
5	CI/CD	2-3 weeks	12 topics	931
PART III: CLOUD YOU CAN REBUILD				
6	Cloud (AWS)	4-6 weeks	18 topics	1137
7	Terraform / Infrastructure as Code	2-3 weeks	9 topics	1403
PART IV: ORCHESTRATION				
8	Kubernetes	6-10 weeks	19 topics	1547
9	Helm	1-2 weeks	6 topics	1823
PART V: SEE IT, SECURE IT				
10	Observability	3-4 weeks	11 topics	1919
11	Security / DevSecOps	3-4 weeks	13 topics	2123
PART VI: UNDER THE HOOD				
12	Advanced Networking	3-4 weeks	14 topics	2321
13	Databases & Distributed Systems	4-6 weeks	15 topics	2577
PART VII: KEEP IT RUNNING				
14	SRE	3-4 weeks	10 topics	2817
15	Disaster Recovery	2-3 weeks	10 topics	2983
PART VIII: PLATFORMS AT SCALE				
16	Advanced Kubernetes	4-6 weeks	10 topics	3137
17	GitOps	2-3 weeks	7 topics	3303
18	Platform Engineering	4-6 weeks	10 topics	3403
PART IX: SENIOR ALTITUDE				
19	Architecture	Ongoing	11 topics	3551
20	Cost Optimization	2-3 weeks	9 topics	3703

Level numbers follow the roadmap document; the recommended learning order differs slightly (see Base Camp). The nine Summit Projects in Part X then chain the levels into one system.

Contents of Volume 2

PART II Build It, Ship It 735

4	**Docker**	737
4.1	How Docker Works Internally	739
4.2	Images and Layers	751
4.3	Containers and Their Lifecycle	761
4.4	Registries	773
4.5	Volumes and Bind Mounts	785
4.6	Docker Networks	795
4.7	Dockerfile	805
4.8	Docker Compose	817
4.9	Image Caching and Layer Optimization	829
4.10	Multi-stage Builds	841
4.11	Non-root Containers	851
4.12	Health Checks	865
4.13	Minimal Base Images	877
4.14	Image Scanning and Container Security	889
★	Capstone: Containerize shop-api with Docker Compose	903
5	**CI/CD**	931
5.1	Designing a CI/CD Pipeline End to End	933
5.2	GitHub Actions for CI/CD	945
5.3	Automated Testing in Pipelines	959
5.4	Docker Image Builds and Registries in CI	971
5.5	Secrets and Environment Variables	987
5.6	Build Artifacts	1003
5.7	Rollback	1019
5.8	Zero-downtime Deployments	1033
5.9	Database Migrations in Pipelines	1049
5.10	Rolling Deployments	1067
5.11	Blue/Green Deployments	1081
5.12	Canary Deployments	1095
★	Capstone: Push-to-Production Pipeline for shop-api	1111

PART III Cloud You Can Rebuild 1135

6	**Cloud (AWS)**	1137
6.1	EC2	1141

6.2	Auto Scaling	1155
6.3	Lambda	1167
6.4	VPC and Subnets	1179
6.5	Route Tables, Internet Gateway and NAT Gateway	1191
6.6	Security Groups and Network ACLs	1203
6.7	IAM	1217
6.8	S3	1231
6.9	EBS and EFS	1245
6.10	RDS and Aurora	1261
6.11	ElastiCache	1273
6.12	Route 53	1285
6.13	CloudFront	1297
6.14	Application Load Balancer	1311
6.15	KMS	1325
6.16	Secrets Manager	1339
6.17	CloudTrail	1353
6.18	CloudWatch	1369
★	Capstone: shop-api on CloudFront, ALB, EC2, Redis, RDS and S3	1383
7	**Terraform / Infrastructure as Code**	1403
7.1	Providers and Resources	1405
7.2	Variables and Outputs	1419
7.3	Modules	1431
7.4	State and Remote Backends	1443
7.5	Data Sources and Dependencies	1457
7.6	Reusable Environments	1471
7.7	Terraform CI/CD	1485
7.8	State Locking	1499
7.9	Drift Detection	1513
★	Capstone: shop-api on AWS with Terraform	1527

PART II

Build It, Ship It

Package software so it runs the same everywhere, then automate the road from a commit to a safe release.



IN THIS PART

4	Docker	737
5	CI/CD	931

LAB NOTES What I ran, what I saw, what it means

LEVEL 4 · PART II

Docker

4

Goal: Go beyond basic container usage and understand how Docker works internally.

RECOMMENDED TIME	TOPICS
2-3 weeks	14

PRACTICE / CAPSTONE

Containerize a Node.js API, PostgreSQL, Redis, Nginx, Prometheus, Grafana and Loki using Docker Compose.

Topics

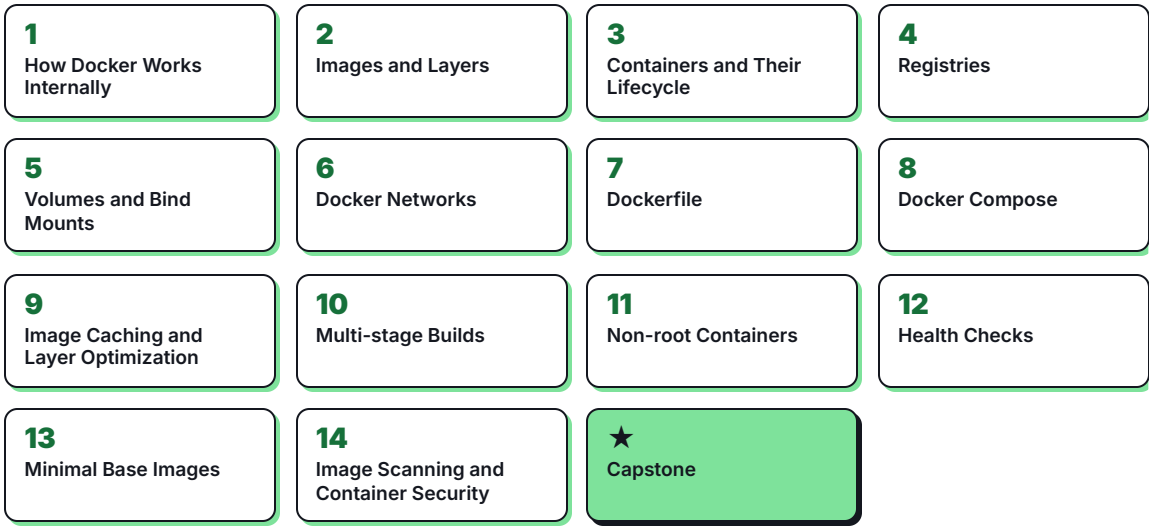
#	Topic	What you will learn
1	How Docker Works Internally **p.739**	namespaces, cgroups, union filesystems, containerd/runc, Docker vs VMs
2	Images and Layers **p.751**	image anatomy, content-addressable layers, tags vs digests
3	Containers and Their Lifecycle **p.761**	run/stop/exec/logs, PID 1, signals, restart policies
4	Registries **p.773**	Docker Hub, ECR, GHCR, tagging/pushing, private registries
5	Volumes and Bind Mounts **p.785**	named volumes, bind mounts, tmpfs, persistence and backups
6	Docker Networks **p.795**	bridge/host/none/overlay, DNS between containers, port publishing
7	Dockerfile **p.805**	instructions, CMD vs ENTRYPOINT, .dockerignore, ARG vs ENV
8	Docker Compose **p.817**	services, networks, volumes, depends_on with healthchecks, profiles, env files
9	Image Caching and Layer Optimization **p.829**	cache invalidation order, BuildKit cache mounts, size reduction
10	Multi-stage Builds **p.841**	build vs runtime stage, copying artifacts, target stages
11	Non-root Containers **p.851**	USER, file ownership, dropping capabilities, read-only rootfs
12	Health Checks **p.865**	HEALTHCHECK, liveness vs readiness concepts, Compose health gating

#	Topic	What you will learn
13	Minimal Base Images **p.877**	alpine vs slim vs distroless vs scratch, glibc/musl trade-offs
14	Image Scanning and Container Security **p.889**	Trivy/Grype, CVEs, SBOM, signing basics, runtime hardening
★	Level capstone project **p.903**	Containerize a Node.js API, PostgreSQL, Redis, Nginx, Prometheus, Grafana and Loki using Docker Compose.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



LEVEL 4: DOCKER

4.1 How Docker Works Internally

Topic 1 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

A Docker container is **not a tiny virtual machine**. It is an ordinary Linux process (or group of processes) that the kernel has been told to *lie to*: it sees its own process tree, its own network stack, its own hostname and its own root filesystem, and the kernel limits how much CPU and memory it may use. Docker is the friendly tooling around three kernel features:

- **Namespaces** — control what a process can *see* (PIDs, network interfaces, mounts, hostname, users...).
- **Control groups (cgroups)** — control how much a process can *use* (CPU, memory, I/O, number of processes).
- **Union filesystems (OverlayFS)** — stack read-only image layers plus one thin writable layer into a single `/`.

On top of those sit a few userspace components: the `docker` CLI, the Docker daemon (`dockerd`), `containerd` (the container lifecycle manager) and `runc` (the tiny program that actually calls the kernel to create the container). Image and runtime formats are standardised by the **Open Container Initiative (OCI)**, which is why an image built with Docker runs on Kubernetes, Podman or AWS Fargate.

2 Why it matters in DevOps

- **Debugging:** "container OOM-killed", "can't bind port", "permission denied on a volume" and "PID 1 ignores SIGTERM" are all kernel-feature problems. If you only know `docker run`, you can only guess.
- **Security:** a container shares the host kernel. A kernel exploit or an over-privileged container (`--privileged`, mounting `/var/run/docker.sock`) is a host compromise. A VM has a much harder boundary.
- **Capacity planning:** CPU and memory limits are cgroup settings. Without them one noisy container starves its neighbours.

- **Kubernetes:** Kubernetes does not need Docker Engine at all; it talks to `containerd` or CRI-O (which call `runc`). Knowing the layers tells you which tool to use on a node when `docker` is not installed.
- See also Processes, PIDs and Signals §1.8 — a container is just such a process.

3 Real-world system example

Kubernetes nodes and AWS Fargate/Lambda. Since Kubernetes 1.24 removed the built-in "dockershim", most clusters run **containerd** (or CRI-O) directly on each node; the kubelet asks containerd to create pods, and containerd calls an OCI runtime (normally `runc`) to create the namespaces and cgroups. The Docker daemon is not involved. Docker itself uses the same `containerd` underneath, which is why the two ecosystems are interchangeable at the image level.

For workloads where the shared-kernel boundary is not strong enough (multi-tenant serverless), cloud providers put a lightweight VM around the container: AWS Lambda and Fargate are built on **Firecracker** microVMs, and projects such as **gVisor** (user-space kernel) and **Kata Containers** (a small VM per container) exist for the same reason. The image format stays the same; only the isolation layer below the runtime changes.

4 Diagram

The path from your keyboard to a running process:

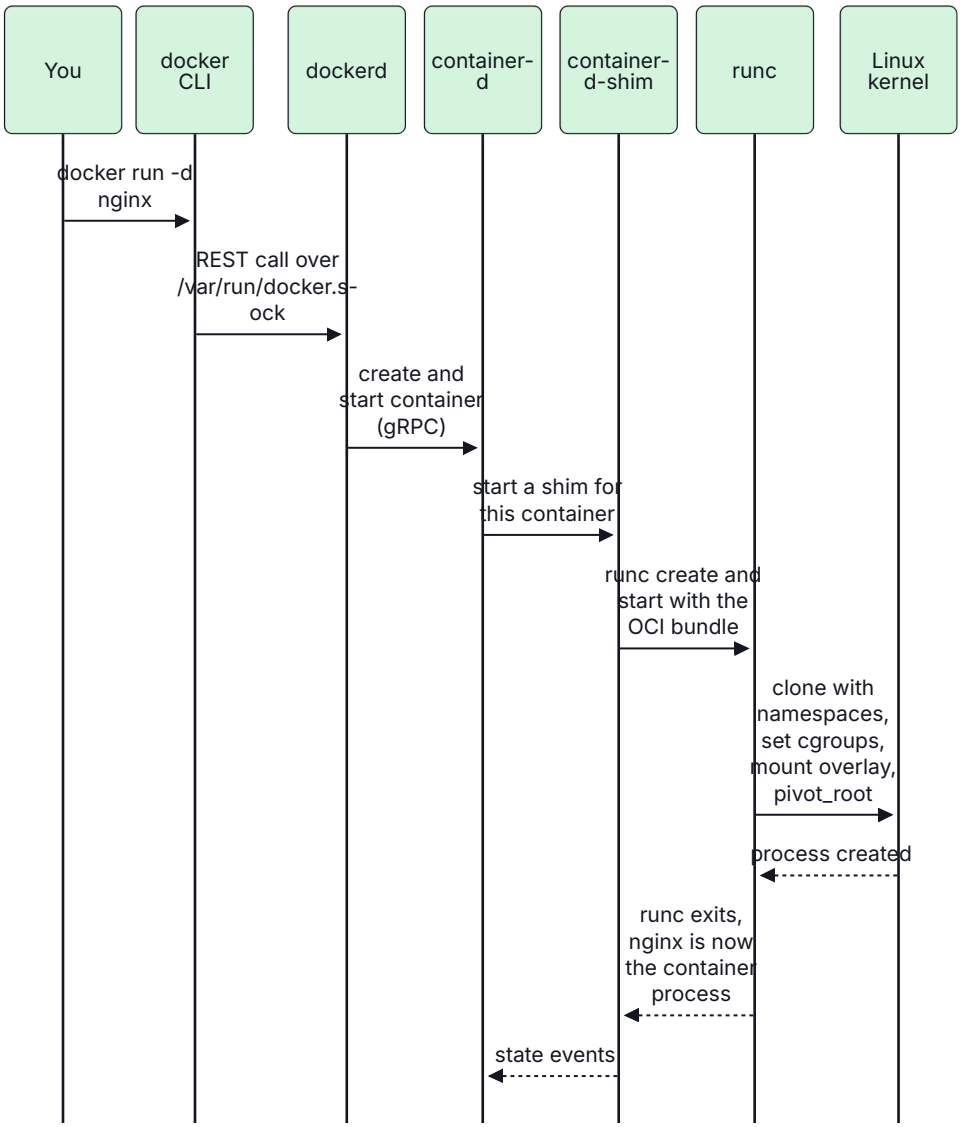


Figure 4.1-1

What the kernel builds for one container:

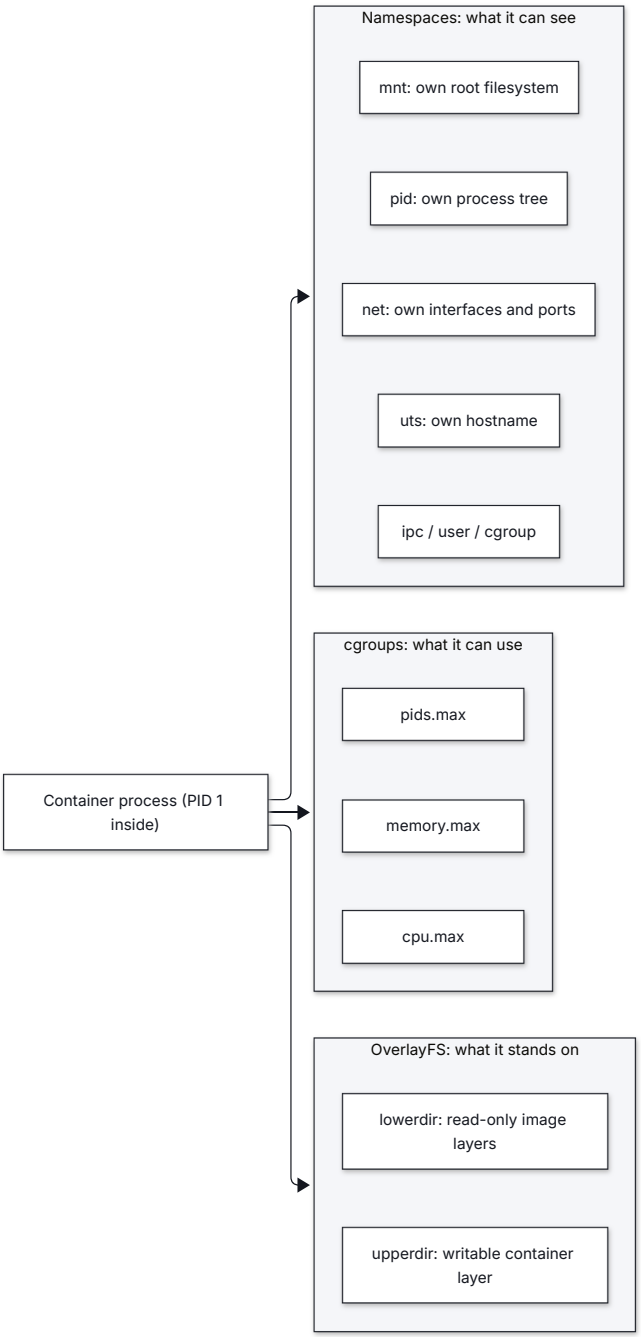


Figure 4.1-2

5 How it works

Namespaces (each is a file under `/proc/<PID>/ns/`):

Namespace	Isolates	Visible effect inside a container
`pid`	process IDs	the app is PID 1; it cannot see host processes
`net`	interfaces, routes, ports, iptables	its own `eth0`; port 80 inside does not clash with port 80 on the host
`mnt`	mount points	its own `/` built from the image
`uts`	hostname	hostname is the container ID by default
`ipc`	System V IPC, POSIX queues	no shared memory with other containers
`user`	UID/GID mapping	optional: root inside can map to an unprivileged user outside (user-namespace remapping, rootless mode)
`cgroup`	view of the cgroup tree	the container sees its own cgroup as the root

cgroups are a hierarchy of directories under `/sys/fs/cgroup` (cgroup v2 on modern distributions). Each directory has control files: `memory.max`, `cpu.max`, `pids.max`, and stat files such as `memory.current`. `docker run --memory 256m --cpus 0.5` just writes those files. Exceed `memory.max` and the kernel's OOM killer terminates a process in the cgroup (exit code 137 = 128 + SIGKILL). CPU limits *throttle* rather than kill.

Union filesystem (OverlayFS). An image is a stack of read-only directories (`lowerdir`). Docker adds an empty writable directory (`upperdir`) and mounts them together as the container's `/`. Reads fall through to the lower layers; the first write to a file **copies it up** into the upper layer (copy-on-write); a delete leaves a *whiteout* marker in the upper layer. When the container is removed the upper layer is discarded — which is why data you want to keep belongs in a volume §4.5.

The component stack:

Component	Job
`docker` CLI	Parses commands, calls the daemon's REST API (over a Unix socket by default)
`dockerd`	Images, networks, volumes, build (BuildKit), the user-facing API
`containerd`	Pulls and stores images, manages container lifecycle and snapshots
`containerd-shim`	One small process per container; keeps it running if `dockerd` or `containerd` restarts, relays stdio and exit status
`runc`	Reference OCI runtime: sets up namespaces, cgroups, mounts, drops capabilities, `exec`s your program, then exits

Containers vs VMs:

	Container	Virtual machine
Kernel	Shared with the host	Own guest kernel on virtualised hardware
Start time	Typically well under a second	Typically seconds to minutes
Overhead	A process plus a few MB	A full OS: GBs of disk, hundreds of MB of RAM
Isolation	Kernel features (weaker boundary)	Hypervisor (stronger boundary)
Good for	Packaging and scaling applications	Different OS kernels, hard multi-tenant isolation

On macOS and Windows, Docker Desktop runs a small Linux VM and your containers run inside it — because containers need a Linux kernel.

6 Step-by-step guide

Prerequisites: a Linux machine or VM where you have `sudo`, and Docker Engine for the later steps. Steps 1-3 need no Docker at all — they use `unshare` from `util-linux`. Output below was captured on a Linux host; PIDs and IDs will differ on yours.

1. **List the namespaces your shell lives in.**

```bash
ls -l /proc/self/ns
```

You see `cgroup`, `ipc`, `mnt`, `net`, `pid`, `time`, `user`, `uts`, each a link such as `pid:[4026531836]`. Two processes with the same number share that namespace.

2. **Create your own PID namespace without Docker.**

```bash
sudo unshare --pid --fork --mount-proc bash -c 'ps -ef; echo "my PID is $$"'
```

```
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 00:41 ?        00:00:00 bash -c ps -ef; echo "my PID is
  $$"
root           2       1  0 00:41 ?        00:00:00 ps -ef
my PID is 1
```

Your command became PID 1 and the host's processes vanished. That is the `pid` namespace, the whole trick.

3. **Change a hostname that only you can see (UTS namespace).**

```bash
sudo unshare --uts bash -c 'hostname demo; hostname'; hostname
```

The first prints `demo`; the second prints the real hostname — the host is unaffected.

4. **Build a union filesystem by hand with OverlayFS.**

```bash
mkdir -p /tmp/ov/{lower,upper,work,merged}
echo base  > /tmp/ov/lower/a.txt
echo hello > /tmp/ov/lower/b.txt
sudo mount -t overlay overlay \
  -o lowerdir=/tmp/ov/lower,upperdir=/tmp/ov/upper,workdir=/tmp/ov/work /tmp/ov/merged
echo changed | sudo tee /tmp/ov/merged/a.txt   # copy-up
sudo rm /tmp/ov/merged/b.txt                    # whiteout
ls -l /tmp/ov/upper
cat /tmp/ov/lower/a.txt
ls /tmp/ov/merged
sudo umount /tmp/ov/merged
```

```
-rw-r--r-- 1 root root    8 ... a.txt
c--------- 2 root root 0, 0 ... b.txt
base
a.txt
```

`a.txt` was copied into `upper` and modified there; the lower copy still says `base`. The deleted `b.txt` is a character device with major/minor `0, 0` — a *whiteout*. This is exactly what Docker does with image layers.

5. **Run a container and find it from the host.** (Needs Docker.)

```bash
docker run -d --name web --memory 128m --cpus 0.5 nginx:1.27
docker top web                                  # host PIDs of the container's
  processes
PID=$(docker inspect -f '{{.State.Pid}}' web)
sudo ls -l /proc/$PID/ns                        # different numbers from your shell's
  namespaces
```

The nginx master process has an ordinary host PID. The namespace link targets differ from step 1 for `mnt`, `net`, `pid`, `uts`, `ipc` (and `cgroup`).

6. **Read the limits you set from the cgroup filesystem.**

```bash
docker exec web cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/cpu.max
```

```
134217728
50000 100000
```

128 MiB in bytes, and 50 ms of CPU per 100 ms period (0.5 CPU). *(These paths are for cgroup v2; on a cgroup v1 host the files live under* `/sys/fs/cgroup/memory/` *and* `/sys/fs/cgroup/cpu/` *.)*

7. **Look at the overlay Docker mounted.**

```bash
docker inspect -f '{{.GraphDriver.Name}}' web
docker inspect -f '{{json .GraphDriver.Data}}' web | python3 -m json.tool
```

With the classic overlay2 storage driver you see `LowerDir`, `UpperDir`, `WorkDir`, `MergedDir`. (Recent Docker versions may default to the containerd image store, where the driver reports as `overlayfs` and the fields differ — the idea is identical.)

8. **Meet the runtime layers.**

```bash
ps -eo pid,ppid,cmd --forest | grep -E 'dockerd|containerd|nginx' | grep -v grep
```

You see `dockerd` and `containerd` as long-lived daemons and a `containerd-shim-runc-v2` process as the parent of `nginx: master process`. There is no `runc` process: it exits after starting the container.

9. **Clean up.**

```bash
docker rm -f web; sudo rm -rf /tmp/ov
```

7 Real-world scenario

(Illustrative.) **"The API container keeps dying every night at 02:00."** The `shop-api` container restarts with no application error in the logs. `docker inspect shop-api --format '{{.State.OOMKilled}} {{.State.ExitCode}}'` prints `true 137`: the kernel OOM-killed it. The nightly report job pushed memory above the `--memory 256m` limit set in the Compose file,

and `memory.max` is enforced by the kernel, not by Node.js. The team raised the limit to 512 MB, set `NODE_OPTIONS=--max-old-space-size=384` so the V8 heap stays below the cgroup limit, and added a memory-usage alert at 80% of the limit. **Lesson:** a container limit is a kernel rule — runtime settings inside the app must be sized to fit under it.

8 Common mistakes and best practices

Don't

- Say "containers are lightweight VMs" in a design review — they share the host kernel and its vulnerabilities.
- Run `--privileged` or mount `/var/run/docker.sock` into containers casually; both are effectively root on the host.
- Store data in the container's writable layer and expect it to survive `docker rm`.
- Leave production containers without memory and PID limits.

Do

- Set `--memory`, `--cpus` (or Compose `deploy.resources` / `mem_limit`) and `--pids-limit` on every service.
- Use `docker inspect`, `docker top` and `/proc/<PID>/ns` when something "impossible" happens; the host can see everything.
- Prefer rootless Docker or user-namespace remapping where you can, and keep the host kernel patched.
- Remember Kubernetes talks to containerd/CRI-O; learn `crictl` for nodes without Docker.

9 Tasks

- [ ] **T1 — Create a PID namespace by hand** *(≈20 min, beginner)*
  - **Goal:** see isolation without Docker.
  - **Steps:**
    1. Run `sudo unshare --pid --fork --mount-proc bash`.
    2. Inside, run `echo $$` and `ps -ef`.
    3. Open a second terminal on the host and run `pgrep -f "unshare --pid"` to find the same process from outside.
    4. `exit` the inner shell.

- **Done when:** inside you are PID 1 with a two-line `ps`, and from outside you can show the same process has a large PID.

- [ ] **T2 — Map a container to the host** *(≈25 min, beginner)*
  - **Goal:** prove a container is just a process.
  - **Steps:**
    1. `docker run -d --name t2 nginx:1.27` and `docker top t2`.
    2. Get its host PID with `docker inspect -f '{{.State.Pid}}' t2` and run `sudo cat /proc/<PID>/cgroup`.
    3. Compare `readlink /proc/<PID>/ns/pid` with `readlink /proc/self/ns/pid`.
  - **Done when:** the PIDs match between `docker top` and `inspect`, and the two `pid` namespace IDs differ.

- [ ] **T3 — Enforce a memory limit** *(≈30 min, intermediate)*
  - **Goal:** watch the cgroup OOM killer act.
  - **Steps:**
    1. Run `docker run --name hog --memory 64m --memory-swap 64m python:3.12-slim python -c "x=bytearray(200*1024*1024)"`.
    2. Run `docker inspect hog -f '{{.State.OOMKilled}} {{.State.ExitCode}}'`.
    3. Remove the container.
  - **Done when:** the output is `true 137` and you can explain where 137 comes from.

- [ ] **T4 — Rebuild copy-on-write by hand** *(≈30 min, intermediate)*
  - **Goal:** understand layers by creating them.
  - **Steps:**
    1. Follow step 4 but add a *second* lower layer: `lowerdir=/tmp/ov/lower2:/tmp/ov/lower` (the left-most is the top layer), with a different `a.txt` in `lower2`.
    2. Read `merged/a.txt`, modify it, and list `upper`.
  - **Done when:** you can show which layer each file came from and that both lower layers are unchanged after writes.

- [ ] **T5 — Break it: no limit vs limit on PIDs** *(≈30 min, intermediate)*
  - **Goal:** see why `--pids-limit` exists.
  - **Steps:**
    1. Run `docker run --rm --pids-limit 20 alpine sh -c 'for i in $(seq 1 100); do sleep 30 & done; wait'` and note the "can't fork" errors.
    2. Re-run with `--pids-limit 200` and observe it succeed (stop it after a few seconds).

- **Done when:** the first run prints fork errors and the second does not.

- [ ] **T6 — Prove it: draw and explain the stack** *(≈30 min, advanced)*
  - **Goal:** explain what happens on `docker run -d nginx`.
  - **Steps:**
    1. Without notes, draw the chain CLI → dockerd → containerd → shim → runc → kernel.
    2. For each box write what it owns, and for the kernel list the three mechanisms (namespaces, cgroups, overlay).
    3. Explain to a colleague why a shared kernel makes the container isolation boundary weaker than a VM's.
  - **Done when:** your diagram and explanation match section 4 and a listener can answer "where does the memory limit live?" (a cgroup file).

10 Self-check questions

1. Which three kernel features make a container, and what does each control?
2. What does `runc` do, and why is there no `runc` process while a container runs?
3. What does exit code 137 mean for a container?
4. Why does deleting a file in a container not shrink the image?
5. Why might a company run containers inside Firecracker microVMs?
6. Does Kubernetes need `dockerd`?

ANSWERS

1. Namespaces (visibility), cgroups (resource limits and accounting), and a union filesystem such as OverlayFS (layered root filesystem).
2. It creates the namespaces/cgroups/mounts per the OCI spec and `exec`s the container's program; once that is done, runc has nothing left to do and exits. The shim stays as parent.
3. 128 + 9: the process was killed by SIGKILL, commonly by the kernel OOM killer when the cgroup memory limit was hit (confirm with `OOMKilled` in `docker inspect`).
4. Image layers are read-only; a delete writes a whiteout in a *new* layer, and the old layer's bytes still ship.
5. To get a hardware-virtualisation boundary between tenants while keeping container-like start times and the same image format.
6. No. The kubelet talks to a CRI runtime such as containerd or CRI-O, which uses an OCI runtime like runc.

11 Further reading

- Docker documentation: "Docker overview" and "Storage drivers" (docs.docker.com).
- `man 7 namespaces`, `man 7 cgroups`, `man 1 unshare` — the primary sources.
- Linux kernel documentation: "Control Group v2" and "Overlay Filesystem".
- Open Container Initiative runtime and image specifications (opencontainers.org).
- *Container Security* by Liz Rice — how namespaces, cgroups and capabilities combine.

4.2 Images and Layers

Topic 2 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

A **container image** is a read-only package containing a filesystem (your app, its runtime, libraries, OS userland) plus metadata: which command to run, which environment variables to set, which user to run as, which port the app expects. A container is an image *plus* a thin writable layer plus a running process.

An image is not one big file. It is a **stack of layers**, each layer being a compressed tar archive of filesystem changes ("add these files, delete those"). Every layer, and the image's configuration, is stored and referenced by the **SHA-256 hash of its content** — this is called **content-addressable storage**. Because the name *is* the hash, two images that share a base (say `node:22-slim`) store those layers only once on disk, in a registry and on the wire.

2 Why it matters in DevOps

- **Speed:** pushes, pulls and CI builds are fast only if layers are reused. A badly ordered Dockerfile re-uploads hundreds of MB on every commit.
- **Reproducibility and rollback:** a *tag* can be moved to different content; a *digest* cannot. Deploying by digest is how you guarantee "the thing I tested is the thing in production".
- **Security:** every layer is permanent. A secret copied in one layer and deleted in the next is still extractable from the earlier layer. Vulnerability scanners work layer by layer.
- **Cost:** registry storage, cross-region transfer and node disk usage all scale with image size.
- See also Image Caching and Layer Optimization §4.9 and Image Scanning and Container Security §4.14.

3 Real-world system example

Docker Hub and the OCI image format. Every image on Docker Hub, GitHub Container Registry or Amazon ECR is described by an **OCI image manifest**: a small JSON document listing one *config* blob and an ordered list of *layer* blobs, each by media type, size and

`sha256:` digest. Multi-architecture images such as `nginx` or `node` publish an **image index** (a "manifest list") that maps `linux/amd64`, `linux/arm64` and so on to per-platform manifests — which is how the same tag works on an Intel server and an Apple-silicon laptop. Kubernetes manifests, Helm charts and GitOps tools commonly pin `image: repo/app@sha256:...` to deploy an exact, immutable artifact. Official Docker images also use mutable tags on purpose (`node:22` moves with each patch release), which is convenient but exactly why production deployments should pin more precisely.

4 Diagram

How a tag leads to bytes:

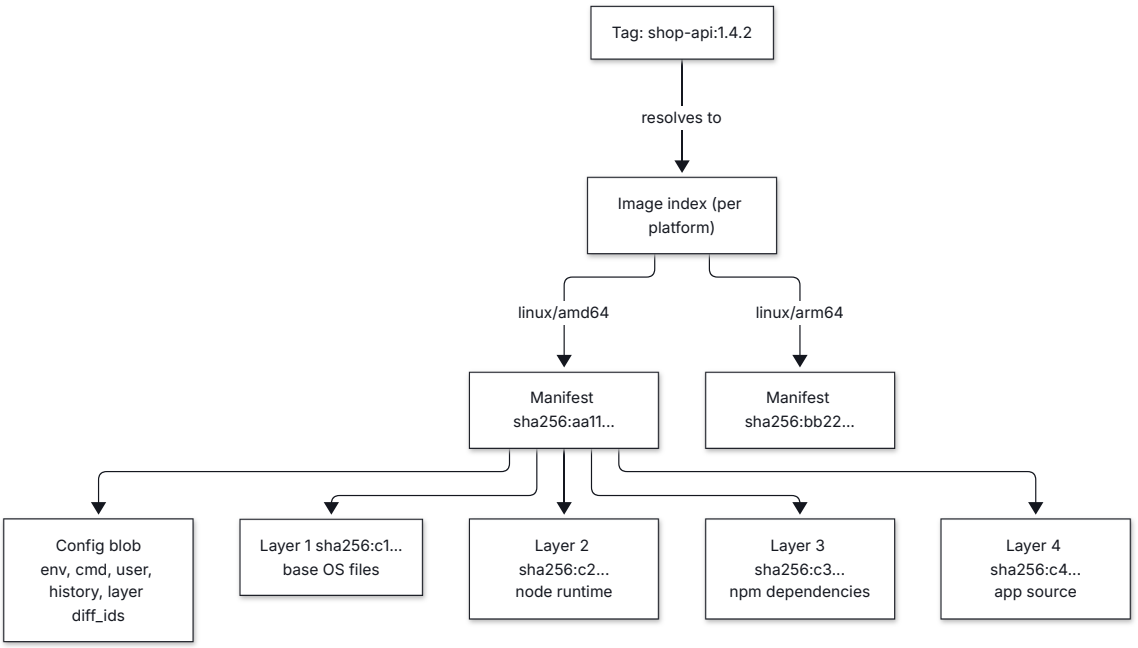


Figure 4.2-1

Sharing between images (each blob is stored once):

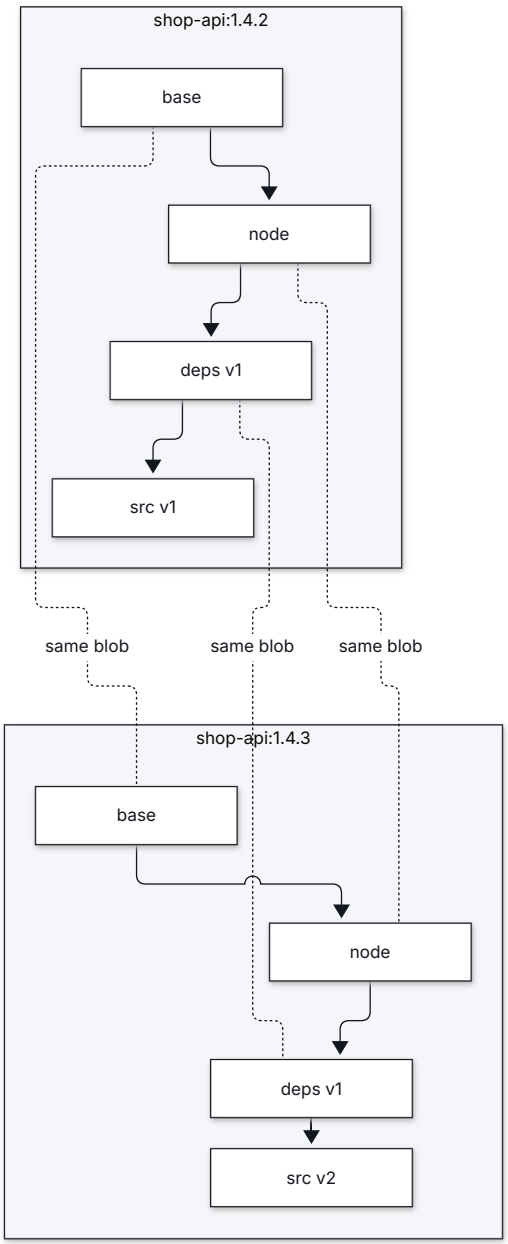


Figure 4.2-2

Only `src v2` has to be pushed and pulled for version 1.4.3.

5 How it works

Anatomy of an image (OCI layout, which you can see with `docker save`):

Piece	What it is
Image index	Optional list of per-platform manifests (multi-arch)
Manifest	JSON: pointer to the config blob + ordered list of layer blobs (digest, size, media type)
Config	JSON: `Env`, `Cmd`, `Entrypoint`, `User`, `ExposedPorts`, `WorkingDir`, `Labels`, build `history`, and `rootfs.diff_ids`
Layers	Compressed tar archives (gzip or zstd) of changes; applied bottom to top

Where layers come from. Dockerfile instructions that change the filesystem (`RUN`, `COPY`, `ADD`) create layers. Instructions that only change metadata (`ENV`, `CMD`, `EXPOSE`, `USER`, `WORKDIR`, `LABEL`) go in the config and add (almost) no size. Two digests matter: the **diff ID** (hash of the *uncompressed* tar, listed in the config) and the **layer digest** (hash of the *compressed* blob, listed in the manifest).

Layers are immutable and additive. Deleting a file in a later layer writes a *whiteout* entry; the earlier layer's bytes are still in the image. That is why `RUN rm -rf /var/cache` as a *separate* step saves nothing, while doing the install and the cleanup in the *same* `RUN` does.

Tags vs digests:

	Tag (`node:22-slim`)	Digest (`node@sha256:9f1c...`)
What it is	A human-friendly, **mutable** pointer	The SHA-256 of the manifest — **immutable**
Can change underneath you	Yes (re-pushed, `latest` moves)	No — different content means a different digest
Use for	Development, human conversation	Production deploys, audit trails, reproducible builds
Common pattern	`1.4.2`, `1.4`, `1`, `latest`, `git-3f9a2c1`	`image: ghcr.io/example/shop-api@sha256:...`

A reference can carry both (`name:tag@sha256:...`); the digest wins and the tag becomes documentation.

Local view. `docker images` shows tags; `docker image ls --digests` adds digests; `docker history <image>` lists each layer with the instruction that created it and its size; `docker inspect <image>` prints the config including `RootFS.Layers`.

6 Step-by-step guide

Prerequisites: Docker Engine (or Docker Desktop) and internet access to pull small public images from Docker Hub. Steps 1 and 2 need only `tar` and `sha256sum`. Image-related output below is *illustrative*: digests, sizes and layer counts change with every upstream release, so expect something similar, not identical.

1. **Feel content addressing with plain tar.** The hash of a layer is just a hash of bytes, so identical content gives an identical name.

```bash
mkdir -p /tmp/layer1 && echo hi > /tmp/layer1/a.txt
tar --sort=name --mtime='UTC 2020-01-01' --owner=0 --group=0 --numeric-owner -cf
  /tmp/l1.tar -C /tmp/layer1 .
sleep 1
tar --sort=name --mtime='UTC 2020-01-01' --owner=0 --group=0 --numeric-owner -cf
  /tmp/l2.tar -C /tmp/layer1 .
sha256sum /tmp/l1.tar /tmp/l2.tar
```

   Both lines show the same hash, even though they were created a second apart. (Change `a.txt` and the hash changes completely.) Real layer hashes work the same way, which is why reproducible builds normalise timestamps.

2. **Inspect an image's layers and config.**

```bash
docker pull node:22-slim
docker image inspect node:22-slim --format '{{json .RootFS.Layers}}' | python3 -m
  json.tool
docker history node:22-slim
```

   `RootFS.Layers` is the ordered list of `sha256:` diff IDs. `docker history` prints one row per layer with `CREATED BY` (the instruction) and `SIZE`; metadata-only instructions show `0B`.

3. **Open the image like an archive.**

```bash
mkdir -p /tmp/img && docker save node:22-slim -o /tmp/node.tar && tar -xf
  /tmp/node.tar -C /tmp/img
ls /tmp/img /tmp/img/blobs/sha256 | head -20
cat /tmp/img/index.json
```

   You should see `index.json`, `oci-layout`, `manifest.json` and a `blobs/sha256/` directory full of files named by their hashes. (Layout details vary slightly between Docker versions and image stores.) Each blob is a manifest, a config or a layer.

4. **Prove layers are shared.** Pull two images that share a base and watch Docker skip layers it already has.

```bash
docker pull node:22-slim
docker pull node:22-bookworm-slim
```

   Lines such as `<id>: Already exists` mean the layer was identified by hash and not downloaded again. (`22-slim` and `22-bookworm-slim` are often the same image under two tags; if both resolve to the same digest, the second pull reports "Image is up to date".)

5. **See tags vs digests.**

```bash
docker image ls --digests node
docker run --rm node:22-slim node --version
DIGEST=$(docker image inspect node:22-slim --format '{{index .RepoDigests 0}}')
echo "$DIGEST"
docker run --rm "$DIGEST" node --version
```

   `RepoDigests` contains `node@sha256:...`. Running by digest always executes exactly those bytes.

6. **Add a layer yourself and look at the history.**

```bash
mkdir -p /tmp/lyr && cd /tmp/lyr
printf 'FROM alpine:3.20\nRUN echo one > /one.txt\nRUN echo two > /two.txt\nENV
  STAGE=demo\nCMD ["cat","/one.txt","/two.txt"]\n' > Dockerfile
docker build -t layers-demo:1 .
docker history layers-demo:1
docker run --rm layers-demo:1
```

   The history shows the base layers, then two layers of a few bytes each, then `0B` metadata rows for `ENV` and `CMD`. The run prints `one` and `two`.

7. **Show that deleting does not shrink an image.**

```bash
printf 'FROM alpine:3.20\nRUN dd if=/dev/zero of=/big bs=1M count=50\nRUN rm /big\n' >
  Dockerfile
docker build -t layers-demo:big .
docker history layers-demo:big
docker image ls layers-demo
```

One layer is ~50 MB, the `rm` layer is `0B`, and the image is still 50 MB larger than `alpine`. Now change it to `RUN dd ... && rm /big` in a single `RUN` and rebuild: the 50 MB disappears.

8. **Tag, retag and clean up.**

```bash
docker tag layers-demo:1 layers-demo:stable
docker image ls layers-demo
docker rmi layers-demo:stable layers-demo:1 layers-demo:big
```

Two tags, same image ID: tags are just names pointing at content.

7 Real-world scenario

(Illustrative.) **"Staging passed, production broke, and the code is identical."** The release pipeline deployed `shop-api:latest` to staging on Monday and to production on Wednesday. On Tuesday the base image `node:22-slim` had been rebuilt upstream with a new OpenSSL, and a rebuild in CI had silently pulled it. Production ran code that nobody had tested. The team traced it with `docker image inspect` on both hosts: different `Id` and different `RepoDigests` for the "same" tag. They changed CI to build once, push `shop-api:git-3f9a2c1`, record the resulting digest, and deploy `shop-api@sha256:<digest>` to every environment, with Renovate/Dependabot opening explicit PRs for base-image bumps. **Lesson:** tags are conversation, digests are evidence.

8 Common mistakes and best practices

Don't

- Deploy `latest` (or any floating tag) to production and call it a version.
- Copy secrets (`.env`, SSH keys, npm tokens) into any layer, even if you delete them later — they remain in earlier layers.
- Put `RUN rm` in a separate step to "clean up" and expect a smaller image.
- Assume `docker pull` today and `docker pull` next month give the same bytes.

Do

- Tag images with an immutable identifier (git SHA or semantic version) and deploy by digest.
- Order instructions from least to most frequently changing so layers are reused (see topic 9).

- Combine install and cleanup in one `RUN`, and use a multi-stage build §4.10 to keep build tools out of the final layers.
- Build multi-arch images (`docker buildx build --platform linux/amd64,linux/arm64`) if your fleet or laptops mix architectures.

9 Tasks

- [ ] **T1 — Read an image's layers** *(≈20 min, beginner)*
  - **Goal:** map instructions to layers.
  - **Steps:**
    1. `docker pull nginx:1.27` (or any small image).
    2. Run `docker history --no-trunc nginx:1.27` and `docker image inspect nginx:1.27 --format '{{len .RootFS.Layers}}'`.
    3. Identify which history rows are `0B` metadata rows and which add files.
  - **Done when:** you can state the layer count and name the instruction behind the biggest layer.

- [ ] **T2 — Crack open** `docker save` *(≈30 min, intermediate)*
  - **Goal:** find the manifest, config and layers on disk.
  - **Steps:**
    1. `docker save alpine:3.20 -o alpine.tar` and extract it into an empty directory.
    2. Locate the manifest (via `index.json` or `manifest.json`), then the config blob, then the layer blob.
    3. Extract the layer tar and list its files; confirm `sha256sum` of the blob equals its filename.
  - **Done when:** you have `sha256sum <blob>` output that matches the blob's name, and you can point to the file that holds `Cmd`.

- [ ] **T3 — Measure layer sharing** *(≈30 min, intermediate)*
  - **Goal:** prove deduplication.
  - **Steps:**
    1. Build two images `FROM alpine:3.20` that each add a different 1 MB file.
    2. Run `docker system df -v` and compare the "SHARED SIZE" and "UNIQUE SIZE" columns.
    3. Explain how much disk two images really consume together.

- **Done when:** you can show the shared base counted once and the unique parts of about 1 MB each.

- [ ] **T4 — Break it: the "deleted" secret** *(≈30 min, intermediate)*
  - **Goal:** learn why secrets in layers leak.
  - **Steps:**
    1. Write a Dockerfile with `RUN echo "API_KEY=FAKE-123" > /secret` followed by `RUN rm /secret`.
    2. Build it, run `docker save` and extract the layers, then `grep -r FAKE-123` through the extracted layer tars (`tar -xOf <layer> | grep FAKE`).
    3. Rewrite the build using a BuildKit secret mount (`RUN --mount=type=secret,id=api_key ...`) and repeat the search.
  - **Done when:** the first image leaks `FAKE-123` and the second does not.

- [ ] **T5 — Pin by digest and detect drift** *(≈30 min, intermediate)*
  - **Goal:** make a build reproducible.
  - **Steps:**
    1. Get the digest of `alpine:3.20` and write it into a Dockerfile as `FROM alpine:3.20@sha256:<digest>`.
    2. Build, then run `docker buildx imagetools inspect alpine:3.20` and compare the digest to the one you pinned.
    3. Write two sentences on how you would update the pin safely (automation + PR review).
  - **Done when:** your Dockerfile builds from the pinned digest and you can explain the update process.

- [ ] **T6 — Prove it: explain a pull** *(≈30 min, advanced)*
  - **Goal:** narrate what a registry pull downloads.
  - **Steps:**
    1. Without notes, write the sequence: tag → (index) → manifest → config + layers → verify digests → unpack to snapshots.
    2. Explain why a second `docker pull` of an updated tag downloads only some layers.
    3. State when you would use a tag and when a digest.
  - **Done when:** your explanation matches sections 4-5 and includes "content-addressable" used correctly.

Self-check questions

10

1. What is in an image manifest, and how does it differ from the image config?
2. Why do two images that share a base take less space than the sum of their sizes?
3. Why is `RUN rm -rf /tmp/build` in its own step useless for reducing image size?
4. What is the difference between a tag and a digest, and which would you put in a production Kubernetes manifest?
5. What is the difference between a layer's diff ID and its digest?

ANSWERS

1. The manifest lists the config blob and the ordered layer blobs by media type, size and digest. The config holds runtime metadata (env, cmd, user), the build history and the uncompressed layer diff IDs.
2. Layers are identified by content hash and stored once; images that reference the same hash share the blob on disk, in the registry and on the wire.
3. Layers are additive: the deletion is recorded as a whiteout in a new layer while the earlier layer still contains the bytes. Delete in the same `RUN` that created the files, or use a multi-stage build.
4. A tag is a mutable pointer; a digest is the immutable hash of the manifest. Production manifests should reference the digest (optionally with a human-readable tag alongside).
5. The diff ID is the hash of the uncompressed tar (stored in the config); the digest is the hash of the compressed blob (stored in the manifest and used by registries).

11 Further reading

- OCI Image Format Specification (opencontainers.org) — the manifest, config and layout definitions.
- Docker documentation: "Images and layers" and "Build best practices" (docs.docker.com).
- Docker documentation: `docker buildx imagetools` and the multi-platform builds guide.
- *Docker Deep Dive* by Nigel Poulton — a clear walk through images and layers.

4.3 Containers and Their Lifecycle

Topic 3 of 14 | ~4 hours | Difficulty: Beginner

1 What it is

A **container** is a running (or stopped) instance of an image: the image's read-only layers, one thin writable layer, a set of isolation settings (namespaces, cgroup limits, network, mounts) and a **main process** that the runtime starts. The container lives exactly as long as that main process — when it exits, the container stops. This "one main process, tied to the container's life" rule explains most beginner surprises: containers that exit immediately, containers that will not stop politely, and containers that restart forever.

The everyday verbs are `docker run` (create + start), `docker ps` (list), `docker logs` (read stdout/stderr), `docker exec` (run another process inside), `docker stop` / `docker kill` (end it), `docker start` / `docker restart` (bring it back) and `docker rm` (delete it and its writable layer).

2 Why it matters in DevOps

- **Graceful shutdown = no dropped requests.** Every deploy, scale-down and node drain sends your container a signal. If the app ignores it, users see errors and in-flight jobs are lost.
- **Restart behaviour decides availability.** A restart policy turns a crash into a blip — or into a crash loop that hides a real bug and burns CPU.
- **Logs are your first debugging tool.** In containers, logs are what the process writes to stdout/stderr, not files you SSH in to read.
- **PID 1 is special.** Getting it wrong causes zombie processes and containers that take 10 seconds to stop on every deploy.
- See also Processes, PIDs and Signals §1.8 for the Linux foundations.

3 Real-world system example

Kubernetes pod termination. When Kubernetes removes a pod it sends `SIGTERM` to the main process of each container, waits up to `terminationGracePeriodSeconds` (default 30 seconds), then sends `SIGKILL`. This is exactly the `docker stop` contract (SIGTERM, wait, SIGKILL; default wait 10 seconds). Services like Nginx and PostgreSQL define their own signal semantics: Nginx treats `SIGQUIT` as "graceful shutdown" and `SIGTERM` as "fast shutdown", which is why the official Nginx image sets `STOPSIGNAL SIGQUIT`; PostgreSQL treats `SIGTERM` as "smart" shutdown and `SIGINT` as "fast", and its image also sets a stop signal to match. Rolling deployments in Kubernetes, ECS and Docker Swarm are only zero-downtime if containers honour this contract.

4 Diagram

Container state machine:

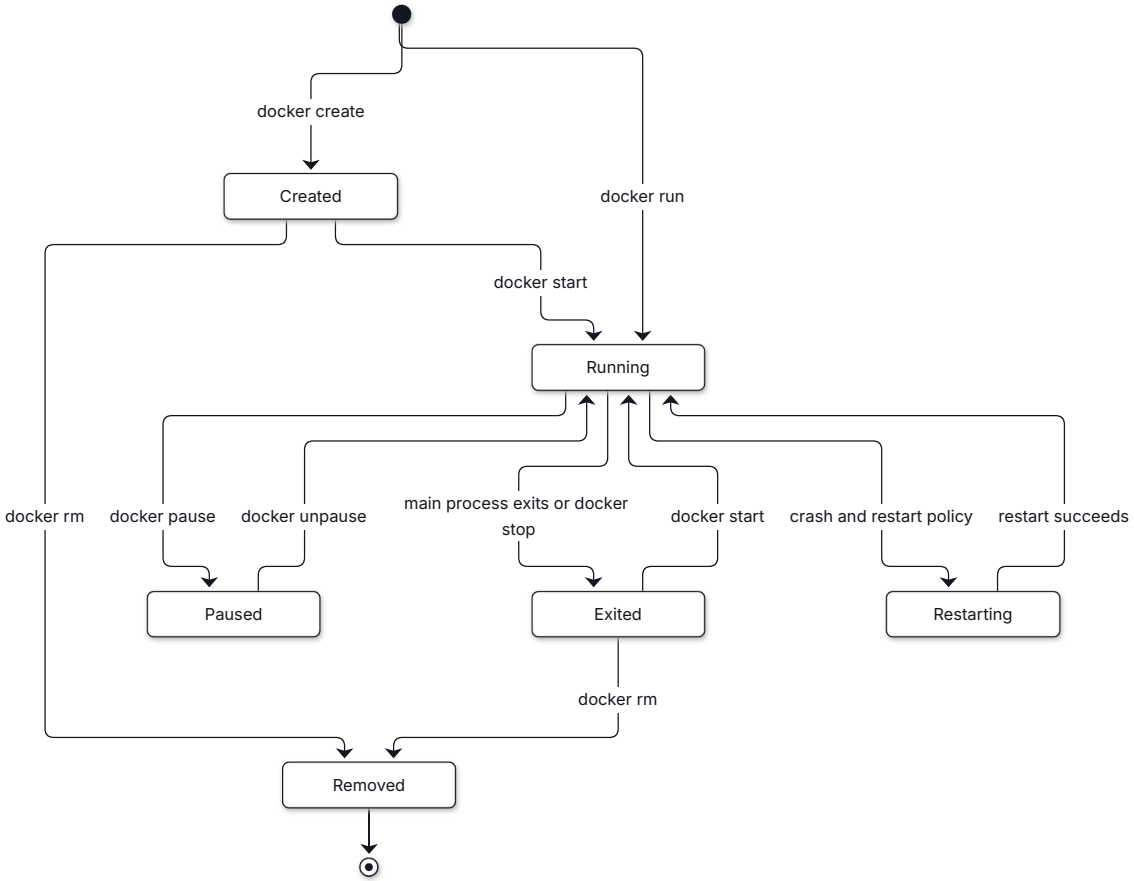


Figure 4.3-1

What `docker stop` does:

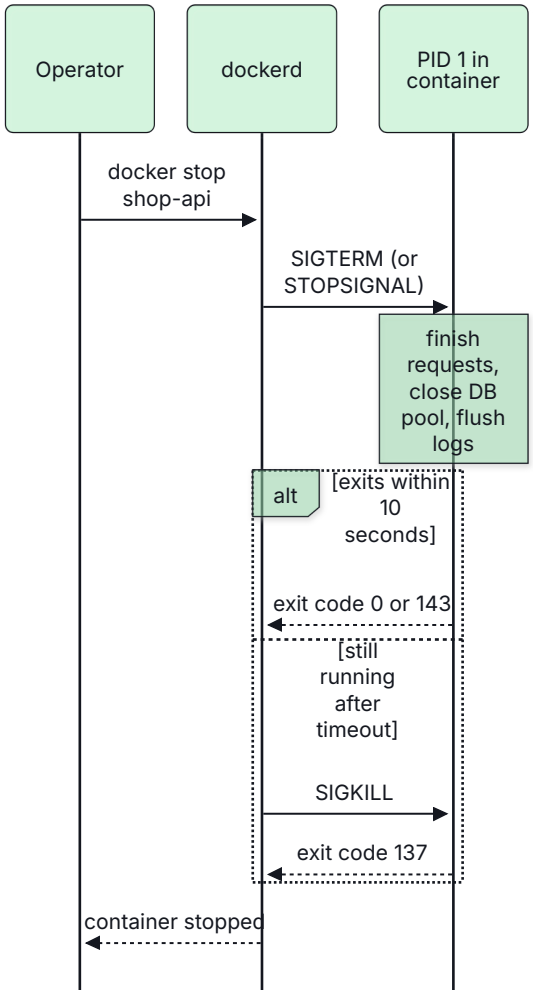


Figure 4.3-2

5 How it works

Core commands:

Command	Purpose	Useful flags
`docker run IMAGE [CMD]`	create and start	`-d` detach, `-it` interactive TTY, `--name`, `--rm`, `-p`, `-e`, `-v`, `--restart`, `--init`, `--memory`, `--cpus`
`docker ps` / `ps -a`	running / all containers	`--filter status=exited`, `--format`

Command	Purpose	Useful flags
`docker logs NAME`	stdout and stderr	`-f` follow, `--tail 100`, `--since 10m`, `-t` timestamps
`docker exec NAME CMD`	run a new process in a running container	`-it` for a shell, `-u` user, `-e` env
`docker stop` / `kill`	SIGTERM then SIGKILL / immediate signal	`-t 30` grace seconds, `-s SIGHUP`
`docker inspect NAME`	full JSON state	`-f '{{.State.ExitCode}}'`
`docker stats`	live CPU, memory, network, PIDs	`--no-stream`
`docker cp`	copy files in or out	
`docker rm`	delete a stopped container	`-f` force (kill first), `-v` remove anonymous volumes

Exit codes you must recognise: `0` success; `1` generic app error; `125` Docker itself failed; `126` command not executable; `127` command not found; `137` = 128 + 9 (SIGKILL — often out-of-memory or a `docker stop` timeout); `143` = 128 + 15 (SIGTERM, a normal stop for a process with no handler).

PID 1 rules. Inside a PID namespace the first process is PID 1, and the kernel treats it specially:

1. Signals with the default action ("terminate") are **not** delivered to PID 1 unless it has installed a handler. A Node.js process that does not call `process.on('SIGTERM', ...)` ignores `docker stop` until the 10-second timeout, then gets `SIGKILL`.
2. PID 1 must **reap** orphaned children (`wait()` on them) or they stay as zombies. Fixes: handle `SIGTERM` in the app, and/or run a tiny init as PID 1 with `docker run --init` (Compose: `init: true`), which uses `tini` to forward signals and reap zombies.

Shell form vs exec form. `CMD node server.js` (shell form) runs `/bin/sh -c "node server.js"`, so `sh` **is PID 1** and your app never sees `SIGTERM`. `CMD ["node", "server.js"]` (exec form) makes node PID 1 directly. Always use exec form (details in Dockerfile §4.7). (Some shells optimise a lone command in `sh -c` by exec-ing it, which can mask the problem; wrappers such as `npm start`, `a && b` or scripts that do not end in `exec` do not get that luck.)

Restart policies (`--restart` or Compose `restart:`):

Policy	Behaviour
`no` (default)	never restart

Policy	Behaviour
`on-failure[:N]`	restart only on non-zero exit, at most N times
`always`	restart whatever the exit code; also restarts when the Docker daemon starts
`unless-stopped`	like `always`, but not if you manually stopped it before the daemon restarted

Docker delays restarts with an increasing back-off, so a crash loop does not spin the CPU, but it still never fixes a bug. In Kubernetes the equivalent is the pod `restartPolicy` and the `CrashLoopBackOff` state.

Logs. The default `json-file` driver stores stdout/stderr under `/var/lib/docker/containers/<id>/` and **never rotates by default**. Set `--log-opt max-size=10m --log-opt max-file=3` (or the same in `daemon.json`) or switch drivers (`local`, `fluentd`, `awslogs`, `loki`).

6 Step-by-step guide

Prerequisites: Docker Engine/Desktop. Outputs are illustrative (IDs, timings differ).

1. **Run, list, read logs.**

```bash
docker run -d --name web -p 8080:80 nginx:1.27
docker ps
curl -sI http://localhost:8080 | head -1
docker logs --tail 3 web
```

   `docker ps` shows `web` with `0.0.0.0:8080→80/tcp`; curl prints `HTTP/1.1 200 OK`; the logs show your request in the access log.

2. **Open a shell inside and look around.**

```bash
docker exec -it web sh
ps aux | head -3        # PID 1 is "nginx: master process"
cat /etc/os-release | head -2
exit
```

   `exec` starts an *additional* process in the same namespaces; exiting it does not stop the container.

3. **Time a stop and read the exit code.**

```bash
time docker stop web
docker inspect -f '{{.State.Status}} exit={{.State.ExitCode}}' web
```

Nginx handles its stop signal, so this returns in about a second with `exited exit=0`.

4. **See PID 1 ignoring SIGTERM, then fix it.** Create a deliberately bad app:

```bash
mkdir -p /tmp/sig && cd /tmp/sig
cat > Dockerfile <<'EOF'
FROM node:22-alpine
CMD ["node", "-e", "setInterval(() => console.log('tick'), 1000)"]
EOF
docker build -t sig-bad .
docker run -d --name bad sig-bad
time docker stop bad
docker inspect -f 'exit={{.State.ExitCode}}' bad
```

It takes about 10 seconds and the exit code is `137`: Docker had to `SIGKILL` it because node as PID 1 had no SIGTERM handler. Now fix it two ways:

```bash
docker run -d --name init --init sig-bad && time docker stop init       # tini
  forwards SIGTERM; node dies at once
docker inspect -f 'exit={{.State.ExitCode}}' init                         # 143
```

Or handle the signal in code (the real fix):

```js
// server.js (excerpt)
process.on('SIGTERM', () => {
  console.log('SIGTERM received, shutting down');
  server.close(() => process.exit(0));   // stop accepting, drain, then exit
});
```

5. **Use a restart policy and watch a crash loop.**

```bash
docker run -d --name flaky --restart on-failure:3 alpine sh -c 'echo boot; sleep 2;
  exit 1'
sleep 15
docker inspect -f 'restarts={{.RestartCount}} status={{.State.Status}}' flaky
docker logs flaky | head
```

After three retries the policy gives up: `restarts=3 status=exited`. The logs show `boot` repeated. With `--restart always` it would never give up.

6. **Limit and watch resources.**

```bash
docker run -d --name limited --memory 64m --cpus 0.25 nginx:1.27
docker stats --no-stream limited
```

The `MEM USAGE / LIMIT` column shows a `64MiB` limit.

7. **Cap the log size, then clean up.**

```bash
docker run -d --name chatty --log-opt max-size=1m --log-opt max-file=2 alpine sh -c
  'while true; do echo spam; done'
sleep 5 && docker inspect -f '{{.HostConfig.LogConfig}}' chatty
docker rm -f web bad init flaky limited chatty
```

The log config shows the `json-file` driver with `max-size:1m` and `max-file:2`. The last line removes every container created in this guide.

7 Real-world scenario

(Illustrative.) **"Every deploy drops a few payments."** `shop-api` is deployed with a rolling update. During each rollout, the checkout endpoint returns a handful of 502s. `docker stop` (and, in the cluster, the pod termination) takes exactly the full grace period, and exit code is `137`. The Dockerfile used `CMD npm start`: `npm` was PID 1, did not forward `SIGTERM` to node, and the app never closed its HTTP server — in-flight requests were cut when the `SIGKILL` arrived. The fix was three lines: exec-form `CMD ["node","server.js"]`, a `SIGTERM` handler that calls `server.close()` and then drains the database pool, and `init: true` in Compose. Deploys now stop in under two seconds with exit code `0`. **Lesson:** graceful shutdown is part of the application's contract with its platform.

8 Common mistakes and best practices

Don't

- Use `CMD npm start` or shell-form `CMD` for production; wrapper processes swallow signals.
- Write logs to files inside the container, and never `docker exec` in to "fix" a production container by hand — changes vanish on restart.
- Use `--restart always` to paper over crashes without alerting on `RestartCount`.

- Leave the default `json-file` log driver unbounded on a long-lived host.
- Run `docker rm -f` as your normal stop command; it skips graceful shutdown.

Do

- Run the app in the foreground as PID 1 (exec form), log to stdout/stderr, and handle `SIGTERM`.
- Use `--init` or `init: true` when the image runs shells or spawns child processes.
- Set `--memory` and log rotation on every container; choose `unless-stopped` for single-host services and `on-failure` for batch jobs.
- Inspect with `docker inspect`, `docker logs --since` and `docker stats` before touching anything.

9 Tasks

- [ ] **T1 — Lifecycle tour** *(≈20 min, beginner)*
  - **Goal:** use every lifecycle verb once.
  - **Steps:**
    1. `docker create --name t1 nginx:1.27`, then `docker ps -a` (status `Created`).
    2. `docker start t1`, `docker pause t1`, `docker unpause t1`, `docker stop t1`, `docker start t1`, `docker rm -f t1`.
    3. After each step run `docker ps -a --filter name=t1` and note the state.
  - **Done when:** you have seen `Created`, `Up`, `Up (Paused)`, `Exited` and the container disappearing.

- [ ] **T2 — Debug a container that exits immediately** *(≈25 min, beginner)*
  - **Goal:** read an exit code and logs.
  - **Steps:**
    1. Run `docker run --name oops alpine sh -c 'echo failing; exit 3'`.
    2. Find the exit code with `docker inspect` and the output with `docker logs`.
    3. Re-run with `--rm` and compare `docker ps -a`.
  - **Done when:** you report exit code `3`, the `failing` log line, and explain what `--rm` changed.

- [ ] **T3 — Fix a slow stop** *(≈30 min, intermediate)*
  - **Goal:** make shutdown graceful.
  - **Steps:**
    1. Build the `sig-bad` image from step 4 and time `docker stop` (about 10 s, exit 137).

2. Add a `SIGTERM` handler (or `--init`) and rebuild.
3. Time `docker stop` again.

- **Done when:** stop takes under 2 seconds and the exit code is not `137`.

- [ ] **T4 — Shell form vs exec form** *(≈30 min, intermediate)*
  - **Goal:** see who PID 1 is.
  - **Steps:**
    1. Build two images from `node:22-alpine` with `CMD node -e "setInterval(()⇒{},1000)"; echo done` (shell, compound so the shell stays resident) and `CMD ["node","-e","setInterval(()⇒{},1000)"]` (exec).
    2. Run each and execute `docker exec <name> ps` (or `docker top <name>`).
    3. Record which process is PID 1 in each.
  - **Done when:** the shell form shows `/bin/sh -c ...` as PID 1 with node as its child, and the exec form shows `node` as PID 1.

- [ ] **T5 — Restart policies under fire** *(≈30 min, intermediate)*
  - **Goal:** compare policies empirically.
  - **Steps:**
    1. Start three crashing containers (`sh -c 'sleep 1; exit 1'`) with `no`, `on-failure:2` and `unless-stopped`.
    2. After 30 seconds compare `RestartCount` and status for each.
    3. Restart the Docker daemon (`sudo systemctl restart docker`, skip on Docker Desktop) and see which come back.
  - **Done when:** you can tabulate the three outcomes and explain them.

- [ ] **T6 — Prove it: write the shutdown contract** *(≈30 min, advanced)*
  - **Goal:** specify graceful shutdown for `shop-api`.
  - **Steps:**
    1. Without notes, write the 5-step sequence from "SIGTERM received" to "exit 0" (stop accepting, drain in-flight, close DB/Redis, flush logs, exit).
    2. Add the Dockerfile and Compose settings it relies on (`exec form`, `init: true`, `stop_grace_period: 30s`).
    3. Demonstrate with a running container and `docker stop`, showing the shutdown log line and exit code.
  - **Done when:** the log shows your shutdown message and `docker inspect` reports `ExitCode=0`.

10 Self-check questions

1. What happens, step by step, when you run `docker stop`?
2. Why does a Node.js container often take 10 seconds to stop, and how do you fix it?
3. What do exit codes 137 and 143 mean?
4. What is the difference between `docker exec` and `docker run`?
5. When would you choose `on-failure:5` instead of `always`?
6. Where do container logs go by default, and what is the risk?

ANSWERS

1. Docker sends the stop signal (SIGTERM by default or the image's `STOPSIGNAL`) to PID 1, waits for the timeout (10 s by default), then sends SIGKILL.
2. Node as PID 1 has no default SIGTERM handler (the kernel ignores signals with default action for PID 1) or a wrapper such as `sh` / `npm` swallows it. Use exec-form CMD, handle SIGTERM, or use `--init`.
3. 137 = killed by SIGKILL (OOM kill or stop timeout); 143 = terminated by SIGTERM.
4. `run` creates a new container from an image; `exec` starts an extra process inside an already running container.
5. For jobs that should stop retrying after repeated failure so the failure is visible, rather than looping indefinitely.
6. Stdout/stderr of PID 1 go to the `json-file` driver under `/var/lib/docker/containers/`; without `max-size` / `max-file` they grow without limit and can fill the disk.

11 Further reading

- Docker documentation: "Run" reference (`docker run`), "Start containers automatically" and "Logging drivers" (docs.docker.com).
- Kubernetes documentation: "Pod Lifecycle" — termination and grace periods.
- `man 7 signal` and `man 7 pid_namespaces` — why PID 1 is special.
- The `tini` project README (github.com/krallin/tini) — a minimal init for containers.

LAB NOTES What I ran, what I saw, what it means

4.4 Registries

Topic 4 of 14 | ~3 hours | Difficulty: Beginner

1 What it is

A **container registry** is a server that stores and serves container images over HTTP, using the OCI Distribution API (the "Docker Registry HTTP API v2"). It is to images what a Git host is to source code: you **push** an image from your build machine, and any server, laptop or cluster with permission **pulls** it. Registries store blobs (layers and configs) by digest and manifests by tag or digest, so they deduplicate layers automatically (see Images and Layers §4.2).

An image name encodes where it lives: `REGISTRY/NAMESPACE/REPOSITORY:TAG`, for example `ghcr.io/example/shop-api:1.4.2`. If you omit the registry, Docker assumes Docker Hub (`docker.io`); if you omit the tag, it assumes `latest`.

2 Why it matters in DevOps

- **The registry is the hand-off point between CI and deployment.** CI builds and pushes; CD pulls. If the registry is down or a credential is wrong, nothing ships.
- **Access control and supply chain security.** Who may push to production images? Who may pull private code? Compromised push rights mean an attacker can run code in your cluster.
- **Cost and reliability.** Pull rate limits on public registries, cross-region transfer and storage growth all show up on bills and in outages ("ImagePullBackOff" at 3 a.m.).
- **Retention and compliance.** Old images pile up forever unless lifecycle rules delete them; vulnerability scanning usually hooks in at the registry.
- See also Docker Image Builds and Registries in CI §5.4.

3 Real-world system example

Where real teams keep images.

- **Docker Hub** hosts the "Docker Official Images" (`nginx`, `postgres`, `redis`, `node`, `alpine`) that most Dockerfiles start `FROM`. Anonymous and free pulls are subject to rate limits, so busy CI systems usually authenticate or mirror the base images they use.
- **Amazon ECR (Elastic Container Registry)** is the natural choice on AWS: it integrates with IAM, is reachable privately from a VPC through endpoints, supports image scanning and **lifecycle policies**, and is what ECS and EKS nodes pull from by default in AWS-centric stacks.
- **GitHub Container Registry (GHCR, `ghcr.io`)** sits next to the code: GitHub Actions can push with the built-in `GITHUB_TOKEN`, and images are linked to repositories.
- Kubernetes itself publishes release images to its own registry, `registry.k8s.io`, and many open-source projects use Quay.io or GHCR.
- Large organisations often add a **pull-through cache or mirror** (for example ECR pull-through cache, Harbor or Artifactory) so that upstream outages and rate limits do not stop deployments.

4 Diagram

What `docker push` actually does (HTTP calls simplified):

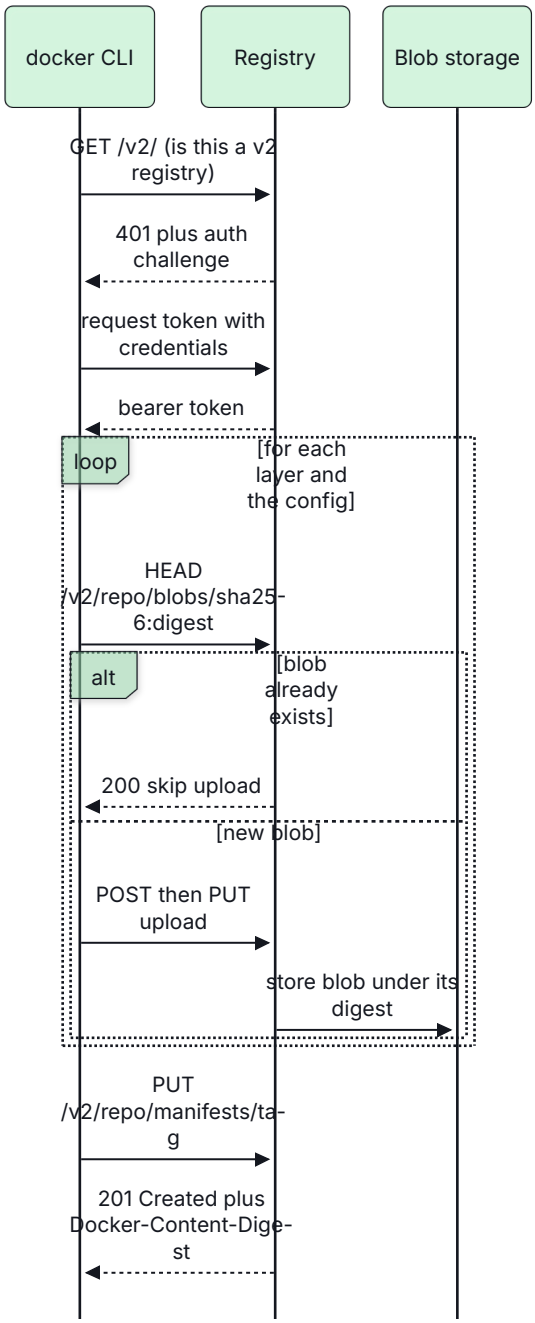


Figure 4.4-1

A typical delivery path:

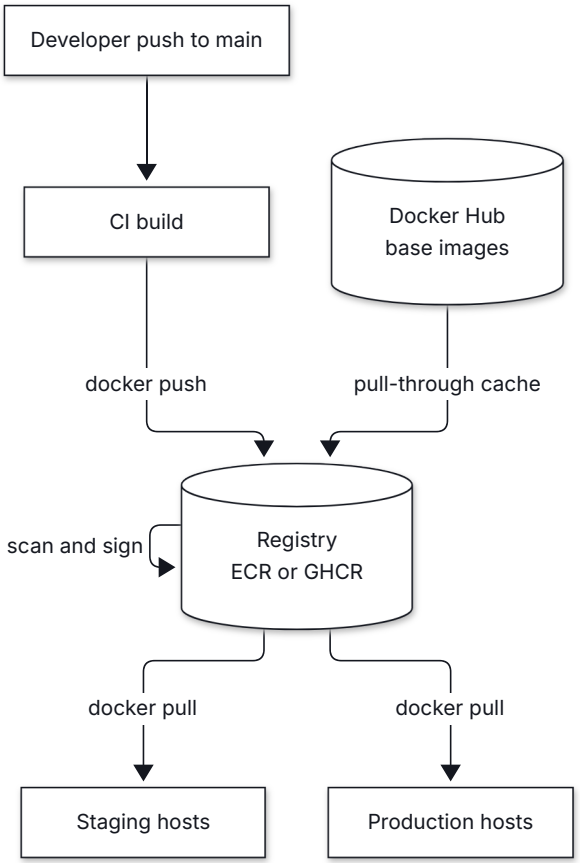


Figure 4.4-2

5 How it works

Naming and tagging

Part	Example	Notes
Registry host	`ghcr.io`, `123456789012.dkr.ecr.us-east-1.amazonaws.com`, `localhost:5000`	default `docker.io`
Namespace / repo	`example/shop-api`	Hub: `username/repo`; official images live under `library/`
Tag	`1.4.2`, `git-3f9a2c1`, `main`	default `latest`; mutable unless the registry enforces immutability

Part	Example	Notes
Digest	`@sha256:...`	immutable content address

`docker tag SOURCE TARGET` just adds another name to the same image locally; `docker push TARGET` uploads it. One build commonly gets several tags: the git SHA (unique), a semantic version, and maybe a moving `1.4` or `main`.

Authentication

Registry	How to log in
Docker Hub	`docker login -u USER` with a **personal access token**, not your password
GHCR	`echo $CR_PAT | docker login ghcr.io -u USERNAME --password-stdin` with a token scoped to packages; in GitHub Actions use `GITHUB_TOKEN` with `packages: write`
Amazon ECR	`aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin ACCOUNT.dkr.ecr.us-east-1.amazonaws.com` — the token lasts 12 hours
Self-hosted	basic auth (htpasswd), token auth, or an OIDC-backed registry such as Harbor

`docker login` stores credentials in `~/.docker/config.json`, ideally via a credential helper (`osxkeychain`, `pass`, `ecr-login`) rather than base64 text.

Private registries. You can run the open-source `registry:2` image (the CNCF Distribution project) for a lab or an internal cache, or use Harbor for a full product with RBAC, replication, scanning and signing. Docker allows plain HTTP only to `localhost` or to hosts listed under `insecure-registries` in `daemon.json`; real deployments need TLS.

Immutability, retention, scanning. ECR supports *tag immutability* (a tag can be pushed only once) and *lifecycle policies* (for example "keep the last 20 images, expire untagged after 7 days"). GHCR and others have retention settings or API-based cleanup. Enable push-time scanning, and prefer deploying by digest.

6 Step-by-step guide

Prerequisites: Docker. Steps 1-5 use a local registry on `localhost:5000` and cost nothing. ECR/GHCR steps (6-7) need an AWS account or a GitHub account; ECR storage is billed per GB-month (a few small images cost cents) and the cleanup step deletes it. Outputs are illustrative.

1. **Run a private registry on your laptop.**

```bash
docker run -d --name registry -p 5000:5000 -v registry-data:/var/lib/registry
  registry:2
curl -s http://localhost:5000/v2/
```

   The curl prints `{}`: the API is alive. The `registry-data` named volume keeps images across restarts.

2. **Tag an image for it and push.**

```bash
docker pull alpine:3.20
docker tag alpine:3.20 localhost:5000/lab/alpine:3.20
docker push localhost:5000/lab/alpine:3.20
```

   You see `Pushed` for the layer and a final line like `3.20: digest: sha256:... size: 528`.

3. **Query the registry API.**

```bash
curl -s http://localhost:5000/v2/_catalog
curl -s http://localhost:5000/v2/lab/alpine/tags/list
```

   Expect `{"repositories":["lab/alpine"]}` and `{"name":"lab/alpine","tags":["3.20"]}`.

4. **Prove a pull comes from the registry.**

```bash
docker rmi localhost:5000/lab/alpine:3.20 alpine:3.20
docker pull localhost:5000/lab/alpine:3.20
docker run --rm localhost:5000/lab/alpine:3.20 echo "served by my registry"
```

5. **Tag the way CI does: SHA + version.**

```bash
SHA=$(git rev-parse --short HEAD 2>/dev/null || echo local1)
docker tag localhost:5000/lab/alpine:3.20 localhost:5000/lab/alpine:git-$SHA
docker tag localhost:5000/lab/alpine:3.20 localhost:5000/lab/alpine:1.0.0
docker push --all-tags localhost:5000/lab/alpine
docker image ls --digests localhost:5000/lab/alpine
```

   All three tags show the *same* digest and image ID: tags are names, content is shared.

6. **Push to Amazon ECR** (real cloud, small cost).

```bash
export AWS_REGION=us-east-1
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
aws ecr create-repository --repository-name shop-api --image-scanning-configuration
  scanOnPush=true \
  --image-tag-mutability IMMUTABLE --region $AWS_REGION
aws ecr get-login-password --region $AWS_REGION | \
  docker login --username AWS --password-stdin
  $ACCOUNT.dkr.ecr.$AWS_REGION.amazonaws.com
docker tag localhost:5000/lab/alpine:3.20
  $ACCOUNT.dkr.ecr.$AWS_REGION.amazonaws.com/shop-api:1.0.0
docker push $ACCOUNT.dkr.ecr.$AWS_REGION.amazonaws.com/shop-api:1.0.0
aws ecr describe-images --repository-name shop-api --region $AWS_REGION
```

`describe-images` lists the image with its digest and tag. Pushing `1.0.0` a second time with different content fails because tags are immutable. Set a billing alarm in your account if you do not have one.

7. **Push to GHCR.**

```bash
echo "$CR_PAT" | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin    #
  CR_PAT = a token with package write scope
docker tag localhost:5000/lab/alpine:3.20 ghcr.io/YOUR_GITHUB_USERNAME/shop-api-
  lab:1.0.0
docker push ghcr.io/YOUR_GITHUB_USERNAME/shop-api-lab:1.0.0
```

The package appears under your GitHub profile's *Packages* tab (private by default for new personal packages; change visibility there if you want it public).

8. **Clean up.**

```bash
docker rm -f registry && docker volume rm registry-data
aws ecr delete-repository --repository-name shop-api --force --region us-east-1
docker logout ghcr.io
```

Delete the GHCR package from its settings page, and revoke the token you created.

7 Real-world scenario

(Illustrative.) **"Production can't scale: ImagePullBackOff."** During a traffic spike the autoscaler starts new nodes, but pods stay in `ImagePullBackOff`. The events say `toomanyrequests` from Docker Hub: dozens of new nodes were pulling `node:22-slim` and

`redis` anonymously from the same NAT IP. Existing nodes were fine because the layers were cached; fresh nodes had nothing. The team mirrored the handful of base images into their own ECR through a pull-through cache, switched Dockerfiles and manifests to the ECR names, and added an alert on registry pull errors. **Lesson:** a third-party registry is a runtime dependency of your autoscaling. Own a copy of what you depend on.

8 Common mistakes and best practices

Don't

- Rely on `latest` or any mutable tag to identify a release.
- Use your account password with `docker login`, commit `~/.docker/config.json`, or paste tokens into Dockerfiles or CI logs.
- Leave a self-hosted registry on plain HTTP or without authentication on a reachable network.
- Let repositories grow forever; storage and scanning costs grow with them.

Do

- Tag with immutable identifiers (git SHA, semver) and deploy by digest; enable tag immutability where available.
- Give CI a narrowly scoped, short-lived credential (OIDC to ECR, `GITHUB_TOKEN` to GHCR); give production hosts pull-only rights.
- Add lifecycle/retention rules, push-time scanning and a pull-through cache for upstream base images.
- Keep the registry in the same region as the compute that pulls from it.

9 Tasks

- [ ] **T1 — Stand up a local registry** *(≈25 min, beginner)*
  - **Goal:** push and pull your own image.
  - **Steps:**
    1. Follow steps 1-4 with an image of your choice.
    2. List repositories and tags with the `/v2/` API.
  - **Done when:** `curl localhost:5000/v2/_catalog` lists your repository and you pulled it back after deleting the local copy.
- [ ] **T2 — Dissect the push** *(≈30 min, intermediate)*
  - **Goal:** observe blobs and manifests.

- **Steps:**
  1. Fetch the manifest: `curl -s -H 'Accept: application/vnd.oci.image.manifest.v1+json, application/vnd.docker.distribution.manifest.v2+json' http://localhost:5000/v2/lab/alpine/manifests/3.20`.
  2. Identify the config digest and layer digests.
  3. Download a blob with `curl http://localhost:5000/v2/lab/alpine/blobs/sha256:<digest>` and verify its `sha256sum`.
- **Done when:** the computed hash matches the digest in the URL.

☐ **T3 — Tagging strategy** *(≈30 min, intermediate)*

- **Goal:** produce SHA, semver and moving tags from one build.
- **Steps:**
  1. Build `shop-api` (any small app) and tag it `git-<sha>`, `1.2.3`, `1.2` and `1`.
  2. Push all tags and show they share one digest.
  3. Write down which tag production should use and why (answer: the digest, tracked from the SHA tag).
- **Done when:** `docker image ls --digests` shows one digest across four tags.

☐ **T4 — Authenticate to a hosted registry** *(≈40 min, intermediate)*

- **Goal:** push to GHCR or ECR.
- **Steps:**
  1. Create a scoped credential (GitHub PAT with package scope or an AWS IAM user/role limited to `ecr:*` on one repository).
  2. Log in using `--password-stdin`, tag, push and pull from a second machine or after `docker rmi`.
  3. Revoke the credential and confirm `docker push` now fails.
- **Done when:** a push succeeds, then fails with an authorization error after revocation. For ECR, run the cleanup step.

☐ **T5 — Break it: retention and immutability** *(≈30 min, advanced)*

- **Goal:** see policy enforcement.
- **Steps:**
  1. In an ECR repository with immutable tags, push `1.0.0`, change the image, and push `1.0.0` again (expect rejection).
  2. Add a lifecycle policy that expires untagged images older than 1 day (`aws ecr put-lifecycle-policy`), and describe what it would remove.

- **Done when:** the second push is rejected with `ImageTagAlreadyExistsException` and the policy JSON is saved in your notes.

☐ **T6 — Prove it: design the registry plan for `shop-api`** *(≈30 min, advanced)*

- **Goal:** justify a registry setup.
- **Steps:**
  1. Without notes, write: which registry, naming convention, tagging scheme, who can push, who can pull, retention, scanning, and how base images are mirrored.
  2. Explain what happens to deployments if Docker Hub is unreachable.
- **Done when:** your plan covers all eight items and a reviewer can find no single point of failure you did not acknowledge.

10 Self-check questions

1. How does Docker decide which registry an image name refers to?
2. Why does a second `docker push` of a slightly changed image upload so little data?
3. Why log in with `--password-stdin` and a token instead of `-p PASSWORD`?
4. How long is an ECR login token valid, and what does that mean for CI?
5. Why mirror base images into your own registry?

ANSWERS

1. By the first path component: if it contains a `.` or `:` (or is `localhost`) it is a registry host; otherwise Docker assumes Docker Hub (`docker.io`).
2. The registry already has the unchanged layers by digest (HEAD returns 200), so only new blobs and the manifest are sent.
3. `-p` leaks the secret into shell history and process lists; tokens can be scoped and revoked, unlike your account password.
4. 12 hours; CI should fetch a fresh token at the start of each job, not cache it.
5. To avoid rate limits and upstream outages affecting your autoscaling and deploys, and to scan and control what enters your supply chain.

11 Further reading

- Docker documentation: "Docker Hub", "Deploy a registry server" and `docker login` reference (docs.docker.com).
- Amazon ECR User Guide — private repositories, lifecycle policies, pull-through cache.
- GitHub Docs — "Working with the Container registry".

- OCI Distribution Specification (opencontainers.org) and the CNCF Distribution (`registry:2`) and Harbor project docs.

LAB NOTES What I ran, what I saw, what it means

4.5 Volumes and Bind Mounts

Topic 5 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

A container's writable layer is **ephemeral**: `docker rm` deletes it, and it is slower than a native filesystem because of copy-on-write. Anything that must outlive the container — database files, uploaded images, a Prometheus time-series store — must live somewhere else. Docker offers three ways to attach storage to a container:

- **Named volume** — storage that Docker creates and manages (by default under `/var/lib/docker/volumes/<name>/_data` on the host). Referenced by name, portable between containers, the right default for data.
- **Bind mount** — a specific host path (`/home/me/project/src`) mapped into the container. Great for development and for injecting config files; tied to the host's directory layout and permissions.
- **tmpfs mount** — an in-memory filesystem that vanishes when the container stops. For scratch data and secrets that must never touch disk.

2 Why it matters in DevOps

- **Data loss is the worst outage.** "We recreated the database container and the data was gone" is a classic, avoidable disaster.
- **Disk-full incidents.** Volumes, images and logs all live under `/var/lib/docker`; knowing what grows where tells you what to clean or move to a bigger disk.
- **Permissions.** Most "permission denied" errors in containers are a UID mismatch on a volume or bind mount (see Non-root Containers §4.11).
- **Backups and portability.** A volume is only as safe as the last tested backup. In Kubernetes the same ideas return as PersistentVolumes: see Volumes, PersistentVolumes and PersistentVolumeClaims §8.9 and, for databases, Backups and Point-in-Time Recovery §13.5.

3 Real-world system example

The official `postgres`, `redis` and `prometheus` images. They are built around volumes. The `postgres` image keeps its cluster in a data directory (`/var/lib/postgresql/data` in most versions; images from PostgreSQL 18 onward move this under `/var/lib/postgresql`, so check the image documentation for your version) and declares it a `VOLUME` so data survives even if you forget to name one (as an anonymous volume). Redis writes its RDB/AOF persistence to `/data`. Prometheus stores its TSDB under `/prometheus` and Grafana keeps dashboards and its SQLite database in `/var/lib/grafana`. In production on a single Docker host, these are usually named volumes on a dedicated disk; on Kubernetes, the equivalent is a PersistentVolumeClaim backed by a cloud disk (for example EBS on AWS). Many teams deliberately run *managed* databases (RDS) so they don't have to run volumes for the database at all.

4 Diagram

Where the data lives:

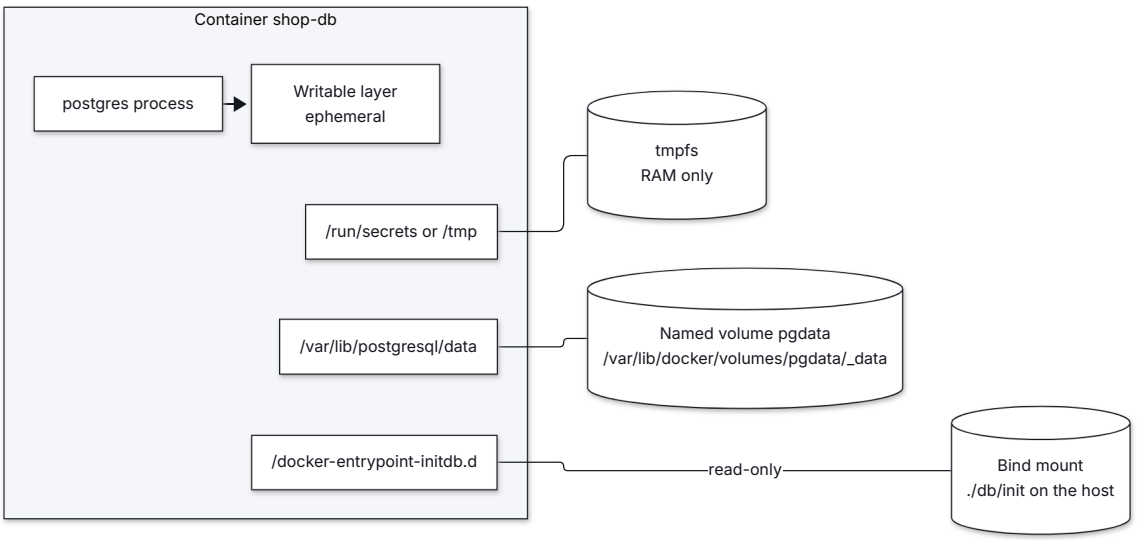


Figure 4.5-1

The lifecycle that makes volumes valuable:

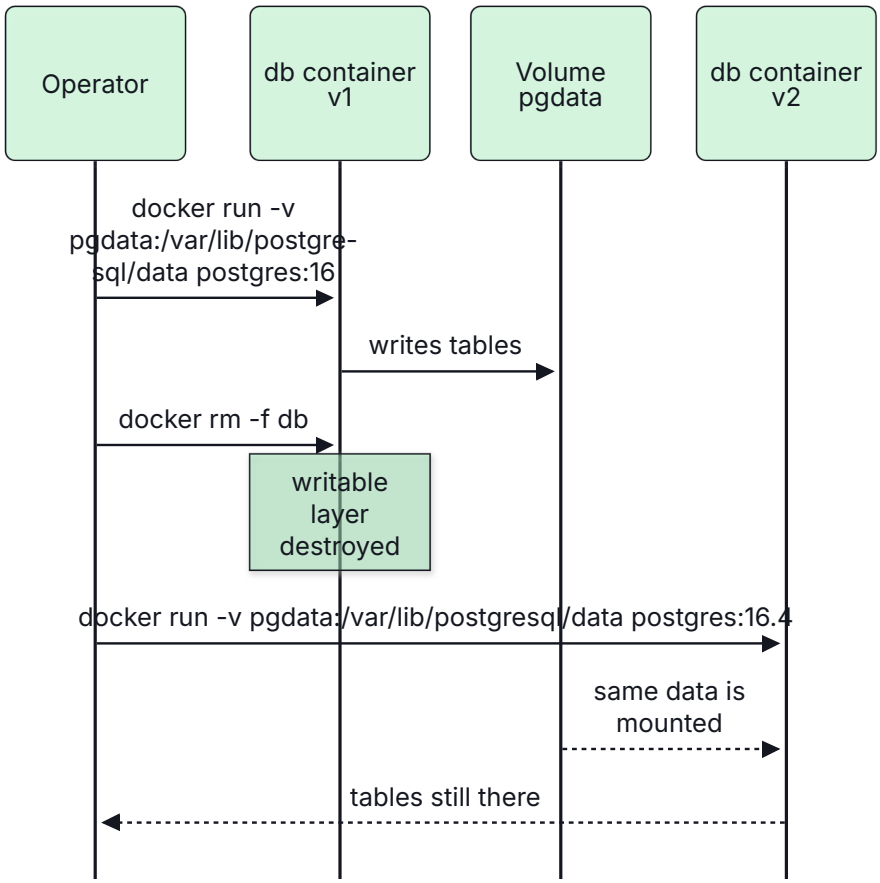


Figure 4.5-2

5 How it works

	Named volume	Bind mount	tmpfs
Created by	Docker (`docker volume create` or on first use)	You (host path must exist with `-v`; `--mount` errors if missing)	Docker, in RAM
Host location	`/var/lib/docker/volumes/NAME/_data`	any path you choose	none (memory)
Survives `docker rm`	Yes (until `docker volume rm`)	Yes (it is your file)	No
Pre-populated from the image	Yes: an empty new volume is initialised with the image's files at that path	No: the host directory hides what the image had there	No

	Named volume	Bind mount	tmpfs
Good for	databases, TSDBs, uploads	source code in dev, config files, certificates	scratch space, secrets, tests
Pitfalls	invisible in your project tree; easy to forget in backups	UID/GID mismatches, host-path coupling, slow on macOS/Windows Docker Desktop, SELinux labels	consumes RAM; size-limit it

Syntax. Two flags do the same thing; prefer the explicit `--mount` for clarity:

```bash
docker run -v pgdata:/var/lib/postgresql/data postgres:16                    # short: named volume
docker run --mount type=volume,src=pgdata,dst=/var/lib/postgresql/data postgres:16
docker run -v "$PWD/conf/nginx.conf:/etc/nginx/nginx.conf:ro" nginx:1.27     # bind, read-only
docker run --mount type=bind,src="$PWD/conf",dst=/etc/nginx/conf.d,readonly nginx:1.27
docker run --tmpfs /tmp:rw,size=64m,mode=1777 alpine                         # tmpfs
docker run --mount type=tmpfs,dst=/tmp,tmpfs-size=67108864 alpine
```

Anonymous volumes. An image with `VOLUME /data` and no `-v` creates an anonymous volume with a random name on every `docker run` — easy to orphan. `docker rm -v` removes a container's anonymous volumes; `docker volume prune` removes unused ones (check what it will delete first).

Persistence and consistency. A volume is just files. Copying live database files while the database runs can produce a corrupt backup. For databases use a logical dump (`pg_dump`) or a proper physical backup tool; for plain files, a `tar` taken through a throw-away container is fine.

Volume drivers. The default `local` driver can mount NFS or other filesystems (`--opt type=nfs ...`); plugins integrate cloud disks. In Swarm and Kubernetes the storage backend is the platform's job.

Compose. Declare named volumes at the top level and reference them from services (see Docker Compose §4.8); `docker compose down` keeps them, `docker compose down -v` deletes them.

6 Step-by-step guide

Prerequisites: Docker. Everything runs locally and is free. Outputs are illustrative.

1. **Prove the writable layer is ephemeral.**

```bash
docker run --name tmp1 alpine sh -c 'echo precious > /data.txt && cat /data.txt'
docker rm tmp1
docker run --rm alpine cat /data.txt
```

The last command fails with `No such file or directory`: a new container starts from the pristine image.

2. **Use a named volume.**

```bash
docker volume create notes
docker run --rm -v notes:/data alpine sh -c 'echo precious > /data/a.txt'
docker run --rm -v notes:/data alpine cat /data/a.txt
docker volume ls
docker volume inspect notes
```

The second container prints `precious`. `inspect` shows `"Mountpoint": "/var/lib/docker/volumes/notes/_data"`.

3. **Persist a PostgreSQL database across container replacement.**

```bash
docker run -d --name db -e POSTGRES_PASSWORD=devpass -e POSTGRES_DB=shop \
  -v pgdata:/var/lib/postgresql/data postgres:16
sleep 8
docker exec db psql -U postgres -d shop -c "CREATE TABLE orders(id serial primary key,
  item text); INSERT INTO orders(item) VALUES ('keyboard');"
docker rm -f db
docker run -d --name db -e POSTGRES_PASSWORD=devpass -v
  pgdata:/var/lib/postgresql/data postgres:16
sleep 8
docker exec db psql -U postgres -d shop -c "SELECT * FROM orders;"
```

The final query returns the `keyboard` row from a brand-new container. (`devpass` is a throw-away lab password. For PostgreSQL 18+ images, mount the volume at `/var/lib/postgresql` instead.)

4. **Use a bind mount for live config, read-only.**

```bash
mkdir -p /tmp/site && echo '<h1>hello from the host</h1>' > /tmp/site/index.html
docker run -d --name web -p 8081:80 -v /tmp/site:/usr/share/nginx/html:ro nginx:1.27
curl -s localhost:8081
echo '<h1>edited live</h1>' > /tmp/site/index.html && curl -s localhost:8081
docker exec web sh -c 'echo x > /usr/share/nginx/html/new.html'
```

   The page changes instantly without a rebuild. The final command fails with `Read-only file system`.

5. **Use tmpfs.**

```bash
docker run --rm --tmpfs /scratch:rw,size=16m alpine sh -c 'df -h /scratch; dd
  if=/dev/zero of=/scratch/f bs=1M count=32'
```

   `df` shows a 16 MB `tmpfs`; the `dd` fails with `No space left on device` after about 16 MB.

6. **Back up and restore a volume with a helper container.**

```bash
docker run --rm -v notes:/data:ro -v "$PWD":/backup alpine tar czf /backup/notes-
  backup.tgz -C /data .
docker volume create notes-restored
docker run --rm -v notes-restored:/data -v "$PWD":/backup alpine tar xzf
  /backup/notes-backup.tgz -C /data
docker run --rm -v notes-restored:/data alpine cat /data/a.txt
```

   The last command prints `precious`. For the database, prefer a logical dump:

```bash
docker exec db pg_dump -U postgres shop > shop-$(date +%F).sql
```

7. **See the UID problem and fix it.**

```bash
mkdir /tmp/owned && docker run --rm -v /tmp/owned:/out alpine sh -c 'echo hi >
  /out/root-file'
ls -l /tmp/owned
docker run --rm -u "$(id -u):$(id -g)" -v /tmp/owned:/out alpine sh -c 'echo hi >
  /out/my-file'
ls -l /tmp/owned
```

The first file is owned by `root`; the second by you. (On Docker Desktop for macOS/Windows ownership is translated, so you may not see the difference.)

8. **Clean up.**

```bash
docker rm -f db web; docker volume rm notes notes-restored pgdata; rm -rf /tmp/site
  /tmp/owned notes-backup.tgz
docker system df -v | head -20   # review what else is using space
```

7 Real-world scenario

(Illustrative.) **"The staging database was empty on Monday."** Over the weekend an engineer ran `docker compose down -v` to "clean up" a colleague's problem. The `-v` flag removed the named volume `pgdata`, and staging, which held three weeks of QA test data, started with an empty database. Nothing in production was touched, but QA lost a week. The team (1) removed `-v` from their cheat sheet and Makefile, (2) added a nightly `pg_dump` to object storage from a sidecar container, (3) labelled volumes (`labels: backup=true`) and wrote a restore runbook, and (4) rehearsed a restore into a new volume. **Lesson:** `down` is safe, `down -v` is data deletion, and a backup you have never restored is a hope.

8 Common mistakes and best practices

Don't

- Store databases or uploads in the container's writable layer.
- Bind-mount `/var/run/docker.sock` or host system directories into containers unless you accept root-equivalent access.
- Run `docker volume prune` or `docker system prune --volumes` on a host you do not fully understand.
- Back up a running database by copying its files, or mount the same volume read-write into two database containers.

Do

- Use named volumes for data, bind mounts for dev code and config (read-only with `:ro`), tmpfs for scratch and secrets.
- Give volumes meaningful names and labels; keep Docker's data root on a separate disk with monitoring.
- Back up with `pg_dump`/native tools, copy the backup off-host, and test the restore.

- Match container UID/GID with volume ownership (run as non-root and `chown` at build time, or use `--user`).

9 Tasks

- [ ] **T1 — Lose data, then keep it** *(≈20 min, beginner)*
  - **Goal:** feel the difference between the writable layer and a volume.
  - **Steps:**
    1. Run steps 1 and 2 of the guide.
    2. Write down in one sentence why the first test lost data and the second did not.
  - **Done when:** `precious` is printed from a *new* container using the named volume, and you demonstrated the loss without it.
- [ ] **T2 — Database survives an upgrade** *(≈30 min, intermediate)*
  - **Goal:** replace a container while keeping the data.
  - **Steps:**
    1. Run `postgres:16` with a named volume and insert rows.
    2. Remove the container and start `postgres:16.4` (or any newer 16.x tag) on the same volume.
    3. Query the table.
  - **Done when:** the rows are still present and `SELECT version();` shows the newer minor version.
- [ ] **T3 — Hot-reload with a bind mount** *(≈30 min, intermediate)*
  - **Goal:** develop without rebuilding.
  - **Steps:**
    1. Create a tiny Node.js server that returns a string; run it with `docker run -v "$PWD":/app -w /app node:22-alpine node --watch server.js`.
    2. Edit the string on the host.
  - **Done when:** `curl` shows the new string without rebuilding or restarting the container yourself.
- [ ] **T4 — Break it: UID mismatch** *(≈30 min, intermediate)*
  - **Goal:** reproduce and fix a classic permission error.
  - **Steps:**
    1. Create a host directory owned by your user with mode `700`, and run a container as `-u 1234:1234` writing into it (expect `Permission denied`).

2. Fix it by `chown 1234:1234` on the host directory, or by running with `-u $(id -u):$(id -g)`.

- **Done when:** the write fails, then succeeds, and you can state the rule: numeric IDs matter, not names.

- [ ] **T5 — Automate backup and restore** *(≈45 min, advanced)*
  - **Goal:** a script that backs up a named volume and restores it into a new one.
  - **Steps:**
    1. Write `backup.sh VOLUME` producing `VOLUME-YYYYMMDD.tgz` using the helper-container pattern.
    2. Write `restore.sh VOLUME FILE` that creates the volume and unpacks the archive.
    3. Round-trip a Postgres volume: dump with `pg_dump`, destroy the volume, restore, verify the rows.
  - **Done when:** the `orders` rows come back after you deliberately deleted the volume.

- [ ] **T6 — Prove it: choose the mount type** *(≈20 min, advanced)*
  - **Goal:** justify storage choices.
  - **Steps:**
    1. Without notes, decide and justify the mount type for each `shop-api` stack item: Postgres data, Redis AOF, Nginx config, Prometheus TSDB, API temp upload buffer, TLS private key.
    2. State for each: backup strategy and who owns the files.
  - **Done when:** each item has a type, a backup answer and an ownership answer, and you can explain why none uses the writable layer.

10 Self-check questions

1. What happens to data in a container's writable layer when you run `docker rm`?
2. When would you pick a bind mount over a named volume, and when the opposite?
3. What does `docker compose down -v` do that `docker compose down` does not?
4. Why is copying a running database's files a bad backup method?
5. What is an anonymous volume and why is it a problem?

ANSWERS

1. It is deleted with the container.

2. Bind mount: development source code, injected config or certificates you manage on the host. Named volume: application data that Docker should manage and that must be portable between hosts' directory layouts.
3. It also deletes the named volumes declared in the Compose file — i.e. your data.
4. Files may be mid-write and inconsistent; use `pg_dump` or a physical backup tool that is transaction-aware.
5. A volume with a random name created from an image's `VOLUME` instruction when none is specified; it is easy to orphan and hard to identify later.

11 Further reading

- Docker documentation: "Volumes", "Bind mounts" and "tmpfs mounts" (docs.docker.com).
- Docker documentation: "Back up, restore, or migrate data volumes".
- The Docker Official Image pages for `postgres`, `redis`, `prometheus` and `grafana` — data directories and permissions notes.
- PostgreSQL documentation: "Backup and Restore".

4.6 Docker Networks

Topic 6 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

Every container gets its own **network namespace**: its own `eth0`, IP address, routing table and port space. Docker connects those namespaces with virtual networks. The most important network type is the **bridge**: a virtual Layer-2 switch inside the host, with each container attached through a virtual Ethernet pair (`veth`). Containers on the same user-defined bridge can reach each other by **container or service name** thanks to Docker's built-in DNS server, and the host reaches the outside world through NAT (Network Address Translation).

To let something *outside* the Docker network (your browser, a load balancer) reach a container, you **publish a port** (`-p 8080:80`), which installs a forwarding rule from a host port to the container's port.

2 Why it matters in DevOps

- **"Connection refused" and "could not resolve host"** between containers are the most common Compose problems, and nearly all are network-name or port mistakes.
- **Security:** a published port is an open door. Binding the database to `0.0.0.0:5432` on a cloud VM is how databases end up on the public internet. Segmenting networks (front-end vs backend) limits blast radius.
- **Performance and debugging:** NAT, userland proxies and DNS add latency; host networking removes it at the cost of isolation.
- **It maps directly to Kubernetes:** pod networking, Services and cluster DNS are the same ideas at larger scale. Background: Routing, NAT, Ports and Firewalls §2.4 and DNS Resolution §2.5.

3 Real-world system example

Docker Compose stacks (and Kubernetes pods). A typical `shop-api` development stack has Nginx, the API, PostgreSQL and Redis. Compose creates one project network; the API connects to `postgres://db:5432` and `redis://cache:6379` using *service names*, and only Nginx publishes a port to the host. The same pattern, "only the edge is reachable, everything else is internal", is how production platforms are laid out: in Kubernetes pods find each other through Service DNS names (`db.default.svc.cluster.local`), and only the ingress controller or load balancer is exposed. Multi-host container networking in Docker Swarm uses **overlay** networks (VXLAN tunnels between hosts); Kubernetes delegates this to CNI plugins such as Calico or Cilium.

4 Diagram

A user-defined bridge network with DNS and one published port:

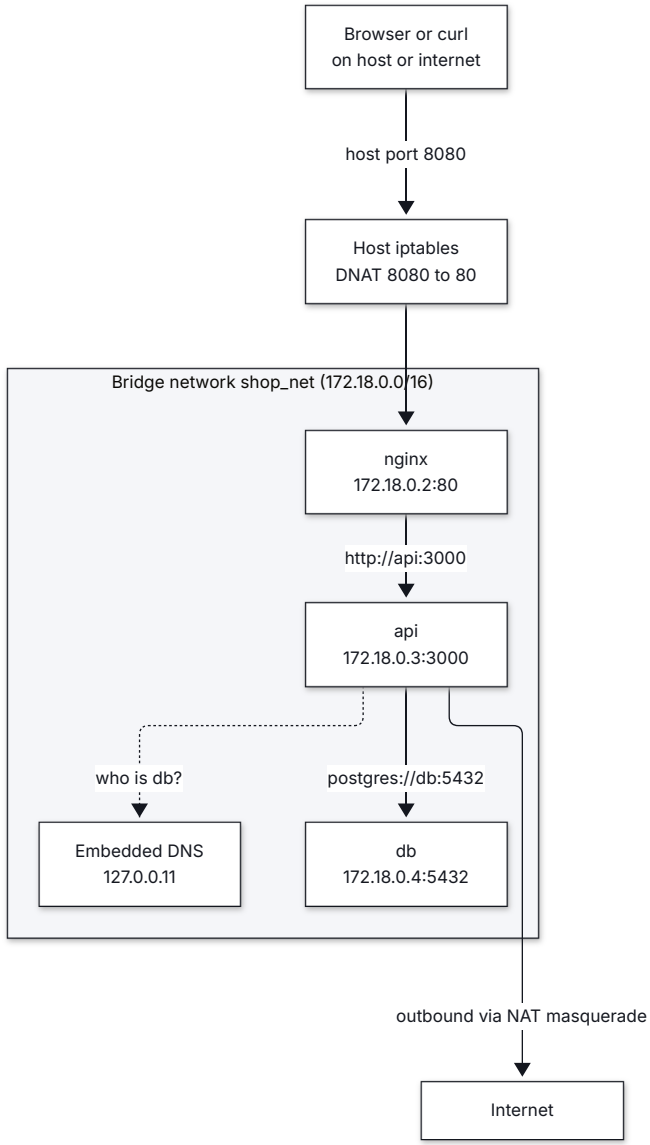


Figure 4.6-1

Name resolution for one request:

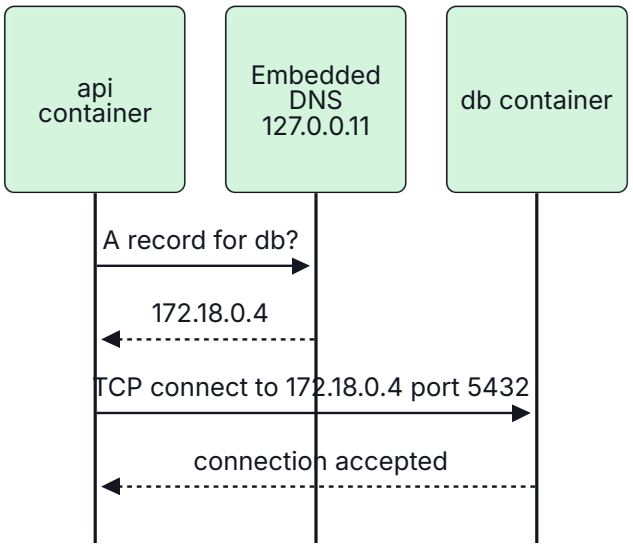


Figure 4.6-2

5 How it works

Network drivers

Driver	What it does	Typical use
`bridge` (default)	Private virtual switch on one host; NAT to the outside	Single-host apps, Compose
`host`	Container shares the host's network namespace — no isolation, no port mapping	Performance-critical or network tooling; Linux hosts
`none`	Only a loopback interface	Fully isolated batch jobs, security tests
`overlay`	Multi-host network over VXLAN (needs Swarm mode)	Swarm services across nodes
`macvlan` / `ipvlan`	Container gets an address on the physical LAN	Legacy apps that need a real LAN IP

Default bridge vs user-defined bridge. Docker always creates a default network called `bridge` (interface `docker0`). Containers on it can reach each other by IP address only. A **user-defined bridge** (`docker network create`) adds automatic DNS by container name, better isolation and the ability to connect/disconnect containers live. Compose always creates a user-defined network for each project, so **never rely on the default bridge**.

Embedded DNS. On user-defined networks the container's `/etc/resolv.conf` points to `127.0.0.11`; Docker answers for container names, **network aliases** and Compose service names, and forwards everything else to the host's resolvers. Name lookups only work between containers that share at least one network. If a container is on several networks, it can resolve names on each.

Port publishing. `-p HOST_PORT:CONTAINER_PORT[/proto]` creates NAT rules (iptables/nftables DNAT, plus a userland proxy for some paths).

- `-p 8080:80` binds **all** host interfaces. `-p 127.0.0.1:8080:80` binds loopback only, the right choice for anything not meant to be public.
- `-P` publishes every `EXPOSE`d port on random high ports. `EXPOSE` itself publishes nothing; it is documentation (and a hint for `-P`).
- Containers talk to each other on the **container port** (`api:3000`), never the host port.
- Docker edits firewall rules directly, so published ports can **bypass host firewalls such as UFW**. Bind to `127.0.0.1` or put a real firewall/security group in front.

Useful commands: `docker network ls`, `docker network inspect NET`, `docker network create --driver bridge --subnet 172.30.0.0/24 NET`, `docker network connect NET CONTAINER`, `docker network disconnect`, `docker port CONTAINER`, `docker run --network NET --network-alias NAME`. Add `--internal` to a network to cut off outbound internet access completely.

6 Step-by-step guide

Prerequisites: Docker on Linux, macOS or Windows. Everything is local and free. Outputs are illustrative. A Linux host is needed for the `host` network demo (step 6).

1. **Look at what Docker created.**

```bash
docker network ls
docker network inspect bridge --format '{{range .IPAM.Config}}{{.Subnet}}{{end}}'
```

You see `bridge`, `host` and `none`. The default bridge subnet is commonly `172.17.0.0/16`.

2. **Show that the default bridge has no name resolution.**

```bash
docker run -d --name a1 alpine sleep 600
docker run -d --name a2 alpine sleep 600
A2_IP=$(docker inspect -f '{{.NetworkSettings.IPAddress}}' a2)
docker exec a1 ping -c1 "$A2_IP"      # works
docker exec a1 ping -c1 a2            # fails: bad address 'a2'
```

3. **Create a user-defined bridge and use DNS.**

```bash
docker network create shop_net
docker run -d --name db  --network shop_net -e POSTGRES_PASSWORD=devpass postgres:16
docker run -d --name api --network shop_net alpine sleep 600
docker exec api ping -c1 db
docker exec api cat /etc/resolv.conf
docker exec api nslookup db 2>/dev/null | tail -3
```

`ping db` succeeds, and `resolv.conf` shows `nameserver 127.0.0.11`. Names work because both containers are on a user-defined network.

4. **Publish a port, then restrict it.**

```bash
docker run -d --name web1 -p 8080:80 nginx:1.27
docker port web1
curl -sI localhost:8080 | head -1
docker run -d --name web2 -p 127.0.0.1:8081:80 nginx:1.27
docker ps --format 'table {{.Names}}\t{{.Ports}}'
```

`web1` shows `0.0.0.0:8080→80/tcp` (reachable from other machines); `web2` shows `127.0.0.1:8081→80/tcp` (this machine only).

5. **Prove "container port, not host port" inside the network.**

```bash
docker network connect shop_net web1
docker exec api wget -qO- http://web1:80 | head -3     # works: container port 80
docker exec api wget -qO- http://web1:8080 || echo "refused: 8080 is only the host
  port"
```

6. **Try host and none networking** (Linux).

```bash
docker run --rm --network host alpine ip -br addr | head -3      # the host's own
  interfaces
docker run --rm --network none alpine ip -br addr                # only lo
```

7. **Segment networks and block egress.**

```bash
docker network create --internal backend
docker run -d --name secret-db --network backend -e POSTGRES_PASSWORD=devpass
  postgres:16
docker run --rm --network backend alpine sh -c 'ping -c1 -W2 1.1.1.1 || echo "no
  internet, as intended"'
docker network connect backend api      # api is now on shop_net and backend
docker exec api ping -c1 secret-db
```

8. **Clean up.**

```bash
docker rm -f a1 a2 db api web1 web2 secret-db
docker network rm shop_net backend
```

7 Real-world scenario

(Illustrative.) **"The API can't reach Postgres, but `psql` from my laptop works."** After moving `shop-api` into Compose, the API logged `ECONNREFUSED 127.0.0.1:5432`. The configuration still said `DB_HOST=localhost`: inside the API container, `localhost` is the API container itself, not the host or the database. The team changed it to the service name `db`. A second bug appeared when someone "fixed" it by using the host port from `ports: "15432:5432"`: `db:15432` is refused, since containers use the **container** port 5432. Finally, a security review noticed `ports: "5432:5432"` exposed the database on the VM's public IP; they removed the mapping entirely (only the API needs the database) and used `docker compose exec db psql` for admin work. **Lesson:** `localhost` means "this container"; use service names and container ports inside the network, and publish only what outsiders need.

8 Common mistakes and best practices

Don't

- Use `localhost` or the host's IP to reach another container.

- Rely on container IP addresses — they change on every recreate.
- Publish database, cache or metrics ports on `0.0.0.0` on an internet-facing host.
- Use `--network host` for convenience in production; you lose isolation and port independence.
- Use `--link`; it is legacy. Use user-defined networks and aliases.

Do

- Create one user-defined network per application (Compose does this), and split into `frontend` / `backend` networks when appropriate.
- Publish only the edge service (Nginx) and bind admin ports to `127.0.0.1`.
- Use service names + container ports in connection strings, configured via environment variables.
- Debug with `docker network inspect`, `docker exec ... nslookup/ping/wget`, and a throw-away `nicolaka/netshoot` container on the same network.

9 Tasks

- [ ] **T1 — DNS by name** *(≈25 min, beginner)*
  - **Goal:** prove name resolution works on a user-defined network and not on the default bridge.
  - **Steps:**
    1. Run steps 2 and 3 of the guide.
    2. Record which `ping` succeeded and which failed, with the error message.
  - **Done when:** you demonstrate `ping db` working on `shop_net` and `ping a2` failing on `bridge`.
- [ ] **T2 — Publish smartly** *(≈20 min, beginner)*
  - **Goal:** control exposure.
  - **Steps:**
    1. Publish Nginx on `127.0.0.1:9090:80`.
    2. From the host, `curl localhost:9090`; then from another machine/VM or by using the host's LAN IP, confirm it is refused.
  - **Done when:** loopback works and the LAN address does not.
- [ ] **T3 — Two-tier network** *(≈40 min, intermediate)*
  - **Goal:** build `nginx → api → db` where `db` is unreachable from `nginx`.
  - **Steps:**

1. Create networks `front` and `back`. Put `nginx` on `front`, `db` on `back`, `api` (a simple container) on both.
2. From `nginx`, try `nslookup db`; from `api`, ping both.

- **Done when:** `nginx` cannot resolve `db` while `api` reaches everything.

- [ ] **T4 — Break it: wrong port and wrong host** *(≈30 min, intermediate)*
  - **Goal:** diagnose the two classic errors.
  - **Steps:**
    1. Run Postgres with `-p 15432:5432` on a user-defined network and a client container on the same network.
    2. Try `psql -h localhost`, `-h db -p 15432`, and `-h db -p 5432`; note which work.
    3. Write the one-line rule for each failure.
  - **Done when:** only `-h db -p 5432` succeeds and your rules mention "localhost is the container itself" and "container port, not host port".

- [ ] **T5 — Inspect the plumbing** *(≈40 min, advanced)*
  - **Goal:** see the bridge, veth and NAT rules on the host (Linux).
  - **Steps:**
    1. Start two containers on `shop_net`; run `ip -br link` and `bridge link` on the host to find `br-<id>` and the `veth*` interfaces.
    2. Run `sudo iptables -t nat -S | grep -E 'DOCKER|MASQUERADE'` (or `sudo nft list ruleset | grep -i docker`) before and after publishing a port.
    3. Match the new rule to the published port.
  - **Done when:** you can point at the DNAT rule created by `-p` and the masquerade rule that gives containers internet access.

- [ ] **T6 — Prove it: explain a request path** *(≈30 min, advanced)*
  - **Goal:** narrate `curl localhost:8080` all the way to the database.
  - **Steps:**
    1. Without notes, write the hops: host port → DNAT → nginx container → DNS lookup of `api` → API → DNS lookup of `db` → Postgres.
    2. Mark which hops use host ports, container ports and DNS.
  - **Done when:** your path matches the diagrams in section 4 and you can say where each of `localhost`, `127.0.0.11`, `172.18.x.x` appears.

10 Self-check questions

1. Why can containers on a user-defined bridge reach each other by name, but not on the default bridge?
2. What does `-p 127.0.0.1:8080:80` do differently from `-p 8080:80`?
3. Inside the `api` container, how should it reach PostgreSQL in a Compose service called `db`?
4. What does `EXPOSE 3000` actually do?
5. When is `--network host` reasonable, and what do you give up?

ANSWERS

1. Only user-defined networks get Docker's embedded DNS server (127.0.0.11); the default bridge supports IP addresses only.
2. It binds the published port to loopback only, so it is unreachable from other machines; the second form binds all interfaces.
3. `db:5432` — the service name and the container port, never `localhost` and never the published host port.
4. It documents the port (image metadata) and enables `-P`; it publishes nothing and does not open a firewall.
5. For network tools or very latency-sensitive workloads on Linux; you give up network isolation and port remapping, and risk port conflicts.

11 Further reading

- Docker documentation: "Networking overview", "Bridge network driver", "Publishing and exposing ports" (docs.docker.com).
- Docker documentation: "Packet filtering and firewalls" — how Docker interacts with iptables and UFW.
- The `nicolaka/netshoot` project README — a debugging toolbox container.
- Kubernetes documentation: "Cluster Networking" and "Service" — the next step up from single-host networking.

4.7 Dockerfile

Topic 7 of 14 | ~5 hours | Difficulty: Intermediate

1 What it is

A **Dockerfile** is a plain-text recipe that tells the builder (BuildKit, driven by `docker build` or `docker buildx build`) how to assemble an image: start `FROM` a base image, copy in your code, run install commands, set the user and the default command. Each instruction that changes the filesystem produces a layer §4.2; instructions that only set metadata (`ENV`, `CMD`, `EXPOSE`, `USER`) edit the image configuration.

The directory you pass to `docker build` (usually `.`) is the **build context**. Everything in it is sent to the builder, and `COPY` can only read from it. A file called `.dockerignore` in the context root excludes paths from that upload, which makes builds faster and keeps secrets out of the image.

2 Why it matters in DevOps

- **The Dockerfile is your deployment artifact's source code.** Reviewers should read it as carefully as application code: it decides image size, build speed, who the process runs as, and what vulnerabilities ship.
- **Reproducible builds** (pinned bases, lockfile installs) are the difference between "works on my machine" and "the same bytes everywhere".
- **Most image-security findings start here:** running as root, secrets baked into layers, unpinned `latest`, build tools left in the final image.
- **CI time and cost.** A badly ordered Dockerfile rebuilds `npm install` on every commit (next topic: Image Caching and Layer Optimization §4.9).

3 Real-world system example

The official Docker images. The Dockerfiles for `nginx`, `postgres`, `redis` and `node` are public on GitHub (the "docker-library" project) and are a good reference. The `nginx` image ends with `ENTRYPOINT ["/docker-entrypoint.sh"]`, `CMD ["nginx", "-g", "daemon off;"]` and

`STOPSIGNAL SIGQUIT`: the entrypoint script runs init steps (templating config, for example) and then `exec "$@"` so the real server becomes PID 1, while `CMD` supplies the default server command that you can override. The `postgres` image's entrypoint does the same thing, additionally creating the database cluster on first start and running any `*.sql` files in `/docker-entrypoint-initdb.d`. Google's `distroless` images and Chainguard images are published as base images precisely so application Dockerfiles can end with a tiny, shell-less runtime stage. When companies say "we containerise with a golden Dockerfile", they usually mean a reviewed template of the pattern below.

4 Diagram

From Dockerfile to a running container:

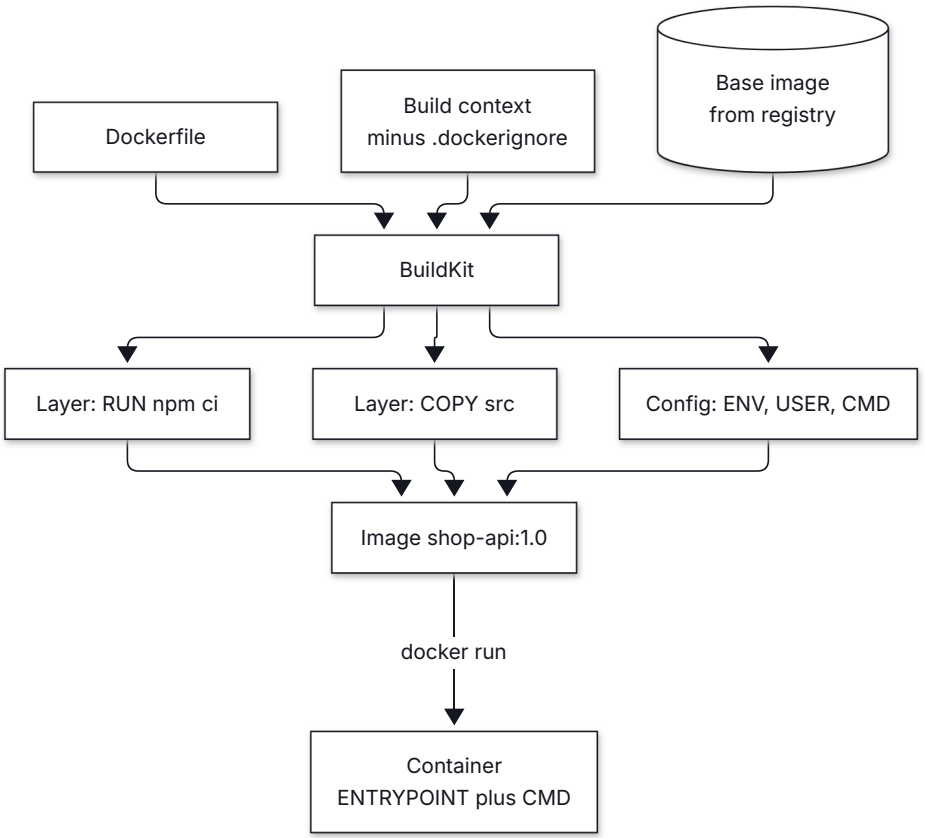


Figure 4.7-1

How ENTRYPOINT and CMD combine into the final command:

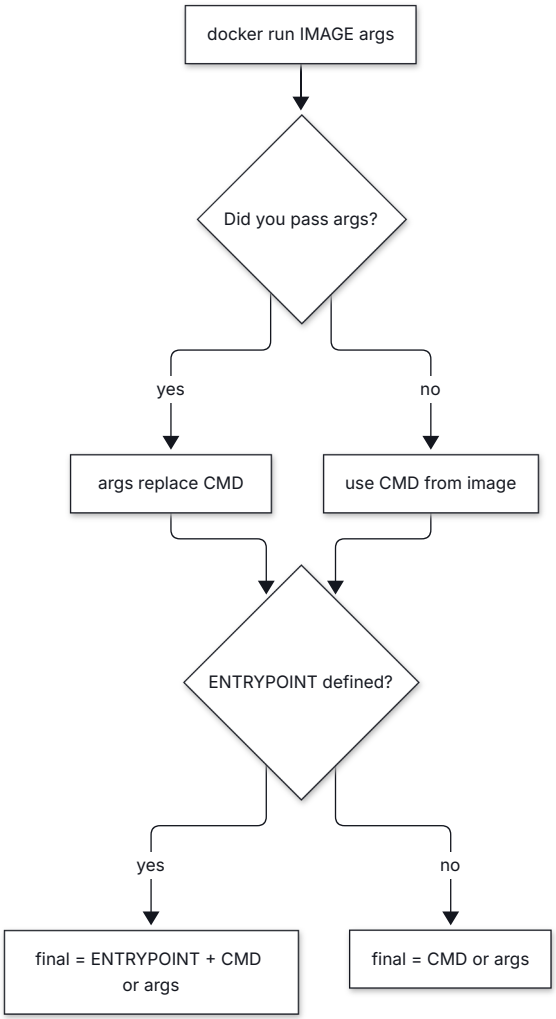


Figure 4.7-2

5 How it works

Core instructions

Instruction	Purpose	Notes
`FROM image[:tag] [@digest] [AS name]`	Start a build stage	Pin a version; multi-stage uses several `FROM`s
`WORKDIR /app`	Set (and create) the working directory	Prefer over `RUN cd`

Instruction	Purpose	Notes
`COPY [--chown=u:g] src dest`	Copy from context (or `--from=stage`)	Use `COPY`, not `ADD`, unless you need auto-extract or a URL
`RUN cmd`	Run a command at build time, creating a layer	Chain related commands with `&&`; `--mount=type=cache/secret` for BuildKit extras
`ENV KEY=value`	Environment variable in the image **and** at runtime	Visible in `docker inspect`
`ARG NAME[=default]`	Build-time variable	Not in the running container's environment
`EXPOSE 3000`	Documents a port	Publishes nothing
`USER 10001`	Switch to a non-root user for later steps and runtime	See Non-root Containers §4.11
`CMD` / `ENTRYPOINT`	What runs on `docker run`	See below
`HEALTHCHECK`	How Docker tests health	See Health Checks §4.12
`LABEL`	Metadata (OCI annotations such as `org.opencontainers.image.source`)	
`VOLUME`, `STOPSIGNAL`, `SHELL`, `ONBUILD`	Declare a volume path / change stop signal / change shell / trigger on child builds	`ONBUILD` is rarely wise

CMD vs ENTRYPOINT.

- `ENTRYPOINT` is "the program this image *is*"; `CMD` is "its default arguments". `docker run IMAGE x y` replaces `CMD` only; `--entrypoint` replaces the entrypoint.
- Pattern 1 — **CMD only** (most application images): `CMD ["node", "server.js"]`; users can run `docker run image sh` to get a shell.
- Pattern 2 — **ENTRYPOINT + CMD** (tools and wrapper scripts): `ENTRYPOINT ["python", "-m", "http.server"]` + `CMD ["8000"]` makes the image behave like a CLI.
- Always use **exec form** (JSON array). Shell form (`CMD node server.js`) runs under `/bin/sh -c`, which becomes PID 1 and swallows `SIGTERM` (see Containers §4.3). A wrapper entrypoint script must end with `exec "$@"` for the same reason.

ARG vs ENV

	ARG	ENV
Available during build	Yes	Yes (after it is set)
Available in the running container	**No**	**Yes**

	ARG	ENV
Set from the CLI	`docker build --build-arg NAME=value`	`docker run -e NAME=value` overrides the default
Stored in the image	In build history (`docker history` can show it)	In the image config
Use for	versions, build flags, `NODE_ENV` choice at build	runtime defaults such as `PORT=3000`
Secrets?	**Never**	**Never** — use `RUN --mount=type=secret` at build and runtime secrets from the platform

A common pattern converts one to the other: `ARG APP_VERSION=dev` then `ENV APP_VERSION=$APP_VERSION`. An `ARG` declared before the first `FROM` is only usable in `FROM` lines; redeclare it inside a stage to use it there.

.dockerignore (same location as the context root; globbing similar to `.gitignore`):

```
node_modules
npm-debug.log
.git
.gitignore
.env
.env.*
Dockerfile
docker-compose*.yml
coverage
*.md
```

Without it, `COPY . .` drags in your local `node_modules` (wrong OS binaries), `.git` history, and `.env` secrets, and invalidates the cache whenever any of them change.

6 Step-by-step guide

Prerequisites: Docker Engine/Desktop with BuildKit (the default builder in current versions). Outputs are illustrative.

1. **Create a tiny app and a first Dockerfile.**

```bash
mkdir -p ~/lab/dockerfile && cd ~/lab/dockerfile
cat > app.js <<'EOF'
const http = require('http');
const port = process.env.PORT || 3000;
const server = http.createServer((req, res) => {
```

```
  res.end(JSON.stringify({ app: 'shop-api', version: process.env.APP_VERSION,
  greeting: process.env.GREETING }) + '\n');
});
server.listen(port, () => console.log('listening on ' + port));
process.on('SIGTERM', () => server.close(() => process.exit(0)));
EOF
cat > Dockerfile <<'EOF'
FROM node:22-alpine
WORKDIR /app
COPY app.js .
ENV PORT=3000 GREETING=hello
EXPOSE 3000
CMD ["node", "app.js"]
EOF
docker build -t dockerfile-lab:1 .
docker run -d --name lab -p 3000:3000 dockerfile-lab:1
curl -s localhost:3000
```

The build prints one step per instruction; curl returns `{"app":"shop-api","greeting":"hello"}` (`version` is absent because it is undefined).

2. **Override ENV at runtime, and add ARG.** Edit the Dockerfile to add after `WORKDIR`:

DOCKERFILE

```
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION
```

BASH

```
docker rm -f lab
docker build --build-arg APP_VERSION=1.2.3 -t dockerfile-lab:2 .
docker run -d --name lab -p 3000:3000 -e GREETING=bonjour dockerfile-lab:2
curl -s localhost:3000
docker exec lab printenv | grep -E 'APP_VERSION|GREETING'
```

The response shows `"version":"1.2.3","greeting":"bonjour"`: the ARG was baked into an ENV at build time, and `-e` overrode the ENV default at runtime.

3. **See the difference between ARG and ENV in the image.**

BASH

```
docker history --no-trunc dockerfile-lab:2 | grep -E 'APP_VERSION|GREETING'
docker image inspect dockerfile-lab:2 --format '{{json .Config.Env}}'
```

Both appear in history and config. This is why neither is a place for secrets.

4. **Prove ENTRYPOINT vs CMD behaviour.**

```bash
mkdir -p ../ep && cd ../ep
printf 'FROM alpine:3.22\nENTRYPOINT ["echo", "hello"]\nCMD ["world"]\n' > Dockerfile
docker build -t ep-lab .
docker run --rm ep-lab
docker run --rm ep-lab there
docker run --rm --entrypoint sh ep-lab -c 'echo replaced entrypoint'
```

   Output: `hello world`, then `hello there` (args replaced CMD), then `replaced entrypoint`.

5. **Make shell form fail to stop quickly, then fix it.**

```bash
cd ../dockerfile
sed 's|CMD \["node", "app.js"\]|CMD node app.js; echo "node exited"|' Dockerfile >
  Dockerfile.shell
docker build -f Dockerfile.shell -t dockerfile-lab:shell .
docker run -d --name shellform dockerfile-lab:shell
docker exec shellform ps
```

   `ps` shows PID 1 is `/bin/sh -c node app.js; echo "node exited"` with node as a child. (Our app handles SIGTERM, but PID 1 is the shell, which does not forward it, so `docker stop` waits the full timeout then kills.) Compare with `docker exec lab ps`, where `node app.js` is PID 1. Note: some shells optimise a *lone* command in `sh -c` by exec-ing it directly, which can hide the problem; wrappers like `npm start` or compound commands like the one above do not get that optimisation.

6. **Add `.dockerignore` and watch the context shrink.**

```bash
mkdir -p junk && dd if=/dev/zero of=junk/big.bin bs=1M count=100 2>/dev/null
docker build -t dockerfile-lab:3 . 2>&1 | grep -i 'transferring context'
printf 'junk\n.git\n.env\nDockerfile*\n' > .dockerignore
docker build -t dockerfile-lab:4 . 2>&1 | grep -i 'transferring context'
```

   The first build transfers about 100 MB of context (lines like `transferring context: 100.00MB`); the second transfers a few kB.

7. **Lint it.** Run a Dockerfile linter such as Hadolint (`docker run --rm -i hadolint/hadolint < Dockerfile`) and BuildKit's own checks:

```bash
docker build --check .
```

Fix warnings (for example "FROM should be pinned", "JSON arguments recommended for CMD").

8. **The real `shop-api` Dockerfile** (used again in the capstone, combined with later topics):

```dockerfile
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY server.js ./
ENV NODE_ENV=production PORT=3000
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

9. **Clean up.**

```bash
docker rm -f lab shellform; docker rmi dockerfile-lab:1 dockerfile-lab:2 dockerfile-lab:3 dockerfile-lab:4 dockerfile-lab:shell ep-lab
rm -rf ~/lab/dockerfile ~/lab/ep
```

7 Real-world scenario

(Illustrative.) **"The image is 1.4 GB and has our `.env` in it."** A security scan of `shop-api` flagged a database password in an image layer. The Dockerfile used `COPY . .` with no `.dockerignore`, so the developer's local `.env`, `node_modules` (built for macOS), and the whole `.git` directory were all copied in. Fixing it took four changes: add a `.dockerignore`; `COPY package*.json` first and run `npm ci --omit=dev`; stop using `ENV DB_PASSWORD=...` and inject secrets at runtime; rotate the leaked password. The image dropped from about 1.4 GB to a few hundred MB and the build stopped re-installing dependencies on every commit. **Lesson:** the build context is an upload, and `COPY . .` copies whatever happens to be on your disk.

8 Common mistakes and best practices

Don't

- `FROM node:latest` or any unpinned base; `ADD` for local files; `COPY . .` without `.dockerignore`.
- Put secrets in `ENV`, `ARG`, or `COPY`ed files — they persist in layers and image metadata.

- Use shell-form `CMD` / `ENTRYPOINT` for the main process, or `RUN apt-get update` on its own line (stale cached package lists).
- Run the app as root, install editors/debug tools in production images, or `curl | bash` in builds.

Do

- Pin base images (`node:22-alpine` , ideally with `@sha256:`), copy dependency manifests before source, and chain `RUN` steps that belong together.
- Use exec-form `CMD` , `USER` with a numeric non-root ID, and `LABEL org.opencontainers.image.*` for provenance.
- Keep a `.dockerignore` ; lint with Hadolint and `docker build --check` ; scan the result.
- Prefer multi-stage builds for compiled languages and bundlers (Multi-stage Builds §4.10).

9 Tasks

- [ ] **T1 — Containerize a tiny app** *(≈30 min, beginner)*
  - **Goal:** write your first Dockerfile from scratch.
  - **Steps:**
    1. Create the `app.js` and Dockerfile from step 1 without copying.
    2. Build, run with `-p 3000:3000` and `curl` it.
  - **Done when:** `curl localhost:3000` returns the JSON and `docker ps` shows the container `Up` .
- [ ] **T2 — ARG vs ENV experiment** *(≈30 min, intermediate)*
  - **Goal:** demonstrate the lifetime of each.
  - **Steps:**
    1. Add `ARG BUILD_TOKEN=none` and `ENV RUNTIME_MODE=dev` ; build with `--build-arg BUILD_TOKEN=abc` .
    2. Run `docker run --rm IMAGE printenv` and `docker history --no-trunc IMAGE` .
    3. Override `RUNTIME_MODE` with `-e` .
  - **Done when:** `printenv` shows `RUNTIME_MODE` but not `BUILD_TOKEN` , `history` reveals `abc` , and you can explain why that proves ARG is not a secret store.
- [ ] **T3 — Control the entrypoint** *(≈30 min, intermediate)*
  - **Goal:** build a CLI-style image.
  - **Steps:**

1. Build an image with `ENTRYPOINT ["node", "-e"]` and `CMD ["console.log('default')"]`. (Or use `curl`: `ENTRYPOINT ["curl", "-s"]`, `CMD ["https://example.com"]`.)
2. Run it with no args, with a custom argument, and with `--entrypoint sh`.

- **Done when:** all three invocations behave as the ENTRYPOINT/CMD rules predict.

☐ **T4 — Break it: context bloat and leaked secret** *(≈40 min, intermediate)*

- **Goal:** feel what `.dockerignore` protects.
- **Steps:**
  1. Create a fake `.env` containing `API_KEY=FAKE-123` and a 100 MB junk file, then build with `COPY . .` and no `.dockerignore`.
  2. Run `docker run --rm IMAGE cat /app/.env`.
  3. Add `.dockerignore`, rebuild, and repeat.
- **Done when:** the first image prints `FAKE-123`, the second fails with "No such file", and the context size dropped.

☐ **T5 — Write the production Dockerfile for `shop-api`** *(≈60 min, intermediate)*

- **Goal:** produce a reviewed Dockerfile you will reuse in the capstone.
- **Steps:**
  1. Create the `shop-api` source (from the capstone) or any Node.js app with `package.json`.
  2. Write the Dockerfile from step 8, a matching `.dockerignore`, and lint it with Hadolint.
  3. Run it and confirm `docker exec CONTAINER id` reports a non-root user.
- **Done when:** Hadolint reports no errors, the container runs as UID 1000 (`node`) and `docker stop` completes in under 2 seconds.

☐ **T6 — Prove it: review a bad Dockerfile** *(≈30 min, advanced)*

- **Goal:** spot problems like a senior reviewer.
- **Steps:**
  1. Write a deliberately bad Dockerfile (latest tag, root, `COPY . .`, shell-form CMD, secret in ENV, `apt-get update` alone).
  2. Without notes, list every issue and the one-line fix for each, then fix the file.
- **Done when:** you found at least 6 issues and the fixed version builds and passes Hadolint.

10 Self-check questions

1. What is the build context and why does `.dockerignore` matter?
2. What is the difference between `CMD` and `ENTRYPOINT`, and how do `docker run IMAGE args` and `--entrypoint` change them?
3. Why is shell-form `CMD` a problem?
4. How do `ARG` and `ENV` differ in lifetime and visibility? Which one is safe for secrets?
5. Why should `COPY package*.json` come before `COPY . .`?
6. When would you use `ADD` instead of `COPY`?

ANSWERS

1. The directory uploaded to the builder; `COPY` can only read from it. `.dockerignore` keeps large or sensitive files out (smaller, faster, safer) and avoids spurious cache invalidations.
2. `ENTRYPOINT` is the fixed program, `CMD` the default arguments. Arguments after the image name replace `CMD`; `--entrypoint` replaces `ENTRYPOINT`.
3. It runs under `/bin/sh -c`, so the shell is PID 1 and does not forward signals to your app: slow, ungraceful stops.
4. `ARG` exists only during the build (but shows in history); `ENV` persists in the image and the container. Neither is safe for secrets — use BuildKit secret mounts at build and runtime secret injection.
5. So the expensive dependency-install layer is cached until the manifest changes, not on every source edit.
6. Rarely: to auto-extract a local tar archive or fetch a remote URL/Git source — otherwise prefer `COPY` for predictability.

11 Further reading

- Docker documentation: "Dockerfile reference" and "Building best practices" (docs.docker.com).
- Docker documentation: "Build secrets" and `.dockerignore` reference.
- Hadolint (github.com/hadolint/hadolint) — Dockerfile linter rules with explanations.
- The `docker-library` GitHub organisation — Dockerfiles of the official images, good examples to read.

LAB NOTES What I ran, what I saw, what it means

4.8 Docker Compose

Topic 8 of 14 | ~5 hours | Difficulty: Intermediate

1 What it is

Docker Compose lets you describe a whole multi-container application — services, networks, volumes, configuration, startup order — in one YAML file and run it with a single command. Instead of typing five long `docker run` commands in the right order with the right flags, you write `compose.yaml` and run `docker compose up`. The file format is the open **Compose Specification**; it has **no top-level** `version:` **key** any more (current Compose ignores it and warns). Compose V2 is a Docker CLI plugin, invoked as `docker compose` (with a space), not the old Python `docker-compose` binary.

A Compose file has top-level sections: `services` (the containers), `networks`, `volumes`, and optionally `configs` and `secrets`. Compose groups everything it creates under a **project** (default name = directory name, or the top-level `name:`), prefixing resource names so that two copies of a stack do not collide.

2 Why it matters in DevOps

- **Local development parity:** "clone, `docker compose up`" gives every developer the same PostgreSQL, Redis and API versions — no more "works on my machine".
- **CI integration tests:** pipelines start the stack, run tests against it and tear it down; flaky tests usually trace back to startup order (the `depends_on` section below).
- **Small production:** many single-host deployments (side projects, internal tools, edge devices) run on Compose. For anything needing multiple hosts, rolling updates and self-healing, the next step is Kubernetes (Level 8).
- **A readable architecture document:** the Compose file shows which services exist, who talks to whom, and which ports are exposed.
- See also Docker Networks §4.6 and Volumes and Bind Mounts §4.5, whose concepts Compose declares in YAML.

3 Real-world system example

The development stack for a typical web product. Public repositories for projects such as Grafana's observability stacks, Sentry's self-hosted distribution and many open-source web apps ship a `docker-compose.yml` / `compose.yaml` that starts the application together with PostgreSQL, Redis and supporting services. For example, Sentry's self-hosted installer is driven by Compose, and the Grafana, Prometheus and Loki projects publish Compose examples in their documentation. Companies commonly use the same approach for local development and CI service containers, then deploy the *images* (not the Compose file) to Kubernetes or ECS. The capstone of this level builds exactly such a stack for `shop-api`.

4 Diagram

What `docker compose up` creates for a project named `shop`:

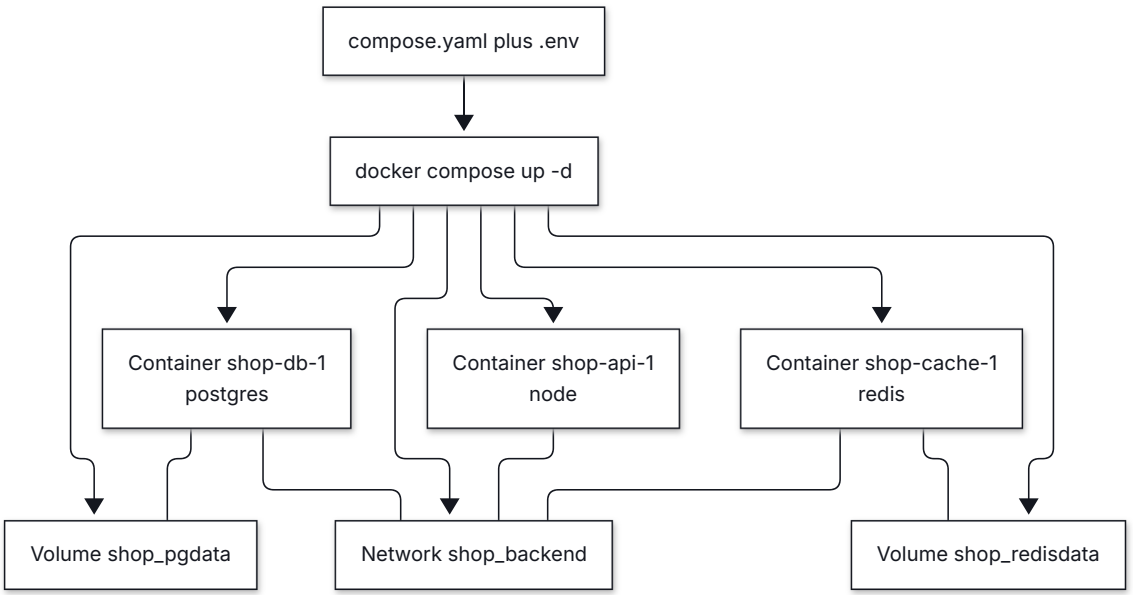


Figure 4.8-1

Startup ordering with health gating:

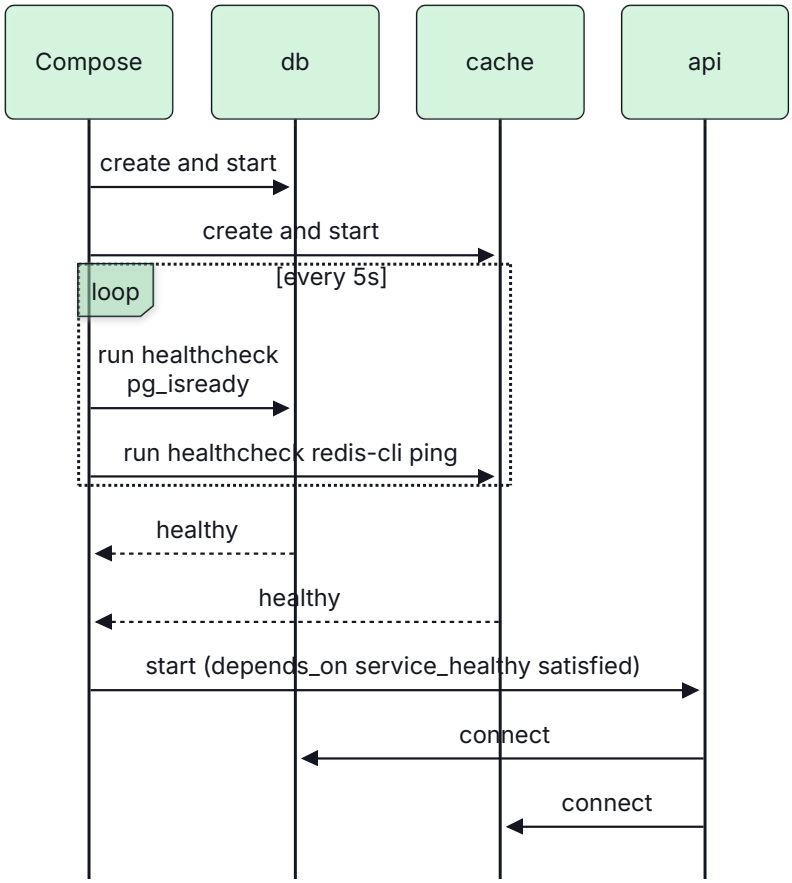


Figure 4.8-2

5 How it works

Anatomy of a service

Key	Meaning
`image` / `build`	use a pulled image or build from a Dockerfile (`build: ./api` , or `{context, dockerfile, args, target}`); both together build and tag
`ports`	publish `"HOST:CONTAINER"` ; prefix `127.0.0.1:` for local-only
`environment` / `env_file`	set variables inline or from files
`volumes`	named volumes, bind mounts (`./src:/app/src`), tmpfs
`networks`	which networks the service joins; others cannot reach it

Key	Meaning
`depends_on`	start order, optionally gated on health (below)
`healthcheck`	how Compose/Docker decides a service is healthy
`restart`	`no`, `on-failure`, `always`, `unless-stopped`
`command` / `entrypoint` / `init` / `user` / `read_only`	override image defaults
`profiles`	only start the service when the profile is enabled
`deploy.resources` / `mem_limit` / `cpus`	resource limits

`depends_on` **and health.** Plain `depends_on: [db]` only orders *container start*; it does **not** wait for PostgreSQL to accept connections, so the API can crash on boot. The long form waits for the dependency's health check:

```yaml
depends_on:
  db:
    condition: service_healthy     # wait until db's healthcheck passes
```

Other conditions are `service_started` (default) and `service_completed_successfully` (for one-shot jobs such as migrations). Combine with an application that retries its connections anyway. See Health Checks §4.12.

Profiles. Services with `profiles: ["tools"]` are skipped by default and started with `docker compose --profile tools up` (or `COMPOSE_PROFILES=tools`). Use them for debug UIs, seeders and one-off jobs.

Configuration and variable precedence. Two different mechanisms are easy to confuse:

- `.env` **file** (next to `compose.yaml`) feeds **interpolation** inside the Compose file: `${POSTGRES_PASSWORD}`, `${TAG:-dev}` (default), `${POSTGRES_PASSWORD:?error message}` (required).
- `env_file:` / `environment:` set variables **inside the container**. Precedence for container variables, highest first: `docker compose run -e` > `environment:` > `env_file:` > the image's `ENV`. Shell variables take priority over `.env` for interpolation. Never commit real secrets: commit `.env.example`, git-ignore `.env`, and use Compose `secrets:` or your platform's secret store for anything sensitive.

Multiple files. `compose.yaml` plus an automatically merged `compose.override.yaml` is the classic dev-override pattern; `-f compose.yaml -f compose.prod.yaml` merges explicitly.

Everyday commands

Command	Purpose
`docker compose up -d [--build] [--wait]`	create/start in background; `--wait` blocks until healthy
`docker compose ps` / `logs -f [svc]`	status / follow logs
`docker compose exec svc sh` / `run --rm svc cmd`	shell in a running service / one-off container
`docker compose config`	render and validate the merged, interpolated file
`docker compose down`	stop and remove containers and networks (**volumes kept**)
`docker compose down -v`	the same **plus named volumes (data deleted)**
`docker compose pull` / `build` / `restart svc`	update images / rebuild / restart one service

6 Step-by-step guide

Prerequisites: Docker with the Compose plugin (`docker compose version` prints a version). Everything is local and free. Outputs are illustrative; `docker compose config` output was checked against a real Compose binary, running containers was not possible in the authoring environment.

1. **Create the project skeleton.** We use the `shop-api` source from the capstone (L4 Capstone Capstone 4). If you don't have it yet, any Node.js app with a Dockerfile works for `api`.

```bash
mkdir -p ~/lab/shop/api ~/lab/shop/db ~/lab/shop/config && cd ~/lab/shop
# put server.js, package.json, package-lock.json, Dockerfile in ./api and init.sql in
  ./db
```

2. **Write the environment files.** `.env` is used for interpolation; `config/api.env` is passed into the API container.

```bash
printf 'POSTGRES_PASSWORD=change-me-dev-only\n' > .env
printf 'LOG_LEVEL=info\nCACHE_TTL_SECONDS=30\n' > config/api.env
printf '.env\n' >> .gitignore
```

3. **Write `compose.yaml`.**

```yaml
name: shop
```

```
services:
  api:
    build: ./api
    image: shop-api:dev
    env_file:
      - config/api.env
    environment:
      DATABASE_URL: postgres://shop:${POSTGRES_PASSWORD}@db:5432/shop
      REDIS_URL: redis://cache:6379
    ports:
      - "127.0.0.1:3000:3000"
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_healthy
    restart: unless-stopped
    init: true
    networks:
      - backend

  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s
    restart: unless-stopped
    networks:
      - backend

  cache:
    image: redis:7.4-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redisdata:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped
    networks:
      - backend
```

```
  adminer:
    image: adminer:5
    profiles: ["tools"]
    ports:
      - "127.0.0.1:8081:8080"
    depends_on:
      db:
        condition: service_healthy
    networks:
      - backend

networks:
  backend:

volumes:
  pgdata:
  redisdata:
```

Note: the database is **not** published to the host — only the API (loopback) is.

4. **Validate before running.**

```bash
docker compose config --quiet && echo "compose file OK"
docker compose config --services
```

The second command lists `db`, `cache` and `api` (not `adminer`, because its profile is off). Try removing `.env` temporarily: `config` fails with `required variable POSTGRES_PASSWORD is missing a value: set POSTGRES_PASSWORD in .env`.

5. **Start the stack and wait for health.**

```bash
docker compose up -d --build --wait
docker compose ps
```

`ps` shows `db` and `cache` as `healthy` and `api` as `running`. Check the application:

```bash
curl -s localhost:3000/readyz
curl -s localhost:3000/products
```

6. **Read logs and exec into services.**

```bash
docker compose logs --tail 5 api
docker compose exec db psql -U shop -d shop -c 'SELECT count(*) FROM products;'
docker compose exec cache redis-cli keys '*'
```

7. **Enable the profile for a debug UI.**

```bash
docker compose --profile tools up -d adminer
```

   Open `http://localhost:8081` (System: PostgreSQL, Server: `db`, user `shop`, password from `.env`).

8. **Prove health gating.** Stop the database and recreate the API:

```bash
docker compose stop db
docker compose up -d --force-recreate api      # waits for db to be healthy first
docker compose start db
docker compose ps
```

   Compose starts `db` again and the API becomes `running` only after `db` reports `healthy`.

9. **Tear down.** Note the difference:

```bash
docker compose down            # keeps pgdata and redisdata
docker compose up -d --wait && curl -s localhost:3000/products | head -c 80   # data
  still there
docker compose down -v         # deletes volumes too
```

7 Real-world scenario

(Illustrative.) **"CI is flaky: the integration tests pass one run in four."** The pipeline ran `docker compose up -d` and immediately `npm test`. `depends_on: [db]` guaranteed that the PostgreSQL *container* had started, not that PostgreSQL was ready: on slow CI runners the API tried to connect while the database was still initialising and exited, and the tests failed with `ECONNREFUSED`. The fix: a `pg_isready` healthcheck on `db`, `condition: service_healthy` in the API's `depends_on`, and `docker compose up -d --wait` in the pipeline so the test step only

starts once everything is healthy. They also added a retry-with-backoff loop to the API's startup connection. Flakiness dropped to zero. **Lesson:** "started" is not "ready"; health checks turn timing guesses into facts.

8 Common mistakes and best practices

Don't

- Add `version: "3.8"` at the top (obsolete), or use the old `docker-compose` binary in new docs.
- Publish database/cache ports on `0.0.0.0` (or at all) when only other containers need them.
- Commit `.env` with real credentials, or hard-code passwords in `compose.yaml`.
- Use `latest` tags, or run `docker compose down -v` casually.
- Use `localhost` for service-to-service connections; use service names.

Do

- Pin image tags, add health checks, and gate dependencies with `condition: service_healthy`.
- Name volumes, keep data services on a private network, and publish only the edge on `127.0.0.1` or behind Nginx.
- Use `docker compose config` in CI to validate, `.env.example` for documentation, profiles for optional tools, and `--wait` in scripts.
- Set `restart:` policies, `init: true`, memory limits and log rotation (`logging: options: max-size`).

9 Tasks

- [ ] **T1 — Bring up the stack** *(≈40 min, beginner)*
  - **Goal:** run `api`, `db` and `cache` with Compose.
  - **Steps:**
    1. Follow steps 1-5.
    2. Call `/readyz` and `/products` twice and note the `X-Cache` header changes from MISS to HIT (`curl -si`).
  - **Done when:** `docker compose ps` shows three healthy/running services and the second `/products` returns `X-Cache: HIT`.
- [ ] **T2 — Profiles and overrides** *(≈30 min, intermediate)*

- **Goal:** add optional services and a dev override.
- **Steps:**
  1. Bring up the `tools` profile and open Adminer.
  2. Create `compose.override.yaml` that bind-mounts `./api/server.js` into the API container and sets `LOG_LEVEL=debug`; run `docker compose config` to see the merged result.
- **Done when:** `docker compose config` shows the bind mount and the override, and Adminer lists the `products` table.

☐ **T3 — Break it: remove the healthcheck** *(≈30 min, intermediate)*

- **Goal:** reproduce start-order flakiness.
- **Steps:**
  1. Change `depends_on` for `api` to the short form `[db, cache]` and remove the `db` healthcheck.
  2. Run `docker compose down && docker compose up -d` several times and check `docker compose logs api`.
  3. Restore the long form and compare.
- **Done when:** you captured at least one failed connection in the API logs without health gating and none with it (the effect is timing dependent; try three runs).

☐ **T4 — Persistence rules** *(≈30 min, intermediate)*

- **Goal:** prove which command keeps data.
- **Steps:**
  1. `POST` an order (`curl -XPOST -H 'content-type: application/json' -d '{"product_id":1,"quantity":2}' localhost:3000/orders`).
  2. Run `docker compose down`, `up -d --wait`, and count orders in `psql`.
  3. Run `docker compose down -v`, `up -d --wait`, and count again.
- **Done when:** the order exists after step 2 and is gone after step 3, and you can explain why.

☐ **T5 — Automate with a Makefile or script** *(≈40 min, advanced)*

- **Goal:** make the stack a one-command workflow.
- **Steps:**
  1. Write `make up` (`docker compose up -d --build --wait`), `make test` (smoke `curl` checks that exit non-zero on failure), `make logs`, `make down`.
  2. Run `make up test down` from a clean checkout in CI (a GitHub Actions job is a good target: see Docker Image Builds and Registries in CI §5.4).

- **Done when:** `make up test down` exits 0 on a clean machine and non-zero when you stop `db` first.

- [ ] **T6 — Prove it: explain the file** *(≈30 min, advanced)*
  - **Goal:** justify every line of your Compose file.
  - **Steps:**
    1. Without notes, add a `worker` service that runs the same image with `command: ["node","worker.js"]`, shares the network, and starts only after `db` is healthy.
    2. For each service explain: why it has (or lacks) published ports, which network it is on, where its data lives, how it is restarted.
  - **Done when:** `docker compose config` validates and a reviewer agrees with your answers.

10 Self-check questions

1. Why is there no `version:` key in modern Compose files?
2. What does plain `depends_on` guarantee, and what does `condition: service_healthy` add?
3. What is the difference between `.env` and `env_file:`?
4. What is the difference between `docker compose down` and `docker compose down -v`?
5. How would you run an optional debug container only when asked?
6. Why shouldn't you publish PostgreSQL's port in a production Compose file?

ANSWERS

1. The Compose Specification is versionless; the key is obsolete and ignored (with a warning).
2. Plain `depends_on` orders container start only; `service_healthy` waits for the dependency's healthcheck to pass.
3. `.env` provides values for `${...}` interpolation in the Compose file itself; `env_file:` injects variables into the container's environment.
4. `down` removes containers and networks; `down -v` also deletes named volumes, destroying data.
5. Give the service a `profiles:` entry and start it with `--profile NAME` (or `COMPOSE_PROFILES`).
6. Anything that reaches the host's public interface can reach the database; only the API on the same private network needs it.

11 Further reading

- Compose Specification (compose-spec.io) and Docker documentation: "Docker Compose" overview, "Use service profiles", "Control startup order".
- Docker documentation: "Environment variables in Compose" — precedence rules.
- `docker compose --help` and `docker compose config` — the best self-service debugging tools.
- Docker Samples repository on GitHub (`docker/awesome-compose`) — worked examples.

4.9 Image Caching and Layer Optimization

Topic 9 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

Building an image runs the Dockerfile instructions top to bottom, and each filesystem-changing instruction produces a layer §4.2. The builder (BuildKit) keeps a **build cache**: before executing an instruction it asks "have I already run this exact instruction on top of this exact parent layer with these exact inputs?" If yes, it reuses the stored layer and prints `CACHED` instead of doing the work again.

Layer optimization is the craft of writing a Dockerfile so that (a) most builds hit the cache, because the things that change often are listed *last*, and (b) the final image contains only what it needs to run. Fast builds and small images are the same discipline viewed from two sides.

2 Why it matters in DevOps

- **Developer and CI time.** Re-running `npm ci` or `pip install` on every commit can turn a 20-second build into a 5-minute one. Multiply by every pull request, every day.
- **Money.** CI minutes, registry storage, cross-region transfer and node disk space all scale with how many bytes you rebuild and ship.
- **Deployment speed.** Smaller images pull faster, which shortens autoscaling reaction time and rolling-deploy duration.
- **Security surface.** Fewer packages in the final image means fewer CVEs to patch (see Image Scanning and Container Security §4.14).
- See also Docker Image Builds and Registries in CI §5.4 for sharing the cache between CI runs.

3 Real-world system example

BuildKit in GitHub Actions and Docker's own tooling. Docker's official GitHub Actions (`docker/build-push-action`, `docker/setup-buildx-action`) are built around BuildKit's exportable cache: a CI runner is a fresh VM every time, so without exporting the cache the pipeline starts cold on every run. The documented cache backends include `type=gha` (the GitHub Actions cache), `type=registry` (store cache as an image in your registry) and `type=local`. Official language base images such as `node`, `python` and `golang` are rebuilt regularly upstream, which is why cached `FROM` layers refresh whenever you pull a newer base. Projects that publish many images, such as the Docker official images themselves, rely on layer sharing so that variants (`alpine`, `slim`, `bookworm`) of the same app do not duplicate work.

4 Diagram

How cache invalidation cascades:

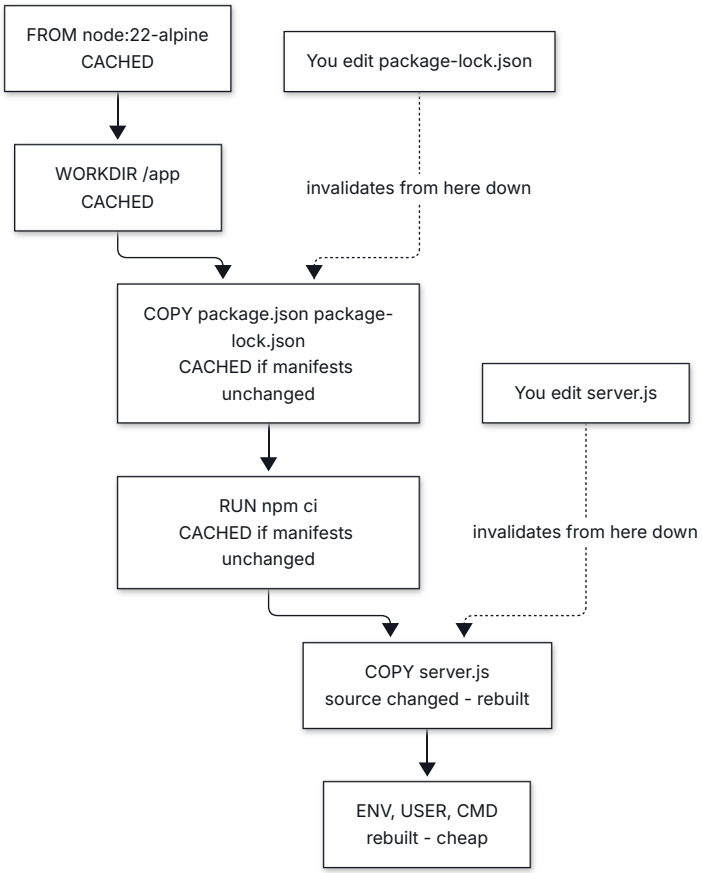


Figure 4.9-1

Bad order versus good order:

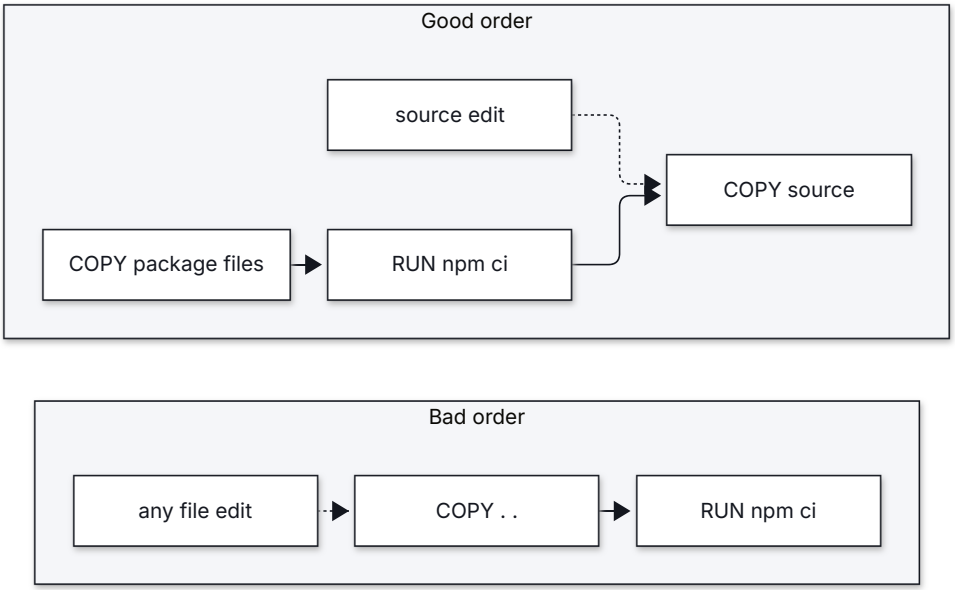


Figure 4.9-2

In the bad order every source edit re-runs `npm ci`; in the good order it only re-copies source.

5 How it works

Cache rules (simplified)

1. A layer is reused only if the **parent layer** is reused. One miss invalidates everything below it.
2. For `RUN`, the cache key is the **command text** (plus parent). It does *not* notice that `apt-get install` would now fetch newer packages — a cached `RUN apt-get update` stays stale forever (until you rebuild with `--no-cache` or change the line).
3. For `COPY`/`ADD`, the key includes a **checksum of the copied files' content and metadata**. Any change to any copied file invalidates that layer and all below.
4. `ARG` values used by a later instruction are part of its key; changing a `--build-arg` invalidates from the first use.
5. A newer upstream base image does not invalidate cache until you `docker pull`/`--pull` it; CI should build with `--pull` on a schedule.

The five habits

Habit	Why
Order instructions from **least to most frequently changing**	maximise cache hits
`COPY` dependency manifests first, install, **then** `COPY` the source	source edits don't re-install dependencies
Use `.dockerignore`	stops irrelevant changes (`.git`, logs, docs) invalidating `COPY . .`
Combine `RUN` steps that belong together and **clean up in the same step**	deleted files in a later layer still occupy space
Install only what you need: `npm ci --omit=dev`, `apt-get install --no-install-recommends`, no build tools in the runtime image	smaller image, fewer CVEs

BuildKit cache mounts. A *cache mount* gives a `RUN` step a persistent directory that is **not part of the image** and survives between builds, so package managers can reuse downloaded files even when the layer itself must re-run:

```
DOCKERFILE
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev
RUN --mount=type=cache,target=/var/cache/apt --mount=type=cache,target=/var/lib/apt \
    apt-get update && apt-get install -y --no-install-recommends curl
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt
```

Also useful: `--mount=type=bind` (read source files without a `COPY` layer) and `--mount=type=secret` (use a credential during one `RUN` without it being stored in any layer). For Debian/Ubuntu images that auto-clean apt caches, remove `/etc/apt/apt.conf.d/docker-clean` when using an apt cache mount (the Docker documentation shows this).

Size-reduction toolbox

Technique	Typical effect
Smaller base (`-slim`, `-alpine`, distroless — see Minimal Base Images §4.13)	hundreds of MB
Multi-stage build §4.10: compile in one stage, copy only artifacts	removes compilers, dev dependencies, sources
`npm ci --omit=dev` / `pip install --no-cache-dir`	tens to hundreds of MB
`apt-get install --no-install-recommends` + `rm -rf /var/lib/apt/lists/*` in the same `RUN`	tens of MB
`.dockerignore`	smaller context; no accidental junk

Technique	Typical effect
Don't `chown -R` big trees in a separate layer — use `COPY --chown=`	avoids duplicating the whole tree in a new layer

CI caching. Export the cache from the builder: `docker buildx build --cache-from type=registry,ref=REG/shop-api:buildcache --cache-to type=registry,ref=REG/shop-api:buildcache,mode=max ...` (`mode=max` also caches intermediate stages). On GitHub Actions use `type=gha`.

Inspecting results: `docker build --progress=plain .` (see every step and `CACHED`), `docker history IMAGE`, `docker image ls`, and the third-party tool `dive` to explore layer contents.

6 Step-by-step guide

Prerequisites: Docker with BuildKit (default in current Docker). Needs internet access to download npm packages from the registry (small). Times and sizes below are illustrative; yours will differ.

1. **Create a small Node project with a real dependency.**

```bash
mkdir -p ~/lab/cache && cd ~/lab/cache
npm init -y >/dev/null && npm install express --package-lock-only >/dev/null
echo "console.log('v1')" > server.js
ls   # package.json  package-lock.json  server.js
```

2. **Build with the naive Dockerfile and time it.**

```bash
cat > Dockerfile.bad <<'EOF'
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN npm ci --omit=dev
CMD ["node", "server.js"]
EOF
time docker build -f Dockerfile.bad -t cache-lab:bad .
echo "console.log('v2')" > server.js
time docker build -f Dockerfile.bad -t cache-lab:bad .
```

The second build re-runs `npm ci` (you see the install output again and the time is similar to the first) even though only `server.js` changed.

3. **Fix the order.**

```bash
cat > Dockerfile <<'EOF'
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY server.js ./
CMD ["node", "server.js"]
EOF
time docker build -t cache-lab:good .
echo "console.log('v3')" > server.js
time docker build --progress=plain -t cache-lab:good . 2>&1 | grep -E
  'CACHED|COPY|RUN'
```

On the rebuild `RUN npm ci` is marked `CACHED` and only the final `COPY` runs: typically a second or less instead of tens of seconds.

4. **Add a BuildKit cache mount.**

```bash
sed -i 's|RUN npm ci --omit=dev|RUN --mount=type=cache,target=/root/.npm npm ci --
  omit=dev|' Dockerfile
time docker build -t cache-lab:mount .
npm install --package-lock-only lodash >/dev/null           # change the manifest on
  purpose
time docker build -t cache-lab:mount .
```

Changing the manifest invalidates the `RUN` layer, but the cache mount means previously downloaded tarballs are reused: the install is faster than a cold one.

5. **Measure size and see where it goes.**

```bash
docker image ls cache-lab
docker history cache-lab:good
```

The `node_modules` layer is the biggest after the base image. Compare `node:22` (Debian, large) to `node:22-slim` and `node:22-alpine` with `docker image ls node`.

6. **Show that cleanup must share a layer.**

```bash
cat > Dockerfile.apt <<'EOF'
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates
  \
    && rm -rf /var/lib/apt/lists/*
EOF
docker build -f Dockerfile.apt -t cache-lab:apt .
printf 'FROM debian:bookworm-slim\nRUN apt-get update && apt-get install -y --no-
  install-recommends curl ca-certificates\nRUN rm -rf /var/lib/apt/lists/*\n' >
  Dockerfile.apt2
docker build -f Dockerfile.apt2 -t cache-lab:apt2 .
docker image ls cache-lab
```

`apt2` is larger than `apt` by roughly the size of the apt lists (tens of MB): the separate `rm` layer did not reclaim it.

7. **Use a secret mount (no trace in layers).**

```bash
echo "super-secret-token" > /tmp/npm_token
printf 'FROM alpine:3.22\nRUN --mount=type=secret,id=npm_token cat
  /run/secrets/npm_token | wc -c\n' > Dockerfile.secret
docker build --progress=plain --secret id=npm_token,src=/tmp/npm_token -f
  Dockerfile.secret -t cache-lab:secret .
docker history cache-lab:secret --no-trunc | grep -c super-secret-token || true
```

The build prints `19` (the token length incl. newline); the history grep count is `0`.

8. **Clean up.**

```bash
docker rmi cache-lab:bad cache-lab:good cache-lab:mount cache-lab:apt cache-lab:apt2
  cache-lab:secret
docker builder prune -f      # remove build cache (this deletes your cache mounts too)
rm -rf ~/lab/cache /tmp/npm_token
```

7 Real-world scenario

(Illustrative.) **"CI takes 9 minutes and the pull-request queue is backed up."** A `shop-api` pipeline spent most of its time in `docker build`: each run started on a fresh runner (empty cache) and the Dockerfile had `COPY . .` before `npm ci`. Two changes: reorder to copy the manifests first and add `--cache-from/--cache-to type=gha,mode=max` (cache exported between runs), and add `--mount=type=cache,target=/root/.npm` for the occasional dependency

bump. Typical PR builds (source-only changes) dropped from about 9 minutes of build time to well under one, and the image shrank by removing dev dependencies with `--omit=dev`. A scheduled nightly build with `--pull --no-cache` makes sure base-image security fixes are not hidden behind a stale cache. **Lesson:** cache hit-rate is a pipeline metric; refresh deliberately so "cached" never means "forever outdated".

8 Common mistakes and best practices

Don't

- `COPY . .` before installing dependencies.
- Put `apt-get update` on its own `RUN` line (cached stale), or clean up in a *later* layer.
- Copy secrets or credentials into layers to use them for one step.
- Rely on the cache for security freshness: a cached `RUN apt-get upgrade` never upgrades again.

Do

- Order layers by volatility, keep `.dockerignore` tight, and use `npm ci`/lockfiles for deterministic installs.
- Use `RUN --mount=type=cache` for package managers and `--mount=type=secret` for credentials.
- Export the cache in CI and rebuild from scratch on a schedule with `--pull --no-cache`.
- Track image size over time and fail CI if it grows past a budget.

9 Tasks

- [ ] **T1 — Measure the cache** *(≈30 min, beginner)*
  - **Goal:** see `CACHED` and its absence.
  - **Steps:**
    1. Build the bad and good Dockerfiles from steps 2-3.
    2. Change only `server.js` and rebuild each, recording build time and which steps print `CACHED`.
  - **Done when:** you have a table of times and the good Dockerfile shows `npm ci` as cached.
- [ ] **T2 — Reorder a real Dockerfile** *(≈30 min, intermediate)*
  - **Goal:** fix a cache-hostile file.
  - **Steps:**

1. Take the `shop-api` Dockerfile and put `COPY . .` first.
2. Measure a source-only rebuild, then restore the right order and measure again.

- **Done when:** the second rebuild is clearly faster and `docker build --progress=plain` shows the install layer as `CACHED`.

- [ ] **T3 — Cache mounts** *(≈30 min, intermediate)*
  - **Goal:** speed up installs when the manifest changes.
  - **Steps:**
    1. Add `--mount=type=cache,target=/root/.npm` to the install step.
    2. Add a dependency, rebuild, and compare with a rebuild using `--no-cache` and no mount.
  - **Done when:** the cached-mount install is measurably faster and the image size is unchanged (the cache is not in the image).

- [ ] **T4 — Break it: the stale** `apt-get update` *(≈40 min, intermediate)*
  - **Goal:** understand why `update` and `install` share a `RUN`.
  - **Steps:**
    1. Build an image with `RUN apt-get update` and a separate `RUN apt-get install -y curl`.
    2. Change only the package list in the second line to add another package, rebuild, and read the output. Then rebuild with `--no-cache`.
    3. Explain in two sentences what could go wrong if the first layer were weeks old.
  - **Done when:** you can show the first layer being `CACHED` and describe the failure mode (`404`/missing package versions from a stale index).

- [ ] **T5 — Shrink the image** *(≈45 min, advanced)*
  - **Goal:** reduce `shop-api` below a size budget.
  - **Steps:**
    1. Record the starting size with `docker image ls`.
    2. Apply, one at a time: `node:22-alpine` base, `npm ci --omit=dev`, `.dockerignore`, removal of build tools. Record the size after each.
  - **Done when:** you have a before/after table and the final image is smaller than half the original (a typical result when starting from the full `node:22` image).

- [ ] **T6 — Prove it: explain cache invalidation** *(≈30 min, advanced)*
  - **Goal:** predict builds.
  - **Steps:**

1. Without running anything, predict which layers rebuild for each of: edit `server.js`; add an npm dependency; change `ENV NODE_ENV`; change base tag; touch `README.md` (with and without `.dockerignore`).
2. Run the builds and check your predictions.

- **Done when:** at least 5 of 5 predictions match, and you can state the "parent layer miss invalidates everything below" rule.

10 Self-check questions

1. What decides whether BuildKit reuses a `RUN` layer? A `COPY` layer?
2. Why does `COPY . .` before `RUN npm ci` slow every build?
3. What does a cache mount do that layer caching alone does not?
4. Why can't you shrink an image by deleting files in a later layer?
5. How do you share the build cache between CI runs on ephemeral runners?

ANSWERS

1. `RUN`: the parent layer and the command text. `COPY`: the parent layer and a checksum of the copied files.
2. Any source change alters the `COPY` checksum, invalidating the layer and everything after it, including the dependency install.
3. It provides a persistent scratch directory (package-manager download cache) that survives across builds without becoming part of the image, so even a re-run `RUN` step avoids re-downloading.
4. Layers are additive; the delete is recorded in a new layer while the earlier layer's data is still shipped.
5. Export and import it: `--cache-to` / `--cache-from` with `type=registry`, `type=gha` or `type=local`.

11 Further reading

- Docker documentation: "Build cache", "Cache backends" and "Optimize cache usage in builds" (docs.docker.com).
- Docker documentation: Dockerfile reference for `RUN --mount` (cache, secret, bind, tmpfs).
- Docker documentation: "GitHub Actions cache" for `docker/build-push-action`.
- `dive` (github.com/wagoodman/dive) — explore image layers and wasted space.

LAB NOTES What I ran, what I saw, what it means

4.10 Multi-stage Builds

Topic 10 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

A **multi-stage build** is a single Dockerfile with several `FROM` instructions. Each `FROM` starts a new, independent **stage** with its own filesystem. You use early stages to *build* things (install compilers, download dev dependencies, compile, run tests) and a final stage to *run* them. The final stage pulls just the finished artifacts out of the earlier stages with `COPY --from=<stage>`; everything else — compilers, source code, package-manager caches, test tools — is left behind and never reaches the image you ship.

The result is a runtime image that contains only what the application needs at runtime: usually smaller, faster to pull, and with a far smaller attack surface, while the whole recipe still lives in one reviewable file.

2 Why it matters in DevOps

- **Size and speed:** a Go service built in `golang:1.24` is several hundred MB with the toolchain; the static binary alone, copied into a minimal image, can be only a few MB.
- **Security:** no compiler, `git`, `curl`, package manager or shell in production means fewer CVEs for scanners to flag and fewer tools for an attacker to use (see Image Scanning and Container Security §4.14).
- **One Dockerfile, several outcomes:** the same file builds a `dev` image with hot-reload tools, a `test` image used in CI, and a lean `prod` image, selected with `--target`.
- **No separate "build script" pipelines:** the older "builder pattern" used two Dockerfiles and a shell script to copy artifacts between them; multi-stage replaces it.

3 Real-world system example

Official images and Go/Rust/Java microservices. The Dockerfiles of widely used Go projects (for example Prometheus and many Kubernetes components) build a static binary in a `golang` stage and place it into a tiny final image (`scratch`, `busybox` or a distroless variant).

The Prometheus Docker image, for instance, is deliberately minimal: a small base with the `prometheus` and `promtool` binaries, and a non-root user. Google's **distroless** project documents multi-stage examples where a language-specific stage compiles and a `gcr.io/distroless/...` stage runs. For interpreted languages (Node.js, Python) teams use the same pattern to install build-time dependencies (compilers for native modules) in one stage and copy only `node_modules` or the virtualenv into the runtime stage.

4 Diagram

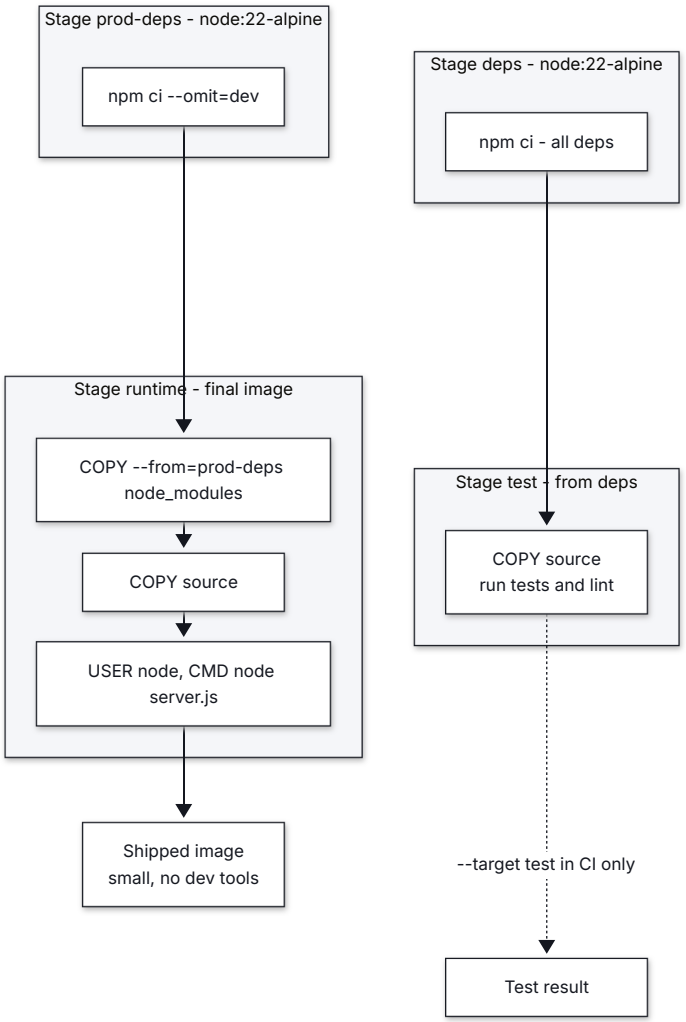


Figure 4.10-1

Size effect (conceptual):

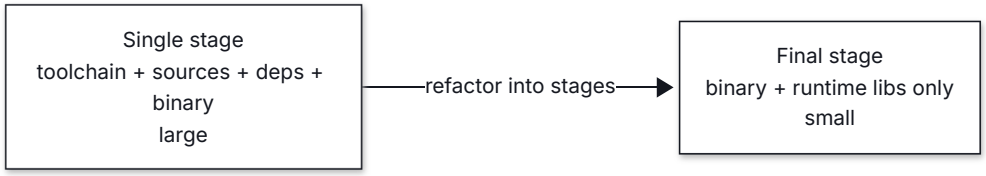


Figure 4.10-2

5 How it works

Syntax

```dockerfile
FROM golang:1.24-alpine AS build          # stage named "build"
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/app .

FROM scratch                              # final stage: the shipped image
COPY --from=build /out/app /app
USER 10001:10001
ENTRYPOINT ["/app"]
```

Key rules

Rule	Detail
Stages are independent filesystems	Only what you `COPY --from` crosses over; `ENV`, `USER`, `WORKDIR` are **not** inherited unless you start a stage `FROM` another stage
The **last stage** is the image you get by default	Everything else is build-time only
Name stages with `AS name`	Refer to them as `--from=name` (or by number `--from=0`, fragile)
`--target name` builds up to that stage	e.g. `docker build --target test .` for CI or `--target dev` for local
`COPY --from=image:tag /path /dest`	Copy from any image, not just a stage (e.g. `--from=nginx:1.28-alpine /etc/nginx/nginx.conf /tmp/`)
BuildKit builds only the stages the target needs	An unreferenced `test` stage is skipped for the default build — it is not a gate unless something depends on it
Stages can share a common ancestor	`FROM base AS dev-deps` and `FROM base AS prod-deps` share the cached `base` layers

Making tests a gate. Run them as a separate CI step with `docker build --target test .` (fails the pipeline if a `RUN` in that stage fails) before building the production target. Alternatively make the runtime stage depend on the test stage (for instance `COPY --from=test /app/.tested /tmp/.tested`), which forces the tests to run in every build at the cost of coupling.

What to copy. Compiled binaries (`/out/app`), built frontend assets (`dist/`), production-only `node_modules`, Python virtualenvs (`/opt/venv`), CA certificates and timezone data if the final image lacks them (`COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/`). Remember that `scratch` has no shell, no `/tmp`, no users file — decide how the app will get CA certificates and a non-root UID.

Cache behaviour. Each stage's layers are cached independently, so a source change does not invalidate `go mod download` or `npm ci` as long as manifests are copied first (caching topic §4.9).

6 Step-by-step guide

Prerequisites: Docker with BuildKit. Go source below was compiled and run locally to check it; image sizes are illustrative (they vary by base-image release).

1. **Create a small Go service.**

```bash
mkdir -p ~/lab/multistage && cd ~/lab/multistage
cat > go.mod <<'EOF'
module example.com/hello

go 1.24
EOF
cat > main.go <<'EOF'
package main

import (
  "fmt"
  "log"
  "net/http"
  "os"
)

func main() {
  addr := ":" + getenv("PORT", "8080")
  http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
          fmt.Fprintln(w, "hello from a multi-stage build")
  })
  log.Printf("listening on %s", addr)
  log.Fatal(http.ListenAndServe(addr, nil))
```

```
}

func getenv(k, d string) string {
  if v := os.Getenv(k); v ≠ "" {
          return v
  }
  return d
}
EOF
```

2. **Build a single-stage image first (the "before").**

```bash
cat > Dockerfile.single <<'EOF'
FROM golang:1.24-alpine
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app .
CMD ["/app"]
EOF
docker build -f Dockerfile.single -t ms-lab:single .
docker image ls ms-lab
```

The image is hundreds of MB (the Go toolchain is in it).

3. **Convert to multi-stage with `scratch`.**

```bash
cat > Dockerfile <<'EOF'
FROM golang:1.24-alpine AS build
WORKDIR /src
COPY go.mod ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/app .

FROM scratch
COPY --from=build /out/app /app
USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["/app"]
EOF
docker build -t ms-lab:multi .
docker image ls ms-lab
```

`ms-lab:multi` is only a few MB (about the size of the binary, roughly 5 MB when built with the flags shown).

4. **Run it and prove there is nothing else inside.**

```bash
docker run -d --name ms -p 8080:8080 ms-lab:multi
curl -s localhost:8080
docker exec ms sh 2>&1 | head -1
docker run --rm ms-lab:single ls /usr/local/go/bin | head -3
```

The service answers `hello from a multi-stage build`. `docker exec ms sh` fails (`executable file not found`) because the image has no shell; the single-stage image still carries the Go toolchain.

5. **Add named stages and `--target` for a Node.js service.** For `shop-api` (create the files from the capstone, or use any Node app):

```dockerfile
FROM node:22-alpine AS base
WORKDIR /app
COPY package.json package-lock.json ./

FROM base AS dev-deps
RUN --mount=type=cache,target=/root/.npm npm ci

FROM base AS prod-deps
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev

FROM dev-deps AS test
COPY server.js ./
RUN node --check server.js          # replace with: npm test

FROM node:22-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production PORT=3000
COPY --from=prod-deps --chown=node:node /app/node_modules ./node_modules
COPY --chown=node:node package.json server.js ./
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

```bash
docker build --target test -t shop-api:test .      # CI gate: fails if the test stage
  fails
docker build -t shop-api:1.0.0 .                   # production image: last stage
docker run --rm shop-api:1.0.0 ls node_modules/.bin 2>&1 | head -3   # no dev tooling
```

6. **Copy from an external image.** Take a config file out of another image without running it:

```dockerfile
FROM nginx:1.28-alpine AS nginxconf
FROM alpine:3.22
COPY --from=nginxconf /etc/nginx/nginx.conf /tmp/nginx.conf.default
```

   This is the pattern for borrowing binaries (`COPY --from=busybox:1.37 /bin/busybox /bin/`, `--from=tini` and similar) into minimal images.

7. **Clean up.**

```bash
docker rm -f ms; docker rmi ms-lab:single ms-lab:multi shop-api:test shop-api:1.0.0
rm -rf ~/lab/multistage
```

7 Real-world scenario

(Illustrative.) **"The vulnerability report has 200 findings for a Go service that uses three libraries."** A scan of the `shop-api` sidecar written in Go showed hundreds of CVEs, almost all in packages the service never executes: `git`, `curl`, `openssl` tools, `perl` and the compiler toolchain from the single-stage `golang` image, plus Debian packages in the base. The team moved to a multi-stage build with a `scratch`/distroless final stage and built the binary statically. The next scan reported near-zero OS findings, the image went from hundreds of MB to a few MB, pulls on new autoscaled nodes became nearly instant, and the on-call team lost a debugging crutch (no shell) — so they added an ephemeral debug workflow (`docker debug` / `kubectl debug` with a toolbox image) instead of shipping `sh` everywhere. **Lesson:** what is not in the image cannot be exploited or flagged. Plan your debugging path before removing the shell.

8 Common mistakes and best practices

Don't

- Copy the entire build stage (`COPY --from=build /src /src`) — you re-import the bloat you were avoiding.
- Forget runtime needs of the final stage: CA certificates, timezone data, a non-root user, a writable `/tmp`.
- Use `scratch` with a dynamically linked binary (see the glibc/musl note in Minimal Base Images §4.13).
- Expect an unreferenced `test` stage to run during a normal `docker build`.

Do

- Name every stage and `COPY --from=name`; put the shipped stage **last**.
- Copy dependency manifests first in each stage for caching; use cache mounts for package managers.
- Run tests in a `--target test` step in CI before building the final image.
- Set `USER` in the final stage (it is not inherited from build stages) and pin base images.

9 Tasks

- [ ] **T1 — Shrink a single-stage image** *(≈30 min, beginner)*
  - **Goal:** see the size difference.
  - **Steps:**
    1. Build the single-stage and multi-stage Go images from the guide.
    2. Compare `docker image ls ms-lab` and `docker history` of both.
  - **Done when:** you have a before/after size table and a one-sentence explanation of what was left behind.
- [ ] **T2 — Multi-stage for `shop-api`** *(≈40 min, intermediate)*
  - **Goal:** separate dev and production dependencies.
  - **Steps:**
    1. Write the Node.js multi-stage Dockerfile from step 5 for your `shop-api`.
    2. Build the default target and compare `docker run --rm IMAGE ls node_modules | wc -l` with a single-stage `npm ci` image that includes dev dependencies.
  - **Done when:** the runtime image has fewer packages and starts normally with `docker run -p 3000:3000`.
- [ ] **T3 — A test gate** *(≈40 min, intermediate)*
  - **Goal:** make a failing test stop the build.
  - **Steps:**
    1. Add a `test` stage with a command that you can make fail (for example `RUN test "$(node -p 1+1)" = 2`).
    2. Run `docker build --target test .` and observe success; change the assertion and observe failure.
  - **Done when:** the second build exits non-zero and prints the failing `RUN` step, and you wrote the CI snippet that calls it before the production build.

- [ ] **T4 — Break it: missing CA certificates** *(≈40 min, intermediate)*
  - **Goal:** discover a classic `scratch` pitfall.
  - **Steps:**
    1. Extend the Go app to call an HTTPS URL (`http.Get("https://example.com")`) and build it into `scratch`.
    2. Run it and read the TLS error (`x509: certificate signed by unknown authority`).
    3. Fix it with `COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/`.
  - **Done when:** the request succeeds after the fix and you can explain why `scratch` has no certificates.

- [ ] **T5 — Targets for dev, test and prod** *(≈45 min, advanced)*
  - **Goal:** one Dockerfile, three behaviours.
  - **Steps:**
    1. Add a `dev` stage that includes dev dependencies and runs `node --watch server.js`.
    2. In Compose, set `build: {context: ./api, target: dev}` for development and `target: runtime` in an override for production.
    3. Run `docker compose config` for both and show the differing `target`.
  - **Done when:** `docker compose up` runs the `dev` stage with hot reload via a bind mount, and the prod override builds the lean image.

- [ ] **T6 — Prove it: review a Dockerfile** *(≈30 min, advanced)*
  - **Goal:** turn a bloated Dockerfile into a multi-stage one without notes.
  - **Steps:**
    1. Take a single-stage Dockerfile that compiles TypeScript (`npm ci`, `npm run build`, `CMD node dist/index.js`).
    2. Rewrite it with a build stage and a runtime stage that contains only `dist/` and production dependencies, running as non-root.
    3. Explain each `COPY --from`.
  - **Done when:** the image runs the compiled app, contains no `src/` or `typescript` package, and you can defend each line.

10 Self-check questions

1. What crosses from one stage to another, and what doesn't?

2. Which stage becomes the image if you don't pass `--target`?
3. Why won't a `test` stage run in a normal build, and how do you make tests a gate?
4. What must you add to a `scratch` image for HTTPS to work?
5. Is `USER` inherited from a build stage to the final stage?

ANSWERS

1. Only files you explicitly `COPY --from`; environment, user, workdir and layers are not inherited.
2. The last stage in the file.
3. BuildKit builds only the stages the requested target depends on. Build it explicitly with `--target test` in CI, or make a later stage depend on it.
4. CA certificates (copy `ca-certificates.crt` from a build stage) — scratch is empty.
5. No. Each stage starts fresh from its own `FROM`; set `USER` in the final stage.

11 Further reading

- Docker documentation: "Multi-stage builds" and Dockerfile reference for `FROM ... AS` and `COPY --from` (docs.docker.com).
- Google `distroless` project README (github.com/GoogleContainerTools/distroless) — multi-stage examples per language.
- Docker documentation: "Build best practices".
- Go documentation: `go build` flags (`-trimpath`, `-ldflags`) and `CGO_ENABLED`.

4.11 Non-root Containers

Topic 11 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

By default a process in a container runs as **UID 0 — `root` inside the container**. Because a container shares the host kernel, root in the container is a powerful identity: if the app is exploited, or if the container is later given extra privileges (a mounted Docker socket, a host path, `--privileged`), the attacker starts from the strongest possible position. A **non-root container** runs the application as an unprivileged numeric user instead, and is further hardened by removing Linux **capabilities** (fine-grained slices of root power), mounting the filesystem **read-only**, and forbidding privilege escalation.

The four building blocks of this topic:

1. `USER` — which UID/GID the process runs as.
2. **File ownership** — making sure that user can read the code and write only where it must.
3. **Dropping capabilities** — `--cap-drop ALL`, then add back only what is needed.
4. **Read-only root filesystem** — `--read-only`, with `tmpfs` or volumes for the few writable paths.

2 Why it matters in DevOps

- **Blast radius.** A remote-code-execution bug in `shop-api` becomes "attacker is `node` with no write access" instead of "attacker is root who can rewrite the app, install tools and probe the host".
- **Platform policy.** Kubernetes' *restricted* Pod Security Standard requires non-root, no privilege escalation, dropped capabilities and a default seccomp profile. Many clusters and compliance regimes (CIS Docker Benchmark, SOC 2 controls) reject root containers.
- **Failure mode: volumes.** Switching to non-root is the most common cause of "permission denied" on mounted volumes; understanding UIDs is a core operational skill (see also Volumes and Bind Mounts §4.5).

- See also Rootless and Minimal Containers §11.12 and Pod Security §16.3 for the same ideas at cluster level.

3 Real-world system example

Official images and Kubernetes Pod Security Standards. Many popular images ship an unprivileged user and an "unprivileged" variant: the `node` image includes a `node` user (UID 1000); the official Nginx project publishes `nginxinc/nginx-unprivileged`, which listens on port 8080 and runs as a non-root user; the `redis` and `postgres` images start as root only long enough to fix ownership of their data directory and then drop to a service user (`redis`, `postgres`). The Prometheus and Grafana images run as unprivileged users (`nobody` and UID 472 respectively), which is why a freshly created bind-mounted data directory owned by root causes "permission denied" until the owner is fixed. On Kubernetes, the **Pod Security Standards** ("baseline" and "restricted" profiles, enforced by the Pod Security Admission controller that replaced PodSecurityPolicy) encode exactly these hardening settings, and managed platforms such as OpenShift run containers with arbitrary non-root UIDs by default.

4 Diagram

Layers of defence around one process:

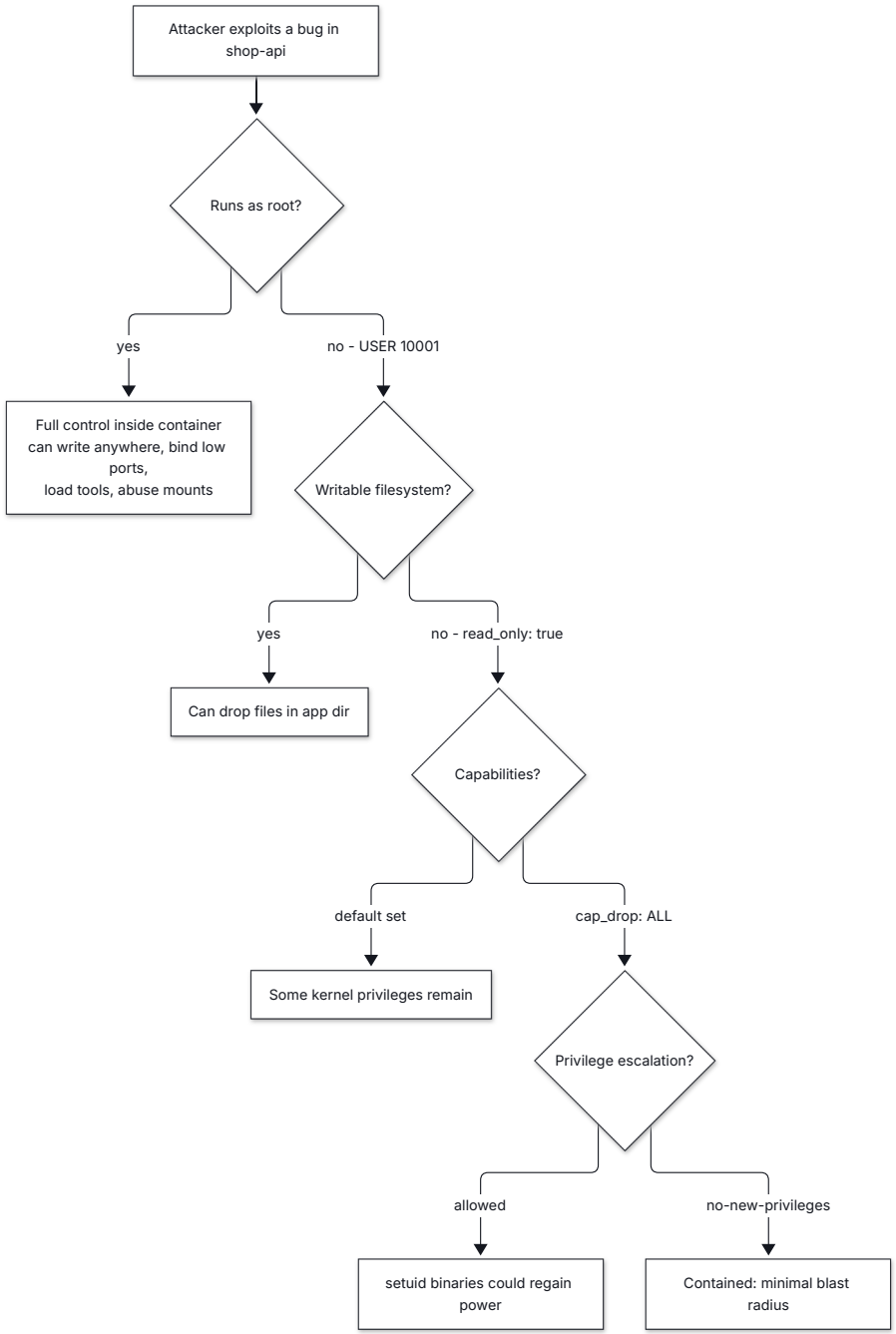

Attacker exploits a bug in shop-api
Runs as root?
yes
no - USER 10001
Full control inside container can write anywhere, bind low ports, load tools, abuse mounts
Writable filesystem?
yes
no - read_only: true
Can drop files in app dir
Capabilities?
default set
cap_drop: ALL
Some kernel privileges remain
Privilege escalation?
allowed
no-new-privileges
setuid binaries could regain power
Contained: minimal blast radius

Figure 4.11-1

How a UID mismatch breaks a volume:

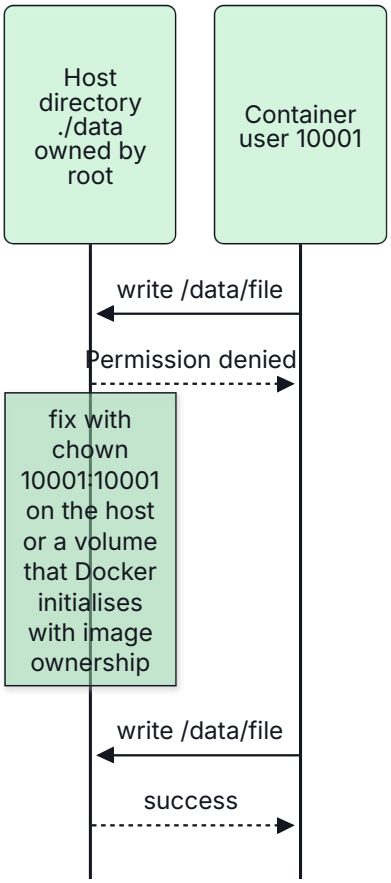


Figure 4.11-2

5 How it works

1. `USER` **.** Create a user with a **fixed numeric UID/GID** and switch to it late in the Dockerfile (after installing packages as root). Prefer numbers (`USER 10001:10001`) over names: Kubernetes' `runAsNonRoot` check can only verify a numeric UID.

```dockerfile
# Debian/Ubuntu based
RUN groupadd --system --gid 10001 app && useradd --system --uid 10001 --gid app --no-
  create-home --shell /usr/sbin/nologin app
# Alpine (busybox) based
RUN addgroup -S -g 10001 app && adduser -S -u 10001 -G app -H -s /sbin/nologin app
# Node images already ship a "node" user with UID 1000: just use USER node
```

2. File ownership. Files `COPY`ed in default to `root:root`. A non-root user can usually *read* them (mode 644/755), which is what you want: the app should **not** be able to modify its own code. Give ownership only to paths that must be written (data, cache, tmp). Use `COPY --chown=10001:10001 ...` (not a separate `RUN chown -R`, which duplicates the files in a new layer). For volumes: Docker initialises an *empty named volume* with the image's ownership at that path, so `RUN mkdir /data && chown 10001:10001 /data` before `USER` makes a named volume writable; **bind mounts** keep the host's ownership, so `chown` the host directory or use `--user "$(id -u):$(id -g)"` in development.

3. Capabilities. Root's powers are split into ~40 capabilities. Docker gives containers a default set of about 14 (including `CHOWN`, `DAC_OVERRIDE`, `SETUID`, `SETGID`, `NET_RAW`, `NET_BIND_SERVICE`, `KILL`) and no more. Best practice is to drop all and add back only what the app proves it needs:

```bash
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE ...    # only if binding ports below 1024 as non-root is required
```

Prefer high ports (`8080`, `3000`) and then no capability is needed. Never use `--privileged` (all capabilities, all devices, no confinement) or `--cap-add SYS_ADMIN` outside deliberate tooling.

4. Read-only root filesystem and friends.

Flag (Compose key)	Effect
`--read-only` (`read_only: true`)	Root filesystem mounted read-only; writes fail with `Read-only file system`
`--tmpfs /tmp` (`tmpfs: [/tmp]`)	In-memory writable scratch directory
`--security-opt no-new-privileges` (`security_opt: ["no-new-privileges:true"]`)	Setuid binaries cannot gain extra privileges
`--pids-limit`, `--memory`, `--cpus`	Limit fork bombs and resource abuse
`--user 10001:10001` (`user:`)	Override the image's user at run time
seccomp / AppArmor	Default profiles already apply; keep them (never `--security-opt seccomp=unconfined`)

Rootless Docker and user namespaces. Everything above hardens *inside* the container. **Rootless mode** runs the Docker daemon itself as an unprivileged host user, and **user-namespace remapping** (`userns-remap`) maps container UID 0 to an unprivileged host UID range, so even "root in the container" is nobody on the host. They complement, not replace, `USER`.

6 Step-by-step guide

Prerequisites: Docker. Everything is local and free. Outputs are illustrative; capability bitmasks are standard for Docker's default profile but may differ on your runtime.

1. **Observe the default: root.**

```bash
docker run --rm alpine id
docker run --rm alpine sh -c 'grep Cap /proc/1/status'
```

`uid=0(root) gid=0(root)` and a non-zero `CapEff` such as `00000000a80425fb` (Docker's default capability set).

2. **Run as another user.**

```bash
docker run --rm --user 10001:10001 alpine id
docker run --rm --user 10001:10001 alpine touch /etc/x    # permission denied
```

The user has no name (`uid=10001 gid=10001`) and cannot modify `/etc`.

3. **Drop all capabilities.**

```bash
docker run --rm --cap-drop ALL alpine sh -c 'grep Cap /proc/1/status; chown 5:5
  /etc/hostname'
```

`CapEff` is all zeros and `chown` fails with `Operation not permitted` even though the user is root: capabilities, not just UID, gate privileged actions.

4. **Make the filesystem read-only.**

```bash
docker run --rm --read-only alpine touch /x                # Read-only file system
docker run --rm --read-only --tmpfs /tmp alpine sh -c 'touch /tmp/ok && echo tmp-ok'
```

5. **Build a non-root `shop-api` image.** (Use your `shop-api`; a minimal Node image already

has the `node` user.)

```dockerfile
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY --chown=root:root server.js ./
ENV NODE_ENV=production PORT=3000
USER 1000:1000
EXPOSE 3000
CMD ["node", "server.js"]
```

```bash
docker build -t shop-api:nonroot .
docker run --rm shop-api:nonroot id           # uid=1000(node) gid=1000(node)
```

Code is owned by root and read-only to the app; the app cannot rewrite itself.

6. **Run it fully hardened** next to throw-away PostgreSQL and Redis containers.

```bash
docker network create hardnet
docker run -d --name pg --network hardnet -e POSTGRES_USER=shop -e
  POSTGRES_PASSWORD=devpass -e POSTGRES_DB=shop postgres:17-alpine
docker run -d --name rd --network hardnet redis:7.4-alpine
docker run -d --name hardened --network hardnet -p 127.0.0.1:3000:3000 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL --security-opt no-new-privileges:true \
  --pids-limit 100 --memory 256m \
  -e DATABASE_URL=postgres://shop:devpass@pg:5432/shop -e REDIS_URL=redis://rd:6379 \
  shop-api:nonroot
sleep 8; docker logs hardened | tail -3
curl -s localhost:3000/readyz
```

`readyz` returns `{"status":"ready"}`. If the app fails to start, `docker logs` names the path it tried to write, which you then provide as a `tmpfs` or volume.

7. **The same in Compose.**

```yaml
services:
  api:
    image: shop-api:nonroot
    user: "1000:1000"
    read_only: true
    tmpfs:
      - /tmp
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    pids_limit: 100
    mem_limit: 256m
```

Validate with `docker compose config`.

8. **See the volume-ownership problem and fix it.**

```bash
mkdir -p /tmp/rootowned && sudo chown root:root /tmp/rootowned
docker run --rm --user 10001:10001 -v /tmp/rootowned:/data alpine sh -c 'echo hi >
  /data/f'      # Permission denied
sudo chown 10001:10001 /tmp/rootowned
docker run --rm --user 10001:10001 -v /tmp/rootowned:/data alpine sh -c 'echo hi >
  /data/f && echo written'
```

(On Docker Desktop for macOS/Windows file ownership is translated and you may not see the first failure.)

9. **Clean up.**

```bash
docker rm -f hardened pg rd; docker network rm hardnet; docker rmi shop-api:nonroot;
  sudo rm -rf /tmp/rootowned
```

7 Real-world scenario

(Illustrative.) **"Security review blocks the production launch."** The audit found three things in `shop-api`: container running as root, `/var/run/docker.sock` mounted into it "for log collection", and the app directory writable. A pen-tester used a deserialisation bug in a dependency to get a shell, then used the Docker socket to start a privileged container and read the host's files. The fix list: `USER 1000:1000`, code owned by root and read-only (`read_only: true`), `cap_drop: [ALL]`, `no-new-privileges`, removal of the Docker socket mount (logs now

go to stdout and are collected by an agent with its own scoped access), and a Kubernetes `restricted` Pod Security label on the namespace so regressions are rejected at admission. The retest could execute code as `node` but could not write anywhere except `/tmp` or reach the host. **Lesson:** each control alone is bypassable; together they turn a critical finding into a low one.

8 Common mistakes and best practices

Don't

- Leave `USER` unset or switch to a *named* user the base image doesn't define.
- `chown -R` huge directories in a separate `RUN` (doubles the layer) or make the app's code writable by the app.
- "Fix" permission errors with `chmod 777` or by running as root again.
- Mount `/var/run/docker.sock`, use `--privileged`, or `--cap-add SYS_ADMIN` in application containers.

Do

- Use a fixed numeric UID/GID (above 10000 is a good habit), set `USER` as late as possible, and verify with `docker exec CONTAINER id`.
- Drop all capabilities, set `no-new-privileges`, and use `read_only: true` with `tmpfs` for scratch.
- Bind to ports above 1024 (the reverse proxy or load balancer publishes 80/443).
- Test the hardened configuration in CI (`docker run --read-only --cap-drop ALL ...` + a smoke test), and enforce it in Kubernetes with Pod Security Admission.

9 Tasks

- [ ] **T1 — Compare root and non-root** *(≈20 min, beginner)*
  - **Goal:** see what changes with `--user`.
  - **Steps:**
    1. Run `id` and `grep Cap /proc/1/status` in `alpine` as default, as `--user 10001:10001`, and with `--cap-drop ALL`.
    2. Try `apk add curl` in each.
  - **Done when:** you have a 3-row table of UID, capability mask and whether `apk add` worked.
- [ ] **T2 — Make `shop-api` non-root** *(≈40 min, intermediate)*

- **Goal:** produce a non-root image.
- **Steps:**
  1. Add `USER 1000:1000` (or a custom UID) and ensure code is root-owned.
  2. Build, run, and verify with `docker exec CONTAINER id` and `docker exec CONTAINER sh -c 'touch /app/x'` (should fail).
- **Done when:** the API works and writing to `/app` is denied.

- [ ] **T3 — Hardened runtime profile** *(≈45 min, intermediate)*
  - **Goal:** find the minimum writable paths.
  - **Steps:**
    1. Run with `--read-only --cap-drop ALL --security-opt no-new-privileges:true`.
    2. If the app fails, read `docker logs` for the path, add only that path as `tmpfs`/volume, and retry.
    3. Capture the final Compose snippet.
  - **Done when:** the container is healthy with a read-only root filesystem and the only writable mounts are the ones you listed.

- [ ] **T4 — Break it: volume permissions** *(≈30 min, intermediate)*
  - **Goal:** reproduce and fix `Permission denied` on a mounted path.
  - **Steps:**
    1. Run step 8 of the guide with a root-owned host directory.
    2. Fix it three ways: `chown` the host dir; `--user "$(id -u):$(id -g)"`; use a named volume initialised from an image directory owned by the app UID.
  - **Done when:** all three approaches write successfully and you can explain which one you would use in production and why.

- [ ] **T5 — Low ports without root** *(≈30 min, advanced)*
  - **Goal:** understand `NET_BIND_SERVICE`.
  - **Steps:**
    1. Run a tiny server on port 80 as a non-root user with `--cap-drop ALL` and note the result (the outcome depends on your Docker version: recent versions allow unprivileged binding to low ports via a sysctl default, older ones fail with `EACCES`).
    2. Add `--cap-add NET_BIND_SERVICE` (or switch to 8080 and publish `-p 80:8080`) and compare.
  - **Done when:** you can state which approach is preferable (high port + publish) and why.

- [ ] **T6 — Prove it: harden and verify** *(≈45 min, advanced)*

- **Goal:** produce evidence of hardening.
- **Steps:**
  1. Without notes, write the Compose service for `shop-api` with every control from this topic.
  2. Write a script that checks them against the running container (`docker inspect` fields `.Config.User`, `.HostConfig.ReadonlyRootfs`, `.HostConfig.CapDrop`, `.HostConfig.SecurityOpt`).
- **Done when:** the script prints PASS for all four settings and FAILs if you remove any one.

10 Self-check questions

1. Why is "root in a container" dangerous even though it is isolated?
2. Why prefer a numeric `USER`?
3. What is a Linux capability, and what does `--cap-drop ALL` do?
4. What breaks when you add `--read-only`, and how do you fix it?
5. Why does a bind-mounted directory give "permission denied" to a non-root container?
6. How do rootless Docker and `USER` differ?

ANSWERS

1. It shares the host kernel; combined with a misconfiguration (docker.sock, privileged, host mounts) or a kernel bug it can lead to host compromise, and it lets an attacker modify files and tools inside the container freely.
2. Orchestrators like Kubernetes can only verify a numeric UID against `runAsNonRoot`; names may not exist or may map differently.
3. A capability is a single privilege that used to be bundled in root (change file owner, bind low ports, send raw packets...). `--cap-drop ALL` removes them all, leaving the process unable to perform privileged operations.
4. Anything that writes to the root filesystem (temp files, caches, PID files). Mount `tmpfs` or volumes at those exact paths.
5. Bind mounts keep the host's ownership and mode; if the container UID doesn't match, access is denied. Fix ownership or run with a matching UID.
6. `USER` limits the process inside the container; rootless Docker runs the whole daemon and containers without host root, so even a container escape lands in an unprivileged host account.

11 Further reading

- Docker documentation: "Run the Docker daemon as a non-root user (Rootless mode)", "Isolate containers with a user namespace", and `docker run` runtime privilege and Linux capabilities reference.
- Kubernetes documentation: "Pod Security Standards" and "Configure a Security Context for a Pod or Container".
- CIS Docker Benchmark (cisecurity.org) — host and container hardening checks.
- `man 7 capabilities` — the full list of capabilities and what each allows.

LAB NOTES What I ran, what I saw, what it means

4.12 Health Checks

Topic 12 of 14 | ~3 hours | Difficulty: Intermediate

1 What it is

A running container is not necessarily a *working* container. The process can be alive but deadlocked, still warming up, or unable to reach its database. A **health check** is a small command that Docker runs inside the container on a schedule; its exit code tells Docker whether the application is healthy. The container then has a health status in addition to its running state: `starting`, `healthy` or `unhealthy`.

You declare it in the Dockerfile with `HEALTHCHECK` or in Compose under `healthcheck:`. Health status is what lets tools answer two different questions:

- **Liveness** — "is this process stuck so badly that it should be restarted?"
- **Readiness** — "is this instance ready to receive traffic or to be depended on right now?"

Docker's own health status is one signal that is used mostly for readiness-style gating (for example Compose's `depends_on: condition: service_healthy`); Kubernetes uses separate *probes* for both concepts.

2 Why it matters in DevOps

- **Startup ordering without guessing.** The API must not start until PostgreSQL accepts connections (Docker Compose §4.8). Health gating replaces `sleep 10` hacks.
- **Zero-downtime deploys.** Load balancers and orchestrators only send traffic to instances that report ready; otherwise users get errors from half-started containers.
- **Faster incident detection.** `unhealthy` in `docker ps` is a clear, queryable signal long before customer complaints.
- **The wrong check is worse than none.** A health check that tests the database in a *liveness* role can restart every API instance at once when the database blips, turning a small incident into an outage.
- See also Liveness and Readiness Probes §8.15 and What to Monitor §10.11.

3 Real-world system example

Kubernetes probes, AWS load balancers and the official database images. Kubernetes runs three kinds of probe per container: **liveness** (restart on failure), **readiness** (remove from Service endpoints on failure) and **startup** (give slow starters time). It ignores a Dockerfile `HEALTHCHECK` entirely; you re-declare the check as a probe. AWS Application Load Balancers use *target group health checks* (an HTTP path, interval and threshold) to decide which targets receive requests, and ECS services can use container health checks to replace tasks. The official `postgres` image documents `pg_isready` and the `redis` image `redis-cli ping` as the canonical checks, and nearly every Compose example for databases uses them. Prometheus and Grafana expose `/-/healthy` and `/-/ready` (Prometheus) and `/api/health` (Grafana) endpoints for exactly this purpose. The pattern is universal: a cheap HTTP endpoint or CLI command that returns success only when the service can do its job.

4 Diagram

Health state machine of a container with a health check:

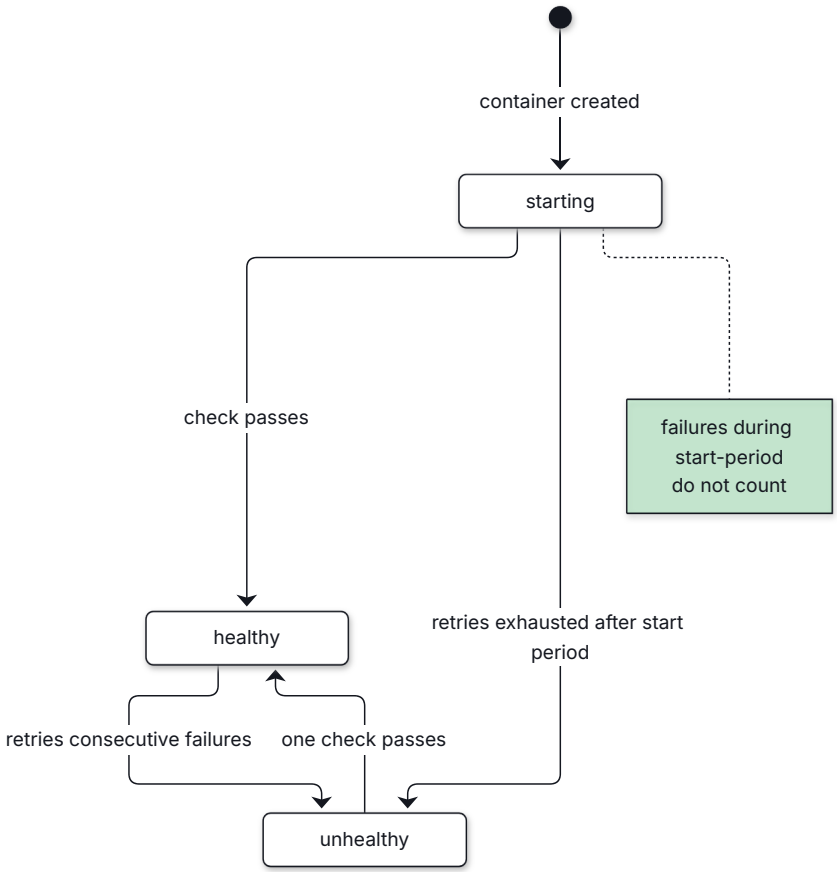


Figure 4.12-1

Health gating in Compose (what the API waits for):

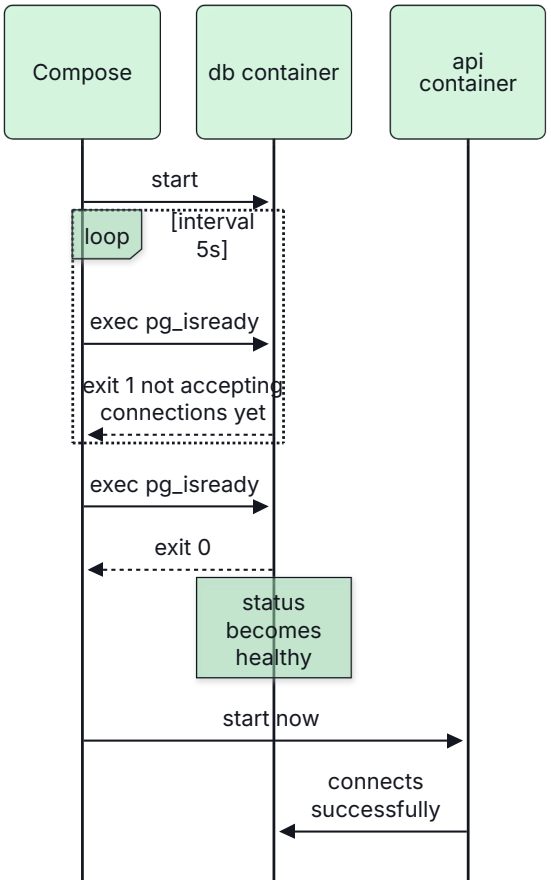


Figure 4.12-2

5 How it works

The instruction

DOCKERFILE

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \
  CMD node -e "fetch('http://127.0.0.1:3000/readyz').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"
```

Option	Default	Meaning
`--interval`	30s	time between checks
`--timeout`	30s	a check taking longer counts as a failure

Option	Default	Meaning
`--start-period`	0s	grace time at boot: failures do not count toward `retries` (a success ends it early)
`--start-interval`	5s	check frequency during the start period (Docker Engine 25 and later)
`--retries`	3	consecutive failures before `unhealthy`

The command's **exit code** decides: `0` = healthy, `1` = unhealthy (code `2` is reserved). `HEALTHCHECK NONE` disables an inherited check. In Compose the same fields appear as `test`, `interval`, `timeout`, `start_period`, `start_interval`, `retries`; `test` is `["CMD", "prog", "arg"]` (exec) or `["CMD-SHELL", "shell command"]`.

What the check runs in. The command runs *inside* the container, so the tool must exist in the image: `curl` is not in `alpine`, `-slim` or distroless images. Options: BusyBox `wget --spider` (Alpine), the language runtime itself (`node -e "fetch(...)"`, `python -c "import urllib.request..."`), a tiny compiled health binary, or a service-native CLI (`pg_isready`, `redis-cli ping`, `mysqladmin ping`). Use `127.0.0.1`, not the service name, inside the check.

Liveness vs readiness — design the endpoints separately

	Liveness (`/healthz`)	Readiness (`/readyz`)
Question	Is the process fundamentally working?	Can it serve requests right now?
Checks	the event loop answers; nothing external	database, cache and other hard dependencies; warm-up complete
On failure	restart the container	stop sending traffic / do not start dependents
Risk if too deep	dependency outage causes mass restarts	too shallow lets traffic hit a broken instance
Must be	very cheap, fast, no dependencies	still fast (use timeouts), cached if expensive

In `shop-api`, `/healthz` returns 200 without touching anything, and `/readyz` pings PostgreSQL and Redis with a 2-second timeout so a dead dependency produces a fast `503`, not a hang. Docker itself does **not** restart an `unhealthy` container on a plain Docker host; it only labels it. Docker Swarm and ECS act on it; on a single host you add `restart: on-failure` plus your own supervisor, or an "autoheal" helper, if you need that behaviour. Kubernetes' liveness probe is the equivalent action.

Tuning. Interval 5-30 s, timeout 2-5 s, retries 3, and a start period that covers real boot time (databases with large recovery need more). Too aggressive checks create load and false alarms; too lax ones react too late.

6 Step-by-step guide

Prerequisites: Docker. Free, local. The `node -e fetch(...)` check command was run against a local copy of `shop-api` to confirm exit codes (0 when ready, 1 when the dependency is down). Docker outputs below are illustrative.

1. **Watch a database become healthy.**

```bash
docker run -d --name pg -e POSTGRES_PASSWORD=devpass \
  --health-cmd 'pg_isready -U postgres' --health-interval 2s --health-timeout 3s --
  health-retries 10 \
  postgres:17-alpine
for i in 1 2 3 4 5 6; do docker inspect -f '{{.State.Health.Status}}' pg; sleep 2;
  done
```

   You see `starting` a few times, then `healthy`. `docker ps` shows `(healthy)` in the STATUS column.

2. **Read the check history.**

```bash
docker inspect --format '{{json .State.Health}}' pg | python3 -m json.tool | head -20
```

   `Log` lists the last five runs with start/end times, `ExitCode` and `Output`.

3. **Make a check fail on purpose.**

```bash
docker run -d --name flaky --health-cmd 'test -f /tmp/ok' --health-interval 2s --
  health-retries 2 alpine sleep 600
sleep 8; docker inspect -f '{{.State.Health.Status}}' flaky        # unhealthy
docker exec flaky touch /tmp/ok; sleep 4
docker inspect -f '{{.State.Health.Status}}' flaky                 # healthy
```

   Recovery needs just one passing run.

4. **Add a HEALTHCHECK to `shop-api`.** Append to the Dockerfile (before `USER` / `CMD` is fine; it applies to the final image):

```dockerfile
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
  CMD node -e "fetch('http://127.0.0.1:3000/readyz').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"
```

Build and run it with the stack from the capstone, then:

```bash
docker compose ps          # api shows (healthy)
docker compose stop cache  # kill a dependency
sleep 40; docker compose ps
```

The API turns `unhealthy` (its `/readyz` returns 503 within ~2 s because Redis is down) while `/healthz` is still 200. Start the cache again and it recovers by itself.

5. **Gate startup in Compose.**

```yaml
services:
  api:
    build: ./api
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set it in .env}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s
```

```bash
docker compose up -d --wait   # returns only when all services with healthchecks are
  healthy
```

6. **Query health in scripts and CI.**

```bash
docker ps --filter health=unhealthy --format '{{.Names}}'
docker inspect -f '{{.State.Health.Status}}' pg | grep -qx healthy && echo READY
```

7. **Clean up.**

```bash
docker rm -f pg flaky; docker compose down
```

7 Real-world scenario

(Illustrative.) **"One database blip restarted the whole fleet."** An orchestrator ran `shop-api` with a single `/health` endpoint that executed `SELECT 1` and was used as the *liveness* check. During a 40-second PostgreSQL failover, every API replica failed its probe at the same moment, was killed and restarted, and came back to a database that was just recovering and got hit by a thundering herd of reconnects. What should have been 40 seconds of degraded service became 8 minutes of outage. The fix split the endpoints: `/healthz` (liveness, no dependencies) and `/readyz` (readiness, checks dependencies with a 2-second timeout), with readiness failures merely removing instances from the load balancer, plus jittered, capped reconnect backoff in the database client. **Lesson:** liveness answers "restart me", readiness answers "don't send me traffic". Mixing them multiplies failures.

8 Common mistakes and best practices

Don't

- Use `curl` in the check when the image doesn't contain it (the check fails forever with exit 127 and the container is `unhealthy`).
- Put dependency checks in liveness, or make checks slow, expensive or stateful (writing to the database every 5 seconds).
- Forget `start_period`, so a slow boot is reported as unhealthy and dependents never start.
- Assume Docker restarts unhealthy containers, or that Kubernetes will read your Dockerfile `HEALTHCHECK`.

Do

- Provide `/healthz` (shallow) and `/readyz` (deep with timeouts), and keep both fast and unauthenticated on an internal port or path.
- Use native checks for datastores (`pg_isready`, `redis-cli ping`) and `condition: service_healthy` for dependents.
- Tune `interval`, `timeout`, `retries`, `start_period` from measured startup times, and alert on `unhealthy` containers.
- Translate checks into Kubernetes liveness, readiness and startup probes when you move to a cluster.

9 Tasks

- [ ] **T1 — Watch the states** *(≈20 min, beginner)*
  - **Goal:** observe `starting` → `healthy` → `unhealthy`.

- **Steps:**
  1. Run steps 1-3 of the guide.
  2. Record the transitions with timestamps from `docker inspect`.
- **Done when:** you captured all three statuses and the `Log` entries behind them.

- [ ] **T2 — Add a real check to `shop-api`** *(≈30 min, intermediate)*
  - **Goal:** a HEALTHCHECK that works in a minimal image.
  - **Steps:**
    1. Add the Node-based `HEALTHCHECK` from step 4 (no `curl` needed).
    2. Run the API with and without Redis reachable.
  - **Done when:** the container is `healthy` with Redis up and `unhealthy` within about a minute with Redis down.

- [ ] **T3 — Gate the startup order** *(≈30 min, intermediate)*
  - **Goal:** replace sleeps with health gating.
  - **Steps:**
    1. In Compose, make `api` depend on `db` and `cache` with `condition: service_healthy`.
    2. Run `docker compose down && docker compose up -d --wait` and read the timestamps in `docker compose logs`.
  - **Done when:** the API's first log line is later than the database's "ready to accept connections" line.

- [ ] **T4 — Break it: wrong check** *(≈30 min, intermediate)*
  - **Goal:** see why checks must match the image.
  - **Steps:**
    1. Use `CMD curl -f http://localhost:3000/healthz || exit 1` in an Alpine- or slim-based image without curl.
    2. Read `docker inspect` output (`exit code 127`, `curl: not found`) and fix it with `wget --spider`, the Node one-liner or by installing curl.
  - **Done when:** the container goes from `unhealthy` to `healthy` and you can explain what exit code 127 meant.

- [ ] **T5 — Auto-heal on a single host** *(≈45 min, advanced)*
  - **Goal:** restart unhealthy containers when the platform won't.
  - **Steps:**

1. Write a small script or sidecar loop that runs `docker ps --filter health=unhealthy -q | xargs -r docker restart`.
2. Break the `shop-api` dependency and confirm the script restarts it (and note the risk: this is a liveness action tied to a readiness signal).

- **Done when:** the container restarts automatically once, and you wrote one sentence on how you would avoid restart storms (backoff, separate liveness check).

☐ **T6 — Prove it: design the endpoints** *(≈30 min, advanced)*

- **Goal:** specify liveness vs readiness for `shop-api`.
- **Steps:**
  1. Without notes, write what `/healthz` and `/readyz` check, their timeouts, their failure behaviour, and the Docker/Compose settings that consume them.
  2. Describe what happens in each of: Redis down, PostgreSQL down, event loop blocked.
- **Done when:** your table matches the design in section 5 and shows that only an event-loop block should trigger a restart.

10 Self-check questions

1. What are the three health statuses and what moves a container between them?
2. What does `start_period` do?
3. Does Docker restart an unhealthy container on a single host? Does Kubernetes use your `HEALTHCHECK`?
4. Why shouldn't a liveness check call the database?
5. How does `depends_on: condition: service_healthy` differ from plain `depends_on`?

ANSWERS

1. `starting`, `healthy`, `unhealthy`. After the start period, `retries` consecutive failures make it unhealthy; a passing check makes it healthy.
2. It gives the app boot time: failing checks during that window don't count toward `retries`, and a success ends the window.
3. No, a plain Docker host only marks it (Swarm/ECS act on it). Kubernetes ignores `HEALTHCHECK` and uses its own probes.
4. A dependency outage would fail every instance's liveness at once and restart them all, amplifying the incident.

5. Plain `depends_on` waits only for the container to start; `service_healthy` waits until the dependency's health check passes.

11 Further reading

- Docker documentation: Dockerfile reference for `HEALTHCHECK`, and the Compose `healthcheck` and `depends_on` references (docs.docker.com).
- Kubernetes documentation: "Configure Liveness, Readiness and Startup Probes".
- PostgreSQL documentation for `pg_isready`; Redis documentation for `PING`.
- *Site Reliability Engineering* (Google) — chapters on monitoring and handling overload.

LAB NOTES What I ran, what I saw, what it means

4.13 Minimal Base Images

Topic 13 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

Every image starts `FROM` a **base image**, and the base decides most of what ends up in production: the C library, the shell, the package manager, hundreds of small utilities — and every security vulnerability (CVE) in them. A **minimal base image** deliberately ships less. The common choices form a ladder from "convenient" to "bare":

Rung	Examples	What you get
Full distro	`node:22`, `python:3.12`, `ubuntu:24.04`	compilers, `git`, `curl`, many libraries; huge
slim	`node:22-slim`, `python:3.12-slim`, `debian:bookworm-slim`	Debian/Ubuntu userland trimmed down; glibc, `apt`, a shell
Alpine	`alpine:3.22`, `node:22-alpine`	BusyBox tools, `apk`, **musl** libc instead of glibc; very small
Distroless	`gcr.io/distroless/static`, `.../base`, `.../nodejs22-debian12`	just the runtime and its libraries: **no shell, no package manager**
scratch	`FROM scratch`	literally empty: only what you `COPY` in (a static binary)

2 Why it matters in DevOps

- **Security:** fewer packages means fewer CVEs for scanners to report and fewer tools (`sh`, `curl`, `apt`) for an attacker who gets code execution. See Image Scanning and Container Security §4.14.
- **Speed and cost:** smaller images pull faster onto freshly scaled nodes and cost less to store and transfer.
- **Reliability trade-offs:** going too minimal moves the pain elsewhere — musl incompatibilities, missing CA certificates or timezone data, nothing to debug with.
- **Policy:** many organisations standardise on approved minimal bases ("golden images") so that patching one base fixes hundreds of services.

- See also Rootless and Minimal Containers §11.12 and Multi-stage Builds §4.10, the technique that makes minimal runtime images practical.

3 Real-world system example

Google's distroless, Alpine, Chainguard and the official images. Google's open-source **distroless** project publishes language-focused images (static, base, cc, Java, Node.js, Python) with no shell or package manager, and is used as the runtime stage of many Go and Java services; Kubernetes' own components are published as small, mostly distroless-style images. **Alpine Linux** is the base of countless Docker Official Image variants (`-alpine` tags) and underpins the `busybox` -style debugging experience many engineers know. Vendors such as Chainguard sell and publish minimal, frequently rebuilt "zero-CVE-goal" images, and Canonical offers "chiselled" Ubuntu images — all responding to the same pressure: scanners and auditors count every package. Docker Official Images for `postgres` , `redis` and `nginx` offer both Debian and Alpine variants because operators disagree about the trade-offs described below.

4 Diagram

Choosing a base:

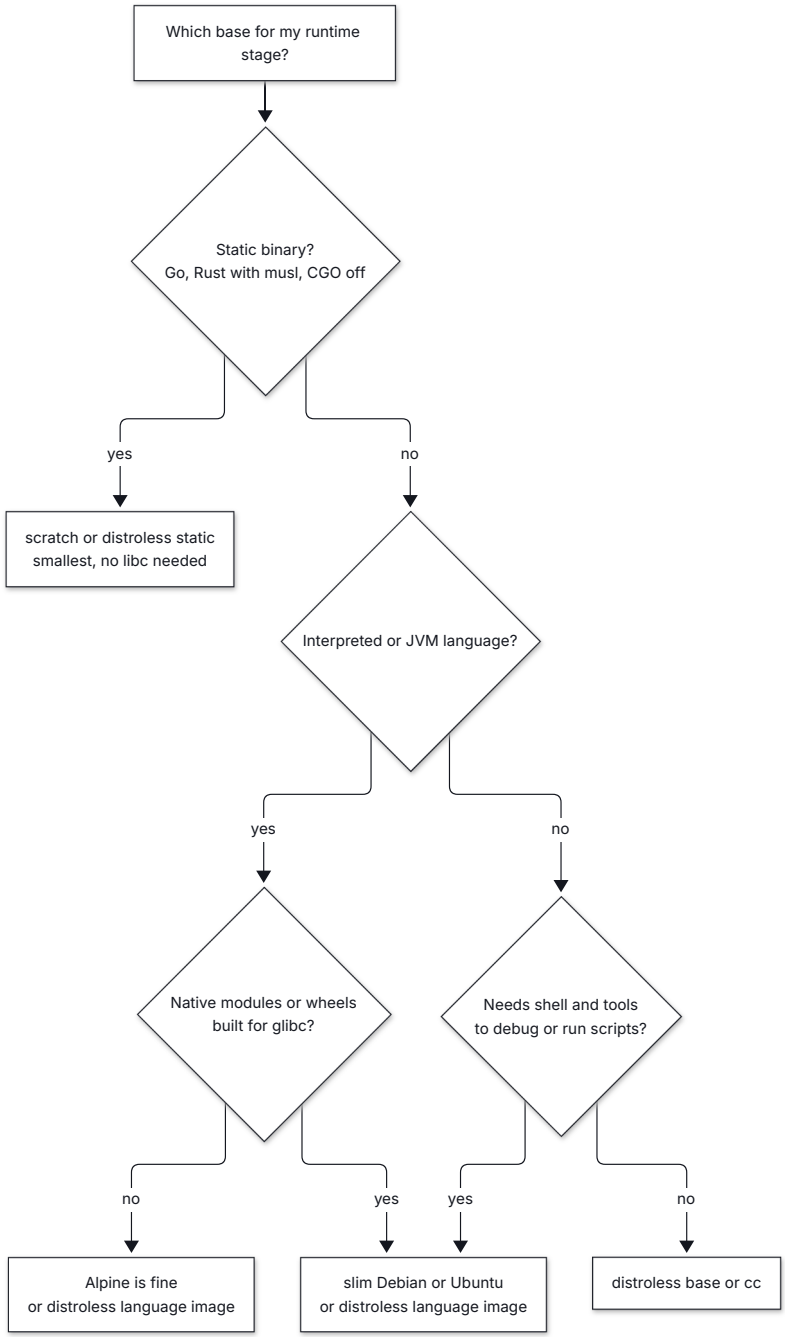

Which base for my runtime stage?
Static binary? Go, Rust with musl, CGO off
yes
no
scratch or distroless static smallest, no libc needed
Interpreted or JVM language?
yes
no
Native modules or wheels built for glibc?
Needs shell and tools to debug or run scripts?
no
yes
yes
no
Alpine is fine or distroless language image
slim Debian or Ubuntu or distroless language image
distroless base or cc

Figure 4.13-1

What differs underneath:

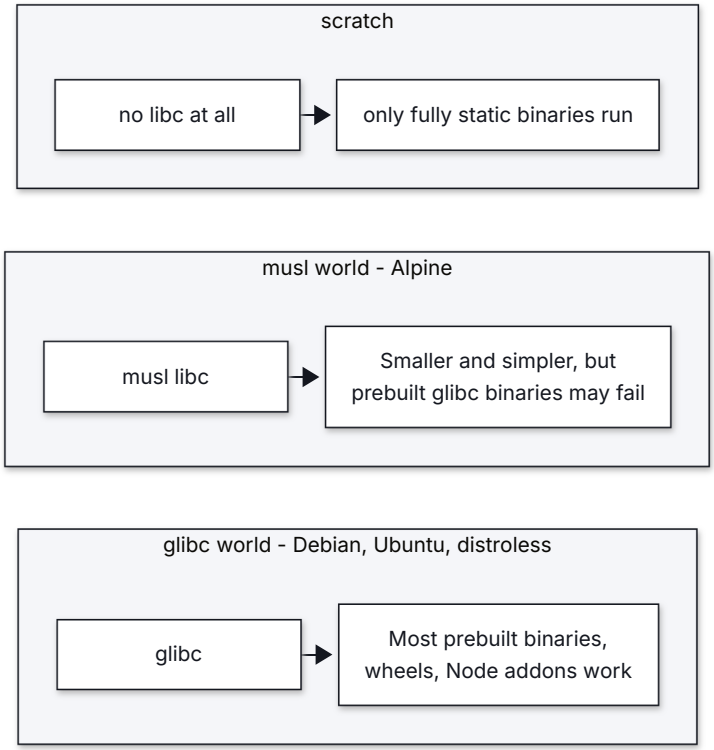


Figure 4.13-2

5 How it works

Approximate sizes (compressed sizes shown on Docker Hub for linux/amd64 when this was written; they change with each release):

Image	Approx. compressed size
`busybox:1.37`	about 2 MB
`alpine:3.22`	about 4 MB
`debian:bookworm-slim`	about 28 MB
`ubuntu:24.04`	about 30 MB
`node:22-alpine`	about 61 MB
`node:22-slim`	about 80 MB
`node:22` (full)	about 410 MB

Image	Approx. compressed size
`python:3.12-alpine` / `python:3.12-slim`	about 21 MB / 43 MB

alpine vs slim vs distroless vs scratch

	Alpine	slim (Debian/Ubuntu)	Distroless	scratch
libc	musl	glibc	glibc (or none for `static`)	none
Shell	`sh` (BusyBox)	`bash` / `dash`	none (a `:debug` tag adds BusyBox)	none
Package manager	`apk`	`apt`	none	none
Debugging	easy (`docker exec -it ... sh`)	easy	hard: use `kubectl debug` / `docker debug` / `:debug` tags	hard
CVE surface	small	medium	very small	minimal (only your binary)
Compatibility	musl gotchas	best	good (matches Debian glibc)	static binaries only
Non-root user	create one	create one	`:nonroot` tags provided (UID 65532)	`USER 10001` works without /etc/passwd entries

glibc vs musl — the real trade-offs. `glibc` is the GNU C library used by mainstream distributions; `musl` is a smaller, stricter alternative used by Alpine. Consequences:

- **Prebuilt binaries** (Python manylinux wheels, Node.js native addons with prebuilt glibc binaries, some vendor agents and database clients) either don't run or must be compiled from source on Alpine, making builds slower and larger.
- **Behavioural differences:** DNS resolution (older musl versions handled `search` domains and TCP fallback differently, which has caused surprises in clusters), locale support, and memory-allocator performance in some workloads.
- **Go** can avoid libc entirely with `CGO_ENABLED=0` (static binary → `scratch` or distroless `static`). **Rust** targets `x86_64-unknown-linux-musl` for static builds. **Java/Node/Python** usually prefer `slim` or distroless-glibc images unless you've tested on Alpine.
- A rule of thumb used by many teams: *"Alpine for tools and static-ish workloads you have tested, Debian slim by default for interpreted languages, distroless/scratch for compiled services."*

What minimal images lack (and what to bring): CA certificates (`ca-certificates`), timezone data (`tzdata`), a non-root user entry, a shell for entrypoint scripts, and `/tmp`. Copy them from a build stage: `COPY --from=build /etc/ssl/certs/ca-certificates.crt`

`/etc/ssl/certs/` and `COPY --from=build /usr/share/zoneinfo /usr/share/zoneinfo`.

Pin and refresh. Pin the base by digest, rebuild on a schedule to pick up security patches, and let a bot (Renovate, Dependabot) propose base updates — a minimal image that is a year stale is still vulnerable.

6 Step-by-step guide

Prerequisites: Docker, internet access for pulls (a few hundred MB in total). Outputs and sizes are illustrative; use `docker image ls` on your machine for real values.

1. **Compare the base sizes.**

```bash
docker pull alpine:3.22 && docker pull debian:bookworm-slim && docker pull node:22-alpine && docker pull node:22-slim
docker image ls --format 'table {{.Repository}}:{{.Tag}}\t{{.Size}}' | grep -E 'alpine|slim'
```

   Disk sizes (uncompressed) are larger than the Hub's compressed numbers but keep the same ordering.

2. **See what is inside each.**

```bash
docker run --rm alpine:3.22 sh -c 'cat /etc/os-release | head -2; ls /bin | wc -l; ldd --version 2>&1 | head -1'
docker run --rm debian:bookworm-slim sh -c 'head -2 /etc/os-release; ls /bin | wc -l; ldd --version | head -1'
```

   Alpine reports musl; Debian reports `GNU libc` (glibc). Alpine's tools are BusyBox symlinks (`ls -l /bin/ls` shows `busybox`).

3. **Build the same Go app three ways** (use the Go program from Multi-stage Builds §4.10).

```dockerfile
# Dockerfile.scratch
FROM golang:1.24-alpine AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/app .
FROM scratch
COPY --from=build /out/app /app
USER 10001:10001
ENTRYPOINT ["/app"]
```

Create `Dockerfile.distroless` by changing the second stage to `FROM gcr.io/distroless/static-debian12:nonroot` (and drop `USER`), and `Dockerfile.alpine` with `FROM alpine:3.22` plus `RUN adduser -S -u 10001 app` and `USER app`.

```bash
for v in scratch distroless alpine; do docker build -f Dockerfile.$v -t mini:$v .;
  done
docker image ls mini
```

Expect roughly: scratch about 5 MB (just the binary), distroless a little larger (adds CA certs, tzdata, passwd entries), alpine about 9-10 MB.

4. **Try to debug each one.**

```bash
docker run -d --name m-alpine -p 8080:8080 mini:alpine
docker exec -it m-alpine sh -c 'id && wget -qO- localhost:8080'      # works
docker run -d --name m-scratch -p 8081:8080 mini:scratch
docker exec -it m-scratch sh                                          # fails: no
  shell
```

For the shell-less image, debug from outside: `docker logs`, `docker cp`, `docker run --rm --pid=container:m-scratch --net=container:m-scratch nicolaka/netshoot ps aux` (a toolbox container joining the target's namespaces), or the `docker debug` command if your Docker edition has it.

5. **Hit the musl/glibc wall.** Run a glibc-linked binary on Alpine:

```bash
docker run --rm -v "$PWD":/w debian:bookworm-slim sh -c 'cp /bin/ls /w/ls-glibc'
docker run --rm -v "$PWD":/w alpine:3.22 /w/ls-glibc /
```

The second command fails with `exec /w/ls-glibc: no such file or directory` — the "missing file" is the glibc dynamic loader the binary asks for. This is what happens with glibc-only wheels and addons.

6. **Python wheels on Alpine vs slim.**

```bash
docker run --rm python:3.12-slim sh -c 'time pip install --no-cache-dir numpy 2>&1 |
  tail -1'
docker run --rm python:3.12-alpine sh -c 'time pip install --no-cache-dir numpy 2>&1 |
  tail -3'
```

On `slim` pip downloads a prebuilt wheel in seconds. Recent versions of pip and numpy publish musl wheels for common platforms, so the Alpine run may also be quick today; with packages that lack musl wheels you will see a long compile instead. Check the output rather than assuming.

7. **Scan each image** (preview of the next topic):

```bash
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.58.0
  image --quiet --severity HIGH,CRITICAL mini:alpine
```

   Compare counts across `mini:scratch`, `mini:distroless` and `mini:alpine`; they should be very low (the Go standard library version still matters).

8. **Clean up.**

```bash
docker rm -f m-alpine m-scratch; docker rmi mini:scratch mini:distroless mini:alpine;
  rm -f ls-glibc
```

7 Real-world scenario

(Illustrative.) **"We moved to Alpine and builds went from 2 minutes to 25."** A team shrank their Python data-export worker by switching `python:3.12` to `python:3.12-alpine`. The image got smaller, but the build now compiled `cryptography` and `psycopg2` from source at every build (needing `gcc`, `musl-dev`, `libffi-dev`, `openssl-dev`), so CI ballooned and the compiler packages bloated the image again until they added a multi-stage build. A month later a DNS-related timeout appeared only in the Alpine image under high load. They reverted to `python:3.12-slim` for the interpreted services (keeping Alpine for small utility containers), and used distroless/scratch for their Go services, with a shared "debug sidecar" workflow to replace `docker exec sh`. **Lesson:** the smallest image isn't the best image — compatibility, build time and debuggability are costs too. Choose per workload and measure.

8 Common mistakes and best practices

Don't

- Pick Alpine reflexively for Python/Node/Java without testing native dependencies and DNS behaviour.
- Use `FROM scratch` with a dynamically linked binary, or forget CA certificates, timezone data and a non-root user.
- Ship `latest` bases or leave a "minimal" image unpatched for months.

- Strip the shell from production with no debugging plan.

Do

- Build in a full image and run in a minimal one (multi-stage §4.10).
- Prefer glibc `-slim` or distroless language images by default; use `scratch` / `static` for static binaries.
- Pin by tag plus digest, rebuild regularly, and scan every build.
- Standardise approved base images across teams, and document the debug workflow (ephemeral debug containers, `:debug` tags).

9 Tasks

- [ ] **T1 — Size table** *(≈25 min, beginner)*
  - **Goal:** quantify the options.
  - **Steps:**
    1. Pull `alpine`, `debian:bookworm-slim`, `ubuntu:24.04`, `node:22`, `node:22-slim`, `node:22-alpine`.
    2. Record `docker image ls` sizes and the layer count (`docker history`).
  - **Done when:** you have a sorted table and can say how many times bigger the full `node:22` is than `node:22-alpine`.
- [ ] **T2 — Same app, three bases** *(≈45 min, intermediate)*
  - **Goal:** compare size and debuggability.
  - **Steps:**
    1. Build `shop-api` on `node:22-slim`, `node:22-alpine` and `gcr.io/distroless/nodejs22-debian12:nonroot` (check the distroless README for current tags; use `CMD ["server.js"]` since its entrypoint is node).
    2. Run each and test `/healthz`; try `docker exec ... sh` on each.
  - **Done when:** all three serve traffic, and you recorded size, shell availability and the user ID for each.
- [ ] **T3 — Break it: glibc on musl** *(≈30 min, intermediate)*
  - **Goal:** reproduce "no such file or directory" for an existing binary.
  - **Steps:**
    1. Run step 5.
    2. Diagnose with `ldd` or `readelf -l ls-glibc | grep interpreter` on the host and explain the missing loader.

3. Fix by running it in `debian:bookworm-slim`, or by installing `gcompat` on Alpine (and note it is a workaround).

- **Done when:** the binary runs and you can explain the interpreter path mismatch.

- [ ] **T4 — Shell-less debugging** *(≈40 min, advanced)*
  - **Goal:** troubleshoot a container with no shell.
  - **Steps:**
    1. Run the `scratch` image and the distroless image.
    2. Use `docker logs`, `docker inspect`, `docker cp` and a `nicolaka/netshoot` container joined with `--pid=container:` and `--network=container:` to list processes and `curl localhost:8080`.
  - **Done when:** you retrieved the app's HTTP response and process list without a shell inside the target.

- [ ] **T5 — Scan and compare** *(≈40 min, advanced)*
  - **Goal:** connect base choice to vulnerability counts.
  - **Steps:**
    1. Scan `node:22`, `node:22-slim`, `node:22-alpine` and your distroless build with Trivy or Grype.
    2. Make a table of HIGH+CRITICAL counts, and note which packages account for most findings.
  - **Done when:** you have a table and one paragraph explaining what drives the differences.

- [ ] **T6 — Prove it: write the base-image policy** *(≈30 min, advanced)*
  - **Goal:** make a decision others can follow.
  - **Steps:**
    1. Without notes, write a half-page policy: default base per language, when to use `scratch`/distroless, required additions (certs, tz, non-root), pinning and refresh rules, debug workflow.
    2. Justify each rule with a trade-off from this topic.
  - **Done when:** a teammate can pick the right base for a new Go service, a Python service and a Node service using only your policy.

10 Self-check questions

1. What is the difference between glibc and musl, and why does it matter for containers?

2. When is `FROM scratch` a good choice, and what must you add to it?
3. What does distroless remove, and how do you debug such a container?
4. Why isn't "smallest image" always the right goal?
5. How do you keep a minimal base from becoming a stale, vulnerable one?

ANSWERS

1. They are different C standard libraries. Alpine uses musl; most distributions use glibc. Binaries, wheels and addons built for glibc may not run on musl, and some runtime behaviours differ.
2. For fully static binaries (Go with `CGO_ENABLED=0`, static Rust). Add CA certificates, timezone data if needed, and a non-root `USER`.
3. The shell, package manager and most utilities. Debug with ephemeral containers (`kubectl debug`, `docker debug`), debug-variant tags, or a toolbox container sharing namespaces.
4. Compatibility, build time, debuggability and operational cost can outweigh a few megabytes.
5. Pin plus rebuild on a schedule, automate base-image update PRs, and scan every build.

11 Further reading

- Google `distroless` project README (github.com/GoogleContainerTools/distroless) — image list, `:nonroot` and `:debug` tags.
- Alpine Linux documentation and the musl libc "Functional differences from glibc" page (wiki.musl-libc.org).
- Docker documentation: "Build best practices — choose the right base image" (docs.docker.com).
- Docker Official Images documentation on Docker Hub (variants: `-slim`, `-alpine`).

4.14 Image Scanning and Container Security

Topic 14 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

Container security has two halves. **Before runtime** you ask "what is inside this image, and is any of it known to be vulnerable or tampered with?" **At runtime** you ask "how do I limit what this container can do if it is compromised?"

- An **image scanner** (Trivy, Grype, Docker Scout, Snyk and others) unpacks an image, inventories the operating-system packages and language dependencies, and matches them against vulnerability databases. Each match is a **CVE** (Common Vulnerabilities and Exposures: a public identifier such as `CVE-2021-44228`) with a **severity** (often from CVSS, the Common Vulnerability Scoring System, rated Low/Medium/High/Critical).
- An **SBOM** (Software Bill of Materials) is the inventory itself: a machine-readable list of every component in the image (formats: SPDX and CycloneDX). Generate it once at build time and you can answer "are we affected?" later without re-scanning.
- **Signing** (for example with Sigstore **cosign**) attaches a cryptographic signature to an image digest so deployers can verify *who built it* and that it was not modified. **Provenance attestations** record how it was built.
- **Runtime hardening** applies the least-privilege controls from Non-root Containers §4.11 plus seccomp/AppArmor, resource limits, and not exposing the Docker socket.

2 Why it matters in DevOps

- **Supply-chain attacks are routine.** Public base images, npm/PyPI packages and CI actions are all attack paths. A scanner in CI is the cheapest control you can add.
- **"Are we affected?" must take minutes, not days.** When a headline vulnerability drops, SBOMs let you query every deployed image instead of grepping repositories.
- **Compliance.** SOC 2, PCI-DSS and customer questionnaires increasingly ask for image scanning, patch SLAs and artifact signing.

- **Noise management is a skill.** An unfiltered scan of a large image reports hundreds of findings, most unreachable; teams need policies (severity thresholds, unfixed-only, suppression with expiry) so the signal is acted on.
- See also Container Image Scanning (Trivy) §11.9, DevSecOps §0.12 and Minimal Base Images §4.13.

3 Real-world system example

Log4Shell and the SBOM argument. In December 2021 a critical flaw in the Java logging library Log4j (CVE-2021-44228, "Log4Shell") was disclosed. Organisations scrambled to learn which of their applications and container images bundled a vulnerable log4j-core JAR, often buried inside fat JARs. The incident is widely cited as the moment SBOMs went mainstream; it contributed to US government guidance on software supply-chain security and to industry adoption of SPDX and CycloneDX. On the tooling side, **Trivy** (open source, by Aqua Security) is widely used in CI and is the default scanner integrated into the **Harbor** registry; **Grype** and **Syft** (Anchore) are a popular scanner/SBOM pair; **cosign** and the Sigstore project provide keyless signing using OIDC identities (for example a GitHub Actions workflow identity) and a public transparency log; Kubernetes admission policies (Kyverno, Sigstore policy-controller, OPA Gatekeeper) can refuse unsigned or vulnerable images. Docker Engine itself ships with a default **seccomp** profile that blocks dozens of risky system calls.

4 Diagram

A secure delivery pipeline for one image:

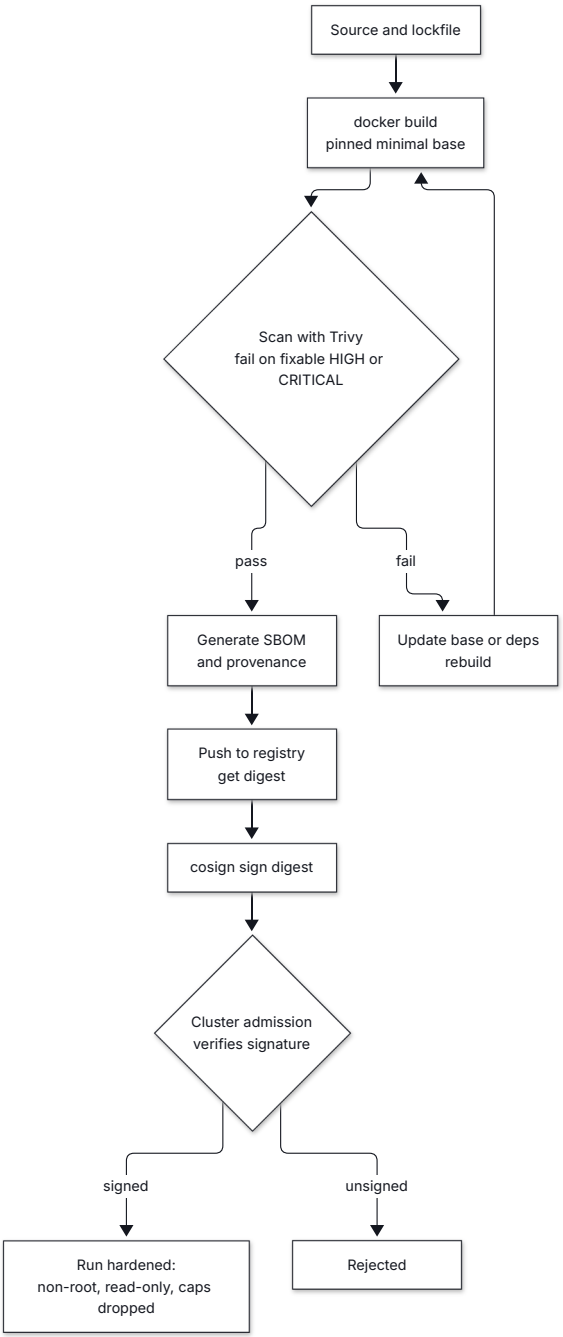

Source and lockfile
docker build
pinned minimal base
Scan with Trivy
fail on fixable HIGH or
CRITICAL
pass
fail
Generate SBOM
and provenance
Update base or deps
rebuild
Push to registry
get digest
cosign sign digest
Cluster admission
verifies signature
signed
unsigned
Run hardened:
non-root, read-only, caps
dropped
Rejected

Figure 4.14-1

Layers of runtime isolation around a container:

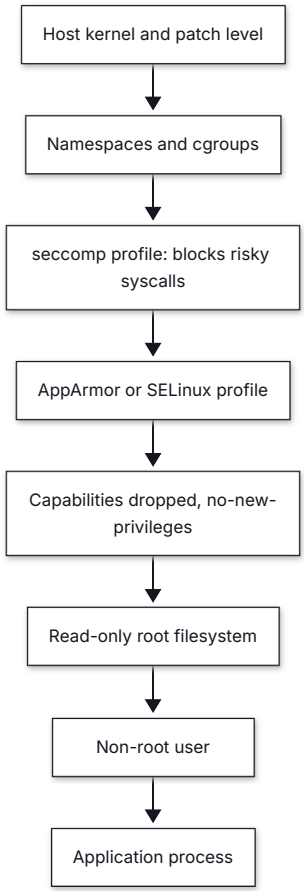


Figure 4.14-2

5 How it works

Scanning. A scanner reads the image layers, identifies the OS (from `/etc/os-release`) and package databases (`dpkg`, `apk`, `rpm`) and language lockfiles/manifests (`package-lock.json`, `go.mod`, `requirements.txt`, JAR metadata), then matches versions against advisories.

Tool	Notes
Trivy	One tool for images, filesystems, Git repos, IaC/Dockerfile misconfiguration (`trivy config`), secrets and SBOMs. `trivy image IMAGE`, `trivy fs .`, `trivy config .`
Grype + **Syft**	Syft generates the SBOM; Grype scans an image or an SBOM (`grype sbom:sbom.json`)

Tool	Notes
Docker Scout	Integrated in Docker Desktop/CLI and Docker Hub
Registry scanners	ECR basic/enhanced scanning, GHCR + Dependabot alerts, Harbor + Trivy

Reading results. A finding has a package, installed version, **fixed version** (may be empty = "no fix yet"), severity and CVE ID. Useful triage questions: Is a fix available? Is the vulnerable code path reachable from our app? Is it a build-time-only package that is not in the runtime image? Is it known-exploited? (CISA publishes a Known Exploited Vulnerabilities catalog; the EPSS score estimates exploitation likelihood.) Common policy: **fail the pipeline on fixable HIGH/CRITICAL** (`--ignore-unfixed --severity HIGH,CRITICAL --exit-code 1`), report the rest, and track exceptions with an owner and an expiry date in an ignore file (`.trivyignore`). **False positives and VEX:** a *VEX* (Vulnerability Exploitability eXchange) statement records "this CVE exists in the package but does not affect us".

Rescan continuously. New CVEs are published for images you shipped months ago. Schedule a daily scan of what is *deployed* (registry scanning or a cron job), not just what you build.

SBOMs and attestations. `trivy image --format cyclonedx --output sbom.cdx.json IMAGE` or `syft IMAGE -o spdx-json=sbom.spdx.json` produce an SBOM. BuildKit can attach SBOM and provenance attestations directly to the pushed image: `docker buildx build --sbom=true --provenance=mode=max --push -t REG/shop-api:1.0.0 .`.

Signing and verification (basics).

1. Build and push; take the immutable **digest** (`REG/shop-api@sha256:...`).
2. `cosign sign REG/shop-api@sha256:...` — keyless in CI (the OIDC token of the workflow becomes a short-lived certificate recorded in a transparency log) or with a managed key (`--key`).
3. Consumers verify: `cosign verify REG/shop-api@sha256:... --certificate-identity <workflow identity> --certificate-oidc-issuer https://token.actions.githubusercontent.com`.
4. A cluster admission policy enforces "only run images signed by our pipeline". Sign **digests**, never mutable tags. (Docker Content Trust, based on Notary v1, is the older mechanism; modern setups use Sigstore cosign or Notation.)

Runtime hardening checklist

Control	How
Non-root, caps dropped, read-only rootfs, `no-new-privileges`	see Non-root Containers §4.11
Keep the default **seccomp** profile and AppArmor/SELinux	never `--security-opt seccomp=unconfined`
Never `--privileged`; don't mount `/var/run/docker.sock` or host paths	the socket is root on the host
Resource limits	`--memory`, `--cpus`, `--pids-limit`
No secrets in images or env files in git	inject at runtime from a secret manager
Isolate networks; publish only the edge	Docker Networks §4.6
Minimal, patched, pinned base images	Minimal Base Images §4.13
Patch the host kernel and Docker Engine; consider rootless mode	containers share the kernel
Runtime detection (Falco and similar) and logs to a central store	detect shells spawned in containers, unexpected outbound traffic

6 Step-by-step guide

Prerequisites: Docker and internet access. Scanner tools run as containers, so nothing needs installing. Vulnerability counts and exact output are illustrative: they change daily as databases update. Keep image tags below pinned to versions that exist (check the tool's release page if a tag has moved on).

1. **Scan a deliberately old image.**

```bash
docker run --rm -v trivy-cache:/root/.cache aquasec/trivy:0.58.0 image --severity
  HIGH,CRITICAL python:3.9-slim-bullseye
```

   You get a table: library/package, CVE ID, severity, installed version, fixed version and a title. The first run downloads the vulnerability database into the `trivy-cache` volume.

2. **Scan your own image and filter the noise.**

```bash
docker build -t shop-api:scan ./api
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-
  cache:/root/.cache \
  aquasec/trivy:0.58.0 image --severity HIGH,CRITICAL --ignore-unfixed shop-api:scan
```

(Mounting the Docker socket into a short-lived scanner is acceptable on a lab machine; in CI scan from a registry or a tar archive: `docker save shop-api:scan -o img.tar` and `trivy image --input img.tar`.)

3. **Make it a gate.** Exit non-zero on fixable high/critical findings:

```bash
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-
  cache:/root/.cache \
  aquasec/trivy:0.58.0 image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1
  shop-api:scan
echo "exit code: $?"
```

`0` means the gate passes; `1` fails the pipeline.

4. **Compare a bloated and a minimal base.** Scan `node:22` versus `node:22-alpine`, or your multi-stage distroless build, and compare counts:

```bash
for img in node:22 node:22-slim node:22-alpine; do
  echo "== $img"; docker run --rm -v trivy-cache:/root/.cache aquasec/trivy:0.58.0
  image --quiet --severity HIGH,CRITICAL --format json "$img" \
    | python3 -c 'import sys,json; d=json.load(sys.stdin);
  print(sum(len(r.get("Vulnerabilities") or []) for r in d.get("Results",[])))'
done
```

The full image usually reports many more findings than `slim` or `alpine`.

5. **Lint the Dockerfile and Compose file for misconfiguration.**

```bash
docker run --rm -v "$PWD":/work -w /work aquasec/trivy:0.58.0 config .
```

Expect findings such as "image user should not be root" and "no HEALTHCHECK defined" if you skipped them.

6. **Generate an SBOM and query it.**

```bash
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v "$PWD":/out
  aquasec/trivy:0.58.0 \
  image --format cyclonedx --output /out/sbom.cdx.json shop-api:scan
python3 -c 'import json; d=json.load(open("sbom.cdx.json"));
  print(len(d["components"]), "components"); print([c["name"]+" "+c.get("version","")
  for c in d["components"] if c["name"]=="express"])'
```

You get a component count and the `express` version: the "are we affected by X?" answer in one query. Scan the SBOM later without the image: `docker run --rm -v "$PWD":/w aquasec/trivy:0.58.0 sbom /w/sbom.cdx.json`.

7. **Sign and verify with cosign** (using a local key pair on a local registry, for learning).

```bash
docker run -d -p 5000:5000 --name registry registry:2
docker tag shop-api:scan localhost:5000/shop-api:1.0.0 && docker push
  localhost:5000/shop-api:1.0.0
DIGEST=$(docker inspect --format '{{index .RepoDigests 0}}' localhost:5000/shop-
  api:1.0.0)
docker run --rm -v "$PWD":/k -w /k --network host -e COSIGN_PASSWORD=lab-only
  gcr.io/projectsigstore/cosign:latest generate-key-pair
docker run --rm -v "$PWD":/k -w /k --network host -e COSIGN_PASSWORD=lab-only
  gcr.io/projectsigstore/cosign:latest sign --yes --key cosign.key --allow-insecure-
  registry "$DIGEST"
docker run --rm -v "$PWD":/k -w /k --network host gcr.io/projectsigstore/cosign:latest
  verify --key cosign.pub --allow-insecure-registry "$DIGEST"
```

`verify` prints the signature payload JSON (with the digest) and "The following checks were performed"; modify the image, push a different digest and verify again to see it fail. In real pipelines prefer keyless signing from CI and verify identities. Flags and image tags can change between cosign releases, so consult `cosign --help` if a flag is rejected.

8. **Run a hardened container and inspect the evidence.**

```bash
docker run -d --name secure --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-
  new-privileges:true \
  --pids-limit 100 --memory 256m --user 1000:1000 -p 127.0.0.1:3000:3000 \
  -e DATABASE_URL=postgres://invalid -e REDIS_URL=redis://invalid shop-api:scan sleep
  300
docker inspect secure --format 'user={{.Config.User}} ro=
  {{.HostConfig.ReadonlyRootfs}} capdrop={{.HostConfig.CapDrop}} secopt=
  {{.HostConfig.SecurityOpt}} pids={{.HostConfig.PidsLimit}}'
```

(The `sleep 300` overrides the app because only the settings are being inspected.) The output lists every control.

9. **Clean up.**

```bash
docker rm -f secure registry; docker rmi shop-api:scan localhost:5000/shop-api:1.0.0;
  docker volume rm trivy-cache; rm -f sbom.cdx.json cosign.key cosign.pub img.tar
```

7 Real-world scenario

(Illustrative.) **"Is any production image running the vulnerable library?"** On a Friday afternoon a critical advisory is published for a popular npm package. Management asks which services are affected. Because the `shop-api` pipeline had been storing a CycloneDX SBOM alongside each image digest, a script queried 40 SBOMs in a few minutes and found the package in two images (one `shop-api` replica set and a worker), with exact versions. Both were patched and redeployed by the evening. Without SBOMs, the previous incident had taken two days of grepping repositories and unpacking images. Afterwards the team added a daily scan of deployed digests, a policy of "fixable critical findings block the merge", and signature verification at admission so that only pipeline-built images can run. **Lesson:** inventory and automation convert emergencies into routine work.

8 Common mistakes and best practices

Don't

- Scan once at build time and never again, or treat a clean scan as proof of safety (scanners only know *published* CVEs).
- Drown the team in findings: unfiltered output teaches people to ignore the scanner.
- Suppress CVEs forever without an owner and an expiry date.
- Sign mutable tags, trust unsigned images from any registry, or run `--privileged` and mount `docker.sock` "just for CI".

Do

- Scan in CI (gate on fixable HIGH/CRITICAL) **and** rescan deployed images on a schedule.
- Prefer minimal, pinned, regularly rebuilt bases; remove build tools from runtime images (multi-stage §4.10).
- Produce SBOMs and provenance for every release; sign digests; verify at admission.
- Combine scanning with runtime hardening (non-root, read-only, no caps, seccomp, limits) and keep host kernels and Docker Engine patched.

9 Tasks

- [ ] **T1 — First scan** *(≈25 min, beginner)*
  - **Goal:** read a vulnerability report.
  - **Steps:**
    1. Run the Trivy scan from step 1 on an old image.

2. Pick three findings and for each note the package, installed vs fixed version, and severity.

- **Done when:** you can explain what a CVE ID, "fixed version" and severity mean for each of your three findings.

- [ ] **T2 — Gate your build** *(≈40 min, intermediate)*
  - **Goal:** make CI fail on fixable high/critical issues.
  - **Steps:**
    1. Run step 3 against `shop-api`; if it passes, build a copy `FROM node:18` (end-of-life) to force a failure.
    2. Turn the command into a pipeline step (a GitHub Actions job or a Makefile target) that returns Trivy's exit code.
  - **Done when:** the old base fails the gate with exit code 1 and the current base passes.

- [ ] **T3 — Reduce findings by changing the base** *(≈45 min, intermediate)*
  - **Goal:** quantify the effect of minimal images.
  - **Steps:**
    1. Scan `shop-api` built on `node:22`, `node:22-slim`, `node:22-alpine` and a distroless runtime.
    2. Record HIGH+CRITICAL counts and the top three packages causing them.
  - **Done when:** you have a table of counts per base and a note on which packages disappear in the minimal builds.

- [ ] **T4 — SBOM-driven triage** *(≈40 min, intermediate)*
  - **Goal:** answer "are we affected?" without the image.
  - **Steps:**
    1. Generate CycloneDX SBOMs for two image versions.
    2. Write a short script that searches the SBOMs for a given package name and version range and prints the matching image.
    3. Re-scan one SBOM with `trivy sbom`.
  - **Done when:** your script finds a known dependency (for example `express`) in the correct SBOM, and the SBOM scan reports the same vulnerabilities as the image scan.

- [ ] **T5 — Sign and verify, then tamper** *(≈50 min, advanced)*
  - **Goal:** see signatures protect a digest.
  - **Steps:**
    1. Follow step 7 to sign and verify.

2. Rebuild with a small change, push it under the same tag, and verify the new digest.
3. Write down why verifying by tag would be unsafe and verifying a digest is safe.

- **Done when:** verification succeeds for the signed digest and fails for the unsigned one.

☐ **T6 — Prove it: security review of `shop-api`** *(≈60 min, advanced)*

- **Goal:** deliver an evidence-based review.
- **Steps:**
  1. Without notes, list the 10 controls you would require before production (base, user, caps, rootfs, secrets, scan gate, SBOM, signing, limits, network exposure).
  2. For each, state how to verify it with a command (`docker inspect`, `trivy`, `cosign verify`, `docker compose config`).
  3. Run the checks against your capstone stack and mark pass/fail.
- **Done when:** you have a table of control, command and result, with every failing item turned into a ticket or fix.

10 Self-check questions

1. What is a CVE, and how is it different from an SBOM?
2. Why use `--ignore-unfixed` in a gating scan, and what is the downside?
3. Why sign a digest rather than a tag?
4. What does the default seccomp profile do, and why shouldn't you disable it?
5. Why should deployed images be rescanned even if the build scan was clean?
6. Why is mounting `/var/run/docker.sock` into a container dangerous?

ANSWERS

1. A CVE is a public identifier for a specific known vulnerability; an SBOM is the inventory of components inside your software, which you match against CVE data.
2. It hides findings that have no available fix so the gate only fails on actionable issues; the downside is that unfixed risks go unseen unless reported separately and tracked.
3. A tag can be moved to different content after signing; a digest identifies exactly one image.
4. It blocks a list of dangerous or rarely needed system calls, reducing kernel attack surface; disabling it removes that safety net.
5. New vulnerabilities are published daily against components you already shipped.
6. Anyone who controls that container can talk to the Docker daemon, start privileged containers and take over the host: it is effectively root on the host.

11 Further reading

- Trivy documentation (trivy.dev) and Grype/Syft documentation (github.com/anchore).
- Sigstore documentation (docs.sigstore.dev) — cosign signing and verification.
- Docker documentation: "Docker security", "Seccomp security profiles" and "Build attestations".
- OWASP Docker Security Cheat Sheet and the CIS Docker Benchmark.
- NIST Secure Software Development Framework (SSDF) and the SLSA framework (slsa.dev) for supply-chain maturity levels.
- CISA Known Exploited Vulnerabilities catalog; the SPDX and CycloneDX specifications for SBOM formats.

LAB NOTES What I ran, what I saw, what it means

★ Containerize shop-api with Docker Compose

Level 4 capstone ~8 hours

1 Goal and outcome

Containerize the whole `shop-api` platform and run it with **one command**: a Node.js API, PostgreSQL, Redis and an Nginx reverse proxy, watched by Prometheus (metrics), Loki with Grafana Alloy (logs) and Grafana (dashboards). Along the way you apply **all 14 topics** of this level: image internals, layers, container lifecycle, registries, volumes, networks, Dockerfiles, Compose, caching, multi-stage builds, non-root containers, health checks, minimal base images, and image scanning.

When you finish you will have:

- `docker compose up -d --wait` brings up **eight containers** (`nginx`, `api`, `db`, `cache`, `prometheus`, `loki`, `alloy`, `grafana`) in dependency order, gated by health checks.
- `curl http://127.0.0.1:8080/products` works through Nginx; the first call is a cache miss and the next a hit.
- A provisioned Grafana dashboard shows request rate, latency percentiles, cache hit ratio and live API/Nginx logs.
- Data survives `docker compose down` and container replacement; `db` and `cache` cannot reach the internet or the host network.
- A hardened, scanned, tagged API image pushed to a registry (a local one in this guide).

2 Architecture

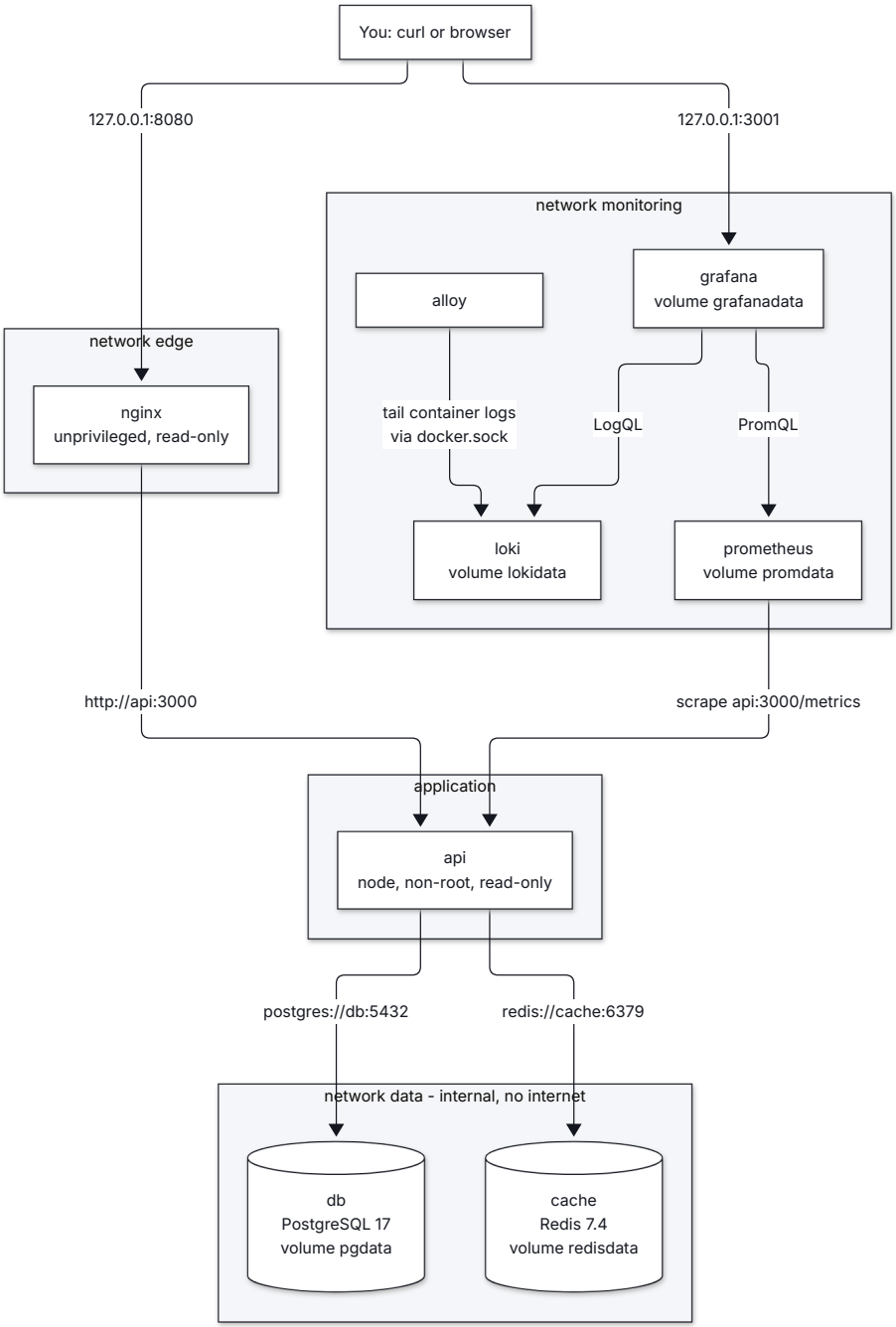


Figure 4-1

Startup order enforced by `depends_on` plus health checks:

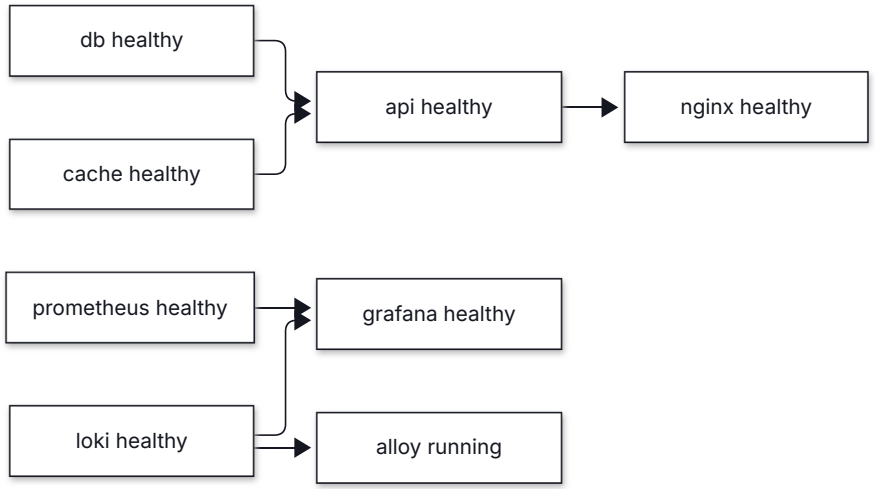


Figure 4-2

Service	Image	Published port	Networks	Persistent data
nginx	`nginxinc/nginx-unprivileged:1.28-alpine`	`127.0.0.1:8080`	edge	none
api	`shop-api:dev` (built here)	none	edge, data, monitoring	none
db	`postgres:17-alpine`	none	data	`pgdata`
cache	`redis:7.4-alpine`	none	data	`redisdata`
prometheus	`prom/prometheus:v3.5.5`	`127.0.0.1:9090`	monitoring	`promdata`
loki	`grafana/loki:3.6`	none	monitoring	`lokidata`
alloy	`grafana/alloy:v1.20.1`	none	monitoring	`alloydata`
grafana	`grafana/grafana:12.2`	`127.0.0.1:3001`	monitoring	`grafanadata`

3 Prerequisites and cost

- **Tools:** Docker Engine 25+ or Docker Desktop with Compose v2 (`docker compose version`), `curl`, `python3` (for JSON one-liners), a text editor. About 4 GB of free RAM and 3-4 GB of disk.

- **Free ports on localhost:** 8080, 9090, 3001. **Cost:** nothing; everything runs locally. (The optional ECR stretch goal has a small cost.)
- **Prior topics:** work through topics 1-14 first, or use this capstone as the checklist of what you must be able to do.
- **Authoring note:** the Compose file passed `docker compose config`, the Nginx configuration passed `nginx -t`, and the API source was run against local PostgreSQL and Redis with Nginx in front. The container images themselves could not be started in the authoring environment, so image versions, log lines and timings below are illustrative. If a tag has moved on, pick the nearest current one.
- **Security note:** Alloy mounts `/var/run/docker.sock` (read-only) to discover containers. Access to that socket is effectively root on the host, so this is a deliberate **lab trade-off**; stretch goal S3 replaces it.

4 Build it step by step

Milestone 1 — Project skeleton and the `shop-api` source *(topics 3, 7)*

```bash
mkdir -p ~/lab/shopstack && cd ~/lab/shopstack
mkdir -p api db nginx prometheus loki alloy grafana/provisioning/datasources
  grafana/provisioning/dashboards grafana/dashboards
```

The API exposes `/healthz` (liveness), `/readyz` (readiness), `/products` (cached in Redis), `POST /orders`, and `/metrics` (Prometheus), logs one JSON object per line to stdout, and shuts down gracefully on `SIGTERM`.

`api/package.json`

```bash
cat > api/package.json <<'EOF'
{
  "name": "shop-api",
  "version": "1.0.0",
  "description": "Sample API used throughout the DevOps roadmap",
  "main": "server.js",
  "private": true,
  "scripts": {
    "start": "node server.js"
  },
  "engines": {
    "node": "⩾22"
  },
  "dependencies": {
    "express": "^5.1.0",
```

```
    "pg": "^8.13.0",
    "prom-client": "^15.1.3",
    "redis": "^5.0.0"
  }
}
EOF
```

`api/server.js`

BASH

```
cat > api/server.js <<'EOF'
'use strict';
const express = require('express');
const { Pool } = require('pg');
const { createClient } = require('redis');
const client = require('prom-client');

const PORT = Number(process.env.PORT || 3000);
const CACHE_TTL = Number(process.env.CACHE_TTL_SECONDS || 30);

const pool = new Pool({ connectionString: process.env.DATABASE_URL, max: 10,
  connectionTimeoutMillis: 2000 });
// disableOfflineQueue: when Redis is down, commands fail immediately instead of hanging.
const redis = createClient({ url: process.env.REDIS_URL, disableOfflineQueue: true });

// One JSON object per line on stdout: easy for Loki (and any log shipper) to parse.
const LEVELS = { debug: 10, info: 20, warn: 30, error: 40 };
const MIN_LEVEL = LEVELS[process.env.LOG_LEVEL] || LEVELS.info;
function log(level, msg, extra = {}) {
  if (LEVELS[level] < MIN_LEVEL) return;
  console.log(JSON.stringify({ ts: new Date().toISOString(), level, msg, ...extra }));
}
redis.on('error', (err) => log('error', 'redis error', { err: err.message }));
pool.on('error', (err) => log('error', 'postgres pool error', { err: err.message }));

// Fail fast instead of hanging when a dependency is down (health checks must answer
  quickly).
function withTimeout(promise, ms) {
  let timer;
  const timeout = new Promise((_, reject) => {
    timer = setTimeout(() => reject(new Error('dependency timeout')), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

// Prometheus metrics
const register = new client.Registry();
client.collectDefaultMetrics({ register });
const httpDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request latency in seconds',
```

```
  labelNames: ['method', 'route', 'status'],
  buckets: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5],
  registers: [register],
});
const cacheResults = new client.Counter({
  name: 'shop_cache_requests_total',
  help: 'Product cache lookups by result',
  labelNames: ['result'],
  registers: [register],
});

const app = express();
app.disable('x-powered-by');
app.use(express.json());

app.use((req, res, next) => {
  const stop = httpDuration.startTimer();
  res.on('finish', () => {
    const route = req.route ? req.route.path : 'unmatched';
    stop({ method: req.method, route, status: res.statusCode });
    if (route !== '/metrics' && route !== '/healthz') {
      log('info', 'request', { method: req.method, path: req.originalUrl, status:
  res.statusCode });
    }
  });
  next();
});

// Liveness: the process is up and the event loop answers. Does NOT touch dependencies.
app.get('/healthz', (req, res) => res.json({ status: 'ok' }));

// Readiness: can we actually serve traffic? Checks PostgreSQL and Redis.
app.get('/readyz', async (req, res) => {
  try {
    await withTimeout(pool.query('SELECT 1'), 2000);
    await withTimeout(redis.ping(), 2000);
    res.json({ status: 'ready' });
  } catch (err) {
    res.status(503).json({ status: 'not ready', error: err.message });
  }
});

app.get('/products', async (req, res, next) => {
  try {
    const cached = await redis.get('products');
    if (cached) {
      cacheResults.inc({ result: 'hit' });
      res.set('X-Cache', 'HIT');
      return res.json(JSON.parse(cached));
    }
    cacheResults.inc({ result: 'miss' });
    const { rows } = await pool.query('SELECT id, name, price_cents FROM products ORDER BY
  id');
```

```
    await redis.set('products', JSON.stringify(rows), { EX: CACHE_TTL });
    res.set('X-Cache', 'MISS');
    res.json(rows);
  } catch (err) {
    next(err);
  }
});

app.post('/orders', async (req, res, next) => {
  try {
    const { product_id: productId, quantity } = req.body || {};
    if (!Number.isInteger(productId) || !Number.isInteger(quantity) || quantity < 1) {
      return res.status(400).json({ error: 'product_id and quantity must be positive
  integers' });
    }
    const { rows } = await pool.query(
      'INSERT INTO orders (product_id, quantity) VALUES ($1, $2) RETURNING id, product_id,
  quantity, created_at',
      [productId, quantity]
    );
    res.status(201).json(rows[0]);
  } catch (err) {
    next(err);
  }
});

app.get('/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});

// Error handler: log details, return a generic message
app.use((err, req, res, next) => {
  log('error', 'unhandled error', { err: err.message, path: req.originalUrl });
  res.status(500).json({ error: 'internal error' });
});

async function main() {
  // Do not block startup on Redis: the node-redis client keeps retrying in the
  background,
  // /healthz stays green and /readyz reports 503 until the connection is up.
  redis.connect().catch((err) => log('error', 'redis connect failed', { err: err.message
  }));
  const server = app.listen(PORT, '0.0.0.0', () => log('info', 'shop-api listening', {
  port: PORT }));

  // Graceful shutdown: stop accepting connections, drain, close dependencies, exit 0.
  let shuttingDown = false;
  async function shutdown(signal) {
    if (shuttingDown) return;
    shuttingDown = true;
    log('info', 'shutdown started', { signal });
    server.close(async () => {
```

```
      try {
        await pool.end();
        await redis.quit();
      } catch (err) {
        log('error', 'error during shutdown', { err: err.message });
      }
      log('info', 'shutdown complete');
      process.exit(0);
    });
    setTimeout(() => process.exit(1), 8000).unref(); // hard stop before Docker's SIGKILL
  }
  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT', () => shutdown('SIGINT'));
}

main().catch((err) => {
  log('error', 'fatal startup error', { err: err.message });
  process.exit(1);
});
EOF
```

`db/init.sql`

```bash
cat > db/init.sql <<'EOF'
CREATE TABLE IF NOT EXISTS products (
  id          SERIAL PRIMARY KEY,
  name        TEXT    NOT NULL,
  price_cents INTEGER NOT NULL CHECK (price_cents >= 0)
);

CREATE TABLE IF NOT EXISTS orders (
  id         SERIAL PRIMARY KEY,
  product_id INTEGER NOT NULL REFERENCES products(id),
  quantity   INTEGER NOT NULL CHECK (quantity > 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO products (name, price_cents) VALUES
  ('Mechanical keyboard', 8999),
  ('USB-C hub', 3499),
  ('27-inch monitor', 21999);
EOF
```

Create the lockfile (the Dockerfile uses `npm ci`, which requires one) and optionally run the API without Docker if you have Node.js 22:

```bash
cd api && npm install --package-lock-only && cd ..
ls api   # package-lock.json  package.json  server.js
```

(Note: at the time of writing the npm registry marks `prom-client` as deprecated in favour of a successor package; its API used here is stable and fine for this lab, and swapping libraries is a stretch goal.)

Milestone 2 — A production-quality API image *(topics 2, 7, 9, 10, 11, 12, 13)*

`api/Dockerfile`

```bash
cat > api/Dockerfile <<'EOF'
# ---- base: manifests only, so dependency layers cache well ----
FROM node:22-alpine AS base
WORKDIR /app
COPY package.json package-lock.json ./

# ---- dev-deps: everything, used by the test stage ----
FROM base AS dev-deps
RUN --mount=type=cache,target=/root/.npm npm ci

# ---- prod-deps: runtime dependencies only ----
FROM base AS prod-deps
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev

# ---- test: run in CI with `docker build --target test` ----
FROM dev-deps AS test
COPY server.js ./
RUN node --check server.js

# ---- runtime: the image we ship ----
FROM node:22-alpine AS runtime
LABEL org.opencontainers.image.title="shop-api" \
      org.opencontainers.image.source="https://github.com/example/shop-api"
WORKDIR /app
ENV NODE_ENV=production PORT=3000
COPY --from=prod-deps /app/node_modules ./node_modules
COPY package.json server.js ./
USER 1000:1000
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
  CMD node -e "fetch('http://127.0.0.1:3000/readyz').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"
CMD ["node", "server.js"]
```

```
EOF
```

`api/.dockerignore`

```bash
cat > api/.dockerignore <<'EOF'
node_modules
npm-debug.log
.git
.gitignore
.env
.env.*
Dockerfile
.dockerignore
*.md
coverage
EOF
```

Build the test stage first (a CI gate), then the runtime image, and inspect it:

```bash
docker build --target test -t shop-api:test ./api
docker build -t shop-api:dev ./api
docker image ls shop-api
docker history shop-api:dev
docker run --rm shop-api:dev id          # uid=1000(node) gid=1000(node)
```

Rebuild after touching only `server.js` and confirm the `npm ci` step prints `CACHED`:

```bash
echo "// touch" >> api/server.js && docker build --progress=plain -t shop-api:dev ./api
  2>&1 | grep -E 'CACHED|npm ci'
```

Milestone 3 — Supporting configuration files *(topics 5, 6, 7)*

Nginx — JSON access logs, a health endpoint, `/metrics` hidden from the outside:

`nginx/default.conf`

```bash
cat > nginx/default.conf <<'EOF'
# One JSON object per request: easy to query in Loki with `| json`.
log_format json_combined escape=json
  '{"ts":"$time_iso8601","remote_addr":"$remote_addr","method":"$request_method",'
  '"uri":"$request_uri","status":$status,"bytes":$body_bytes_sent,'
  '"request_time":$request_time,"upstream_time":"$upstream_response_time",'
  '"request_id":"$request_id"}';
```

```
upstream shop_api {
    server api:3000;
    keepalive 16;
}

server {
    listen 8080;
    server_name _;

    access_log /var/log/nginx/access.log json_combined;
    error_log  /var/log/nginx/error.log warn;

    # Cheap endpoint for the container health check (does not touch the API).
    location = /nginx-health {
        access_log off;
        default_type text/plain;
        return 200 "ok\n";
    }

    # Prometheus scrapes the API directly on the private network; never expose metrics
  publicly.
    location = /metrics {
        return 404;
    }

    location / {
        proxy_pass http://shop_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Request-Id $request_id;
        proxy_connect_timeout 3s;
        proxy_read_timeout 30s;
    }
}
EOF
```

Prometheus — scrape configuration and two alert rules:

`prometheus/prometheus.yml`

BASH

```
cat > prometheus/prometheus.yml <<'EOF'
global:
  scrape_interval: 15s
  evaluation_interval: 15s
```

```
rule_files:
  - /etc/prometheus/rules.yml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: shop-api
    metrics_path: /metrics
    static_configs:
      - targets: ["api:3000"]

  - job_name: loki
    static_configs:
      - targets: ["loki:3100"]

  - job_name: grafana
    static_configs:
      - targets: ["grafana:3000"]

  - job_name: alloy
    static_configs:
      - targets: ["alloy:12345"]
EOF
```

`prometheus/rules.yml`

BASH

```
cat > prometheus/rules.yml <<'EOF'
groups:
  - name: shop-api
    rules:
      - alert: ShopApiDown
        expr: up{job="shop-api"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "shop-api is not being scraped"

      - alert: ShopApiHighErrorRate
        expr: |
          sum(rate(http_request_duration_seconds_count{job="shop-api",status=~"5.."}[5m]))
            /
          sum(rate(http_request_duration_seconds_count{job="shop-api"}[5m])) > 0.05
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "More than 5 percent of shop-api requests are failing"
```

```
EOF
```

Loki — single-process, filesystem storage, 7-day retention:

`loki/loki-config.yaml`

```bash
cat > loki/loki-config.yaml <<'EOF'
auth_enabled: false

server:
  http_listen_port: 3100
  grpc_listen_port: 9096

common:
  instance_addr: 127.0.0.1
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory

schema_config:
  configs:
    - from: 2024-04-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

limits_config:
  retention_period: 168h        # keep 7 days of logs
  allow_structured_metadata: true

compactor:
  working_directory: /loki/compactor
  retention_enabled: true
  delete_request_store: filesystem

analytics:
  reporting_enabled: false
EOF
```

Alloy — discover containers through the Docker socket and ship their logs to Loki:

`alloy/config.alloy`

```bash
cat > alloy/config.alloy <<'EOF'
// Discover every container through the Docker socket.
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"
}

// Turn Docker metadata into Loki labels.
discovery.relabel "containers" {
  targets = discovery.docker.containers.targets

  rule {
    source_labels = ["__meta_docker_container_name"]
    regex         = "/(.*)"
    target_label  = "container"
  }
  rule {
    source_labels = ["__meta_docker_container_label_com_docker_compose_service"]
    target_label  = "service"
  }
  rule {
    source_labels = ["__meta_docker_container_label_com_docker_compose_project"]
    target_label  = "project"
  }
}

// Tail container stdout/stderr and forward to Loki.
loki.source.docker "containers" {
  host          = "unix:///var/run/docker.sock"
  targets       = discovery.docker.containers.targets
  relabel_rules = discovery.relabel.containers.rules
  forward_to    = [loki.write.default.receiver]
}

loki.write "default" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
EOF
```

Grafana — provision data sources and a dashboard so there is nothing to click through:

`grafana/provisioning/datasources/datasources.yaml`

```bash
cat > grafana/provisioning/datasources/datasources.yaml <<'EOF'
apiVersion: 1
datasources:
```

```
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
  - name: Loki
    uid: loki
    type: loki
    access: proxy
    url: http://loki:3100
EOF
```

`grafana/provisioning/dashboards/dashboards.yaml`

BASH

```
cat > grafana/provisioning/dashboards/dashboards.yaml <<'EOF'
apiVersion: 1
providers:
  - name: shop
    folder: Shop
    type: file
    allowUiUpdates: true
    options:
      path: /etc/grafana/dashboards
EOF
```

`grafana/dashboards/shop-api.json`

BASH

```
cat > grafana/dashboards/shop-api.json <<'EOF'
{
  "uid": "shop-api",
  "title": "shop-api overview",
  "schemaVersion": 39,
  "version": 1,
  "refresh": "10s",
  "time": {"from":"now-30m","to":"now"},
  "tags": ["shop"],
  "panels": [
    {"id":1,"title":"Request rate by status","type":"timeseries","gridPos":
  {"x":0,"y":0,"w":12,"h":8},"datasource":
  {"type":"prometheus","uid":"prometheus"},"targets":[{"refId":"A","expr":"sum by
  (status) (rate(http_request_duration_seconds_count{job=\"shop-api\"}
  [1m]))","legendFormat":"{{status}}"}],"fieldConfig":{"defaults":
  {"unit":"reqps"},"overrides":[]}},
    {"id":2,"title":"Latency p95 by route","type":"timeseries","gridPos":
  {"x":12,"y":0,"w":12,"h":8},"datasource":
  {"type":"prometheus","uid":"prometheus"},"targets":
```

```
    [{"refId":"A","expr":"histogram_quantile(0.95, sum by (le, route)
    (rate(http_request_duration_seconds_bucket{job=\"shop-api\"}[5m])))","legendFormat":"
    {{route}}"}],"fieldConfig":{"defaults":{"unit":"s"},"overrides":[]}},
      {"id":3,"title":"Cache hit ratio","type":"stat","gridPos":
    {"x":0,"y":8,"w":6,"h":6},"datasource":
    {"type":"prometheus","uid":"prometheus"},"targets":
    [{"refId":"A","expr":"sum(rate(shop_cache_requests_total{result=\"hit\"}[5m])) /
    sum(rate(shop_cache_requests_total[5m]))"}],"fieldConfig":{"defaults":
    {"unit":"percentunit"},"overrides":[]}},
      {"id":4,"title":"Scrape targets up","type":"stat","gridPos":
    {"x":6,"y":8,"w":6,"h":6},"datasource":
    {"type":"prometheus","uid":"prometheus"},"targets":[{"refId":"A","expr":"sum(up)"}]},
      {"id":5,"title":"shop-api logs","type":"logs","gridPos":
    {"x":12,"y":8,"w":12,"h":6},"datasource":{"type":"loki","uid":"loki"},"targets":
    [{"refId":"A","expr":"{service=\"api\"} | json"}]},
      {"id":6,"title":"nginx access logs","type":"logs","gridPos":
    {"x":0,"y":14,"w":24,"h":8},"datasource":{"type":"loki","uid":"loki"},"targets":
    [{"refId":"A","expr":"{service=\"nginx\"} | json | status ⩾ 400"}]}
    ]
}
EOF
```

Milestone 4 — The Compose file and secrets *(topics 5, 6, 8, 11, 12)*

Secrets live in `.env` (git-ignored); the template `.env.example` is committed:

BASH

```
cat > .env.example <<'EOF'
POSTGRES_PASSWORD=change-me-dev-only
GRAFANA_ADMIN_PASSWORD=change-me-too
EOF
cp .env.example .env
printf '.env\n' > .gitignore
```

`compose.yaml`

BASH

```
cat > compose.yaml <<'EOF'
name: shopstack

x-logging: &default-logging
  driver: json-file
  options:
    max-size: "10m"
    max-file: "3"

services:
  # ---------- edge ----------
  nginx:
    image: nginxinc/nginx-unprivileged:1.28-alpine
```

```
    ports:
      - "127.0.0.1:8080:8080"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      api:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider http://127.0.0.1:8080/nginx-health || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 3
      start_period: 5s
    read_only: true
    tmpfs:
      - /tmp
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    mem_limit: 128m
    pids_limit: 100
    init: true
    restart: unless-stopped
    logging: *default-logging
    networks: [edge]

  # ---------- application ----------
  api:
    build:
      context: ./api
      target: runtime
    image: shop-api:dev
    environment:
      PORT: "3000"
      LOG_LEVEL: info
      CACHE_TTL_SECONDS: "30"
      DATABASE_URL: postgres://shop:${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in
  .env}@db:5432/shop
      REDIS_URL: redis://cache:6379
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_healthy
    read_only: true
    tmpfs:
      - /tmp
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    mem_limit: 256m
    cpus: 0.5
    pids_limit: 100
    init: true
    stop_grace_period: 15s
```

```
    restart: unless-stopped
    logging: *default-logging
    networks: [edge, data, monitoring]

  # ---------- data ----------
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s
    mem_limit: 512m
    pids_limit: 200
    restart: unless-stopped
    logging: *default-logging
    networks: [data]

  cache:
    image: redis:7.4-alpine
    command: ["redis-server", "--appendonly", "yes", "--maxmemory", "64mb", "--maxmemory-
  policy", "allkeys-lru"]
    volumes:
      - redisdata:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10
    mem_limit: 128m
    restart: unless-stopped
    logging: *default-logging
    networks: [data]

  # ---------- observability ----------
  prometheus:
    image: prom/prometheus:v3.5.5
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=7d
      - --web.enable-lifecycle
    ports:
      - "127.0.0.1:9090:9090"
    volumes:
```

```
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/rules.yml:/etc/prometheus/rules.yml:ro
      - promdata:/prometheus
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider http://localhost:9090/-/ready || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 10s
    mem_limit: 512m
    restart: unless-stopped
    logging: *default-logging
    networks: [monitoring]

  loki:
    image: grafana/loki:3.6
    command: ["-config.file=/etc/loki/loki-config.yaml"]
    volumes:
      - ./loki/loki-config.yaml:/etc/loki/loki-config.yaml:ro
      - lokidata:/loki
    healthcheck:
      # Uses wget, falling back to curl, so it works whichever tool the image provides.
      test: ["CMD-SHELL", "wget -q --spider http://localhost:3100/ready || curl -sf
http://localhost:3100/ready"]
      interval: 10s
      timeout: 3s
      retries: 10
      start_period: 30s
    mem_limit: 512m
    restart: unless-stopped
    logging: *default-logging
    networks: [monitoring]

  alloy:
    image: grafana/alloy:v1.20.1
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy/data
      - /etc/alloy/config.alloy
    user: "0:0"        # needed to read the Docker socket; see the security note in the
capstone
    volumes:
      - ./alloy/config.alloy:/etc/alloy/config.alloy:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - alloydata:/var/lib/alloy/data
    depends_on:
      loki:
        condition: service_healthy
    mem_limit: 256m
    restart: unless-stopped
    logging: *default-logging
    networks: [monitoring]
```

```
  grafana:
    image: grafana/grafana:12.2
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:?set GRAFANA_ADMIN_PASSWORD in
  .env}
      GF_USERS_ALLOW_SIGN_UP: "false"
    ports:
      - "127.0.0.1:3001:3000"
    volumes:
      - grafanadata:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/etc/grafana/dashboards:ro
    depends_on:
      prometheus:
        condition: service_healthy
      loki:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider http://localhost:3000/api/health || curl -sf
  http://localhost:3000/api/health"]
      interval: 10s
      timeout: 3s
      retries: 10
      start_period: 20s
    mem_limit: 512m
    restart: unless-stopped
    logging: *default-logging
    networks: [monitoring]

networks:
  edge:
  data:
    internal: true        # db and cache have no route to the internet
  monitoring:

volumes:
  pgdata:
  redisdata:
  promdata:
  lokidata:
  alloydata:
  grafanadata:
EOF
```

Validate it before running anything:

```bash
docker compose config --quiet && echo "compose file OK"
docker compose config --services
```

Milestone 5 — Bring the stack up in stages *(topics 1, 3, 6, 8, 12)*

```bash
docker compose up -d --wait db cache        # data tier first
docker compose ps                            # both (healthy)
docker compose up -d --wait api nginx        # app and edge
curl -s http://127.0.0.1:8080/readyz         # {"status":"ready"}
curl -si http://127.0.0.1:8080/products | grep -iE 'HTTP/|x-cache'   # X-Cache: MISS
curl -si http://127.0.0.1:8080/products | grep -i x-cache            # X-Cache: HIT
curl -s -X POST -H 'content-type: application/json' -d '{"product_id":1,"quantity":2}'
  http://127.0.0.1:8080/orders
```

Then the observability tier, and finally everything together:

```bash
docker compose up -d --wait prometheus loki alloy grafana
docker compose up -d --wait                  # idempotent: converges the whole stack
docker compose ps
```

Expected: eight containers, seven of them `healthy` (Alloy has no health check and shows `running`).

Milestone 6 — Generate traffic and observe *(topics 1, 5, 12)*

```bash
for i in $(seq 1 200); do
  curl -s -o /dev/null http://127.0.0.1:8080/products
  curl -s -o /dev/null -X POST -H 'content-type: application/json' -d
  '{"product_id":2,"quantity":1}' http://127.0.0.1:8080/orders
done
```

- **Prometheus** `http://127.0.0.1:9090/targets` shows five targets up (`prometheus`, `shop-api`, `loki`, `grafana`, `alloy`). Try `sum(rate(http_request_duration_seconds_count[1m]))`.
- **Grafana** `http://127.0.0.1:3001` (user `admin`, password from `.env`) → *Dashboards* → *Shop* → *shop-api overview*. You should see request rate by status, latency, cache hit ratio and the two log panels.
- **Loki via Grafana Explore:** run `{service="api"} | json | level="info"` and `{service="nginx"} | json | status ⩾ 400`.

A scripted smoke test you can reuse (and call from CI):

```bash
cat > smoke.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
```

```
set -a; . ./.env; set +a
base=http://127.0.0.1:8080
curl -fsS "$base/readyz" >/dev/null && echo "PASS readyz via nginx"
curl -fsS "$base/products" >/dev/null                    # prime the cache
hdr=$(curl -fsS -D - -o /dev/null "$base/products")
grep -qi 'x-cache: hit' <<<"$hdr" && echo "PASS cache hit"
test "$(curl -s -o /dev/null -w '%{http_code}' "$base/metrics")" = 404 && echo "PASS
  metrics hidden from edge"
curl -fsS 'http://127.0.0.1:9090/api/v1/targets?state=active' | python3 -c '
import sys, json
t = json.load(sys.stdin)["data"]["activeTargets"]
bad = [x["labels"]["job"] for x in t if x["health"] != "up"]
print("targets:", len(t), "down:", bad); sys.exit(1 if bad else 0)' && echo "PASS
  prometheus targets"
curl -fsS -u "admin:${GRAFANA_ADMIN_PASSWORD}" http://127.0.0.1:3001/api/health >/dev/null
  && echo "PASS grafana"
labels=$(curl -fsS -u "admin:${GRAFANA_ADMIN_PASSWORD}"
  http://127.0.0.1:3001/api/datasources/proxy/uid/loki/loki/api/v1/labels)
grep -q service <<<"$labels" && echo "PASS loki has service labels"
echo "ALL CHECKS PASSED"
EOF
chmod +x smoke.sh && ./smoke.sh
```

Milestone 7 — Harden, scan, tag and publish *(topics 2, 4, 11, 13, 14)*

BASH

```
# 1. Evidence of hardening
docker inspect shopstack-api-1 --format 'user={{.Config.User}} ro=
  {{.HostConfig.ReadonlyRootfs}} capdrop={{.HostConfig.CapDrop}} secopt=
  {{.HostConfig.SecurityOpt}}'

# 2. Scan the image (fail on fixable HIGH or CRITICAL)
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-cache:/root/.cache \
  aquasec/trivy:0.58.0 image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 shop-
  api:dev

# 3. SBOM
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v "$PWD":/out
  aquasec/trivy:0.58.0 \
  image --format cyclonedx --output /out/sbom.cdx.json shop-api:dev

# 4. Tag by git SHA and version, push to a local registry, pull back by digest
docker run -d --name registry -p 127.0.0.1:5000:5000 registry:2
SHA=$(git rev-parse --short HEAD 2>/dev/null || echo local1)
docker tag shop-api:dev localhost:5000/shop/shop-api:1.0.0
docker tag shop-api:dev localhost:5000/shop/shop-api:git-$SHA
docker push --all-tags localhost:5000/shop/shop-api
docker image ls --digests localhost:5000/shop/shop-api
```

(Container names follow `<project>-<service>-1`; run `docker compose ps` if yours differ.) Whatever the scan reports, record the count and decide on each finding: fix, accept with an expiry, or document as a false positive.

5 Tasks

- [ ] **T1 — Build and run the API image alone** *(≈45 min, intermediate; milestones 1-2; topics 2, 3, 7, 9, 10, 11, 12)*
  - **Goal:** a lean, non-root, health-checked API image.
  - **Steps:**
    1. Create the files from milestones 1-2 and build `shop-api:dev` and the `test` target.
    2. Run `docker run --rm -d --name api-solo -p 3000:3000 -e DATABASE_URL=postgres://x -e REDIS_URL=redis://x shop-api:dev`, then check `curl localhost:3000/healthz` (200) and `/readyz` (503 after about 2 s).
    3. Run `docker stop api-solo` and time it; read `docker inspect api-solo --format '{{.State.Health.Status}}'`.
  - **Done when:** `/healthz` is 200, `/readyz` is 503 (dependencies absent), the container reports `unhealthy` after a minute, `docker stop` takes under 3 seconds, and `id` shows UID 1000.

- [ ] **T2 — Bring up the whole stack with health gating** *(≈60 min, intermediate; milestones 3-5; topics 6, 8, 12)*
  - **Goal:** the eight-container stack, started in the right order.
  - **Steps:**
    1. Create all configuration files and `compose.yaml`; run `docker compose config --quiet`.
    2. Run `docker compose up -d --wait` and then `docker compose ps`.
    3. Run `./smoke.sh`.
  - **Done when:** `smoke.sh` prints `ALL CHECKS PASSED` and `docker compose ps` shows seven healthy services plus a running Alloy.

- [ ] **T3 — Prove persistence and backups** *(≈45 min, intermediate; milestone 5; topic 5)*
  - **Goal:** data outlives containers; you can restore it.
  - **Steps:**
    1. `POST` three orders and note `SELECT count(*) FROM orders` via `docker compose exec db psql -U shop -d shop`.

2. `docker compose down` then `up -d --wait`: the count must be unchanged. Then `docker compose rm -sf db && docker compose up -d --wait db`: still unchanged.
3. Take a backup: `docker compose exec -T db pg_dump -U shop --clean --if-exists shop > backup.sql` (`--clean --if-exists` makes the dump drop and recreate the tables). Run `docker compose down -v`, bring the stack up (a fresh volume runs `init.sql` again), restore with `docker compose exec -T db psql -U shop -d shop < backup.sql`, and recount.

- **Done when:** the order count is identical after `down`, after container replacement, and after the restore; and you can state what `down -v` deleted.

- [ ] **T4 — Prove the network design** *(≈45 min, intermediate; milestone 5; topic 6)*
  - **Goal:** demonstrate isolation instead of assuming it.
  - **Steps:**
    1. From the host confirm only `8080`, `9090`, `3001` are published, all on `127.0.0.1` (`docker compose ps`, `ss -ltn`).
    2. `docker compose exec nginx nslookup db` must fail (different network); `docker compose exec api nslookup db` must succeed.
    3. `docker compose exec cache sh -c 'wget -T 3 -qO- http://example.com || echo no-internet'` must print `no-internet` (the `data` network is internal).
    4. Explain the role of the embedded DNS server (`127.0.0.11`) with `docker compose exec api cat /etc/resolv.conf`.
  - **Done when:** each of the three negative/positive checks behaves as stated and you can draw the three networks from memory.

- [ ] **T5 — Observe a failure end to end** *(≈60 min, advanced; milestone 6; topics 1, 3, 12)*
  - **Goal:** use metrics and logs to find a broken dependency.
  - **Steps:**
    1. Run a background load loop on `/products` and `/orders`.
    2. `docker compose stop cache`. Watch `/readyz` go 503, the `api` container turn `unhealthy`, the error-rate panel rise, and the log panel show `redis error` lines.
    3. `docker compose start cache`; confirm recovery. Note how long each signal took to show up.
  - **Done when:** you have a short timeline (event, first signal, time) showing a metric, a log line and a health status all reflecting the outage, and the stack recovers without restarting the API by hand.

- [ ] **T6 — Break it three ways and fix each** *(≈60 min, advanced; milestones 4-5; topics 3, 5, 8, 11)*

- **Goal:** diagnose common failures.
- **Steps:**
  1. **Wrong password:** change `POSTGRES_PASSWORD` in `.env` only (not in the volume) and `docker compose up -d`. Explain why the API can no longer connect although the DB container is healthy (the password is baked into the existing volume).
  2. **Memory limit:** set `mem_limit: 20m` on `api` and run load; find `OOMKilled=true` and exit code 137 with `docker inspect`, then restore a sane limit.
  3. **Read-only root:** remove `tmpfs: [/tmp]` from `nginx` and observe the failure message in `docker compose logs nginx`; restore it.
- **Done when:** for each failure you can show the exact log or `inspect` line that identified it and the one-line fix.

- [ ] **T7 — Measure caching and base-image choices** *(≈60 min, advanced; milestone 2; topics 9, 10, 13)*
  - **Goal:** quantify build speed and size.
  - **Steps:**
    1. Record cold build time (`docker build --no-cache`) and warm rebuild time after editing only `server.js`.
    2. Build variants on `node:22-slim` and a distroless Node.js runtime stage (check the distroless README for current tags) and compare `docker image ls` sizes and shell availability.
    3. Add `COPY . .` before `npm ci` to the Dockerfile, repeat the warm rebuild, and note the regression.
  - **Done when:** you have a table of build times and sizes for at least three variants and can explain each number.

- [ ] **T8 — Scan, gate and generate an SBOM** *(≈45 min, advanced; milestone 7; topics 13, 14)*
  - **Goal:** a security gate with evidence.
  - **Steps:**
    1. Run the Trivy gate from milestone 7 on `shop-api:dev` and on a copy built `FROM node:18` (end of life).
    2. Generate the CycloneDX SBOM and find `express` and its version inside it.
    3. Run `trivy config .` over the project and fix at least one misconfiguration it reports.
  - **Done when:** the old image fails the gate (exit 1) and the current one passes (or every remaining finding has a written decision), and the SBOM contains `express`.

- [ ] **T9 — Publish and pull by digest** *(≈30 min, intermediate; milestone 7; topics 2, 4)*
  - **Goal:** a registry round trip with immutable references.
  - **Steps:**
    1. Push the tags from milestone 7 to the local registry; note that tags share one digest.
    2. Change `compose.yaml` so `api` uses `image: localhost:5000/shop/shop-api@sha256:<digest>` and remove `build:`; run `docker compose pull api && docker compose up -d api`.
  - **Done when:** `docker compose ps` shows the API running from the digest reference and `smoke.sh` still passes.
- [ ] **T10 — Prove it: rebuild from nothing and explain it** *(≈90 min, advanced; all milestones; all topics)*
  - **Goal:** reproduce the platform without the guide, then teach it.
  - **Steps:**
    1. Run `docker compose down -v`, `docker system prune -af --volumes` (on a lab machine only), and delete your working directory except your notes.
    2. Rebuild everything from memory plus your own notes, in under two hours, until `./smoke.sh` passes.
    3. Present (aloud or in writing) the story of one request: Nginx container port, DNS lookup of `api`, cgroup limits, overlay layers, the volume holding the row, the metric emitted, and the log line shipped to Loki.
  - **Done when:** `smoke.sh` passes on the rebuilt stack and your explanation covers namespaces, cgroups, layers, networks, volumes, health checks and the security controls, without opening this page.

6 Verification checklist

- [ ] `docker compose config --quiet` succeeds and the file has no `version:` key.
- [ ] `docker compose ps` shows 8 containers: `nginx`, `api`, `db`, `cache`, `prometheus`, `loki`, `grafana` healthy and `alloy` running.
- [ ] `curl -si http://127.0.0.1:8080/products` returns 200 with `X-Cache: MISS`, then `HIT`.
- [ ] `curl http://127.0.0.1:8080/metrics` returns 404 (metrics are not public) while Prometheus shows `shop-api` as `UP`.
- [ ] `ss -ltn` shows 8080, 9090 and 3001 bound to `127.0.0.1` only; nothing publishes 5432 or 6379.

- [ ] `docker compose exec api id` shows UID 1000; `docker inspect` shows `ReadonlyRootfs=true` and `CapDrop=[ALL]` for `api` and `nginx`.
- [ ] Data survives `docker compose down && docker compose up -d --wait` (order count unchanged).
- [ ] `docker compose exec cache sh -c 'wget -T 3 -qO- http://example.com'` fails (internal network).
- [ ] Grafana dashboard *shop-api overview* shows data for all panels, and Loki returns lines for `{service="api"}` and `{service="nginx"}`.
- [ ] `docker stop` / `docker compose stop api` completes in a few seconds with exit code 0 (graceful `SIGTERM` handling).
- [ ] Trivy gate passes (or each finding has a documented decision) and an SBOM exists.
- [ ] `./smoke.sh` prints `ALL CHECKS PASSED`.

7 Teardown

```bash
docker compose down -v                  # containers, networks and ALL named volumes (data is deleted)
docker rm -f registry api-solo 2>/dev/null
docker rmi shop-api:dev shop-api:test localhost:5000/shop/shop-api:1.0.0 2>/dev/null
docker volume rm trivy-cache 2>/dev/null
docker builder prune -f                 # BuildKit cache
rm -rf ~/lab/shopstack
```

If you pushed anything to ECR or GHCR in a stretch goal, delete the repository or package and revoke any tokens you created.

8 Stretch goals and what to learn next

- **S1 — Distroless runtime:** switch the `runtime` stage to a distroless Node.js image, and debug it with an ephemeral toolbox container instead of a shell.
- **S2 — More exporters:** add `postgres-exporter`, `redis_exporter` and `cadvisor` services, scrape them from Prometheus, and add panels for database connections and container CPU/memory.
- **S3 — Remove the Docker socket:** replace Alloy's socket mount with the Loki Docker logging driver plugin, or a file-based approach, and compare the security trade-offs.

- **S4 — TLS at the edge:** generate a local certificate (for example with `mkcert`), terminate TLS in Nginx, and redirect HTTP to HTTPS.
- **S5 — Resolver-aware Nginx:** Nginx resolves `api` once at startup. Add `resolver 127.0.0.11 valid=10s;` with a variable in `proxy_pass` so it survives API container recreation, and test it with `docker compose up -d --force-recreate api`.
- **S6 — Compose profiles and overrides:** add a `dev` override with bind mounts and `node --watch`, and a `tools` profile with Adminer.
- **S7 — Sign the image** with cosign and verify the digest before `docker compose pull`.
- **S8 — Alerting:** connect Prometheus rules to Alertmanager (or Grafana alerting) and trigger `ShopApiDown` on purpose.

Next: push this build into a pipeline in Level 5: CI/CD §5.1 (automated tests, image builds and registry pushes), then run the same images on a cluster in Level 8: Kubernetes §8.1. For a deeper treatment of the monitoring half of this stack, see Level 10: Observability §10.1.

LEVEL 5 · PART II

CI/CD

5

Goal: Build a fully automated pipeline from git push to verified production deployment with rollback.

RECOMMENDED TIME
2-3 weeks

TOPICS
12

PRACTICE / CAPSTONE
Pipeline: Git Push → GitHub → Lint → Tests → Build → Docker Build → Security Scan → Push Image → Deploy → Health Check → Notify.

Topics

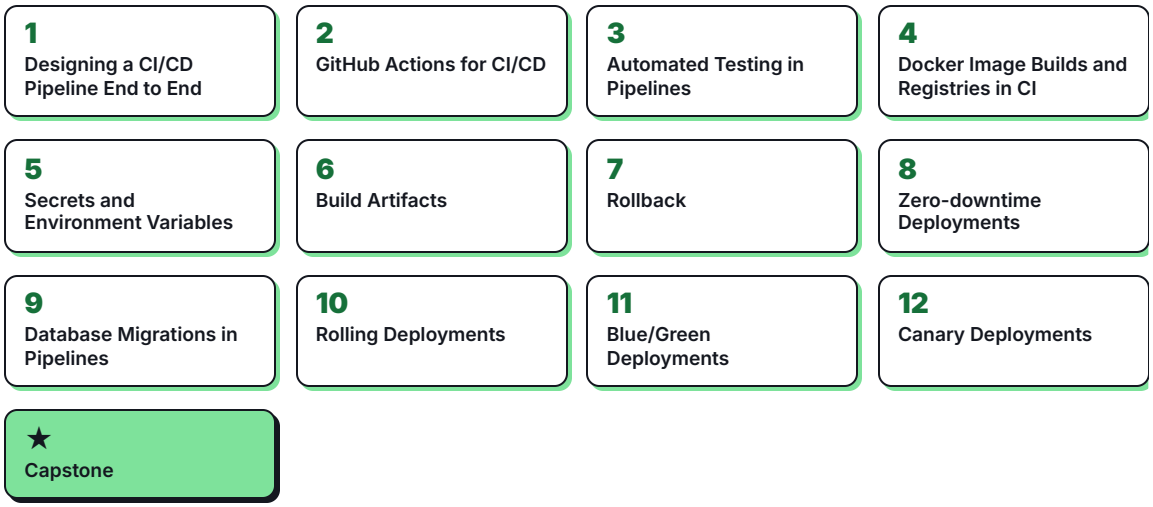
#	Topic	What you will learn
1	Designing a CI/CD Pipeline End to End **p.933**	stages, fail-fast, lint, tests, build, scan, push, deploy, health check, notify
2	GitHub Actions for CI/CD **p.945**	reusable workflows, composite actions, matrix, caching, concurrency
3	Automated Testing in Pipelines **p.959**	unit/integration/e2e, test pyramid, service containers, flaky tests
4	Docker Image Builds and Registries in CI **p.971**	buildx, layer cache in CI, tagging strategy (sha/semver), pushing to GHCR/ECR
5	Secrets and Environment Variables **p.987**	per-env config, secret stores, OIDC, never in images, rotation
6	Build Artifacts **p.1003**	build once deploy many, artifact promotion, retention, provenance
7	Rollback **p.1019**	image rollback, versioned releases, rollback triggers, roll-forward
8	Zero-downtime Deployments **p.1033**	graceful shutdown, connection draining, readiness, health gates
9	Database Migrations in Pipelines **p.1049**	expand/contract, backwards compatible migrations, Flyway/Prisma/Alembic, locking pitfalls
10	Rolling Deployments **p.1067**	batch size, max surge/unavailable, trade-offs

#	Topic	What you will learn
11	Blue/Green Deployments **p.1081**	two environments, traffic switch, instant rollback, cost
12	Canary Deployments **p.1095**	progressive traffic shift, metrics-based promotion, Argo Rollouts/Flagger overview
★	Level capstone project **p.1111**	Pipeline: Git Push → GitHub → Lint → Tests → Build → Docker Build → Security Scan → Push Image → Deploy → Health Check → Notify.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



5.1 Designing a CI/CD Pipeline End to End

Topic 1 of 12 | ~4 hours | Difficulty: Intermediate

1 What it is

A **pipeline** is an automated assembly line that carries a code change from `git push` to running, verified software in production. Each station on the line is a **stage** (lint, test, build, scan, push, deploy, health check, notify). A stage either passes and hands its output to the next stage, or fails and stops the line so a broken change never reaches users.

Three related terms are often mixed up:

Term	Meaning	Human approval before production?
Continuous Integration (CI)	Every change is merged often and automatically built and tested	n/a
Continuous Delivery (CD)	Every change that passes the pipeline is *releasable*; a human presses the button	Yes
Continuous Deployment (CD)	Every change that passes the pipeline is released automatically	No

This topic is about **designing** the whole line: which stages exist, in what order, what each one proves, how fast it must be, and what happens when something fails. The next topics fill in the tools (GitHub Actions, Docker, secrets) and the safe-release techniques (rollback, rolling, blue/green, canary). For the conceptual background see CI/CD in Level 0 §0.2.

2 Why it matters in DevOps

- **Speed of feedback:** a developer who learns in 5 minutes that a change broke the build fixes it while the context is fresh. A developer who learns after 3 hours (or from a customer) pays a context-switch tax.
- **Safety through repetition:** a deploy done by a script 500 times is more reliable than a runbook followed by a tired human at 11 p.m. Pipelines make the boring path the safe path.

- **Cost of a bad design:** a pipeline that runs 40-minute end-to-end tests on every push is ignored; one with no scan stage ships known-vulnerable images; one that rebuilds the artifact per environment ships something nobody tested.
- **Auditability:** the pipeline run is the record of *who changed what, what was tested, and what went where*.

3 Real-world system example

Amazon's automated deployment pipelines. The Amazon Builders' Library article *"Automating safe, hands-off deployments"* documents the design Amazon teams aim for: the pipeline starts from a commit to the source repository, builds and tests the change, deploys to pre-production environments, then to production in **waves** (a small slice first, typically one Region or cell, widening only if monitoring stays healthy), with a **bake time** between waves and **automatic rollback** when alarms fire. Humans are removed from the routine path; they design the pipeline and respond to alarms.

Netflix and Spinnaker. Netflix created and open-sourced the **Spinnaker** continuous delivery platform. Its pipelines model deployment as stages (bake an image, deploy a server group, run canary analysis, manual judgment, destroy the old group). Netflix and Google also open-sourced **Kayenta**, the automated canary analysis service used by Spinnaker. Both are good references for what a mature pipeline looks like: stages are explicit, the artifact is immutable, and a deployment only proceeds when measured health says so.

You do not need Amazon's scale to copy the *shape*: ordered stages, an immutable artifact, small blast radius first, automatic checks, automatic rollback.

4 Diagram

The pipeline we build in this level (and in the capstone Capstone 5), including where it stops early:

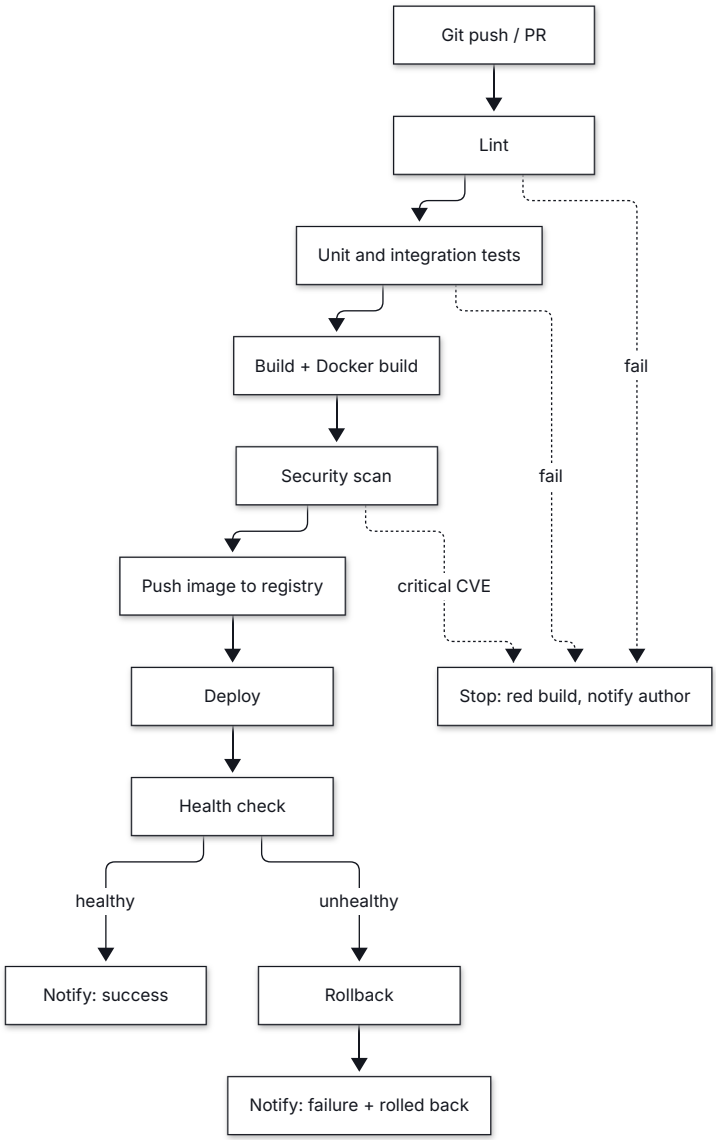


Figure 5.1-1

Order stages by **cost and likelihood of failure**: cheapest and most likely to fail first.

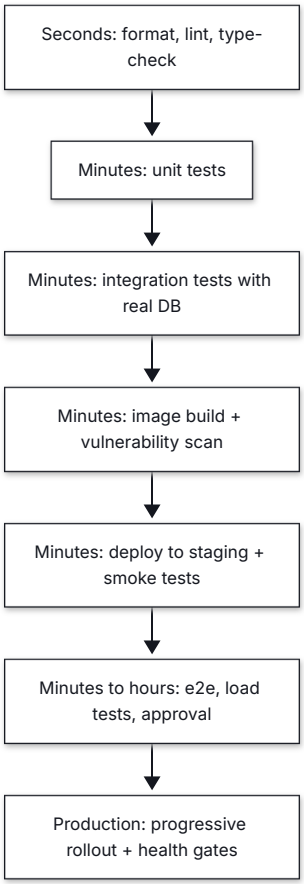


Figure 5.1-2

5 How it works

The stages and what each one proves

Stage	Question it answers	Typical tools	Fails when
Trigger	When does the pipeline run?	`push`, `pull_request`, tag, schedule, manual	n/a
Lint / static analysis	Is the code well-formed and consistent?	ESLint, ruff, hadolint, `terraform fmt`	style or obvious bug patterns
Test	Does it behave correctly?	Jest, pytest, JUnit, service containers	assertion fails

Stage	Question it answers	Typical tools	Fails when
Build	Can we produce a runnable artifact?	npm, Maven, Go, `docker build`	compile or packaging error
Scan	Is it safe to ship?	Trivy, Grype, Snyk, secret scanners	critical CVE, leaked secret
Push	Is the artifact stored immutably?	GHCR, ECR, Artifactory	auth or registry failure
Deploy	Is the new version running where we want?	SSH + Compose, `kubectl`, Helm, CodeDeploy	rollout does not complete
Health check	Does it actually work in that environment?	`curl /healthz`, smoke tests, metrics	wrong status, wrong version
Notify	Does the team know the outcome?	Slack, email, GitHub deployments API	n/a
Rollback	Can we undo it automatically?	previous image, traffic switch	health gate failed

Design principles

1. **Fail fast.** Put cheap, likely-to-fail checks first and run independent checks in parallel (lint and unit tests in parallel; build waits for both). Aim for a CI result in **under ~10 minutes**; this "ten-minute build" target is an old Extreme Programming guideline that still works as a rule of thumb.
2. **Build once, deploy many.** Create the artifact (an image identified by digest) one time, then *promote* that exact artifact through staging and production. Only configuration differs. See L5.6.
3. **Every stage is a gate.** A non-zero exit code stops the line. No "continue-on-error" without a written reason.
4. **Pipeline as code.** The definition lives in the repo (`.github/workflows/*.yml`), is reviewed in pull requests, and is versioned with the app.
5. **Idempotent and re-runnable.** Re-running a deploy of the same image changes nothing. This is what makes retry and rollback safe.
6. **Environments are not snowflakes.** Staging and production are created by the same automation; secrets and sizes differ. See L5.5.
7. **Deploy verification is part of the deploy.** "The command exited 0" is not "the app works". A health gate decides.
8. **Least privilege.** The pipeline gets short-lived, scoped credentials, not an admin key.
9. **Observable.** Log durations per stage; a pipeline that got 3x slower over a year is a bug.

Branch and environment strategy

Event	Runs	Deploys to
Pull request	lint, tests, image build + scan (no push)	nothing (or a preview environment)
Merge to `main`	everything + push image	staging automatically
Staging healthy	approval gate (optional)	production
Manual "rollback" run	deploy of an older image	the chosen environment

6 Step-by-step guide

Prerequisites: a free GitHub account and a repository (call it `shop-api`). Optional: Python 3 with PyYAML to validate files locally. Cost: none (public repos and the free tier include enough Actions minutes for this).

1. **Write the pipeline on paper first.** Create `docs/pipeline.md` in the repo with a table: stage, input, output, time budget, what happens on failure. Use the table in section 5 as a starting point. Decide the total budget (for example: CI 8 min, staging deploy 3 min).
2. **Create a skeleton workflow with every stage stubbed.** Save as `.github/workflows/pipeline-skeleton.yml`:

```yaml
name: pipeline-skeleton
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  lint:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - run: echo "lint ok"
  test:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - run: echo "tests ok"
  build:
    needs: [lint, test]
    runs-on: ubuntu-24.04
    steps:
```

```
      - run: echo "docker build ok"
  scan:
    needs: build
    runs-on: ubuntu-24.04
    steps:
      - run: echo "scan ok"
  push:
    needs: scan
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-24.04
    steps:
      - run: echo "push image"
  deploy:
    needs: push
    runs-on: ubuntu-24.04
    steps:
      - run: echo "deploy"
  health-check:
    needs: deploy
    runs-on: ubuntu-24.04
    steps:
      - run: echo "health check"
  notify:
    needs: [push, deploy, health-check]
    if: always() && github.ref == 'refs/heads/main'
    runs-on: ubuntu-24.04
    steps:
      - run: echo "deploy=${{ needs.deploy.result }} health=${{ needs.health-
  check.result }}"
```

`lint` and `test` have no dependency on each other, so they run **in parallel**. Everything after waits via `needs`.

3. **Validate the YAML locally (optional but quick).**

```bash
python3 -c 'import yaml,sys; yaml.safe_load(open(".github/workflows/pipeline-
  skeleton.yml")); print("valid YAML")'
```

Expected output: `valid YAML`. For GitHub-specific checks install `actionlint` (a static checker for workflows) and run it in the repo.

4. **Push and watch the graph.** Commit to a branch, open a pull request, then merge. In the **Actions** tab open the run: the graph shows `lint` and `test` side by side, then the chain. On the pull request the `push`, `deploy`, `health-check` jobs are **skipped** (`if` is false) — exactly the behaviour in the table above.
5. **Prove fail-fast.** Change the `lint` job to `run: exit 1`, push. Result: `lint` is red, and `build`, `scan`, `push`, `deploy` never start (shown as skipped). Revert.

6. **Measure.** Open the run summary and note each job's duration. Write the numbers in `docs/pipeline.md` as your baseline; in later topics you will make real stages replace the stubs and keep the total inside the budget.

7 Real-world scenario

(Illustrative.) A team of six at a small e-commerce company deploys `shop-api` by SSH-ing into a server and running `git pull && npm install && pm2 restart`. One Friday a developer deploys a change that works on their laptop but needs a new environment variable; the app boots and crashes in a loop. There is no health check and the previous `node_modules` were overwritten, so nobody can quickly return to the old version. It takes 90 minutes to recover.

The team designs a pipeline: PR checks (lint + tests) required to merge; on merge an image tagged with the commit SHA is built, scanned and pushed; a deploy job pulls *that exact image* on the server and then calls `/healthz`; if the response is not `ok` for the new version within two minutes, the job redeploys the previous image and posts to Slack. The next missing-variable mistake is caught by the health check, auto-rolled-back in under three minutes, and shows up as a red Slack message instead of a customer complaint. **Lesson:** the pipeline is not about speed alone; it is about turning "unknown unknowns at deploy time" into cheap, automatic, reversible failures.

8 Common mistakes and best practices

Don't

- Rebuild the artifact per environment ("it passed on staging" then means nothing for production).
- Put slow end-to-end tests *before* lint and unit tests.
- Let the pipeline pass when a stage is skipped or errors are ignored (`|| true` , `continue-on-error`).
- Deploy by tag name like `latest` — you cannot tell what is running or return to it.
- Give the pipeline a long-lived admin credential, or print secrets in logs.
- Treat a flaky pipeline as normal: people learn to click "re-run" and stop trusting red.

Do

- Keep a time budget per stage and alert when it is exceeded.
- Run independent jobs in parallel; use caching for dependencies and image layers.
- Make deploys idempotent and always verify with a health gate that checks the **version**, not just "200 OK".

- Require the pipeline to pass before merge (branch protection) — see protected branches §3.8.
- Notify on failure *and* on automatic rollback, and include a link to the run.
- Keep a documented, tested manual override (a "deploy this SHA" workflow) for when the automation is the problem.

9 Tasks

- [ ] **T1 — Run the skeleton pipeline** *(≈30 min, beginner)*
  - **Goal:** see a multi-stage pipeline with parallel and sequential jobs.
  - **Steps:**
    1. Create the repo and add `pipeline-skeleton.yml` from step 2.
    2. Open a pull request, then merge it.
    3. Compare the run graph for the PR and for the merge.
  - **Done when:** the PR run skips `push` / `deploy` / `health-check` and the `main` run executes all of them.
- [ ] **T2 — Prove fail-fast** *(≈20 min, beginner)*
  - **Goal:** show that a failed early stage prevents later stages.
  - **Steps:**
    1. Make `lint` exit with code 1.
    2. Push and open the run.
    3. Revert and confirm green.
  - **Done when:** you can point to `build` .. `deploy` marked skipped in the failing run and green in the fixed run.
- [ ] **T3 — Write the stage contract** *(≈40 min, intermediate)*
  - **Goal:** define what each stage consumes, produces and how long it may take.
  - **Steps:**
    1. Create `docs/pipeline.md` with a row per stage: input, output, time budget, failure action.
    2. Add the sequence number of the stage and whether it blocks merge.
    3. Add a "who is notified" column.
  - **Done when:** every stage in the diagram has a row and no row says "TBD".
- [ ] **T4 — Optimise order** *(≈45 min, intermediate)*

- **Goal:** reduce feedback time by re-ordering and parallelising.
- **Steps:**
  1. Add `sleep 60` to `test` and `sleep 20` to `lint`.
  2. Record total time with `lint` and `test` chained (`test` needs `lint`) versus parallel.
  3. Record time until the *first* failure if lint fails.
- **Done when:** you have two measured totals and can explain which layout gives the fastest failure signal.

☐ **T5 — Add a manual gate** *(≈30 min, intermediate)*

- **Goal:** model continuous delivery (human approval) for production.
- **Steps:**
  1. Create a GitHub **Environment** named `production` with a required reviewer (Settings → Environments).
  2. Add `environment: production` to a new `deploy-prod` job that `needs: health-check`.
  3. Merge a change and approve the deployment from the run page.
- **Done when:** the run pauses at `deploy-prod` showing "Waiting for approval" and continues after you approve.

☐ **T6 — Prove it: design a pipeline for a new service** *(≈45 min, advanced)*

- **Goal:** apply the principles to a service you have not seen.
- **Steps:**
  1. Without the guide, draw (Mermaid or paper) a pipeline for a Python API with a PostgreSQL migration and a container image.
  2. Mark the fail-fast order, where the artifact is created, where it is promoted, and the health gate.
  3. Explain to a teammate (or write 6 sentences) why each stage is where it is.
- **Done when:** your diagram has all ten stages from section 5, one artifact build, and a stated rollback path.

10 Self-check questions

1. What is the difference between continuous delivery and continuous deployment?
2. Why should lint run before integration tests, and why can lint and unit tests run in parallel?
3. What does "build once, deploy many" mean, and what goes wrong if you rebuild per environment?

4. Why is a health check that only tests for HTTP 200 insufficient after a deploy?
5. Name three things that make a pipeline slow and trusted less over time.

ANSWERS

1. Delivery keeps every passing change releasable but requires a human to release; deployment releases every passing change automatically.
2. Lint is seconds and catches many errors, so it saves minutes of expensive stages. Lint and unit tests do not depend on each other's output, so parallel execution shortens the critical path.
3. One immutable artifact (image digest) is created and then promoted through environments with only configuration changing. Rebuilding can pick up different dependencies or base-image content, so production runs something never tested.
4. The old version might still be answering 200 (deploy did not take effect), or the app can be up but broken (database unreachable). Check the reported version and a dependency-aware endpoint.
5. Serial stages that could be parallel, no dependency or layer caching, slow end-to-end tests on every push, flaky tests that train people to re-run, undersized runners, rebuilding things that did not change.

11 Further reading

- *Continuous Delivery* by Jez Humble and David Farley — the origin of the deployment pipeline pattern.
- Amazon Builders' Library: *Automating safe, hands-off deployments*.
- *Accelerate* by Nicole Forsgren, Jez Humble and Gene Kim — research linking delivery practices to performance (the DORA metrics).
- Spinnaker documentation (spinnaker.io) — a reference for pipeline stage concepts.
- GitHub Docs: *Understanding GitHub Actions* (documentation root: docs.github.com/actions).
- See also Deployment strategies in Level 0 §0.7 and GitHub Actions workflows in Level 3 §3.11.

LAB NOTES What I ran, what I saw, what it means

5.2 GitHub Actions for CI/CD

Topic 2 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

GitHub Actions is GitHub's built-in automation engine. A **workflow** is a YAML file in `.github/workflows/` that starts on an **event** (push, pull request, schedule, manual dispatch, another workflow), runs one or more **jobs** on **runners** (fresh virtual machines or containers), and each job is a sequence of **steps** (shell commands or reusable **actions**). If you have not used it yet, read GitHub Actions: Workflows, Jobs and Triggers §3.11 first.

This topic is about the features that turn a single workflow file into a maintainable **CI/CD system**:

- **Reusable workflows** — a whole workflow that other workflows call like a function.
- **Composite actions** — a bundle of steps packaged as one step.
- **Matrix builds** — one job definition fanned out over versions/platforms.
- **Caching** — reusing downloaded dependencies and build layers between runs.
- **Concurrency** — controlling what happens when two runs of the same thing overlap.

2 Why it matters in DevOps

- **Copy-paste drift:** a team with 30 repositories and 30 slightly different `deploy.yml` files has 30 places to fix a security problem. Reusable workflows make it one.
- **Wasted minutes (money):** installing dependencies from scratch on every run is the most common cause of slow, expensive pipelines. Caching and cancelling superseded runs commonly cut runtime and billed minutes substantially.
- **Race conditions in production:** two merges a minute apart can start two overlapping deploys. Without `concurrency`, the older one may finish *last* and leave production on the older version.
- **Compatibility:** a library that supports Node 20 and 22 must be tested on both; a matrix does it without duplicating YAML.

3 Real-world system example

Homebrew. The Homebrew package manager's CI runs on GitHub Actions: every pull request to its core formulae repository triggers builds and tests across several macOS versions and Linux — a natural matrix — and built packages ("bottles") are produced by workflows rather than by hand. Large open-source projects commonly run the same job across many OS and language-version combinations, which is exactly the matrix feature.

GitHub itself publishes the runner images. The virtual machine images used by GitHub-hosted runners (`ubuntu-24.04`, `windows-2022`, `macos-14` ...) are built from a public repository, `actions/runner-images`, which also lists the preinstalled software for each image. That is why you can write `runs-on: ubuntu-24.04` and rely on `docker`, `git`, `jq` and `curl` being present.

Many organisations keep one shared repository (often called something like `.github` or `workflows`) containing the company-standard reusable workflows (`build-image.yml`, `deploy.yml`) that every service calls with a few inputs.

4 Diagram

How a caller, a reusable workflow and a composite action nest:

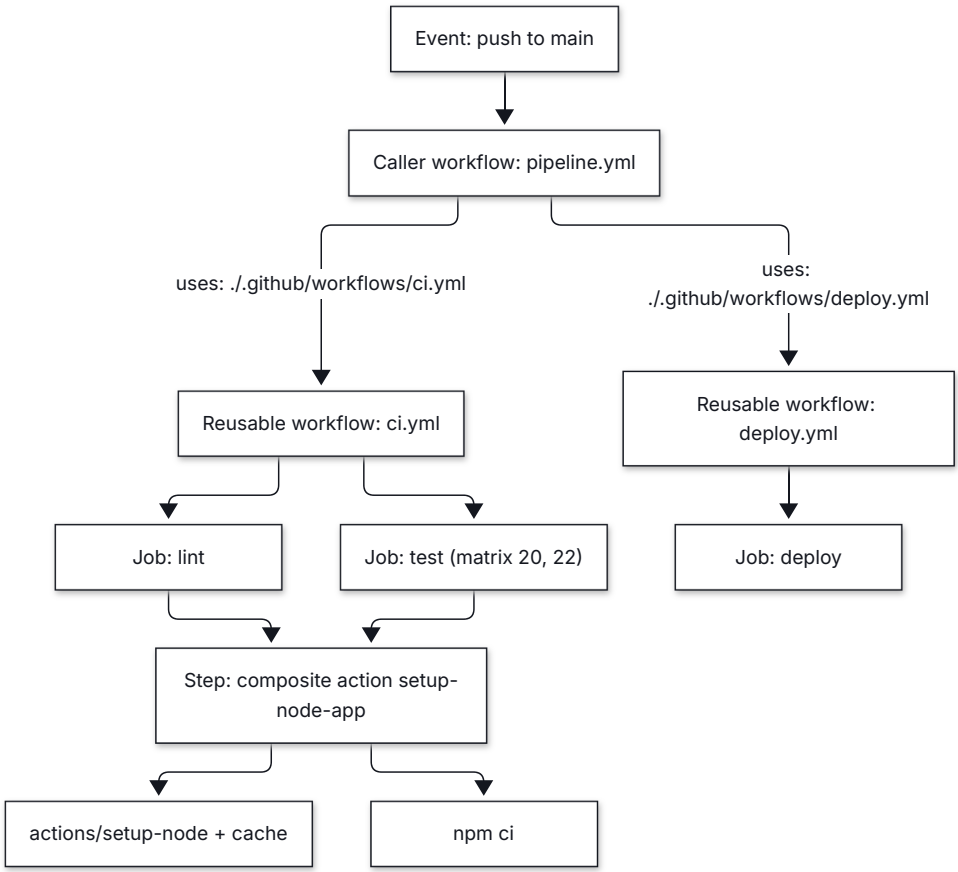


Figure 5.2-1

What `concurrency` does to overlapping runs:

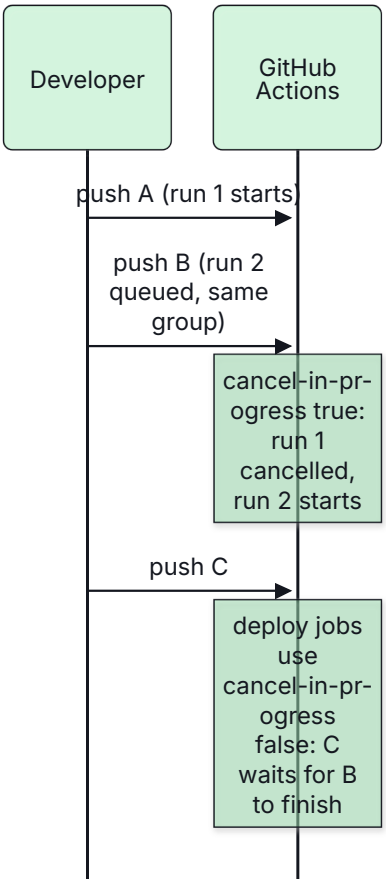


Figure 5.2-2

5 How it works

Reusable workflows (`workflow_call`)

A reusable workflow declares inputs, secrets and outputs under `on: workflow_call`. A caller uses it at the **job** level:

YAML

```yaml
# .github/workflows/ci.yml  (the reusable one)
on:
  workflow_call:
    inputs:
      node-version:
        type: string
        default: "22"
    outputs:
      coverage:
        value: ${{ jobs.test.outputs.coverage }}
    secrets:
      NPM_TOKEN:
        required: false
```

YAML

```yaml
# caller
jobs:
  ci:
    uses: ./.github/workflows/ci.yml          # same repo
    with:
      node-version: "22"
    secrets: inherit                           # pass the caller's secrets down
  other:
    uses: my-org/shared-workflows/.github/workflows/build.yml@v1   # another repo, pinned
```

Rules worth memorising: a reusable workflow **cannot be a step** (only a job); the caller job can have `needs`, `if`, `with`, `secrets`, `permissions` (the callee can only keep or reduce them) but not `runs-on` or `steps`; the called workflow's `github` context is the *caller's*; nesting depth is limited, so check the docs rather than building deep chains.

Composite actions

A composite action packages **steps** (not jobs). It lives in a folder with an `action.yml`:

	Reusable workflow	Composite action
Unit of reuse	whole job(s)	a group of steps inside one job
Runs on	its own runner(s)	the caller job's runner

	Reusable workflow	Composite action
Can use `secrets:`	yes (declared)	no, pass as inputs
Called with	`jobs.<id>.uses`	`steps[*].uses`
Typical use	"build and test", "deploy to env"	"set up Node, restore cache, install"

Matrix

```yaml
strategy:
  fail-fast: false            # let all combinations finish; default is true
  max-parallel: 4
  matrix:
    node: [20, 22]
    os: [ubuntu-24.04, ubuntu-22.04]
    include:
      - node: 24
        os: ubuntu-24.04
    exclude:
      - node: 20
        os: ubuntu-22.04
```

Use `${{ matrix.node }}` in steps. Each combination is its own job (and its own billed run). In branch protection, require the matrix jobs' names, or add a final "gate" job with `needs` on all of them.

Caching

Mechanism	What it caches	Key idea
`actions/setup-node` with `cache: npm`	npm download cache	keyed on `package-lock.json` hash
`actions/cache@v4`	any path	`key` (exact) + `restore-keys` (prefix fallback)
Docker `cache-from/to: type=gha`	BuildKit layers	see L5.4

```yaml
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ runner.os }}-${{ hashFiles('requirements*.txt') }}
    restore-keys: |
      pip-${{ runner.os }}-
```

Facts to know: a cache is **immutable** once saved for a key (change the key to refresh it); caches are scoped by branch (a run can restore caches from its own branch and from the default branch, not from sibling branches); each repository has a size limit (10 GB at the time of writing) and unused entries are evicted (after 7 days without access). A cache is an *optimisation*: your build must succeed on a cache miss. Never cache secrets or build outputs you intend to ship — use artifacts §5.6 for that.

Concurrency

```yaml
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true        # for CI: only the newest commit matters
```

```yaml
concurrency:
  group: deploy-production
  cancel-in-progress: false       # for deploys: queue, never kill a half-finished deploy
```

At most one run **per group** is running and (at most) one is pending; a newer pending run replaces an older pending one. **Pitfall:** if a caller and the workflow it calls declare the *same* group, the callee can cancel itself or its caller. Put the group on the caller, or use distinct group names.

Other things you will use constantly

- `permissions:` — set to `contents: read` by default at the top, widen per job (least privilege).
- `needs`, `if`, `outputs` — wire jobs together; `if: always()` runs notification jobs even after failures.
- `environment:` — links a job to an Environment with secrets, variables and approvals (Environments §3.12).
- `timeout-minutes:` — always set it; the default is 360 minutes.
- Pin third-party actions to a **full commit SHA** (or at least a major tag you trust); actions run with your token.

6 Step-by-step guide

Prerequisites: a GitHub repository with a small Node.js project (`package.json` with a `test` script; `npm init -y` and `"test": "node --test"` is enough). Cost: free for public repos.

1. **Create a composite action.** Save as `.github/actions/setup-app/action.yml`:

```yaml
name: Setup app
description: Install Node.js with npm cache and install dependencies
inputs:
  node-version:
    description: Node.js version
    default: "22"
runs:
  using: composite
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: npm
    - name: Install
      shell: bash
      run: npm ci
```

   Every `run` step in a composite action **must** declare `shell`.

2. **Create a reusable workflow with a matrix.** Save as `.github/workflows/test.yml`:

```yaml
name: test
on:
  workflow_call:
    inputs:
      versions:
        type: string
        default: '["20","22"]'

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    strategy:
      fail-fast: false
      matrix:
        node: ${{ fromJSON(inputs.versions) }}
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-app
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
```

`fromJSON` turns a JSON string input into a list — the standard trick for a dynamic matrix.

3. **Create the caller with concurrency.** Save as `.github/workflows/ci.yml`:

```yaml
name: ci
on:
  pull_request:
  push:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  test:
    uses: ./.github/workflows/test.yml
    with:
      versions: '["20","22"]'
```

4. **Run it and read the logs.** Push a branch, open a PR. In **Actions** you see two jobs: `test / test (20)` and `test / test (22)`. On the first run the `Setup Node` step ends with `Cache not found`.
5. **Prove caching works.** Re-run the workflow (Re-run all jobs). The log of `Setup Node` should now say `Cache restored from key: node-cache-Linux-x64-npm-<hash>` and `npm ci` becomes noticeably faster.
6. **Prove concurrency works.** Add `sleep 60` to the test step, push, then push again within 30 seconds. The first run shows **Cancelled**; the second runs to completion.
7. **Validate locally** (optional). Run `python3 -c 'import yaml;[yaml.safe_load(open(f)) for f in ["a.yml"]]'` for syntax or install `actionlint` for GitHub-specific errors like a missing `shell:` or a bad `needs` reference.

7 Real-world scenario

(Illustrative.) A platform team owns 24 microservices. Each has its own copy of a 120-line `deploy.yml`. A routine audit shows 9 of them still use a deprecated action version, 5 deploy with an over-broad token, and 3 can deploy two builds at once. Fixing it means 24 pull requests.

The team writes one reusable `deploy.yml` in a shared repository with inputs (`environment`, `image`, `version`), sets `permissions` explicitly, adds a `concurrency` group per environment with `cancel-in-progress: false`, and publishes it as a tagged release `v1`. Each service shrinks to a 15-line caller. The next security fix is one commit and a retag; services pick it up on their next run (or via a Dependabot bump of the `@v1` ref). **Lesson:** reuse turns CI/CD from per-repo craft into a maintained product.

8 Common mistakes and best practices

Don't

- Give workflows `permissions: write-all` or leave the default broad token scope.
- Use the same `concurrency.group` in a caller and the workflow it calls.
- Use `cancel-in-progress: true` on production deploys.
- Rely on a cache for correctness (a miss must still work), or put secrets in a cached path.
- Reference unpinned third-party actions on `@main`.
- Forget `shell:` in composite action `run` steps, or try to use `secrets` directly inside a composite action.

Do

- Put shared logic in reusable workflows (jobs) and shared step groups in composite actions, and version them (`@v1`).
- Use `fail-fast: false` for compatibility matrices where you want the full picture.
- Add `timeout-minutes` to every job and `if: always()` for notification jobs.
- Use a final "gate" job so branch protection needs only one required check.
- Prefer `${{ hashFiles('**/package-lock.json') }}`-style keys so the cache invalidates when dependencies change.

9 Tasks

- [ ] **T1 — Build and call a composite action** *(≈30 min, beginner)*
  - **Goal:** package repeated setup steps.
  - **Steps:**
    1. Create `.github/actions/setup-app/action.yml` as in step 1.
    2. Use it in a workflow with `uses: ./.github/actions/setup-app` after `actions/checkout`.

- **Done when:** the run shows the composite action expanded into its inner steps and `npm ci` succeeded.

- [ ] **T2 — Matrix across versions** *(≈30 min, beginner)*
  - **Goal:** test on Node 20 and 22 from one job definition.
  - **Steps:**
    1. Add the `strategy.matrix` block from step 2.
    2. Add a third version and an `exclude` entry for one combination.
  - **Done when:** the Actions UI lists exactly the combinations you intended, each with its own green check.

- [ ] **T3 — Prove the cache** *(≈30 min, intermediate)*
  - **Goal:** measure caching.
  - **Steps:**
    1. Run once on a cold cache and note the `npm ci` step time.
    2. Re-run and note the time again.
    3. Change a dependency in `package-lock.json` and run a third time.
  - **Done when:** you have three timings and a log line proving a cache hit on run 2 and a miss (or `restore-keys` fallback) on run 3.

- [ ] **T4 — Concurrency: cancel and queue** *(≈40 min, intermediate)*
  - **Goal:** use both concurrency modes.
  - **Steps:**
    1. Add `cancel-in-progress: true` to CI and verify with two quick pushes.
    2. Add a dummy `deploy` job with `concurrency: { group: deploy, cancel-in-progress: false }` and `sleep 45`.
    3. Trigger two deploys in a row.
  - **Done when:** CI shows a cancelled run; the second deploy shows `Pending` until the first completes.

- [ ] **T5 — Break and fix a reusable workflow** *(≈40 min, intermediate)*
  - **Goal:** learn the sharp edges.
  - **Steps:**
    1. In the reusable workflow, reference `secrets.MY_SECRET` without declaring it under `workflow_call.secrets` and call it without `secrets: inherit`.
    2. Observe that the value is empty; fix by declaring and passing it (or use `secrets: inherit`).

3. Set the same `concurrency.group` in caller and callee and watch the problem; rename one.

- **Done when:** you can explain each failure from the run logs and both are fixed.

☐ **T6 — Prove it: shared workflow library** *(≈60 min, advanced)*

- **Goal:** publish a versioned reusable workflow and consume it from a second repository.
- **Steps:**
  1. Without the guide, create repo `workflows` with `.github/workflows/test.yml` (`workflow_call`), tag it `v1`.
  2. In `shop-api`, call it with `uses: <you>/workflows/.github/workflows/test.yml@v1`.
  3. Explain in writing the difference between this and a composite action and when you would use each.
- **Done when:** the `shop-api` run executes the shared workflow from the other repository at ref `v1`.
- *(The shared repository must be public, or its Actions access setting must allow other repos to use it.)*

10 Self-check questions

1. When would you choose a reusable workflow over a composite action?
2. What does `cancel-in-progress: true` do, and why is it wrong for deployments?
3. A cache key is `npm-${{ hashFiles('package-lock.json') }}`. When is a new cache created?
4. Why can a reusable workflow not use `runs-on` set by the caller?
5. What is the risk of `uses: someone/action@main`?

ANSWERS

1. When you want to reuse whole jobs (own runner, own secrets/permissions handling, outputs), e.g. "deploy to environment". Composite actions reuse a handful of steps inside a job you already have.
2. It cancels the running run of the same group when a newer one starts. Killing a deploy halfway can leave the system in a mixed state; use queueing (`false`) for deploys.
3. When the hash of the lockfile changes (dependencies changed). With an unchanged lockfile the key matches and the existing cache is restored.

4. The reusable workflow defines its own jobs, and each job declares its own `runs-on`; the caller only invokes the workflow (it can pass the runner label as an input if the author supports it).
5. The ref is mutable; an upstream change (or compromise) runs inside your workflow with your token. Pin a commit SHA or a trusted release tag.

11 Further reading

- GitHub Docs: *Reusing workflows*, *Creating a composite action*, *Running variations of jobs in a workflow* (matrix), *Dependency caching reference*, *Control the concurrency of workflows and jobs* — all under docs.github.com/actions.
- `actions/runner-images` repository — the software installed on GitHub-hosted runners.
- `actionlint` project (static checker for workflow files).
- Next: Automated testing in pipelines §5.3 and Docker builds in CI §5.4.

LAB NOTES What I ran, what I saw, what it means

LEVEL 5: CI/CD

5.3 Automated Testing in Pipelines

Topic 3 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

Automated testing in a pipeline means that every change is checked by machines, on a clean runner, before a human has to trust it. Tests come in layers that trade **speed** against **realism**:

- **Unit tests** exercise one function or class in isolation, with no network or database. Milliseconds each.
- **Integration tests** exercise several components together with *real* dependencies — your API talking to a real PostgreSQL and Redis. Seconds each.
- **End-to-end (e2e) tests** drive the whole running system the way a user or client would (a browser, or HTTP calls against a deployed environment). Seconds to minutes each.

The **test pyramid** is the rule of thumb for how many of each you should have: many fast unit tests at the base, fewer integration tests in the middle, a small number of e2e tests at the top. A **service container** is how a CI job gets real dependencies cheaply: the platform starts a throwaway PostgreSQL/Redis container next to your job. A **flaky test** is one that sometimes passes and sometimes fails on the same code — the single biggest threat to a team's trust in CI.

2 Why it matters in DevOps

- **Tests are the pipeline's first gate:** without them, "deploy automatically" just means "break production automatically".
- **Speed:** an upside-down pyramid (mostly slow e2e tests) makes the pipeline take an hour and developers stop running it.
- **Environment parity:** unit tests with mocks pass while the real SQL query is wrong; integration tests against real PostgreSQL catch it before staging does.
- **Trust:** if 1 in 20 runs fails randomly, engineers re-run until green and real failures slip through ("cry wolf").

- **Cost:** a failing test found in CI costs minutes; the same bug in production costs an incident.

3 Real-world system example

Google. In *Software Engineering at Google* and on the Google Testing Blog, Google describes classifying tests by size (small, medium, large) and recommends a distribution that is heavily weighted to small tests — roughly 80% small, 15% medium and 5% large is quoted as a guideline, not a law. Google's blog post *"Flaky Tests at Google and How We Mitigate Them"* is a public account of flakiness at scale: it reports that a small percentage of test runs were flaky and that a considerably larger share of individual tests showed some flakiness at some point, and describes mitigations such as re-running failing tests, tracking flakiness per test, and quarantining unreliable tests so they do not block everybody.

GitHub Actions service containers. GitHub documents service containers (PostgreSQL and Redis examples are in its own docs) precisely so projects can run integration tests against real databases in CI without maintaining shared test servers.

4 Diagram

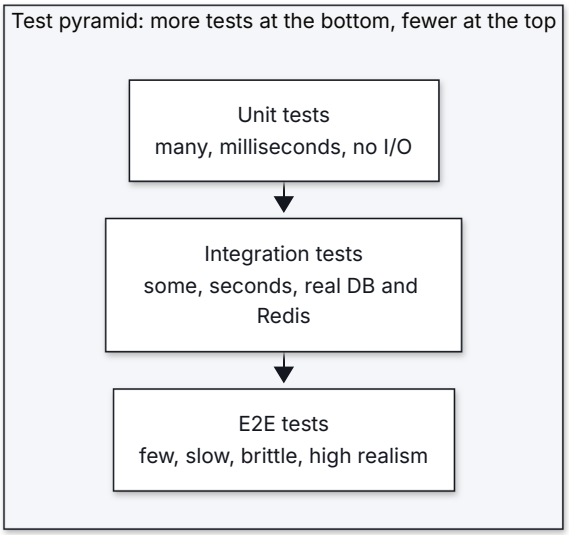


Figure 5.3-1

How an integration test job runs on a CI runner:

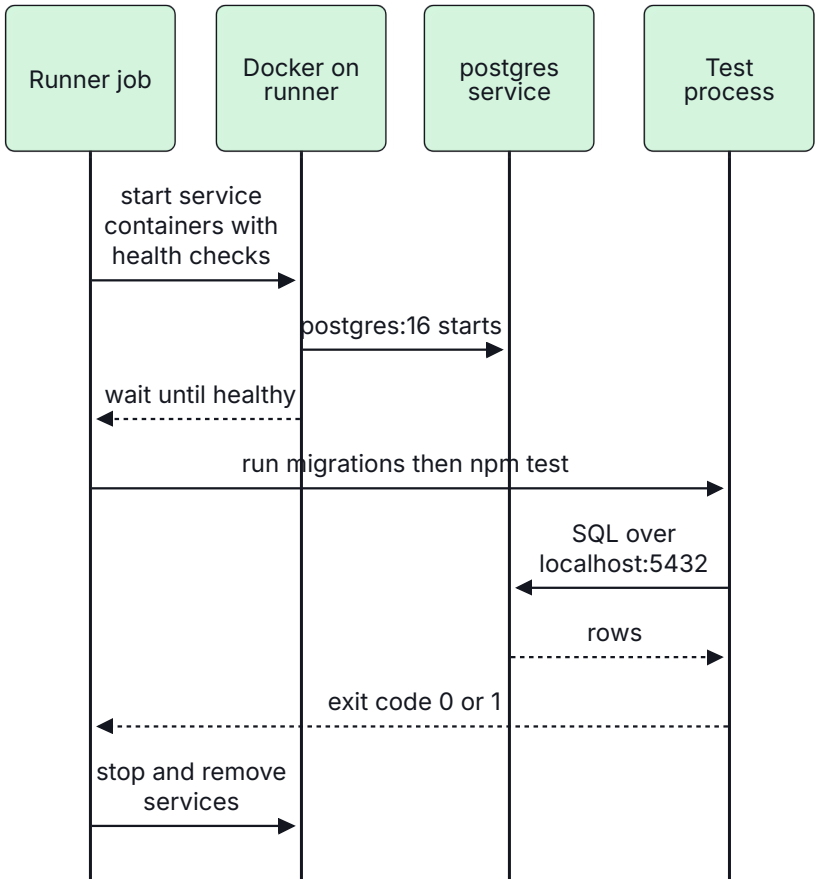


Figure 5.3-2

5 How it works

What belongs where

Layer	Scope	Dependencies	Speed target	Runs
Unit	one function or class	none (pure) or fakes	< 1 ms to 50 ms each; whole suite under 1-2 min	every push and PR
Integration	API + DB, API + cache, module boundaries	real PostgreSQL/Redis via service containers	whole suite a few minutes	every push and PR

Layer	Scope	Dependencies	Speed target	Runs
Contract	API shape between two services	stub or recorded provider	seconds	PR
E2E / smoke	the deployed system	a real environment	minutes	after deploy (staging), small set
Non-functional	load, security (DAST), accessibility	environment	long	nightly or before release

Sensible rules:

1. **Test the logic at the lowest layer that can see the bug.** A pricing rule is a unit test, not a browser test.
2. **Use real dependencies for integration, not mocks of the database.** Mocking the thing you most need to verify (SQL, migrations) proves nothing.
3. **Keep e2e small** — a handful of critical user journeys, plus a post-deploy **smoke test** (hit `/healthz`, fetch one resource).
4. **Make tests hermetic:** each test creates its own data, uses random ports, and does not depend on order or wall-clock time.
5. **Fail on coverage regressions sparingly:** a coverage gate (for example "lines >= 80%") prevents rot, but 100% coverage is a vanity target.

Service containers

In GitHub Actions, `services:` starts containers on the runner network before the steps. Health options make the job wait until the service is ready:

YAML

```yaml
services:
  postgres:
    image: postgres:16
    env:
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: shop_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd "pg_isready -U postgres"
      --health-interval 5s
      --health-timeout 5s
      --health-retries 10
```

When the job runs directly on the runner VM (the default), reach the service at `localhost:<mapped port>`. When the job itself runs in a container (`container:`), reach it by service name (`postgres:5432`). Service containers exist for the life of the job and are destroyed with it, so every run starts with an empty database.

Flaky tests: causes and handling

Typical cause	Fix
Timing: `sleep(1)` then assert	wait for a condition with a timeout, not a fixed delay
Shared state / order dependence	isolate data per test; run in random order to expose it
External network calls	stub them, or use a local fake
Time and randomness	inject a clock and seed the RNG
Resource limits on CI	right-size runners; avoid port collisions with port 0
Race in the app itself	a real bug, the test is the messenger — fix the code

Policy that works: (1) **detect** flakiness by re-running a suspect test many times and by tracking pass rate per test; (2) **quarantine** a flaky test quickly (move it to a non-blocking job, open an issue with an owner and a deadline); (3) **fix or delete**; (4) allow *limited* automatic retries only as a visible, measured stopgap — silent retries hide bugs.

Making test output useful

- Emit a machine-readable report (JUnit XML, TAP) and upload it as an artifact §5.6 so failures can be inspected later.
- Print only what is needed; show the failing assertion and the diff.
- Shard long suites across parallel jobs (a matrix of `shard: [1,2,3,4]`) once a single job exceeds your time budget.

6 Step-by-step guide

Prerequisites: Node.js 22 (`node -v`), a GitHub repo. For the optional local part: Docker. Cost: free.

1. **Write a unit test.** Node has a built-in test runner, so no dependencies are needed.

```bash
mkdir -p src test
cat > src/price.js <<'EOF'
export function totalCents(items, taxRate = 0.2) {
  const subtotal = items.reduce((sum, i) => sum + i.priceCents * i.qty, 0);
  return Math.round(subtotal * (1 + taxRate));
```

```
}
EOF
cat > test/price.test.js <<'EOF'
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { totalCents } from '../src/price.js';

test('adds 20 percent tax', () => {
  assert.equal(totalCents([{ priceCents: 1000, qty: 2 }]), 2400);
});

test('empty cart is free', () => {
  assert.equal(totalCents([]), 0);
});
EOF
echo '{"type":"module","scripts":{"test":"node --test"}}' > package.json
npm test
```

Expected (similar to): two lines `ok 1 - adds 20 percent tax` / `ok 2 - empty cart is free` and `# pass 2`, `# fail 0`.

2. **Add coverage and a gate.** Recent Node 22 releases include a coverage report and thresholds:

```bash
node --test --experimental-test-coverage --test-coverage-lines=90
echo "exit code: $?"
```

A table with `price.js` at 100% appears and the exit code is `0`. Set `--test-coverage-lines=101` to see the gate fail with exit code `1`.

3. **Write an integration test that needs PostgreSQL.** Run a local database first:

```bash
docker run -d --name pg-test -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shop_test -p
  5432:5432 postgres:16
npm install pg
cat > test/db.test.js <<'EOF'
import { test, after } from 'node:test';
import assert from 'node:assert/strict';
import pg from 'pg';

const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL ??
  'postgres://postgres:postgres@localhost:5432/shop_test',
});
after(() => pool.end());

test('can create and read an order', async () => {
```

```
  await pool.query('CREATE TEMP TABLE orders (id serial PRIMARY KEY, total_cents int
  NOT NULL)');
  await pool.query('INSERT INTO orders (total_cents) VALUES ($1)', [2400]);
  const { rows } = await pool.query('SELECT total_cents FROM orders');
  assert.deepEqual(rows, [{ total_cents: 2400 }]);
});
EOF
npm test
```

Expected: three passing tests. Stop the database afterwards: `docker rm -f pg-test`.

4. **Run the same tests in CI with service containers.** Save as `.github/workflows/test.yml`:

```yaml
name: test
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: shop_test
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
      redis:
        image: redis:7
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10
    env:
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/shop_test
      REDIS_URL: redis://localhost:6379
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - name: Run tests with JUnit report
        run: node --test --test-reporter=spec --test-reporter-destination=stdout --
  test-reporter=junit --test-reporter-destination=junit.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: junit-report
          path: junit.xml
          retention-days: 7
```

In the run, expand **Initialize containers**: you see the PostgreSQL image pulled and a health status of `healthy` before the first step. The `junit-report` artifact appears on the run summary page. The password is a throwaway value for a database that only exists for this job.

5. **Add a post-deploy smoke test (the thin e2e layer).** After a deploy, a tiny script proves the deployed system answers:

```bash
cat > scripts/smoke.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
BASE_URL="${1:?usage: smoke.sh https://staging.example.com}"
curl -fsS --retry 5 --retry-delay 3 --connrefused "$BASE_URL/healthz" | jq -e
  '.status == "ok"'
curl -fsS "$BASE_URL/api/products" | jq -e 'type == "array"'
echo "smoke ok"
EOF
chmod +x scripts/smoke.sh
```

Run it later against a real environment, for example `scripts/smoke.sh http://localhost:3000`; output ends with `smoke ok`.

6. **Detect a flaky test.** Run the suite repeatedly and stop at the first failure:

```bash
for i in $(seq 1 30); do npm test --silent >/dev/null || { echo "FAILED on run $i";
  break; }; done; echo done
```

A deterministic suite prints just `done`. Add a deliberately flaky test (`assert.ok(Math.random() > 0.2)`) and re-run to see how quickly repetition exposes it.

7 Real-world scenario

(Illustrative.) The `shop-api` pipeline is red about one run in six on the `checkout` integration test. Engineers have started pressing "Re-run failed jobs" without reading the error. One day a real regression in the discount calculation is merged because "it was probably the flaky one", and customers are over-charged for four hours.

Post-incident the team measures: the same test passes or fails on identical commits. The test inserts an order and then `sleep(200)` before asserting that a Redis-backed cache has been invalidated; on a busy runner 200 ms is sometimes not enough. They replace the sleep with a poll-until-true helper (with a 5 s timeout), add a `shard` matrix to bring the suite under 6 minutes, and add a rule: any test failing twice in a week without a code change is quarantined within a day and assigned an owner. **Lesson:** a flaky test is not an annoyance, it is a security hole in your safety net.

8 Common mistakes and best practices

Don't

- Mock the database in the tests that exist to verify database behaviour.
- Share one long-lived test database between runs (state leaks, parallel runs collide).
- Use `sleep` to wait for asynchronous work.
- Normalise "just re-run it", or add silent automatic retries across the whole suite.
- Build an ice-cream cone: a huge e2e suite on top of a few unit tests.
- Put real credentials in test configuration; use disposable values for disposable services.

Do

- Run lint and unit tests first, then integration, then (post-deploy) smoke tests — fail fast.
- Use service containers with health checks so tests never start against a half-booted database.
- Upload test reports and coverage as artifacts; keep a flaky-test list with owners.
- Make each test create and clean its own data (transactions rolled back, temp tables, unique IDs).
- Keep the PR test run under ~10 minutes; shard or parallelise when it grows beyond that.

9 Tasks

- [ ] **T1 — Unit tests, locally** *(≈30 min, beginner)*
  - **Goal:** write and run unit tests with the built-in runner.

- **Steps:**
  1. Create `src/price.js` and `test/price.test.js` as in step 1.
  2. Add a test for a 0% tax rate and one for a quantity of 0.
  3. Run `npm test`.
- **Done when:** `npm test` reports your tests passing and exits with code 0.

☐ **T2 — Integration test with a real database** *(≈40 min, intermediate)*

- **Goal:** test code against real PostgreSQL.
- **Steps:**
  1. Start PostgreSQL in Docker (step 3).
  2. Write a test that creates a `products` table, inserts two rows and reads them back sorted by price.
  3. Stop the container and confirm the test now fails with a connection error.
- **Done when:** the test passes with the database up and fails clearly with it down.

☐ **T3 — Move it into CI with service containers** *(≈40 min, intermediate)*

- **Goal:** the same integration test runs on GitHub Actions.
- **Steps:**
  1. Add the workflow from step 4.
  2. Push a PR and open **Initialize containers** in the logs.
  3. Download the `junit-report` artifact.
- **Done when:** the run is green, shows both services healthy, and the artifact contains your test names.

☐ **T4 — Hunt a flaky test** *(≈40 min, intermediate)*

- **Goal:** detect, quarantine and fix flakiness.
- **Steps:**
  1. Add a test with `await new Promise(r ⇒ setTimeout(r, 10))` followed by an assertion on something that takes a random 0-30 ms.
  2. Run the loop from step 6 until it fails.
  3. Replace the sleep with a poll-until-true helper that has a timeout; re-run 30 times.
- **Done when:** the flaky version fails within 30 runs and the fixed version passes 30 of 30.

☐ **T5 — Add a coverage gate and break it** *(≈30 min, intermediate)*

- **Goal:** enforce a minimum coverage in CI.

- **Steps:**
  1. Add `--experimental-test-coverage --test-coverage-lines=80` to the CI test command.
  2. Add an untested function with several branches and push.
  3. Observe the failure, then add tests to fix it.
- **Done when:** CI is red because of the coverage threshold, then green after you add tests.

☐ **T6 — Prove it: choose the layer** *(≈30 min, advanced)*

- **Goal:** justify where each test lives.
- **Steps:**
  1. Without the guide, list five behaviours of `shop-api` (tax calculation, order insert, `/healthz` output, Redis cache expiry, checkout journey in a browser).
  2. Assign each to unit, integration, or e2e/smoke and give the reason and expected runtime.
  3. Explain to a teammate what your pipeline does when the checkout journey test fails twice in a week with no code change.
- **Done when:** you have a written table of five rows with reasoning and a stated quarantine policy.

10 Self-check questions

1. Why is a pyramid better than an "ice-cream cone" test suite?
2. Why use a real PostgreSQL service container instead of mocking the database client in integration tests?
3. How do you reach a service container from a step when the job runs directly on the runner? And when the job runs inside a container?
4. Give two common causes of flaky tests and the fix for each.
5. What is the purpose of a post-deploy smoke test, given that tests already passed in CI?

ANSWERS

1. Unit tests are fast, cheap and precise; e2e tests are slow, brittle and expensive. A pyramid gives quick, targeted feedback; a cone gives slow, vague feedback and flakiness.
2. A mock only proves the code calls the client as expected, not that the SQL, constraints, migrations and types work. A real database verifies behaviour.
3. Via `localhost:<mapped host port>` when on the runner; via the service name and container port (for example `postgres:5432`) when the job runs in a container.

4. Fixed sleeps → wait for a condition with a timeout. Shared state between tests → isolate data per test and randomise order. (Also: network calls → stub them; clock/randomness → inject them.)
5. CI tests the code in a test environment; the smoke test verifies the *deployed* system (configuration, secrets, networking, real dependencies) actually works, and it provides the signal for rollback.

11 Further reading

- *Software Engineering at Google* (Winters, Manshreck, Wright), chapters on testing — test sizes and the pyramid.
- Google Testing Blog: *Flaky Tests at Google and How We Mitigate Them*.
- Martin Fowler: *TestPyramid* and *Eradicating Non-Determinism in Tests* (martinfowler.com).
- GitHub Docs: *About service containers* (docs.github.com/actions).
- Node.js documentation: the `node:test` module and test runner (nodejs.org/api).
- See also Docker image builds in CI §5.4 and Dynamic testing with OWASP ZAP §11.13.

LEVEL 5: CI/CD

5.4 Docker Image Builds and Registries in CI

Topic 4 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

In CI, a Docker image is the **deployable artifact**: the build stage turns source code into an image, gives it a unique name, and stores it in a **registry** (a server that stores and serves images) so that every later stage and every environment pulls the *same bytes*. If you need a refresher on images, layers and registries, read Images and Layers §4.2 and Registries §4.4 first.

Three ideas make this work well in a pipeline:

- **BuildKit / `buildx`** — `docker buildx build` is the modern builder front-end (BuildKit under the hood). It builds layers in parallel, supports **cache import/export** to places that survive a throw-away CI runner, and can build **multi-platform** images (for example `linux/amd64` and `linux/arm64`) in one command.
- **Layer cache in CI** — a CI runner starts empty every time, so without an external cache each build re-downloads and re-installs everything. You export the cache to GitHub's cache service or to the registry and import it on the next run.
- **Tagging strategy** — a tag is a movable label on an image. The tags you create decide whether you can answer "what exactly is running in production, and can I get back to the previous one?"

2 Why it matters in DevOps

- **Reproducibility and rollback:** if production runs `shop-api:latest`, you cannot say which commit it is, and "roll back" has no target. If it runs `shop-api:sha-3f2a9c1...`, the answer is exact and the previous version is still in the registry.
- **Speed and cost:** an uncached Node.js or Java image build can take many minutes of billed runner time; a good cache turns it into seconds when only application code changed.
- **Supply chain:** the registry is where an attacker would love to slip in a modified image. Immutable tags, short-lived push credentials, scanning before pushing, and provenance attestations all live in this stage.

- **Cost of registries:** untagged and stale images accumulate and are billed as storage; lifecycle rules are part of the design.

3 Real-world system example

GitHub Container Registry (GHCR) and Docker's official build actions. Docker maintains the GitHub Actions used in most public pipelines: `docker/setup-buildx-action`, `docker/login-action`, `docker/metadata-action` (computes tags and labels from the Git event) and `docker/build-push-action`. The Docker documentation describes the `gha` cache backend, which stores BuildKit layer cache in the GitHub Actions cache service, and a `registry` backend, which stores it as an image in your registry. GitHub's own registry, `ghcr.io`, accepts pushes authenticated with the workflow's built-in `GITHUB_TOKEN` when the job is granted `packages: write`, so a public project needs no stored registry password at all.

Amazon ECR. On AWS, teams push to Amazon Elastic Container Registry. ECR supports **immutable tags** (a repository setting that rejects pushing a tag that already exists), **lifecycle policies** that expire old images, and **image scanning**. In CI the recommended pattern is for GitHub Actions to assume an IAM role through OpenID Connect (OIDC) instead of storing AWS keys; see L5.5.

Kubernetes-ecosystem projects also publish their official images as **multi-architecture** images, so one tag works on both Intel/AMD and Arm nodes. That is the same `--platform linux/amd64,linux/arm64` feature shown below.

4 Diagram

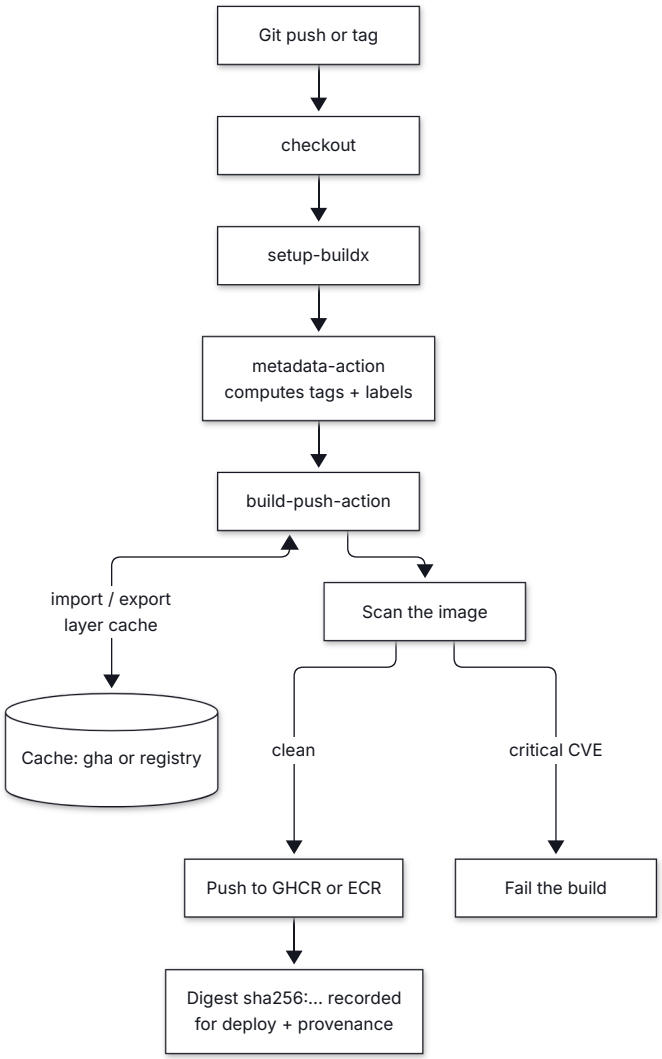


Figure 5.4-1

How the cache turns a rebuild into seconds (a Node.js Dockerfile where only `src/` changed):

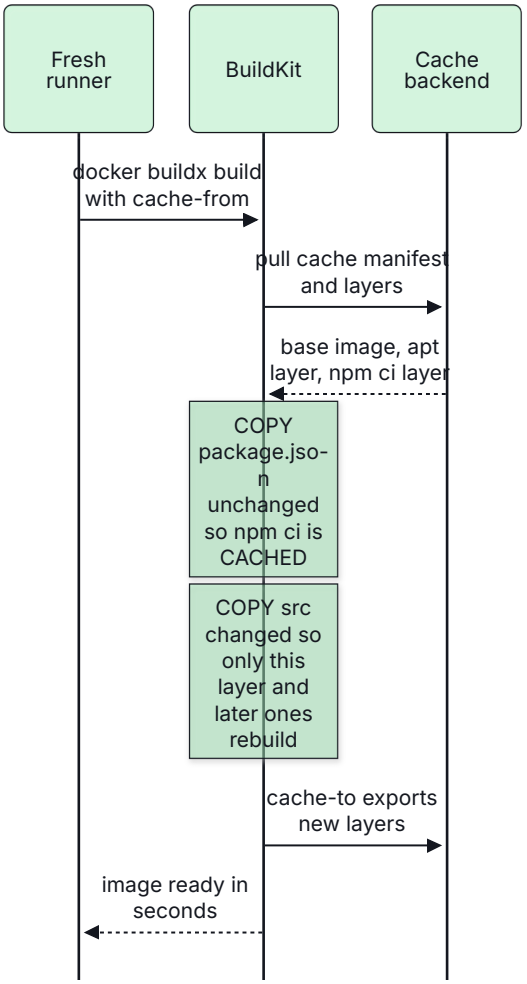


Figure 5.4-2

5 How it works

Tagging strategy

A tag points at a **manifest digest** (`sha256:...`). Tags can be moved; digests cannot. Use tags for humans and digests for machines.

Tag	Example	Mutable?	Use
Commit SHA	`sha-3f2a9c1e...` (full 40 chars)	No (treat as write-once)	The deploy identity. Every build gets one. Trace any running container to a commit.

Tag	Example	Mutable?	Use
Semantic version	`1.4.2`, `1.4`, `1`	`1.4.2` no, `1.4` and `1` move	Release tags created from Git tags `v1.4.2` (SemVer §3.10).
Branch	`main`, `pr-123`	Yes	Convenience for humans and preview environments. Never deploy production from it.
`latest`	`latest`	Yes	Convenience only. Avoid in manifests: it is not a version, so rollback and audit break.
Digest	`shop-api@sha256:9f86d0...`	No, by construction	The strongest pin; what GitOps tools and admission policies compare.

Rule of thumb: **build and push by SHA on every merge; add SemVer tags only on release; deploy by SHA or digest.** Set repository tag immutability where the registry supports it (ECR does; GHCR tags are mutable, so rely on convention + digests).

Cache backends

Backend	`cache-to` / `cache-from`	Notes
`gha`	`type=gha,mode=max`	Easiest in GitHub Actions. Shares the repository cache quota (10 GB at the time of writing; old entries are evicted), scoped by branch rules.
`registry`	`type=registry,ref=ghcr.io/acme/shop-api:buildcache,mode=max`	Works on any CI and between machines; costs registry storage.
`inline`	`type=inline`	Embeds cache metadata into the pushed image. Simple, but only exports the final stage layers.
`local`	`type=local,dest=/tmp/cache`	For self-hosted runners with a persistent disk.

`mode=max` exports layers of **all** stages (essential for multi-stage builds); the default `mode=min` exports only the final image's layers. Layer order in the Dockerfile still matters most: copy lockfiles and install dependencies **before** copying source (Layer caching §4.9). A cache is an optimisation, never a correctness mechanism: a cache miss must still build correctly.

Key actions and what they do

Action	Purpose
`docker/setup-buildx-action@v3`	Creates a BuildKit builder (needed for the `gha` cache and multi-platform).
`docker/setup-qemu-action@v3`	Registers CPU emulators so an x86 runner can build Arm images (slow; native Arm runners are faster).

Action	Purpose
`docker/login-action@v3`	Logs in to `ghcr.io`, ECR or Docker Hub.
`docker/metadata-action@v5`	Turns the Git event (branch, tag, SHA) into image tags and OCI labels.
`docker/build-push-action@v6`	Runs the build, applies the cache, optionally pushes, outputs the `digest`.

Action major versions keep moving; check each action's releases page and pin to a full commit SHA in security-sensitive pipelines.

Registry authentication

Registry	Authentication in CI	Stored secret?
GHCR	`GITHUB_TOKEN` with job permission `packages: write`	none
ECR	`aws-actions/configure-aws-credentials` with OIDC role, then `aws-actions/amazon-ecr-login`	none
Docker Hub	access token in `secrets.DOCKERHUB_TOKEN`	yes, rotate it

Provenance and SBOM

BuildKit can attach **attestations** to the pushed image: a *provenance* record (which workflow, commit and builder produced it) and an *SBOM* (software bill of materials: the list of packages inside). Enable with `provenance: mode=max` and `sbom: true` on `build-push-action`; GitHub can also sign build provenance with `actions/attest-build-provenance`. Recent `buildx` versions add a minimal provenance attestation by default, which makes the pushed object an index (a manifest list) containing extra "unknown/unknown" entries; some older tools display that oddly. See L5.6 for how provenance is used.

6 Step-by-step guide

Prerequisites: a GitHub repository `shop-api` (public is free; private uses your plan's minutes and package storage), Docker installed locally to test (`docker version`). No cloud account needed for the GHCR path; the ECR variant at the end needs an AWS account (a single repository costs almost nothing, but follow the cleanup in task T4).

1. **Write a cache-friendly Dockerfile.** Save as `Dockerfile` in the repo root:

```dockerfile
# syntax=docker/dockerfile:1
```

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev

FROM node:22-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY src ./src
COPY package.json ./
USER node
EXPOSE 3000
CMD ["node", "src/server.js"]
```

The lockfile is copied *before* `src`, so editing code does not invalidate the `npm ci` layer.

2. **Build locally with buildx and see the cache work.**

```bash
docker buildx version
docker buildx build -t shop-api:dev --load .
touch src/server.js            # simulate a code change
docker buildx build -t shop-api:dev --load .
```

The second build prints `CACHED` next to the `npm ci` step and only re-runs `COPY src`. (Output is similar to `⇒ CACHED [deps 3/3] RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev`.)

3. **Create the workflow.** Save as `.github/workflows/docker-image.yml`:

```yaml
name: docker-image

on:
  push:
    branches: [main]
    tags: ["v*.*.*"]
  pull_request:

concurrency:
  group: docker-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}
```

```yaml
jobs:
  image:
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    permissions:
      contents: read
      packages: write          # push to GHCR with GITHUB_TOKEN
      id-token: write          # needed by attestation signing
      attestations: write
    outputs:
      digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@v4

      - uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
        if: github.event_name ≠ 'pull_request'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Compute tags and labels
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          flavor: latest=false
          tags: |
            type=sha,format=long,prefix=sha-
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=ref,event=branch
            type=ref,event=pr

      - name: Build (and push on main or tags)
        id: build
        uses: docker/build-push-action@v6
        with:
          context: .
          push: ${{ github.event_name ≠ 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          provenance: mode=max
          sbom: true

      - name: Attest build provenance
        if: github.event_name ≠ 'pull_request'
```

```
        uses: actions/attest-build-provenance@v2
        with:
          subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          subject-digest: ${{ steps.build.outputs.digest }}
          push-to-registry: true

      - name: Summary
        run: |
          echo "### Image" >> "$GITHUB_STEP_SUMMARY"
          echo "digest: \`${{ steps.build.outputs.digest }}\`" >>
  "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"
          echo "${{ steps.meta.outputs.tags }}" >> "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"
```

On a pull request the image is **built but not pushed** (it proves the Dockerfile works and warms the cache). `metadata-action` lowercases the image name for you, which matters because GHCR names must be lowercase.

4. **Run it.** Push to a branch and open a PR: the job builds, nothing is pushed. Merge to `main`: the run pushes `ghcr.io/<you>/shop-api:sha-<40-char-sha>` and `:main`. The step summary shows the digest. Create and push a tag:

```bash
git tag v0.1.0 && git push origin v0.1.0
```

The tag run pushes `0.1.0` and `0.1` as well.

5. **Prove the cache.** Re-run the job, or push a change that touches only `src/`. In the `build-push-action` log look for `CACHED` lines and for `importing cache manifest from gha:...`. The job duration should drop clearly compared with run 1.
6. **Pull the exact image by digest and inspect it.**

```bash
docker pull ghcr.io/<you>/shop-api@sha256:<digest-from-summary>
docker buildx imagetools inspect ghcr.io/<you>/shop-api:main
docker image inspect ghcr.io/<you>/shop-api:main --format '{{ index .Config.Labels
  "org.opencontainers.image.revision" }}'
```

The `revision` label (added by `metadata-action`) equals the commit SHA. For a private package, run `echo <YOUR_TOKEN> | docker login ghcr.io -u <you> --password-stdin` first with a token that has `read:packages`.

7. **Scan before you push (recommended).** Build locally in the job, scan, and only then

push. Insert before the push step:

```yaml
      - name: Build image for scanning (single platform, not pushed)
        uses: docker/build-push-action@v6
        with:
          context: .
          load: true
          tags: shop-api:scan
          cache-from: type=gha
      - name: Scan with Trivy
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:scan
          severity: CRITICAL,HIGH
          exit-code: "1"
          ignore-unfixed: true
```

The second `build-push-action` then reuses the cache, so the extra build costs seconds. Details of scanning are in Image Scanning §4.14 and Container Image Scanning with Trivy §11.9.

8. **Multi-architecture (optional).** Add `docker/setup-qemu-action@v3` before buildx and `platforms: linux/amd64,linux/arm64` to the push step. Emulated Arm builds are several times slower; keep the cache on, and note that `load: true` cannot load a multi-platform result, which is why the scan step above builds one platform.
9. **ECR variant.** Replace the login step and image name:

```yaml
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/shop-api-ci-push
          aws-region: us-east-1
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      # then: images: ${{ steps.ecr.outputs.registry }}/shop-api   (in metadata-
  action)
```

Create the repository once, with immutable tags and scanning on push:

```bash
aws ecr create-repository --repository-name shop-api --region us-east-1 \
  --image-tag-mutability IMMUTABLE \
  --image-scanning-configuration scanOnPush=true
```

The push role needs `ecr:GetAuthorizationToken` (on `*`) plus `ecr:BatchCheckLayerAvailability`, `ecr:InitiateLayerUpload`, `ecr:UploadLayerPart`, `ecr:CompleteLayerUpload` and `ecr:PutImage` on that one repository, and a trust policy limited to your repo and branch (see L5.5). With **immutable tags**, do not push a moving `main` tag to ECR; use SHA and SemVer tags only.

7 Real-world scenario

(Illustrative.) A team deploys `shop-api` by pulling `shop-api:latest` onto two servers. On a Tuesday, checkout errors start. The on-call engineer asks "which commit is running?" and nobody can say: one server pulled `latest` before the last merge, the other after. Rolling back means rebuilding an old commit from scratch, 14 minutes of builds plus a hope that the dependencies resolve the same way.

After the incident the pipeline is changed: every merge builds once, tags the image `sha-<commit>`, records the digest in the deploy step, and the Compose file on the servers references that tag. The `npm ci` layer is cached in `gha`, so builds drop from about 4 minutes to well under 1 minute for code-only changes. The next bad release is rolled back by redeploying the previous SHA tag, already in the registry, in under two minutes. **Lesson:** an image name that cannot answer "what is running" is a deployment bug waiting to happen; tags are the index of your rollback history.

8 Common mistakes and best practices

Don't

- Deploy `latest` (or any mutable tag) to production.
- Rebuild the image in the deploy stage or per environment; promote the one you tested (L5.6).
- Put credentials into the build (`ARG NPM_TOKEN` or `COPY .env`): they stay in image layers and history forever (L5.5).
- Use `COPY . .` before installing dependencies: any file change busts the dependency layer.
- Forget `.dockerignore` (`.git`, `node_modules`, `.env`, test output); it bloats the context and can leak files.
- Export cache with the default mode for a multi-stage build and wonder why nothing is cached.
- Let the registry grow without a lifecycle policy.

Do

- Tag with the full commit SHA on every build; add SemVer tags on release; deploy by SHA or digest.
- Build pull-request images without pushing, so PRs cannot publish anything.
- Use `GITHUB_TOKEN` or OIDC for registry authentication; grant `packages: write` only to the job that pushes.
- Scan before pushing, fail on critical findings, and pin base images (`node:22-alpine` at minimum, a digest for strict environments).
- Add OCI labels (`metadata-action` does this) so any running container links back to its commit.
- Turn on immutable tags and lifecycle rules in ECR; delete untagged GHCR versions on a schedule.

9 Tasks

- [ ] **T1 — Build with buildx and read the cache** *(≈30 min, beginner)*
  - **Goal:** see layer caching locally before using it in CI.
  - **Steps:**
    1. Create the `Dockerfile` from step 1 for `shop-api` (any small Node.js app with a `src/server.js` is fine).
    2. Run `docker buildx build -t shop-api:dev --load .` twice, editing a file in `src/` between runs.
    3. Reorder the Dockerfile so `COPY . .` comes before `npm ci`, edit a source file again, and rebuild.
  - **Done when:** you can point at `CACHED` on the `npm ci` step in run 2, and at the same step re-running in the reordered version, and explain why.

- [ ] **T2 — Push to GHCR with SHA and SemVer tags** *(≈45 min, intermediate)*
  - **Goal:** an automated, traceable image push.
  - **Steps:**
    1. Add `.github/workflows/docker-image.yml` from step 3 to `shop-api`.
    2. Merge to `main`, then push tag `v0.1.0`.
    3. Open the package page (profile, then Packages) and list the tags.
  - **Done when:** the package shows `sha-<40 chars>`, `main`, `0.1.0` and `0.1`, and `docker pull` by digest works.

- [ ] **T3 — Measure and tune the cache** *(≈45 min, intermediate)*
  - **Goal:** prove the pipeline gets faster.

- **Steps:**
  1. Record the build step duration on a cold run (change `cache-from` / `cache-to` temporarily to remove them, or use a new branch).
  2. Re-enable `type=gha,mode=max`; run twice and record the second run.
  3. Switch to `type=registry,ref=ghcr.io/<you>/shop-api:buildcache,mode=max` and record a third pair.
- **Done when:** you have a small table (cold, warm gha, warm registry) and can say which you would use for a team and why.

- [ ] **T4 — Push to ECR with OIDC, then clean up** *(≈60 min, advanced; AWS free tier or a few cents — set a billing alarm first)*
  - **Goal:** push without any stored AWS keys.
  - **Steps:**
    1. Create the ECR repository with immutable tags (step 9) and an IAM role trusted by your repo via GitHub's OIDC provider.
    2. Change the workflow to log in with `configure-aws-credentials` + `amazon-ecr-login` and push only SHA and SemVer tags.
    3. Re-run the same commit and observe what happens to an immutable tag that already exists.
    4. **Cleanup:** `aws ecr delete-repository --repository-name shop-api --force --region us-east-1`, and delete the IAM role.
  - **Done when:** `aws ecr describe-images --repository-name shop-api` lists your SHA tag before cleanup, and the repository and role are gone afterwards.

- [ ] **T5 — Break and fix: leak a secret into a layer** *(≈40 min, intermediate)*
  - **Goal:** understand why secrets never belong in images.
  - **Steps:**
    1. In a throw-away Dockerfile add `ARG API_KEY=hunter2-demo` and `RUN echo $API_KEY > /tmp/k`, build it.
    2. Run `docker history --no-trunc <image>` and `docker run --rm <image> cat /tmp/k`.
    3. Fix it using a BuildKit secret mount (`RUN --mount=type=secret,id=api_key ...` with `secrets: api_key=...` in the action) and rebuild.
  - **Done when:** the leaked value is visible in `docker history` before the fix and absent in history and filesystem after it.

- [ ] **T6 — Prove it: from push to pinned deploy** *(≈60 min, advanced)*

- **Goal:** show end-to-end traceability without notes.
- **Steps:**
  1. Without the guide, extend the workflow so the job outputs the image digest, and a second job prints `ghcr.io/<you>/shop-api@<digest>`.
  2. Pick a running container (or just a pulled image) and, using only registry data and labels, find the commit that built it.
  3. Explain in writing why a deploy should reference the digest or SHA tag rather than `main`.
- **Done when:** you can go from an image reference to its commit and workflow run in under two minutes, and your explanation covers tag mutability.

10 Self-check questions

1. What is the difference between a tag and a digest, and which should a deployment reference?
2. Why does the order of `COPY` instructions in a Dockerfile change CI build time?
3. When do you need `mode=max` for `--cache-to`?
4. Why build pull-request images without pushing them?
5. How does a GitHub Actions job push to GHCR without a stored password, and how to ECR?
6. Why is `latest` a poor deployment tag?

ANSWERS

1. A tag is a movable name; a digest (`sha256:...`) is the content hash of the manifest and cannot change. Deploy by digest or by a write-once SHA tag.
2. Layers are cached top to bottom; once one layer changes, every later layer rebuilds. Copying lockfiles and installing dependencies before copying source keeps the expensive layer cached.
3. For multi-stage builds, so that intermediate stage layers are exported too; the default only exports the final image's layers.
4. Untrusted code from a pull request must not publish artifacts to your registry or access push credentials; the build still validates the Dockerfile and warms the cache.
5. GHCR: the workflow's `GITHUB_TOKEN` with `packages: write`. ECR: an IAM role assumed through GitHub's OIDC provider, then `amazon-ecr-login` exchanges it for a short-lived registry token.
6. It moves, so it does not identify a version: you cannot tell what is running, reproduce it or roll back to "the previous latest".

11 Further reading

- Docker Docs: *Docker Build* (BuildKit, cache backends, multi-platform builds) and *Continuous integration with Docker / GitHub Actions* at docs.docker.com.
- Docker's `build-push-action`, `metadata-action`, `setup-buildx-action` and `login-action` repositories and their README examples.
- GitHub Docs: *Working with the Container registry*; *Using artifact attestations* under docs.github.com.
- AWS Docs: *Amazon ECR User Guide* (image tag mutability, lifecycle policies, scanning).
- Next: Secrets and environment variables §5.5 and Build artifacts §5.6.

LAB NOTES What I ran, what I saw, what it means

LEVEL 5: CI/CD

5.5 Secrets and Environment Variables

Topic 5 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

An **environment variable** is a named string handed to a process when it starts (`DATABASE_URL=...`). It is the standard way to give one build artifact different **configuration** in different places: the same `shop-api` image runs in staging with a staging database and in production with the production database, and only the variables differ. This is the third factor ("Config") of the Twelve-Factor App `(12factor.net)` method: config lives in the environment, not in the code or the image.

A **secret** is configuration whose exposure causes harm: database passwords, API keys, signing keys, tokens. Secrets need extra rules on top of ordinary config: they are stored encrypted, handed out only to the jobs and people that need them, masked in logs, rotated, and (best of all) replaced by **short-lived credentials** that never need storing. The modern way to get those in a pipeline is **OpenID Connect (OIDC)**: the CI system proves its identity to the cloud with a signed token, and the cloud returns temporary credentials.

2 Why it matters in DevOps

- **Leaks are permanent:** a key committed to Git, baked into an image layer or printed in a log should be treated as compromised the moment it is public. Removing it later does not un-leak it.
- **CI is a high-value target:** the pipeline holds the keys to production. Anyone who can change a workflow file or an action it uses can try to read them.
- **Blast radius:** one shared admin key in one repository setting means a single compromised job can destroy everything. Per-environment, least-privilege credentials limit the damage.
- **Operations:** a rotation that takes two weeks of coordination never happens. A rotation that is one command is done on schedule, and during an incident.

3 Real-world system example

GitHub Actions OIDC with AWS, Google Cloud and Azure. GitHub documents OpenID Connect hardening for workflows: each job can request a short-lived signed JWT whose claims describe the run (repository, branch or tag, environment, workflow). AWS IAM, Google Cloud workload identity federation and Azure federated credentials can trust GitHub's issuer (`https://token.actions.githubusercontent.com`) and exchange that token for temporary cloud credentials, with trust conditions such as "only repository `acme/shop-api`, only environment `production`". No long-lived cloud key is stored in GitHub at all.

The 2025 `tj-actions/changed-files` incident. In March 2025 a popular third-party GitHub Action was compromised and modified to print secrets from the runner's memory into public workflow logs for repositories that used it. It is widely documented in security advisories (CVE-2025-30066). The practical lessons: pin third-party actions to full commit SHAs, give jobs minimal `permissions`, prefer OIDC over stored keys (there was little for the attacker to steal), and rotate anything that might have been exposed.

CircleCI, January 2023. CircleCI disclosed a security incident and advised all customers to rotate every secret stored in the platform, including project environment variables and tokens. It is a good reminder that a CI vendor's secret store is still a single place an attacker wants, and that rotation must be something you can do quickly.

4 Diagram

OIDC: how a job gets cloud access without any stored key.

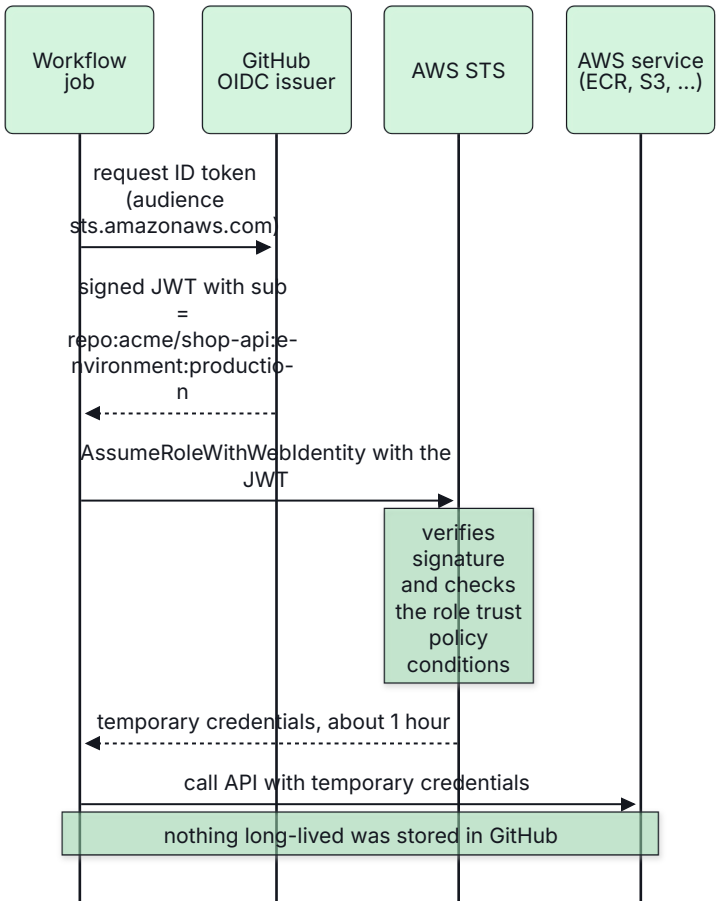


Figure 5.5-1

Where configuration and secrets come from at each stage:

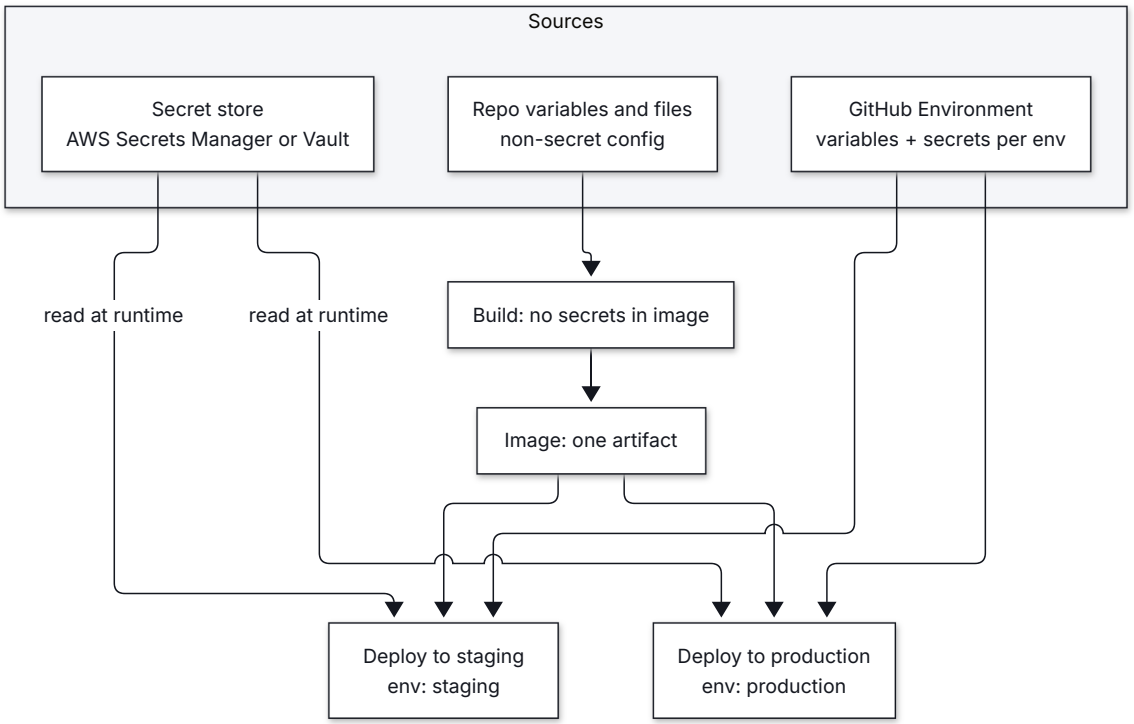


Figure 5.5-2

5 How it works

Kinds of configuration

Kind	Example	Where it lives	Secret?
Build-time constant	Node version, base image tag	Dockerfile, workflow file	No
Per-environment config	`LOG_LEVEL`, `PUBLIC_URL`, replica count	GitHub Environment **variables**, Helm values, `.env.<env>` in Git	No
Per-environment secret	`DATABASE_URL`, `STRIPE_API_KEY`	GitHub Environment **secrets**, cloud secret store	**Yes**
Pipeline credential	registry push, cloud deploy	OIDC role (preferred), or a scoped token	**Yes**

Where to keep secrets

Store	Good for	Notes
GitHub encrypted secrets (repo, environment, organization)	CI-only values and tokens	Masked in logs; **not** passed to workflows triggered from forks; environment secrets are released only to jobs that target that environment, after any required reviewers approve.
AWS Secrets Manager / SSM Parameter Store	Runtime secrets for the app	Encrypted with KMS, IAM-controlled, auditable, rotation support in Secrets Manager. See Secrets Manager §6.16.
HashiCorp Vault or cloud equivalents	Dynamic, short-lived credentials	Can issue a database user per deployment that expires.
Kubernetes Secrets	Pod-consumed values	Only base64-encoded by default; combine with encryption at rest, RBAC, and an external store (Secrets §8.7).
Encrypted files in Git (SOPS, Sealed Secrets)	GitOps flows	Secret is encrypted with a key; the ciphertext is reviewed and versioned.

See Secrets Management and Encryption §11.4 for the deeper treatment.

GitHub Environments

An **Environment** (`staging` , `production`) groups secrets, variables and **protection rules** (required reviewers, a wait timer, allowed branches). The same name holds a different value in each environment, so the deploy job is identical and only `environment:` changes:

YAML

```yaml
jobs:
  deploy:
    runs-on: ubuntu-24.04
    environment: production      # selects production secrets/vars, applies approval rules
    steps:
      - run: ./deploy.sh
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}   # production value here
          LOG_LEVEL: ${{ vars.LOG_LEVEL }}
```

More background: Environments and Secrets §3.12.

OIDC trust: the part that matters

The cloud role's **trust policy** decides who may assume it. The `sub` claim of the token looks like `repo:<owner>/<repo>:environment:<name>`, `repo:<owner>/<repo>:ref:refs/heads/main`, or `...:pull_request`. Narrow it as far as you can:

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-
  provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:acme/shop-
  api:environment:production"
      }
    }
  }]
}
```

A trust policy that omits the `sub` condition lets *any* GitHub repository assume your role. Always include both `aud` and `sub`. The job needs `permissions: id-token: write` to request the token.

Rules for secrets in pipelines

1. **Never in the image.** No `ENV API_KEY=...`, no `ARG` secrets (both land in `docker history`), no `COPY .env`. Pass config at run time. If a *build* needs a secret (a private npm registry), use a BuildKit secret mount, which never lands in a layer.
2. **Never in Git.** Enable secret scanning and push protection (Secrets Scanning §11.11).
3. **Never printed.** GitHub masks registered secrets in logs, but only exact matches: a transformed value (base64, a substring) is **not** masked. Do not `echo` them, do not enable `set -x` around them. For values computed at run time call `echo "::add-mask::$VALUE"`.
4. **Least privilege per environment.** The staging role cannot touch production. Pull requests get no deploy secrets.
5. **Untrusted code never meets secrets.** Secrets are not provided to workflows from forks. Be very careful with `pull_request_target` and `workflow_run`, which run with secrets in the base repository's context: never check out and run the pull request's code there.
6. **Prefer short-lived over long-lived.** OIDC, then tokens with expiry, then static keys as a last resort.

7. **Rotate on a schedule and on suspicion.** See below.

Rotation

Rotation is replacing a secret with a new value and retiring the old one without downtime. The safe pattern is **overlap**:

1. Create the new credential (the database now accepts both old and new).
2. Update the store (Secrets Manager version, GitHub secret).
3. Roll the application so it reads the new value (a deploy or restart; or a library that re-reads the secret).
4. Verify with a health check and metrics.
5. Revoke the old credential.

Automating it (Secrets Manager rotation with a Lambda, Vault dynamic credentials) is better than a calendar reminder. Whatever you choose, practise it: the first time you rotate must not be during an incident.

6 Step-by-step guide

Prerequisites: a GitHub repository `shop-api` (a public repo or a plan that supports Environments on private repos), the `gh` CLI logged in (optional, the web UI works), Node.js 22 and `jq` locally. No cloud account is needed for steps 1 to 6; step 7 (AWS) needs one and is free to configure (IAM roles cost nothing), but set a billing alarm anyway.

1. **Make the app fail fast on missing config.** Save as `src/config.js`:

```javascript
const required = ["DATABASE_URL", "REDIS_URL", "SESSION_SECRET"];
const missing = required.filter((k) => !process.env[k]);
if (missing.length) {
  console.error(`FATAL: missing required env vars: ${missing.join(", ")}`);
  process.exit(1);
}
module.exports = {
  logLevel: process.env.LOG_LEVEL || "info",
  databaseUrl: process.env.DATABASE_URL,
};
```

Test it locally:

```bash
node -e 'require("./src/config.js")'; echo "exit=$?"
DATABASE_URL=x REDIS_URL=y SESSION_SECRET=z node -e
  'console.log(require("./src/config.js").logLevel)'
```

Expected: `FATAL: missing required env vars: DATABASE_URL, REDIS_URL, SESSION_SECRET` and `exit=1`; then `info`. A missing variable now fails at startup (and in the health check), not at 3 a.m. on the first request that needs it.

2. **Create two environments with different values.** In the repository, **Settings, Environments, New environment**: `staging` and `production`. In each, add the *secret* `DATABASE_URL` and the *variable* `LOG_LEVEL` (`debug` for staging, `info` for production). With the CLI:

```bash
gh api -X PUT repos/<you>/shop-api/environments/staging
gh api -X PUT repos/<you>/shop-api/environments/production
gh secret set DATABASE_URL --env staging    --body 'postgres://shop:
  <STAGING_PW>@10.0.1.10:5432/shop'
gh secret set DATABASE_URL --env production --body 'postgres://shop:
  <PROD_PW>@10.0.2.10:5432/shop'
gh variable set LOG_LEVEL --env staging    --body debug
gh variable set LOG_LEVEL --env production --body info
```

On `production`, add **Required reviewers** (yourself) and restrict **Deployment branches** to `main`.

3. **Prove the same job gets different config.** Save as `.github/workflows/show-config.yml`:

```yaml
name: show-config
on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
        default: staging

permissions:
  contents: read

jobs:
  show:
    runs-on: ubuntu-24.04
    timeout-minutes: 5
```

```
    environment: ${{ inputs.environment }}
    steps:
      - name: Show config (never the secret itself)
        env:
          LOG_LEVEL: ${{ vars.LOG_LEVEL }}
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
        run: |
          echo "environment=${{ inputs.environment }} LOG_LEVEL=$LOG_LEVEL"
          echo "db host: $(echo "$DATABASE_URL" | sed -E 's#.*@([^:/]+).*#\1#')"
          echo "attempting to print the secret: $DATABASE_URL"
```

Run it for each environment (Actions tab, Run workflow). The production run waits for your approval. The log shows `LOG_LEVEL=debug` vs `info`, the two different host names, and the last line prints `***` instead of the connection string because GitHub masks it. (The host extraction is a derived value; deriving fragments is fine, but a derived *piece* of a secret such as a password would not be masked.)

4. **See the log-masking limit for yourself.** Add a step that prints `echo "$DATABASE_URL" | base64`. Run it. The base64 text is shown in clear, because only the exact registered value is masked. Delete the step, and treat the run logs as sensitive; if this had been a real secret you would now rotate it.
5. **Inspect an OIDC token's claims (no cloud needed).** Add this job; it requests a token and prints only the claims, not the token:

YAML

```
  oidc-claims:
    runs-on: ubuntu-24.04
    timeout-minutes: 5
    permissions:
      id-token: write
      contents: read
    environment: staging
    steps:
      - name: Decode token claims
        run: |
          TOKEN=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN"
  \
            "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sts.amazonaws.com" | jq -r .value)
          echo "$TOKEN" | python3 -c 'import sys,base64,json;
  p=sys.stdin.read().strip().split(".")[1]; p+="="*(-len(p)%4);
  c=json.loads(base64.urlsafe_b64decode(p)); print(json.dumps({k:c.get(k) for k in
  ("iss","sub","aud","ref","environment","repository")},indent=2))'
```

Output is similar to:

```json
{
  "iss": "https://token.actions.githubusercontent.com",
  "sub": "repo:<you>/shop-api:environment:staging",
  "aud": "sts.amazonaws.com",
  "ref": "refs/heads/main",
  "environment": "staging",
  "repository": "<you>/shop-api"
}
```

The `sub` value is exactly what a cloud trust policy matches against.

6. **Keep secrets out of the image.** Show the wrong and right way:

```bash
# WRONG: leaks into history
printf 'FROM alpine\nARG API_KEY\nRUN echo "$API_KEY" > /dev/null\n' >
  /tmp/Dockerfile.bad
docker build -f /tmp/Dockerfile.bad --build-arg API_KEY=demo-not-real -t leak:test
  /tmp
docker history --no-trunc leak:test | grep API_KEY
```

The build arg value appears in the history output. Fix with a BuildKit secret mount (build-time only):

```dockerfile
# syntax=docker/dockerfile:1
FROM alpine
RUN --mount=type=secret,id=api_key sh -c 'test -s /run/secrets/api_key && echo "secret
  was available"'
```

```bash
echo -n 'demo-not-real' > /tmp/api_key.txt
docker build --secret id=api_key,src=/tmp/api_key.txt -t noleak:test .
docker history --no-trunc noleak:test | grep -c demo-not-real     # 0
```

In CI the same thing is `secrets: |` + `api_key=${{ secrets.API_KEY }}` on `docker/build-push-action`.

7. **Replace a stored AWS key with OIDC (AWS account required).**

```bash
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com
```

(Older CLI versions also required `--thumbprint-list`; follow the AWS documentation for your version.) Create a role with the trust policy shown in section 5 (replace the account ID and repository) and a minimal permission policy, then use it:

```yaml
jobs:
  deploy:
    runs-on: ubuntu-24.04
    environment: production
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/shop-api-deploy-production
          aws-region: us-east-1
      - run: aws sts get-caller-identity
```

`get-caller-identity` returns the assumed-role ARN. Change the repository name in the trust policy to a wrong value and re-run: the step must fail with `Not authorized to perform sts:AssumeRoleWithWebIdentity`. Cleanup: delete the role and the provider if this was only practice.

8. **Rotate a secret with overlap (rehearsal).** Update `DATABASE_URL` (staging) to a second database user, redeploy, and verify with `curl -fsS https://staging.example.com/healthz`, then drop the old user. Time the whole procedure and write the steps into `docs/runbook-rotate-secrets.md`.

7 Real-world scenario

(Illustrative.) A developer debugging a failing deploy adds `env | sort` to the workflow and pushes it to a branch. The job runs, prints the environment, and GitHub masks the exact secret values it knows about. But the workflow also base64-encodes a kubeconfig before passing it to a tool, and the encoded string is printed unmasked in the log, which is world-readable on this public repository. A secret-scanning alert fires within minutes.

The on-call engineer treats the cluster credential as compromised: revokes it, issues a new one, updates the environment secret, re-runs the deploy, and checks the cloud audit log (CloudTrail on AWS) for use of the old credential. The follow-up work moves the pipeline to OIDC (no stored kubeconfig), adds the `production` environment with required reviewers and a branch restriction, and turns on push protection. **Lesson:** masking is a safety net, not a control. The real controls are not having long-lived secrets, scoping them, and being able to rotate quickly.

8 Common mistakes and best practices

Don't

- Commit `.env` files with real values, or bake `ENV SECRET=...` / `ARG` secrets into the Dockerfile.
- Use one organization-wide admin key for every repository and environment.
- Echo secrets, or transform them (base64, split) and print the result; masking will not catch it.
- Run untrusted pull-request code with `pull_request_target` or `workflow_run` and secrets available.
- Omit the `sub` condition from an OIDC trust policy.
- Keep a rotation procedure "in someone's head".

Do

- Build one image; supply config at deploy time via environment variables or mounted files.
- Use GitHub Environments: separate secrets per environment, required reviewers and branch rules for production.
- Prefer OIDC and other short-lived credentials; use `permissions:` to grant `id-token: write` only to the job that needs it.
- Make the app validate required config at startup and fail with a clear message (never the values).
- Pin third-party actions by commit SHA; they can read what your job can read.
- Scan for leaked secrets (push protection, scheduled scans) and rehearse rotation.

9 Tasks

- [ ] **T1 — Fail fast on missing config** *(≈20 min, beginner)*
  - **Goal:** make configuration errors loud and early.

- **Steps:**
  1. Add `src/config.js` from step 1 to `shop-api` and require it at the top of `src/server.js`.
  2. Start the server with and without the variables set.
- **Done when:** without the variables the process exits 1 listing the names; with them it starts and `LOG_LEVEL` defaults to `info`.

☐ **T2 — Two environments, one job** *(≈40 min, beginner)*

- **Goal:** same workflow, different config and secrets.
- **Steps:**
  1. Create the `staging` and `production` environments with different `LOG_LEVEL` and `DATABASE_URL` (placeholders are fine).
  2. Add required reviewers and a `main`-only branch rule to `production`.
  3. Run `show-config.yml` for both.
- **Done when:** the logs show different `LOG_LEVEL` and host names, the secret prints as `***`, and the production run paused for approval.

☐ **T3 — Break it: defeat the masking, then fix** *(≈30 min, intermediate)*

- **Goal:** understand what masking does and does not do.
- **Steps:**
  1. Print a base64 of a (fake) secret and observe it unmasked.
  2. Fix by deleting the step, and add `echo "::add-mask::$DERIVED"` for a computed sensitive value; show a second print is masked.
  3. Write two sentences on why you would rotate a real secret after step 1.
- **Done when:** the first log shows the leak, the fixed log shows `***` for the derived value, and your note says "rotate".

☐ **T4 — Keep a build secret out of the image** *(≈30 min, intermediate)*

- **Goal:** use BuildKit secret mounts.
- **Steps:**
  1. Reproduce the `ARG` leak and inspect it with `docker history --no-trunc`.
  2. Rebuild using `--mount=type=secret`.
  3. Add the same to `build-push-action` with `secrets:` in your workflow (using a dummy repo secret).
- **Done when:** `docker history --no-trunc` shows the value before the fix and not after, and the CI build passes.

- [ ] **T5 — Replace a stored key with OIDC** *(≈60 min, advanced; AWS account, no cost beyond a billing alarm)*
  - **Goal:** pipeline access to AWS with no stored credentials, and a trust policy that actually restricts.
  - **Steps:**
    1. Create the OIDC provider and a role trusted only by `repo:<you>/shop-api:environment:production`.
    2. Use `configure-aws-credentials` and run `aws sts get-caller-identity` from the production environment.
    3. Run the same job from `staging` (or a fork or another repository) and confirm it is denied.
    4. **Cleanup:** delete the role (and provider if unused).
  - **Done when:** production succeeds, the other context fails with `Not authorized to perform sts:AssumeRoleWithWebIdentity`, and no AWS key exists in repository secrets.
- [ ] **T6 — Prove it: write and rehearse the rotation runbook** *(≈60 min, advanced)*
  - **Goal:** rotate a secret with zero downtime and without the guide.
  - **Steps:**
    1. Write `docs/runbook-rotate-secrets.md`: the overlap procedure for `DATABASE_URL`, including verification and rollback.
    2. Execute it against staging while polling `/healthz` in a loop (`while sleep 1; do curl -s -o /dev/null -w '%{http_code}\n' https://staging.example.com/healthz; done`).
    3. Explain in writing why an exposed secret in a log must be rotated even if the log is deleted.
  - **Done when:** the polling loop never shows a non-200 during rotation and the old credential no longer works.

10 Self-check questions

1. What is the difference between configuration and a secret, and where does each live for `shop-api`?
2. Why must a secret never be an `ENV` or `ARG` in a Dockerfile?
3. How does OIDC remove the need for a stored cloud key, and which claim restricts which environment may assume the role?
4. Why are secrets not available to workflows triggered by pull requests from forks?

5. GitHub masks secrets in logs. Why is that not sufficient protection?
6. Describe a zero-downtime rotation of a database password.

ANSWERS

1. Config varies by environment but is harmless (`LOG_LEVEL`); a secret causes harm if exposed (`DATABASE_URL`). Config: environment variables/Helm values/GitHub variables. Secrets: GitHub environment secrets or a secret store, injected at deploy/run time.
2. Both are recorded in image metadata/layers; anyone who can pull the image can read them with `docker history` . Use runtime injection or BuildKit secret mounts.
3. The job presents a signed JWT from GitHub; the cloud verifies it and returns temporary credentials. The `sub` claim (for example `repo:acme/shop-api:environment:production`) is matched in the role's trust policy.
4. Fork code is untrusted; if it received secrets, anyone could open a pull request that prints them.
5. Masking is exact-match string replacement in logs. Encoded or partial forms, other outputs (artifacts, network calls) and anyone with write access to the workflow are not covered.
6. Add the new credential alongside the old, store the new value, roll the app so it uses it, verify health, then revoke the old credential.

11 Further reading

- GitHub Docs: *Using secrets in GitHub Actions*, *Managing environments for deployment*, *About security hardening with OpenID Connect*, *Security hardening for GitHub Actions* (docs.github.com).
- AWS Docs: *Configuring OpenID Connect in AWS* and the `aws-actions/configure-aws-credentials` README.
- The Twelve-Factor App (12factor.net), factor III: Config.
- Docker Docs: *Build secrets* (BuildKit secret mounts).
- Next: Build artifacts §5.6; deeper security material in Secrets Management and Encryption §11.4.

LAB NOTES What I ran, what I saw, what it means

5.6 Build Artifacts

Topic 6 of 12 | ~4 hours | Difficulty: Intermediate

1 What it is

A **build artifact** is the output of a build that you intend to run, ship or keep: a container image, a `.jar`, a `.tgz` of a Node.js app, a compiled binary, a Helm chart, a test report, a coverage file. A pipeline *produces* artifacts in early stages and *consumes* them in later ones.

The core rule of this topic is **build once, deploy many**: compile and package the software **one time**, give the result an immutable identity (a content hash called a **digest**), and move that exact artifact through every environment (test, staging, production). Only the *configuration* changes between environments (L5.5). Moving the same artifact forward is called **promotion**. The opposite, rebuilding "the same commit" in each environment, produces artifacts that are merely *similar*.

Two supporting ideas complete the picture:

- **Retention:** how long artifacts are kept before they are deleted (cheap build leftovers for days, released versions for years).
- **Provenance:** verifiable metadata saying *which source commit, which workflow and which builder produced this artifact*, so that you can prove it is the one your pipeline built and not something an attacker swapped in.

2 Why it matters in DevOps

- **"It passed in staging" only means something if production runs the same bytes.** A rebuild can pull a newer base image or dependency (`npm install` without a lockfile, an `apt-get` layer) and ship code nobody tested.
- **Fast, safe rollback:** if every release is stored and addressable, rollback is "deploy the previous artifact", not "rebuild an old commit and hope" (L5.7).
- **Speed:** promotion is a retag or a copy (seconds). A rebuild per environment costs minutes and adds failure points.

- **Cost:** keeping every PR build forever costs storage money; deleting too eagerly deletes the version you need to roll back to.
- **Security and audit:** regulators and customers increasingly ask "how do you know what is running was built from reviewed code?" Provenance answers that.

3 Real-world system example

Spinnaker and "bake once". Netflix's open-source continuous delivery platform Spinnaker models deployment as promotion of an immutable image: a pipeline bakes a machine or container image once and then deploys that same image to successive clusters and accounts, rather than building per environment. This immutable-artifact approach is one of the central design ideas of the tool.

SLSA and Sigstore. SLSA ("salsa", *Supply-chain Levels for Software Artifacts*, a framework from the Open Source Security Foundation) defines levels of build integrity; its core ask is that artifacts come with signed **provenance** produced by the build platform, not by the developer's laptop. The **Sigstore** project (including `cosign`) provides signing and verification for artifacts and container images, and GitHub's **artifact attestations** (`actions/attest-build-provenance`, verified with `gh attestation verify`) produce signed SLSA-style provenance for builds that run on GitHub Actions.

Container registries as artifact stores. Amazon ECR and GitHub Container Registry store images by digest; npm, Maven Central and PyPI store versioned packages that are immutable once published. The same principle: a published version number is never overwritten.

4 Diagram

Build once, promote the same digest through environments (compare with the anti-pattern below):

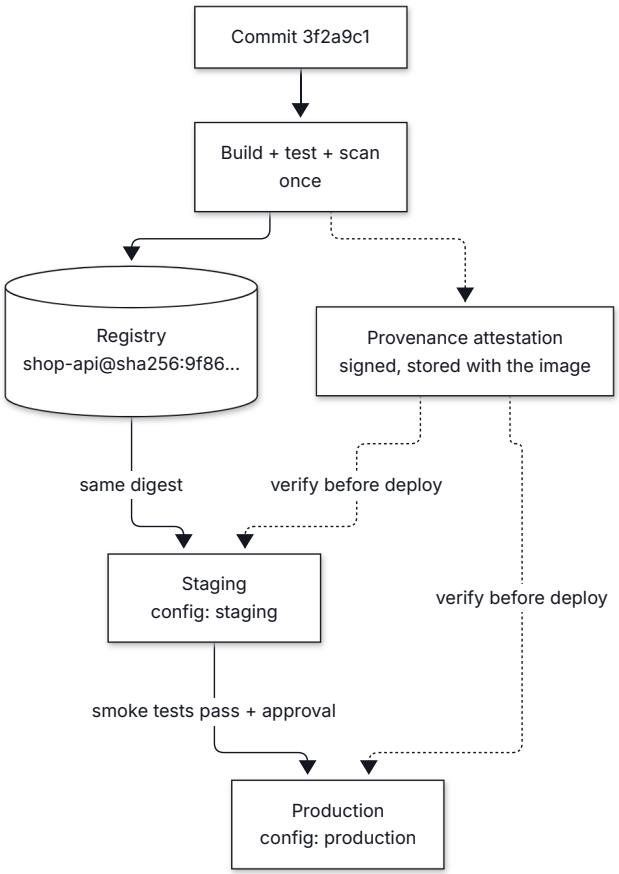


Figure 5.6-1

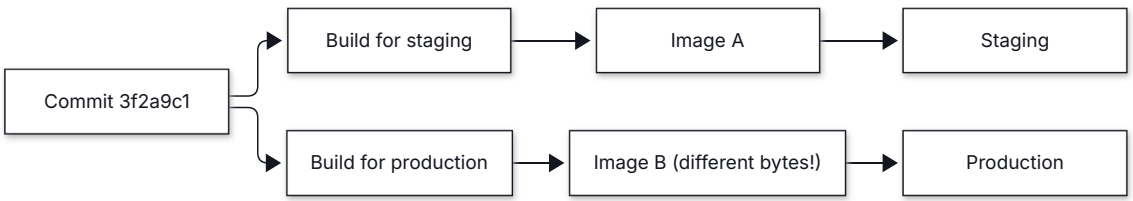


Figure 5.6-2

The second picture is the anti-pattern: image B was never tested.

Artifact lifecycle and retention:

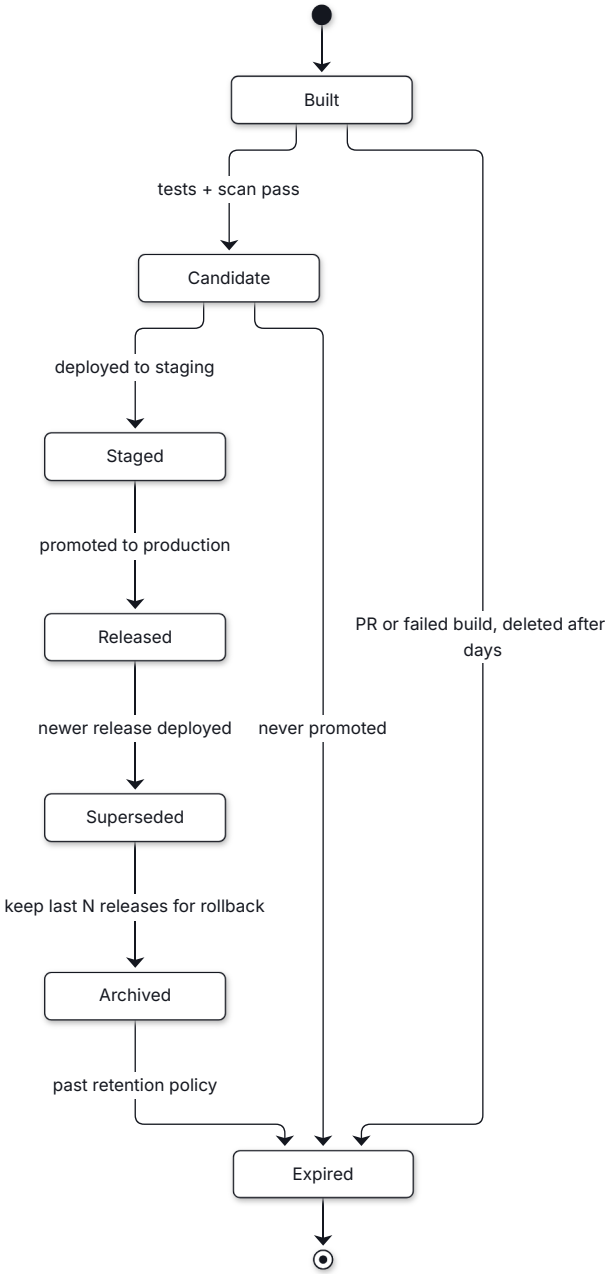


Figure 5.6-3

5 How it works

What counts as an artifact

Kind	Examples	Where to store	Typical retention
Deployable	container image, `.jar`, `.tgz`, Helm chart	registry (GHCR, ECR), package repo (npm, Artifactory, Nexus), S3	releases: long (a year or more); main builds: last N; PR builds: days
Pipeline hand-off	compiled `dist/` passed between jobs	workflow artifacts (`upload-artifact`)	days
Evidence	test reports, coverage, SBOM, scan results	workflow artifacts and/or attached to the release	weeks to years (compliance)
Cache	`node_modules`, BuildKit layers	cache service	automatic eviction; not an artifact!

Cache vs artifact: a cache is a speed-up that may disappear; an artifact is a *product* of the build that later stages depend on. Never rely on a cache for correctness (L5.2).

Identity: tag vs digest

A version tag says what you *meant*; the **digest** says what you *got*. Promote and deploy by digest (or a write-once SHA tag). Verify that what is running equals what was tested: `docker inspect` / `kubectl get pod -o jsonpath` can show the digest.

Promotion mechanisms

Mechanism	How	Cost
Retag in the same registry	`docker buildx imagetools create -t repo:staging repo@sha256:...`	zero copy: a new tag pointing at the same manifest
Copy between registries/accounts	`crane copy`, `skopeo copy`, ECR replication, or `docker buildx imagetools create` to another repo	copies layers once, digest preserved
Environment pointer in Git (GitOps)	a PR bumping `image: ...@sha256:` in the staging then production overlay	a Git commit is the promotion (Environment Promotion §17.4)
Package repo promote	Artifactory/Nexus "promote" from `snapshot` to `release` repository	metadata move

What does not count as promotion: running `docker build` again on the same commit.

Promotion must be **gated**: the staging deployment, smoke tests and (for production) an approval or health gate decide whether the digest advances. The pipeline records *which digest was promoted, by whom, when*: that is your release history.

Workflow artifacts (GitHub Actions)

```yaml
# producer job
- uses: actions/upload-artifact@v4
  with:
    name: dist-${{ github.sha }}          # names must be unique within a run in v4
    path: dist/
    retention-days: 7                     # default is 90 days (repo setting can lower it)
    if-no-files-found: error

# consumer job (same run)
- uses: actions/download-artifact@v4
  with:
    name: dist-${{ github.sha }}
    path: dist/

# consumer in a different run: pass run-id and a token
- uses: actions/download-artifact@v4
  with:
    name: dist-<sha>
    run-id: ${{ inputs.build_run_id }}
    github-token: ${{ secrets.GITHUB_TOKEN }}
```

Artifacts of `upload-artifact@v4` are immutable per name within a run, count against your storage quota, and expire after `retention-days`. Use them for hand-offs and evidence, not as the long-term home of releases: put releases in a registry or in a GitHub Release.

Retention policy: decide, don't default

Artifact	Suggested policy
PR images / workflow artifacts	delete after 1 to 7 days
`main` build images (SHA tags)	keep the newest 30 to 50 builds, or 30 days
Released versions (`v1.4.2`)	keep for at least as long as you may need to roll back or audit (often 1 year or more)
Untagged images	delete after 1 to 7 days (ECR lifecycle: `"tagStatus": "untagged"`)
Evidence (SBOM, scan report, attestations)	same lifetime as the artifact they describe

Never let a retention rule delete the image currently running in any environment or the one you would roll back to. ECR lifecycle policies and scheduled cleanup jobs (for GHCR, a workflow using the Packages API) must never match releases or anything referenced by deployments (see step 7 in the guide: a separate repository per retention class avoids multi-tag surprises).

Provenance and verification

1. **Build** produces the image; the builder records a statement: *subject* (image digest), *builder* (workflow identity), *source* (repository, commit, ref).
2. **Sign:** the statement is signed with a short-lived certificate tied to the workflow's OIDC identity (Sigstore) so there is no private key to steal.
3. **Store:** attached to the registry next to the image (and logged in a transparency log).
4. **Verify** before deploy: check that the digest has a valid attestation produced by *your* repository's workflow on *the expected branch*.

An **SBOM** (software bill of materials: list of components) is a separate attestation that answers "are we affected by this new CVE?" without unpacking images.

6 Step-by-step guide

Prerequisites: the `shop-api` repository and the `docker-image.yml` workflow from L5.4 (it pushes `ghcr.io/<you>/shop-api:sha-<commit>`, outputs `digest`, and attests provenance). Local tools: `git`, Node.js 22 and `sha256sum` (Linux/macOS: `shasum -a 256`). Optional: `gh` CLI. Cost: none on a public repository.

1. **Feel what "same bytes" means with a plain artifact.** Package the app twice and compare:

```bash
npm pack --silent                       # creates shop-api-0.1.0.tgz
sha256sum shop-api-0.1.0.tgz | tee SHA256SUMS
sha256sum -c SHA256SUMS                 # shop-api-0.1.0.tgz: OK
tar tzf shop-api-0.1.0.tgz              # lists package/package.json and
  package/src/...
```

   The checksum is the artifact's identity. Verify it after every copy ("did it arrive unchanged?"). Note that a rebuild is not guaranteed to produce the same hash (timestamps, ordering) unless the build is **reproducible**; this is why you build once.

2. **Hand an artifact between jobs.** Save as `.github/workflows/build-handoff.yml`:

```yaml
name: build-handoff
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read
```

```yaml
jobs:
  build:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm pack
      - run: sha256sum shop-api-*.tgz | tee SHA256SUMS
      - uses: actions/upload-artifact@v4
        with:
          name: shop-api-${{ github.sha }}
          path: |
            shop-api-*.tgz
            SHA256SUMS
          retention-days: 7
          if-no-files-found: error

  verify-and-deploy:
    needs: build
    runs-on: ubuntu-24.04
    timeout-minutes: 5
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: shop-api-${{ github.sha }}
      - name: Verify checksum before using the artifact
        run: sha256sum -c SHA256SUMS
      - run: echo "deploying exactly the tested tarball"
```

The second job never rebuilds: it downloads, verifies the checksum, and uses the same file.

3. **Look at retention.** Open the run, scroll to **Artifacts** and note the size and expiry date. Under **Settings, Actions, General** see the repository's artifact and log retention setting.
4. **Promote an image by digest, not by rebuild.** Take the digest from the L5.4 run summary

(or read it):

```bash
IMG=ghcr.io/<you>/shop-api
DIGEST=$(docker buildx imagetools inspect $IMG:main --format '{{json .Manifest}}' | jq
  -r .digest)
echo "$DIGEST"                                  # sha256:...
docker buildx imagetools create -t $IMG:staging $IMG@$DIGEST
docker buildx imagetools inspect $IMG:staging --format '{{json .Manifest}}' | jq -r
  .digest   # identical digest
```

Both digests must be identical: the `staging` tag is just another name for the same manifest. (Output is similar to the above; the digest value is yours.)

5. **Make promotion a gated workflow.** Save as `.github/workflows/promote.yml`:

```yaml
name: promote
on:
  workflow_dispatch:
    inputs:
      sha:
        description: Full commit SHA to promote (image tag sha-<sha>)
        required: true
      target:
        type: choice
        options: [staging, production]
        default: staging

permissions:
  contents: read
  packages: write
  attestations: read

jobs:
  promote:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    environment: ${{ inputs.target }}          # production can require reviewers
    env:
      GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
    steps:
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Resolve digest of the tested image
        id: d
        env:
```

```
          SHA: ${{ inputs.sha }}
        run: |
          IMG=ghcr.io/${GITHUB_REPOSITORY,,}
          DIGEST=$(docker buildx imagetools inspect "$IMG:sha-$SHA" --format '{{json
  .Manifest}}' | jq -r .digest)
          echo "image=$IMG" >> "$GITHUB_OUTPUT"
          echo "digest=$DIGEST" >> "$GITHUB_OUTPUT"
      - name: Verify provenance (must come from this repo's workflow)
        env:
          IMAGE: ${{ steps.d.outputs.image }}
          DIGEST: ${{ steps.d.outputs.digest }}
        run: gh attestation verify "oci://$IMAGE@$DIGEST" --repo "$GITHUB_REPOSITORY"
      - name: Promote by retagging (no rebuild)
        env:
          IMAGE: ${{ steps.d.outputs.image }}
          DIGEST: ${{ steps.d.outputs.digest }}
          TARGET: ${{ inputs.target }}
        run: docker buildx imagetools create -t "$IMAGE:$TARGET" "$IMAGE@$DIGEST"
      - name: Record the promotion
        env:
          DIGEST: ${{ steps.d.outputs.digest }}
          SHA: ${{ inputs.sha }}
          TARGET: ${{ inputs.target }}
        run: echo "promoted $DIGEST (commit $SHA) to $TARGET by $GITHUB_ACTOR" >>
  "$GITHUB_STEP_SUMMARY"
```

Run it with a SHA from a merged build. If the image has no valid attestation (for example, it was pushed from a laptop), `gh attestation verify` fails and the job stops before any retag.

6. **Verify provenance by hand.**

```bash
gh attestation verify oci://ghcr.io/<you>/shop-api@$DIGEST --repo <you>/shop-api
```

On success it prints `Verification succeeded!` and the workflow, repository and commit that built the image. (Exact wording varies by `gh` version.)

7. **Protect the rollback set with a retention policy.** Retention rules act on tags, and one image can carry several tags (in L5.4 a release build has both `sha-<commit>` and `1.4.2`). A rule that expires old `sha-` images would therefore also expire old releases. The robust pattern is one repository per retention class: `shop-api` (every build, short retention) and

`shop-api-release` (production-promoted images, long retention), with promotion copying the digest from the first to the second. For the build repository, expire untagged images quickly and keep only the newest 50 `sha-` images:

```json
{
  "rules": [
    { "rulePriority": 1, "description": "expire untagged after 3 days",
      "selection": { "tagStatus": "untagged", "countType": "sinceImagePushed",
  "countUnit": "days", "countNumber": 3 },
      "action": { "type": "expire" } },
    { "rulePriority": 2, "description": "keep last 50 sha- images",
      "selection": { "tagStatus": "tagged", "tagPrefixList": ["sha-"], "countType":
  "imageCountMoreThan", "countNumber": 50 },
      "action": { "type": "expire" } }
  ]
}
```

Save it as `policy.json` and always **preview before applying**:

```bash
aws ecr start-lifecycle-policy-preview --repository-name shop-api --lifecycle-policy-
  text file://policy.json
aws ecr get-lifecycle-policy-preview --repository-name shop-api      # lists images
  that would expire
aws ecr put-lifecycle-policy --repository-name shop-api --lifecycle-policy-text
  file://policy.json
```

Cleanup of a sandbox repository: `aws ecr delete-repository --repository-name shop-api --force`.

7 Real-world scenario

(Illustrative.) A team's pipeline rebuilds the image in the production deploy job "so it can use production build args". One Thursday a deploy goes out and checkout latency doubles. Staging, tested an hour earlier, was fine. The investigation compares the two images: the `npm ci` layer in the production build pulled a patch release of a transitive dependency published between the two builds, because the base image tag `node:22` also moved. The two environments never ran the same code.

The fix: a single `build` job pushes `sha-<commit>`, the staging and production deploy jobs reference its digest, environment differences move into runtime configuration, and a `promote` workflow verifies provenance before retagging. Rollback becomes "promote the previous di-

gest". Staging results now predict production because they are the same bytes. **Lesson:** if you rebuild, you have two artifacts and one test result.

8 Common mistakes and best practices

Don't

- Rebuild per environment or per deploy.
- Bake environment-specific values into the artifact (use runtime config and secrets).
- Deploy by a movable tag (`latest`, `main`) and call it a release.
- Use workflow caches as the hand-off between jobs; use artifacts (or the registry).
- Let a cleanup job delete the version you are running or the previous one you would roll back to.
- Treat a green build as proof of provenance: an image pushed by a person looks the same until you verify the attestation.

Do

- Build once, record the digest, and promote that digest through gated stages.
- Store deployable artifacts in a registry/package repo with immutable versions; keep workflow artifacts for short-lived hand-offs and evidence.
- Verify checksums or attestations before using an artifact in a later stage.
- Define retention per artifact type, in code (lifecycle policy, scheduled cleanup).
- Generate and keep an SBOM and provenance with every release image.
- Keep a release record: digest, commit, who approved, when, where.

9 Tasks

- [ ] **T1 — Checksum a hand-off** *(≈30 min, beginner)*
  - **Goal:** pass a tarball between jobs and prove it is unchanged.
  - **Steps:**
    1. Create `build-handoff.yml` from step 2.
    2. Run it, then edit the workflow so the second job tampers with the file (`echo x >> shop-api-*.tgz`) before `sha256sum -c`.
  - **Done when:** the clean run passes and the tampered run fails with `FAILED` in the `sha256sum -c` step.
- [ ] **T2 — Promote by digest** *(≈30 min, intermediate)*

- **Goal:** retag an image without rebuilding.
- **Steps:**
  1. Run the `imagetools create` commands from step 4 against your GHCR image.
  2. Pull `:main` and `:staging` and compare `docker image inspect --format '{{.Id}}'`.
- **Done when:** both tags resolve to the same digest and no build job ran.

☐ **T3 — Gated promotion workflow** *(≈45 min, intermediate)*

- **Goal:** make promotion an auditable, approved action.
- **Steps:**
  1. Add `promote.yml` (step 5) and require reviewers on the `production` environment.
  2. Promote a SHA to `staging`, then to `production`.
  3. Open the run summary and find the line recording the digest.
- **Done when:** the production promotion waits for approval and the summary shows the promoted digest and commit.

☐ **T4 — Break it: promote an unattested image** *(≈45 min, advanced)*

- **Goal:** see provenance verification stop a bad artifact.
- **Steps:**
  1. Build a tiny image locally and push it to your GHCR repo as `sha-0000000000000000000000000000000000000000` (manual push with a PAT: `<YOUR_TOKEN>` with `write:packages`).
  2. Run `promote.yml` with that SHA.
  3. Observe the failure at `gh attestation verify`, then delete the test tag.
- **Done when:** the promote job fails before the retag step, and you can explain why an image built on a laptop is refused.

☐ **T5 — Retention that protects rollback** *(≈45 min, intermediate)*

- **Goal:** write a lifecycle policy and prove it with a preview.
- **Steps:**
  1. Write the ECR policy from step 7 (or a scheduled GHCR cleanup workflow that keeps the newest 30 `sha-` versions and all semver tags).
  2. Preview what it would delete (`aws ecr start-lifecycle-policy-preview`) or run the cleanup in dry-run mode.
  3. **Cleanup:** remove any sandbox repository you created.
- **Done when:** the preview lists only old `sha-` or untagged images from the build repository, and nothing from the release repository nor the currently deployed image.

- [ ] **T6 — Prove it: release record and rollback target** *(≈60 min, advanced)*
  - **Goal:** answer "what is in production and what do we roll back to?" from artifacts alone.
  - **Steps:**
    1. Without the guide, extend `promote.yml` so every promotion appends `environment, digest, commit, actor, time` to a `deployments.md` release record (or creates a GitHub Deployment via the API).
    2. Promote three different SHAs in sequence.
    3. Explain in writing why you promote by digest and not rebuild, and how the record gives you the rollback target.
  - **Done when:** the record lists the three promotions in order and its second-newest entry still exists in the registry and verifies.

10 Self-check questions

1. What does "build once, deploy many" mean, and what changes between environments?
2. Why is a digest a better identity than a tag?
3. What is the difference between a cache and an artifact?
4. What does an attestation prove that a successful build log does not?
5. What retention rules would you set for PR images, `main` builds and release tags?
6. A promote job runs `docker build` again from the same commit. What is wrong?

ANSWERS

1. The artifact (image) is created once and the same digest is deployed everywhere; only configuration and secrets change per environment.
2. A tag can be moved to different content; a digest is the hash of the content, so it identifies exactly one artifact.
3. A cache is an optional speed-up that may be evicted; an artifact is a required product of the build consumed by later stages or kept as a release.
4. That the image with this digest was produced by a specific workflow in a specific repository from a specific commit, signed so it cannot be forged after the fact.
5. PR images: days. `main` SHA images: newest N or ~30 days, never the deployed or previous release. Release tags: long, often a year or more, matching rollback and audit needs.
6. The rebuild can differ (new base image or dependency versions, timestamps), so what is deployed is not what was tested; it must retag or copy the existing digest.

11 Further reading

- GitHub Docs: *Storing workflow data as artifacts*, *Using artifact attestations to establish provenance for builds*, `actions/upload-artifact`, `actions/download-artifact`.
- SLSA specification (slsa.dev) and Sigstore documentation (docs.sigstore.dev, `cosign`).
- Docker Docs: *Attestations* (SBOM and provenance) and `docker buildx imagetools`.
- *Continuous Delivery* by Jez Humble and David Farley: the chapters on the deployment pipeline and "build your binaries only once".
- Next: Rollback §5.7. Related: Docker builds and registries §5.4, Environment Promotion in GitOps §17.4.

LAB NOTES What I ran, what I saw, what it means

LEVEL 5: CI/CD

5.7 Rollback

Topic 7 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

A **rollback** returns production to the last known-good version after a bad release. In a container-based pipeline that means "run the previous image again": the old artifact is still in the registry, so the operation is a deploy of an older tag or digest, not a rebuild. The alternative is a **roll-forward**: leave the bad version running and ship a new fix as quickly as possible.

Four ideas make rollback dependable instead of heroic:

- **Versioned, immutable releases** — every release has an unambiguous identity (`sha-<commit>`, `1.4.2`, a digest) and stays in the registry (L5.4, L5.6).
- **Rollback is a normal deploy** — it uses the same pipeline and the same script, so it is rehearsed every day.
- **Rollback triggers** — rules (a failed health check, a spike in 5xx errors, a person) that decide *when* to go back, ideally automatically.
- **State compatibility** — code rolls back easily, **data does not**. A migration that dropped a column cannot be undone by redeploying old code (L5.9).

2 Why it matters in DevOps

- **Mean time to recovery (MTTR):** the fastest way to end a customer-visible incident is usually to undo the last change. Teams that can roll back in minutes tolerate failures far better than teams that debug live in production.
- **Confidence to ship:** frequent small deploys are only safe if each one is cheaply reversible.
- **Rollback is a feature you must test.** An untested rollback path fails exactly when you need it (old image garbage-collected, config incompatible, migration irreversible).
- **Not every incident is a rollback.** Knowing when to roll forward (a data migration already ran; the fix is a one-liner) is part of the skill.

3 Real-world system example

Amazon. The Amazon Builders' Library article *"Automating safe, hands-off deployments"* describes pipelines that deploy in waves, watch alarms during a bake period, and **roll back automatically** when an alarm fires, with the rollback itself being just a deployment of the previous version through the same system. Making that path common, and not an emergency procedure, is the point.

Kubernetes and Helm. Kubernetes `Deployment` objects keep a history of ReplicaSets (controlled by `revisionHistoryLimit`) so `kubectl rollout undo` can return to a previous revision. Helm records every release revision, and `helm rollback <release> <revision>` re-applies an earlier one. Both are built-in expressions of the "previous version is kept and addressable" principle (Releases §9.4).

Knight Capital, August 2012 (a cautionary tale). A trading firm deployed new software to its servers, but one of eight servers did not receive the new code, and a repurposed flag caused that server to run old, dormant logic. In roughly 45 minutes the firm lost on the order of 440 million US dollars (the figure the firm reported at the time; the SEC documented the incident in a public 2013 order). Reports of the incident also note that reverting the new code on the other servers made things worse, because the old code contained the faulty behaviour. The lessons are well known: deploys must be consistent and verified on every host, dead code must be removed, and **rolling back is not automatically safe**. You need to know what the previous version does in the *current* environment.

4 Diagram

Automatic rollback in a pipeline:

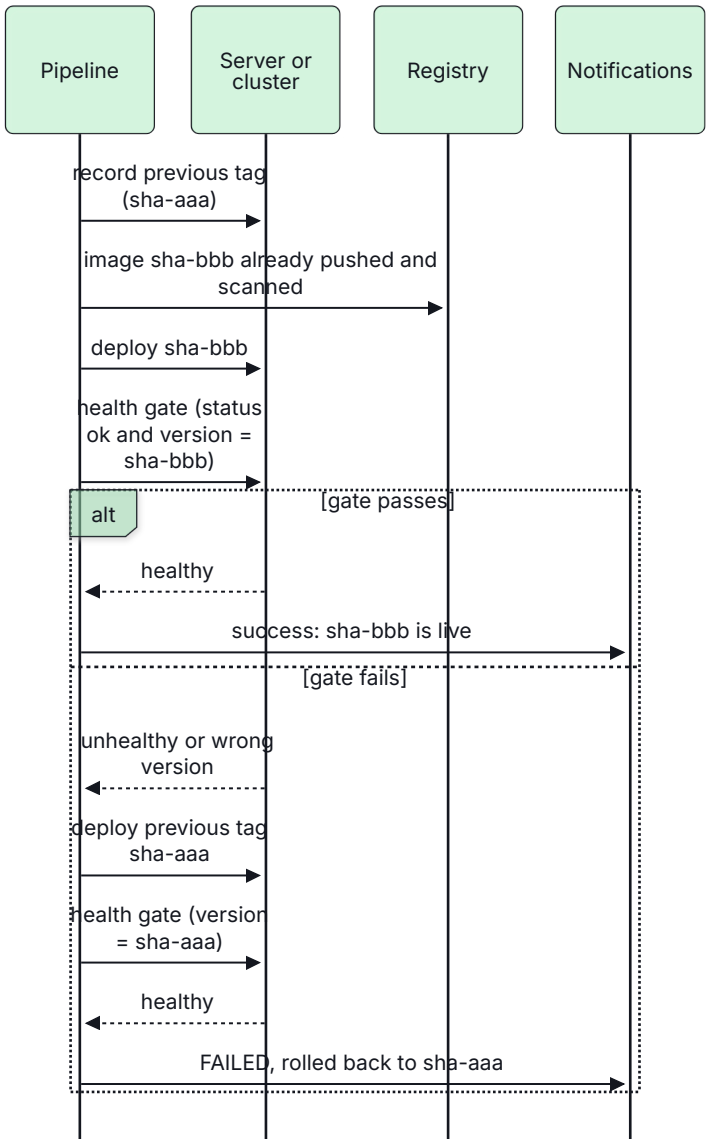


Figure 5.7-1

Rollback or roll forward?

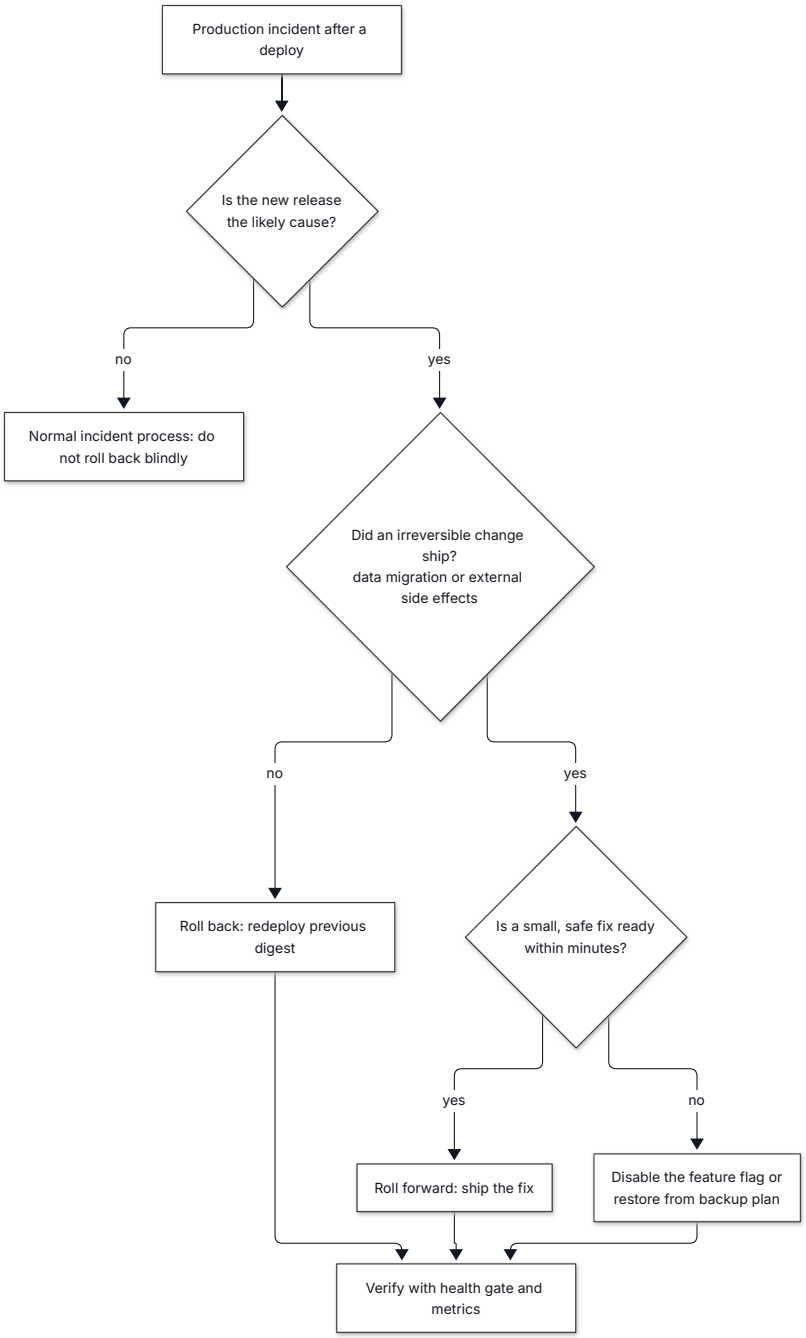


Figure 5.7-2

5 How it works

What you roll back, and how

Layer	Rollback action	Speed	Watch out for
Container image on a host	redeploy previous tag/digest (`IMAGE_TAG=<old> docker compose up -d`)	seconds to minutes	old image must still exist; config must match the old version
Kubernetes Deployment	`kubectl rollout undo deployment/shop-api` (or to a specific revision)	seconds to minutes	`revisionHistoryLimit`; GitOps controllers may re-apply the bad version
Helm release	`helm rollback shop-api <revision> --wait`	minutes	rolls back chart state, not the database
Blue/green	switch traffic back to the old environment	seconds	old environment must be kept warm (L5.11)
Canary	set canary weight to 0 / abort	seconds	only the canary slice was affected (L5.12)
Feature flag	turn the flag off	seconds, no deploy	flag must be in place before release
GitOps	`git revert` the change and let the controller reconcile	minutes	the audit trail is the Git history (Rollback through Git §17.6)
Database	restore from backup / point-in-time recovery, or forward-fix	slow, risky	avoid needing this: design migrations to be backwards compatible

Rollback triggers

Trigger	Detected by	Action
Container fails its health check / does not start	orchestrator, deploy script	automatic rollback
Smoke test fails after deploy (wrong version, `/healthz` not ok)	pipeline health gate	automatic rollback
Error rate or latency SLO burn during the bake period	monitoring alarm (Prometheus/CloudWatch)	automatic rollback, or page then manual
Business metric drops (checkout conversions)	dashboard / alert	human decision
Security issue found in the release	scanner, report	human decision; may need roll forward
"Something feels wrong"	a person	one-click rollback workflow

An automatic trigger needs a **bake time**: after a deploy, watch for several minutes, because many failures (memory leaks, a slow dependency) appear only under real traffic. See Observability §0.10 for choosing signals, and SLOs and error budgets §14.4.

Making rollback safe

1. **Keep the previous N artifacts**, and never let retention delete the running or previous release (L5.6).
2. **Record the deployed version** on the host or in a deployment record: `current.tag`, `previous.tag`, a GitHub Deployment, a Helm revision.
3. **Make the app report its version** in `/healthz` (`{"status":"ok","version":"<sha>"}`) so the gate can prove the *new* code is serving, not the old container that is still up.
4. **Backwards-compatible schema changes.** Version N-1 of the code must run on the schema left by version N. Use expand/contract (L5.9).
5. **Config compatibility.** A new required environment variable breaks a rollback if the old version cannot start with the new config (it usually can; check removed variables).
6. **Rehearse.** Roll back in staging every release, or on a schedule.
7. **Prefer small releases.** The smaller the change, the easier to decide and the less data it can touch.

Rollback versus roll-forward

	Rollback	Roll-forward
What you do	redeploy the previous good version	write, test and deploy a fix
Time to relief	minutes, predictable	depends on the fix, usually longer
Risk	old code vs new data/config	new, less-tested code under pressure
Choose it when	the cause is the release and nothing irreversible shipped	a migration or external side effect already happened; or the fix is trivial and safe

Even when you roll forward, the fix goes through the normal pipeline. The emergency path should be a faster *path through the same gates*, not a way around them.

6 Step-by-step guide

Prerequisites: a Linux VM or VPS (a free tier or a local VM is fine) with Docker Engine and the Compose plugin, a registry holding at least two `shop-api` images (for example `sha-aaa...` and `sha-bbb...` from L5.4), and `jq` and `curl` on the server. The app exposes `GET`

`/healthz` returning `{"status":"ok","version":"<tag>"}`. Cost: nothing beyond the VM. The deploy script below was tested for its control flow (success, failure and rollback paths) against stubbed `docker` and `curl`; running it for real needs your server.

1. **Describe the stack with an `IMAGE_TAG` variable and a health check.** Save as `/opt/shop-api/compose.yaml`:

```yaml
services:
  api:
    image: ghcr.io/acme/shop-api:${IMAGE_TAG:?set IMAGE_TAG}
    restart: unless-stopped
    env_file: .env
    ports:
      - "127.0.0.1:3000:3000"
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://127.0.0.1:3000/healthz"]
      interval: 5s
      timeout: 3s
      retries: 6
      start_period: 10s
```

2. **Write the deploy script with automatic rollback.** Save as `/opt/shop-api/deploy.sh` and `chmod +x` it:

```bash
#!/usr/bin/env bash
# Deploy a tag with automatic rollback. Usage: deploy.sh <image-tag>
set -euo pipefail

NEW_TAG="${1:?usage: deploy.sh <image-tag>}"
APP_DIR="${APP_DIR:-/opt/shop-api}"
HEALTH_URL="${HEALTH_URL:-http://127.0.0.1:3000/healthz}"
cd "$APP_DIR"

PREV_TAG="$(cat current.tag 2>/dev/null || true)"
echo "previous=${PREV_TAG:-none} new=$NEW_TAG"

start() {   # start <tag>: bring the stack up on that tag and wait for the container
  healthcheck
  IMAGE_TAG="$1" docker compose up -d --wait --wait-timeout 90
}

healthy() { # healthy <tag>: the app must answer ok AND report the version we just
  deployed
  for _ in $(seq 1 20); do
    if curl -fsS --max-time 3 "$HEALTH_URL" | jq -e --arg v "$1" '.status == "ok" and
  .version == $v' >/dev/null; then
      return 0
```

```
    fi
    sleep 3
  done
  return 1
}

if start "$NEW_TAG" && healthy "$NEW_TAG"; then
  echo "$NEW_TAG" > current.tag
  [ -n "$PREV_TAG" ] && echo "$PREV_TAG" > previous.tag
  echo "DEPLOY OK: $NEW_TAG"
  exit 0
fi

echo "DEPLOY FAILED: $NEW_TAG" >&2
if [ -n "$PREV_TAG" ]; then
  echo "rolling back to $PREV_TAG" >&2
  start "$PREV_TAG"
  if healthy "$PREV_TAG"; then
    echo "ROLLED BACK to $PREV_TAG" >&2
    exit 2        # 2 = failed but rolled back cleanly
  fi
  echo "ROLLBACK ALSO UNHEALTHY" >&2
  exit 3          # 3 = page a human
fi
exit 1
```

Exit codes carry meaning: `0` success, `2` failed and safely rolled back, `3` rollback also failed (page someone), `1` failed with nothing to roll back to.

3. **First deploy, then a good upgrade.**

```bash
cd /opt/shop-api
./deploy.sh sha-aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa     # use your real tags
./deploy.sh sha-bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
cat current.tag previous.tag
```

Expected output ends with `DEPLOY OK: sha-bbbb...`; `current.tag` holds `sha-bbbb...`, `previous.tag` holds `sha-aaaa...`.

4. **Simulate a bad release and watch the automatic rollback.** Build a deliberately broken image (for example one that exits at startup when `FORCE_FAIL=1` is set, or whose `/healthz` returns 500) and push it as `sha-cccc...`; then:

```bash
./deploy.sh sha-cccccccccccccccccccccccccccccccccccccccc; echo "exit=$?"
cat current.tag
curl -s http://127.0.0.1:3000/healthz
```

Expected: `DEPLOY FAILED`, `rolling back to sha-bbbb...`, `ROLLED BACK`, `exit=2`; `current.tag` still `sha-bbbb...`; `/healthz` shows the `bbbb` version. Production kept serving the previous version.

5. **Add the deploy job with the rollback-aware notification.** Fragment of `.github/workflows/deploy.yml` (secrets are per environment, see L5.5):

```yaml
name: deploy
on:
  workflow_dispatch:
    inputs:
      tag:
        description: Image tag to deploy (sha-<commit>)
        required: true
  workflow_call:
    inputs:
      tag:
        type: string
        required: true

concurrency:
  group: deploy-production
  cancel-in-progress: false

permissions:
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    environment: production
    steps:
      - uses: actions/checkout@v4
      - name: Configure SSH
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.SSH_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519 && chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts
      - name: Deploy with automatic rollback
        id: deploy
        env:
          TAG: ${{ inputs.tag }}
          HOST: ${{ vars.DEPLOY_HOST }}
        run: |
          set +e
```

```
          ssh "deploy@$HOST" "APP_DIR=/opt/shop-api bash -s -- '$TAG'" <
  scripts/deploy.sh
          rc=$?
          echo "rc=$rc" >> "$GITHUB_OUTPUT"
          exit $rc
      - name: Report
        if: always()
        env:
          RC: ${{ steps.deploy.outputs.rc }}
        run: |
          case "$RC" in
            0) echo "Deployed ${{ inputs.tag }}" >> "$GITHUB_STEP_SUMMARY" ;;
            2) echo "FAILED and rolled back to the previous version" >>
  "$GITHUB_STEP_SUMMARY" ;;
            3) echo "FAILED and rollback unhealthy: PAGE A HUMAN" >>
  "$GITHUB_STEP_SUMMARY" ;;
            *) echo "FAILED, nothing to roll back to (rc=$RC)" >>
  "$GITHUB_STEP_SUMMARY" ;;
          esac
```

The job is red whenever the new version did not go live (exit codes 1, 2, 3), but the summary tells the on-call engineer whether production is safe. `scripts/deploy.sh` is the script from step 2 kept in the repository so the pipeline and the server never drift.

6. **Provide the human rollback button.** Rollback is the same workflow with an older tag. Add a `rollback.yml` that is a thin wrapper:

```yaml
name: rollback
on:
  workflow_dispatch:
    inputs:
      tag:
        description: Previously released tag to restore (see Deployments or
  previous.tag)
        required: true
jobs:
  rollback:
    uses: ./.github/workflows/deploy.yml
    with:
      tag: ${{ inputs.tag }}
    secrets: inherit
```

The `deploy.yml` from step 5 already declares `workflow_call`, so the wrapper reuses it unchanged: the same script, gates and notifications run for a rollback as for a release. Run **Actions, rollback, Run workflow** with the previous tag from `previous.tag` (or from the deployment history).

7. **Kubernetes version (if you use it).**

```bash
kubectl rollout history deployment/shop-api
kubectl rollout undo deployment/shop-api                    # previous revision
kubectl rollout undo deployment/shop-api --to-revision=3    # a specific one
kubectl rollout status deployment/shop-api --timeout=120s
# Helm:
helm history shop-api
helm rollback shop-api 3 --wait --timeout 2m
```

If Argo CD or Flux manages the Deployment, undoing by hand will be reverted by the controller on the next sync; roll back by reverting the Git commit instead (L17 §17.6).

7 Real-world scenario

(Illustrative.) At 14:05 a `shop-api` release adds a new `discount_code` lookup. Tests passed, but in production the lookup runs a query without an index. The health check passes (it only checks the process is up), and by 14:11 the p95 latency of `/checkout` has risen from 180 ms to 4 s and orders are timing out. The alert fires.

The on-call engineer checks the deploy log: the only change in the last hour is this release, and its migration was additive (a new nullable column), so the old code runs fine on the new schema. They run the rollback workflow with the previous tag; the same deploy script brings up the old image, the version-aware health gate confirms it, and latency is back to normal at 14:15, ten minutes after the first symptom. The next day the team adds the index, writes a test with a realistic data volume, and extends the post-deploy gate to query a synthetic checkout and compare latency with a threshold. **Lesson:** a cheap, rehearsed rollback converts an outage into a short blip; and the health gate should test what customers do, not only that the process is alive.

8 Common mistakes and best practices

Don't

- Roll back by rebuilding an old commit; redeploy the existing artifact.
- Let retention delete old images, or use `latest`, so there is nothing to go back to.
- Write irreversible migrations in the same release as the code that needs them.
- Treat "container started" as success; check the served version and a real request.
- Roll back by hand on a GitOps-managed cluster without reverting Git (the controller will undo you).

- Assume rollback is always safe: check whether data, caches, queues or external systems changed.

Do

- Record `current` and `previous` versions (and who deployed them) on every deploy.
- Automate the common case: failed health gate means automatic rollback and a notification.
- Keep rollback as a normal pipeline path with a one-click manual trigger.
- Bake after deploy, and wire rollback to SLO-based alarms where you can.
- Use backwards-compatible schema changes and feature flags to decouple deploy from release.
- Practise rollbacks, and review the decision (rollback vs roll-forward) in post-incident reviews.

9 Tasks

- [ ] **T1 — Version-aware health endpoint** *(≈30 min, beginner)*
  - **Goal:** let the pipeline prove which version is serving.
  - **Steps:**
    1. Make `/healthz` return `{"status":"ok","version":"<value of APP_VERSION>"}` and set `APP_VERSION` from the image tag in the Dockerfile/compose file.
    2. Run two containers with different `APP_VERSION` values and `curl` each.
  - **Done when:** `curl -s localhost:3000/healthz | jq .version` prints the tag you started, and differs between runs.
- [ ] **T2 — Deploy with `current.tag` / `previous.tag`** *(≈45 min, intermediate)*
  - **Goal:** a scripted deploy that records history.
  - **Steps:**
    1. Install `compose.yaml` and `deploy.sh` on a VM or locally (use `HEALTH_URL` and a local registry or local images).
    2. Deploy tag A, then tag B.
  - **Done when:** `cat current.tag previous.tag` shows B then A and `/healthz` reports B.
- [ ] **T3 — Break it: bad release, automatic rollback** *(≈45 min, intermediate)*
  - **Goal:** watch the gate catch a failure and restore service.
  - **Steps:**

1. Create image C that fails (exits at start, or serves `/healthz` 500, or reports the wrong version).
2. Run `./deploy.sh C` while polling the old endpoint in another terminal (`while sleep 1; do curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/healthz; done`).
3. Note the exit code and the length of any gap.

- **Done when:** exit code is `2`, `current.tag` is unchanged, `/healthz` reports B, and you can state how many seconds of errors users would have seen.

- [ ] **T4 — One-click rollback workflow** *(≈45 min, intermediate)*
  - **Goal:** give the on-call a button.
  - **Steps:**
    1. Create `deploy.yml` (with `workflow_call` and `workflow_dispatch`) and `rollback.yml` as in steps 5 and 6, using a local runner or `echo` stand-ins for SSH if you have no server.
    2. Trigger a rollback to a previous tag from the Actions tab.
  - **Done when:** the run summary shows the older tag deployed and the history in `current.tag`/the run list lets you see who rolled back and when.

- [ ] **T5 — Rollback vs roll-forward decision drill** *(≈40 min, advanced)*
  - **Goal:** practise the judgment, not just the command.
  - **Steps:**
    1. Create a release whose migration drops a column (do it on a scratch database).
    2. Redeploy the previous code and observe the failure.
    3. Write which option you would choose for (a) a CSS bug, (b) a bad index, (c) a dropped column, and the reasoning in two lines each.
  - **Done when:** you can show the failing rollback and your decision notes mention schema compatibility and data.

- [ ] **T6 — Prove it: timed rollback drill** *(≈60 min, advanced)*
  - **Goal:** make rollback boring.
  - **Steps:**
    1. Without the guide, have a teammate (or your future self) break a release in staging in a way you do not know in advance.
    2. Detect it from the pipeline/monitoring signal, decide rollback or roll-forward, and recover. Time it from first signal to healthy.
    3. Write a five-line runbook: trigger, decision, command, verification, follow-up.

- **Done when:** staging is healthy on the previous version, the elapsed time is recorded, and the runbook exists in the repository.

10 Self-check questions

1. Why is rollback faster and safer when you deploy immutable, versioned images?
2. What must `/healthz` return for a deploy gate to detect "the old container is still up"?
3. When is rolling forward better than rolling back?
4. What can make a code rollback fail even though the old image is intact?
5. On a cluster managed by Argo CD, why does `kubectl rollout undo` not stick?
6. Name three automatic rollback triggers.

ANSWERS

1. The previous version already exists exactly as it ran before; redeploying it needs no build and carries no new surprises.
2. The deployed version identifier (for example the commit SHA) in addition to a healthy status, and the gate must compare it with the version it just deployed.
3. When irreversible changes already shipped (a destructive migration, external side effects) or the fix is small and quick; when the old version cannot run on the current data.
4. An incompatible database schema or data, changed or removed configuration, a cache or message format the old code cannot read, or the old image having been deleted.
5. The controller continuously reconciles the cluster to the desired state in Git and reverts manual changes; revert the Git change instead.
6. Failed container health check, failed post-deploy smoke test or version mismatch, and error-rate or latency alarm breach during the bake time.

11 Further reading

- Amazon Builders' Library: *Automating safe, hands-off deployments* and *Ensuring rollback safety during deployments*.
- Kubernetes Docs: *Deployments* (rolling back a Deployment); Helm Docs: `helm rollback` and `helm history`.
- *Site Reliability Engineering* (Google) and *The SRE Workbook*: chapters on release engineering and canarying.
- SEC Administrative Proceeding on Knight Capital Americas LLC (2013), the public record of the August 2012 incident.
- Next: Zero-downtime deployments §5.8. Related: Deployment Strategies §0.7.

5.8 Zero-downtime Deployments

Topic 8 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

A **zero-downtime deployment** replaces the running version of a service with a new one without users noticing: no refused connections, no `502` errors, no dropped requests, no "we are down for maintenance" page. The old version keeps serving until the new version is **ready**, and the old version is only stopped after it has finished the work it already accepted.

It rests on four cooperating mechanisms, all of which you must get right:

1. **Readiness** — the new instance tells the platform when it can take traffic (for example `GET /readyz` returns 200 only after the database pool is connected).
2. **Health gates** — the pipeline (or orchestrator) refuses to move on until readiness and a real request succeed for the *new* version.
3. **Connection draining** — before an old instance stops, the load balancer stops sending it *new* requests while existing requests finish.
4. **Graceful shutdown** — the old process handles the termination signal (`SIGTERM`), finishes in-flight requests, closes connections and exits cleanly.

How the instances are swapped (rolling, blue/green, canary) is covered in L5.10, L5.11 and L5.12. Those strategies only deliver zero downtime if the four mechanisms above work.

2 Why it matters in DevOps

- **Deploy frequency:** if every deploy drops a few seconds of requests, teams deploy at night and rarely. If deploys are invisible, teams deploy ten times a day.
- **Hidden errors:** a deploy that causes 0.5% errors for 30 seconds is easy to miss in dashboards but it breaks real checkouts, retried webhooks and payments.
- **Error budgets:** every dropped request spends your SLO error budget (Error Budgets §14.4).
- **Most "mystery 502s at deploy time" have one of three causes:** the app ignores `SIGTERM`; the load balancer still routes to an instance that is shutting down; or the new instance receives traffic before it is ready.

3 Real-world system example

Kubernetes pod termination. The Kubernetes documentation describes the sequence exactly: when a pod is deleted, it is marked `Terminating`, a `preStop` hook (if defined) runs, the container receives `SIGTERM`, and after `terminationGracePeriodSeconds` (30 seconds by default) it gets `SIGKILL`. At the same time the pod is removed from Service endpoints. These two things happen **concurrently**, which is why well-run services add a short `preStop` sleep or keep serving for a few seconds after `SIGTERM`: the endpoint removal needs time to propagate to proxies and load balancers. A Deployment's rolling update then uses readiness probes plus `maxUnavailable: 0` to keep capacity during the swap.

Nginx. `nginx -s reload` is the classic graceful reload: the master process starts new worker processes with the new configuration and tells the old workers to stop accepting new connections and exit after finishing their current ones. It is the same drain-then-exit idea at the process level.

AWS load balancers. Application Load Balancers use a target-group **deregistration delay** (300 seconds by default): when a target is deregistered, it enters `draining`, receives no new requests, and in-flight requests may finish until the delay expires. ECS and CodeDeploy rely on this and on health checks to shift traffic safely.

4 Diagram

What happens to one instance during a graceful shutdown (the order matters):

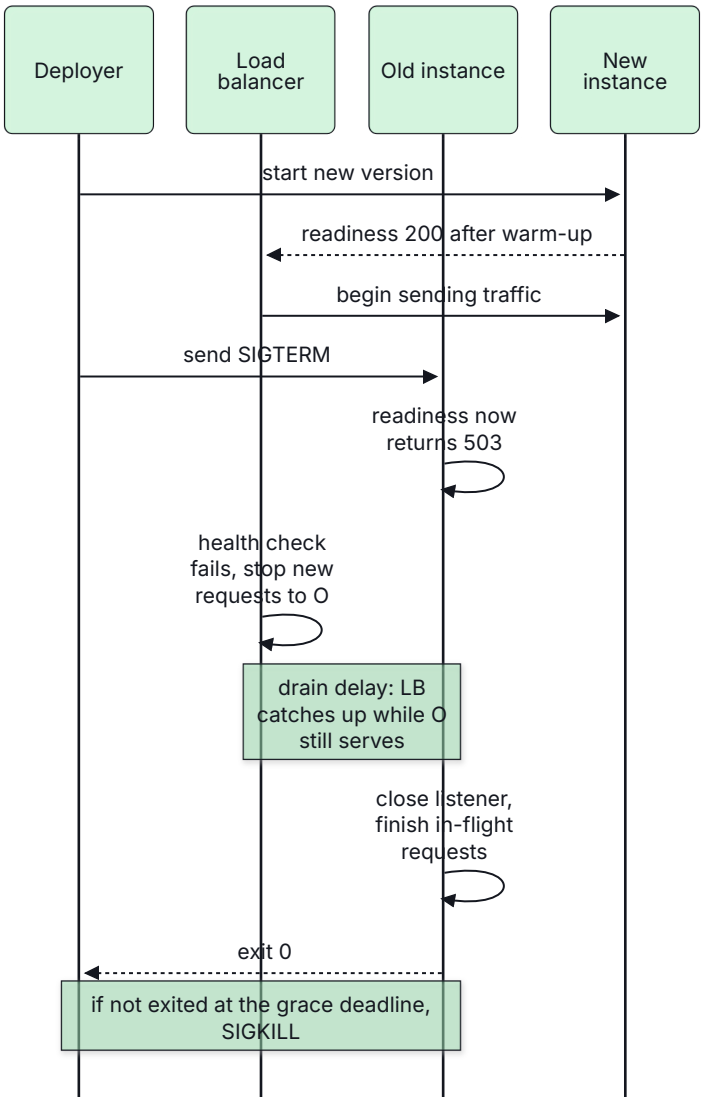


Figure 5.8-1

Where zero downtime is lost:

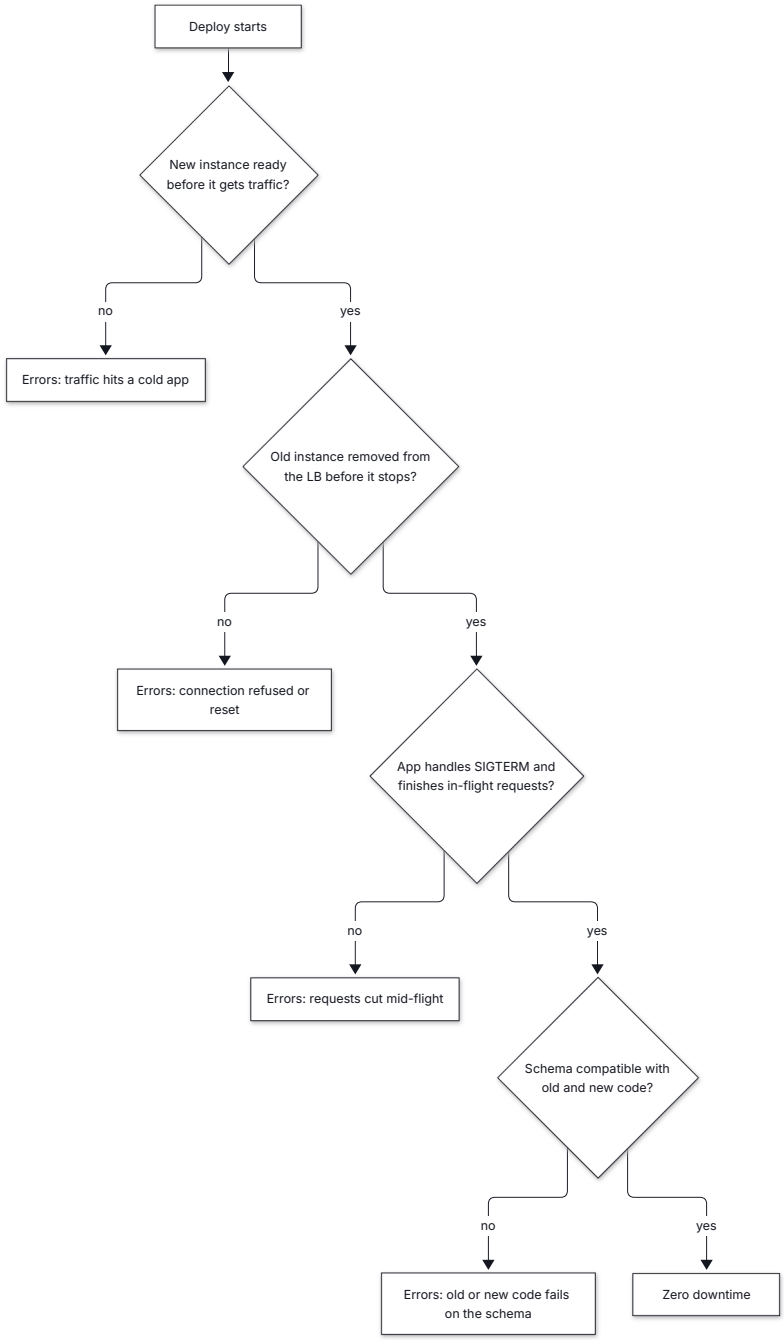


Figure 5.8-2

5 How it works

Liveness, readiness and startup are different questions

Check	Question	If it fails	Typical content
Liveness (`/livez`)	Is the process stuck beyond recovery?	restart the container	returns 200 if the event loop runs; **never** checks the database (a DB outage must not restart-loop every pod)
Readiness (`/readyz`)	Can this instance take traffic *right now*?	remove from load balancing, do not restart	DB pool connected, caches warmed, not shutting down
Startup	Has the slow initialisation finished?	delay the other probes	used for apps with long boot times
Deploy health gate (pipeline)	Is the *new version* actually serving correct responses?	roll back	`status ok` , `version == deployed tag` , a smoke request, N consecutive successes

See Health Checks §4.12 and Liveness and Readiness Probes §8.15.

The graceful shutdown recipe

1. Receive `SIGTERM` (Processes and Signals §1.8). Your process must be PID 1 in its container *or* receive forwarded signals: use the exec form `CMD ["node", "src/server.js"]` , never `CMD npm start` (npm does not forward signals), or add an init such as `docker run --init` .
2. **Immediately fail readiness** (return 503) so the load balancer stops choosing this instance.
3. **Keep serving for a short drain delay** (a few seconds) because the load balancer needs time to notice. Skipping this is the number one cause of deploy-time errors.
4. **Stop accepting new connections** (`server.close()`), finish in-flight requests, close idle keep-alive sockets, then close database pools and flush logs and metrics.
5. Exit `0` before the platform's kill deadline. Set the app's own hard timeout *below* it (`docker stop` waits 10 seconds by default; Kubernetes 30 seconds; ECS and others have `stopTimeout`).

Connection draining at each layer

Layer	Setting	Notes
Docker / Compose	`stop_grace_period: 30s`	time between `SIGTERM` and `SIGKILL`
Kubernetes	`terminationGracePeriodSeconds` , `lifecycle.preStop` , readiness probe	grace period must exceed `preStop` + drain + longest request

Layer	Setting	Notes
AWS ALB	target group deregistration delay	default 300 seconds; lower it if your requests are short
Nginx	`nginx -s reload` / `quit`	old workers finish current connections
HAProxy / Envoy	drain a server (`set server ... state drain`) / hot restart	stop new sessions, keep existing
HTTP keep-alive	server closes connection after current response	long-lived keep-alive connections will otherwise stay on the old instance

Capacity and rollout settings (Kubernetes example)

YAML

```yaml
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1            # start one extra new pod
      maxUnavailable: 0      # never drop below 3 ready pods
  template:
    spec:
      terminationGracePeriodSeconds: 40
      containers:
        - name: api
          readinessProbe:
            httpGet: { path: /readyz, port: 3000 }
            periodSeconds: 5
            failureThreshold: 2
          lifecycle:
            preStop:
              exec: { command: ["sleep", "10"] }   # let endpoints propagate before
  SIGTERM is handled
```

`maxUnavailable: 0` with `maxSurge: 1` means a new pod must become **ready** before an old one is removed. A `PodDisruptionBudget` protects capacity during node drains, which are a different kind of "deploy". (Newer Kubernetes versions also offer a built-in sleep action for `preStop`; check your version's documentation.)

Database compatibility

During a deploy both versions run at once, against the *same* database. The schema must work for N-1 and N code. That is the expand/contract discipline of L5.9. Also check: message formats on queues, cache key formats, and API contracts between services.

6 Step-by-step guide

Prerequisites: Node.js 22, `curl`, and Nginx installed locally (Debian/Ubuntu: `sudo apt-get install -y nginx`; the commands below run Nginx from a private directory and do not touch the system service). No cloud account. The server and Nginx sequence below was run in a sandbox; request counts will differ on your machine ("similar to").

1. **Write an app that shuts down gracefully.** Save as `server.js`:

```javascript
const http = require("node:http");

let ready = false;          // readiness: may we receive new traffic?
let shuttingDown = false;
let inFlight = 0;

const server = http.createServer((req, res) => {
  if (req.url === "/livez") {            // liveness: process alive, no dependency
  checks
    res.writeHead(200).end("alive\n");
    return;
  }
  if (req.url === "/readyz") {           // readiness: flips to 503 as soon as
  shutdown starts
    const ok = ready && !shuttingDown;
    res.writeHead(ok ? 200 : 503, { "content-type": "application/json" });
    res.end(JSON.stringify({ status: ok ? "ok" : "draining", version:
  process.env.APP_VERSION || "dev" }) + "\n");
    return;
  }
  if (req.url.startsWith("/slow")) {     // simulates a 3 second request
    inFlight++;
    setTimeout(() => { inFlight--; res.end("slow done\n"); }, 3000);
    return;
  }
  res.end("hello\n");
});

const PORT = Number(process.env.PORT || 3000);
server.listen(PORT, () => {
  ready = true;
  console.log("listening on", PORT, "version", process.env.APP_VERSION || "dev");
});

function shutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;                   // 1. fail readiness first
  const drain = Number(process.env.DRAIN_MS || 5000);
  console.log(`${signal} received: readiness=503, waiting ${drain}ms for the load
  balancer`);
  setTimeout(() => {                     // 2. keep serving while the LB notices
```

```
    console.log("closing server, in-flight requests:", inFlight);
    server.close(() => {                // 3. stop accepting, wait for in-flight
  requests
      console.log("all connections closed, exiting 0");
      process.exit(0);
    });
    server.closeIdleConnections();      // 4. drop idle keep-alive sockets so close()
  can finish
  }, drain);
  setTimeout(() => {                    // 5. hard deadline, below the platform kill
  timeout
    console.error("forced exit after timeout");
    process.exit(1);
  }, Number(process.env.FORCE_MS || 25000)).unref();
}
process.on("SIGTERM", () => shutdown("SIGTERM"));
process.on("SIGINT", () => shutdown("SIGINT"));
```

2. **Prove the in-flight request survives `SIGTERM`.** Run the server in one terminal, then in a second:

```bash
APP_VERSION=v2 DRAIN_MS=2000 node server.js &
PID=$!
curl -s -w ' [%{http_code}]\n' localhost:3000/readyz        #
  {"status":"ok","version":"v2"} [200]
curl -s -w ' [%{http_code}]\n' localhost:3000/slow &         # a 3 second request
  starts
sleep 0.5; kill -TERM $PID
sleep 0.3
curl -s -w ' [%{http_code}]\n' localhost:3000/readyz        #
  {"status":"draining",...} [503]
curl -s -w ' [%{http_code}]\n' localhost:3000/livez         # alive [200]
wait
```

   Expected: the `/slow` request still prints `slow done [200]` after about 3 seconds, `/readyz` flips to `503 draining` immediately, `/livez` stays 200, and the server log says `SIGTERM received...`, `closing server, in-flight requests: 1`, `all connections closed, exiting 0`.

3. **Prove what happens without a handler (the broken version).**

```bash
node -e 'require("node:http").createServer((q,r)=>setTimeout(()=>r.end("slow
  done\n"),3000)).listen(3001)' &
P=$!; sleep 0.7
( curl -s -w ' [%{http_code}]\n' localhost:3001/ ; echo "curl exit=$?" ) &
sleep 0.5; kill -TERM $P; wait
```

The request fails with `[000]` and `curl: (52) Empty reply from server` (similar output). Node's default `SIGTERM` behaviour is to exit immediately; the customer's request is cut.

4. **Run two instances behind Nginx.** Create a private Nginx directory and config:

```bash
mkdir -p nginx/logs nginx/tmp
cat > nginx/nginx.conf <<EOF
worker_processes 1;
pid $PWD/nginx/nginx.pid;
error_log $PWD/nginx/logs/error.log warn;
events { worker_connections 256; }
http {
  access_log $PWD/nginx/logs/access.log;
  client_body_temp_path $PWD/nginx/tmp/body; proxy_temp_path $PWD/nginx/tmp/proxy;
  fastcgi_temp_path $PWD/nginx/tmp/fcgi; uwsgi_temp_path $PWD/nginx/tmp/uwsgi;
  scgi_temp_path $PWD/nginx/tmp/scgi;
  upstream api {
    server 127.0.0.1:3000 max_fails=0;
    server 127.0.0.1:3001 max_fails=0;
  }
  server {
    listen 8080;
    location / {
      proxy_pass http://api;
      proxy_next_upstream error timeout http_502 http_503;
      proxy_connect_timeout 1s;
    }
  }
}
EOF
nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx
PORT=3000 APP_VERSION=v1 DRAIN_MS=2000 node server.js > a.log 2>&1 &  A=$!
PORT=3001 APP_VERSION=v1 DRAIN_MS=2000 node server.js > b.log 2>&1 &  B=$!
sleep 1; curl -s localhost:8080/readyz
```

`proxy_next_upstream` makes Nginx retry another server on connection errors and `503`, which is how open-source Nginx copes with a draining instance (it has only *passive* health checks). `max_fails=0` disables its "mark the server down" behaviour; see the note after step 6.

5. **Create constant traffic and record every status code.** Save as `load.sh` (`chmod +x`

`load.sh`):

```bash
#!/usr/bin/env bash
# print one status code per request for N seconds
end=$((SECONDS+$1))
while [ $SECONDS -lt $end ]; do
  curl -s -o /dev/null -w '%{http_code}\n' --max-time 5 http://127.0.0.1:8080/
  sleep 0.1
done
```

6. **Do a rolling restart under load and count errors.** Run it in the same shell as step 4 (it uses `$A` and `$B`):

```bash
./load.sh 16 > codes.txt &  L=$!
sleep 1
# replace instance A (v1 → v2): SIGTERM, wait for the clean exit, start v2, wait
  until ready
kill -TERM $A; while kill -0 $A 2>/dev/null; do sleep 0.2; done
PORT=3000 APP_VERSION=v2 DRAIN_MS=2000 node server.js > a2.log 2>&1 &  A=$!
until curl -sf localhost:3000/readyz >/dev/null; do sleep 0.2; done
# replace instance B
kill -TERM $B; while kill -0 $B 2>/dev/null; do sleep 0.2; done
PORT=3001 APP_VERSION=v2 DRAIN_MS=2000 node server.js > b2.log 2>&1 &  B=$!
until curl -sf localhost:3001/readyz >/dev/null; do sleep 0.2; done
wait $L
sort codes.txt | uniq -c
```

(Keep using the same shell as step 4 so that `$A` and `$B` still hold the process IDs.) Expected result is similar to `134 200` and **no other codes**: both instances were replaced while traffic flowed, and the readiness-then-drain-then-exit sequence meant no request failed.

> **What went wrong in my first attempts:** with Nginx's default `max_fails=1 fail_timeout=10s`, a single `503` from a draining instance marks that server unavailable for 10 seconds. When the second instance started draining, Nginx had *no live upstream* and returned `502 no live upstreams`; in the sandbox this produced a burst of `502` responses. Managed load balancers (ALB, Kubernetes Services) use active health checks and readiness instead. With plain Nginx, either use `max_fails=0` as above, run enough instances, or use Nginx Plus / HAProxy active checks.

7. **Add the same checks to the pipeline as a deploy health gate.** After the deploy step, require several consecutive successes for the *new* version:

```yaml
      - name: Health gate
        env:
          URL: ${{ vars.HEALTH_URL }}        # e.g. https://staging.example.com/readyz
          WANT: ${{ inputs.tag }}
        run: |
          ok=0
          for i in $(seq 1 30); do
            if curl -fsS --max-time 3 "$URL" | jq -e --arg v "$WANT" '.status == "ok"
  and .version == $v' >/dev/null; then
              ok=$((ok+1))
            else
              ok=0
            fi
            [ "$ok" -ge 5 ] && echo "healthy: $ok consecutive successes" && exit 0
            sleep 2
          done
          echo "health gate FAILED" >&2
          exit 1
```

Requiring consecutive successes avoids a false pass while a load balancer is still rotating between old and new instances. Pair the gate with automatic rollback (L5.7).

8. **Clean up.** `kill -TERM $A $B` and `nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx -s quit`.

7 Real-world scenario

(Illustrative.) Every `shop-api` deploy produces a handful of `502` errors for about 10 seconds. The team has two instances behind a load balancer and a rolling script that runs `docker stop` on one container at a time. The logs show clients receiving `connection reset` exactly when containers stop. Investigation: the container's command is `npm start`, so the `SIGTERM` goes to `npm`, not to Node, which is killed by `SIGKILL` after the 10 second grace period, cutting requests. The load balancer's health check runs every 15 seconds with two failures required, so for up to 30 seconds it keeps sending traffic to the stopped instance.

Fixes: change to `CMD ["node", "src/server.js"]`; add the `SIGTERM` handler (fail readiness, wait 5 seconds, close the server); lower the health-check interval to 5 seconds with `unhealthy threshold 2`; raise `stop_grace_period` to 30 seconds; and add a pipeline gate that checks `/readyz` for the new version. The next deploy shows zero non-2xx responses in the load balancer metrics. **Lesson:** zero downtime is not a property of the deployment tool; it is the cooperation of app, load balancer and pipeline.

8 Common mistakes and best practices

Don't

- Start sending traffic to an instance because its process is running (rather than ready).
- Run the app as `npm start` or via a shell script that swallows `SIGTERM`.
- Make liveness depend on the database or other services; an outage will restart every healthy pod.
- Stop an instance before the load balancer has stopped routing to it.
- Set the grace period shorter than the longest legitimate request plus the drain delay.
- Ship a migration that breaks the previous version while the deploy is in progress.

Do

- Implement `/livez` (cheap) and `/readyz` (real dependencies and shutdown awareness), and include the version in the response.
- On `SIGTERM`: fail readiness, keep serving for a drain delay, finish in-flight requests, exit 0.
- Keep spare capacity during the swap (`maxUnavailable: 0`, surge by one).
- Gate the pipeline on the new version being healthy for several consecutive checks; roll back automatically on failure.
- Test deploys under load (a loop of `curl` is enough) and count non-2xx responses.
- Make the application and schema compatible across adjacent versions (L5.9).

9 Tasks

- [ ] **T1 — Graceful shutdown** *(≈45 min, beginner)*
  - **Goal:** an in-flight request survives `SIGTERM`.
  - **Steps:**
    1. Create `server.js` from step 1 and run the demonstration in step 2.
    2. Read the server log lines in order.
  - **Done when:** the slow request returns `200` after the signal, `/readyz` returned `503` immediately, and the log shows `exiting 0`.
- [ ] **T2 — Break it: no handler** *(≈30 min, beginner)*
  - **Goal:** see the failure that graceful shutdown prevents.
  - **Steps:**
    1. Run the handler-less server from step 3 and send `SIGTERM` during a request.

2. Then run it with `CMD npm start` in a container (`docker run`, then `docker stop`) and time how long the stop takes.

- **Done when:** you have a `curl` failure and a stop time of about 10 seconds, and can explain that npm did not forward the signal.

- [ ] **T3 — Rolling restart under load** *(≈60 min, intermediate)*
  - **Goal:** deploy two instances with zero failed requests.
  - **Steps:**
    1. Set up Nginx and two instances (steps 4 to 6).
    2. Run `load.sh` and perform the rolling restart.
    3. Count status codes with `sort | uniq -c`.
  - **Done when:** every recorded status code is `200`.

- [ ] **T4 — Break it: skip the drain** *(≈45 min, intermediate)*
  - **Goal:** understand why the drain delay and readiness matter.
  - **Steps:**
    1. Run the rolling restart with `kill -9` instead of `SIGTERM` (or `DRAIN_MS=0`) and the default Nginx `max_fails` / `fail_timeout` (remove `max_fails=0`).
    2. Record the status codes; read `nginx/logs/error.log`.
    3. Restore the correct sequence and re-measure.
  - **Done when:** you have two `uniq -c` outputs (with and without the correct sequence) and an explanation of each error line.

- [ ] **T5 — Kubernetes rollout with `maxUnavailable: 0`** *(≈75 min, advanced; kind or minikube, free)*
  - **Goal:** the same guarantee under an orchestrator.
  - **Steps:**
    1. Create a kind cluster, a Deployment of three `shop-api` pods with the readiness probe, `maxSurge: 1`, `maxUnavailable: 0`, a `preStop` sleep and `terminationGracePeriodSeconds: 40`.
    2. Expose it with a Service and port-forward; run the `curl` loop against it.
    3. Update the image tag (`kubectl set image`) and watch `kubectl rollout status`.
    4. **Cleanup:** `kind delete cluster`.
  - **Done when:** the loop shows only `200` during the rollout and `kubectl get pods -w` shows new pods becoming Ready before old ones terminate.

- [ ] **T6 — Prove it: a deploy gate that catches a bad release** *(≈60 min, advanced)*

- **Goal:** make the pipeline refuse a release that cannot serve traffic.
- **Steps:**
  1. Without the guide, add the health gate of step 7 to a deploy workflow.
  2. Deploy a good version and a deliberately bad one (wrong version string, or `/readyz` returning 500).
  3. Explain in writing what the gate checks, why it needs consecutive successes, and what your pipeline does on failure.
- **Done when:** the good version passes, the bad version fails the gate and triggers rollback, and the workflow log shows the consecutive-success counting.

10 Self-check questions

1. Why should liveness not check the database, while readiness may?
2. Why does a graceful shutdown keep serving for a few seconds after `SIGTERM` instead of closing the listener at once?
3. What does `npm start` as a container command do to `SIGTERM`?
4. What do `maxSurge: 1` and `maxUnavailable: 0` guarantee during a Kubernetes rolling update?
5. Why must a deploy health gate check the version, not only the status?
6. Why does zero-downtime deployment depend on backwards-compatible database changes?

ANSWERS

1. A failing liveness check restarts the container; a database outage would restart every pod, making things worse. A failing readiness check only removes the pod from traffic, which is the correct reaction to "dependency down".
2. The load balancer needs time (health-check interval, endpoint propagation) to stop sending new requests. Closing immediately causes refused connections for requests already routed to this instance.
3. `npm` becomes the main process and does not forward the signal to Node, so the app never runs its shutdown logic and is killed by `SIGKILL` after the grace period.
4. The controller creates one extra new pod and never lets the number of ready pods fall below the desired count: a new pod must be Ready before an old one is removed.
5. The old instance may still answer `ok`; only the version proves that the new code is the one serving.
6. Old and new application versions run against the same database at the same time during the swap, so the schema must work for both.

11 Further reading

- Kubernetes Docs: *Pod Lifecycle* (termination of pods), *Configure Liveness, Readiness and Startup Probes*, *Performing a Rolling Update*.
- Nginx Docs: *Controlling nginx* (`reload`, `quit`) and `ngx_http_upstream_module` (`max_fails`, `proxy_next_upstream`).
- AWS Docs: *Target groups for your Application Load Balancers* (deregistration delay) and *Health checks*.
- Node.js Docs: `http.Server.close()` and `closeIdleConnections()`.
- Next: Database migrations in pipelines §5.9. Related: Health Checks §4.12, Deployment Strategies §0.7.

LAB NOTES What I ran, what I saw, what it means

5.9 Database Migrations in Pipelines

Topic 9 of 12 | ~6 hours | Difficulty: Advanced

1 What it is

A **database migration** is a versioned, repeatable script that changes a database schema (tables, columns, indexes, constraints) or its data, from one version to the next. A **migration tool** (Flyway, Prisma Migrate, Alembic, Liquibase, Rails/Django migrations) keeps a table in the database (for example `flyway_schema_history`) that records which migrations have run, so applying "all pending migrations" is safe to repeat and every environment ends in the same state.

In a pipeline the problem is not *running* a migration; it is **timing and compatibility**. Code and schema do not change at the same instant:

- During a rolling, blue/green or canary deploy, **old and new application versions run at the same time against one database**.
- A rollback puts the **old code back against a schema the new code has already changed**.

So every migration must be compatible with the version before it and the version after it. The standard technique is **expand and contract** (also called "parallel change"): first *expand* the schema in a backwards-compatible way, deploy code that works with both shapes, migrate the data, and only later *contract* by removing what is no longer used.

2 Why it matters in DevOps

- **Database changes are the main reason "rollback" and "zero downtime" fail.** Code is stateless and replaceable; data is not (L5.7, L5.8).
- **Locks cause outages without any error in the deploy.** An innocent `ALTER TABLE` on a busy table can queue behind a long transaction and then block *every* query on that table behind it.
- **Table rewrites are slow at scale:** a change that takes 20 ms on a laptop with 1,000 rows can hold a lock for many minutes on a table with 200 million.

- **Automation without discipline makes mistakes fast:** a pipeline that applies a destructive migration to production on every merge needs review rules, previews and a plan for failure.

3 Real-world system example

Stripe. Stripe's engineering blog post *"Online migrations at scale"* describes migrating a very large datastore with no downtime using a four-step pattern: **dual write** to the old and new structures, **change reads** to the new one, **change writes** to the new one only, then **remove the old** data. The pipeline is expand and contract applied to a huge, live database, with backfills done gradually and verification between steps.

GitHub and `gh-ost`. GitHub open-sourced `gh-ost`, an online schema-change tool for MySQL that builds a shadow copy of a table, applies ongoing changes to it by reading the binary log, and swaps it in at the end, avoiding the long locks of a direct `ALTER`. (Percona's `pt-online-schema-change` takes a similar approach with triggers.) PostgreSQL gets much of this behaviour natively through `CREATE INDEX CONCURRENTLY` and by making many `ALTER TABLE` forms metadata-only.

GitLab. GitLab's public development guidelines have a long section on migrations: a separation between regular migrations and **post-deployment migrations** (run after the new code is live), requirements that migrations be backwards compatible and run within a time budget, and rules to avoid long locks. The Ruby on Rails ecosystem has a popular gem, `strong_migrations`, that blocks unsafe operations in review for the same reasons.

4 Diagram

Expand and contract across three releases (the schema is always compatible with the code that runs against it):

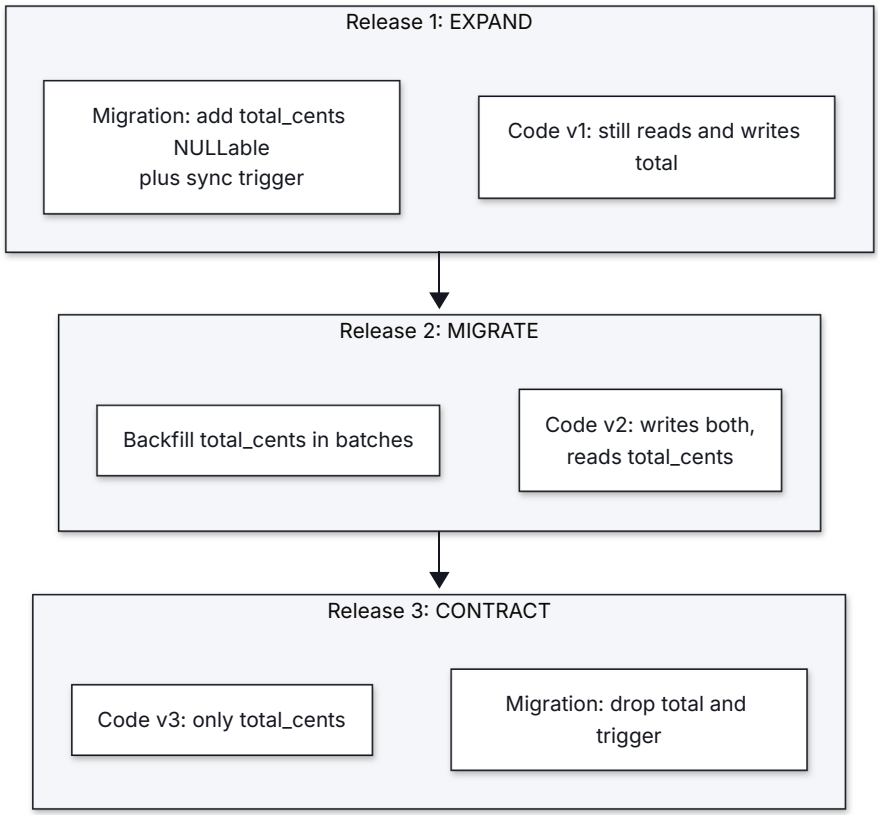


Figure 5.9-1

Why a rename in one step breaks a rolling deploy:

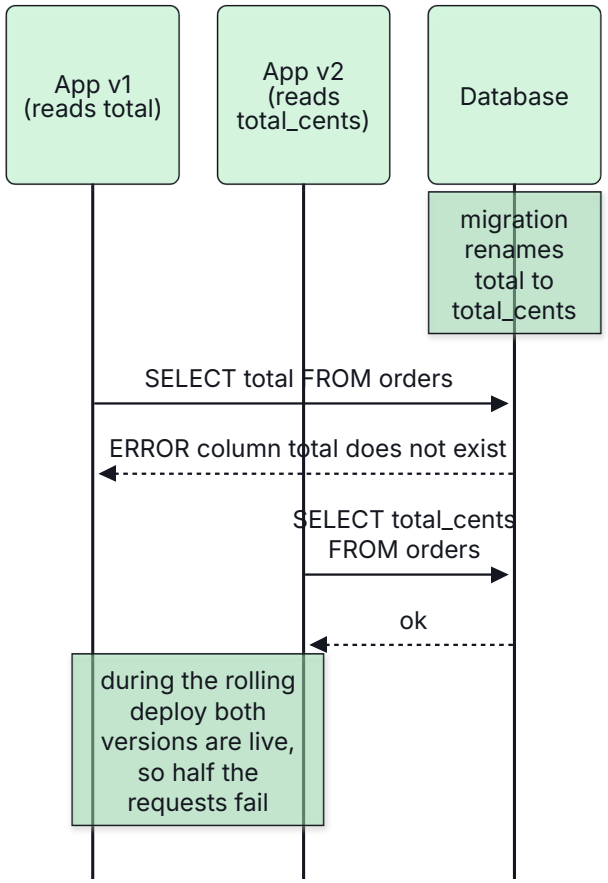


Figure 5.9-2

The migration step inside the pipeline:

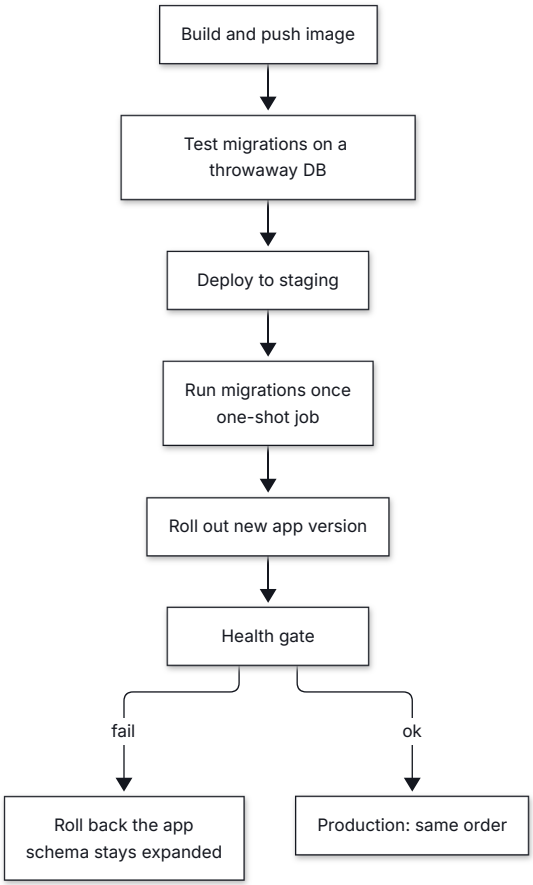


Figure 5.9-3

5 How it works

Safe and unsafe changes (PostgreSQL)

Change	Safe in one step?	Why / safer way
Add a nullable column	Yes	Metadata-only, very fast
Add a column with a constant `DEFAULT`	Yes on PostgreSQL 11+	Stored as metadata, no rewrite (older versions rewrite the table)
Add `NOT NULL` to an existing column	No (scans the table under lock)	`ADD CONSTRAINT ... CHECK (col IS NOT NULL) NOT VALID`, then `VALIDATE CONSTRAINT`, then `SET NOT NULL` (PostgreSQL 12+ reuses the validated check)

Change	Safe in one step?	Why / safer way
Add an index	No (blocks writes)	`CREATE INDEX CONCURRENTLY` (cannot run inside a transaction block)
Add a foreign key	No (locks both tables while validating)	`ADD CONSTRAINT ... NOT VALID`, then `VALIDATE CONSTRAINT`
Rename a column or table	**No**	Breaks old code; add new, dual write, switch, drop old
Change a column type	**No**	Often rewrites the table; add new column, backfill, switch, drop
Drop a column or table	**Only after** no deployed code uses it	Contract step in a later release
Backfill millions of rows	No in one statement	Batches of 1,000 to 20,000 rows with pauses; one transaction each

The locking pitfall

Most `ALTER TABLE` statements need an `ACCESS EXCLUSIVE` lock, which conflicts with every other lock. If a long-running query or an open transaction holds any lock on the table, the `ALTER` **waits**, and while it waits, **every new query on that table queues behind it**, including plain `SELECT`s. A migration that waits 30 seconds for an idle-in-transaction session can therefore stall the whole application. Defences:

1. `SET lock_timeout = '3s';` at the top of risky migrations: fail fast instead of queueing everyone; retry the migration.
2. `SET statement_timeout` for the statements that should be quick.
3. Run migrations at a low-traffic time *and* keep application transactions short; alert on `idle in transaction` sessions.
4. Prefer metadata-only operations and the `NOT VALID` / `CONCURRENTLY` patterns above.
5. Test on a copy of production-sized data, not only on an empty database.

Tools compared

Tool	Migration format	Typical pipeline command	Notes
Flyway	SQL files `V2__add_total_cents.sql` (plus repeatable `R__` and Java)	`flyway migrate`, `flyway validate`, `flyway info`	Versioned SQL, simple and explicit; checksum validation detects edited, already-applied files.

Tool	Migration format	Typical pipeline command	Notes
Prisma Migrate	SQL generated from `schema.prisma` into `prisma/migrations/<timestamp>_name/`	`prisma migrate deploy` (CI and production), `prisma migrate dev` (local only)	Review and, when needed, hand-edit the generated SQL (`--create-only`). Never run `migrate dev` or `migrate reset` against production.
Alembic (Python/SQLAlchemy)	Python revision files with `upgrade()` and `downgrade()`	`alembic upgrade head`, `alembic current`, `alembic history`	Autogenerate is a draft only. `postgresql_concurrently=True` index builds need an `autocommit_block()`.
Liquibase	XML, YAML or SQL changelogs	`liquibase update`	Supports rollback scripts and contexts.

Common rules, whatever the tool:

- **Never edit a migration that has been applied anywhere shared**; add a new one. Tools verify checksums and will refuse.
- **One tool, one source of truth;** the migrations live in the same repository and are reviewed like code.
- **Run migrations once per deploy, not at app startup in every replica.** Parallel replicas race, a slow migration makes health checks fail, and an app crash loop can leave a half-applied change. Use a one-shot step: a pipeline job on a runner that can reach the database, a `migrate` container run before the app, a Kubernetes Job or Helm `pre-upgrade` hook, or an ECS task.
- **Prefer forward fixes over down-migrations.** Down scripts for data-destroying changes are fiction; keep the rollback story as "old code works with the expanded schema".
- **Give the migration account its own credentials:** DDL rights for the migrator, not for the running app (L5.5).

Migration-aware pipeline stages

1. **PR:** start a throwaway Postgres (a service container), apply *all* migrations from scratch, then run the test suite. Also apply only the *new* migration to a database built from the main branch.
2. **Compatibility test:** run the previous release's test suite (or smoke tests) against the schema after the new migration; this proves N-1 code works with N schema, the rollback case.
3. **Lint** migrations (for example with a SQL migration linter such as `squawk` for PostgreSQL, or code-review rules) to flag blocking operations.
4. **Staging:** run the same migration job as production against a production-like dataset.

5. **Production:** migration job, then rollout, then health gate. Contract migrations run in a **later** release.

6 Step-by-step guide

Prerequisites: PostgreSQL 14 or newer and `psql`. Fastest: `docker run --rm -d --name pg -e POSTGRES_PASSWORD=<YOUR_PASSWORD> -p 5432:5432 postgres:16` (then `psql "postgresql://postgres:<YOUR_PASSWORD>@localhost:5432/postgres"`), or a local installation. Optional for the tool parts: Docker (for Flyway), Node.js (Prisma) or Python 3 (Alembic). The SQL in steps 1 to 5 was run against PostgreSQL 16; timings will differ on your machine ("similar to").

1. **Create the starting schema and 100,000 rows.** Save as `seed.sql` and run `psql ... -d postgres -c "create database shop"` then `psql ... -d shop -f seed.sql`:

```sql
create table orders (
  id          bigserial primary key,
  customer_id integer not null,
  total       numeric(10,2) not null,
  status      text not null default 'new',
  created_at  timestamptz not null default now()
);
insert into orders (customer_id, total)
select (random()*1000)::int, (random()*200)::numeric(10,2) from generate_series(1,
  100000);
```

Goal of the scenario: change money from `total numeric` to integer `total_cents` (store cents to avoid rounding), without downtime. A direct `RENAME` / `TYPE` change is exactly what we avoid.

2. **Release 1, EXPAND.** Save as `db/migration/V2__expand_total_cents.sql`:

```sql
-- additive and nullable: old code is unaffected
ALTER TABLE orders ADD COLUMN total_cents bigint;

-- old code only writes "total": keep the new column in sync for its writes
CREATE OR REPLACE FUNCTION orders_sync_total() RETURNS trigger AS $$
BEGIN
  IF NEW.total_cents IS NULL THEN
    NEW.total_cents := (NEW.total * 100)::bigint;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER orders_sync_total_trg BEFORE INSERT OR UPDATE ON orders
  FOR EACH ROW EXECUTE FUNCTION orders_sync_total();
```

   Apply it with `psql -d shop -f db/migration/V2__expand_total_cents.sql`. Then prove old code still works and new rows are filled:

```sql
insert into orders (customer_id, total) values (1, 12.34) returning id, total,
  total_cents;
```

   Expected: `total_cents` is `1234` even though the insert only mentioned `total`.

3. **Release 2, MIGRATE: backfill in batches, then validate.** Run repeatedly (a loop in a script or a job) until `remaining` is 0:

```sql
UPDATE orders
SET total_cents = (total * 100)::bigint
WHERE id IN (SELECT id FROM orders WHERE total_cents IS NULL ORDER BY id LIMIT 20000);
```

```bash
for i in $(seq 1 6); do psql -d shop -qc "UPDATE orders SET total_cents =
  (total*100)::bigint WHERE id IN (SELECT id FROM orders WHERE total_cents IS NULL
  ORDER BY id LIMIT 20000)"; sleep 1; done
psql -d shop -c "select count(*) filter (where total_cents is null) as remaining from
  orders"
```

Expected: `remaining` is `0`. Small batches keep each transaction short, so locks and replication lag stay small. Then make the column `NOT NULL` without a long scan under lock, in `V3__total_cents_not_null.sql`:

```sql
ALTER TABLE orders ADD CONSTRAINT orders_total_cents_nn CHECK (total_cents IS NOT
  NULL) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT orders_total_cents_nn;   -- scans, but allows
  concurrent writes
ALTER TABLE orders ALTER COLUMN total_cents SET NOT NULL;       -- instant: PostgreSQL
  12+ uses the validated check
ALTER TABLE orders DROP CONSTRAINT orders_total_cents_nn;
```

Deploy application v2 now: it writes both `total` and `total_cents` and reads `total_cents`. Rolling back to v1 is safe at every point, because `total` is still there and still correct.

4. **Release 3, CONTRACT** (only after v1 is gone everywhere and the rollback window has passed). Save as `V4__contract_drop_total.sql`:

```sql
DROP TRIGGER orders_sync_total_trg ON orders;
DROP FUNCTION orders_sync_total();
ALTER TABLE orders DROP COLUMN total;
```

Prove the danger of doing this too early: after it, run the old query `select total from orders;`. It fails with `ERROR: column "total" does not exist`; any still-running v1 instance, or a rollback to v1, would hit this on every request.

5. **See the lock-queue problem yourself.** Open three `psql` sessions on database `shop`:

```sql
-- session 1: an "innocent" long transaction
begin; select count(*) from orders; -- leave it open (or: select pg_sleep(60);)

-- session 2: a migration with no protection (here limited to 5 s so you can observe)
set lock_timeout = '5s';
alter table orders add column note text;     -- WAITS

-- session 3: a plain read, started while session 2 is waiting
select 1 from orders limit 1;                -- also WAITS, behind the ALTER
```

In a fourth session `select pid, left(query,40), wait_event_type, wait_event from pg_stat_activity where datname='shop';` shows the `ALTER` and the `SELECT` both waiting on `Lock / relation`. After 5 seconds session 2 fails with `ERROR: canceling statement due`

`to lock timeout` and session 3 immediately completes. In my run the plain `SELECT` was delayed by about 4 seconds by a migration that did nothing. With `lock_timeout` the blast radius is bounded; without it, it lasts as long as session 1 stays open. Finish with `rollback;` in session 1.

6. **Index without blocking writes.** In a transaction block it is refused; outside it works:

```sql
begin; create index concurrently idx_orders_customer on orders(customer_id); commit;
-- ERROR:  CREATE INDEX CONCURRENTLY cannot run inside a transaction block
create index concurrently if not exists idx_orders_customer on orders(customer_id);
```

Keep such a statement alone in its own migration file. Tools differ on how they wrap migrations in transactions: with Flyway on PostgreSQL, a script that contains only the concurrent statement is the safe layout (it can also be set to run outside a transaction in the script configuration; check the Flyway documentation for your version).

7. **Run migrations with a tool, as a one-shot step.** With Flyway in Docker (files named `V<version>__<description>.sql`; note the double underscore):

```bash
docker run --rm --network host \
  -v "$PWD/db/migration:/flyway/sql:ro" \
  -e FLYWAY_URL=jdbc:postgresql://localhost:5432/shop \
  -e FLYWAY_USER=postgres -e FLYWAY_PASSWORD=<YOUR_PASSWORD> \
  flyway/flyway:11 info
docker run --rm --network host -v "$PWD/db/migration:/flyway/sql:ro" \
  -e FLYWAY_URL=jdbc:postgresql://localhost:5432/shop -e FLYWAY_USER=postgres -e
  FLYWAY_PASSWORD=<YOUR_PASSWORD> \
  flyway/flyway:11 migrate
```

`info` lists each version as `Pending` / `Success`; `migrate` applies pending ones in order and records them in `flyway_schema_history`. Editing an applied file makes the next run fail checksum validation; that is deliberate. Use a fresh database for this step: put the step 1

seed into `db/migration/V1__init.sql` so Flyway applies V1 to V4 itself. (To adopt Flyway on an existing database, run it once with `-baselineOnMigrate=true -baselineVersion=1`.) Equivalents:

```bash
# Prisma: develop locally, deploy in CI/production
npx prisma migrate dev --create-only --name expand_total_cents   # generate SQL, then review/edit it
npx prisma migrate deploy                                         # apply pending migrations, no shadow DB, no reset

# Alembic
alembic revision -m "expand total_cents"
alembic upgrade head
alembic current
```

In Alembic, a safe index build looks like this (Alembic 1.12+ for `if_not_exists`):

```python
def upgrade():
    with op.get_context().autocommit_block():
        op.create_index("idx_orders_customer", "orders", ["customer_id"],
                        postgresql_concurrently=True, if_not_exists=True)
```

8. **Wire the migration into the pipeline.** Migrations run as their own job *before* the rollout, and PRs test them on a throwaway database:

```yaml
name: migrations-check
on:
  pull_request:
    paths: ["db/migration/**"]

permissions:
  contents: read

jobs:
  migrate-from-scratch:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: ci-only-password
          POSTGRES_DB: shop
        ports: ["5432:5432"]
        options: >-
```

```
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-timeout 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - name: Apply all migrations with Flyway
        run: |
          docker run --rm --network host \
            -v "$PWD/db/migration:/flyway/sql:ro" \
            -e FLYWAY_URL=jdbc:postgresql://localhost:5432/shop \
            -e FLYWAY_USER=postgres -e FLYWAY_PASSWORD=ci-only-password \
            flyway/flyway:11 migrate
      - name: Check for blocking operations (warn on risky DDL)
        run: |
          if grep -Eiq 'ALTER TABLE .* (DROP COLUMN|RENAME|ALTER COLUMN .* TYPE)'
  db/migration/*.sql; then
            echo "::warning::migration contains DROP/RENAME/TYPE change: confirm
  expand/contract is respected"
          fi
```

And in the deploy job, run the migration against the target environment *before* the app rollout (from a runner that can reach the database, or via SSH/`docker compose run --rm migrate` on the host), and fail the deployment if it fails:

```yaml
services:
  migrate:
    image: flyway/flyway:11
    command: -connectRetries=20 migrate
    environment:
      FLYWAY_URL: jdbc:postgresql://db:5432/shop
      FLYWAY_USER: ${DB_MIGRATOR_USER}
      FLYWAY_PASSWORD: ${DB_MIGRATOR_PASSWORD}
    volumes: ["./db/migration:/flyway/sql:ro"]
    profiles: ["migrate"]
```

`docker compose --profile migrate run --rm migrate` runs it once and exits.

9. **Clean up.** `docker rm -f pg` (or `drop database shop;` on a local server).

7 Real-world scenario

(Illustrative.) A developer needs `orders.total` as integer cents. Their migration is one line: `ALTER TABLE orders RENAME COLUMN total TO total_cents;`, and the code change is merged with it. The pipeline is green: tests run against a fresh database that has the new schema. The deploy is a rolling update over four instances. Within seconds the first new pod becomes ready and the migration has already run, so the three remaining old pods start returning `500: column "total" does not exist` for every checkout. The health gate fails, the

pipeline rolls the application back, but the rolled-back old code still hits the renamed column, so rollback does not help. The on-call engineer restores service by adding a view/column alias manually (`ALTER TABLE orders ADD COLUMN total numeric GENERATED ALWAYS AS (total_cents / 100.0) STORED`) at 02:10; recovery takes about 25 minutes.

Follow-up: the team adopts expand/contract for all schema changes, adds a `migrations-check` PR job, runs migrations as a separate pre-rollout job with `lock_timeout`, and writes the rule in `CONTRIBUTING.md`: "A migration must work with the previous release's code. Rename, drop and type changes take three releases." **Lesson:** green tests on a fresh database prove nothing about *mixed-version* operation; compatibility between adjacent versions has to be designed and tested.

8 Common mistakes and best practices

Don't

- Rename or drop columns, or change types, in the same release as the code that stops using them.
- Edit a migration file that has already been applied in any shared environment.
- Run migrations from every app replica at startup.
- Run `ALTER TABLE` on a busy table without `lock_timeout`, or with a long transaction open.
- Add indexes without `CONCURRENTLY` on large production tables.
- Backfill millions of rows in a single statement or transaction.
- Run `prisma migrate dev` / `reset` or auto-generated destructive Alembic output against production.
- Rely on down-migrations as your rollback plan.

Do

- Use expand, migrate, contract across separate releases; document which phase each migration is in.
- Keep migrations small, idempotent where practical, and reviewed; test them from scratch and from the previous release's schema.
- Run migrations once, as a dedicated, retryable pipeline step that fails the deploy on error.
- Set `lock_timeout` (and `statement_timeout`) and retry; monitor `idle in transaction` sessions.
- Use separate credentials: a migrator with DDL rights, an application user with only DML.
- Take a backup or snapshot before risky production changes (Backups and PITR §13.5).

- Test on production-sized data in staging so lock and timing surprises appear there.

9 Tasks

- [ ] **T1 — Expand with a trigger and a backfill** *(≈45 min, intermediate)*
  - **Goal:** add `total_cents` without breaking old writes.
  - **Steps:**
    1. Build the `orders` table with 100,000 rows (step 1).
    2. Apply `V2__expand_total_cents.sql` and insert a row using only `total`.
    3. Backfill existing rows in batches.
  - **Done when:** `select count(*) from orders where total_cents is null` returns `0` and the inserted row has the right `total_cents`.
- [ ] **T2 — Add NOT NULL without a long lock** *(≈30 min, intermediate)*
  - **Goal:** use `NOT VALID` then `VALIDATE`.
  - **Steps:**
    1. Run the three-statement pattern from step 3.
    2. In another session, insert rows continuously (`pgbench -f` with a one-line insert script, or a shell loop) while you validate.
  - **Done when:** `\d orders` shows `total_cents` as `not null` and the concurrent inserts never blocked for more than a moment.
- [ ] **T3 — Break it: reproduce the lock queue** *(≈40 min, advanced)*
  - **Goal:** see a harmless migration stall unrelated queries.
  - **Steps:**
    1. Follow step 5 with and without `lock_timeout`.
    2. Record how long the plain `SELECT` in session 3 waited in both cases and capture the `pg_stat_activity` rows.
  - **Done when:** you have timings for both cases and can explain why the SELECT waited behind the ALTER.
- [ ] **T4 — Break it: the rename that kills the old version** *(≈40 min, intermediate)*
  - **Goal:** prove that a one-step rename is not rollback-safe.
  - **Steps:**
    1. Run an "old app" loop (`while sleep 0.5; do psql -d shop -tc "select total from orders limit 1"; done`).
    2. In another session run `ALTER TABLE orders RENAME COLUMN total TO total_cents`.

3. Undo it and redo the change with the three-release pattern.

- **Done when:** the loop shows `column "total" does not exist` after the rename and shows no error during the expand/contract version until the contract migration.

- [ ] **T5 — Migration job in a pipeline** *(≈75 min, advanced)*
  - **Goal:** a PR check and an ordered deploy.
  - **Steps:**
    1. Add `migrations-check.yml` (step 8) and open a PR with a new migration; make it fail with a syntax error and fix it.
    2. Add a `migrate` Compose service (or Kubernetes Job) and run it before the app in your deploy script; verify the deploy stops if the migration fails.
  - **Done when:** a bad migration turns the PR check red, and a deploy with a failing migration never starts the new app version.

- [ ] **T6 — Prove it: plan a column split without notes** *(≈60 min, advanced)*
  - **Goal:** design expand/contract for a new change.
  - **Steps:**
    1. Without the guide, write the three-release plan to split `customers.name` into `first_name` and `last_name`: migrations, application behaviour per release, backfill method, and the rollback story at each step.
    2. Execute it on a scratch database while a read/write loop runs; confirm zero errors.
    3. Explain which step you could not safely compress, and why.
  - **Done when:** the loop reported no errors during all three phases, and your write-up states that v(N-1) code worked after every migration.

10 Self-check questions

1. Why must a migration be compatible with both the previous and the next application version?
2. Describe expand and contract using a column rename as the example.
3. Why can a quick `ALTER TABLE` take an application down, and what two defences reduce the risk?
4. Why run migrations as a separate step rather than at app startup?
5. How do you add a `NOT NULL` constraint or a foreign key to a large PostgreSQL table with minimal locking?
6. Why is a down-migration a weak rollback plan?

ANSWERS

1. During rolling, blue/green and canary deploys both versions run against one database, and a rollback runs the old version against the new schema.
2. Add the new column (nullable) plus a way to keep it in sync, deploy code that writes both and reads the new one, backfill, then in a later release stop using and drop the old column.
3. It needs an exclusive lock and waits behind open transactions, and everything else queues behind it. Use `lock_timeout` (with retry) and metadata-only/`NOT VALID` / `CONCURRENTLY` patterns, plus keep application transactions short.
4. Replicas race and duplicate work, a slow migration makes health checks fail, a crash loop can leave a half-applied change, and the app account should not have DDL rights; a single retryable job gives control and clear failure.
5. Add the constraint as `NOT VALID` (or a `CHECK ... IS NOT NULL NOT VALID`), then `VALIDATE CONSTRAINT`, which scans without blocking writes; then `SET NOT NULL` (PostgreSQL 12+ skips the rescan).
6. Data changes (dropped columns, rewritten values, lost rows) cannot be reversed by a script; it is safer to keep the schema backwards compatible so old code works, and to fix forward.

11 Further reading

- Stripe Engineering: *Online migrations at scale*.
- PostgreSQL Documentation: *ALTER TABLE*, *CREATE INDEX* (the `CONCURRENTLY` section) and *Explicit Locking*; the `lock_timeout` setting.
- Flyway Documentation (Redgate): *Migrations*, *Concepts*, *Commands*; Prisma Docs: *Prisma Migrate* (development and production workflows); Alembic Documentation: tutorial and operations reference.
- GitHub `gh-ost` repository documentation; GitLab Development Docs: *Migration style guide*.
- *Refactoring Databases* by Scott Ambler and Pramod Sadalage (evolutionary database design).
- Next: Rolling deployments §5.10. Related: Transactions and Locks §13.2, Zero-downtime deployments §5.8.

LAB NOTES What I ran, what I saw, what it means

5.10 Rolling Deployments

Topic 10 of 12 | ~4 hours | Difficulty: Intermediate

1 What it is

A **rolling deployment** replaces the running instances of a service with the new version a few at a time. At every moment some instances run the old version and some the new one, until all have been replaced. Because the service never has zero healthy instances, users keep being served, and because only a slice is replaced at once, a bad release is noticed before it has replaced everything.

Two numbers define a rolling update:

- **Batch size:** how many instances are being replaced at the same time.
- **Capacity floor and ceiling:** how much capacity may disappear (**max unavailable**) and how much extra capacity may be created temporarily (**max surge**) during the swap.

Rolling is the default strategy of Kubernetes Deployments and Amazon ECS services, and the cheapest strategy in infrastructure terms: it needs no second full environment. Compare it with blue/green §5.11 and canary §5.12 in Deployment Strategies §0.7.

2 Why it matters in DevOps

- **It is what you get by default**, so you must understand its settings. The defaults may be too aggressive for a small service or too slow for a big one.
- **Mixed versions are normal.** For the length of the rollout both versions serve traffic and share the database, queues and caches. Anything not backwards compatible breaks here (L5.9).
- **Capacity during deploys:** with three replicas and `maxUnavailable: 1`, a deploy removes a third of your capacity. At peak traffic that can cause the very outage the deploy was supposed to avoid.
- **Speed vs safety:** a big batch finishes fast but a bad version hurts more users; a batch of one is gentle but a 200-instance fleet then takes hours.

- **Rollback is another rolling update** back to the old version, so it takes as long as the roll forward.

3 Real-world system example

Kubernetes Deployments. The Deployment controller implements `RollingUpdate` by scaling a new ReplicaSet up and the old one down, within the limits `maxSurge` and `maxUnavailable` (both default to 25%). Percentages are rounded: the surge is rounded up and the unavailable count is rounded down. A pod counts as available only after its readiness probe passes (and `minReadySeconds`, if set), so a crashing new version stalls the rollout instead of replacing healthy pods. If the rollout makes no progress for `progressDeadlineSeconds` (600 by default), the Deployment is reported as failed (it is not rolled back automatically; you or your pipeline do that with `kubectl rollout undo`).

Amazon ECS. ECS services use rolling updates by default, controlled by `minimumHealthyPercent` and `maximumPercent` (documented defaults of 100 and 200 for the rolling update type), and can enable a **deployment circuit breaker** that stops a failing deployment and rolls back automatically. **EC2 Auto Scaling instance refresh** replaces instances in a group in batches, governed by a minimum healthy percentage and an instance warm-up time. All three are the same idea expressed in different APIs.

4 Diagram

A rolling update of four instances with batch size 1 (`maxSurge 1`, `maxUnavailable 0`):

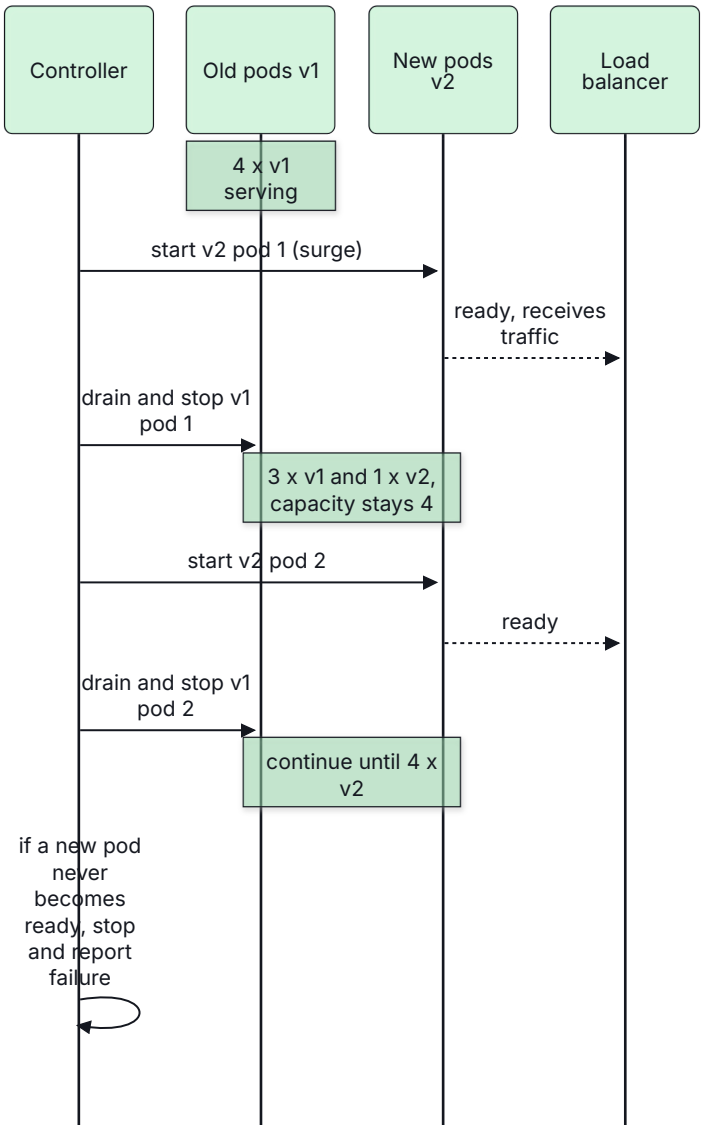


Figure 5.10-1

How the settings trade off capacity, speed and cost:

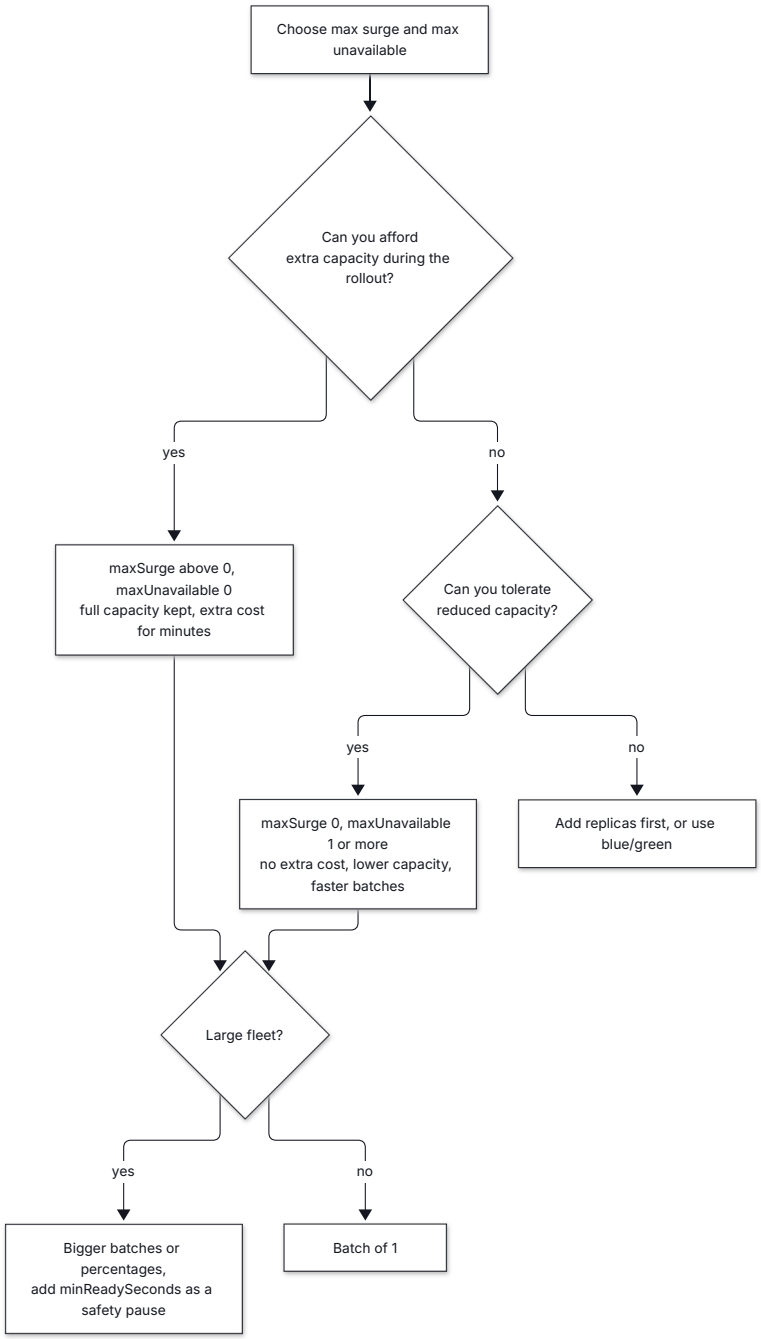


Figure 5.10-2

5 How it works

The settings, per platform

Platform	Batch / pace	Capacity control	Safety net
Kubernetes Deployment	implied by `maxSurge` and `maxUnavailable`	`maxSurge` (extra pods allowed), `maxUnavailable` (pods that may be down), integers or percentages	readiness probe, `minReadySeconds`, `progressDeadlineSeconds`, `kubectl rollout pause/undo`
Amazon ECS	tasks started and stopped within the limits	`minimumHealthyPercent`, `maximumPercent`	health checks, deployment circuit breaker with rollback, CloudWatch alarms
EC2 Auto Scaling instance refresh	replaces instances in batches	minimum healthy percentage, instance warm-up	checkpoints (pause at 20%, 50%…), automatic rollback option
Docker Swarm	`update_config.parallelism`, `delay`	`order: start-first` or `stop-first`	`failure_action: rollback`, `monitor`
Your script / Ansible	`serial: 1` or `serial: "25%"` in Ansible	you decide	stop the play when a host fails the health gate

Kubernetes examples for 4 replicas:

`maxSurge`	`maxUnavailable`	Behaviour	Capacity during rollout	Extra cost
1	0	start one new pod, wait ready, remove one old pod	always 4 or more	one extra pod briefly
0	1	remove one old pod, then start its replacement	3 or 4	none
25% (rounds to 1)	25% (rounds to 1)	default; may overlap a bit	3 to 5	small
2	1	fast; two start while one is removed	3 or more	two extra pods
100%	0	everything new first (blue/green-like)	4 to 8	doubles for the rollout

Choosing batch size

- **Capacity:** `batch ⩽ spare_capacity`. If you run at 70% utilisation with 4 instances, removing one leaves the other three at about 93%. Over-provision, or surge instead of removing.

- **Blast radius:** the first batch is your canary. If one instance of four is bad, about a quarter of requests hit it until the gate stops the rollout.
- **Warm-up:** JVM apps, caches and connection pools need time; add `minReadySeconds` or a warm-up period so a fresh instance is not counted healthy the instant it answers.
- **Stateful or sticky systems:** databases, queues with consumer groups and websocket servers need ordering and draining care (StatefulSets update one pod at a time in reverse ordinal order by default).

What a rolling update does not give you

Limitation	Consequence	Answer
Mixed versions for the whole rollout	schema, API and message formats must be compatible	expand/contract (L5.9); version APIs
No instant rollback	going back is another rolling update	blue/green (L5.11) or feature flags
Traffic split is by instance count, not a chosen percentage	a bad version gets `new/total` of the traffic, uncontrolled by metrics	canary with metric gates (L5.12)
Health gate quality decides everything	a weak readiness probe lets bad pods replace good ones	meaningful `/readyz`, post-deploy smoke tests (L5.8)
Long-lived connections stay on old instances	uneven load after the rollout	drain with a deadline; clients reconnect

6 Step-by-step guide

Prerequisites: Node.js 22, `curl`, Nginx (Debian/Ubuntu: `sudo apt-get install -y nginx`, used from a private directory; the system service is untouched), and the `server.js` from L5.8 (graceful shutdown with `/readyz` returning `{"status":"ok","version":"..."}`). No cloud account. The local rollout below was run in a sandbox; timings are "similar to" on your machine. A Kubernetes version follows in step 6 (kind or minikube, free).

1. **Create a private Nginx with four backends.** In an empty directory containing `server.js`:

```bash
mkdir -p nginx/logs nginx/tmp run
cat > nginx/nginx.conf <<EOF
worker_processes 1;
pid $PWD/nginx/nginx.pid;
error_log $PWD/nginx/logs/error.log warn;
events { worker_connections 256; }
http {
  access_log off;
  client_body_temp_path $PWD/nginx/tmp/body; proxy_temp_path $PWD/nginx/tmp/proxy;
```

```
  fastcgi_temp_path $PWD/nginx/tmp/fcgi; uwsgi_temp_path $PWD/nginx/tmp/uwsgi;
  scgi_temp_path $PWD/nginx/tmp/scgi;
  upstream api {
    server 127.0.0.1:3000 max_fails=0;
    server 127.0.0.1:3001 max_fails=0;
    server 127.0.0.1:3002 max_fails=0;
    server 127.0.0.1:3003 max_fails=0;
  }
  server {
    listen 8080;
    location / {
      proxy_pass http://api;
      proxy_next_upstream error timeout http_502 http_503;
      proxy_connect_timeout 1s;
    }
  }
}
EOF
nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx
```

(`max_fails=0` is explained in L5.8: it stops open-source Nginx from marking a draining server down.)

2. **Write the rolling-update script with a batch size and a health gate.** Save as `rolling.sh` and `chmod +x rolling.sh`:

```bash
#!/usr/bin/env bash
# Rolling update of local instances. Usage: rolling.sh <version> <batch-size>
set -euo pipefail
VERSION="${1:?version}"; BATCH="${2:-1}"
PORTS=(3000 3001 3002 3003)
mkdir -p run

start() {   # start <port>
  PORT="$1" APP_VERSION="$VERSION" DRAIN_MS="${DRAIN_MS:-2000}" node server.js >
  "run/$1.log" 2>&1 &
  echo $! > "run/$1.pid"
}
stop() {    # stop <port>: SIGTERM, wait for the graceful exit
  [ -f "run/$1.pid" ] || return 0
  local pid; pid=$(cat "run/$1.pid")
  kill -TERM "$pid" 2>/dev/null || return 0
  while kill -0 "$pid" 2>/dev/null; do sleep 0.2; done
}
ready() {   # ready <port>: gate = 5 consecutive good readiness answers for the NEW
  version
  local ok=0
  for _ in $(seq 1 60); do
    if curl -fs --max-time 2 "http://127.0.0.1:$1/readyz" 2>/dev/null | grep -q
  "\"version\":\"$VERSION\""; then ok=$((ok+1)); else ok=0; fi
```

```
    [ "$ok" -ge 5 ] && return 0
    sleep 0.2
  done
  return 1
}

i=0
while [ $i -lt ${#PORTS[@]} ]; do
  batch=("${PORTS[@]:$i:$BATCH}")
  echo "batch: ${batch[*]}"
  stoppers=()
  for p in "${batch[@]}"; do stop "$p" & stoppers+=($!); done
  wait "${stoppers[@]}"                        # wait only for the stop jobs, not for
  the servers
  for p in "${batch[@]}"; do start "$p"; done
  for p in "${batch[@]}"; do ready "$p" || { echo "GATE FAILED on $p: stop the
  rollout" >&2; exit 1; }; done
  i=$((i+BATCH))
done
echo "rollout of $VERSION complete"
```

The `wait` takes explicit PIDs on purpose: a bare `wait` would also wait for the long-running servers you started and hang. The gate (five consecutive good answers *reporting the new version*) plays the role of a Kubernetes readiness probe plus `minReadySeconds`.

3. **Start version v1 on all four instances.**

```bash
export DRAIN_MS=2000
./rolling.sh v1 4          # bootstraps: batch of 4 = everything at once
curl -s localhost:8080/readyz      # {"status":"ok","version":"v1"}
```

(The first run finds no PID files, so "stop" does nothing and all four start.)

4. **Generate traffic and record status code and version for every request.** Save as `load.sh` (`chmod +x load.sh`):

```bash
#!/usr/bin/env bash
# one line per request: "<status> <version>" for N seconds
end=$((SECONDS+$1))
while [ $SECONDS -lt $end ]; do
  out=$(curl -s -w ' %{http_code}' --max-time 5 http://127.0.0.1:8080/readyz || echo '
  000')
  code=${out##* }
  ver=$(echo "$out" | grep -o '"version":"[^"]*"' | cut -d'"' -f4)
  echo "$code ${ver:-none}"
  sleep 0.1
done
```

5. **Roll out v2 with batch size 1, under load, and compare with batch size 2.**

```bash
(./load.sh 22 > codes-b1.txt) &
sleep 1; time ./rolling.sh v2 1; wait
sort codes-b1.txt | uniq -c

(./load.sh 22 > codes-b2.txt) &
sleep 1; time ./rolling.sh v3 2; wait
sort codes-b2.txt | uniq -c
```

Expected shape (similar to; counts depend on timing): `batch: 3000`, `batch: 3001`, … then `rollout of v2 complete`, taking around 15 seconds, with output like

```
     49 200 v1
    131 200 v2
```

Every request succeeded and you can see both versions served during the rollout (mixed-version operation). Batch size 2 finishes sooner (fewer rounds) but half the instances are down at once, so each remaining instance carries double load for a few seconds. In `codes-b2.txt` watch for slower responses or a short burst of `v3` appearing all at once.

6. **Prove the health gate protects you.** Make the new version unready and watch the rollout stop after the first batch:

```bash
# break v4: it starts but reports a different version than the gate expects (simulates
  a failed readiness)
sed 's/process.env.APP_VERSION || "dev"/"broken"/' server.js > server-broken.js
cp server.js server-good.js
cp server-broken.js server.js
./rolling.sh v4 1; echo "exit=$?"
cp server-good.js server.js
```

Expected: `batch: 3000`, then (after the gate times out, about 12 seconds) `GATE FAILED on 3000: stop the rollout` and `exit=1`. Only the first batch (1 of 4 instances) ever ran the bad version; the other three still run the previous one. Run `./load.sh` during this experiment and count: in my run about 18% of requests (30 of 163) were answered by the `broken` instance, which is about one quarter of the traffic for as long as it stays in rotation. That is the blast radius of a batch of one on four instances. In this local model the load balancer does not know the gate failed, so the bad instance keeps receiving traffic until you act; a real orchestrator keeps an instance that fails readiness out of rotation. Recover with `./rolling.sh v3 1` (restores a single version) and put the good `server.js` back.

7. **The same thing on Kubernetes** (kind: `kind create cluster`). Save as `deploy.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 4
  minReadySeconds: 5
  progressDeadlineSeconds: 120
  selector:
    matchLabels: { app: shop-api }
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels: { app: shop-api }
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: api
          image: ghcr.io/<you>/shop-api:sha—<old>
          ports: [{ containerPort: 3000 }]
          readinessProbe:
            httpGet: { path: /readyz, port: 3000 }
            periodSeconds: 3
```

```bash
kubectl apply -f deploy.yaml
kubectl rollout status deployment/shop-api
kubectl set image deployment/shop-api api=ghcr.io/<you>/shop-api:sha—<new>
kubectl get pods -w                              # watch: new pod Ready before an old
  one terminates
kubectl rollout status deployment/shop-api --timeout=120s
kubectl rollout history deployment/shop-api
kubectl rollout undo deployment/shop-api         # a rollout of the previous
  ReplicaSet
```

Change to `maxSurge: 0, maxUnavailable: 2` and repeat to see capacity dip instead of surge. To pause after the first batch as a manual canary: `kubectl rollout pause deployment/shop-api`, inspect, then `kubectl rollout resume deployment/shop-api`. Clean up with `kind delete cluster`.

8. **Clean up the local sandbox.** `for f in run/*.pid; do kill -TERM $(cat $f); done; nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx -s quit`.

7 Real-world scenario

(Illustrative.) `shop-api` runs three replicas with the Kubernetes defaults (`25%` rounds to `maxSurge 1`, `maxUnavailable 0`) and everything is fine. The team cuts costs by right-sizing to two replicas running at 75% CPU, and sets `maxUnavailable: 1, maxSurge: 0` to avoid "extra pods that cannot be scheduled" on a small cluster. At the next lunchtime deploy, one of two pods is removed, the remaining one saturates, latency climbs from 150 ms to 3 s and the readiness probe of the surviving pod starts timing out. Kubernetes removes it from the Service and the site is briefly fully down while the first new pod starts.

The postmortem finds that the rollout settings had been changed without considering capacity. Fixes: restore `maxSurge: 1, maxUnavailable: 0` and size the cluster for one extra pod; add a `PodDisruptionBudget`; deploy outside the daily peak unless needed; and add an alert for `available replicas < desired - 0` during rollouts. **Lesson:** rolling update parameters are a capacity decision, and the safest default is "surge before you remove".

8 Common mistakes and best practices

Don't

- Reduce `maxSurge` and raise `maxUnavailable` to "make deploys faster" without checking capacity at peak.
- Use a readiness probe that only checks the process is up; the rollout then happily replaces good pods with broken ones.
- Assume both versions never overlap: ship incompatible schema or API changes and discover it halfway through the rollout.
- Rely on rolling for a risky change where you need an instant switch back or a measured exposure; use blue/green or canary.
- Forget `progressDeadlineSeconds` and alerts, so a stalled rollout sits half-finished for hours.
- Let the pipeline report success when `kubectl apply` returned; wait for `rollout status`.

Do

- Prefer surge-first (`maxUnavailable: 0`) when you can afford the extra instance for a few minutes.

- Add `minReadySeconds`/warm-up and a real `/readyz`; gate the pipeline on `kubectl rollout status` and a post-deploy smoke test.
- Keep rollbacks rehearsed: `kubectl rollout undo` or redeploy of the previous digest (L5.7).
- Size batches by capacity and blast radius: small for critical services, percentages for large fleets, with checkpoints.
- Make every change backwards compatible for one version, and design clients to retry safely.
- Watch error rate and latency by version during the rollout (label metrics with the version).

9 Tasks

- [ ] **T1 — Roll four instances with a gate** *(≈45 min, intermediate)*
  - **Goal:** run and read a rolling update.
  - **Steps:**
    1. Set up Nginx and `rolling.sh` (steps 1 to 3).
    2. Run `./rolling.sh v2 1` under `load.sh`.
    3. Count responses by status and version.
  - **Done when:** all responses are `200`, both `v1` and `v2` appear, and you can say which part of the log shows mixed-version operation.
- [ ] **T2 — Compare batch sizes** *(≈45 min, intermediate)*
  - **Goal:** measure the speed versus capacity trade-off.
  - **Steps:**
    1. Roll with batch sizes 1, 2 and 4, each under load; record total time and any non-200.
    2. Add an artificial per-request delay (`/slow`) to the load script to simulate heavy load and repeat batch 1 vs 2.
  - **Done when:** you have a table of batch size, duration and error count, and a sentence justifying a batch size for a four-instance service.
- [ ] **T3 — Break it: a bad release halts at the gate** *(≈45 min, intermediate)*
  - **Goal:** see the health gate limit the blast radius.
  - **Steps:**
    1. Do step 6 (broken version).
    2. Count how many instances ran the bad version and what the load script saw.

3. Recover with a rollout of the previous version.

- **Done when:** the rollout exits non-zero after the first batch and the system is restored to a single version with `200` responses.

- [ ] **T4 — Kubernetes: surge versus unavailable** *(≈60 min, advanced; kind, free)*
  - **Goal:** observe capacity under both settings.
  - **Steps:**
    1. Deploy `shop-api` with `maxSurge: 1, maxUnavailable: 0` and trigger an update while running a `curl` loop through a port-forward.
    2. Repeat with `maxSurge: 0, maxUnavailable: 2` and a deliberately slow readiness probe.
    3. In a second terminal record `kubectl get deploy shop-api -w` (READY and AVAILABLE counts).
    4. **Cleanup:** `kind delete cluster`.
  - **Done when:** you captured the READY counts for both settings and can show where capacity dipped below 4.

- [ ] **T5 — Auto-fail a stuck rollout in the pipeline** *(≈60 min, advanced)*
  - **Goal:** make the pipeline decide, not a human watching.
  - **Steps:**
    1. Write a deploy step: `kubectl set image ...` then `kubectl rollout status --timeout=120s`.
    2. On failure run `kubectl rollout undo` and exit non-zero.
    3. Test with an image that fails its readiness probe.
  - **Done when:** the job is red, the Deployment is back on the previous ReplicaSet (`kubectl rollout history` shows it), and the notification says "rolled back".

- [ ] **T6 — Prove it: choose settings for three services** *(≈45 min, advanced)*
  - **Goal:** justify rolling parameters from capacity, not habit.
  - **Steps:**
    1. Without the guide, choose `maxSurge` / `maxUnavailable` / `minReadySeconds` for: (a) 2 replicas at 70% load, (b) 100 replicas at 40% load, (c) a stateful consumer group with 6 pods.
    2. For each, state the minimum capacity during the rollout, the expected duration and the main risk.
    3. Run (a) on kind to check your numbers.

- **Done when:** your table is internally consistent (capacity numbers match the settings) and the kind run confirms (a).

10 Self-check questions

1. What do `maxSurge` and `maxUnavailable` control, and what do percentages round to?
2. Why does a rolling update need backwards-compatible schema and API changes?
3. How can a rolling update take down a service with zero failed pods?
4. What stops Kubernetes from replacing all healthy pods with a broken version?
5. When would you pick blue/green or canary over rolling?
6. How does rollback differ for a rolling deployment compared with blue/green?

ANSWERS

1. `maxSurge` is how many extra pods above the desired count may exist during the update; `maxUnavailable` is how many may be unavailable. Surge rounds up, unavailable rounds down.
2. Old and new versions serve traffic simultaneously against the same data stores during the rollout (and during rollback), so both must work with the schema and messages in use.
3. If the settings remove too much capacity (for example `maxUnavailable` too high at peak load), the remaining pods saturate and fail even though the rollout is "healthy".
4. Readiness probes (plus `minReadySeconds`): a new pod that never becomes ready does not count as available, so the controller does not remove more old pods and the rollout stalls.
5. Blue/green when you need an instant, whole-environment switch back; canary when you want a controlled small percentage of traffic and metric-based promotion.
6. Rolling rollback is another rolling update (takes as long and passes through mixed versions again); blue/green rollback is a traffic switch back to the still-running old environment.

11 Further reading

- Kubernetes Docs: *Deployments* (rolling update strategy, `maxSurge`, `maxUnavailable`, `minReadySeconds`, `progressDeadlineSeconds`) and `kubectl rollout`.
- AWS Docs: *Amazon ECS rolling update deployments* and the deployment circuit breaker; *Amazon EC2 Auto Scaling instance refresh*.
- Ansible Docs: the `serial` keyword for rolling updates across hosts.
- *Continuous Delivery* by Jez Humble and David Farley, chapter on deployment strategies.
- Next: Blue/green deployments §5.11. Related: Deployments and ReplicaSets §8.3.

5.11 Blue/Green Deployments

Topic 11 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

A **blue/green deployment** keeps **two complete production environments**. One, say **blue**, is **live** and serves all users. The other, **green**, is idle. To release, you deploy the new version to the idle environment, test it there with no user traffic, and then flip the router so that all traffic goes to green in one step. Blue stays running, untouched, as your instant way back.

The key properties:

- **Two environments** of the same shape (same instance counts, same configuration, same database connectivity), named by colour so nobody confuses them with "staging".
- **One traffic switch**: a load balancer listener, a Kubernetes Service selector, a DNS record or a reverse-proxy upstream. Flipping it is fast and atomic compared with replacing instances one by one.
- **Instant rollback:** flip the switch back. No rebuild, no redeploy, no waiting for pods.
- **Pre-flight testing in production conditions:** the idle environment runs on the real infrastructure and can be smoke-tested through a private URL before it takes any customer traffic.

The price is **cost** (two environments) and the **database problem**: both colours normally share one database, so schema changes must be backwards compatible (L5.9).

2 Why it matters in DevOps

- **Rollback in seconds:** the fastest recovery from a bad release is switching traffic back. This turns "we need to roll back" from a 15-minute operation into a one-line command (L5.7).
- **No mixed versions on the same traffic:** unlike rolling, a given request path sees either all-blue or all-green. This simplifies reasoning about caches and APIs (but not about the shared database).

- **Safer big or risky changes:** framework upgrades, runtime upgrades and infrastructure changes can be validated on the idle side first.
- **Cost-benefit decision:** teams must decide whether instant rollback justifies running twice the capacity during the deploy window. For most web services a short overlap is cheap; for large stateful fleets it is not.

3 Real-world system example

The pattern's origin and AWS tooling. The blue/green name and idea were popularised by Jez Humble and Dan North and described on Martin Fowler's site (*BlueGreenDeployment*). AWS supports it in several services: **AWS CodeDeploy** blue/green deployments for EC2 and ECS (with Application Load Balancer target groups as the "colours" and a configurable wait before the old tasks are terminated), **AWS Elastic Beanstalk** environment *swap CNAMEs*, and weighted target groups on Application Load Balancers. In Kubernetes, the same effect is achieved with two Deployments and a Service whose selector is switched, or by a traffic manager (Argo Rollouts supports a blue/green strategy with a preview service and a promotion step).

Cloud Foundry popularised "blue-green deploy" on its platform with a documented zero-downtime procedure of mapping the application route to the new app and then unmapping the old one.

Practical constraint seen in all of these: the databases and queues are not duplicated, which is why practitioners pair blue/green with expand/contract migrations and feature flags.

4 Diagram

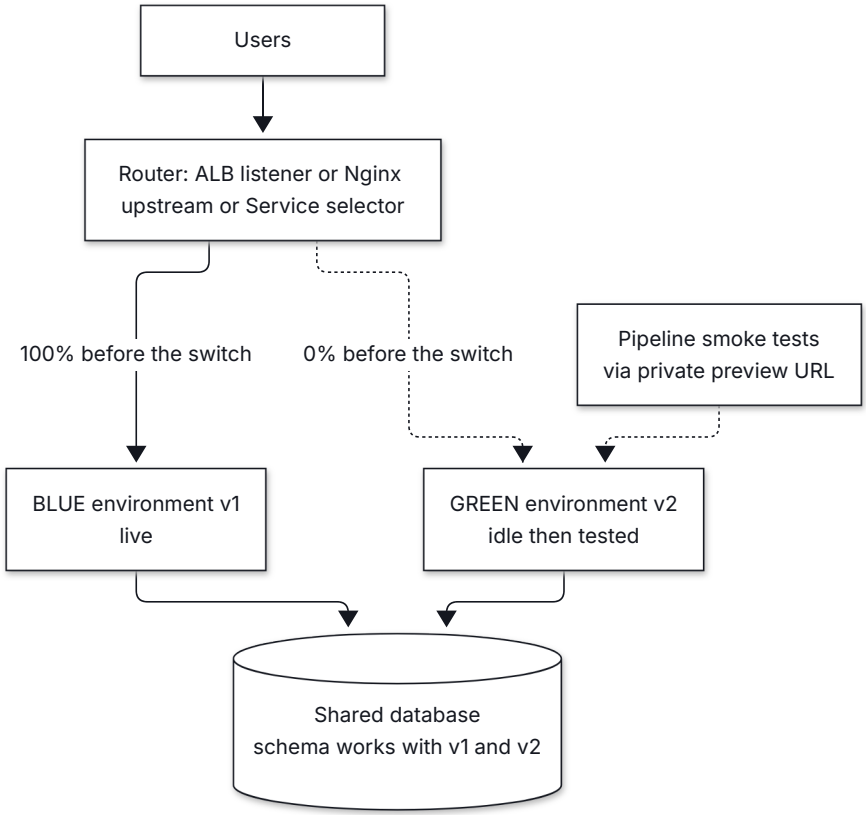


Figure 5.11-1

The release timeline (switch and rollback are the same action in opposite directions):

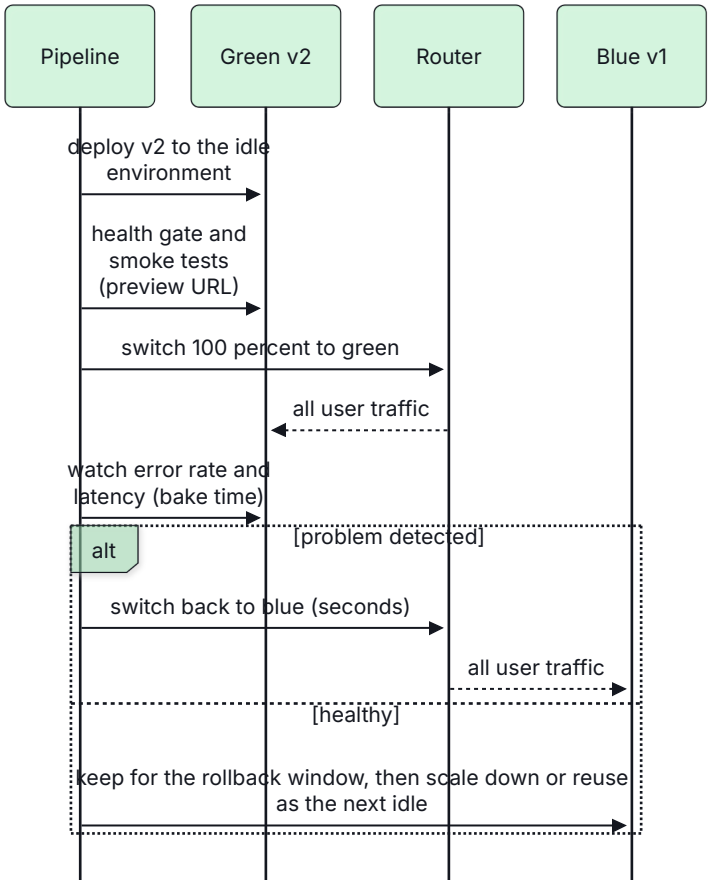


Figure 5.11-2

5 How it works

Ways to implement the switch

Mechanism	Switch time	Notes
Reverse proxy upstream (Nginx, HAProxy, Envoy)	instant after reload	simple on one host or VM; reload is graceful, in-flight requests finish on the old upstream
ALB listener rule / weighted target groups	seconds	`modify-listener` (or rule) to forward 100% to the green target group; target groups are the colours
Kubernetes Service selector	seconds	two Deployments labelled `track: blue` / `track: green`; patch the Service selector

Mechanism	Switch time	Notes
Ingress or Gateway weights / Argo Rollouts blue/green	seconds	controller adds a preview Service and promotion step
DNS (Route 53 records)	minutes or longer	clients and resolvers cache answers beyond the TTL; the old environment keeps receiving traffic for a while. Acceptable for coarse switches (regions), not for instant rollback
Elastic Beanstalk CNAME swap	DNS-bound	uses DNS under the hood, with the same caching caveat

Prefer a switch that lives **in your infrastructure** (proxy, load balancer) over DNS when you want instant, reliable rollback.

The stages of a blue/green release

1. **Find the idle colour** (the one not currently live).
2. **Run migrations** in the expand phase (L5.9); both colours must work with the resulting schema.
3. **Deploy the new version to the idle colour** from the same artifact digest (L5.6); configure it exactly like the live one.
4. **Gate it**: health check with version, then smoke tests through a private route (a preview hostname or a path that only your pipeline uses).
5. **Switch** the router to the idle colour, which becomes the live one.
6. **Bake**: watch metrics for a defined period; automatic rollback on SLO or alarm breach (L5.7).
7. **Decommission or reuse**: keep the previous colour warm for the rollback window (an hour, a day), then scale it down or make it the next idle target. Never run the contract migration until the window has passed.

What blue/green does not solve

Concern	Detail	Mitigation
Database/schema	one shared database; an incompatible migration breaks the colour you roll back to	expand/contract, backwards-compatible changes
Stateful sessions and caches	users on blue have in-memory sessions or local caches that green does not have	external session store (Redis), stateless services; accept cache warm-up

Concern	Detail	Mitigation
Warm-up	the green side has cold caches and JIT; the switch moves 100% of traffic to it at once	pre-warm with synthetic traffic, or ramp traffic with weighted routing (which makes it a canary)
In-flight work and queues	background jobs consumed by both colours, long-running requests, websockets	drain consumers on the old side, version job payloads, let long requests finish
Cost	two full environments	the idle one exists only for the deploy window and rollback window; scale it down in between
Third-party callbacks and cron jobs	duplicates if both colours run schedulers	run schedulers on one colour only, or use leader election
Hidden coupling	hard-coded hostnames or IPs of "the" environment	resolve through configuration and service discovery

Cost model

If one environment costs `C` per hour and a release keeps both running for `T` hours, a release costs about `C × T` extra. Releasing four times a day with a one-hour overlap on a 10 USD/hour environment is roughly 40 USD/day of overlap; keeping the idle side permanently warm would instead double the bill (`C × 24`). With containers or auto scaling, the idle side can be scaled to zero or to a minimum between releases; with fixed VMs it is a real standing cost. (Illustrative arithmetic; use your own prices.)

6 Step-by-step guide

Prerequisites: Node.js 22, `curl`, Nginx (Debian/Ubuntu: `sudo apt-get install -y nginx`, used from a private directory), and the `server.js` from L5.8. The local switch below was run in a sandbox and the traffic loop saw only `200` responses across the switch and the rollback (counts "similar to" yours). Cost: none; the AWS and Kubernetes snippets are reference material.

1. **Create two environments (blue on 3000, green on 3001) and a switchable Nginx.** In an empty directory containing `server.js`:

```bash
mkdir -p nginx/logs nginx/tmp
cat > nginx/nginx.conf <<EOF
worker_processes 1;
pid $PWD/nginx/nginx.pid;
error_log $PWD/nginx/logs/error.log warn;
events { worker_connections 256; }
http {
```

```
  access_log off;
  client_body_temp_path $PWD/nginx/tmp/body; proxy_temp_path $PWD/nginx/tmp/proxy;
  fastcgi_temp_path $PWD/nginx/tmp/fcgi; uwsgi_temp_path $PWD/nginx/tmp/uwsgi;
  scgi_temp_path $PWD/nginx/tmp/scgi;
  upstream blue  { server 127.0.0.1:3000; }
  upstream green { server 127.0.0.1:3001; }
  include $PWD/nginx/active.conf;      # defines: map \$host \$live_env
  server {
    listen 8080;
    location / { proxy_pass http://\$live_env; }                    # live traffic
    location = /preview-green { proxy_pass http://green/readyz; }   # pipeline-only
  test route
    location = /preview-blue  { proxy_pass http://blue/readyz; }
  }
}
EOF
printf 'map $host $live_env { default blue; }\n' > nginx/active.conf
nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx
```

The file `active.conf` holds the single fact "which colour is live". Everything else is static. (The `\$` in the here-document keeps Nginx variables from being expanded by the shell.)

2. **Start blue as v1 (live) and green as v2 (idle, being tested).**

```bash
export DRAIN_MS=2000
PORT=3000 APP_VERSION=v1 node server.js > blue.log 2>&1 &
PORT=3001 APP_VERSION=v2 node server.js > green.log 2>&1 &
sleep 1
curl -s localhost:8080/readyz              # {"status":"ok","version":"v1"}   ← users
  see blue
curl -s localhost:8080/preview-green       # {"status":"ok","version":"v2"}   ← the
  pipeline sees green
```

Green is fully running and reachable for tests, yet no user traffic reaches it.

3. **Write the switch script.** Save as `switch.sh` and `chmod +x switch.sh`:

```bash
#!/usr/bin/env bash
# Usage: switch.sh blue|green   (atomic switch + graceful reload)
set -euo pipefail
TARGET="${1:?blue|green}"
CONF="$PWD/nginx/active.conf"
printf 'map $host $live_env { default %s; }\n' "$TARGET" > "$CONF.new"
mv "$CONF.new" "$CONF"                          # atomic replace
nginx -t -c "$PWD/nginx/nginx.conf" -p "$PWD/nginx" 2>/dev/null
nginx -s reload -c "$PWD/nginx/nginx.conf" -p "$PWD/nginx" 2>/dev/null
echo "live environment is now: $TARGET"
```

`nginx -t` validates the new configuration before the reload, and the reload is graceful, so in-flight requests finish on the old upstream.

4. **Switch under load, then roll back, and read the result.** Use the `load.sh` from L5.10 (one line per request: status and version). Run this in one shell:

```bash
(./load.sh 14 > codes.txt) &  LOAD=$!
sleep 3; ./switch.sh green          # release v2
sleep 4; ./switch.sh blue           # instant rollback to v1
sleep 4
wait $LOAD
uniq -c codes.txt
```

Expected (similar to): three blocks and no errors,

```
     84 200 v1        ← before the switch (and the rollback reuses v1)
     35 200 v2        ← while green was live
     27 200 v1        ← after rolling back to blue
```

Note `wait $LOAD` waits for the load generator only; a bare `wait` would also wait for the long-running servers. The version flips at a single point in the log each time: no gradual mix, no failures. That is the blue/green signature.

5. **Gate the switch in a script.** The release decision as code, using the preview route:

```bash
#!/usr/bin/env bash
# release.sh <new-version>: assumes the idle colour already runs <new-version>
set -euo pipefail
NEW="$1"; IDLE="${2:-green}"; LIVE="${3:-blue}"
body=$(curl -fsS --max-time 3 "http://127.0.0.1:8080/preview-$IDLE")
echo "$body" | jq -e --arg v "$NEW" '.status == "ok" and .version == $v' >/dev/null \
  || { echo "idle env not healthy on $NEW: NOT switching" >&2; exit 1; }
./switch.sh "$IDLE"
sleep 10                                  # bake: replace with a real SLO or error-rate
  query
curl -fsS --max-time 3 http://127.0.0.1:8080/readyz | jq -e --arg v "$NEW" '.version
  == $v' >/dev/null \
  || { echo "post-switch check failed: ROLLING BACK" >&2; ./switch.sh "$LIVE"; exit 2;
  }
echo "released $NEW; $LIVE kept as rollback target"
```

The same structure, pre-switch gate then post-switch bake and rollback, applies to every platform below.

6. **The same thing on a Kubernetes Service.** Two Deployments (`shop-api-blue`, `shop-api-green`, each with label `track`) and one Service:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  selector:
    app: shop-api
    track: blue            # the switch
  ports:
    - port: 80
      targetPort: 3000
```

```bash
kubectl rollout status deployment/shop-api-green --timeout=120s          # green is ready
kubectl port-forward deployment/shop-api-green 8081:3000 &               # private test route
curl -s localhost:8081/readyz
kubectl patch service shop-api -p '{"spec":{"selector":{"app":"shop-api","track":"green"}}}'    # switch
kubectl patch service shop-api -p '{"spec":{"selector":{"app":"shop-api","track":"blue"}}}'     # rollback
```

Endpoints update within seconds; with a readiness probe and `preStop` drain (L5.8) the switch is invisible to clients. Clean up with `kind delete cluster` if you created one.

7. **The same thing on AWS with an Application Load Balancer** (reference; creates no resources by itself). Two target groups are the colours; the listener decides:

```bash
aws elbv2 modify-listener --listener-arn <LISTENER_ARN> --default-actions '[
  {"Type":"forward","ForwardConfig":{"TargetGroups":[
    {"TargetGroupArn":"<GREEN_TG_ARN>","Weight":100},
    {"TargetGroupArn":"<BLUE_TG_ARN>","Weight":0}]}}]'
```

Rollback is the same command with the weights reversed. A weighted split such as 90/10 first, then 100/0, turns the same mechanism into a coarse canary (L5.12). AWS CodeDeploy automates the whole sequence (deploy to the replacement task set, test listener, shift traffic, wait, terminate the original) for ECS services. Any real use of ALBs, target groups and listeners costs money while they exist; follow ALB §6.14 and delete them afterwards.

8. **Clean up.** `kill -TERM $(pgrep -x node)`, then `nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx -s quit`.

7 Real-world scenario

(Illustrative.) A team upgrades `shop-api` from Node.js 20 to 22 and swaps the HTTP framework. The change passes tests and a staging deployment. They deploy to the idle green environment on the production network and run synthetic checkouts through the preview route: everything passes. At 10:00 they switch; the dashboards are green for two minutes, then p99 latency for `/checkout` climbs and a downstream payments client begins timing out under production load; staging never saw that connection volume.

The pipeline's bake check sees the alert threshold crossed and runs `switch.sh blue`: traffic is back on the old version within seconds, about three minutes after the release. The post-incident work happens calmly on green, which is still running the new version and can be inspected (heap profile, connection counts) with no customer traffic. The root cause is a connection-pool default that changed in the new framework. **Lesson:** blue/green gave an instant rollback *and* a live specimen to debug; the cost was one extra environment for 20 minutes.

8 Common mistakes and best practices

Don't

- Switch with DNS when you need instant, reliable rollback (caches keep sending traffic to the old side).
- Run an incompatible database migration "because we can switch back": the old side cannot run on the new schema.
- Let blue and green drift in configuration, secrets or sizes; the idle one must be a faithful copy of production.
- Tear down the old environment immediately after the switch; you lose the rollback.
- Forget that both sides may run schedulers, consumers or cron jobs that now run twice.
- Assume the switch is a test: green must be smoke-tested *before* it takes traffic.

Do

- Keep the switch in one place and one command, tested by a drill; rollback = the same command in reverse.
- Deploy the same artifact digest to the idle side; only configuration differs.
- Use backwards-compatible migrations (expand/contract) and run the contract step only after the rollback window.

- Warm green (synthetic traffic, cache priming) or ramp the traffic if cold starts matter.
- Bake with automatic checks on error rate, latency and business metrics, and roll back automatically on failure.
- Scale the old environment down after the window, or make it the next idle target; track the cost of the overlap.

9 Tasks

- [ ] **T1 — Build the switchable proxy** *(≈45 min, intermediate)*
  - **Goal:** two environments, one switch.
  - **Steps:**
    1. Set up Nginx with `blue`, `green` upstreams and `active.conf` (steps 1 and 2).
    2. Start blue as v1 and green as v2.
    3. Confirm live traffic shows v1 and `/preview-green` shows v2.
  - **Done when:** `curl localhost:8080/readyz` returns v1 while `curl localhost:8080/preview-green` returns v2.
- [ ] **T2 — Switch and roll back under load** *(≈45 min, intermediate)*
  - **Goal:** see the signature of an atomic switch.
  - **Steps:**
    1. Run `load.sh` for 14 seconds in the background.
    2. `./switch.sh green`, wait, then `./switch.sh blue`.
    3. Summarise with `uniq -c`.
  - **Done when:** the summary shows v1, then v2, then v1 blocks, with zero non-200 responses.
- [ ] **T3 — Break it: switch to an unhealthy environment** *(≈45 min, intermediate)*
  - **Goal:** understand why the pre-switch gate exists.
  - **Steps:**
    1. Start green with a version whose `/readyz` returns 500 (or kill green).
    2. Run `./switch.sh green` directly without the gate and watch `load.sh` output.
    3. Run `release.sh` instead and see it refuse to switch; then roll back with `switch.sh blue`.
  - **Done when:** the direct switch shows errors in the load output, and `release.sh` exits non-zero without switching.
- [ ] **T4 — Kubernetes Service selector switch** *(≈60 min, advanced; kind, free)*

- **Goal:** the same release on Kubernetes.
- **Steps:**
  1. Create two Deployments (`track: blue`, `track: green`) and the Service from step 6.
  2. Test green with `kubectl port-forward`, then patch the Service selector, then patch it back, with a `curl` loop running through a second port-forward to the Service.
  3. **Cleanup:** `kind delete cluster`.
- **Done when:** the loop shows the version change at the patch time and again at rollback, and you can show `kubectl get endpoints shop-api` changing.

☐ **T5 — Break it: database incompatibility** *(≈60 min, advanced)*

- **Goal:** feel why the database is the hard part.
- **Steps:**
  1. With a scratch PostgreSQL (L5.9), let v2 depend on a column that v1 does not know about, and (separately) on a rename that v1 cannot read.
  2. Switch to v2, then roll back to v1 and observe which variant fails.
  3. Redo the change with expand/contract so both versions work at every step.
- **Done when:** you have one rollback that fails and one that succeeds, and you can state the rule that makes the difference.

☐ **T6 — Prove it: cost and decision memo** *(≈45 min, advanced)*

- **Goal:** decide when blue/green is worth it.
- **Steps:**
  1. Without the guide, write a one-page memo for `shop-api`: how the switch works, how rollback works, what happens to the database and sessions, and the extra cost per release for your assumed environment price and overlap.
  2. Compare with rolling and canary in a three-row table (rollback speed, cost, risk exposure).
  3. Present it to a teammate and record their questions.
- **Done when:** the memo contains numbers for cost and rollback time, and a recommendation with the conditions under which you would change it.

10 Self-check questions

1. What is the difference between "idle" green and a staging environment?
2. Why is rollback in blue/green faster than in rolling deployments?
3. Why is DNS a poor switch for instant rollback?

4. What happens to the shared database in a blue/green release, and what discipline does it require?
5. What cost does blue/green add, and how do you reduce it?
6. What must be true of green before you switch to it?

ANSWERS

1. Green runs on the production infrastructure with production configuration and data connections, only without user traffic; staging is a separate, usually smaller environment with different data and dependencies.
2. Rollback is flipping a router back to an environment that is still running the old version; rolling needs another full instance-by-instance rollout.
3. Resolvers and clients cache answers beyond the TTL, so some traffic keeps going to the old side for minutes or longer, and you cannot guarantee a precise or instant cutover.
4. Both colours use the same database, so every schema change must work with the old and new code (expand/contract), and destructive contract steps wait until the rollback window has passed.
5. Double capacity during the overlap; reduce by scaling the idle side to a minimum between releases, keeping the overlap short, and automating teardown.
6. It runs the intended version from the same artifact, passes health and smoke tests through a private route, has production-equivalent configuration, and its dependencies (database schema, queues) are compatible.

11 Further reading

- Martin Fowler's site: *BlueGreenDeployment*; *Continuous Delivery* by Jez Humble and David Farley (release strategies).
- AWS Docs: *AWS CodeDeploy blue/green deployments*, *Application Load Balancer listener rules and weighted target groups*, *Elastic Beanstalk blue/green deployments*.
- Argo Rollouts Docs: BlueGreen strategy; Kubernetes Docs: *Services* (selectors).
- Cloud Foundry Docs: *Using blue-green deployment to reduce downtime and risk*.
- Next: Canary deployments §5.12. Related: Deployment Strategies §0.7, Immutable Infrastructure §0.5.

LAB NOTES What I ran, what I saw, what it means

5.12 Canary Deployments

Topic 12 of 12 | ~5 hours | Difficulty: Advanced

1 What it is

A **canary deployment** releases a new version to a small slice of real traffic first (for example 1%, then 5%, 25%, 50%, 100%), measures how that slice behaves compared with the version already running, and only **promotes** the release further when the measurements are good. If they are bad, the canary is aborted and traffic returns to the stable version. The name comes from the canary in a coal mine: a small, early warning that costs little when it dies.

Compared with the other strategies in this level:

Strategy	Who sees the new version first	How you decide to continue
Rolling §5.10	whoever hits the first replaced instances	readiness probes only
Blue/green §5.11	nobody until the switch, then everybody	tests before the switch, bake after
Canary	a **chosen, small percentage** of real users	**metrics** compared with the stable version, step by step

The three ingredients: **progressive traffic shifting** (weights or a header/cookie rule), **automated analysis** (error rate, latency, saturation and business metrics) and **automatic promotion or abort** based on thresholds you defined in advance.

2 Why it matters in DevOps

- **Smallest blast radius with real traffic:** tests cannot reproduce production data, load patterns and odd clients. A canary exposes the new code to a few percent of them and costs a few percent of requests if it fails.
- **Objective release decisions:** "it looks fine" becomes "the canary's error rate was 0.1% against 0.1% for stable, p99 latency within 5%, for 15 minutes".
- **Pairs with SLOs:** the thresholds are your service level objectives applied to the new version (Error Budgets §14.4). A canary that burns budget quickly is aborted.

- **Enables frequent deploys:** if each deploy exposes only a sliver of users and aborts itself, teams deploy more often with less fear.
- **Needs investment:** you need good metrics, traffic control, and a rollback-friendly design. Without metrics, a canary is just a slow rolling deploy.

3 Real-world system example

Netflix, Google and Kayenta. Netflix and Google jointly open-sourced **Kayenta**, the automated canary analysis platform integrated with Spinnaker. It runs a **baseline** (a fresh deployment of the current version) and a **canary** (the new version) side by side with comparable traffic, collects metrics from both, and uses statistical tests to produce a pass/fail score. Using a fresh baseline rather than the long-running production fleet avoids bias from cache age, uptime and instance history.

Google SRE. *The Site Reliability Engineering Workbook* has a chapter on **canarying releases**, describing partial, time-limited deployments with evaluation against metrics, and discussing how to pick the canary size, duration and the metrics that matter.

Argo Rollouts and Flagger. On Kubernetes, **Argo Rollouts** (a controller and a `Rollout` resource that replaces `Deployment`) and **Flagger** (a progressive delivery operator that is part of the Flux project) automate canaries: they shift traffic through a service mesh, ingress controller or Gateway API implementation in steps, query Prometheus (and other providers) for metrics, and promote or roll back automatically. Both are widely used open-source tools documented by their projects.

4 Diagram

The control loop of a metrics-based canary:

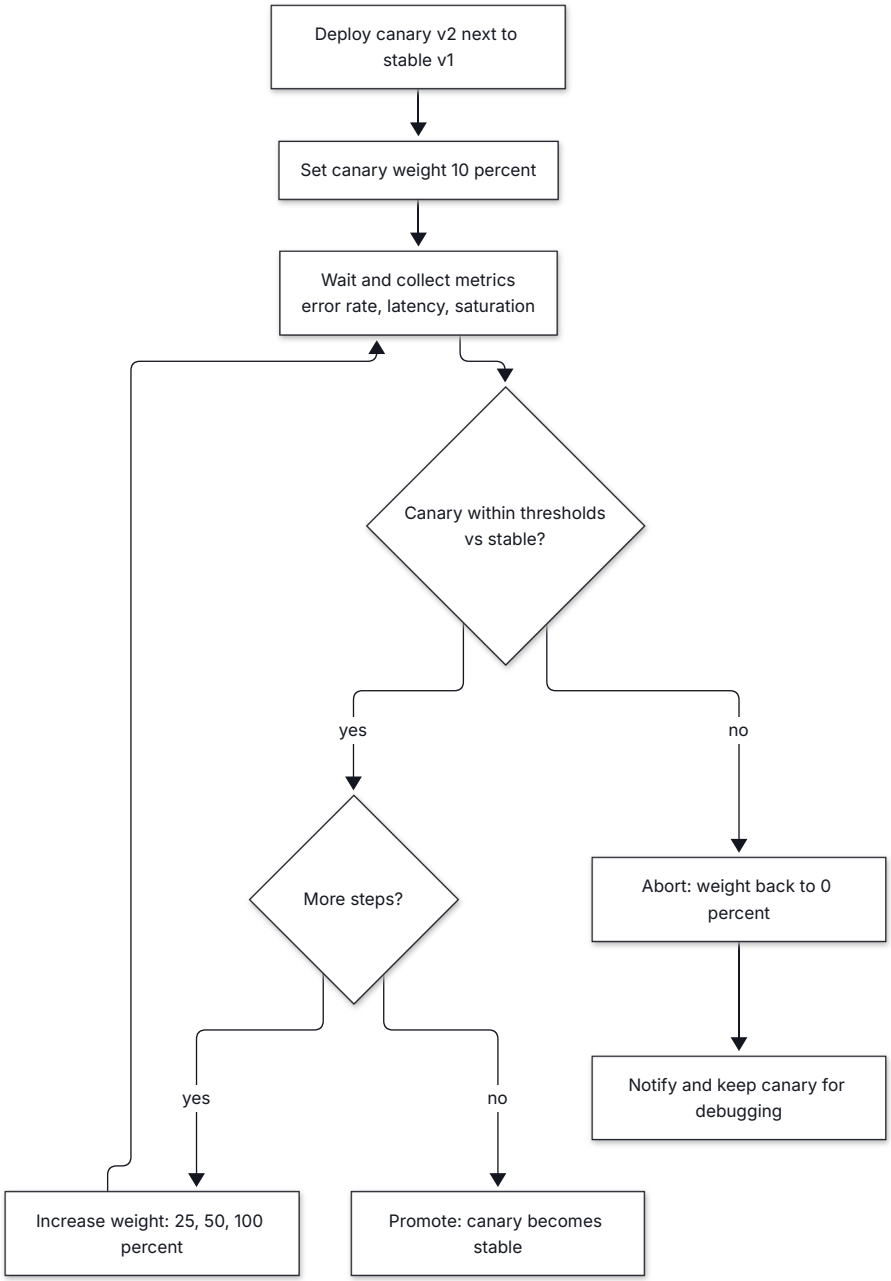


Figure 5.12-1

Traffic during a successful canary and during an aborted one:

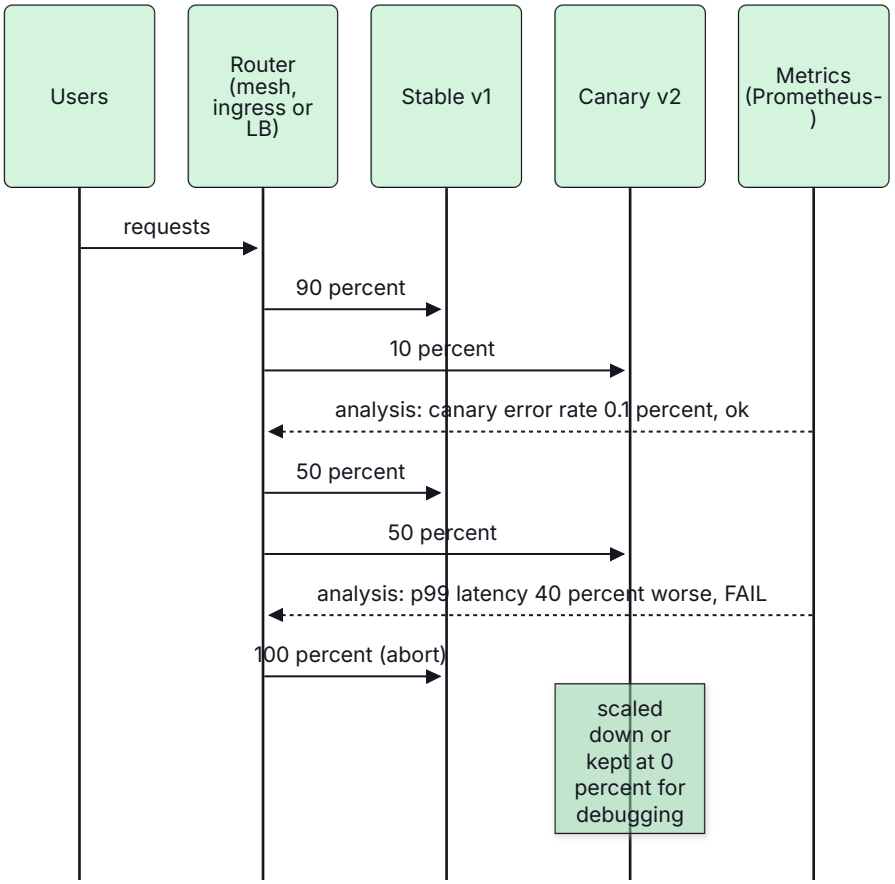


Figure 5.12-2

5 How it works

The moving parts

1. **Traffic splitter:** something that sends N% to the canary. Options: weighted target groups (ALB), Nginx `weight` / `split_clients`, ingress annotations, a service mesh (Istio/Linkerd weights), Gateway API `HTTPRoute` weights, or simply replica ratio (crude: 1 canary pod out of 10 is about 10%).
2. **Metrics with a version label:** every request metric (`http_requests_total`, latency histograms) must be attributable to `version="v2"` vs `version="v1"` (Prometheus and PromQL §10.2, What to Monitor §10.11).

3. **Analysis:** thresholds on the canary's own metrics (absolute), or better a comparison with the stable/baseline (relative).
4. **Controller:** the loop that applies weights, waits, evaluates and either continues or aborts (your pipeline, Argo Rollouts, Flagger, Spinnaker + Kayenta, CodeDeploy).

Which metrics to gate on

Signal	Example query or check	Why
Error rate	share of 5xx among canary requests	crashes, bad queries, broken dependencies
Latency	p95 or p99 of canary versus stable	regressions, missing indexes, N+1 queries
Saturation	CPU, memory, connection pool use per pod	leaks, inefficiency
Application errors	exceptions per minute by version (Sentry, logs)	errors that return 200
Business metrics	checkout success rate, payment errors, cart conversions	the thing that actually pays the bills
Logs/traces	new error signatures	caught by looking, not only by thresholds

Designing the canary so the result means something

- **Enough traffic and time:** with 10 canary requests you cannot tell 0% from 3% errors. Set a *minimum sample size* and duration per step; low-traffic services need longer steps or synthetic traffic.
- **Compare like with like:** same time window, same request mix. Compare the canary against stable (or a fresh baseline), not against last week.
- **Sticky or random?** Random per-request splitting can show one user both versions. Use session affinity or user-ID hashing (cookie, header) when the user experience must stay consistent, or when the API is stateful. Header-based routing (`X-Canary: true`) lets internal users and testers see the canary first.
- **Choose conservative first steps** (1% to 5%), then bigger jumps. Time between steps should cover your metrics scrape interval and alerting delay several times over.
- **Include a failure for the unknown:** also abort on "no data" (a canary that reports no metrics is not healthy).
- **Database and downstream:** both versions still share the database and dependencies; use backwards-compatible migrations (L5.9) and watch downstream saturation.
- **Feature flags** complement canaries: a canary controls which *binary* serves users; a flag controls which *feature* is on, per user, without a deploy.

Argo Rollouts overview

A `Rollout` replaces a `Deployment` and describes the strategy as steps; an `AnalysisTemplate` defines the metrics queries.

YAML

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: shop-api
spec:
  replicas: 5
  selector:
    matchLabels: { app: shop-api }
  template:
    metadata:
      labels: { app: shop-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/acme/shop-api@sha256:<digest>
          ports: [{ containerPort: 3000 }]
          readinessProbe:
            httpGet: { path: /readyz, port: 3000 }
  strategy:
    canary:
      stableService: shop-api-stable
      canaryService: shop-api-canary
      trafficRouting:
        nginx:
          stableIngress: shop-api           # needs the NGINX ingress controller; mesh
  and Gateway API options exist too
      steps:
        - setWeight: 10
        - pause: { duration: 2m }
        - analysis:
            templates: [{ templateName: success-rate }]
            args: [{ name: service, value: shop-api-canary }]
        - setWeight: 50
        - pause: { duration: 5m }
        - setWeight: 100
---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
spec:
  args:
    - name: service
  metrics:
    - name: success-rate
      interval: 1m
```

```
      count: 5
      failureLimit: 1
      successCondition: result[0] >= 0.99
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: |
            sum(rate(http_requests_total{service="{{args.service}}",status!~"5.."}[2m]))
            /
            sum(rate(http_requests_total{service="{{args.service}}"}[2m]))
```

Without a `trafficRouting` block, Argo Rollouts approximates weights by the ratio of canary to stable pods. Operations: `kubectl argo rollouts get rollout shop-api --watch`, `kubectl argo rollouts promote shop-api`, `kubectl argo rollouts abort shop-api` (the plugin is installed separately). It also supports a blue/green strategy.

Flagger overview

Flagger keeps your normal `Deployment` and a `Canary` resource that points to it. It creates the primary/canary workloads and services, shifts traffic through your mesh or ingress (Istio, Linkerd, NGINX, Gateway API, and others), runs metric checks and webhooks (load tests, smoke tests), and promotes or rolls back:

YAML

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: shop-api
spec:
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: shop-api }
  service: { port: 80, targetPort: 3000 }
  analysis:
    interval: 1m          # how often to evaluate
    threshold: 5          # failed checks before rollback
    maxWeight: 50         # stop shifting at 50, then promote
    stepWeight: 10
    metrics:
      - name: request-success-rate
        thresholdRange: { min: 99 }
        interval: 1m
      - name: request-duration
        thresholdRange: { max: 500 }     # milliseconds, p99
        interval: 1m
```

Argo Rollouts vs Flagger vs your own pipeline

	Your pipeline script	Argo Rollouts	Flagger
Resource model	anything	replaces `Deployment` with `Rollout`	keeps `Deployment`, adds `Canary`
Traffic control	proxy or LB API calls	ingress, mesh, Gateway API, replica ratio	mesh, ingress, Gateway API
Metrics	queries you write	`AnalysisTemplate` (Prometheus, Datadog, others)	built-in checks and custom metric templates
Best for	VMs, simple setups, learning	teams using Argo CD and wanting a UI and kubectl plugin	teams using Flux or meshes

6 Step-by-step guide

Prerequisites: Node.js 22, `curl`, Nginx (Debian/Ubuntu: `sudo apt-get install -y nginx`, used from a private directory). No cloud account. The local demo below was run in a sandbox, including a promotion and an abort; counts are "similar to" on your machine. The Kubernetes sections are reference material that you can apply to kind in the tasks.

1. **A tiny API whose error rate you can dial.** Save as `api.js` (a stand-in for `shop-api`; `FAIL_PCT` makes that percentage of requests fail with 500):

```javascript
const http = require("node:http");
const version = process.env.APP_VERSION || "dev";
const failPct = Number(process.env.FAIL_PCT || 0);   // percent of requests that fail
  with 500
http.createServer((req, res) => {
  const fail = Math.random() * 100 < failPct;
  res.writeHead(fail ? 500 : 200, { "content-type": "application/json", "x-version":
  version });
  res.end(JSON.stringify({ version, ok: !fail }) + "\n");
}).listen(Number(process.env.PORT || 3000));
```

2. **Nginx with a rewritable upstream.** Stable on 3000 and canary on 3001; the weights live in an included file:

```bash
mkdir -p nginx/logs nginx/tmp
cat > nginx/nginx.conf <<EOF
worker_processes 1;
pid $PWD/nginx/nginx.pid;
error_log $PWD/nginx/logs/error.log warn;
events { worker_connections 256; }
```

```
http {
  access_log off;
  client_body_temp_path $PWD/nginx/tmp/body; proxy_temp_path $PWD/nginx/tmp/proxy;
  fastcgi_temp_path $PWD/nginx/tmp/fcgi; uwsgi_temp_path $PWD/nginx/tmp/uwsgi;
  scgi_temp_path $PWD/nginx/tmp/scgi;
  include $PWD/nginx/upstream.conf;          # defines: upstream api { ... }
  server {
    listen 8080;
    location / { proxy_pass http://api; }
  }
}
EOF
printf 'upstream api { server 127.0.0.1:3000; }\n' > nginx/upstream.conf
nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx
PORT=3000 APP_VERSION=stable node api.js > s.log 2>&1 &
PORT=3001 APP_VERSION=canary FAIL_PCT=0 node api.js > c.log 2>&1 &
sleep 1; curl -s localhost:8080/api/orders      # {"version":"stable","ok":true}
```

3. **Write the canary controller.** Save as `canary.sh` and `chmod +x canary.sh`. It shifts traffic in stages, analyses the canary's error rate at each stage, and aborts on a bad result or too little data:

BASH

```
#!/usr/bin/env bash
# Progressive canary: stable on :3000, canary on :3001.
set -euo pipefail
STAGES=(10 25 50 100)
MAX_ERR_PCT=5          # abort if the canary error rate exceeds this
MIN_SAMPLES=10         # need at least this many canary requests to judge
PROBES=100             # requests per analysis window
URL=http://127.0.0.1:8080/api/orders

set_weight() {   # set_weight <canary-percent>: rewrite the upstream and reload
  gracefully
  local w=$1
  if   [ "$w" -ge 100 ]; then printf 'upstream api { server 127.0.0.1:3001; }\n' >
  nginx/upstream.conf
  elif [ "$w" -le 0 ];   then printf 'upstream api { server 127.0.0.1:3000; }\n' >
  nginx/upstream.conf
  else printf 'upstream api {\n  server 127.0.0.1:3000 weight=%d;\n  server
  127.0.0.1:3001 weight=%d;\n}\n' $((100-w)) "$w" > nginx/upstream.conf
  fi
  nginx -s reload -c "$PWD/nginx/nginx.conf" -p "$PWD/nginx" 2>/dev/null
}

analyse() {      # sends PROBES requests; prints "<canary_total> <canary_errors>"
  local total=0 errs=0 code ver
  for _ in $(seq 1 $PROBES); do
    out=$(curl -s -w ' %{http_code}' --max-time 3 "$URL" || echo ' 000')
    code=${out##* }; ver=$(echo "$out" | grep -o '"version":"[^"]*"' | cut -d'"' -f4)
```

```
    if [ "$ver" = "canary" ]; then total=$((total+1)); [ "$code" != 200 ] &&
  errs=$((errs+1)); fi
    sleep 0.02
  done
  echo "$total $errs"
}

for w in "${STAGES[@]}"; do
  set_weight "$w"
  echo "stage: canary weight ${w}%"
  read -r total errs < <(analyse)
  if [ "$w" -lt 100 ] && [ "$total" -lt "$MIN_SAMPLES" ]; then echo "not enough canary
  samples ($total)"; set_weight 0; exit 1; fi
  pct=0; [ "$total" -gt 0 ] && pct=$((100*errs/total))
  echo "  canary requests=$total errors=$errs error_rate=${pct}% (limit
  ${MAX_ERR_PCT}%)"
  if [ "$pct" -gt "$MAX_ERR_PCT" ]; then
    echo "ANALYSIS FAILED: aborting canary, all traffic back to stable"; set_weight 0;
  exit 2
  fi
done
echo "PROMOTED: canary is now 100%"
```

4. **Run a healthy canary.** Expected output (similar to):

```
BASH
./canary.sh; echo "exit=$?"
```

```
stage: canary weight 10%
  canary requests=10 errors=0 error_rate=0% (limit 5%)
stage: canary weight 25%
  canary requests=25 errors=0 error_rate=0% (limit 5%)
stage: canary weight 50%
  canary requests=49 errors=0 error_rate=0% (limit 5%)
stage: canary weight 100%
  canary requests=99 errors=0 error_rate=0% (limit 5%)
PROMOTED: canary is now 100%
exit=0
```

Nginx's `weight=` gives a smooth weighted round robin, so 10% is almost exactly 10 of 100 requests: the demo is deterministic, unlike real random traffic.

5. **Run a bad canary and watch it abort.** Reset to stable only, restart the canary with a 30%

failure rate, and run the controller again:

```bash
printf 'upstream api { server 127.0.0.1:3000; }\n' > nginx/upstream.conf
nginx -s reload -c $PWD/nginx/nginx.conf -p $PWD/nginx
pkill -x node; sleep 0.5            # stops the demo servers (careful on a shared
  machine)
PORT=3000 APP_VERSION=stable node api.js > s.log 2>&1 &
PORT=3001 APP_VERSION=canary FAIL_PCT=30 node api.js > c.log 2>&1 &
sleep 1
./canary.sh; echo "exit=$?"
curl -s localhost:8080/api/orders
```

Expected (similar to): at the first stage `canary requests=10 errors=3 error_rate=30% (limit 5%)`, then `ANALYSIS FAILED: aborting canary, all traffic back to stable`, `exit=2`, and the final `curl` answers from `stable`. Only 10% of requests could ever reach the bad version, and only for as long as one analysis window.

6. **Make the same decision from Prometheus (reference).** In production the analysis queries a metrics system. Compare canary and stable error ratios, per version label:

```promql
(
  sum(rate(http_requests_total{app="shop-api",version="v2",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{app="shop-api",version="v2"}[5m]))
)
-
(
  sum(rate(http_requests_total{app="shop-api",version="v1",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{app="shop-api",version="v1"}[5m]))
) > 0.01
```

This expression returns a result only when the canary's error ratio exceeds stable's by more than one percentage point: a relative check that tolerates a generally bad hour for both versions. Add a latency check on `histogram_quantile(0.99, ...)` by version and require a minimum request rate.

7. **Wire it into the pipeline.** After the image is pushed and a baseline deploy is healthy, a job runs the controller (script, Argo Rollouts, or Flagger) and fails if it aborts:

```yaml
jobs:
  canary:
    runs-on: ubuntu-24.04
    timeout-minutes: 45
    environment: production
    steps:
```

```
      - uses: actions/checkout@v4
      - name: Progressive rollout with automatic abort
        env:
          TAG: ${{ inputs.tag }}
        run: |
          ssh "deploy@${{ vars.DEPLOY_HOST }}" "bash -s -- '$TAG'" < scripts/canary-
  deploy.sh
      - name: Notify
        if: always()
        run: echo "canary result for ${{ inputs.tag }} is ${{ job.status }}" >>
  "$GITHUB_STEP_SUMMARY"
```

On Kubernetes with Argo Rollouts, the job instead updates the image in Git or with `kubectl argo rollouts set image shop-api api=ghcr.io/<you>/shop-api@sha256:<digest>` and then waits with `kubectl argo rollouts status shop-api --timeout 20m`, which exits non-zero if the rollout is aborted or degraded.

8. **Clean up.** `pkill -x node` (only in your sandbox), then `nginx -c $PWD/nginx/nginx.conf -p $PWD/nginx -s quit`.

7 Real-world scenario

(Illustrative.) `shop-api` v2 changes the way totals are computed. The team enables a Flagger canary: 10% steps every minute, abort after 3 failed checks, gating on 99% request success and p99 latency below 500 ms. At the 20% step the success-rate check passes, but the p99 latency check fails: a new query is slow for customers with many saved addresses, who are rare in staging but common in production. After three failed checks Flagger routes 100% back to stable and posts to Slack. About 20% of requests saw slower responses for a few minutes; no one was paged.

The engineer reproduces the slow path against a copy of the canary's traces (Distributed Tracing §10.8), adds an index, and re-releases; the second canary passes all steps. Without the canary, the same change in a rolling update would have reached all pods in ten minutes and been caught by customers. **Lesson:** a canary is only as good as its metrics: the latency check, not the error check, caught this one.

8 Common mistakes and best practices

Don't

- Run a canary without version-labelled metrics, or with checks that have too little traffic to be meaningful.
- Gate only on error rate: slow, wrong-but-200 and business-metric regressions pass it.

- Treat "no data" as success.
- Judge a step in seconds; allow for scrape intervals and alert delay.
- Promote automatically past a step that was skipped, paused or never analysed.
- Forget the database: a canary with an incompatible migration breaks stable too.

Do

- Define thresholds (SLO-based) before the release, in code, next to the rollout definition.
- Start tiny (1% to 5%), compare against stable or a fresh baseline, and require a minimum sample size and duration.
- Use sticky routing when users must not flip between versions; use header routing for internal-first canaries.
- Automate abort and rollback, keep the aborted canary available for debugging, and notify the author.
- Combine canaries with feature flags and with backwards-compatible schema changes.
- Review canary outcomes: aborts are the system working; record why they happened.

9 Tasks

- [ ] **T1 — Healthy canary promotion** *(≈45 min, intermediate)*
  - **Goal:** run a staged, analysed canary end to end.
  - **Steps:**
    1. Set up `api.js`, Nginx and `canary.sh` (steps 1 to 3).
    2. Run the canary with `FAIL_PCT=0`.
    3. Read the weights and sample counts per stage.
  - **Done when:** the output ends with `PROMOTED` and exit code `0`, and you can explain why 10% stage had about 10 canary requests out of 100.

- [ ] **T2 — Break it: bad canary aborts** *(≈45 min, intermediate)*
  - **Goal:** prove the abort path protects users.
  - **Steps:**
    1. Restart the canary with `FAIL_PCT=30` and run `canary.sh`.
    2. Count how many failed requests users could have seen (canary errors at the first stage).
    3. Confirm traffic is back to stable afterwards.
  - **Done when:** exit code is `2`, the final `curl` answers `stable`, and you have the number of failed canary requests.

- [ ] **T3 — Add a latency check** *(≈60 min, advanced)*
  - **Goal:** gate on more than errors.
  - **Steps:**
    1. Add `SLOW_MS` to `api.js` (a delay for canary requests) and measure latency per request in `analyse` (`curl -w '%{time_total}'`).
    2. Abort if the canary's p95 (or average) exceeds stable's by more than 30%.
    3. Test with `SLOW_MS=300`.
  - **Done when:** a version with correct responses but slower latency is aborted by the latency check while the error check alone would have passed it.
- [ ] **T4 — Minimum samples and "no data"** *(≈30 min, intermediate)*
  - **Goal:** avoid false confidence.
  - **Steps:**
    1. Reduce `PROBES` to 20 and the first stage to 1% and run `canary.sh`.
    2. Observe the "not enough canary samples" abort.
    3. Explain how you would adapt for a low-traffic service (longer windows, synthetic traffic).
  - **Done when:** the script aborts on insufficient data rather than promoting, and your notes contain two ways to get more signal.
- [ ] **T5 — Argo Rollouts on kind** *(≈90 min, advanced; kind, free)*
  - **Goal:** run a canary with the Kubernetes controller.
  - **Steps:**
    1. Create a kind cluster, install Argo Rollouts and its kubectl plugin following the project's installation documentation.
    2. Apply a `Rollout` with `setWeight: 20`, `pause: {}`, `setWeight: 60`, `pause: {duration: 30s}` and a replica-ratio split (no traffic router needed).
    3. Update the image; use `kubectl argo rollouts get rollout shop-api --watch`, then `promote`; repeat with `abort`.
    4. **Cleanup:** `kind delete cluster`.
  - **Done when:** you saw the rollout pause at 20%, promote to 100% on `promote`, and return to the stable ReplicaSet on `abort`.
- [ ] **T6 — Prove it: design the canary policy for** `shop-api` *(≈60 min, advanced)*
  - **Goal:** choose steps, metrics and thresholds from the service's SLOs.
  - **Steps:**

1. Without the guide, write the policy: step weights and durations, three gating metrics with thresholds (including one business metric), the minimum sample size, what "no data" does, and who is notified on abort.
2. Implement the gates in `canary.sh` (or an `AnalysisTemplate`) and run one passing and one failing release.
3. Explain in writing when you would choose canary over rolling or blue/green for this service.

- **Done when:** both runs behave as your policy predicts and the write-up states the trade-offs (complexity, metric quality, traffic volume).

10 Self-check questions

1. How does a canary differ from a rolling update in how it decides to continue?
2. Why compare the canary against the stable version instead of an absolute threshold only?
3. What happens if the canary receives too little traffic during a step?
4. Why must metrics carry a version label?
5. What does Argo Rollouts add to a plain `Deployment`, and how does Flagger work with one?
6. Give two situations where a canary is the wrong tool.

ANSWERS

1. A rolling update proceeds when readiness probes pass; a canary proceeds only when measured metrics for a controlled traffic slice stay within thresholds, step by step.
2. Conditions change (a traffic spike, a slow dependency) and affect both versions; a relative comparison isolates the effect of the new code, and a fresh baseline avoids bias from cache age and uptime.
3. The result is statistically meaningless; require a minimum sample size/duration, extend steps, add synthetic traffic, and treat missing data as failure.
4. Without it you cannot separate the canary's errors and latency from the stable version's, so the analysis cannot judge the change.
5. Argo Rollouts replaces `Deployment` with a `Rollout` resource that has steps, traffic routing and analysis; Flagger keeps the Deployment and manages primary/canary copies and traffic through a `Canary` resource, both promoting or aborting on metric results.
6. Very low-traffic services (not enough signal), changes that cannot be exposed to a subset (a database cutover with incompatible schema), or releases that need an instant full switch with a pre-tested environment (blue/green).

11 Further reading

- *The Site Reliability Engineering Workbook* (Google), chapter *Canarying Releases*; Spinnaker documentation on Kayenta and automated canary analysis.
- Argo Rollouts documentation (canary strategy, analysis, traffic management) and Flagger documentation (progressive delivery, metrics, deployment strategies).
- Prometheus Docs: *Querying basics* and *Histograms and summaries*.
- Amazon Builders' Library: *Automating safe, hands-off deployments*.
- Next: the Level 5 capstone Capstone 5. Related: Deployment Strategies §0.7, Prometheus and PromQL §10.2, Argo CD and Flux §17.2.

LEVEL 5: CI/CD

★ Push-to-Production Pipeline for shop-api

Level 5 capstone ~12 hours

1 Goal and outcome

Build the roadmap's practice pipeline end to end for the `shop-api` sample application:

Git Push → GitHub → Lint → Tests → Build → Docker Build → Security Scan → Push Image → Deploy → Health Check → Notify, with an automatic **rollback** when the health gate fails.

When you finish, a `git push` to `main` will: lint the code, the Dockerfile and the workflows; run unit and integration tests against real PostgreSQL and Redis containers; build the image with a layer cache; scan it and fail on fixable HIGH or CRITICAL vulnerabilities; push it by digest to GHCR with signed provenance; deploy the same digest to **staging** and then (after approval) **production** on a Linux server using a **blue/green** switch with a health gate and bake time; roll back by itself if the new version misbehaves; and post the result to Slack. You will also be able to roll back with one button, and you will have rehearsed the failure cases.

This ties together every topic of the level: L5.1 pipeline design, L5.2 GitHub Actions, L5.3 testing, L5.4 Docker builds and registries, L5.5 secrets, L5.6 artifacts, L5.7 rollback, L5.8 zero-downtime, L5.9 migrations, L5.10 rolling, L5.11 blue/green and L5.12 canary.

2 Architecture

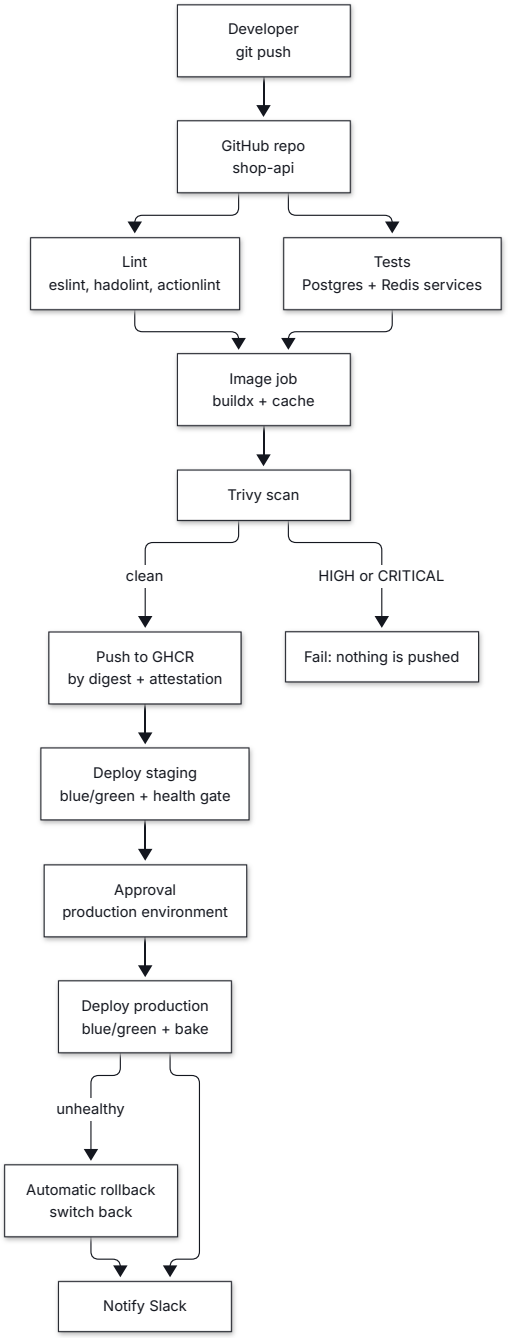


Figure 5-1

What runs on the server (one small Linux VM hosts both environments, separated by ports and directories):

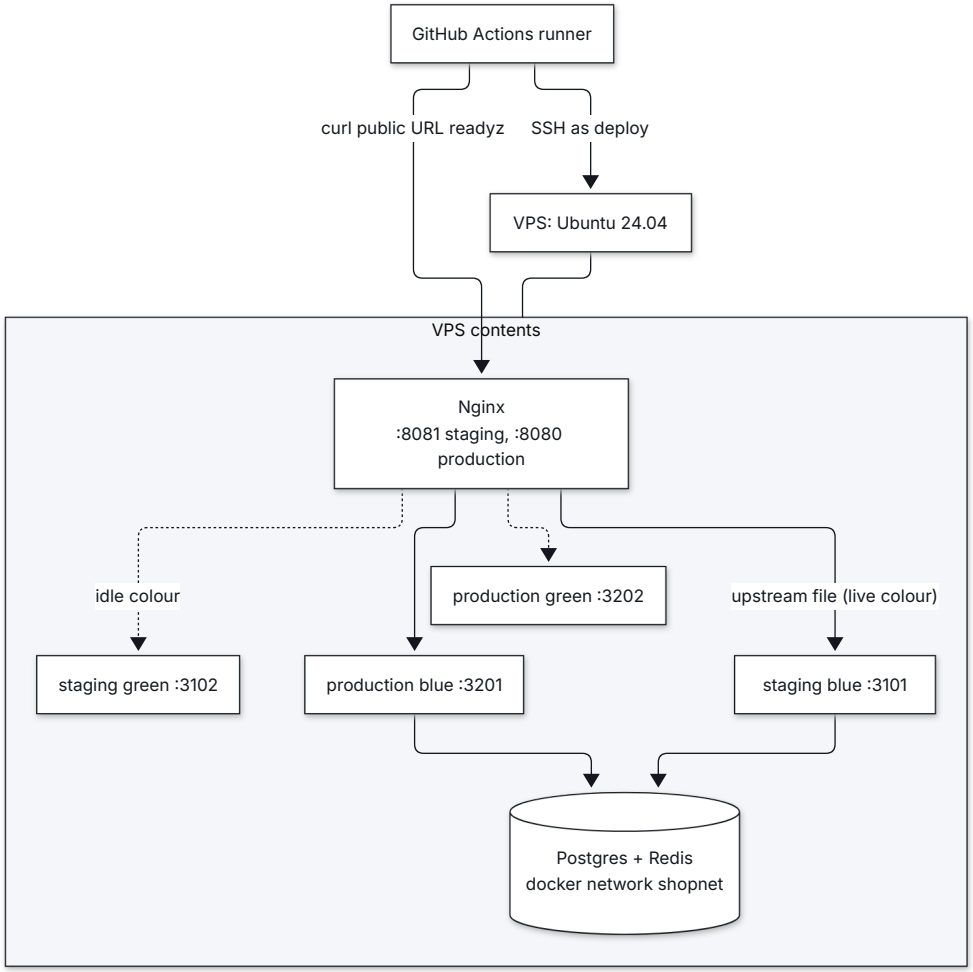


Figure 5-2

3 Prerequisites and cost

- **Accounts and tools:** a GitHub account (public repositories get free Actions minutes and free GHCR storage; private repositories use your plan's quota), `git`, `gh` (GitHub CLI, logged in), `ssh`, `jq`, Docker locally, Node.js 22.
- **A Linux server** reachable over SSH on a public IP: a small VPS (Ubuntu 24.04, 1 vCPU / 1 to 2 GB RAM, typically a few US dollars per month at most providers), a free-tier cloud VM, or a local VM with a bridged network if you only use your own runner or tunnel. The

capstone uses `203.0.113.10` as the illustrative address (RFC 5737; replace it).

- **Slack (optional):** an incoming webhook URL. Without it, the notify job prints to the run summary only.
- **Cost warning:** the VPS is the only cost. Destroy it in the teardown step. Do not leave a server with a public SSH port and open ports 8080/8081 running unattended. Set a billing alert with your provider.
- **Knowledge:** you should have completed topics L5.1 to L5.12 (or have them open), plus Docker §4.7 and Git and GitHub §3.11.
- **Time:** about 12 hours, spread over a few sessions. Milestones are independent enough to stop after each.

4 Build it step by step

Replace `<you>` with your GitHub user and `203.0.113.10` with your server's address throughout.

Milestone 0 — The application contract (≈1 h)

The pipeline assumes `shop-api` provides the following. If your sample app lacks them, add them now (you wrote most of this in earlier topics).

Requirement	Detail	Topic
`npm run lint` and `npm test`	lint exits non-zero on problems; tests use `DATABASE_URL`, `REDIS_URL`, `SESSION_SECRET` from the environment	L5.3
`GET /livez`	200 if the process is alive; no dependency checks	L5.8
`GET /readyz`	`{"status":"ok","version":"<APP_VERSION>"}` with 200 when ready, 503 while starting or shutting down	L5.8
Graceful `SIGTERM`	fail readiness, drain, close server, exit 0	L5.8
Fail-fast config	exit with a clear message if a required variable is missing; no secrets in the image	L5.5
`Dockerfile`	multi-stage, runs as non-root, `CMD ["node","src/server.js"]`, listens on `$PORT` (3000)	L5.4
Backwards-compatible migrations	optional `npm run migrate` that applies only expand-phase changes	L5.9

Create the repository and push it:

```bash
gh repo create shop-api --public --source=. --remote=origin --push
```

Milestone 1 — Prepare the server (≈1.5 h)

SSH to the server as root (or a sudo user) and run the following once. It installs Docker and Nginx, creates a restricted `deploy` user, a Docker network for the app, PostgreSQL and Redis containers, and an Nginx front door whose "live colour" is controlled by one small file per environment.

BASH

```
apt-get update && apt-get install -y nginx docker.io jq ufw
adduser --disabled-password --gecos "" deploy
usermod -aG docker deploy

# the deploy user may only validate and reload nginx, nothing else
echo 'deploy ALL=(root) NOPASSWD: /usr/sbin/nginx -t, /usr/bin/systemctl reload nginx' >
  /etc/sudoers.d/deploy-nginx
chmod 440 /etc/sudoers.d/deploy-nginx && visudo -cf /etc/sudoers.d/deploy-nginx

# upstream files: the pipeline rewrites these to switch colour (blue=3×01, green=3×02)
mkdir -p /etc/nginx/shop-api && chown deploy: /etc/nginx/shop-api
printf 'upstream shop_api_staging { server 127.0.0.1:3101; }\n'    > /etc/nginx/shop-
  api/staging-upstream.conf
printf 'upstream shop_api_production { server 127.0.0.1:3201; }\n' > /etc/nginx/shop-
  api/production-upstream.conf
chown deploy: /etc/nginx/shop-api/*.conf

cat > /etc/nginx/conf.d/shop-api.conf <<'EOF'
include /etc/nginx/shop-api/*.conf;

server {
  listen 8081;                      # staging
  location / {
    proxy_pass http://shop_api_staging;
    proxy_next_upstream error timeout http_502 http_503;
    proxy_set_header Host $host;
  }
}
server {
  listen 8080;                      # production
  location / {
    proxy_pass http://shop_api_production;
    proxy_next_upstream error timeout http_502 http_503;
    proxy_set_header Host $host;
  }
}
EOF
nginx -t && systemctl reload nginx

# firewall: SSH plus the two demo ports (restrict 8080/8081 to your IP if you can)
ufw allow 22/tcp && ufw allow 8080/tcp && ufw allow 8081/tcp && ufw --force enable

# shared network, database and cache (demo credentials: replace the passwords)
```

```
docker network create shopnet
docker run -d --name db --network shopnet --restart unless-stopped \
  -e POSTGRES_PASSWORD=<DB_PASSWORD> -v shop_pgdata:/var/lib/postgresql/data postgres:16
docker run -d --name cache --network shopnet --restart unless-stopped redis:7
sleep 5
docker exec db psql -U postgres -c "create database shop_staging" -c "create database
  shop_production"

# per-environment application settings (runtime configuration; never committed, never in
  the image)
for env in staging production; do
  install -d -o deploy -g deploy /home/deploy/shop-api-$env
  cat > /home/deploy/shop-api-$env/app.env <<EOF
DATABASE_URL=postgres://postgres:<DB_PASSWORD>@db:5432/shop_$env
REDIS_URL=redis://cache:6379
SESSION_SECRET=<RANDOM_STRING_$env>
LOG_LEVEL=$( [ $env = staging ] && echo debug || echo info )
EOF
  chown deploy: /home/deploy/shop-api-$env/app.env && chmod 600 /home/deploy/shop-
  api-$env/app.env
done
```

Create an SSH key for the pipeline on **your laptop** and authorise it on the server:

BASH

```
ssh-keygen -t ed25519 -N '' -C shop-api-pipeline -f deploy_key
ssh root@203.0.113.10 "install -d -m 700 -o deploy -g deploy /home/deploy/.ssh"
cat deploy_key.pub | ssh root@203.0.113.10 "cat >> /home/deploy/.ssh/authorized_keys &&
  chown deploy: /home/deploy/.ssh/authorized_keys && chmod 600
  /home/deploy/.ssh/authorized_keys"
ssh -i deploy_key deploy@203.0.113.10 'docker ps && sudo -n /usr/sbin/nginx -t'    #
  should list db and cache, and nginx syntax is ok
```

Milestone 2 — GitHub environments, variables and secrets (≈45 min)

Per-environment configuration lives in **Environments** (L5.5). The same workflow runs for both; only these values differ.

BASH

```
REPO=<you>/shop-api
gh api -X PUT repos/$REPO/environments/staging
gh api -X PUT repos/$REPO/environments/production     # then add Required reviewers in
  Settings > Environments

# shared (repository) values
gh variable set SSH_KNOWN_HOSTS --repo $REPO --body "$(ssh-keyscan -t ed25519 203.0.113.10
  2>/dev/null)"

for envname in staging production; do
  if [ $envname = staging ]; then P=1; PORT=8081; else P=2; PORT=8080; fi
```

```
  gh variable set DEPLOY_HOST    --repo $REPO --env $envname --body 203.0.113.10
  gh variable set PUBLIC_URL     --repo $REPO --env $envname --body
  "http://203.0.113.10:$PORT"
  gh variable set APP_NAME       --repo $REPO --env $envname --body "shop-api-$envname"
  gh variable set APP_DIR        --repo $REPO --env $envname --body "/home/deploy/shop-
  api-$envname"
  gh variable set BLUE_PORT      --repo $REPO --env $envname --body "3${P}01"
  gh variable set GREEN_PORT     --repo $REPO --env $envname --body "3${P}02"
  gh variable set PUBLIC_PORT    --repo $REPO --env $envname --body "$PORT"
  gh variable set UPSTREAM_NAME  --repo $REPO --env $envname --body "shop_api_$envname"
  gh variable set UPSTREAM_FILE  --repo $REPO --env $envname --body "/etc/nginx/shop-
  api/$envname-upstream.conf"
  gh secret set DEPLOY_SSH_KEY   --repo $REPO --env $envname < deploy_key
done
# optional
gh secret set SLACK_WEBHOOK_URL --repo $REPO --body '<YOUR_SLACK_WEBHOOK_URL>'
# optional: run migrations before the switch, e.g. gh variable set MIGRATE_CMD --env
  staging --body 'npm run migrate'
rm deploy_key deploy_key.pub    # the private key now lives only in GitHub's encrypted
  secrets (never commit it)
```

In **Settings, Environments, production** add yourself as a required reviewer and restrict deployment branches to `main`. In **Settings, Branches** protect `main` and require the checks `lint` and `test` (Protected Branches §3.8).

Milestone 3 — The pipeline: lint, test, build, scan, push (≈2.5 h)

Save as `.github/workflows/pipeline.yml`. It is the whole pipeline from the push to the call of the deploy workflow and the Slack notification. Pull requests run `lint`, `test` and the build-and-scan (no push, no deploy); pushes to `main` run everything.

YAML

```yaml
name: pipeline

on:
  push:
    branches: [main]
  pull_request:

concurrency:
  group: pipeline-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

permissions:
  contents: read

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}
```

```yaml
jobs:
  lint:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm run lint
      - name: Lint the Dockerfile
        uses: hadolint/hadolint-action@v3.1.0
        with:
          dockerfile: Dockerfile
      - name: Lint the workflows
        run: docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.7 -color

  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: ci-only-password
          POSTGRES_DB: shop_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-timeout 5s --health-retries 10
      redis:
        image: redis:7
        ports: ["6379:6379"]
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 5s --health-timeout 5s --health-retries 10
    env:
      DATABASE_URL: postgres://postgres:ci-only-password@localhost:5432/shop_test
      REDIS_URL: redis://localhost:6379
      SESSION_SECRET: ci-only-not-a-secret
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm test

  image:
    needs: [lint, test]
```

```
    runs-on: ubuntu-24.04
    timeout-minutes: 25
    permissions:
      contents: read
      packages: write
      id-token: write
      attestations: write
    outputs:
      image: ${{ steps.ref.outputs.image }}
      version: ${{ github.sha }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3

      - name: Build for scanning (not pushed)
        uses: docker/build-push-action@v6
        with:
          context: .
          load: true
          tags: shop-api:scan
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Security scan (fail on fixable HIGH or CRITICAL)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:scan
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"

      - name: Log in to GHCR
        if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Compute tags and labels
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          flavor: latest=false
          tags: |
            type=sha,format=long,prefix=sha-
            type=ref,event=branch

      - name: Push image
        id: push
        if: github.event_name == 'push'
        uses: docker/build-push-action@v6
```

```
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          provenance: mode=max
          sbom: true

      - name: Attest build provenance
        if: github.event_name == 'push'
        uses: actions/attest-build-provenance@v2
        with:
          subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          subject-digest: ${{ steps.push.outputs.digest }}
          push-to-registry: true

      - name: Export the immutable image reference
        id: ref
        if: github.event_name == 'push'
        env:
          DIGEST: ${{ steps.push.outputs.digest }}
        run: |
          IMAGE="${REGISTRY}/${IMAGE_NAME,,}@${DIGEST}"
          echo "image=$IMAGE" >> "$GITHUB_OUTPUT"
          echo "### Image" >> "$GITHUB_STEP_SUMMARY"
          echo "\`$IMAGE\`" >> "$GITHUB_STEP_SUMMARY"

  staging:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: image
    uses: ./.github/workflows/deploy.yml
    with:
      environment: staging
      image: ${{ needs.image.outputs.image }}
      version: ${{ needs.image.outputs.version }}
    secrets: inherit
    permissions:
      contents: read
      packages: read
      attestations: read

  production:
    needs: [image, staging]
    uses: ./.github/workflows/deploy.yml
    with:
      environment: production
      image: ${{ needs.image.outputs.image }}
      version: ${{ needs.image.outputs.version }}
    secrets: inherit
    permissions:
      contents: read
```

```
      packages: read
      attestations: read

  notify:
    if: always() && github.event_name == 'push'
    needs: [lint, test, image, staging, production]
    runs-on: ubuntu-24.04
    timeout-minutes: 5
    steps:
      - name: Post the result to Slack
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
          LINT: ${{ needs.lint.result }}
          TEST: ${{ needs.test.result }}
          IMAGE: ${{ needs.image.result }}
          STAGING: ${{ needs.staging.result }}
          PRODUCTION: ${{ needs.production.result }}
        run: |
          MSG="shop-api ${GITHUB_SHA::7} by ${GITHUB_ACTOR}: lint=$LINT test=$TEST
image=$IMAGE staging=$STAGING production=$PRODUCTION
${GITHUB_SERVER_URL}/${GITHUB_REPOSITORY}/actions/runs/${GITHUB_RUN_ID}"
          echo "$MSG" >> "$GITHUB_STEP_SUMMARY"
          if [ -z "${SLACK_WEBHOOK_URL:-}" ]; then
            echo "SLACK_WEBHOOK_URL not set: skipping Slack"
            exit 0
          fi
          curl -fsS -X POST -H 'Content-type: application/json' \
            --data "$(jq -n --arg text "$MSG" '{text: $text}')" "$SLACK_WEBHOOK_URL"
```

Things to notice (each maps to a topic):

- `lint` and `test` run **in parallel**; `image` waits for both (fail fast, L5.1). `concurrency` cancels superseded PR runs but queues `main` runs (L5.2).
- The image is **built once**, scanned **before** it is pushed, and only then pushed (re-using the cache) with provenance and an SBOM; the output is the immutable `repo@sha256:...` reference (L5.4, L5.6).
- Both deploy jobs receive the **same digest**; only the environment differs (L5.6).
- `permissions` are minimal at the top and widened only on the image job; the registry login uses `GITHUB_TOKEN`, with no stored password (L5.5).

Milestone 4 — The deploy script: blue/green, health gate, bake, rollback (≈2 h)

Save as `scripts/deploy.sh`. It runs **on the server** (the workflow streams it over SSH, so the version in Git is always the version that runs). It deploys to the idle colour, gates on `/readyz` reporting the new version, switches Nginx, bakes, and switches back automatically if the

public endpoint misbehaves. The control flow of every path (first deploy, upgrade, never-healthy, failing during bake, instant and slow rollback) was exercised against stubbed `docker`, `curl` and `nginx` commands; on a real server expect timing differences.

BASH

```
#!/usr/bin/env bash
# Blue/green deploy with health gate, bake time and automatic rollback.
#   deploy.sh deploy <image-ref> <version>     deploy a new version to the idle colour
#   deploy.sh rollback                         switch back to the previous colour (or
  redeploy previous)
# Required env: APP_NAME APP_DIR BLUE_PORT GREEN_PORT PUBLIC_PORT UPSTREAM_NAME
  UPSTREAM_FILE
# Optional env: MIGRATE_CMD (run once in a throwaway container before the switch),
  BAKE_SECONDS, DOCKER_NETWORK
# Exit codes: 0 ok, 1 failed before switching, 2 failed after switching and rolled back, 3
  rollback failed
set -euo pipefail

CMD="${1:?usage: deploy.sh deploy <image> <version> | rollback}"
: "${APP_NAME:?}" "${APP_DIR:?}" "${BLUE_PORT:?}" "${GREEN_PORT:?}" "${PUBLIC_PORT:?}"
  "${UPSTREAM_NAME:?}" "${UPSTREAM_FILE:?}"
BAKE_SECONDS="${BAKE_SECONDS:-30}"
NETWORK="${DOCKER_NETWORK:-shopnet}"      # user-defined network that also holds the db
  and cache containers
mkdir -p "$APP_DIR"
cd "$APP_DIR"

live="$(cat live.color 2>/dev/null || echo none)"
port_of() { if [ "$1" = blue ]; then echo "$BLUE_PORT"; else echo "$GREEN_PORT"; fi; }
other()   { if [ "$1" = blue ]; then echo green; else echo blue; fi; }

serving() {   # serving <port> <version>: readiness says ok AND reports this version
  curl -fsS --max-time 3 "http://127.0.0.1:$1/readyz" | jq -e --arg v "$2" '.status ==
  "ok" and .version == $v' >/dev/null 2>&1
}

wait_ready() {   # wait_ready <port> <version>: 5 consecutive good answers within ~2
  minutes
  local ok=0
  for _ in $(seq 1 60); do
    if serving "$1" "$2"; then ok=$((ok+1)); else ok=0; fi
    [ "$ok" -ge 5 ] && return 0
    sleep 2
  done
  return 1
}

switch_to() {   # switch_to <colour>: atomic upstream swap + validated graceful reload
  printf 'upstream %s { server 127.0.0.1:%s; }\n' "$UPSTREAM_NAME" "$(port_of "$1")" >
  "$UPSTREAM_FILE.new"
  mv "$UPSTREAM_FILE.new" "$UPSTREAM_FILE"
```

```
  sudo -n /usr/sbin/nginx -t
  sudo -n /usr/bin/systemctl reload nginx
  echo "$1" > live.color
  echo "traffic now goes to $1 ($(port_of "$1"))"
}

start_colour() {   # start_colour <colour> <image> <version>
  local name="$APP_NAME-$1" port
  port="$(port_of "$1")"
  docker rm -f "$name" >/dev/null 2>&1 || true
  docker pull "$2"
  docker run -d --name "$name" --network "$NETWORK" --restart unless-stopped --stop-
  timeout 30 \
    --env-file "$APP_DIR/app.env" -e APP_VERSION="$3" -e PORT=3000 \
    -p "127.0.0.1:$port:3000" \
    --health-cmd "wget -qO- http://127.0.0.1:3000/livez || exit 1" --health-interval 5s --
  health-retries 5 \
    "$2" >/dev/null
}

bake() {   # bake <version>: watch the PUBLIC endpoint for BAKE_SECONDS, tolerate at most
  2 failures
  local fails=0 end=$((SECONDS+BAKE_SECONDS))
  while [ "$SECONDS" -lt "$end" ]; do
    serving "$PUBLIC_PORT" "$1" || fails=$((fails+1))
    [ "$fails" -gt 2 ] && return 1
    sleep 2
  done
  return 0
}

do_deploy() {   # do_deploy <image> <version>
  local image="$1" version="$2" idle
  idle="$(other "$live")"; [ "$live" = none ] && idle=blue
  echo "live=$live idle=$idle version=$version"

  if [ -n "${MIGRATE_CMD:-}" ]; then
    echo "running migrations (expand only): $MIGRATE_CMD"
    docker run --rm --network "$NETWORK" --env-file "$APP_DIR/app.env" "$image" sh -c
  "$MIGRATE_CMD"
  fi

  start_colour "$idle" "$image" "$version"
  if ! wait_ready "$(port_of "$idle")" "$version"; then
    echo "DEPLOY FAILED: $idle never became healthy on $version; NOT switching" >&2
    docker logs --tail 50 "$APP_NAME-$idle" >&2 || true
    exit 1
  fi

  switch_to "$idle"
  if ! bake "$version"; then
    echo "BAKE FAILED: switching back to $live" >&2
    if [ "$live" ≠ none ]; then
```

```
      switch_to "$live"
      exit 2
    fi
    exit 1
  fi

  [ -f current.version ] && cp current.version previous.version && cp current.image
  previous.image
  echo "$version" > current.version
  echo "$image" > current.image
  echo "DEPLOY OK: $version live on $idle ($live kept warm for rollback)"
}

do_rollback() {
  [ "$live" =≠ none ] || { echo "nothing to roll back" >&2; exit 3; }
  local target prev_version
  target="$(other "$live")"
  prev_version="$(cat previous.version 2>/dev/null || true)"
  if [ -n "$prev_version" ] && serving "$(port_of "$target")" "$prev_version"; then
    echo "instant rollback: $target is still running $prev_version"
    switch_to "$target"
    for f in version image; do                        # swap the current/previous records
      mv "current.$f" "tmp.$f"; mv "previous.$f" "current.$f"; mv "tmp.$f" "previous.$f"
    done
  elif [ -f previous.image ]; then
    echo "previous colour is gone: redeploying $(cat previous.image)"
    do_deploy "$(cat previous.image)" "$prev_version"
  else
    echo "ROLLBACK IMPOSSIBLE: no previous release recorded" >&2
    exit 3
  fi
  echo "ROLLED BACK to $prev_version"
}

case "$CMD" in
  deploy)   do_deploy "${2:?image}" "${3:?version}" ;;
  rollback) do_rollback ;;
  *) echo "unknown command $CMD" >&2; exit 1 ;;
esac
```

Exit codes carry the story: `0` success, `1` failed before any switch (live version untouched), `2` failed after switching and was rolled back, `3` the rollback itself failed (page a human). The previous colour keeps running after a successful deploy, so a manual rollback is instant (L5.7, L5.11). Migrations, when configured through `MIGRATE_CMD`, run once in a throwaway container before the switch and must be expand-only (L5.9).

Milestone 5 — The reusable deploy workflow (≈1.5 h)

Save as `.github/workflows/deploy.yml`. It is called by the pipeline for staging and production, and can also be run by hand with an image reference. It verifies the image's **attestation** (only images built by this repository's workflow are deployed), runs the script, then checks the result **from outside**, through the public URL, as a user would.

YAML

```yaml
name: deploy

on:
  workflow_call:
    inputs:
      environment:
        type: string
        required: true
      image:
        description: Immutable image reference (registry/repo@sha256:...)
        type: string
        required: true
      version:
        description: Version reported by /readyz (the commit SHA)
        type: string
        required: true
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
        required: true
      image:
        description: Immutable image reference (registry/repo@sha256:...)
        required: true
      version:
        description: Commit SHA that built the image
        required: true

permissions:
  contents: read
  packages: read
  attestations: read

jobs:
  deploy:
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    environment: ${{ inputs.environment }}
    concurrency:
      group: deploy-${{ inputs.environment }}
      cancel-in-progress: false
    env:
```

```
      GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      IMAGE: ${{ inputs.image }}
      VERSION: ${{ inputs.version }}
      HOST: ${{ vars.DEPLOY_HOST }}
      PUBLIC_URL: ${{ vars.PUBLIC_URL }}
    steps:
      - uses: actions/checkout@v4

      - name: Verify provenance (must be built by this repository's workflow)
        run: gh attestation verify "oci://$IMAGE" --repo "$GITHUB_REPOSITORY"

      - name: Configure SSH
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.SSH_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts

      - name: Deploy (migrate, blue/green, health gate, bake, auto rollback)
        id: deploy
        env:
          APP_NAME: ${{ vars.APP_NAME }}
          APP_DIR: ${{ vars.APP_DIR }}
          BLUE_PORT: ${{ vars.BLUE_PORT }}
          GREEN_PORT: ${{ vars.GREEN_PORT }}
          PUBLIC_PORT: ${{ vars.PUBLIC_PORT }}
          UPSTREAM_NAME: ${{ vars.UPSTREAM_NAME }}
          UPSTREAM_FILE: ${{ vars.UPSTREAM_FILE }}
          MIGRATE_CMD: ${{ vars.MIGRATE_CMD }}
        run: |
          set +e
          ssh "deploy@$HOST" \
            "APP_NAME='$APP_NAME' APP_DIR='$APP_DIR' BLUE_PORT='$BLUE_PORT'
GREEN_PORT='$GREEN_PORT' PUBLIC_PORT='$PUBLIC_PORT' UPSTREAM_NAME='$UPSTREAM_NAME'
UPSTREAM_FILE='$UPSTREAM_FILE' MIGRATE_CMD='$MIGRATE_CMD' bash -s -- deploy '$IMAGE'
'$VERSION'" \
            < scripts/deploy.sh
          rc=$?
          echo "rc=$rc" >> "$GITHUB_OUTPUT"
          exit $rc

      - name: External health check (from the runner, through the public URL)
        run: |
          ok=0
          for i in $(seq 1 30); do
            if curl -fsS --max-time 5 "$PUBLIC_URL/readyz" | jq -e --arg v "$VERSION"
'.status == "ok" and .version == $v' >/dev/null; then
              ok=$((ok+1))
            else
              ok=0
```

```
            fi
            [ "$ok" -ge 5 ] && { echo "healthy: $ok consecutive successes"; exit 0; }
            sleep 2
          done
          echo "external health check FAILED" >&2
          exit 1

      - name: Summary
        if: always()
        env:
          RC: ${{ steps.deploy.outputs.rc }}
          ENVIRONMENT: ${{ inputs.environment }}
        run: |
          case "${RC:-unknown}" in
            0) R="deployed" ;;
            1) R="FAILED before switching (live version untouched)" ;;
            2) R="FAILED after switching and was ROLLED BACK automatically" ;;
            3) R="FAILED and rollback failed: PAGE A HUMAN" ;;
            *) R="did not complete (rc=${RC:-unknown})" ;;
          esac
          echo "### $ENVIRONMENT: $R" >> "$GITHUB_STEP_SUMMARY"
          echo "image: \`$IMAGE\`" >> "$GITHUB_STEP_SUMMARY"
```

Milestone 6 — One-click rollback (≈45 min)

Save as `.github/workflows/rollback.yml`. It reuses the same script: it switches to the previous colour if that colour is still running the previous release (seconds), otherwise it redeploys the previous image.

YAML

```yaml
name: rollback

on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
        required: true

permissions:
  contents: read

jobs:
  rollback:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    environment: ${{ inputs.environment }}
    concurrency:
      group: deploy-${{ inputs.environment }}
      cancel-in-progress: false
```

```
    steps:
      - uses: actions/checkout@v4

      - name: Configure SSH
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.SSH_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts

      - name: Roll back to the previous release
        env:
          HOST: ${{ vars.DEPLOY_HOST }}
          APP_NAME: ${{ vars.APP_NAME }}
          APP_DIR: ${{ vars.APP_DIR }}
          BLUE_PORT: ${{ vars.BLUE_PORT }}
          GREEN_PORT: ${{ vars.GREEN_PORT }}
          PUBLIC_PORT: ${{ vars.PUBLIC_PORT }}
          UPSTREAM_NAME: ${{ vars.UPSTREAM_NAME }}
          UPSTREAM_FILE: ${{ vars.UPSTREAM_FILE }}
        run: |
          ssh "deploy@$HOST" \
            "APP_NAME='$APP_NAME' APP_DIR='$APP_DIR' BLUE_PORT='$BLUE_PORT'
    GREEN_PORT='$GREEN_PORT' PUBLIC_PORT='$PUBLIC_PORT' UPSTREAM_NAME='$UPSTREAM_NAME'
    UPSTREAM_FILE='$UPSTREAM_FILE' bash -s -- rollback" \
            < scripts/deploy.sh

      - name: Show what is live now
        env:
          PUBLIC_URL: ${{ vars.PUBLIC_URL }}
        run: |
          curl -fsS "$PUBLIC_URL/readyz" | tee /dev/stderr | jq -e '.status == "ok"'
    >/dev/null
          echo "rolled back; live version: $(curl -fsS "$PUBLIC_URL/readyz" | jq -r
    .version)" >> "$GITHUB_STEP_SUMMARY"
```

Milestone 7 — First run, then validate (≈1 h)

BASH

```
git add . && git commit -m "ci: add pipeline" && git push origin main
gh run watch                                    # follow the pipeline
curl -s http://203.0.113.10:8081/readyz         # staging: {"status":"ok","version":"
  <commit sha>"}
```

After the staging job passes, the `production` job pauses for approval: approve it in the Actions UI, then `curl -s http://203.0.113.10:8080/readyz`. Make the GHCR package public (package settings) or run `docker login ghcr.io` once as `deploy` if the server cannot pull the image. Lint your workflows locally any time with `docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.7`.

Milestone 8 — Compare the other strategies (≈2 h, optional but recommended)

Blue/green is what you built. To compare with the other two strategies from this level without extra cost, use a local kind cluster: deploy `shop-api` as a rolling update with `maxSurge: 1, maxUnavailable: 0` (L5.10), then install Argo Rollouts and run a canary with `setWeight: 20`, an analysis step, and a promotion (L5.12). Document in a table: rollback speed, extra cost, exposure of a bad release, operational complexity.

5 Tasks

- [ ] **T1 — Bootstrap the server and prove it is safe** *(≈90 min, intermediate; milestones 1 and 2)*
  - **Goal:** a hardened deploy target and complete per-environment configuration. Exercises L5.5 and L5.8.
  - **Steps:**
    1. Run the bootstrap commands; create the SSH key and authorise it; store it only as environment secrets.
    2. Set all variables for `staging` and `production` and enable required reviewers on production.
    3. As the `deploy` user, verify you can run `docker ps` and `sudo -n /usr/sbin/nginx -t` but not `sudo -n bash`.
  - **Done when:** `ssh -i key deploy@host 'sudo -n bash -c id'` is refused, `gh variable list --env production` shows all values, and no secret exists in the repository files.
- [ ] **T2 — Green CI with real services** *(≈90 min, intermediate; milestone 3)*
  - **Goal:** lint and test jobs that run in parallel and gate everything. Exercises L5.1, L5.2, L5.3.
  - **Steps:**
    1. Add `pipeline.yml` (lint and test jobs first) and open a pull request.
    2. Make a test fail on purpose and a lint rule fail on purpose, in separate commits; observe that `image` never starts.
    3. Add the required status checks to branch protection.

- **Done when:** the PR shows `lint` and `test` side by side, a red commit blocks merging, and the run page shows `image` as skipped.

- [ ] **T3 — Image, cache, scan, provenance** *(≈2 h, intermediate; milestone 3)*
  - **Goal:** an image that is scanned before it is pushed and traceable to a commit. Exercises L5.4 and L5.6.
  - **Steps:**
    1. Merge to `main` and read the image job summary for the digest.
    2. Re-run to see the cache hit and note the build time difference.
    3. Add a deliberately vulnerable base image (an old `node` tag) in a branch and confirm the scan fails the job; revert.
    4. Run `gh attestation verify oci://<image-ref> --repo <you>/shop-api`.
  - **Done when:** the registry has `sha-<commit>` and the digest verifies, the second build is clearly faster, and the vulnerable branch never pushed an image.

- [ ] **T4 — Deploy to staging and production with a health gate** *(≈2 h, advanced; milestones 4 to 7)*
  - **Goal:** the first complete push-to-production run. Exercises L5.5, L5.8, L5.9, L5.11.
  - **Steps:**
    1. Add `scripts/deploy.sh`, `deploy.yml` and `rollback.yml`; push to `main`.
    2. Watch staging go green, approve production, and then deploy a second commit so that colours swap.
    3. While deploying the second commit, run `while sleep 0.5; do curl -s -o /dev/null -w '%{http_code}\n' http://203.0.113.10:8081/readyz; done | sort | uniq -c` in another terminal.
    4. Make a harmless expand-only migration (add a nullable column) and run it through `MIGRATE_CMD` on staging.
  - **Done when:** both environments serve the latest commit's SHA at `/readyz`, the second deployment changed the live colour (`cat live.color` on the server) and the status-code count shows only `200`.

- [ ] **T5 — Break it: bad releases and the rollback paths** *(≈2 h, advanced; milestones 4 to 6)*
  - **Goal:** prove that each failure ends safely. Exercises L5.7 and L5.8.
  - **Steps:**
    1. Add drill switches to the app, for example `FAIL_READY=1` (readiness always 500) and `FAIL_AFTER=20` (healthy for 20 seconds, then 500), set through `app.env` on staging. With `FAIL_READY=1`, deploy: expect exit code 1 and the live version unchanged.

2. With `FAIL_AFTER=20`, deploy again (healthy at start, failing during the bake): expect exit code 2 and an automatic switch back.
3. Deploy a good version, then run the `rollback` workflow and measure how long traffic takes to move back.
4. Break the rollback target (stop the idle container with `docker stop`) and roll back again; observe the redeploy path.

- **Done when:** you have the run summaries for all four cases, each stating the right outcome, and `/readyz` always showed a healthy version during and after.

- [ ] **T6 — Expand/contract migration drill** *(≈2 h, advanced; milestone 4)*
  - **Goal:** change the schema without an outage or an unrecoverable rollback. Exercises L5.9.
  - **Steps:**
    1. Plan the three-release change to turn `orders.total` into `orders.total_cents`: expand with a trigger, backfill and dual-write release, then contract.
    2. Ship the expand release through the pipeline; roll back to the previous release and confirm it still works on the new schema.
    3. Ship the migrate release; ship the contract release only after the rollback window.
  - **Done when:** each release deployed and passed its health gate, a rollback after the expand release succeeded, and the contract release was the only one that could not be rolled back past.

- [ ] **T7 — Compare rolling, blue/green and canary** *(≈2 h, advanced; milestone 8)*
  - **Goal:** choose a strategy with evidence. Exercises L5.10, L5.11, L5.12.
  - **Steps:**
    1. Run a rolling update on kind with a loop of requests and count the codes.
    2. Run the local canary script from L5.12 with a healthy and a failing canary.
    3. Fill in a table comparing rollback time, cost, exposure of a bad version, and complexity for all three strategies using your measurements and this capstone's blue/green runs.
  - **Done when:** the table has measured values (not guesses) for at least rollback time and error count, with a recommendation for `shop-api`.

- [ ] **T8 — Prove it: rebuild the pipeline from a blank repository** *(≈3 h, advanced; all milestones)*
  - **Goal:** demonstrate you can design and operate this pipeline without the guide.
  - **Steps:**

1. In a new repository with only the application, write the workflows and the deploy script from memory and a clean server; use the guide only for syntax lookups.
2. Trigger one successful release and one bad release that rolls back automatically.
3. Present it in 10 minutes: the stage diagram, what each gate proves, where secrets live, how rollback works, and what you would change at ten times the traffic.

- **Done when:** the second release is rolled back with no human action, a reviewer can follow the run summaries, and you can answer why the image is built once, scanned before pushing and deployed by digest.

6 Verification checklist

- [ ] A push to `main` triggers the full pipeline; a pull request runs lint, test, build and scan but pushes and deploys nothing.
- [ ] `lint` and `test` run in parallel; a failing test or lint rule blocks the image job.
- [ ] The scan runs before the push and fails on fixable HIGH or CRITICAL findings.
- [ ] The registry contains `sha-<commit>`, the deploy used the `@sha256:` digest, and `gh attestation verify` succeeds for it.
- [ ] Staging and production run the **same digest**, built once; the only differences are environment variables and secrets.
- [ ] Production waits for a human approval; only `main` may deploy to it.
- [ ] No secret appears in the repository, in logs (check the run logs), or in the image (`docker history --no-trunc`).
- [ ] `curl http://<host>:<port>/readyz` returns the commit SHA of the latest release.
- [ ] A release that never becomes healthy leaves the live version untouched (exit code 1).
- [ ] A release that fails during the bake is rolled back automatically (exit code 2) and the Slack or summary message says so.
- [ ] The `rollback` workflow returns to the previous release in seconds when the previous colour is still running.
- [ ] A request loop during a normal deploy shows no non-2xx responses.
- [ ] Schema changes were released as expand, migrate, contract; a rollback after the expand release works.
- [ ] You can state the cost of the blue/green overlap and the retention policy for old images.

7 Teardown

1. **Destroy the server** at your provider (or delete the VM), and remove its firewall rules and any reserved IP. This stops the only recurring cost.
2. **Revoke access:** delete the `shop-api-pipeline` public key wherever you copied it, remove the repository secrets and variables (`gh secret delete DEPLOY_SSH_KEY --env staging`, and likewise for production and `SLACK_WEBHOOK_URL`), and revoke the Slack webhook.
3. **Delete the registry data:** GHCR package versions for `shop-api` (Packages, Package settings, Delete) if you no longer need them.
4. **Remove the environments:** `gh api -X DELETE repos/<you>/shop-api/environments/staging` and `.../production`.
5. **Clean local files:** `rm -f deploy_key deploy_key.pub`, remove kind clusters (`kind delete cluster`) and Docker leftovers (`docker system prune`).
6. Verify the provider billing page shows no running instances and no unattached volumes or IPs.

8 Stretch goals and what to learn next

- Add a **canary** stage: keep 90/10 weights in the Nginx upstream for 10 minutes with a Prometheus-based error-rate check before 100% (L5.12), or replace the VM with Kubernetes and Argo Rollouts.
- Add **preview environments** per pull request (a temporary container on another port, destroyed on PR close).
- Replace SSH deployment with **OIDC to a cloud account** (no stored keys) and deploy to ECS or EKS; build the infrastructure with Terraform.
- Move the desired state to Git and let a controller apply it (**GitOps**): the pipeline only builds and bumps an image digest in a config repository.
- Add DAST (OWASP ZAP) against staging, secret scanning in CI, and Dependabot for actions and dependencies (Level 11 §11.9).
- Add metrics and SLO-based gates to the bake step (Level 10 §10.2, Level 14 §14.4).

Next level: Level 6 — Cloud (AWS) §6.1 moves this pipeline onto managed infrastructure: EC2, IAM roles for OIDC, ALB for the blue/green switch, and Secrets Manager.

LAB NOTES What I ran, what I saw, what it means

PART III

Cloud You Can Rebuild

The AWS building blocks that production runs on, and the Terraform practice that turns them into reviewable, repeatable code.



IN THIS PART

6	Cloud (AWS)	1137
7	Terraform / Infrastructure as Code	1403

LAB NOTES What I ran, what I saw, what it means

LEVEL 6 · PART III

Cloud (AWS)

6

Goal: Learn one cloud deeply; understand AWS building blocks well enough to design and secure a production architecture.

RECOMMENDED TIME
4-6 weeks

TOPICS
18

PRACTICE / CAPSTONE
Build CloudFront → ALB → EC2 instances → Redis → RDS → S3 (use free tier / tear down after; a billing-alarm task is mandatory).

Topics

#	Topic	What you will learn
1	EC2 **p.1141**	instances, AMIs, instance types, key pairs, user data, pricing models intro
2	Auto Scaling **p.1155**	launch templates, ASG, scaling policies, health checks, lifecycle hooks
3	Lambda **p.1167**	event-driven functions, cold starts, triggers, limits, when not to use
4	VPC and Subnets **p.1179**	CIDR planning, public/private subnets, AZs, multi-AZ design
5	Route Tables, Internet Gateway and NAT Gateway **p.1191**	how traffic leaves a subnet, NAT cost, VPC endpoints
6	Security Groups and Network ACLs **p.1203**	stateful vs stateless, SG referencing, layered defense
7	IAM **p.1217**	users, roles, policies, least privilege, instance profiles, STS, policy evaluation
8	S3 **p.1231**	buckets, policies, versioning, lifecycle, encryption, presigned URLs, static hosting
9	EBS and EFS **p.1245**	block vs file storage, snapshots, volume types, when to use which
10	RDS and Aurora **p.1261**	managed Postgres/MySQL, Multi-AZ vs read replicas, backups, Aurora architecture
11	ElastiCache **p.1273**	Redis/Memcached, caching patterns, clusters, failover

#	Topic	What you will learn
12	Route 53 **p.1285**	hosted zones, record types, routing policies, health checks, failover
13	CloudFront **p.1297**	CDN distributions, origins, caching behaviors, OAC, invalidation
14	Application Load Balancer **p.1311**	listeners, rules, target groups, health checks, TLS with ACM
15	KMS **p.1325**	keys, envelope encryption, key policies, rotation
16	Secrets Manager **p.1339**	storing/rotating secrets, app retrieval, vs SSM Parameter Store
17	CloudTrail **p.1353**	API audit logs, management vs data events, investigation queries
18	CloudWatch **p.1369**	metrics, logs, alarms, dashboards, Logs Insights, billing alarm
★	Level capstone project **p.1383**	Build CloudFront → ALB → EC2 instances → Redis → RDS → S3 (use free tier / tear down after; a billing-alarm task is mandatory).

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



17
CloudTrail

18
CloudWatch

★
Capstone

LAB NOTES What I ran, what I saw, what it means

LEVEL 6: CLOUD (AWS)

6.1 EC2

Topic 1 of 18 | ~5 hours | Difficulty: Beginner

1 What it is

Amazon EC2 (Elastic Compute Cloud) rents you virtual machines, called **instances**, by the second. You choose an **AMI (Amazon Machine Image)** — the template containing the operating system and pre-installed software — an **instance type** (how many vCPUs, how much memory, which network and storage capabilities), a network location (a subnet) and a **security group** (a virtual firewall). AWS finds a physical host, boots a copy of the AMI on it and hands you a running Linux or Windows machine in well under a minute.

Everything else in AWS that "runs your code on a server" is built on the same idea. Auto Scaling groups launch EC2 instances, an RDS database runs on EC2-class hardware you cannot see, and EKS worker nodes are EC2 instances. If you understand EC2 — images, sizes, boot-time configuration, access and pricing — you understand the foundation.

2 Why it matters in DevOps

- **Cost:** compute is usually the largest line on the bill. A forgotten `m7i.4xlarge` costs real money every hour; a wrongly chosen pricing model wastes 30-70%.
- **Reliability:** an instance is *cattle, not a pet*. It can be stopped, retired by AWS or lost with its AZ. Designs that treat a single instance as permanent wake people up at 3 a.m.
- **Security:** instance metadata credentials (IMDS), open SSH ports and over-broad key pairs are the classic entry points in cloud breaches.
- **Automation:** AMIs, user data and launch templates are how Infrastructure as Code (Level 0 topic §0.3) and immutable infrastructure (Level 0 topic §0.5) become real.

3 Real-world system example

Netflix has publicly described running its streaming control plane on EC2 and pioneering the "bake an AMI, then launch it" model: the build pipeline produces a machine image containing the application, and the deployment tool (Spinnaker, which Netflix open-sourced)

launches new instances from that image instead of patching running servers. Rolling back means launching the previous image. That pattern — immutable, image-based deployment on EC2 behind Auto Scaling groups — is the one this level builds toward.

Smaller teams use the same primitives: a typical `shop-api` deployment is an AMI (or stock Amazon Linux plus user data) running Nginx and Node.js behind a load balancer, with the database on RDS and static files on S3.

4 Diagram

What happens when you call `RunInstances`:

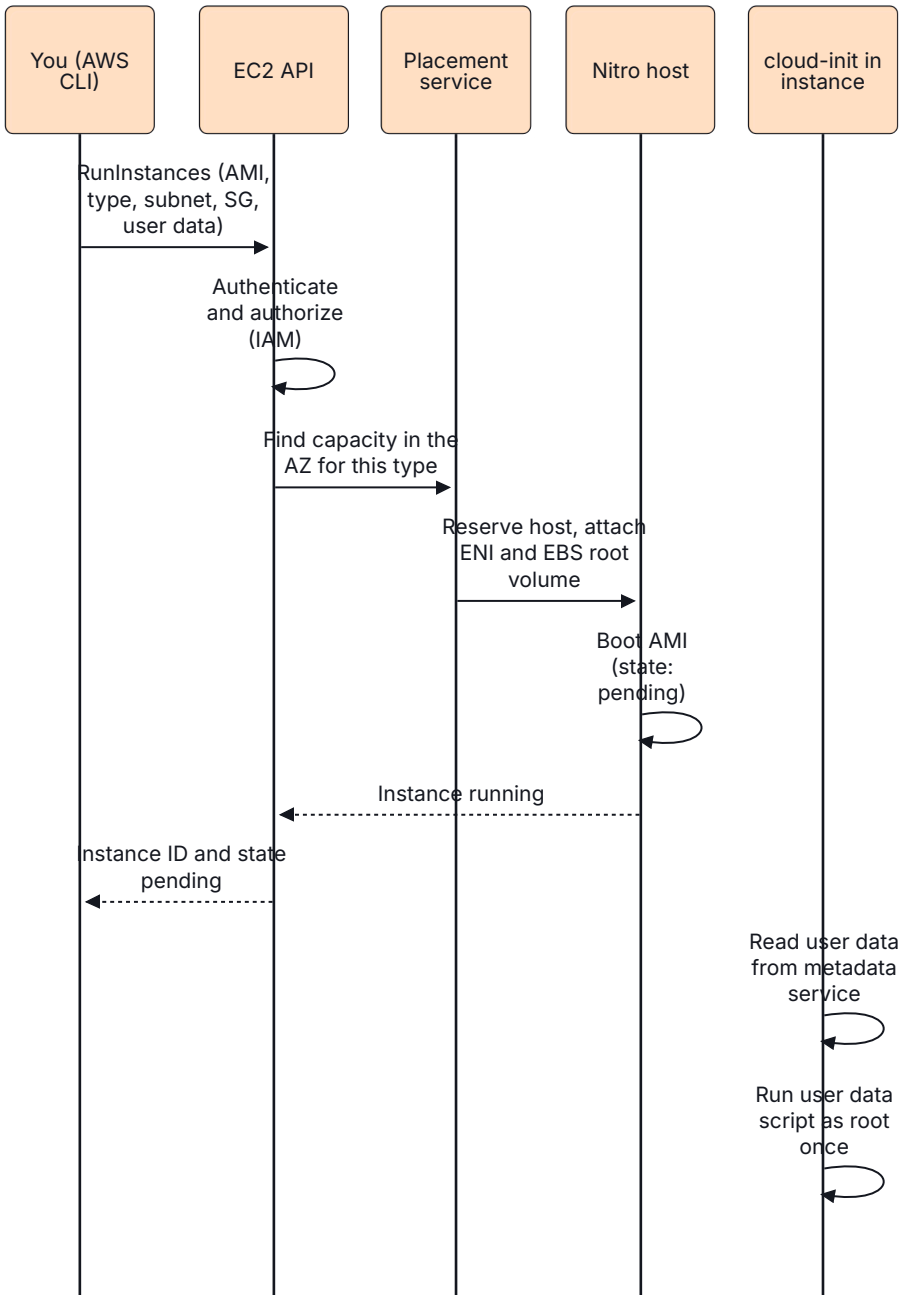


Figure 6.1-1

Instance lifecycle:

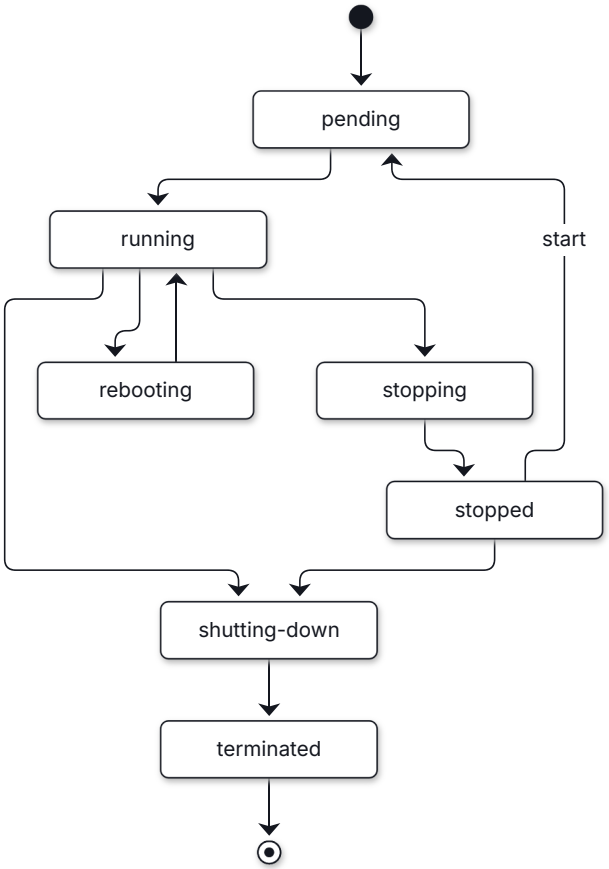


Figure 6.1-2

5 How it works

Instance type naming — `m7i.large` breaks down as family `m`, generation `7`, attributes `i`, size `large`:

Family	Optimized for	Typical use
`t` (t3, t4g)	Burstable CPU with credits; cheap baseline	dev boxes, small web apps, bastions
`m`	General purpose, balanced CPU/RAM	app servers such as `shop-api`
`c`	Compute optimized	CI runners, encoding, high-QPS proxies
`r`, `x`	Memory optimized	caches, in-memory databases, JVM-heavy services

Family	Optimized for	Typical use
`i`, `d`	Local NVMe storage	databases needing very high local IOPS
`g`, `p`	GPUs / accelerators	ML training and inference

Attribute letters: `a` = AMD, `i` = Intel, `g` = AWS Graviton (Arm), `d` = local NVMe disk, `n` = enhanced networking. Graviton (`t4g`, `m7g`) is usually cheaper per unit of work but needs an `arm64` AMI and arm-compatible builds.

Key concepts

Concept	What to remember
AMI	Region-specific. Backed by EBS snapshots. Public (Amazon Linux 2023, Ubuntu), marketplace, or your own baked images. Look up the latest Amazon Linux via the public SSM parameter `/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64`.
Key pair	AWS stores the *public* key and injects it for the default user (`ec2-user` on Amazon Linux, `ubuntu` on Ubuntu). You keep the private `.pem`; AWS cannot recover it. Prefer `ed25519`. See SSH keys §1.12.
User data	A script (max 16 KB) that **cloud-init** runs as root **once, at first boot**. Output goes to `/var/log/cloud-init-output.log`. Not a secret store: anyone who can read instance attributes or metadata can read it.
Instance metadata (IMDS)	HTTP endpoint at `169.254.169.254` exposing instance ID, AZ, and IAM role credentials. **IMDSv2** requires a session token obtained with a `PUT` request, which blocks most SSRF-based theft. New Amazon Linux 2023 images default to IMDSv2 only; set `HttpTokens=required` explicitly everywhere.
Root volume	EBS, usually `gp3`. `DeleteOnTermination` defaults to true for the root volume and false for extra volumes.
Public IPv4	Auto-assigned public IPs change on stop/start. Since February 2024 AWS charges for every public IPv4 address (about $0.005 per hour each, in use or not). An **Elastic IP** is a fixed address you own until released.
Placement	An instance lives in one subnet, therefore one AZ. Spread across AZs for availability.

Pricing models (introduction)

Model	Discount vs On-Demand	Commitment	Use for
On-Demand	none; billed per second (60-second minimum for Linux)	none	spiky, short, unknown workloads, learning
Savings Plans / Reserved Instances	roughly up to 70% depending on term and type	1 or 3 years of steady spend	steady baseline such as the always-on web tier

Model	Discount vs On-Demand	Commitment	Use for
Spot	often up to ~90%	none, but AWS can reclaim with a 2-minute warning	stateless, fault-tolerant work: CI, batch, extra web capacity
Dedicated Hosts / Capacity Reservations	varies	varies	licensing or guaranteed capacity

The discounts are indicative: check the pricing pages for exact numbers. A stopped instance costs **no instance charge**, but its EBS volumes and Elastic IPs keep billing. Deep dives live in Level 20 (Reserved Instances and Savings Plans §20.2, Spot §20.3).

6 Step-by-step guide

Prerequisites: an AWS account, **AWS CLI v2** configured (`aws sts get-caller-identity` works; prefer IAM Identity Center via `aws configure sso`, or a non-root IAM user — never root access keys), `ssh`, `curl`. Region `us-east-1`.

Cost: a `t3.micro` (or whichever type your account marks free-tier eligible) costs roughly a cent per hour. Free-tier rules changed in 2025: newer accounts get a credit-based free plan, older accounts keep the 12-month 750-hour allowance — check **Billing > Free Tier** in the console for what applies to you. The public IPv4 address is billed separately. **Do the cleanup in step 9.** LocalStack can mock these API calls (`aws --endpoint-url http://localhost:4566 ec2 ...`), but it does not give you a real booting OS to SSH into, so it is useful for scripting practice only.

1. **Set a billing safety net first** (mandatory for this level). Create a monthly cost budget that emails you at 80%:

```bash
export AWS_DEFAULT_REGION=us-east-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
cat > budget.json <<'EOF'
{"BudgetName":"roadmap-monthly-10-usd","BudgetLimit":
  {"Amount":"10","Unit":"USD"},"TimeUnit":"MONTHLY","BudgetType":"COST"}
EOF
cat > notifications.json <<'EOF'
[{"Notification":
  {"NotificationType":"ACTUAL","ComparisonOperator":"GREATER_THAN","Threshold":80,"Thr
  esholdType":"PERCENTAGE"},
  "Subscribers":[{"SubscriptionType":"EMAIL","Address":"you@example.com"}]}]
EOF
aws budgets create-budget --account-id "$ACCOUNT_ID" --budget file://budget.json --
  notifications-with-subscribers file://notifications.json
```

No output means success. Budget alerts lag by hours, so they are a safety net, not a substitute for teardown.

2. **Pick an AMI and an instance type.**

```bash
AMI_ID=$(aws ssm get-parameter \
  --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query Parameter.Value --output text)
echo "$AMI_ID"                      # ami-0123456789abcdef0 (differs per region and
  over time)
aws ec2 describe-instance-types --filters Name=free-tier-eligible,Values=true \
  --query 'InstanceTypes[].InstanceType' --output text   # for example: t3.micro
  t3.small ...
```

3. **Create a key pair** and lock down the file:

```bash
aws ec2 create-key-pair --key-name shop-api-lab --key-type ed25519 \
  --query KeyMaterial --output text > shop-api-lab.pem
chmod 400 shop-api-lab.pem
```

4. **Create a security group that only admits your own IP** (in the default VPC):

```bash
VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true --query
  'Vpcs[0].VpcId' --output text)
SG_ID=$(aws ec2 create-security-group --group-name shop-web-sg \
  --description "shop-api lab web" --vpc-id "$VPC_ID" --query GroupId --output text)
MY_IP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" --protocol tcp --port 22
  --cidr "$MY_IP/32"
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" --protocol tcp --port 80
  --cidr "$MY_IP/32"
```

5. **Write user data** that installs Nginx and prints which instance served the page (using

IMDSv2):

```bash
cat > userdata.sh <<'EOF'
#!/bin/bash
set -euxo pipefail
dnf install -y nginx
TOKEN=$(curl -s -X PUT http://169.254.169.254/latest/api/token -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/placement/availability-zone)
echo "<h1>shop-api web</h1><p>served by $IID in $AZ</p>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
EOF
```

6. **Launch the instance**, enforcing IMDSv2 and tagging it so you can find it later:

```bash
INSTANCE_ID=$(aws ec2 run-instances --image-id "$AMI_ID" --instance-type t3.micro \
  --key-name shop-api-lab --security-group-ids "$SG_ID" --user-data file://userdata.sh \
  --metadata-options HttpTokens=required,HttpEndpoint=enabled,HttpPutResponseHopLimit=1 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=shop-web-1},{Key=project,Value=devops-roadmap}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-running --instance-ids "$INSTANCE_ID"
PUBLIC_IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
echo "$INSTANCE_ID $PUBLIC_IP"
```

The default subnet assigns a public IP automatically. `wait instance-running` returns when the state is `running`; user data may need another minute.

7. **Connect and verify boot-time configuration.**

```bash
ssh -i shop-api-lab.pem ec2-user@"$PUBLIC_IP" 'cloud-init status --wait; tail -3 /var/log/cloud-init-output.log'
curl -s "http://$PUBLIC_IP/"      # <h1>shop-api web</h1><p>served by i-0abc... in us-east-1x</p>
```

Expect `status: done`. If the page is missing, read `/var/log/cloud-init-output.log` on the instance.

8. **See IMDSv2 enforcement in action** (run on the instance):

```bash
ssh -i shop-api-lab.pem ec2-user@"$PUBLIC_IP" '
  echo -n "no token → HTTP "; curl -s -o /dev/null -w "%{http_code}\n"
  http://169.254.169.254/latest/meta-data/
  TOKEN=$(curl -s -X PUT http://169.254.169.254/latest/api/token -H "X-aws-ec2-
  metadata-token-ttl-seconds: 60")
  echo -n "with token → "; curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
  http://169.254.169.254/latest/meta-data/instance-type; echo'
```

   Expected: `no token → HTTP 401` and `with token → t3.micro`. Optionally stop and start the instance (`aws ec2 stop-instances`, `start-instances`) and notice the public IP changes.

9. **Clean up (do not skip).**

```bash
aws ec2 terminate-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-terminated --instance-ids "$INSTANCE_ID"
aws ec2 delete-security-group --group-id "$SG_ID"
aws ec2 delete-key-pair --key-name shop-api-lab && rm -f shop-api-lab.pem
aws ec2 describe-instances --filters Name=tag:project,Values=devops-roadmap \
  --query 'Reservations[].Instances[].[InstanceId,State.Name]' --output text   # only
  "terminated" rows remain
```

   Keep the budget from step 1; it is free.

7 Real-world scenario

Illustrative. The `shop-api` team ran a single `m5.large` hand-configured over SSH. One Friday AWS emailed that the underlying host was scheduled for retirement. The engineer stopped and started the instance (which moves it to new hardware), but the site went down for ten minutes: the public IP changed, DNS still pointed at the old one, and a manually installed cron job had lived on an instance-store disk that was wiped. **Investigation** showed no AMI, no user data, no Elastic IP and no documentation. **Fix:** the team baked an AMI with the app, moved config into user data and a launch template, put an Elastic Load Balancer in front and let an Auto Scaling group (next topic) replace instances. **Lesson:** instances will be replaced; anything that is not reproducible from an image plus boot config is a liability.

8 Common mistakes and best practices

Don't

- Open port 22 to `0.0.0.0/0`, or share one key pair across the whole team.
- Leave IMDSv1 enabled, or put secrets in user data.
- Keep irreplaceable data on the instance-store (ephemeral) disks or only on the root volume.
- Forget stopped instances: their EBS volumes, snapshots and Elastic IPs still bill.
- Treat `t` instances as steady-state servers without watching CPU credit balance.

Do

- Launch from templates and AMIs you can rebuild; tag everything (`Name`, `project`, `owner`, `env`).
- Require IMDSv2 (`HttpTokens=required`), and prefer SSM Session Manager or EC2 Instance Connect over long-lived SSH keys in production.
- Use Graviton where your stack supports it, and use gp3 volumes.
- Set a budget and a billing alarm in every account before launching anything.
- Right-size using CloudWatch data, then commit to Savings Plans for the stable baseline.

9 Tasks

Cost note: tasks T2-T5 use paid resources (pennies if you clean up). Finish each task with the cleanup commands.

- [ ] **T1 — Billing alarm and budget** *(≈20 min, beginner)*
  - **Goal:** make sure a mistake cannot silently cost you hundreds of dollars.
  - **Steps:**
    1. Create the budget from step 1 with your real email address.
    2. In the console open **Billing > Billing preferences** and enable "Receive CloudWatch billing alerts", then in region `us-east-1` create an alarm on the `AWS/Billing` metric `EstimatedCharges` (currency USD) with threshold 5 and an SNS email topic.
    3. Confirm both subscription emails.
  - **Done when:** `aws budgets describe-budgets --account-id "$ACCOUNT_ID"` lists your budget and the CloudWatch alarm shows state `OK` or `INSUFFICIENT_DATA` with an email action.
- [ ] **T2 — Launch and prove it is yours** *(≈30 min, beginner)*
  - **Goal:** run the guide end to end.
  - **Steps:** follow steps 2-8, then clean up with step 9.

- **Done when:** `curl http://<ip>/` returned a page naming your instance ID, and the final `describe-instances` shows only terminated instances.

- [ ] **T3 — Change the shape** *(≈30 min, intermediate)*
  - **Goal:** understand stop/modify/start and what changes.
  - **Steps:**
    1. Launch an instance as in the guide, note its public IP and instance type.
    2. Stop it, change the type with `aws ec2 modify-instance-attribute --instance-id "$INSTANCE_ID" --instance-type "{\"Value\":\"t3.small\"}"`, start it.
    3. Compare the public IP and the output of `curl` against the metadata `instance-type` path.
  - **Done when:** metadata reports the new type, the public IP differs, and you can explain why. Then terminate it.

- [ ] **T4 — Break it: lose SSH and recover** *(≈40 min, intermediate)*
  - **Goal:** diagnose connectivity problems systematically.
  - **Steps:**
    1. Revoke the SSH rule (`aws ec2 revoke-security-group-ingress ...`) and observe `ssh` hanging with no error.
    2. Diagnose in order: instance state, public IP, security group, then on-instance logs via `aws ec2 get-console-output --instance-id "$INSTANCE_ID" --latest --output text`.
    3. Restore the rule with your current IP.
  - **Done when:** SSH works again and you can state the difference between "connection timed out" (network/firewall) and "Permission denied (publickey)" (key/user).

- [ ] **T5 — Automate it** *(≈45 min, advanced)*
  - **Goal:** a single script that creates and destroys the whole lab.
  - **Steps:**
    1. Put steps 2-6 into `up.sh` and step 9 into `down.sh`, using tags to find resources.
    2. Make `down.sh` safe to run twice (ignore not-found errors).
    3. Run `up.sh`, verify, run `down.sh` twice.
  - **Done when:** a fresh run of `up.sh` yields a working page and `down.sh` leaves nothing tagged `project=devops-roadmap` running.

- [ ] **T6 — Prove it: explain IMDSv2 and pricing** *(≈30 min, advanced)*
  - **Goal:** teach two concepts that matter in security reviews and cost reviews.

- **Steps:**
  1. Without the guide, demonstrate the 401-versus-token behaviour on a fresh instance.
  2. Write five sentences: why the PUT-token flow blunts SSRF, and which pricing model you would pick for the `shop-api` web tier, a nightly batch job and a CI runner fleet.
- **Done when:** a teammate agrees your explanation is correct, and your demo ran from memory.

10 Self-check questions

1. What are the three things you must choose to launch an instance, and what extra network object is mandatory?
2. When does user data run, and how do you debug it?
3. Why does the public IP of an instance change, and how do you avoid it?
4. What does IMDSv2 add, and why does it matter for credential theft?
5. An instance is stopped. What do you still pay for?
6. When would you choose Spot over Savings Plans?

ANSWERS

1. AMI, instance type and subnet (placement). A security group is attached too (the default one if you pick none).
2. As root by cloud-init at first boot only (unless configured otherwise). Read `/var/log/cloud-init-output.log` and `cloud-init status`.
3. Auto-assigned public IPs are released on stop and reassigned on start. Use an Elastic IP, or better a load balancer with DNS in front.
4. A session token obtained with a PUT request is required for every metadata read. Typical SSRF bugs only allow GETs and cannot add the header, so credentials are harder to steal.
5. EBS volumes, snapshots, Elastic IPs and any other attached resources — not the instance compute.
6. For interruptible, stateless, flexible workloads (CI, batch, queue workers); Savings Plans suit the steady always-on baseline.

11 Further reading

- *Amazon EC2 User Guide for Linux Instances* (AWS documentation) — instance types, AMIs, IMDS, user data.

- *AWS CLI v2 Command Reference*: `aws ec2 run-instances`.
- *AWS Well-Architected Framework* — Cost Optimization and Security pillars.
- *Amazon EC2 pricing* page — On-Demand, Spot, Savings Plans.
- *Docker/LocalStack documentation* — to script EC2 API calls without spending money.

LAB NOTES What I ran, what I saw, what it means

6.2 Auto Scaling

Topic 2 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

An **Auto Scaling group (ASG)** is a managed fleet of identical EC2 instances. You tell it a **minimum**, a **desired** and a **maximum** size, which subnets (and therefore **AZs, Availability Zones**) to use, and a **launch template** that describes what an instance looks like (AMI, instance type, security groups, user data, IAM instance profile). The ASG then does three jobs continuously: it **launches** instances until the fleet matches the desired capacity, it **replaces** instances that fail health checks, and it **scales** the desired capacity up or down according to policies you define (for example "keep average CPU at 50%").

An ASG is the bridge between "I have one server" and "my servers are disposable". It is free to use — you pay only for the instances (and related EBS, load balancer and data transfer) it creates.

2 Why it matters in DevOps

- **Availability:** a crashed or retired instance is replaced automatically, in another AZ if needed, without a human.
- **Cost:** you run 3 instances at 4 a.m. and 12 at the evening peak instead of 12 all day.
- **Deployments:** new application version means a new launch-template version and an *instance refresh* — the basis of rolling and blue/green deployments (Level 5 rolling deployments §5.10).
- **Capacity events:** a marketing email or a retry storm doubles traffic in minutes; people cannot launch servers that fast.

3 Real-world system example

Netflix built much of its early AWS deployment tooling around ASGs and has publicly described two ideas that shaped the industry: deployments as "red/black" pushes (Spinnaker creates a *new* ASG with the new AMI, shifts traffic, then disables the old ASG, which remains

available for instant rollback), and **Scryer**, an internal predictive autoscaling system described on the Netflix Tech Blog that forecast demand to pre-provision capacity because reactive scaling alone was too slow for their daily traffic curve. AWS later added its own *predictive scaling* policy type based on the same idea. For most teams, the pattern is simpler: one ASG per service, spread over three AZs, behind an Application Load Balancer, scaled on request count or CPU.

4 Diagram

Reactive scale-out with a target-tracking policy:

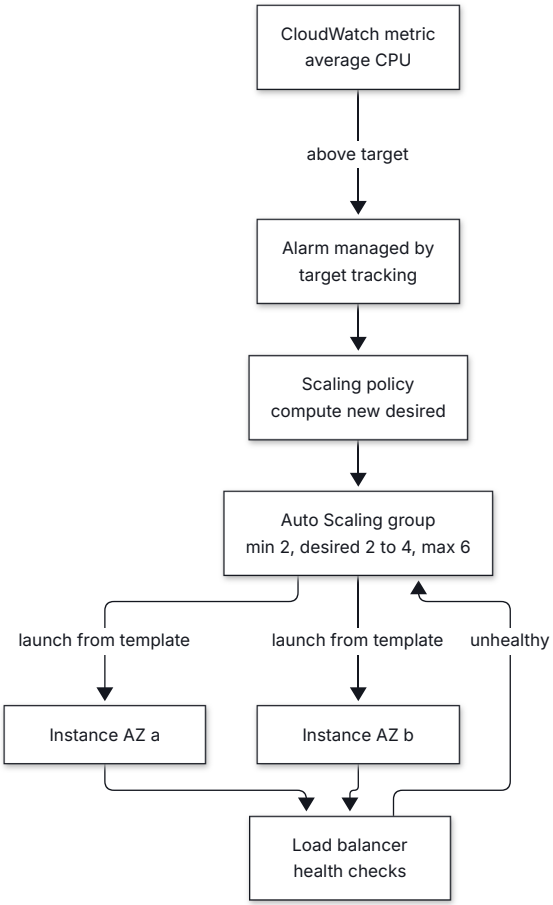


Figure 6.2-1

Lifecycle hooks pause an instance so you can run custom steps (warm a cache, drain connections, ship logs):

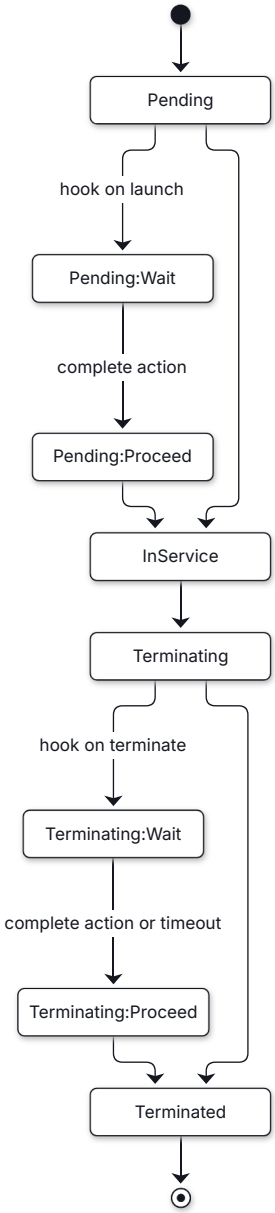


Figure 6.2-2

5 How it works

Launch template. A versioned definition of an instance (`aws ec2 create-launch-template`). New versions are immutable; the ASG points at a specific version, `$Latest` or `$Default`. Use launch templates, not the older *launch configurations*: AWS no longer lets new accounts create launch configurations and new EC2 features are only added to templates.

Capacity settings.

Setting	Meaning
Minimum	never go below (your floor for availability, usually one instance per AZ at least)
Desired	what the ASG tries to run right now; scaling policies change this
Maximum	hard ceiling; protects your wallet from runaway scale-out

Health checks. The default check is **EC2** (instance status is `running` and passes status checks). If the ASG is attached to a load balancer target group, also enable **ELB** health checks, otherwise an instance whose app returned 500s forever stays "healthy" to the ASG. The **health check grace period** (default 300 seconds) stops the ASG from killing instances that are still booting.

Scaling policies.

Policy	How it works	When to use
Target tracking	You pick a metric and target (e.g. `ASGAverageCPUUtilization` 50, or `ALBRequestCountPerTarget`); AWS creates and manages the alarms	default choice
Step scaling	Alarm thresholds map to bigger or smaller adjustments	when you need different reactions at different severities
Simple scaling	One adjustment per alarm, then a cooldown	legacy; prefer the others
Scheduled	Change min/desired/max at set times	known peaks (9 a.m. weekday ramp)
Predictive	ML forecast of load from history (needs ~24 h of data)	cyclical traffic; can run in forecast-only mode first

Instance warmup tells the policy how long a new instance needs before its metrics count, which prevents over-scaling. **Scale-in** removes instances according to the *termination policy* (by default it balances AZs, then prefers older launch templates/oldest instances); *scale-in protection* shields specific instances.

Instance refresh replaces instances to roll out a new template version: `MinHealthyPercentage` controls how many must stay in service, `InstanceWarmup` how long to wait before continuing.

Lifecycle hooks add a `Pending:Wait` or `Terminating:Wait` pause (default timeout 1 hour, max 48 hours of total wait) until your code calls `complete-lifecycle-action` with `CONTINUE` or `ABANDON`, or the timeout elapses and the hook's default result applies. Notifications go to EventBridge, SNS or SQS.

Other features worth knowing: mixed instances policy with On-Demand base and Spot top-up (see Spot Instances §20.3), warm pools (pre-initialized stopped instances for faster scale-out), and AZ rebalancing (the ASG launches in the under-filled AZ first, then terminates the surplus).

6 Step-by-step guide

Prerequisites: AWS CLI v2 configured, the default VPC in `us-east-1`, a budget set (L6.1 step 1).

Cost: the ASG itself is free; 2-4 `t3.micro` instances run for about an hour cost cents (check free-tier eligibility on the Billing console). Detailed (1-minute) monitoring costs extra, so this lab uses the default 5-minute metrics and scaling is slower than in production. **Clean up in step 8.** LocalStack's community edition can create launch templates and ASGs as API objects, but it does not run real scaling; use it only to practise the CLI syntax.

1. **Collect subnets, AMI and a security group.**

```bash
export AWS_DEFAULT_REGION=us-east-1
SUBNETS=$(aws ec2 describe-subnets --filters Name=default-for-az,Values=true \
  --query 'Subnets[0:2].SubnetId' --output text | tr '\t' ',')
echo "$SUBNETS"          # subnet-0aaa...,subnet-0bbb...
AMI_ID=$(aws ssm get-parameter --name /aws/service/ami-amazon-linux-latest/al2023-ami-
  kernel-default-x86_64 \
  --query Parameter.Value --output text)
VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true --query
  'Vpcs[0].VpcId' --output text)
SG_ID=$(aws ec2 create-security-group --group-name shop-asg-sg --description "asg lab"
  \
  --vpc-id "$VPC_ID" --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" --protocol tcp --port 80
  --cidr "$(curl -s https://checkip.amazonaws.com)/32"
```

2. **Create the launch template** (the user data from L6.1, base64-encoded, plus IMDSv2 required):

```bash
cat > userdata.sh <<'EOF'
#!/bin/bash
dnf install -y nginx stress-ng
TOKEN=$(curl -s -X PUT http://169.254.169.254/latest/api/token -H "X-aws-ec2-metadata-
  token-ttl-seconds: 300")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
  http://169.254.169.254/latest/meta-data/instance-id)
echo "<h1>shop-api web v1</h1><p>$IID</p>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
EOF
cat > lt-data.json <<EOF
{"ImageId":"$AMI_ID","InstanceType":"t3.micro","SecurityGroupIds":["$SG_ID"],
 "UserData":"$(base64 -w0 userdata.sh)",
 "MetadataOptions":{"HttpTokens":"required","HttpEndpoint":"enabled"},
 "TagSpecifications":[{"ResourceType":"instance","Tags":
  [{"Key":"project","Value":"devops-roadmap"}]}]}
EOF
aws ec2 create-launch-template --launch-template-name shop-api-lt --version-
  description v1 \
  --launch-template-data file://lt-data.json
```

Expect JSON with `"LatestVersionNumber": 1`. (`stress-ng` is in the Amazon Linux repositories; if `dnf` cannot find it, step 5 uses a shell loop instead.)

3. **Create the ASG across two AZs**, with a short grace period:

```bash
aws autoscaling create-auto-scaling-group --auto-scaling-group-name shop-api-asg \
  --launch-template 'LaunchTemplateName=shop-api-lt,Version=$Latest' \
  --min-size 2 --max-size 4 --desired-capacity 2 \
  --vpc-zone-identifier "$SUBNETS" \
  --health-check-type EC2 --health-check-grace-period 120
sleep 60
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names shop-api-asg \
  --query 'AutoScalingGroups[0].Instances[].
  [InstanceId,AvailabilityZone,LifecycleState,HealthStatus]' --output table
```

Expect two instances in different AZs, `Pending` then `InService`.

4. **Self-healing:** terminate one instance and watch the replacement.

```bash
VICTIM=$(aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names shop-
  api-asg \
  --query 'AutoScalingGroups[0].Instances[0].InstanceId' --output text)
aws ec2 terminate-instances --instance-ids "$VICTIM"
sleep 90
aws autoscaling describe-scaling-activities --auto-scaling-group-name shop-api-asg \
  --query 'Activities[0:3].[StatusCode,Description]' --output text
```

You should see a `Launching a new EC2 instance` activity. The ASG noticed the instance was no longer running and restored desired capacity.

5. **Add a target-tracking policy** and generate load.

```bash
cat > tt.json <<'EOF'
{"PredefinedMetricSpecification":
  {"PredefinedMetricType":"ASGAverageCPUUtilization"},"TargetValue":50.0}
EOF
aws autoscaling put-scaling-policy --auto-scaling-group-name shop-api-asg \
  --policy-name cpu50 --policy-type TargetTrackingScaling --target-tracking-
  configuration file://tt.json
```

To burn CPU without SSH, create a second template version whose user data also runs `stress-ng --cpu 2 --timeout 1200` in the background, then refresh the fleet onto it:

```bash
sed -i 's#^systemctl enable --now nginx#systemctl enable --now nginx\nstress-ng --cpu
  2 --timeout 1200 \&#' userdata.sh
aws ec2 create-launch-template-version --launch-template-name shop-api-lt --source-
  version 1 \
  --version-description v2-load --launch-template-data "{\"UserData\":\"$(base64 -w0
  userdata.sh)\"}"
aws autoscaling start-instance-refresh --auto-scaling-group-name shop-api-asg \
  --preferences '{"MinHealthyPercentage":50,"InstanceWarmup":120}'
```

After ~10-15 minutes (5-minute metrics, three data points), `describe-scaling-activities` shows desired capacity rising toward 4 with a cause such as "target tracking … alarm … in ALARM". Check `aws autoscaling describe-instance-refreshes --auto-scaling-group-name shop-api-asg` to see the rolling replacement.

6. **Lifecycle hook on termination** (give yourself 2 minutes to drain):

```bash
aws autoscaling put-lifecycle-hook --auto-scaling-group-name shop-api-asg --lifecycle-
  hook-name drain \
  --lifecycle-transition autoscaling:EC2_INSTANCE_TERMINATING --heartbeat-timeout 120
  --default-result CONTINUE
aws autoscaling set-desired-capacity --auto-scaling-group-name shop-api-asg --desired-
  capacity 2
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names shop-api-asg \
  --query 'AutoScalingGroups[0].Instances[].[InstanceId,LifecycleState]' --output text
  # one instance shows Terminating:Wait
```

   Finish early with `aws autoscaling complete-lifecycle-action --auto-scaling-group-name shop-api-asg --lifecycle-hook-name drain --lifecycle-action-result CONTINUE --instance-id <i-...>`; otherwise the timeout applies after 120 seconds.

7. **Roll back** by pointing the ASG at version 1 and refreshing:

```bash
aws autoscaling update-auto-scaling-group --auto-scaling-group-name shop-api-asg \
  --launch-template 'LaunchTemplateName=shop-api-lt,Version=1'
aws autoscaling start-instance-refresh --auto-scaling-group-name shop-api-asg --
  preferences '{"MinHealthyPercentage":50}'
```

8. **Clean up.**

```bash
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name shop-api-asg --
  force-delete
sleep 120      # wait for instances to terminate
aws ec2 delete-launch-template --launch-template-name shop-api-lt
aws ec2 delete-security-group --group-id "$SG_ID"
aws ec2 describe-instances --filters Name=tag:project,Values=devops-roadmap
  Name=instance-state-name,Values=pending,running \
  --query 'Reservations[].Instances[].InstanceId' --output text    # expect empty
  output
```

   If the security group delete fails with a dependency error, instances are still terminating; wait and retry.

7 Real-world scenario

Illustrative. A `shop-api` launch-day email tripled traffic. CPU hit 90% on both instances, but the ASG did not react for nearly ten minutes: the policy used 5-minute metrics, new instances took four minutes to boot and install packages, and the health-check grace period (60 s) was shorter than boot time, so the ASG **killed instances that were still starting**, creating a flapping loop. **Investigation:** `describe-scaling-activities` showed launches immediately followed by terminations with cause "health check failed". **Fix:** raise the grace period above the real boot time, bake an AMI instead of installing packages at boot, enable 1-minute monitoring for the template, scale on `ALBRequestCountPerTarget`, and raise the minimum before planned campaigns with a scheduled action. **Lesson:** scaling speed = metric delay + decision time + boot time; you can only cheaply shrink the last term.

8 Common mistakes and best practices

Don't

- Run an ASG in one AZ or set min = max = 1 and call it "highly available".
- Use ELB-backed ASGs with only EC2 health checks, or a grace period shorter than boot time.
- Install packages and pull large artifacts at boot for production fleets.
- Scale on a metric that does not drop when load drops, or leave `max` unbounded.
- Store state (sessions, uploads) on instances that scale in.

Do

- Spread across at least two (preferably three) AZs and set min to at least the number of AZs for important services.
- Bake an AMI per release; use instance refresh with a health-based `MinHealthyPercentage`.
- Prefer target tracking; add scheduled actions for known peaks.
- Handle `Terminating:Wait` (or SIGTERM-style draining) so in-flight requests finish.
- Alarm on "ASG at max capacity" and on failed launches (capacity or quota limits).

9 Tasks

Cost note: instances run while you work; always finish with step 8's cleanup.

- [ ] **T1 — Build and heal** *(≈45 min, beginner)*
  - **Goal:** create an ASG and watch it self-heal.
  - **Steps:** follow guide steps 1-4.

- **Done when:** after you terminate an instance, `describe-scaling-activities` shows a launch and the group is back to 2 `InService` instances in two AZs.

☐ **T2 — Scale with load** *(≈60 min, intermediate)*

- **Goal:** observe target tracking add capacity, then remove it.
- **Steps:** add the `cpu50` policy, generate CPU load with the v2 template, then remove the load (new template version without `stress-ng`, instance refresh) and watch scale-in.
- **Done when:** the activity history shows at least one scale-out and one scale-in, each with the target-tracking cause text, and you can point to the CloudWatch graph that justified each.

☐ **T3 — Zero-touch release** *(≈45 min, intermediate)*

- **Goal:** deploy v2 of the page by instance refresh and roll back.
- **Steps:**
  1. Create template version 2 whose page says `v2`; update the ASG to it and start an instance refresh with `MinHealthyPercentage=50`.
  2. Poll `describe-instance-refreshes` until `Successful`.
  3. Roll back to version 1 as in step 7.
- **Done when:** the page on a surviving instance flips v1 -> v2 -> v1 and the refresh history shows both operations.

☐ **T4 — Break it: a bad template version** *(≈45 min, advanced)*

- **Goal:** see how ASG launch failures and failed refreshes look, and recover.
- **Steps:**
  1. Create template version 3 with a nonexistent `ImageId` (for example `ami-0000000000000000`) and point the ASG at it.
  2. Terminate an instance (or start an instance refresh) and read `describe-scaling-activities` for the `Failed` activity and its status message.
  3. Fix by pointing the ASG back at a good version and confirm desired capacity is restored.
- **Done when:** you can show the failed activity text, the recovery, and explain why the fleet kept serving traffic on the instances it already had.

☐ **T5 — Lifecycle hook that does real work** *(≈60 min, advanced)*

- **Goal:** automate the hook response.
- **Steps:** create an EventBridge rule on `EC2 Instance-terminate Lifecycle Action` that invokes a small Lambda (see L6.3) which writes the instance ID to CloudWatch Logs and calls `complete-lifecycle-action`.

- **Done when:** scale-in produces a log line from your function and the instance leaves `Terminating:Wait` in seconds, not after the timeout.

☐ **T6 — Prove it: design the scaling** *(≈30 min, advanced)*

- **Goal:** justify a design on paper.
- **Steps:** without the guide, write the ASG definition for `shop-api` (min/desired/max, AZs, health checks and grace period, policy and metric, warmup, hook) and explain what limits its reaction time.
- **Done when:** a reviewer cannot find an availability or cost hole in under five minutes.

10 Self-check questions

1. What are the three jobs an ASG performs?
2. Why enable ELB health checks, and what does the grace period protect against?
3. Target tracking versus step scaling: which would you start with and why?
4. What is the difference between a launch template and a launch configuration?
5. A scale-in terminates an instance mid-request. Name two mechanisms that reduce this.
6. How does an instance refresh differ from changing the template version alone?

ANSWERS

1. Launch to reach desired capacity, replace unhealthy instances, adjust desired capacity per policies/schedules.
2. EC2 checks only see the VM; ELB checks see whether the app answers. Grace period prevents terminating instances that are still booting.
3. Target tracking: you state the goal and AWS manages alarms and math; step scaling needs hand-tuned thresholds.
4. Templates are versioned, support newer EC2 features and Spot mixes; launch configurations are legacy, immutable and no longer available to new accounts.
5. Connection draining / deregistration delay on the load balancer, and a `Terminating:Wait` lifecycle hook (plus apps handling SIGTERM).
6. Changing the version only affects future launches; an instance refresh actively replaces existing instances in controlled batches.

11 Further reading

- *Amazon EC2 Auto Scaling User Guide* (AWS documentation) — scaling policies, lifecycle hooks, instance refresh, warm pools.

- *AWS Well-Architected Framework* — Reliability pillar.
- Netflix Tech Blog — posts on Scryer and on Spinnaker red/black deployments.
- *Site Reliability Engineering* (Google) — capacity and load chapters for the underlying ideas.
- See also High Availability and Scalability §0.8.

LEVEL 6: CLOUD (AWS)

6.3 Lambda

Topic 3 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

AWS Lambda runs a function of your code in response to an **event**, without you managing servers. You upload code (a zip or a container image) and choose a runtime such as Node.js 22 or Python 3.13. Something happens — an object lands in S3, an HTTP request hits a **function URL**, a message arrives in an SQS queue, a schedule fires — and Lambda starts an isolated **execution environment**, calls your **handler** function with the event, and bills you for the milliseconds and memory used. When there is no traffic, nothing runs and nothing is billed. When traffic spikes, Lambda runs many copies in parallel.

Lambda is "serverless" in the sense that patching, capacity, scaling and AZ placement are AWS's job. Your job is the code, its permissions (an **execution role** in IAM), its configuration (memory, timeout, environment variables) and what triggers it.

2 Why it matters in DevOps

- **Glue and automation:** react to infrastructure events (instance terminating, S3 upload, CloudWatch alarm, GuardDuty finding) with ten lines of code instead of a VM.
- **Cost:** spiky or rare workloads cost cents. Constant, high-volume workloads can cost more than containers, so you must be able to compare.
- **Operational model:** no SSH, no OS patches — but also no long-lived state, hard time limits and distributed debugging.
- **Failure modes are different:** cold starts, throttling from concurrency limits, retry storms and poison messages are the new 3 a.m. pages.

3 Real-world system example

Amazon Prime Video's Video Quality Analysis tool is a documented lesson in *both* directions. The team's 2023 engineering blog post described how their first design — distributed components orchestrated with AWS Step Functions and Lambda — hit scaling and cost lim-

its, and that moving the pipeline into a single process on containers cut infrastructure cost dramatically (they reported about 90%). It does not show that serverless is bad: it shows that per-request orchestration overhead matters at very high, steady volume, and that Lambda excels elsewhere.

The canonical successful pattern, used from AWS's own launch examples to many production systems, is event-driven glue: a photo uploaded to S3 triggers a function that creates thumbnails; a CloudWatch alarm triggers a function that posts to chat; an ASG lifecycle hook (L6.2) triggers a function that drains an instance.

4 Diagram

Where events come from, and how Lambda invokes your code:



Figure 6.3-1

Cold start versus warm start:

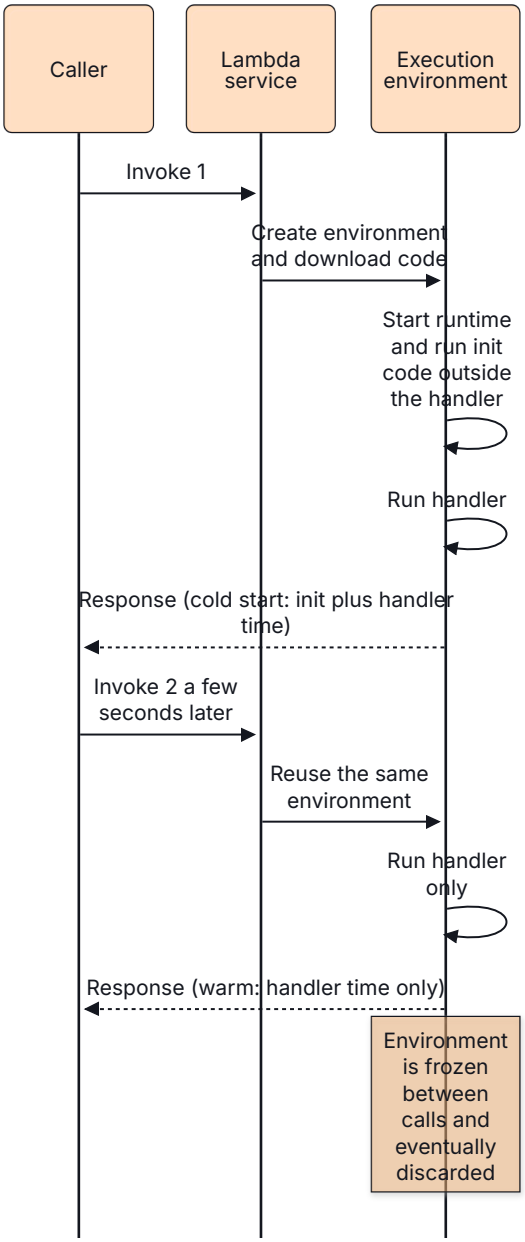


Figure 6.3-2

5 How it works

Invocation models

Model	Examples	Error behaviour
Synchronous	function URL, API Gateway, ALB, `aws lambda invoke`	error returned to the caller, caller retries
Asynchronous	S3, SNS, EventBridge	Lambda queues the event and retries failures twice by default; configure an on-failure destination or DLQ (dead-letter queue)
Event source mapping (poll)	SQS, Kinesis, DynamoDB Streams	Lambda polls and batches; failed batches are retried until success or expiry; use *partial batch responses* so one bad record does not block the rest

Cold starts. The first request to a new execution environment pays the **init** phase: download the package, start the runtime, run top-level code. Typical cold starts are tens to a few hundred milliseconds for Node.js and Python, longer for big packages, Java/.NET without optimization, or functions attached to a VPC with heavy init work. Reduce them with: small packages, lazy-loading heavy libraries, initializing SDK clients **outside** the handler (so warm calls reuse them), more memory (more CPU), arm64 (Graviton), **SnapStart** (supported on selected runtimes such as Java), and **provisioned concurrency** (keeps environments pre-initialized, billed continuously).

Limits to know (always check the current Lambda quotas page; they change):

Limit	Value to remember
Max duration	15 minutes per invocation
Memory	128 MB to 10,240 MB; CPU scales with memory
Synchronous payload	6 MB each for request and response
Package size	50 MB zipped (direct upload), 250 MB unzipped including layers; container images up to 10 GB
`/tmp`	512 MB by default, configurable up to 10,240 MB; not shared between environments
Concurrency	account-level regional quota (commonly 1,000 by default, may be lower on new accounts; raise via Service Quotas); **reserved concurrency** caps or guarantees a function's share

Pricing. You pay per request (about $0.20 per million) plus per GB-second of duration, with a monthly always-free tier (1 million requests and 400,000 GB-seconds). Recent pricing changes bill the init phase for more configurations, so check the pricing page. Related costs: CloudWatch Logs ingestion and storage, NAT Gateway if the function needs internet from a VPC (L6.5), provisioned concurrency.

Permissions have two sides: the **execution role** (what the function may call, e.g. `s3:GetObject`) and the **resource policy** (who may invoke it, e.g. `s3.amazonaws.com` for one bucket). See IAM §6.7.

When not to use Lambda

- Steady, high-throughput, latency-sensitive services where a few containers or instances are cheaper and more predictable.
- Jobs longer than 15 minutes, or needing large local state, GPUs or special kernel features (use ECS/Fargate, Batch or EC2).
- Long-lived connections (WebSockets held open by your code), or heavy connection pooling to a relational database without RDS Proxy.
- Workloads that cannot tolerate occasional cold-start latency and cannot justify provisioned concurrency.
- Complex orchestration expressed as a web of tiny functions that nobody can debug end to end.

6 Step-by-step guide

Prerequisites: AWS CLI v2 configured, `zip`, region `us-east-1`. **Cost:** effectively zero for this lab (well inside the always-free tier); CloudWatch Logs retain forever by default, so the cleanup deletes the log group. **LocalStack** supports Lambda well in its community edition (it needs access to the Docker socket): replace `aws` with `awslocal`, or add `--endpoint-url http://localhost:4566`, and use a dummy role ARN.

1. **Create an execution role** that can only write logs:

```bash
export AWS_DEFAULT_REGION=us-east-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
cat > trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
  {"Service":"lambda.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name shop-hello-role --assume-role-policy-document
  file://trust.json
aws iam attach-role-policy --role-name shop-hello-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
sleep 10          # IAM is eventually consistent
```

2. **Write the function** (Node.js 22, ES module). Note the client-style code outside the han-

dler runs once per environment:

```javascript
// index.mjs
const bootedAt = new Date().toISOString();   // init phase: runs once per execution
  environment
let calls = 0;

export const handler = async (event) => {
  calls += 1;
  const rec = event.Records?.[0];
  if (rec?.s3) {
    const key = decodeURIComponent(rec.s3.object.key.replace(/\+/g, " "));
    console.log(`new object s3://${rec.s3.bucket.name}/${key}
  size=${rec.s3.object.size}`);
    return { ok: true };
  }
  return { statusCode: 200, body: JSON.stringify({ message: `hello ${event.name ??
  "shop"}`, bootedAt, calls }) };
};
```

3. **Package and create the function** (arm64 is cheaper per GB-second and fine for plain JavaScript):

```bash
zip function.zip index.mjs
aws lambda create-function --function-name shop-hello --runtime nodejs22.x --
  architectures arm64 \
  --handler index.handler --zip-file fileb://function.zip \
  --role "arn:aws:iam::$ACCOUNT_ID:role/shop-hello-role" --memory-size 128 --timeout
  10
aws lambda wait function-active-v2 --function-name shop-hello
```

4. **Invoke it and read the REPORT line** (cold start first):

```bash
aws lambda invoke --function-name shop-hello --cli-binary-format raw-in-base64-out \
  --payload '{"name":"damilare"}' --log-type Tail --query LogResult --output text
  out.json | base64 -d
cat out.json
```

Expect a line similar to `REPORT RequestId: ... Duration: 3.1 ms Billed Duration: ... Memory Size: 128 MB Max Memory Used: 64 MB Init Duration: 110 ms`. `Init Duration` appears **only on cold starts**. Run the command again: no `Init Duration`, and `calls` in `out.json` is now 2. (CLI v2 needs `--cli-binary-format raw-in-base64-out` to accept raw JSON for `--payload`.)

5. **Force a new cold start** by changing configuration, then invoke again:

```bash
aws lambda update-function-configuration --function-name shop-hello --environment
  'Variables={STAGE=dev}'
aws lambda wait function-updated-v2 --function-name shop-hello
aws lambda invoke --function-name shop-hello --payload '{}' --cli-binary-format raw-
  in-base64-out --log-type Tail \
  --query LogResult --output text out.json | base64 -d | grep REPORT
```

`Init Duration` is back and `calls` reset to 1, proving the old environment was replaced.

6. **Add an event trigger: S3 upload invokes the function.**

```bash
B="shop-lambda-lab-$ACCOUNT_ID"
aws s3api create-bucket --bucket "$B"          # us-east-1 needs no LocationConstraint
FN_ARN=$(aws lambda get-function --function-name shop-hello --query
  Configuration.FunctionArn --output text)
aws lambda add-permission --function-name shop-hello --statement-id s3-invoke \
  --action lambda:InvokeFunction --principal s3.amazonaws.com \
  --source-arn "arn:aws:s3:::$B" --source-account "$ACCOUNT_ID"
cat > notif.json <<EOF
{"LambdaFunctionConfigurations":[{"LambdaFunctionArn":"$FN_ARN","Events":
  ["s3:ObjectCreated:*"],
  "Filter":{"Key":{"FilterRules":[{"Name":"prefix","Value":"uploads/"}]}}}]}
EOF
aws s3api put-bucket-notification-configuration --bucket "$B" --notification-
  configuration file://notif.json
echo "hello" > a.txt && aws s3 cp a.txt "s3://$B/uploads/a.txt"
sleep 5; aws logs tail /aws/lambda/shop-hello --since 5m
```

The log shows `new object s3://shop-lambda-lab-.../uploads/a.txt size=6`. Uploading outside `uploads/` triggers nothing. The resource policy (`add-permission`) is what lets S3 call your function; without it the notification call fails.

7. **Clean up.**

```bash
aws s3 rb "s3://$B" --force
aws lambda delete-function --function-name shop-hello
aws logs delete-log-group --log-group-name /aws/lambda/shop-hello
aws iam detach-role-policy --role-name shop-hello-role --policy-arn
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam delete-role --role-name shop-hello-role
rm -f function.zip out.json a.txt notif.json trust.json
```

7 Real-world scenario

Illustrative. A `shop-api` order-confirmation function consumed an SQS queue and wrote to PostgreSQL. After a marketing push the database hit `too many connections` and orders were delayed. **Investigation:** CloudWatch showed Lambda concurrency rising to several hundred, each environment opening its own database connection; a bug also made one malformed message fail every batch, so the queue kept redelivering it and starving valid orders. **Fix:** set **reserved concurrency** (e.g. 20) to cap parallel connections, introduce RDS Proxy for pooling, enable *partial batch responses* and a dead-letter queue with a `maxReceiveCount` so poison messages move aside, and alarm on queue age. **Lesson:** infinite scale on one side meets finite capacity on the other; always bound concurrency toward stateful backends.

8 Common mistakes and best practices

Don't

- Give a function `AdministratorAccess` or `Resource: "*"` "to get it working".
- Create SDK clients or load config inside the handler on every call.
- Put secrets in plaintext environment variables in source control; fetch from Secrets Manager or Parameter Store.
- Use recursive triggers (a function that writes to the bucket that triggers it) — a classic runaway bill.
- Assume exactly-once delivery: events can arrive twice or out of order.

Do

- Make handlers **idempotent** and keep them small and fast.
- Set a sensible timeout (not the 15-minute max) and memory (measure; more memory often lowers cost via shorter duration).
- Set log retention, structured JSON logs and alarms on `Errors`, `Throttles` and `Duration`.
- Use DLQs or failure destinations on async functions, and reserved concurrency to protect downstreams.
- Deploy with Infrastructure as Code and versions/aliases for safe rollouts.

9 Tasks

Cost note: all tasks stay within the free tier if you clean up; delete log groups at the end.

- ☐ **T1 — Hello, cold start** *(≈30 min, beginner)*
  - **Goal:** deploy a function and observe a cold start.

- **Steps:** do guide steps 1-5.
- **Done when:** you captured two REPORT lines, one with `Init Duration` and one without, and `out.json` showed `calls` incrementing.

☐ **T2 — Measure memory versus cost** *(≈40 min, intermediate)*

- **Goal:** learn that more memory can mean faster and not always costlier.
- **Steps:**
  1. Add a CPU-bound loop to the handler (e.g. hash a string 200,000 times).
  2. Invoke at 128, 512 and 1769 MB (`update-function-configuration --memory-size`), three times each.
  3. Compare `Duration` and compute duration times memory in GB-seconds.
- **Done when:** you have a table of memory versus billed duration versus GB-seconds and a one-line conclusion.

☐ **T3 — Queue-driven worker** *(≈60 min, intermediate)*

- **Goal:** wire an event source mapping with failure handling.
- **Steps:**
  1. Create an SQS queue and a dead-letter queue with `maxReceiveCount` 3; attach `AWSLambdaSQSQueueExecutionRole` to the role.
  2. `aws lambda create-event-source-mapping --function-name shop-hello --event-source-arn <queue-arn> --batch-size 5`.
  3. Send one good and one malformed message (make the function throw on `"poison"`).
- **Done when:** the good message is processed once and the poison message appears in the DLQ after three attempts.

☐ **T4 — Break it: throttling** *(≈30 min, advanced)*

- **Goal:** recognise throttle errors and recover.
- **Steps:**
  1. Run `aws lambda put-function-concurrency --function-name shop-hello --reserved-concurrent-executions 0`.
  2. Invoke the function and capture the `TooManyRequestsException`/throttle response.
  3. Remove the setting with `aws lambda delete-function-concurrency` and invoke again.
- **Done when:** you reproduced a throttle, found the `Throttles` metric in CloudWatch, and fixed it.

- [ ] **T5 — Automate deployment** *(≈45 min, advanced)*
  - **Goal:** make deploys one command with a rollback.
  - **Steps:** write a script that zips, runs `update-function-code --publish`, points an alias `live` at the new version, and can repoint it to the previous version.
  - **Done when:** `aws lambda get-alias --function-name shop-hello --name live` shows the version change and rolls back on request. (If you adopt Terraform later, see Level 7 §7.1.)

- [ ] **T6 — Prove it: Lambda or container?** *(≈30 min, advanced)*
  - **Goal:** make a defensible compute decision.
  - **Steps:** without notes, choose compute for three `shop-api` workloads (nightly report taking 40 minutes, image thumbnails on upload, the public REST API at steady 500 requests per second) and justify each with limits, cost and cold-start reasoning.
  - **Done when:** each choice cites at least one limit or cost factor and a colleague can follow the logic.

10 Self-check questions

1. What are the three invocation models and how do retries differ?
2. What is a cold start and name three ways to reduce it?
3. Why create SDK clients outside the handler?
4. What two policies are involved when S3 triggers a function?
5. Name four situations where Lambda is the wrong tool.
6. What does reserved concurrency do, and why is it a safety feature?

ANSWERS

1. Synchronous (caller handles errors), asynchronous (Lambda retries twice and can send failures to a destination/DLQ), and polling event source mappings (batches retried until success or expiry).
2. The init phase of a new environment (download, runtime start, init code). Smaller packages and lazy loading, more memory/arm64, SnapStart, provisioned concurrency.
3. Warm invocations reuse the initialized objects and connections, saving time on every call after the first.
4. The function's resource-based policy (lets S3 invoke it) and the execution role (what the code may do once running).
5. Jobs over 15 minutes, steady high-volume latency-sensitive APIs, unpooled relational DB access at scale, workloads needing GPUs/special kernels or big local state, over-fragmented orchestration.

6. It caps (and reserves) the number of concurrent executions of a function, protecting downstream systems and your bill from runaway scaling.

11 Further reading

- *AWS Lambda Developer Guide* (AWS documentation) — runtimes, concurrency, event source mappings, quotas.
- *AWS Lambda pricing* page — always-free tier, GB-second rates, provisioned concurrency.
- Amazon Prime Video technology blog, 2023: "Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%".
- *Serverless Applications Lens* of the AWS Well-Architected Framework.
- See also Infrastructure Automation §0.6.

6.4 VPC and Subnets

Topic 4 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

A **VPC (Virtual Private Cloud)** is your own logically isolated network inside AWS. You pick its private IP range (a **CIDR block**, such as `10.0.0.0/16`), carve it into **subnets**, and decide which subnets can talk to the internet and which cannot. Nothing enters or leaves a VPC unless you build a path for it. If you have read IPv4, IPv6, CIDR and Subnets §2.3, a VPC is the same idea with the router, switches and cabling replaced by an API.

A **subnet** is a slice of the VPC's address range that lives in **exactly one Availability Zone (AZ)**, one isolated data-centre group inside a region. This is the key fact for resilience: to survive the loss of an AZ you need subnets in at least two AZs and resources spread across them. A subnet is called **public** only because its route table has a route to an Internet Gateway (covered in Route Tables, IGW and NAT §6.5); the word is a convention about routing, not a property you set on the subnet itself.

2 Why it matters in DevOps

- **Security boundary:** databases and app servers belong in subnets with no route from the internet. The VPC layout is the first and cheapest layer of defence.
- **Irreversible choices:** the VPC's primary CIDR cannot be changed after creation, and two networks with overlapping ranges cannot be peered or joined through a Transit Gateway or VPN without painful NAT. A sloppy `/24` chosen on day one blocks the merger, the VPN to the office or the second region two years later.
- **Availability:** single-AZ designs fail when that AZ does. Multi-AZ is mostly a subnet-and-placement decision.
- **Capacity:** subnets that are too small run out of IP addresses during a scale-out, and Auto Scaling or Kubernetes pods then fail to launch. See Auto Scaling §6.2.
- **Infrastructure as code:** the network is the first thing Terraform creates and everything else depends on it (IaC §0.3).

3 Real-world system example

Amazon RDS, EKS and ElastiCache all consume subnets you provide. An RDS DB subnet group must span subnets in at least two AZs so that a Multi-AZ failover has somewhere to go. EKS places control-plane network interfaces in your subnets and, with the default AWS VPC CNI plugin, gives every Pod a real VPC IP address, which is why undersized subnets are a well-known cause of "Pods stuck in ContainerCreating" on EKS.

AWS's own well-architected guidance and its "VPC with public and private subnets" reference layouts use the pattern in this topic: load balancers in public subnets, application instances in private subnets, data stores in isolated private subnets, repeated across at least two (commonly three) AZs. The `shop-api` layout below follows it.

4 Diagram

The target layout for `shop-api` in `us-east-1` (one VPC, three AZs, three tiers):

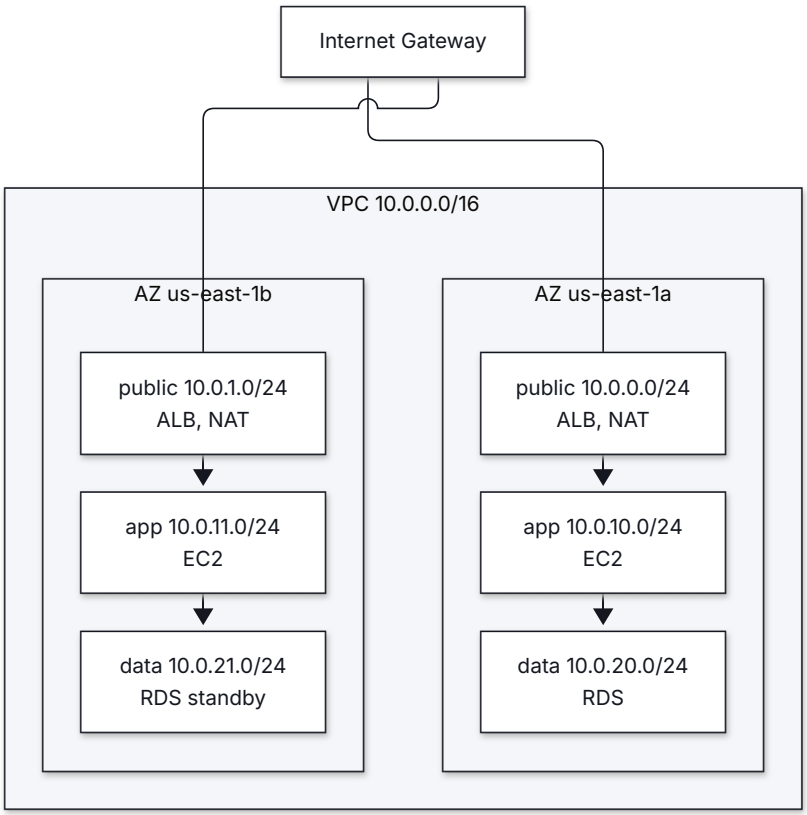


Figure 6.4-1

Carving a `/16` into planned blocks (the third octet encodes the tier, which makes route tables and logs readable):

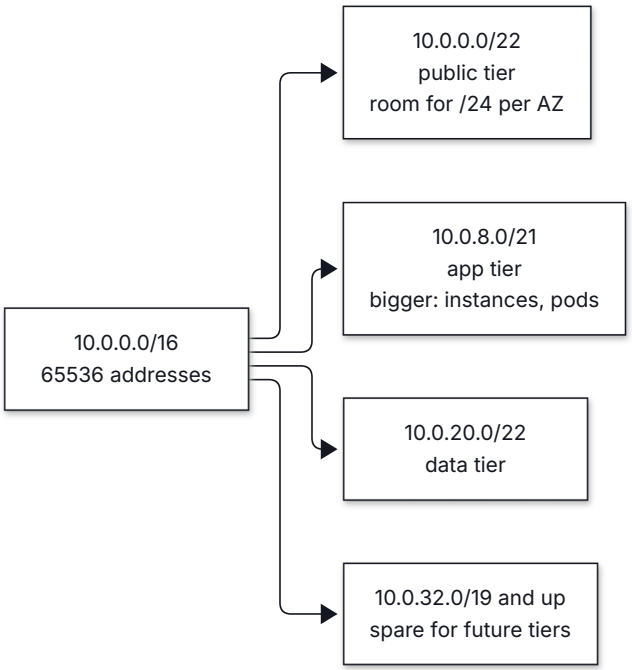


Figure 6.4-2

5 How it works

CIDR rules in AWS

Rule	Detail
VPC size	Between `/16` (65,536 addresses) and `/28` (16 addresses) for IPv4
Address space	Use RFC 1918 ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`
Reserved per subnet	AWS reserves **5 addresses** in every subnet: the network address, VPC router (`.1`), AWS DNS (`.2`), one reserved for future use (`.3`) and the broadcast address (last). A `/24` therefore gives 251 usable addresses, a `/28` only 11
Subnet scope	One subnet = one AZ. It cannot span AZs and cannot overlap another subnet in the VPC
Growing	You can add **secondary CIDR blocks** to a VPC later, but you cannot resize or remove the primary
IPv6	Optionally associate an AWS-provided `/56` block; subnets then take a `/64`. IPv6 addresses are globally unique, so there is no NAT; use an egress-only internet gateway for outbound-only

Rule	Detail
Default VPC	Every region has one (`172.31.0.0/16`, one `/20` public subnet per AZ). Fine for experiments; build your own VPC for anything real

Planning checklist

1. **Choose a range that will not collide.** Inventory your office, other VPCs, other regions, partner networks and any on-prem ranges. Avoid `10.0.0.0/16` if you know a VPN will land on it, and avoid `172.17.0.0/16` (the default Docker bridge). Teams with many accounts allocate non-overlapping blocks centrally (AWS offers **VPC IP Address Manager, IPAM**, for this).
2. **Size for the biggest consumer.** A `/16` VPC is the usual choice; running out of VPC space is far more painful than wasting private addresses. Kubernetes with the VPC CNI needs large app subnets (`/20` or `/19`).
3. **Use at least two AZs, three for production.** Three tolerates one AZ failing while still keeping quorum-based systems healthy.
4. **Separate tiers into separate subnets:** *public* (load balancers, NAT gateways, rarely a bastion), *private app* (EC2, containers) and *private data* (RDS, ElastiCache). Different subnets let you give each tier its own route table and Network ACL (Security Groups and NACLs §6.6).
5. **Think in AZ IDs, not names.** The name `us-east-1a` maps to a different physical AZ in different accounts. The AZ ID (`use1-az1`) is stable and matters when two accounts must share a physical AZ (for example, to avoid cross-AZ data charges).
6. **Leave gaps.** Do not allocate the whole VPC on day one; keep a spare block for a new tier, a new endpoint subnet or a Transit Gateway attachment subnet.

Multi-AZ design principles

- Every tier has one subnet per AZ, with the same size in each.
- Stateless tiers (ALB, app servers) run in all AZs. Stateful tiers use the service's own multi-AZ feature (RDS and Aurora §6.10).
- Cross-AZ traffic is billed (about $0.01 per GB in each direction at the time of writing), so chatty services should prefer same-AZ calls where the design allows.
- The ALB must be given subnets in at least two AZs (Application Load Balancer §6.14).

The VPC itself costs nothing. Subnets, route tables, security groups and Internet Gateways are free. What you *attach* costs money: NAT gateways, public IPv4 addresses, VPC endpoints (interface type), Transit Gateway attachments and the data you move.

6 Step-by-step guide

Prerequisites: AWS CLI v2 configured (`aws sts get-caller-identity` works), region `us-east-1`. Use a non-root identity. **Cost:** everything in this guide is free (a VPC and subnets have no hourly charge). **LocalStack:** the free edition emulates the VPC and subnet APIs, so you can rehearse the same commands offline: `docker run -d --rm -p 4566:4566 localstack/localstack`, then add `--endpoint-url http://localhost:4566` to every command (any dummy credentials work). LocalStack does not enforce real networking or the exact AZ ID mapping.

1. **Set variables and look at the AZs.**

```bash
export AWS_DEFAULT_REGION=us-east-1
aws ec2 describe-availability-zones \
  --query 'AvailabilityZones[].[ZoneName,ZoneId,State]' --output table
```

   You see rows such as `us-east-1a | use1-az1 | available`. The name-to-ID pairing differs per account; note yours.

2. **Verify your CIDR plan before creating anything.** Python's standard library does the arithmetic:

```bash
python3 - <<'EOF'
import ipaddress
vpc = ipaddress.ip_network("10.0.0.0/16")
plan = {"public-a":"10.0.0.0/24","public-b":"10.0.1.0/24",
        "app-a":"10.0.10.0/24","app-b":"10.0.11.0/24",
        "data-a":"10.0.20.0/24","data-b":"10.0.21.0/24"}
nets = {k: ipaddress.ip_network(v) for k, v in plan.items()}
for k, n in nets.items():
    assert n.subnet_of(vpc), k
    print(f"{k:9} {n}  usable={n.num_addresses-5}")
items = list(nets.values())
assert not any(a.overlaps(b) for i, a in enumerate(items) for b in items[i+1:])
print("no overlaps")
EOF
```

   Expected: six lines with `usable=251`, then `no overlaps`.

3. **Create the VPC and enable DNS hostnames.**

```bash
VPC_ID=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=shop-api-vpc}]' \
  --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --vpc-id "$VPC_ID" --enable-dns-hostnames
  '{"Value":true}'
echo "$VPC_ID"
```

   Expected: an ID like `vpc-0abc1234def567890`. DNS support is on by default; DNS hostnames let instances with public IPs get names, and are needed by several features such as interface VPC endpoints.

4. **Create the six subnets** (two AZs shown; copy the pattern for a third):

```bash
mk() { # name cidr az
  aws ec2 create-subnet --vpc-id "$VPC_ID" --cidr-block "$2" --availability-zone "$3"
  \
    --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=shop-api-$1}]" \
    --query Subnet.SubnetId --output text
}
PUB_A=$(mk public-a 10.0.0.0/24  us-east-1a)
PUB_B=$(mk public-b 10.0.1.0/24  us-east-1b)
APP_A=$(mk app-a    10.0.10.0/24 us-east-1a)
APP_B=$(mk app-b    10.0.11.0/24 us-east-1b)
DAT_A=$(mk data-a   10.0.20.0/24 us-east-1a)
DAT_B=$(mk data-b   10.0.21.0/24 us-east-1b)
```

5. **Inspect the result.**

```bash
aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC_ID" \
  --query 'Subnets[].[Tags[?Key==`Name`]|
  [0].Value,CidrBlock,AvailabilityZone,AvailableIpAddressCount]' \
  --output table
```

   Each `/24` shows `AvailableIpAddressCount` of `251`.

6. **Mark the public subnets to auto-assign public IPs.** This only matters once a route to an Internet Gateway exists (next topic), and it is best practice to leave it off for private

subnets:

```bash
aws ec2 modify-subnet-attribute --subnet-id "$PUB_A" --map-public-ip-on-launch
aws ec2 modify-subnet-attribute --subnet-id "$PUB_B" --map-public-ip-on-launch
```

Remember that every public IPv4 address now has an hourly charge (about $0.005 per hour), so avoid public IPs on anything that does not need one.

7. **Check which subnets are actually public.** Right now none are: the VPC has the implicit "main" route table with only the local route.

```bash
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=$VPC_ID" \
  --query 'RouteTables[].Routes[].[DestinationCidrBlock,GatewayId]' --output table
```

Expected: a single `10.0.0.0/16 | local` row. A route like `0.0.0.0/0 | igw-...` is what would make a subnet public.

8. **Clean up** (delete subnets first, then the VPC):

```bash
for s in "$PUB_A" "$PUB_B" "$APP_A" "$APP_B" "$DAT_A" "$DAT_B"; do
  aws ec2 delete-subnet --subnet-id "$s"
done
aws ec2 delete-vpc --vpc-id "$VPC_ID"
```

If `delete-vpc` fails with `DependencyViolation`, something is still attached (an endpoint, gateway or network interface). Later topics reuse this VPC; if you want to keep it for the next topic, skip this step. It is free either way.

7 Real-world scenario

Illustrative scenario, not a documented incident. The `shop-api` team launched with a single `/24` VPC (`10.0.0.0/24`) split into two `/25` subnets "because it was only a prototype". A year later, three problems arrive at once. Black Friday scale-out fails halfway: the Auto Scaling group reports `InsufficientFreeAddressesInSubnet` because the app subnet only has 123 usable addresses and a rolling deployment temporarily doubles the instance count. The company then acquires a startup whose network is `10.0.0.0/16`, and the VPN between the two cannot be built because the ranges overlap. And RDS Multi-AZ cannot be enabled cleanly because both subnets are in one AZ.

Fix: they create a new VPC with a planned `/16` range registered in a central IPAM pool, three AZs and three tiers, then migrate behind the load balancer using blue/green deployment §5.11, tearing the old VPC down afterwards. **Lesson:** CIDR and AZ decisions are the hardest to change; spend an hour on the plan now to avoid a quarter-long migration later.

8 Common mistakes and best practices

Don't

- Use the default VPC for production workloads (all subnets public, no planned ranges).
- Pick a tiny VPC or reuse `10.0.0.0/16` in every account without an allocation plan.
- Put everything in one AZ, or give each tier a different number of AZs.
- Assume "private subnet" means "secure": it only means no inbound path from the internet. Security groups and IAM still apply.
- Forget that subnets reserve 5 addresses, and that Kubernetes Pods, Lambda in a VPC and ENIs consume subnet addresses.

Do

- Plan CIDRs across every account and region first (a spreadsheet or AWS IPAM) and leave unallocated space.
- Create one subnet per tier per AZ with consistent sizes and a naming scheme (`shop-api-app-a`).
- Use three AZs for production and refer to AZ IDs when cross-account placement matters.
- Define the network in code (Terraform §7.1) so it can be reviewed and rebuilt.
- Tag every resource (`Name`, `Environment`, `Owner`) so unknown spend can be traced.

9 Tasks

- [ ] **T1 — Plan a CIDR layout on paper and verify it** *(≈30 min, beginner)*
  - **Goal:** produce a non-overlapping plan for three environments (dev, staging, prod) and three tiers across three AZs.
  - **Steps:**
    1. Give each environment its own `/16` (`10.10.0.0/16`, `10.20.0.0/16`, `10.30.0.0/16`).
    2. For prod, assign a `/24` per tier per AZ (9 subnets) plus a documented spare block.
    3. Encode the plan in the Python snippet from step 2 and run it.
  - **Done when:** the script prints every subnet with `usable=251` and reports `no overlaps`.
- [ ] **T2 — Build the VPC and subnets with the CLI** *(≈30 min, beginner, free)*

- **Goal:** create the layout from section 4 for real (or in LocalStack).
- **Steps:**
  1. Follow steps 1-5 of the guide with two AZs.
  2. Save the IDs in a file for the next topic: `echo "VPC_ID=$VPC_ID" > shop-vpc.env`.
  3. Run the `describe-subnets` query from step 5.
- **Done when:** the table lists six subnets across two AZs, each with `AvailableIpAddressCount` 251.

☐ **T3 — Break it on purpose** *(≈30 min, intermediate)*

- **Goal:** learn the error messages that appear in real pipelines.
- **Steps:**
  1. Try to create a subnet `10.0.5.0/24` and then another `10.0.5.0/25`, in the same VPC.
  2. Try a subnet `192.168.0.0/24` that is outside the VPC range.
  3. Try `aws ec2 create-vpc --cidr-block 10.0.0.0/8` and one with a `/29` block.
  4. Write down each error code and why AWS refused (overlap, out of range, invalid prefix size).
- **Done when:** you have three different error messages (similar to `InvalidSubnet.Conflict`, `InvalidSubnet.Range`, `InvalidVpc.Range`) and an explanation for each.

☐ **T4 — Run out of addresses** *(≈30 min, intermediate)*

- **Goal:** feel the effect of the 5 reserved addresses and of undersized subnets.
- **Steps:**
  1. Create a `/28` subnet and read its `AvailableIpAddressCount`.
  2. Compute the smallest subnet that would hold 50 instances plus 50 Pods' worth of IPs, with 30 percent growth headroom.
  3. Add a secondary CIDR to the VPC (`aws ec2 associate-vpc-cidr-block --vpc-id "$VPC_ID" --cidr-block 100.64.0.0/16`) and create a subnet in it.
- **Done when:** the `/28` reports `11` available addresses, you have a written sizing answer, and the secondary-CIDR subnet exists.

☐ **T5 — Automate it as a reusable script** *(≈45 min, advanced)*

- **Goal:** make network creation repeatable and idempotent-ish.
- **Steps:**
  1. Write `create-vpc.sh` that takes an environment name and a `/16`, creates the VPC and a subnet per tier per AZ from a loop, and prints a JSON summary.

2. Add a `destroy` mode that deletes subnets then the VPC.
3. Run create then destroy twice.

- **Done when:** the second create/destroy cycle works with no leftover resources (`aws ec2 describe-vpcs --filters Name=tag:Name,Values=shop-api-vpc` returns nothing).

☐ **T6 — Prove it: design review** *(≈30 min, advanced)*

- **Goal:** defend a VPC design without notes.
- **Steps:**
  1. On a blank page, draw a three-AZ, three-tier VPC for `shop-api` with CIDRs.
  2. Explain out loud: why `/16`, why three AZs, what the reserved five addresses are, why the primary CIDR is hard to change, how an overlap with the office network would be handled.
- **Done when:** a teammate (or you, recorded) can follow the reasoning and finds no gaps.

10 Self-check questions

1. Can a subnet span two AZs? What does that imply for high availability?
2. How many usable IPv4 addresses are in a `/24` subnet in AWS, and why not 254?
3. What makes a subnet "public"?
4. Why is choosing a VPC CIDR a one-way door, and what is the escape hatch?
5. Why should you plan with AZ IDs rather than AZ names in multi-account setups?

ANSWERS

1. No. A subnet lives in exactly one AZ, so high availability needs at least one subnet per AZ and resources spread across them.
2. 251: AWS reserves the network address, the VPC router, the DNS server, one future-use address and the broadcast address.
3. A route in its route table to an Internet Gateway for `0.0.0.0/0` (plus resources with public IPs). It is routing, not a checkbox.
4. The primary CIDR cannot be changed, and overlapping networks cannot be peered. The escape hatch is adding secondary CIDR blocks, or building a new VPC and migrating.
5. AZ names are shuffled per account; AZ IDs refer to the same physical zone everywhere.

11 Further reading

- AWS documentation: *Amazon VPC User Guide* (CIDR blocks, subnet sizing, IPv6 support).
- AWS documentation: *Amazon VPC IP Address Manager (IPAM)*.
- AWS whitepaper: *Building a Scalable and Secure Multi-VPC AWS Network Infrastructure*.
- *AWS Well-Architected Framework*, Reliability pillar (workload architecture across AZs).
- Level 2: IPv4, IPv6, CIDR and Subnets §2.3 for the underlying math.

LAB NOTES What I ran, what I saw, what it means

6.5 Route Tables, Internet Gateway and NAT Gateway

Topic 5 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

Subnets by themselves are islands. What connects them to each other, to the internet and to AWS services is the **route table**: a short list of rules of the form "traffic for *this destination range* goes to *this target*". Every subnet is associated with exactly one route table, and every packet leaving an instance is matched against it using **longest-prefix match** (the most specific rule wins), exactly as in Routing, NAT, Ports and Firewalls §2.4.

Three targets matter most. The **local** route (always present, cannot be deleted) lets all subnets in the VPC reach each other. An **Internet Gateway (IGW)** is the doorway between the VPC and the public internet. A **NAT Gateway** lets instances in *private* subnets start outbound connections to the internet (for package updates, third-party APIs) without being reachable from it. A fourth family, **VPC endpoints**, gives private subnets a direct path to AWS services such as S3 without touching the internet at all, which often saves real money.

2 Why it matters in DevOps

- **"It can't reach the internet" / "it is reachable and shouldn't be"** are the two most common AWS networking tickets, and both are route-table questions.
- **Cost:** a NAT Gateway is one of the most frequent surprise line items on an AWS bill (an hourly charge per gateway plus a per-GB processing charge on all traffic). Understanding the traffic paths is how you cut it.
- **Security:** the only thing that stops a database subnet from being internet-facing is the absence of a `0.0.0.0/0 → igw` route. Reviewing routes is a core security audit step.
- **Availability:** a NAT Gateway lives in one AZ. A design with one NAT for three AZs loses outbound access (and pays cross-AZ charges) when that AZ fails.
- **Infrastructure as code:** routes and associations are exactly the resources Terraform modules get wrong when a subnet silently falls back to the VPC's *main* route table (Providers and Resources §7.1).

3 Real-world system example

Amazon EKS and ECS on private subnets. The common production pattern for a container platform on AWS is worker nodes in private subnets, with the load balancer in public subnets. Those private nodes must pull images from Amazon ECR, send logs to CloudWatch and call AWS APIs. AWS documents two ways: route everything through a NAT Gateway, or use VPC endpoints (an S3 *gateway* endpoint, because ECR stores image layers in S3, plus *interface* endpoints for ECR, CloudWatch Logs and STS). Fully private clusters with no NAT at all are a documented, supported setup, and teams use it both to reduce the NAT bill and to ensure workloads cannot reach arbitrary internet destinations.

The same split applies to `shop-api` on EC2: Nginx behind the ALB in public subnets, the Node.js instances in private subnets that reach S3 through an endpoint and `registry.npmjs.org` through NAT during deploys.

4 Diagram

The path of an outbound request from a private instance, and the return path:

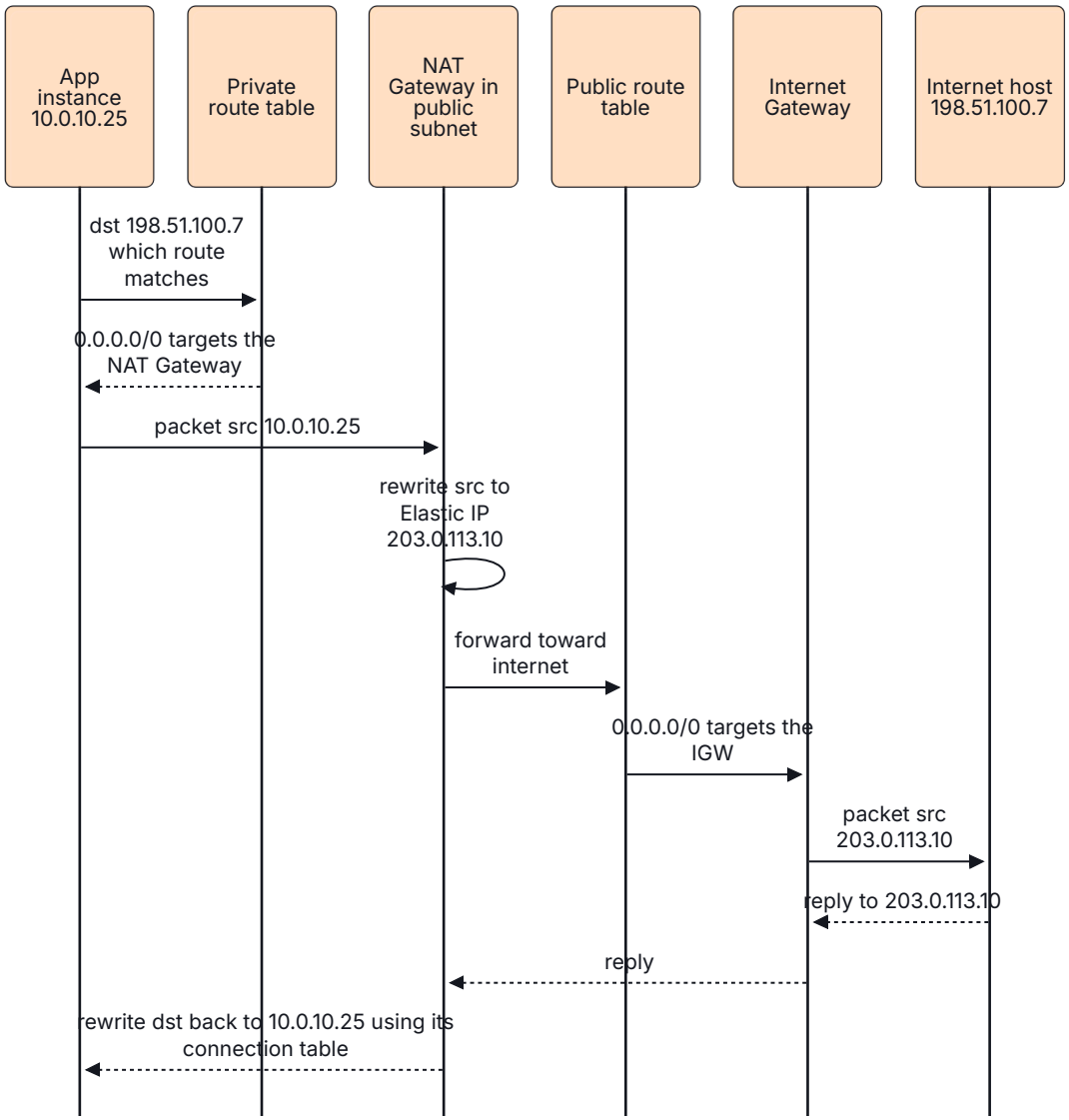


Figure 6.5-1

Three exits from a private subnet, and what each costs:

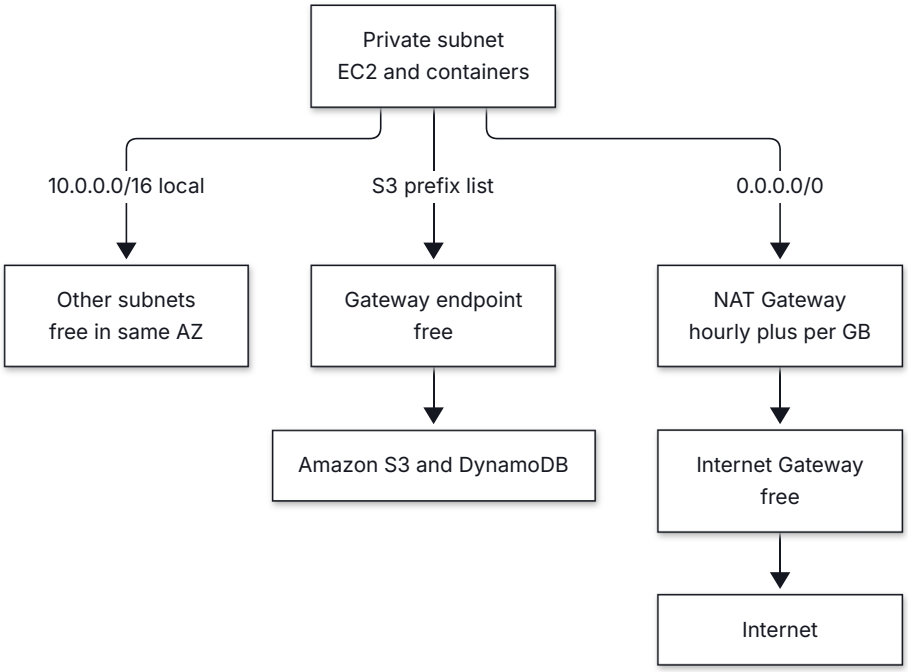


Figure 6.5-2

5 How it works

Route table anatomy (the public table of the `shop-api` VPC):

Destination	Target	Meaning
`10.0.0.0/16`	`local`	Anything inside the VPC stays inside; always present
`0.0.0.0/0`	`igw-0abc...`	Everything else goes to the Internet Gateway: this single line makes the subnet "public"

The private table has `local` plus either `0.0.0.0/0 → nat-0abc...` (outbound internet through NAT) or nothing (fully isolated).

Rules worth memorising:

- **Longest prefix wins.** `10.0.0.0/16 → local` beats `0.0.0.0/0 → nat` for VPC traffic. A more specific route to a peering connection or Transit Gateway beats the default route.

- **Main route table:** every VPC has one, and any subnet without an explicit association silently uses it. Keep the main table *private* (local only) and associate public subnets explicitly, so a forgotten subnet fails closed.
- **One table per subnet, but a table can serve many subnets.** Create one private table **per AZ** if each AZ has its own NAT Gateway.
- **Blackhole:** if a route's target is deleted (a NAT Gateway, a peering connection), the route shows state `blackhole` and traffic is dropped.

Internet Gateway. Horizontally scaled, redundant, no bandwidth limit to plan for, and **free**. It attaches to one VPC. For IPv4 it performs one-to-one NAT between an instance's private address and its public/Elastic IP. For an instance to reach the internet through the IGW **all four** must be true: the subnet route table has `0.0.0.0/0 → igw`, the instance has a public IPv4 (or IPv6) address, the security group allows the traffic, and the network ACL allows it (next topic §6.6). Missing any one is the classic "I added the route but it still doesn't work".

NAT Gateway (public). A managed NAT device you place in a **public subnet** and give an **Elastic IP**. Private subnets route `0.0.0.0/0` to it; it translates source addresses and tracks connections, so replies come back but no one on the internet can initiate a connection in. It is zonal, so use one per AZ in production.

Cost item	Approximate US East (N. Virginia) price at time of writing
NAT Gateway hour	about $0.045 per hour per gateway (roughly $33 per month if left running)
NAT data processing	about $0.045 per GB, **on top of** normal data transfer charges
Elastic IP / public IPv4	about $0.005 per hour

Three AZs with one NAT each is roughly $100 per month **before** any traffic. Always check the current pricing page. Cheaper options: share one NAT across AZs in non-production (accepting the AZ risk and cross-AZ charges); run a NAT instance (a small EC2 instance doing the translation, used by cost-sensitive teams, you patch and scale it); or avoid NAT for AWS traffic with endpoints. AWS has also introduced a *regional* NAT gateway mode that spans AZs; read the current VPC documentation before choosing, since the details have been changing.

IPv6: addresses are globally routable, so no NAT. For outbound-only IPv6 access use an **egress-only internet gateway**.

VPC endpoints keep traffic to AWS services on the AWS network:

Type	How it works	Cost	Services
Gateway endpoint	Adds a route (to an AWS-managed *prefix list*) in the route tables you choose	**Free**	S3 and DynamoDB only

Type	How it works	Cost	Services
Interface endpoint (AWS PrivateLink)	Creates ENIs with private IPs in your subnets; DNS name resolves to them	Roughly $0.01 per hour per AZ plus about $0.01 per GB	most other services (ECR, STS, CloudWatch, SSM, Secrets Manager…) and partner services

Rule of thumb: **always** create the free S3 gateway endpoint in any VPC that uses S3 heavily, then decide on interface endpoints by comparing their fixed cost with the NAT data charges they would replace.

6 Step-by-step guide

Prerequisites: AWS CLI v2, region `us-east-1`, and the VPC from topic 4 §6.4. If you deleted it, recreate a minimal version:

```bash
export AWS_DEFAULT_REGION=us-east-1
VPC_ID=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=shop-api-vpc}]' --query
  Vpc.VpcId --output text)
mk() { aws ec2 create-subnet --vpc-id "$VPC_ID" --cidr-block "$2" --availability-zone "$3"
  \
  --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=shop-api-$1}]" --query
  Subnet.SubnetId --output text; }
PUB_A=$(mk public-a 10.0.0.0/24 us-east-1a);  PUB_B=$(mk public-b 10.0.1.0/24 us-east-1b)
APP_A=$(mk app-a 10.0.10.0/24 us-east-1a);    APP_B=$(mk app-b 10.0.11.0/24 us-east-1b)
```

Cost: steps 1-5 are free (IGW, route tables and gateway endpoints cost nothing). **Step 6 creates a NAT Gateway and Elastic IP: about 5 cents per hour combined plus data. Do it for under an hour and run the cleanup in step 8.** Set a billing alarm first (see EC2 step 1 §6.1 for a budget). LocalStack (`docker run -d --rm -p 4566:4566 localstack/localstack` and `--endpoint-url http://localhost:4566`) accepts these API calls for practice, but it does not route real packets.

1. **Create and attach an Internet Gateway.**

```bash
IGW_ID=$(aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=shop-api-
  igw}]' \
  --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id "$IGW_ID" --vpc-id "$VPC_ID"
```

2. **Create a public route table, add the default route, and associate the public subnets.**

```bash
RT_PUB=$(aws ec2 create-route-table --vpc-id "$VPC_ID" \
  --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=shop-api-
  public}]' \
  --query RouteTable.RouteTableId --output text)
aws ec2 create-route --route-table-id "$RT_PUB" --destination-cidr-block 0.0.0.0/0 --
  gateway-id "$IGW_ID"
aws ec2 associate-route-table --route-table-id "$RT_PUB" --subnet-id "$PUB_A"
aws ec2 associate-route-table --route-table-id "$RT_PUB" --subnet-id "$PUB_B"
```

`create-route` prints `{"Return": true}`.

3. **Create one private route table per AZ** and associate the app subnets (no internet route yet):

```bash
mkrt() { aws ec2 create-route-table --vpc-id "$VPC_ID" \
  --tag-specifications "ResourceType=route-table,Tags=[{Key=Name,Value=shop-api-
  private-$1}]" \
  --query RouteTable.RouteTableId --output text; }
RT_PRIV_A=$(mkrt a); RT_PRIV_B=$(mkrt b)
aws ec2 associate-route-table --route-table-id "$RT_PRIV_A" --subnet-id "$APP_A"
aws ec2 associate-route-table --route-table-id "$RT_PRIV_B" --subnet-id "$APP_B"
```

4. **Inspect every table.** A summary of destination, target and state:

```bash
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=$VPC_ID" \
  --query 'RouteTables[].[Tags[?Key==`Name`]|[0].Value,Routes[].
  [DestinationCidrBlock,GatewayId,NatGatewayId,State]]' \
  --output json
```

The public table shows `10.0.0.0/16 local` and `0.0.0.0/0 igw-...`. The private tables show only `local`.

5. **Add the free S3 gateway endpoint to the private tables.**

```bash
aws ec2 create-vpc-endpoint --vpc-id "$VPC_ID" --service-name com.amazonaws.us-east-
  1.s3 \
  --route-table-ids "$RT_PRIV_A" "$RT_PRIV_B" --query VpcEndpoint.VpcEndpointId --
  output text
aws ec2 describe-route-tables --route-table-ids "$RT_PRIV_A" \
  --query 'RouteTables[].Routes[].
  [DestinationCidrBlock,DestinationPrefixListId,GatewayId]' --output table
```

A new route appears with a destination of the form `pl-xxxxxxxx` (the S3 prefix list) and a target `vpce-...`. Instances in the private subnets now reach S3 without any internet path.

6. **(Optional, costs money) Add a NAT Gateway for outbound internet from the private subnets.**

```bash
EIP_ALLOC=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
NAT_ID=$(aws ec2 create-nat-gateway --subnet-id "$PUB_A" --allocation-id "$EIP_ALLOC"
  \
  --query NatGateway.NatGatewayId --output text)
aws ec2 wait nat-gateway-available --nat-gateway-ids "$NAT_ID"      # about 1-2
  minutes
aws ec2 create-route --route-table-id "$RT_PRIV_A" --destination-cidr-block 0.0.0.0/0
  --nat-gateway-id "$NAT_ID"
```

The NAT Gateway sits in the **public** subnet (it needs the IGW route to reach the internet). Only the AZ-a private table points at it, which shows the "one NAT per AZ" design (the AZ-b table would need its own NAT).

7. **Verify with a real instance (optional).** Launch a `t3.micro` into `APP_A` with no public IP (see EC2 §6.1), connect through Session Manager or a bastion, and run `curl -s https://checkip.amazonaws.com`. The address returned is the NAT's Elastic IP (`aws ec2 describe-addresses --allocation-ids "$EIP_ALLOC"`). Remove the NAT route (`aws ec2 delete-route --route-table-id "$RT_PRIV_A" --destination-cidr-block 0.0.0.0/0`) and the same `curl` hangs. Without an instance, the route listing from step 4 is your evidence.
8. **Clean up in dependency order.** Do not skip the NAT Gateway and Elastic IP: they bill by the hour.

```bash
aws ec2 delete-nat-gateway --nat-gateway-id "$NAT_ID"
until [ "$(aws ec2 describe-nat-gateways --nat-gateway-ids "$NAT_ID" --query
  'NatGateways[0].State' --output text)" = deleted ]; do sleep 10; done
aws ec2 release-address --allocation-id "$EIP_ALLOC"
# endpoint, IGW, route tables, then subnets and VPC
VPCE=$(aws ec2 describe-vpc-endpoints --filters "Name=vpc-id,Values=$VPC_ID" --query
  'VpcEndpoints[].VpcEndpointId' --output text)
aws ec2 delete-vpc-endpoints --vpc-endpoint-ids $VPCE
aws ec2 detach-internet-gateway --internet-gateway-id "$IGW_ID" --vpc-id "$VPC_ID"
aws ec2 delete-internet-gateway --internet-gateway-id "$IGW_ID"
for rt in "$RT_PUB" "$RT_PRIV_A" "$RT_PRIV_B"; do
  for a in $(aws ec2 describe-route-tables --route-table-ids "$rt" --query
  'RouteTables[0].Associations[].RouteTableAssociationId' --output text); do
    aws ec2 disassociate-route-table --association-id "$a"; done
  aws ec2 delete-route-table --route-table-id "$rt"
done
```

```
for s in "$PUB_A" "$PUB_B" "$APP_A" "$APP_B"; do aws ec2 delete-subnet --subnet-id
  "$s"; done
aws ec2 delete-vpc --vpc-id "$VPC_ID"
```

Finish with `aws ec2 describe-nat-gateways --filter Name=state,Values=available,pending` and `aws ec2 describe-addresses`; both should list nothing from this exercise. If you plan to continue to the next topic, keep the VPC, IGW and route tables (all free) but delete the NAT Gateway and Elastic IP.

7 Real-world scenario

Illustrative scenario, not a documented incident. The `shop-api` team's monthly AWS bill jumps by roughly $1,400 after they move their nightly report job into private subnets. Cost Explorer filtered by usage type shows `NatGateway-Bytes` as the new top item. The job reads about 30 TB a month of export files from S3 and writes results back, and every byte now crosses the NAT Gateway and pays the per-GB processing fee.

Investigation: VPC Flow Logs (CloudWatch §6.18 or S3 as destination) confirm the destination addresses are S3 ranges. **Fix:** add the free S3 gateway endpoint to the private route tables. The route for S3's prefix list is more specific than `0.0.0.0/0`, so traffic shifts immediately with no instance restarts; the next day's NAT bytes fall by more than 95 percent. **Lesson:** map your traffic to its path. S3 and DynamoDB traffic should never ride a NAT, and a cost anomaly alert on the NAT usage type would have caught it in a day instead of a month.

8 Common mistakes and best practices

Don't

- Put `0.0.0.0/0 → igw` in the main route table (every new subnet becomes public by accident).
- Place a NAT Gateway in a private subnet: it needs a route to the IGW, so it must sit in a public subnet.
- Forget to delete NAT Gateways, Elastic IPs and interface endpoints after a lab; they bill hourly even with zero traffic.
- Assume a route is enough: public IP, security group and NACL must also allow the flow.
- Send S3 or DynamoDB traffic through NAT.

Do

- Keep the main route table local-only and associate every subnet explicitly.

- Use one NAT Gateway per AZ in production and one private route table per AZ; accept a single NAT only for dev.
- Add the S3 gateway endpoint to every VPC by default; add interface endpoints where the numbers justify them.
- Use VPC Reachability Analyzer (a small per-analysis charge) or Flow Logs to debug paths instead of guessing.
- Alert on `NatGateway-Bytes` and hourly NAT charges with AWS Budgets or Cost Anomaly Detection.

9 Tasks

- [ ] **T1 — Make a subnet public** *(≈30 min, beginner, free)*
  - **Goal:** build the IGW, public route table and associations.
  - **Steps:**
    1. Run steps 1-4 of the guide on your VPC.
    2. Run the `describe-route-tables` query and highlight which line makes the subnet public.
  - **Done when:** the public table shows `0.0.0.0/0 → igw-...` and both public subnets list it under their associations.
- [ ] **T2 — Add the S3 gateway endpoint and read the route** *(≈20 min, beginner, free)*
  - **Goal:** see that an endpoint is just a route to a prefix list.
  - **Steps:**
    1. Run step 5.
    2. Look up the prefix list with `aws ec2 describe-managed-prefix-lists --filters Name=prefix-list-name,Values=com.amazonaws.us-east-1.s3`.
    3. Explain why this route wins over `0.0.0.0/0` for S3 traffic.
  - **Done when:** you can point at the `pl-` route and state "longest prefix match" as the reason.
- [ ] **T3 — Break it: blackhole and wrong association** *(≈30 min, intermediate, uses NAT for minutes)*
  - **Goal:** recognise the two silent failures.
  - **Steps:**
    1. Do step 6, then `aws ec2 delete-nat-gateway` without removing the route. Inspect the private table.

2. Disassociate `APP_B` from its private table (`aws ec2 disassociate-route-table --association-id ...`). It now silently uses the VPC's main route table; show that table with `aws ec2 describe-route-tables --filters Name=vpc-id,Values=$VPC_ID Name=association.main,Values=true`. To see the danger, add a temporary `0.0.0.0/0 → igw` route to the main table and note that `APP_B` is now public.
3. Repair both.

- **Done when:** you saw a route with state `blackhole`, explained the implicit main-table association, and restored valid routes (including removing the temporary main-table IGW route).

- [ ] **T4 — Price the NAT before you build it** *(≈30 min, intermediate)*
  - **Goal:** estimate cost like a reviewer.
  - **Steps:**
    1. Using the AWS pricing page, compute the monthly cost of three NAT Gateways plus 2 TB of processing.
    2. Compare with one NAT plus an S3 gateway endpoint when 1.5 TB of that traffic is S3.
    3. Write the numbers and your assumptions.
  - **Done when:** you have two figures, the saving, and a cited pricing page date.

- [ ] **T5 — Automate the public/private network as code** *(≈45 min, advanced)*
  - **Goal:** repeatable routing with correct associations and cleanup.
  - **Steps:**
    1. Extend your `create-vpc.sh` from topic 4 to create the IGW, public and per-AZ private tables, S3 endpoint and an optional `--with-nat` flag.
    2. Make `destroy` reverse everything in the right order (step 8).
  - **Done when:** a create plus destroy cycle with `--with-nat` leaves `describe-nat-gateways` and `describe-addresses` empty.

- [ ] **T6 — Prove it: trace a packet** *(≈30 min, advanced)*
  - **Goal:** explain every hop without notes.
  - **Steps:**
    1. Draw the path of (a) a browser request to a public ALB, (b) an app instance calling `api.stripe.com`, (c) an app instance writing to S3.
    2. For each, name the route table entry used and whether it costs per-GB.
  - **Done when:** your drawing matches the sequence diagram's logic and you can answer "what happens if the NAT's AZ fails?".

10 Self-check questions

1. What exactly makes a subnet public?
2. Why must a NAT Gateway sit in a public subnet, and why do private instances not need public IPs?
3. A private instance can reach S3 but not `github.com`. Name two possible causes.
4. Which route wins for `S3 prefix list` versus `0.0.0.0/0`, and why?
5. How can a design with a NAT Gateway still fail when one AZ goes down?

ANSWERS

1. A route `0.0.0.0/0 → Internet Gateway` in its associated route table (plus public IPs on its instances).
2. The NAT Gateway itself needs a path to the internet through the IGW and an Elastic IP; private instances use the NAT's address so they need none.
3. The private route table has no NAT route (or it is a blackhole), the NAT Gateway was deleted or is in a failed AZ, or security group, NACL or DNS blocks it. S3 works through the gateway endpoint regardless.
4. The prefix-list route wins: longest-prefix match prefers the more specific destination.
5. If a single NAT serves all AZs and its AZ fails, the others lose outbound internet. Use one NAT per AZ with per-AZ route tables.

11 Further reading

- AWS documentation: *Amazon VPC User Guide* (route tables, internet gateways, NAT gateways, gateway endpoints, AWS PrivateLink).
- AWS pricing pages: *Amazon VPC pricing* (NAT Gateway, PrivateLink and public IPv4 charges).
- AWS documentation: *VPC Reachability Analyzer* and *VPC Flow Logs*.
- Level 20: Network Egress Costs §20.7 for the wider cost picture.

6.6 Security Groups and Network ACLs

Topic 6 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

AWS gives you two firewalls inside a VPC. A **Security Group (SG)** is a virtual firewall attached to a network interface (so, in practice, to an EC2 instance, a load balancer, an RDS database and so on). It is **stateful**: if you allow a request in, the reply is automatically allowed out, and vice versa. It has only **allow** rules, and everything not allowed is denied. A **Network ACL (NACL, Network Access Control List)** is a firewall attached to a **subnet**. It is **stateless**: it judges every packet in each direction separately, so return traffic needs its own rule. It has numbered **allow and deny** rules evaluated in order.

The most useful trick in the whole topic is **security group referencing**: instead of writing "allow port 5432 from 10.0.10.0/24", you write "allow port 5432 from *the app tier's security group*". The rule then follows the instances as they scale, get replaced and change IP addresses.

2 Why it matters in DevOps

- **Most cloud breaches begin with exposure:** an SSH port, a database or an admin UI open to `0.0.0.0/0`. Security groups are the control that prevents it, and reviewing them is a standard audit step.
- **Debugging connectivity** is mostly "which of the SG, NACL, route table or OS firewall is dropping this?" Knowing stateful versus stateless saves hours of staring at a timeout.
- **Blast radius:** tiered security groups mean a compromised web server cannot talk to the database unless the app tier is also allowed to.
- **Automation:** SG rules are among the most frequently edited resources in Terraform; a rule that references an IP instead of a group becomes stale as soon as autoscaling replaces a host (Auto Scaling §6.2).
- See the security view in Level 11: Cloud Security Groups and Policies §11.3.

3 Real-world system example

The three-tier reference pattern used throughout AWS documentation. AWS's own guides for Elastic Load Balancing and RDS recommend this chain: the load balancer's security group accepts HTTPS from the internet; the application instances' group accepts the app port **only from the load balancer's group**; the database group accepts the database port **only from the application group**. Amazon EKS follows the same idea: it creates a *cluster security group* that allows the managed control plane and the worker nodes to talk to each other, and you attach additional groups for your own rules. Teams at large companies also run automated scanners over all security groups (Netflix open-sourced one early example, Security Monkey, to flag risky rules; today AWS Config rules and Security Hub do the same job natively).

Our `shop-api` uses exactly this chain, plus a Redis group that only the app tier may reach.

4 Diagram

Layered defence for `shop-api`; arrows are the only allowed flows:

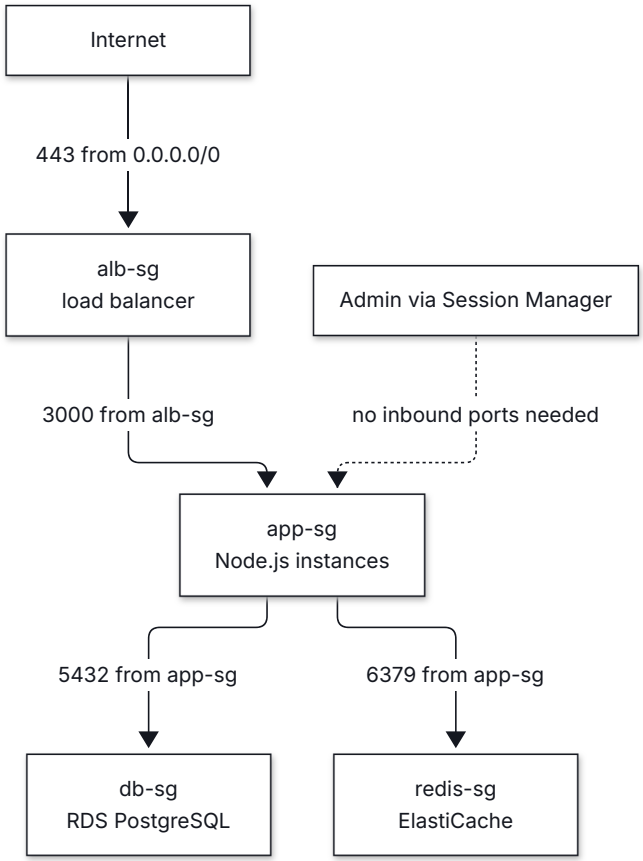


Figure 6.6-1

Stateful versus stateless on one HTTPS request. The SG remembers the connection; the NACL does not:

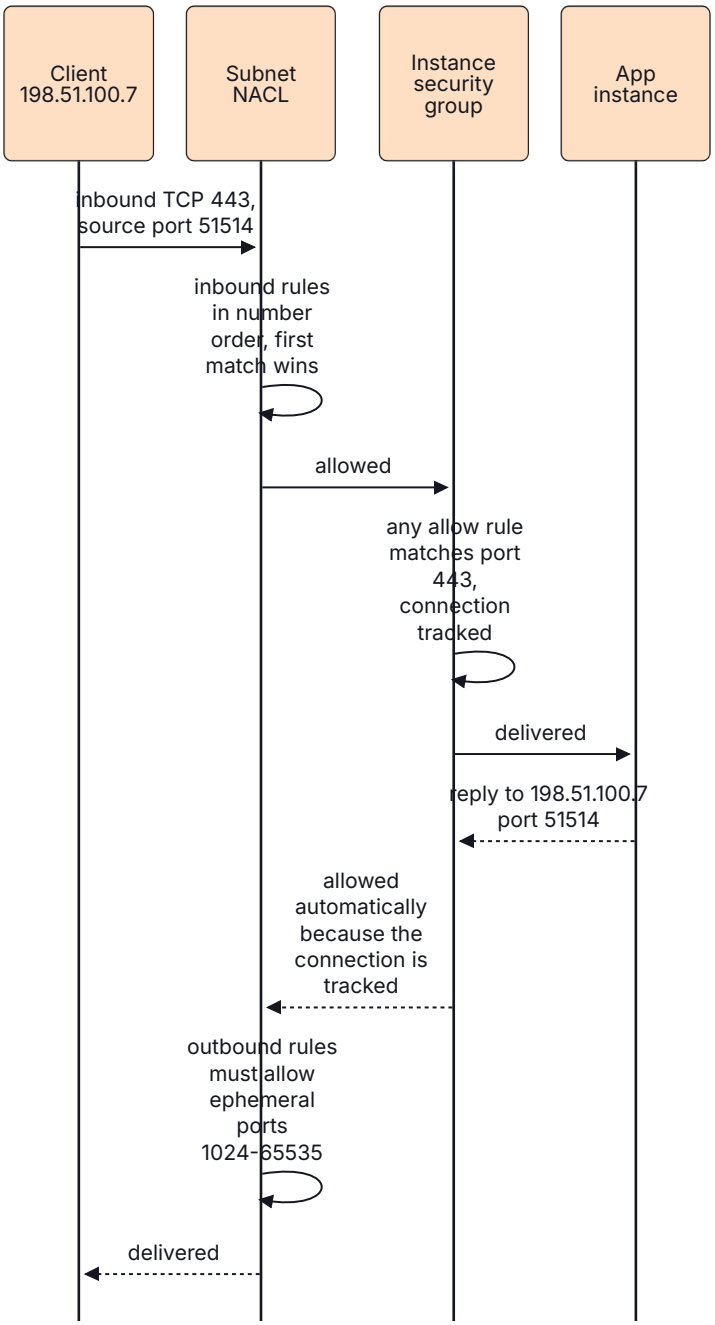


Figure 6.6-2

5 How it works

	Security group	Network ACL
Attached to	Network interface (instance, ALB, RDS, Lambda in VPC…)	Subnet (every subnet has exactly one)
State	**Stateful** (return traffic automatic)	**Stateless** (allow both directions yourself)
Rule types	Allow only	Allow and deny
Evaluation	All rules are evaluated together; any allow permits	Rules in ascending number; first match wins; final `*` rule denies
Default for new resource	No inbound; **all outbound** allowed	The default NACL allows everything; a *custom* NACL denies everything until you add rules
Sources	CIDR, prefix list or **another security group**	CIDR only
Typical use	The main control, per tier	Coarse backstop: block a bad CIDR, subnet-level guardrail

Details that cause real incidents:

- **Referenced SGs match private IPs.** A rule that allows `app-sg` matches traffic whose *source private IP* belongs to an interface with that group. Traffic that arrives via a public or Elastic IP does not match; use the private address in the connection string.
- **Rules are additive.** An instance can have several security groups (the default limit is five per network interface); the effective policy is the union. A single stray `0.0.0.0/0` rule in any attached group opens it.
- **Removing a rule does not always kill an existing flow.** Tracked connections can persist briefly; do not rely on a rule deletion to cut an active session.
- **Ephemeral ports.** A client connects from a random high port (Linux defaults to 32768-60999; Windows and some NAT devices use others). The usual stateless NACL pattern is to allow outbound TCP `1024-65535` to cover replies.
- **NACLs do not filter everything.** Traffic to the VPC DNS resolver, the instance metadata service and DHCP is not evaluated by NACLs.
- **Quotas** (adjustable): by default 60 inbound and 60 outbound rules per security group and 5 groups per interface.
- **Egress is open by default.** Locking outbound traffic down is a legitimate hardening step for sensitive tiers, but plan for the destinations you need: package mirrors, S3 endpoints, the metadata service (link-local, not filtered by SGs).

Typical rule sets for `shop-api`

SG	Inbound	Why
`alb-sg`	TCP 443 from `0.0.0.0/0` (and 80 for redirect)	Public entry point
`app-sg`	TCP 3000 from `alb-sg`	Only the load balancer reaches the app
`db-sg`	TCP 5432 from `app-sg`	Only the app reaches PostgreSQL
`redis-sg`	TCP 6379 from `app-sg`	Only the app reaches Redis

For shell access prefer **AWS Systems Manager Session Manager** (needs an instance role and either outbound internet/NAT or VPC endpoints) over opening port 22. If you must allow SSH, restrict it to your own `/32` and remove it after use. Compare with host firewalls in Level 2 §2.4.

6 Step-by-step guide

Prerequisites: AWS CLI v2, region `us-east-1`, and a VPC (from topic 4 §6.4; the default VPC works too: `VPC_ID=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true --query 'Vpcs[0].VpcId' --output text)`). **Cost:** security groups and NACLs are free. Step 6 optionally launches a `t3.micro` (a few cents per hour); clean up in step 8. **LocalStack** (`docker run -d --rm -p 4566:4566 localstack/localstack`, add `--endpoint-url http://localhost:4566`) supports the SG and NACL APIs for rehearsal but does not enforce them on traffic.

1. **Create the four groups.**

```bash
export AWS_DEFAULT_REGION=us-east-1
sg() { aws ec2 create-security-group --vpc-id "$VPC_ID" --group-name "shop-api-$1" \
       --description "$2" --query GroupId --output text; }
ALB_SG=$(sg alb   "ALB public entry")
APP_SG=$(sg app   "Node.js app tier")
DB_SG=$(sg db     "PostgreSQL")
REDIS_SG=$(sg redis "Redis")
echo "$ALB_SG $APP_SG $DB_SG $REDIS_SG"
```

2. **Add the rules, referencing groups rather than CIDRs.**

```bash
aws ec2 authorize-security-group-ingress --group-id "$ALB_SG" --protocol tcp --port
  443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id "$APP_SG" --protocol tcp --port
  3000 --source-group "$ALB_SG"
aws ec2 authorize-security-group-ingress --group-id "$DB_SG"  --protocol tcp --port
  5432 --source-group "$APP_SG"
aws ec2 authorize-security-group-ingress --group-id "$REDIS_SG" --protocol tcp --port
  6379 --source-group "$APP_SG"
```

   Each call prints the new rule (`SecurityGroupRules`).

3. **Read the result back.**

```bash
aws ec2 describe-security-group-rules --filters "Name=group-id,Values=$DB_SG" \
  --query 'SecurityGroupRules[].
  [IsEgress,IpProtocol,FromPort,ToPort,CidrIpv4,ReferencedGroupInfo.GroupId]' --output
  table
```

   You see an inbound TCP 5432 rule whose source is `$APP_SG` (no CIDR) and the default egress-all rule.

4. **Tighten the database group's egress.** A database rarely needs to start outbound connections:

```bash
aws ec2 revoke-security-group-egress --group-id "$DB_SG" \
  --ip-permissions 'IpProtocol=-1,IpRanges=[{CidrIp=0.0.0.0/0}]'
```

   Note: RDS managed features (backups, monitoring) use AWS-side paths and do not depend on this rule, but test any non-trivial change in a non-production environment first.

5. **Add a stateless NACL for the data subnets and see the ordering rules.** First a model you can run anywhere. This toy evaluator mirrors the NACL algorithm (lowest number that matches wins, implicit final deny):

```bash
python3 - <<'EOF'
import ipaddress
def evaluate(rules, proto, ip, port):
    for num, action, p, cidr, lo, hi in sorted(rules):
        if p in ("all", proto) and ipaddress.ip_address(ip) in
  ipaddress.ip_network(cidr) and lo ⩽ port ⩽ hi:
            return f"{action} (rule {num})"
```

```
    return "deny (rule *)"
inbound  = [(100,"allow","tcp","0.0.0.0/0",443,443),
  (90,"deny","tcp","203.0.113.0/24",0,65535)]
outbound = [(100,"allow","tcp","0.0.0.0/0",1024,65535)]
print("in  443 from 198.51.100.7:", evaluate(inbound,"tcp","198.51.100.7",443))
print("in  443 from 203.0.113.9 :", evaluate(inbound,"tcp","203.0.113.9",443))
print("in  22  from 198.51.100.7:", evaluate(inbound,"tcp","198.51.100.7",22))
print("out reply to port 51514  :", evaluate(outbound,"tcp","198.51.100.7",51514))
print("out reply to port 80     :", evaluate(outbound,"tcp","198.51.100.7",80))
EOF
```

Expected output:

```
in  443 from 198.51.100.7: allow (rule 100)
in  443 from 203.0.113.9 : deny (rule 90)
in  22  from 198.51.100.7: deny (rule *)
out reply to port 51514  : allow (rule 100)
out reply to port 80     : deny (rule *)
```

Rule 90 is evaluated before rule 100, so the denied range loses even though a later rule would allow it.

6. **Create a real custom NACL and attach it** (use a subnet you can afford to break; the default VPC's subnet is fine):

```bash
NACL=$(aws ec2 create-network-acl --vpc-id "$VPC_ID" --query NetworkAcl.NetworkAclId -
  -output text)
# inbound: HTTPS from anywhere; outbound: ephemeral replies
aws ec2 create-network-acl-entry --network-acl-id "$NACL" --ingress --rule-number 100
  \
  --protocol tcp --port-range From=443,To=443 --cidr-block 0.0.0.0/0 --rule-action
  allow
aws ec2 create-network-acl-entry --network-acl-id "$NACL" --egress --rule-number 100 \
  --protocol tcp --port-range From=1024,To=65535 --cidr-block 0.0.0.0/0 --rule-action
  allow
ASSOC=$(aws ec2 describe-network-acls --filters "Name=association.subnet-
  id,Values=$SUBNET_ID" \
  --query 'NetworkAcls[0].Associations[?
  SubnetId==`'"$SUBNET_ID"'`].NetworkAclAssociationId' --output text)
aws ec2 replace-network-acl-association --association-id "$ASSOC" --network-acl-id
  "$NACL"
```

Set `SUBNET_ID` to the chosen subnet first. Any instance in it is now reachable only on 443; SSH, ping and even `dnf` downloads on port 80 fail until you add rules. That is the "custom NACL denies everything" default in action.

7. **Restore the original NACL** so you are not locked out:

```bash
DEFAULT_NACL=$(aws ec2 describe-network-acls --filters "Name=vpc-id,Values=$VPC_ID"
  "Name=default,Values=true" \
  --query 'NetworkAcls[0].NetworkAclId' --output text)
NEW_ASSOC=$(aws ec2 describe-network-acls --network-acl-ids "$NACL" --query
  'NetworkAcls[0].Associations[0].NetworkAclAssociationId' --output text)
aws ec2 replace-network-acl-association --association-id "$NEW_ASSOC" --network-acl-id
  "$DEFAULT_NACL"
```

8. **Clean up.** Delete the custom NACL, then the groups (a group still referenced by another group cannot be deleted, so remove referencing groups first):

```bash
aws ec2 delete-network-acl --network-acl-id "$NACL"
for g in "$DB_SG" "$REDIS_SG" "$APP_SG" "$ALB_SG"; do aws ec2 delete-security-group --
  group-id "$g"; done
```

   `DependencyViolation` means something still uses the group (an instance, a network interface or another group's rule). Terminate any test instance and delete the VPC if you created one just for this topic.

9. **Audit your account for risky rules** (read-only, free). It lists groups open to the world:

```bash
aws ec2 describe-security-group-rules \
  --filters Name=group-id,Values="$APP_SG" --query 'SecurityGroupRules[?
  CidrIpv4==`0.0.0.0/0` && !IsEgress]'
aws ec2 describe-security-groups --filters Name=ip-permission.cidr,Values=0.0.0.0/0 \
  --query 'SecurityGroups[].[GroupId,GroupName]' --output table
```

   Anything on this list other than a public load balancer on 80/443 needs a justification.

7 Real-world scenario

Illustrative scenario, not a documented incident. After a deploy, `shop-api` pages start timing out at random. The database is fine, the app logs show `ETIMEDOUT` connecting to Redis. Nothing changed in the security groups, which is suspicious. The engineer checks the instance's groups and finds the new Auto Scaling launch template attached `shop-api-app` correctly, but the Redis group's rule was written months ago as "6379 from 10.0.10.0/24". The new instances landed in the new `10.0.11.0/24` subnet in the second AZ, which that CIDR does not cover. Half the fleet works; half cannot reach Redis.

Fix: replace the CIDR rule with a reference to `app-sg`, in Terraform, and add a test that connects from each AZ. **Lesson:** reference groups, not addresses. The rule should describe *who* may connect, not *where* they currently live.

8 Common mistakes and best practices

Don't

- Open SSH (22), RDP (3389), databases or admin dashboards to `0.0.0.0/0`.
- Use the VPC's *default* security group for workloads; it allows all traffic between members of the group.
- Rely on NACLs as your main control; they are hard to reason about and easy to break with a missing ephemeral-port rule.
- Reuse one big "allow everything internal" group for all tiers.
- Write CIDR rules for traffic that comes from your own instances.

Do

- Chain groups by reference: internet to ALB to app to data.
- Name groups by role (`shop-api-app`) and describe every rule (the description field is searchable in audits).
- Prefer Session Manager to inbound SSH; if SSH is required, allow only a specific `/32`.
- Use prefix lists for sets of CIDRs (office ranges, partner ranges) so there is one place to update.
- Keep NACLs at the default or add only coarse deny rules; check rules continuously with AWS Config or Security Hub.

9 Tasks

- [ ] **T1 — Build the security-group chain** *(≈30 min, beginner, free)*
  - **Goal:** reproduce the four-group layout using references.
  - **Steps:**
    1. Run guide steps 1-3.
    2. Print each group's rules and underline where a group, not a CIDR, is the source.
  - **Done when:** `db-sg` has exactly one inbound rule whose source is `app-sg`.
- [ ] **T2 — Prove a SG reference works on real instances** *(≈45 min, intermediate, about 10 cents)*
  - **Goal:** show the chain with live traffic.

- **Steps:**
  1. Launch two `t3.micro` instances (see EC2 §6.1): one with `alb-sg` and one with `app-sg`, in the same VPC, no key pair, Session Manager access if your account has an instance profile ready (see IAM §6.7).
  2. On the `app-sg` instance run `python3 -m http.server 3000`.
  3. From the `alb-sg` instance run `curl -s -m 5 http://<app-private-ip>:3000/`, then repeat from a third instance with a different group.
  4. Terminate all instances.
- **Done when:** the call from `alb-sg` returns the directory listing and the call from the unrelated group times out.

☐ **T3 — Break it with a NACL and fix it** *(≈30 min, intermediate)*

- **Goal:** experience stateless filtering.
- **Steps:**
  1. Do guide steps 6-7 but omit the egress ephemeral rule first.
  2. Observe that connections to port 443 hang even though the SG and inbound NACL allow them.
  3. Add the egress rule 100 and see the connection succeed; restore the default NACL.
- **Done when:** you can explain, in one sentence, why the reply was dropped and show the rule that fixed it.

☐ **T4 — Simulate rule ordering** *(≈20 min, intermediate)*

- **Goal:** predict NACL decisions.
- **Steps:**
  1. Save the Python evaluator from step 5 as `nacl.py`.
  2. Add a deny rule numbered 150 for port 443 from `198.51.100.0/24` and predict the result for `198.51.100.7:443` before running.
  3. Renumber the rule to 50 and predict again.
- **Done when:** both predictions match the script output (rule 100 allows when the deny is 150; the deny wins as rule 50).

☐ **T5 — Automate a security-group audit** *(≈45 min, advanced)*

- **Goal:** a script that fails CI when a world-open sensitive port exists.
- **Steps:**
  1. Write `audit-sg.sh` that runs `describe-security-groups` with `--query` and exits non-zero if any inbound rule for ports 22, 3389, 5432, 6379 or 3306 has `0.0.0.0/0` or `::/0`.

2. Test it against a deliberately bad group, then clean up.
3. Add it to a GitHub Actions workflow with read-only credentials (Level 5 pipelines §5.2).

- **Done when:** the script exits 1 on the bad group and 0 after you delete it.

☐ **T6 — Prove it: design the firewall plan** *(≈30 min, advanced)*

- **Goal:** defend a layered design without notes.
- **Steps:**
  1. Without the guide, draw the network for `shop-api` with ALB, app, RDS, Redis and a bastion-free admin path.
  2. Write the security group rules and say what, if anything, you would add as NACL rules and why.
  3. Explain stateful versus stateless using one packet's round trip.
- **Done when:** a reviewer can find no world-open ports except 443 on the load balancer.

10 Self-check questions

1. What does "stateful" mean for a security group, and what extra rules does a NACL need because it is stateless?
2. Why is allowing `app-sg` as a source better than allowing `10.0.10.0/24`?
3. If a security group allows a connection but the NACL has no matching rule, what happens?
4. An instance has two security groups. One allows TCP 22 from `0.0.0.0/0`, the other allows nothing. Is SSH open?
5. When is a NACL the right tool?

ANSWERS

1. The SG remembers accepted connections and allows replies automatically. A NACL needs explicit rules for return traffic, typically outbound ephemeral ports 1024-65535 (and inbound ones for outbound-initiated connections).
2. The reference tracks membership: new or replaced instances are covered immediately, whatever their IP or subnet.
3. The packet is dropped (implicit deny at the end of the NACL). Both must allow it.
4. Yes. Rules are the union across all attached groups.
5. For coarse subnet-wide guardrails such as blocking a hostile CIDR, or when a compliance requirement demands an explicit deny layer.

11 Further reading

- AWS documentation: *Amazon VPC User Guide* (security groups, network ACLs, security group rules).
- AWS documentation: *AWS Systems Manager Session Manager*.
- AWS documentation: *VPC Reachability Analyzer* and *VPC Flow Logs* for debugging.
- *AWS Well-Architected Framework*, Security pillar (infrastructure protection).

LAB NOTES What I ran, what I saw, what it means

6.7 IAM

Topic 7 of 18 | ~6 hours | Difficulty: Intermediate

1 What it is

IAM (Identity and Access Management) is the AWS service that decides **who** (a *principal*) may do **what** (an *action*, such as `s3:GetObject`) on **which thing** (a *resource*, identified by an **ARN**, Amazon Resource Name), and under which *conditions*. Every single AWS API call, from the console, the CLI or your code, is authenticated and authorized by IAM. IAM itself is free.

The building blocks: **users** (long-lived identities for a person or program, with passwords or access keys), **groups** (collections of users), **roles** (identities with no permanent credentials that anyone allowed can *assume* to receive temporary ones) and **policies** (JSON documents that allow or deny actions). A policy attached to an identity says what it can do; a **trust policy** on a role says who may assume it; a **resource policy** (for example an S3 bucket policy) says who may touch that resource. Short-lived credentials are issued by the **STS (Security Token Service)**.

2 Why it matters in DevOps

- **It is the real perimeter.** Network controls matter, but nearly every serious cloud incident involves leaked or over-powered credentials. IAM mistakes are how a leaked key becomes a full account takeover.
- **Everything automated needs an identity:** Terraform, CI pipelines, Lambda functions, EC2 instances and Kubernetes pods each need a role with just enough permission (Level 11: Least-privilege IAM §11.2).
- **Debugging:** `AccessDenied` is the most frequent error in AWS. Knowing the evaluation logic turns a day of guessing into a ten-minute read of the error and the policies.
- **Cost of mistakes is asymmetric:** an over-broad policy costs nothing until the day it costs everything; static access keys committed to Git are found by bots within minutes.

3 Real-world system example

GitHub Actions deploying to AWS. Instead of storing long-lived AWS keys as repository secrets, the documented modern pattern is OpenID Connect (OIDC) federation: AWS IAM trusts GitHub's identity provider (`token.actions.githubusercontent.com`), the workflow requests a signed token for a specific repository and branch, and exchanges it with STS (`AssumeRoleWithWebIdentity`) for temporary credentials of a role whose trust policy matches only that repository. No secret is stored, and credentials expire in an hour or less (Secrets and Environments §3.12).

Inside AWS, **EC2 instance profiles** and **EKS IAM roles for service accounts** work the same way: the platform hands the workload short-lived credentials from STS. Netflix has also publicly described and open-sourced tooling (Repokid) that uses IAM's "last accessed" data to shrink role permissions automatically: least privilege as a continuous process.

4 Diagram

How a request is evaluated (simplified from the AWS "policy evaluation logic"

documentation):

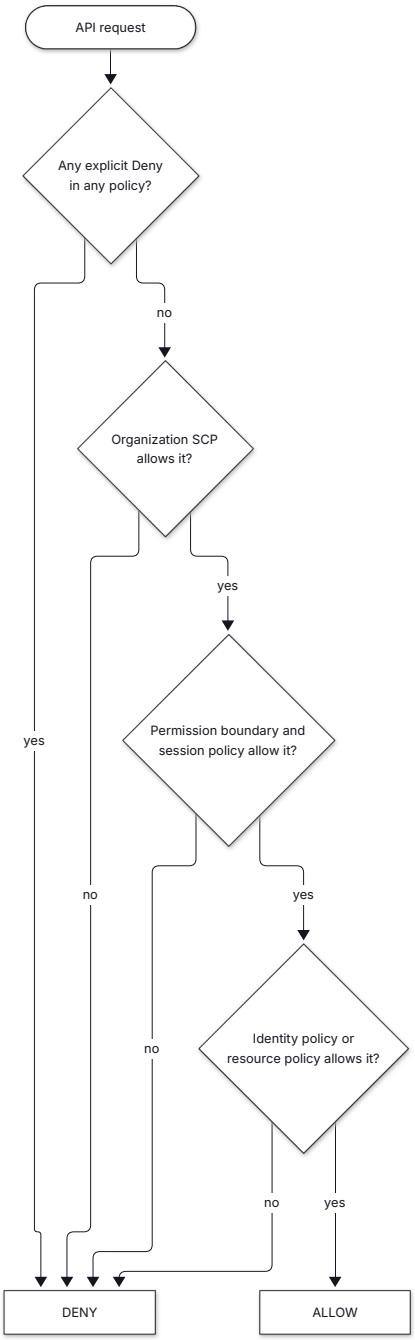


Figure 6.7-1

How an EC2 instance gets credentials with no stored secrets (instance profile and STS):

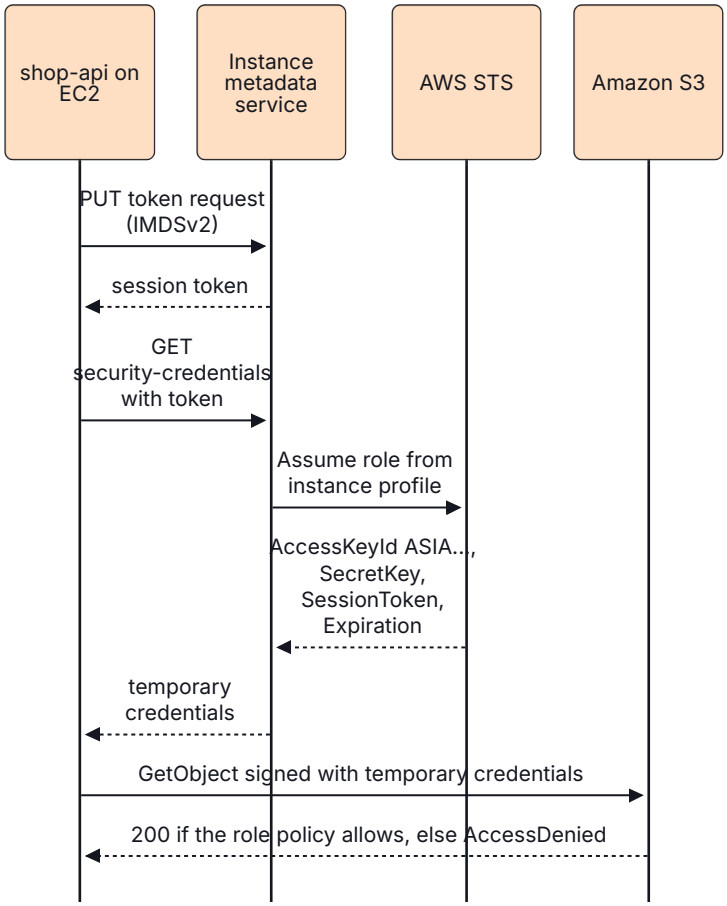


Figure 6.7-2

5 How it works

Identity types

Type	Credentials	Use for
Root user	Email + password (+ MFA)	Almost nothing: account setup, billing settings, a few break-glass tasks. Enable MFA, never create root access keys
IAM user	Password and/or long-lived access keys (`AKIA...`)	Legacy; avoid for humans and workloads
IAM Identity Center user	SSO login, short-lived CLI credentials	The recommended way for **people**, especially across many accounts

Type	Credentials	Use for
IAM role	None of its own; temporary keys (`ASIA...`) + session token from STS	EC2, Lambda, ECS/EKS tasks, CI via OIDC, cross-account access, human federation

Policy types

Policy	Attached to	Effect
Identity-based (managed or inline)	user, group, role	Grants permissions
Resource-based (S3 bucket policy, KMS key policy, SQS queue policy, role **trust policy**)	the resource	Grants access to named principals, even in other accounts
Permissions boundary	user or role	Sets the **maximum** an identity could ever get
Service control policy (SCP)	AWS Organizations account or OU	Guardrail for whole accounts; never grants, only limits
Session policy	an assumed-role session	Further narrows a single session

Anatomy of a policy (a least-privilege policy for the `shop-api` app role):

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadWriteUploads",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::shop-api-assets-<ACCOUNT_ID>/uploads/*"
    },
    {
      "Sid": "ListUploadsPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::shop-api-assets-<ACCOUNT_ID>",
      "Condition": { "StringLike": { "s3:prefix": ["uploads/*"] } }
    }
  ]
}
```

`Effect`, `Action`, `Resource` and optional `Condition` are the four moving parts. Note that object actions use the object ARN (`bucket/key`) while `s3:ListBucket` uses the bucket ARN, a common source of confusing `AccessDenied` errors.

Evaluation rules to memorise

1. **Default deny.** If nothing allows it, it is denied.
2. **Explicit deny always wins**, over any number of allows.
3. **Same account:** an allow in the identity policy *or* in the resource policy is enough (unless a boundary/SCP blocks it).
4. **Cross account:** both sides must allow: the caller's identity policy **and** the target's resource policy (or trusting role).
5. SCPs, permission boundaries and session policies only *restrict*; they never grant.

Roles in practice

- A role has a **trust policy** ("who can assume me") and **permission policies** ("what I can do"). Both must be right: a perfect permission policy with the wrong trust policy fails at `AssumeRole`.
- **Instance profile:** the container that passes a role to EC2. The console creates one automatically; the CLI needs `create-instance-profile` and `add-role-to-instance-profile`.
- **STS** operations: `AssumeRole` (cross-account, humans, chaining), `AssumeRoleWithWebIdentity` (OIDC such as GitHub or EKS), `GetCallerIdentity` (who am I?). Default role session lasts one hour; credentials always carry an expiry and a session token.
- **Cross-account to third parties:** use an `ExternalId` condition to prevent the "confused deputy" problem.

Tools: `aws sts get-caller-identity` (who am I), IAM **policy simulator** (`aws iam simulate-principal-policy`), **IAM Access Analyzer** (finds public/cross-account access and can generate a least-privilege policy from CloudTrail §6.17 activity), and **last accessed information** (which services and actions a role actually used) to prune permissions.

6 Step-by-step guide

Prerequisites: AWS CLI v2 authenticated as an admin-level identity in a **sandbox account** (prefer IAM Identity Center, `aws configure sso`; do not use root keys). **Cost:** IAM, STS and the simulator are free. Nothing in this guide creates billable resources. **LocalStack:** the IAM and STS APIs work for practice with `--endpoint-url http://localhost:4566`, but policy *enforcement* is not part of the free edition, so use real AWS (still free) to see `AccessDenied` for real.

1. **See who you are and your account ID.**

```bash
export AWS_DEFAULT_REGION=us-east-1
aws sts get-caller-identity
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
```

   Output: `UserId`, `Account` (12 digits) and `Arn` (for SSO this is an `assumed-role/...` ARN).

2. **Write the permission policy** shown above to a file (replace the placeholder with your account ID):

```bash
cat > shop-api-s3.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid":"ReadWriteUploads","Effect":"Allow","Action":
  ["s3:GetObject","s3:PutObject"],
     "Resource":"arn:aws:s3:::shop-api-assets-${ACCOUNT_ID}/uploads/*"},
    {"Sid":"ListUploadsPrefix","Effect":"Allow","Action":"s3:ListBucket",
     "Resource":"arn:aws:s3:::shop-api-assets-${ACCOUNT_ID}",
     "Condition":{"StringLike":{"s3:prefix":["uploads/*"]}}}
  ]
}
EOF
```

3. **Create the role with an EC2 trust policy, attach the policy, and make an instance profile.**

```bash
cat > trust-ec2.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name shop-api-app --assume-role-policy-document
  file://trust-ec2.json
aws iam put-role-policy --role-name shop-api-app --policy-name s3-uploads --policy-
  document file://shop-api-s3.json
aws iam create-instance-profile --instance-profile-name shop-api-app
aws iam add-role-to-instance-profile --instance-profile-name shop-api-app --role-name
  shop-api-app
```

You can now pass `--iam-instance-profile Name=shop-api-app` to `aws ec2 run-instances` (EC2 §6.1).

4. **Test the permissions without running anything** using the policy simulator:

```bash
ROLE_ARN=arn:aws:iam::${ACCOUNT_ID}:role/shop-api-app
BUCKET=shop-api-assets-${ACCOUNT_ID}
aws iam simulate-principal-policy --policy-source-arn "$ROLE_ARN" \
  --action-names s3:GetObject s3:PutObject s3:DeleteObject \
  --resource-arns "arn:aws:s3:::${BUCKET}/uploads/a.png" \
  --query 'EvaluationResults[].[EvalActionName,EvalDecision]' --output table
```

   Expected: `s3:GetObject allowed`, `s3:PutObject allowed`, `s3:DeleteObject implicitDeny`. Repeat with the resource `arn:aws:s3:::${BUCKET}/private/a.png`; all three are `implicitDeny` because the path is outside `uploads/`. (The simulator may take a few seconds after role creation because IAM changes are eventually consistent.)

5. **Practice assuming a role and using temporary credentials.** Create a second role that your own account can assume:

```bash
cat > trust-self.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"AWS":"arn:aws:iam::${ACCOUNT_ID}:root"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name lab-readonly --assume-role-policy-document
  file://trust-self.json
aws iam attach-role-policy --role-name lab-readonly --policy-arn
  arn:aws:iam::aws:policy/ReadOnlyAccess
sleep 10
aws sts assume-role --role-arn arn:aws:iam::${ACCOUNT_ID}:role/lab-readonly \
  --role-session-name lab --query Credentials --output json
```

   The output has `AccessKeyId` starting `ASIA`, `SecretAccessKey`, `SessionToken` and `Expiration` about an hour ahead. Trusting `:root` of the account means any principal in the account **that also has** `sts:AssumeRole` **permission** may assume the role; that is why it is a lab shortcut, not a production pattern.

6. **Use the credentials, then try something forbidden.**

```bash
eval "$(aws sts assume-role --role-arn arn:aws:iam::${ACCOUNT_ID}:role/lab-readonly --
  role-session-name lab \
  --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' --output text |
  awk '{printf "export AWS_ACCESS_KEY_ID=%s AWS_SECRET_ACCESS_KEY=%s
  AWS_SESSION_TOKEN=%s", $1,$2,$3}')"
aws sts get-caller-identity                    # Arn now shows assumed-role/lab-
  readonly/lab
aws iam create-user --user-name should-fail    # AccessDenied: ReadOnlyAccess does
  not allow writes
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN
```

   The error reads similar to `User: arn:aws:sts::...:assumed-role/lab-readonly/lab is not authorized to perform: iam:CreateUser ...`. Read it carefully: it names the principal, the action and the resource, which is everything you need to fix it.

7. **See an explicit deny beat an allow.** Add a deny and re-run the simulator from step 4:

```bash
aws iam put-role-policy --role-name shop-api-app --policy-name deny-put --policy-
  document \
'{"Version":"2012-10-17","Statement":
  [{"Effect":"Deny","Action":"s3:PutObject","Resource":"*"}]}'
```

   `s3:PutObject` now shows `explicitDeny` even though step 3 allowed it. Remove it with `aws iam delete-role-policy --role-name shop-api-app --policy-name deny-put`.

8. **Clean up.**

```bash
aws iam remove-role-from-instance-profile --instance-profile-name shop-api-app --role-
  name shop-api-app
aws iam delete-instance-profile --instance-profile-name shop-api-app
aws iam delete-role-policy --role-name shop-api-app --policy-name s3-uploads
aws iam delete-role --role-name shop-api-app
aws iam detach-role-policy --role-name lab-readonly --policy-arn
  arn:aws:iam::aws:policy/ReadOnlyAccess
aws iam delete-role --role-name lab-readonly
```

   If you want to keep `shop-api-app` for the S3 and EC2 labs (S3 §6.8), skip its deletion; it is free.

7 Real-world scenario

Illustrative scenario, not a documented incident. A developer pushes a `.env` file containing a `shop-api` IAM user's access key to a public repository. Within minutes the account's bill alarm fires: dozens of large GPU instances are running in regions the company has never used. The key belonged to a user with `AdministratorAccess` because "it was quicker".

Response: deactivate the key immediately (`aws iam update-access-key --status Inactive`), review CloudTrail §6.17 for what the key did, terminate the instances, delete anything the attacker created, and rotate everything it could read. **Prevention:** no IAM users for workloads; the app runs with an instance role limited to its bucket prefix; CI uses OIDC roles; an SCP denies all regions except `us-east-1` and `eu-west-1`; secret scanning (Level 11 §11.11) blocks the next accidental commit. **Lesson:** the blast radius of a leaked credential is exactly the permission set attached to it, so the work is making that set tiny and short-lived.

8 Common mistakes and best practices

Don't

- Use the root user for daily work, or create root access keys.
- Create long-lived access keys for applications, or commit them anywhere.
- Attach `AdministratorAccess` or `"Action": "*", "Resource": "*"` "just to make it work".
- Forget that `Resource: "*"` with `iam:PassRole` or `iam:*` is a privilege-escalation path.
- Trust `:root` of another account or `"Principal": "*"` without strong conditions.

Do

- Enable MFA on root and every human; use IAM Identity Center for people.
- Use roles for every workload: instance profiles, task roles, OIDC for CI.
- Start from a narrow policy, iterate on `AccessDenied` messages, then confirm with Access Analyzer or last-accessed data.
- Add conditions (`aws:PrincipalOrgID`, `aws:SourceVpce`, `aws:RequestedRegion`, MFA required) where they fit.
- Put organisation-wide guardrails in SCPs and keep accounts separate (prod, staging, tooling).
- Manage IAM as code so every permission change is reviewed.

9 Tasks

- [ ] **T1 — Secure the account basics** *(≈30 min, beginner, free)*

- **Goal:** remove the standing risks before building.
- **Steps:**
  1. Confirm MFA is enabled on the root user and that no root access keys exist (**IAM > Security recommendations** or `aws iam get-account-summary`, field `AccountAccessKeysPresent` should be `0`).
  2. Confirm you work through an SSO/role session, not an IAM user key.
  3. Confirm a monthly budget alert exists (see EC2 step 1 §6.1); billing alarms are mandatory for this level.
- **Done when:** `AccountAccessKeysPresent` is `0` and you can name your budget.

☐ **T2 — Build the least-privilege app role** *(≈45 min, beginner, free)*

- **Goal:** an instance role scoped to one prefix of one bucket.
- **Steps:**
  1. Run guide steps 2-4.
  2. Use the simulator to prove four cases: read in `uploads/`, write in `uploads/`, read in `private/`, delete in `uploads/`.
- **Done when:** the table shows allowed, allowed, implicitDeny, implicitDeny respectively.

☐ **T3 — Assume a role and read the error** *(≈30 min, intermediate, free)*

- **Goal:** use temporary credentials and learn to read `AccessDenied`.
- **Steps:**
  1. Run guide steps 5-6.
  2. Copy the full `AccessDenied` message; underline the principal, action and resource.
  3. Fix it by creating a policy that allows exactly that action, retest, then remove it.
- **Done when:** the forbidden call succeeds only after you add one precise statement.

☐ **T4 — Break it: trust policy versus permission policy** *(≈30 min, intermediate)*

- **Goal:** distinguish the two failure points.
- **Steps:**
  1. Change `trust-self.json` so the principal is a non-existent account ID and update with `aws iam update-assume-role-policy`.
  2. Run `assume-role` and record the error.
  3. Fix the trust policy; then remove the `ReadOnlyAccess` attachment and show that assuming works but calls fail.

- **Done when:** you can state which error means "I may not become the role" and which means "the role may not do this".

- [ ] **T5 — Automate a policy lint and design the CI role** *(≈60 min, advanced)*
  - **Goal:** catch wildcards before they merge and design OIDC access.
  - **Steps:**
    1. Write `lint_policy.py` that loads a policy JSON file and exits 1 if any `Allow` statement has `Action` `*`, `service:*` or `Resource` `*`. Test on a good and a bad file.
    2. Draft (do not need to deploy) a role trust policy for GitHub OIDC with `Federated` set to `arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com`, action `sts:AssumeRoleWithWebIdentity`, and a `StringEquals` condition on `token.actions.githubusercontent.com:aud` of `sts.amazonaws.com` and a `StringLike` condition on `...:sub` of `repo:<ORG>/shop-api:ref:refs/heads/main`.
    3. Run the lint script in a GitHub Actions job (GitHub Actions for CI/CD §5.2).
  - **Done when:** the lint job fails on a wildcard policy and passes on a scoped one, and your trust policy limits the role to one repository and branch.

- [ ] **T6 — Prove it: explain a denied request** *(≈30 min, advanced)*
  - **Goal:** teach policy evaluation.
  - **Steps:**
    1. Without notes, draw the evaluation flow and give an example where an identity allow is overridden by (a) an explicit deny, (b) an SCP, (c) a permission boundary.
    2. Explain the difference between a role's trust policy and its permission policy and show where an instance profile fits.
  - **Done when:** a teammate can use your explanation to debug a made-up `AccessDenied`.

10 Self-check questions

1. What is the difference between a role's trust policy and its permission policy?
2. If an identity policy allows `s3:GetObject` and a bucket policy denies it, what is the result?
3. Why are role credentials safer than access keys?
4. A cross-account S3 read fails although the caller's role has `s3:GetObject`. What is probably missing?
5. What does an SCP do that an IAM policy cannot?

ANSWERS

1. The trust policy says who may assume the role; the permission policy says what the role can do once assumed.
2. Denied. An explicit deny in any applicable policy overrides every allow.
3. They are temporary (they expire, typically within an hour), are issued automatically by STS and are never stored in code or files.
4. The bucket's resource policy (or the target-account role) must also allow the caller; cross-account access needs both sides.
5. It sets a maximum permission boundary for every identity in an account or OU, including the account's admins, without granting anything itself.

11 Further reading

- AWS documentation: *IAM User Guide* (policy evaluation logic, roles, STS, policy reference).
- AWS documentation: *IAM Access Analyzer* and *Security best practices in IAM*.
- AWS documentation: *AWS IAM Identity Center User Guide*.
- GitHub documentation: *Configuring OpenID Connect in Amazon Web Services*.
- *AWS Well-Architected Framework*, Security pillar (identity and access management).

LEVEL 6: CLOUD (AWS)

6.8 S3

Topic 8 of 18 | ~6 hours | Difficulty: Intermediate

1 What it is

Amazon S3 (Simple Storage Service) is AWS's **object storage**. You create a **bucket** (a container with a globally unique name that lives in one region) and put **objects** in it: a blob of bytes up to 5 TB plus metadata, addressed by a **key** such as `uploads/2026/10/avatar.png`. There are no real directories (the `/` in a key is just a character that tools display as folders), no file locking, no in-place edits: you write whole objects, read them back over HTTPS, overwrite them or delete them.

S3 is designed for 99.999999999% (eleven nines) durability by storing data redundantly across multiple Availability Zones in the region (except the One Zone classes), and offers strong read-after-write consistency: after a successful write, any read sees the new data. You pay for the gigabytes stored, the requests made and the data transferred out. S3 is the default place for backups, build artifacts, logs, static websites, user uploads, data lakes and Terraform state.

2 Why it matters in DevOps

- **It is everywhere in the platform:** CloudTrail and ALB logs, CloudFront origins, Lambda packages, AMI snapshots' cousins, Terraform remote state (State and Remote Backends §7.4), database backups (Backups §15.1) and container image layers.
- **Security:** misconfigured public buckets have been behind many publicly reported data leaks. Block Public Access, bucket policies and encryption are table stakes.
- **Cost:** storage is cheap but forgotten data, old versions, incomplete multipart uploads, request-heavy workloads and internet egress add up. Lifecycle rules are the cost control (Storage Lifecycle Policies §20.6).
- **Reliability:** versioning and replication turn "someone ran `rm -r`" from a disaster into a one-command restore.

3 Real-world system example

Terraform remote state. The standard AWS backend for Terraform is an S3 bucket with versioning enabled, so every state change is recoverable. Recent Terraform versions can lock state with a lock file stored in the same bucket (the S3 backend's `use_lockfile` setting, from Terraform 1.10), replacing the older approach of a separate DynamoDB table (State Locking §7.8).

AWS itself eats S3: CloudTrail delivers audit logs to a bucket, load balancers write access logs to one, and ECR keeps image layers in S3-backed storage, which is why a private cluster needs an S3 endpoint to pull images (Route Tables, IGW and NAT §6.5). Dropbox has publicly written that it originally stored file data on S3 before building its own storage system, an example of how far a single service can carry a product.

For `shop-api`, product images live in `s3://shop-api-assets-<ACCOUNT_ID>/uploads/`, served through CloudFront (CloudFront §6.13) and uploaded by browsers directly via presigned URLs.

4 Diagram

Direct browser upload using a presigned URL, so file bytes never pass through the API servers:

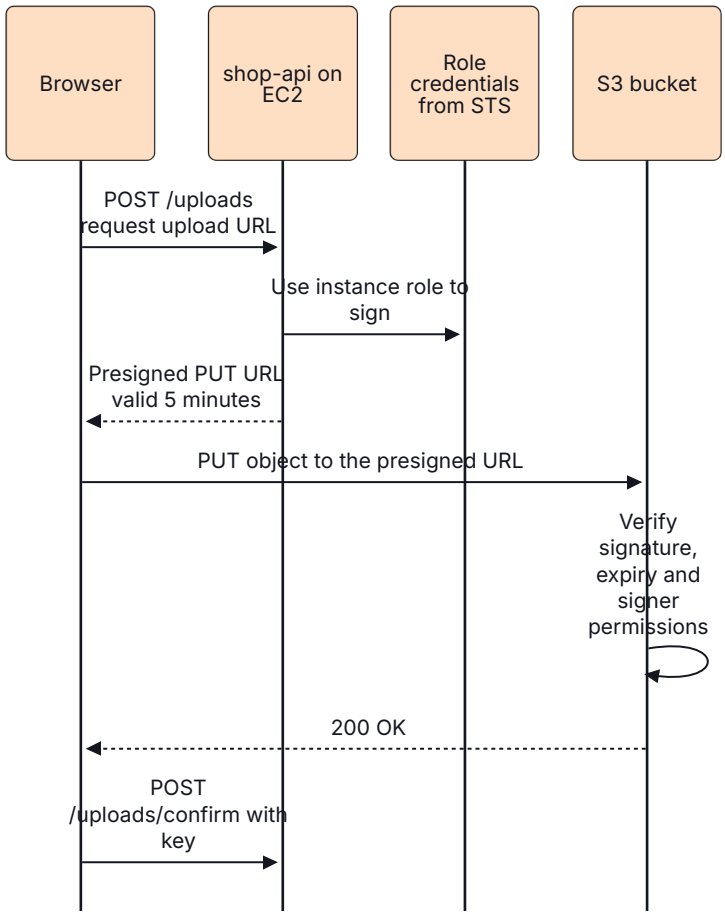


Figure 6.8-1

Lifecycle of storage cost for objects under `logs/`:

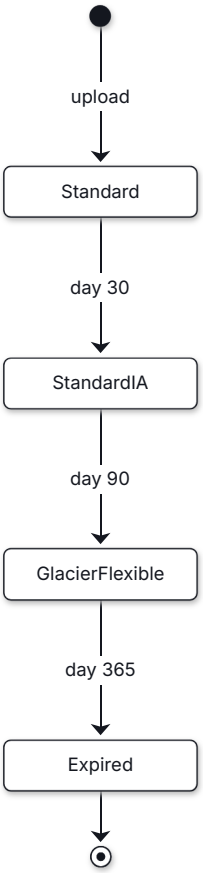


Figure 6.8-2

5 How it works

Core concepts

Concept	What to remember
Bucket	Name is globally unique across all AWS accounts (3-63 chars, lowercase). Region-specific; data never leaves the region unless you replicate it
Object	Key + data + metadata (+ tags, + version ID). Max 5 TB; one `PUT` up to 5 GB; use **multipart upload** above ~100 MB (the CLI does it automatically)
Consistency	Strongly consistent for reads after writes and for listings
Block Public Access	Four switches at account and bucket level. **On by default for new buckets**, along with ACLs disabled ("bucket owner enforced")

Concept	What to remember
Access control	IAM policies, **bucket policies** (resource-based, can grant other accounts or enforce rules such as TLS only), and legacy ACLs (avoid)
Versioning	Keeps every version. A delete adds a **delete marker**; removing the marker restores the object. Cannot be turned off, only suspended
Lifecycle rules	Automatically transition objects between storage classes, expire current and non-current versions, and abort incomplete multipart uploads
Encryption at rest	All new objects are encrypted automatically with **SSE-S3** (AES-256, AWS-managed keys). Upgrade to **SSE-KMS** when you need key policies and audit trails (KMS §6.15). Always encrypted in transit with HTTPS
Presigned URL	A URL carrying a signature that grants time-limited access to one object operation with the signer's permissions. Valid up to 7 days when signed by an IAM user, but never longer than the temporary credentials of a role or SSO session that signed it
Static website hosting	Serves an index page via a **website endpoint** that supports HTTP only and requires public content. For HTTPS and a private bucket, put CloudFront in front with Origin Access Control

Storage classes (pick by access pattern)

Class	Use for	Notes
Standard	Frequently accessed data	Default
Intelligent-Tiering	Unknown or changing access	Moves objects between tiers automatically for a small monitoring fee
Standard-IA / One Zone-IA	Infrequent access, still millisecond	Cheaper storage, per-GB retrieval fee, minimum duration and size charges. One Zone keeps data in a single AZ
Glacier Instant / Flexible / Deep Archive	Archives, compliance, backups	Cheapest storage; Flexible takes minutes to hours to retrieve and Deep Archive up to about half a day

Approximate cost shape (US East, at time of writing; verify on the pricing page): Standard storage is a few cents per GB-month (about $0.023); requests are fractions of a cent per thousand; data in is free; data out to the internet costs roughly $0.09 per GB after a monthly free allowance; transfer to CloudFront is free. New accounts get a limited S3 allowance in the free tier, and rules for free-tier accounts changed in 2025, so check **Billing > Free Tier** to see yours.

6 Step-by-step guide

Prerequisites: AWS CLI v2, a sandbox account, region `us-east-1`. **Cost:** a handful of tiny objects for an hour costs effectively nothing, but the cleanup in step 9 is mandatory because versioned buckets cannot be deleted while any version or delete marker exists. **LocalStack**

can do everything except real HTTPS/CloudFront behaviour: `docker run -d --rm -p 4566:4566 localstack/localstack` and add `--endpoint-url http://localhost:4566` (any credentials).

1. **Create a private bucket and inspect the secure defaults.**

```bash
export AWS_DEFAULT_REGION=us-east-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
B=shop-api-lab-${ACCOUNT_ID}
aws s3api create-bucket --bucket "$B"          # in other regions add: --create-
  bucket-configuration LocationConstraint=<region>
aws s3api get-public-access-block --bucket "$B"
aws s3api get-bucket-encryption --bucket "$B"
```

   Expect all four Block Public Access flags `true`, and an encryption rule with `SSEAlgorithm` `AES256` (SSE-S3).

2. **Upload, download and list.**

```bash
echo "hello s3 v1" > hello.txt
aws s3 cp hello.txt "s3://$B/uploads/hello.txt"
aws s3 ls "s3://$B/uploads/"
aws s3 cp "s3://$B/uploads/hello.txt" -
```

   The last command prints `hello s3 v1`. Try `aws s3api head-object --bucket "$B" --key uploads/hello.txt` and note `ServerSideEncryption`.

3. **Turn on versioning and see a delete marker.**

```bash
aws s3api put-bucket-versioning --bucket "$B" --versioning-configuration
  Status=Enabled
echo "hello s3 v2" > hello.txt && aws s3 cp hello.txt "s3://$B/uploads/hello.txt"
aws s3 rm "s3://$B/uploads/hello.txt"
aws s3api list-object-versions --bucket "$B" --prefix uploads/hello.txt \
  --query '{Versions:Versions[].[VersionId,IsLatest],Markers:DeleteMarkers[].
  [VersionId,IsLatest]}'
```

The object has one version from before versioning (`VersionId` `null`), one new version, and a delete marker that is the latest. `aws s3 ls` no longer shows the key. Restore it by deleting the marker:

```bash
MARK=$(aws s3api list-object-versions --bucket "$B" --prefix uploads/hello.txt --query
  'DeleteMarkers[0].VersionId' --output text)
aws s3api delete-object --bucket "$B" --key uploads/hello.txt --version-id "$MARK"
aws s3 cp "s3://$B/uploads/hello.txt" -        # prints hello s3 v2 again
```

4. **Enforce TLS with a bucket policy.**

```bash
cat > policy.json <<EOF
{"Version":"2012-10-17","Statement":
  [{"Sid":"DenyInsecureTransport","Effect":"Deny","Principal":"*",
  "Action":"s3:*","Resource":["arn:aws:s3:::${B}","arn:aws:s3:::${B}/*"],
  "Condition":{"Bool":{"aws:SecureTransport":"false"}}}]}
EOF
aws s3api put-bucket-policy --bucket "$B" --policy file://policy.json
```

The CLI uses HTTPS, so it keeps working; a plain `http://` request would be denied. This is a Deny, so it overrides any allow elsewhere (see IAM §6.7).

5. **Add a lifecycle rule** (tiering, expiry, cleanup of old versions and abandoned multipart uploads):

```bash
cat > lifecycle.json <<'EOF'
{"Rules":[{"ID":"logs-tiering","Status":"Enabled","Filter":{"Prefix":"logs/"},
  "Transitions":[{"Days":30,"StorageClass":"STANDARD_IA"},
  {"Days":90,"StorageClass":"GLACIER"}],
  "Expiration":{"Days":365},
  "NoncurrentVersionExpiration":{"NoncurrentDays":30},
  "AbortIncompleteMultipartUpload":{"DaysAfterInitiation":7}}]}
EOF
aws s3api put-bucket-lifecycle-configuration --bucket "$B" --lifecycle-configuration
  file://lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket "$B" --query 'Rules[].ID'
```

Lifecycle actions run asynchronously, typically at midnight UTC, so you will not see transitions in this session.

6. **Generate presigned URLs.** For reading (CLI):

```bash
URL=$(aws s3 presign "s3://$B/uploads/hello.txt" --expires-in 60)
curl -s "$URL"                  # prints the object even though the bucket is private
sleep 65; curl -s "$URL" | head -3
```

The second request returns an XML `AccessDenied` error with message `Request has expired`. For uploading, sign a `PUT` with an SDK (the CLI only presigns downloads). Install `pip install boto3` and run:

```bash
python3 - <<EOF
import boto3
s3 = boto3.client("s3", region_name="us-east-1")
url = s3.generate_presigned_url("put_object",
      Params={"Bucket": "$B", "Key": "uploads/from-browser.txt", "ContentType":
  "text/plain"}, ExpiresIn=300)
print(url)
EOF
```

Then upload with `curl -X PUT -H "Content-Type: text/plain" --data "uploaded via presigned URL" "<that URL>"`. The `Content-Type` must match what was signed. Check with `aws s3 ls "s3://$B/uploads/"`.

7. **Host a static website (sandbox only).** Use a separate bucket, because this one must be public:

```bash
W=shop-api-site-${ACCOUNT_ID}
aws s3api create-bucket --bucket "$W"
aws s3api put-public-access-block --bucket "$W" --public-access-block-configuration \

  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=false,RestrictPublicBuc
  kets=false
aws s3api put-bucket-website --bucket "$W" --website-configuration '{"IndexDocument":
  {"Suffix":"index.html"}}'
aws s3api put-bucket-policy --bucket "$W" --policy "{\"Version\":\"2012-10-
  17\",\"Statement\":
  [{\"Sid\":\"PublicRead\",\"Effect\":\"Allow\",\"Principal\":\"*\",\"Action\":\"s3:Ge
  tObject\",\"Resource\":\"arn:aws:s3:::${W}/*\"}]}"
echo '<h1>shop-api status: ok</h1>' > index.html
aws s3 cp index.html "s3://$W/index.html" --content-type text/html
curl -s "http://${W}.s3-website-us-east-1.amazonaws.com/"
```

You should see the `<h1>` line. Note this endpoint speaks HTTP only. In production, keep the bucket **private**, put CloudFront §6.13 in front with Origin Access Control for HTTPS, caching and a custom domain (Route 53 §6.12).

8. **(Optional) Inspect request cost drivers.** Count what you did: a few PUT/GET/LIST requests. At a few tenths of a cent per thousand requests, this lab costs a tiny fraction of a cent; a service doing millions of tiny GETs per day is where request charges matter.
9. **Clean up.** Versioned buckets must be emptied of all versions and delete markers first:

```bash
empty() {
  for key in Versions DeleteMarkers; do
    aws s3api list-object-versions --bucket "$1" --query "${key}[].
  {Key:Key,VersionId:VersionId}" --output json |
    jq -c 'if . == null then empty else . end | . as $a | range(0; length; 1000) |
  {Objects: $a[.:.+1000], Quiet: true}' |
    while read -r batch; do aws s3api delete-objects --bucket "$1" --delete "$batch"
  >/dev/null; done
  done
}
empty "$B"; aws s3api delete-bucket --bucket "$B"
aws s3 rm "s3://$W" --recursive; aws s3api delete-bucket --bucket "$W"
aws s3api list-buckets --query 'Buckets[].Name'
```

(`jq` is required; the helper sends batches of 1,000, the API limit. Alternatively use the console's **Empty** button.) The list at the end must not contain either lab bucket.

7 Real-world scenario

Illustrative scenario, not a documented incident. A `shop-api` engineer runs a cleanup script with the wrong prefix and deletes the `uploads/` folder of 40,000 product images. Customers see broken images within minutes. Because versioning was enabled when the bucket was created, nothing is actually gone: each object has a delete marker on top. The team lists delete markers under the prefix and removes them in a batch; images return as the CloudFront cache is refreshed. The whole recovery takes under an hour.

Follow-ups: MFA Delete and a restrictive bucket policy for bulk deletes, S3 Cross-Region Replication to a second region for the "region disappeared" case (Disaster Recovery §0.9), a `NoncurrentVersionExpiration` rule to keep the extra versions from growing forever, and an alarm on deletion-request metrics. **Lesson:** versioning is cheap insurance; turn it on before you need it.

8 Common mistakes and best practices

Don't

- Make a bucket public to "quickly share" a file; use a presigned URL or CloudFront.
- Put secrets, `.env` files or database dumps in a bucket with broad read access.
- Use S3 like a filesystem for small, frequently updated files (no atomic rename, no locking).
- Leave versioned buckets without a non-current-version expiry rule (silent cost growth).
- Forget that bucket names are public: do not embed secrets or customer names in them.

Do

- Keep **Block Public Access** on at the account level and open exceptions deliberately.
- Enable **versioning** on anything you cannot regenerate; add lifecycle rules for tiering, expiry and multipart cleanup.
- Use bucket policies for guardrails (TLS only, specific VPC endpoint, organisation only) and IAM roles for access, never access keys in code.
- Use SSE-KMS where you need key-level audit or separation, and a bucket key to reduce KMS request cost.
- Enable access logging or CloudTrail data events for sensitive buckets (CloudTrail §6.17).
- Prefer CloudFront with Origin Access Control to the S3 website endpoint.

9 Tasks

- [ ] **T1 — Create a private, encrypted bucket and prove the defaults** *(≈20 min, beginner, free)*
  - **Goal:** see Block Public Access and default encryption.
  - **Steps:**
    1. Run guide step 1 and step 2.
    2. Try `curl -I https://$B.s3.amazonaws.com/uploads/hello.txt` and note the status.
  - **Done when:** Block Public Access shows four `true` values, `head-object` reports `ServerSideEncryption`, and the anonymous `curl` returns 403.
- [ ] **T2 — Delete and recover with versioning** *(≈30 min, beginner)*
  - **Goal:** recover an object after deletion.
  - **Steps:**
    1. Run guide step 3.

2. Now overwrite the object three times; use `list-object-versions` to download version 2 by `--version-id` with `aws s3api get-object`.

- **Done when:** you retrieved the content of an older version and restored a deleted object.

☐ **T3 — Presign, expire, and break it** *(≈30 min, intermediate)*

- **Goal:** understand what a presigned URL actually grants.
- **Steps:**
  1. Generate a 60-second download URL and a 300-second upload URL (guide step 6).
  2. Test them before and after expiry.
  3. Change the `Content-Type` in the upload request and observe the `SignatureDoesNotMatch` error.
  4. Use `aws sts get-caller-identity` to note who signed; then explain why a URL signed by a role session dies when the session does.
- **Done when:** you have seen both an `AccessDenied` (expired) and a `SignatureDoesNotMatch` and can explain each.

☐ **T4 — Lifecycle and cost plan** *(≈40 min, intermediate)*

- **Goal:** turn an access pattern into a rule set.
- **Steps:**
  1. Take `shop-api` logs (written daily, read for 7 days, kept for a year for audit) and write a lifecycle JSON using the right transitions and expiry.
  2. Apply it to the lab bucket under prefix `logs/` and read it back.
  3. Estimate monthly storage cost for 500 GB/month ingestion using the pricing page and compare Standard-only versus tiered.
- **Done when:** `get-bucket-lifecycle-configuration` returns your rule and you have a two-column cost comparison with assumptions.

☐ **T5 — Automate: private site with CloudFront or a TLS-only audit** *(≈60 min, advanced)*

- **Goal:** secure delivery as code.
- **Steps:**
  1. Write a script that loops over all your buckets and reports any without Block Public Access, default encryption or a TLS-deny policy.
  2. Or deploy the static site behind CloudFront with Origin Access Control after finishing CloudFront §6.13, with the bucket fully private.

- **Done when:** the audit lists your lab bucket as compliant (or a `curl https://<cloudfront-domain>/` returns the page while the direct S3 URL returns 403).

☐ **T6 — Prove it: design an upload flow** *(≈30 min, advanced)*

- **Goal:** justify an S3 design.
- **Steps:**
  1. Without notes, sketch how `shop-api` should accept 5 MB image uploads and serve them globally, naming bucket settings, IAM, presigned URLs, lifecycle and the CDN.
  2. State what happens when the user deletes an image and what recovers it.
- **Done when:** your design has no public bucket, no long-lived keys and a recovery story, and a teammate agrees.

10 Self-check questions

1. Why is S3 not a filesystem? Name two things you cannot do.
2. A user deletes an object in a versioned bucket. Is the data gone? How do you bring it back?
3. What does a presigned URL grant, and what limits its lifetime?
4. Why prefer CloudFront with OAC over the S3 website endpoint?
5. A bucket cannot be deleted. Name two likely reasons.

ANSWERS

1. No real directories, no append or in-place edits, no file locking, no atomic rename; objects are written whole.
2. Not gone: a delete marker hides it. Delete the marker (or fetch/copy a specific version) to restore.
3. Time-limited access to one operation on one object, with the signer's permissions; it expires at its set time or when the signer's temporary credentials expire, whichever is earlier.
4. The website endpoint is HTTP only and needs a public bucket; CloudFront gives HTTPS, caching, custom domains and keeps the bucket private.
5. It still contains objects, old versions or delete markers (versioning), or incomplete multipart uploads/replication settings still reference it.

11 Further reading

- AWS documentation: *Amazon S3 User Guide* (buckets, versioning, lifecycle, bucket policies, presigned URLs, storage classes).
- AWS pricing page: *Amazon S3 pricing* for current numbers.
- AWS documentation: *Security best practices for Amazon S3*.
- Terraform documentation: *S3 backend* (state storage and locking).

LAB NOTES What I ran, what I saw, what it means

6.9 EBS and EFS

Topic 9 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

Amazon EBS (Elastic Block Store) gives an EC2 instance a virtual hard disk. It is **block storage**: the instance sees a raw device (such as `/dev/nvme1n1`), puts a filesystem like `xfs` or `ext4` on it and mounts it. An EBS volume is network-attached, lives in **one Availability Zone**, survives instance stop and (for non-root volumes) termination, and is normally attached to **one instance at a time**. Your instance's root disk is an EBS volume too.

Amazon EFS (Elastic File System) is a managed **file storage** service: an NFS (Network File System, version 4.1) share that many Linux instances, containers or Lambda functions mount **at the same time**, across multiple AZs. It grows and shrinks automatically, so you never provision capacity. Where EBS is "a disk for one server", EFS is "a shared folder for a fleet". (Object storage, the third kind, is S3 §6.8.)

2 Why it matters in DevOps

- **Data lives or dies here.** Databases you run yourself, queues, Prometheus and Elasticsearch data all sit on EBS. A wrong volume type is a performance incident; a missing snapshot is a data-loss incident (Backups §15.1, Snapshots §15.3).
- **Cost:** unattached ("available") volumes, orphaned snapshots and over-provisioned IOPS silently bill every hour. `gp2` to `gp3` is one of the easiest savings in AWS.
- **Scaling:** autoscaled fleets cannot share an EBS volume; shared state needs EFS, S3 or a database (Auto Scaling §6.2).
- **Kubernetes:** PersistentVolumes on AWS are EBS (single-node, `ReadWriteOnce`) or EFS (shared, `ReadWriteMany`) (Volumes, PV and PVC §8.9).
- **Recovery time:** restoring from a snapshot has a "first read is slow" behaviour that decides whether your RTO is five minutes or an hour.

3 Real-world system example

Amazon RDS for PostgreSQL and MySQL store database files on EBS volumes managed for you: when you choose "gp3, 200 GB, 6000 IOPS" or "io2" in the RDS console, you are choosing an EBS volume type, and RDS snapshots are EBS snapshots under the hood (RDS and Aurora §6.10). Aurora instead uses its own distributed storage layer.

Kubernetes on EKS uses the AWS EBS CSI driver to turn a `PersistentVolumeClaim` into an EBS volume attached to whichever node the Pod lands on, and the AWS EFS CSI driver for volumes shared across Pods and nodes, such as a shared media directory. A WordPress fleet is the classic EFS use case: many web servers must see the same `wp-content/uploads`.

For `shop-api`, the PostgreSQL data lives on RDS (so EBS under the hood), Redis on ElastiCache, uploads on S3, and EBS appears as instance root volumes plus an optional data volume; EFS is only needed if several app servers must share files.

4 Diagram

Where the data lives:

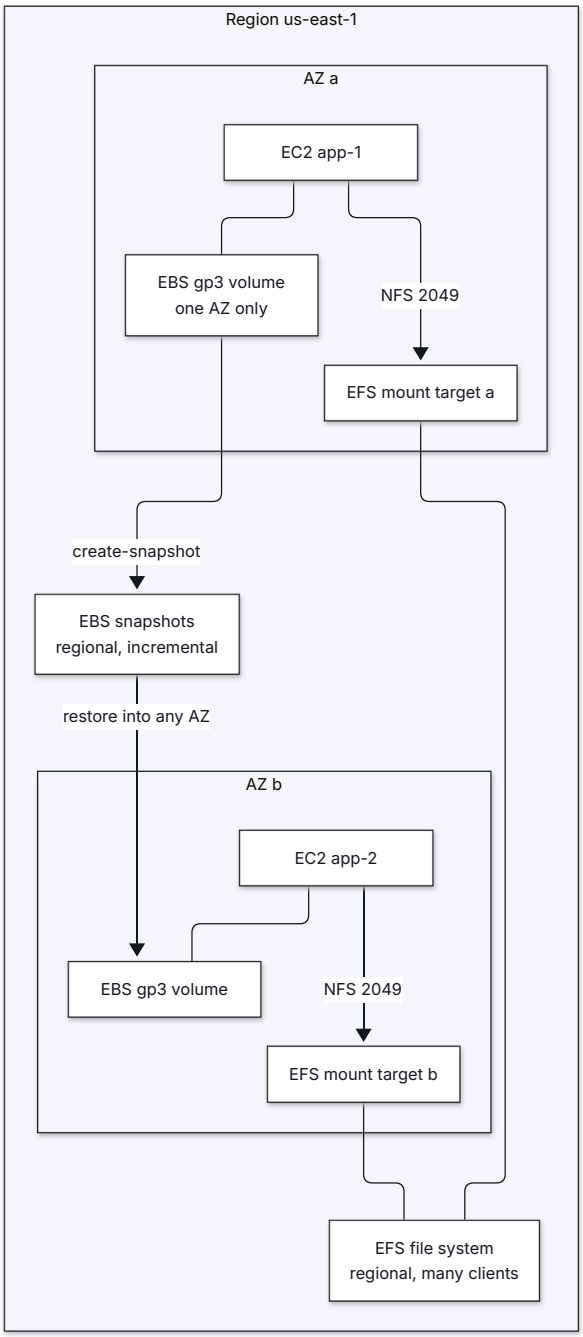

Region us-east-1
AZ a
EC2 app-1
EBS gp3 volume
one AZ only
NFS 2049
EFS mount target a
create-snapshot
EBS snapshots
regional, incremental
restore into any AZ
AZ b
EC2 app-2
EBS gp3 volume
NFS 2049
EFS mount target b
EFS file system
regional, many clients

Figure 6.9-1

Snapshots are incremental: the second snapshot stores only blocks changed since the first, yet each is independently restorable:

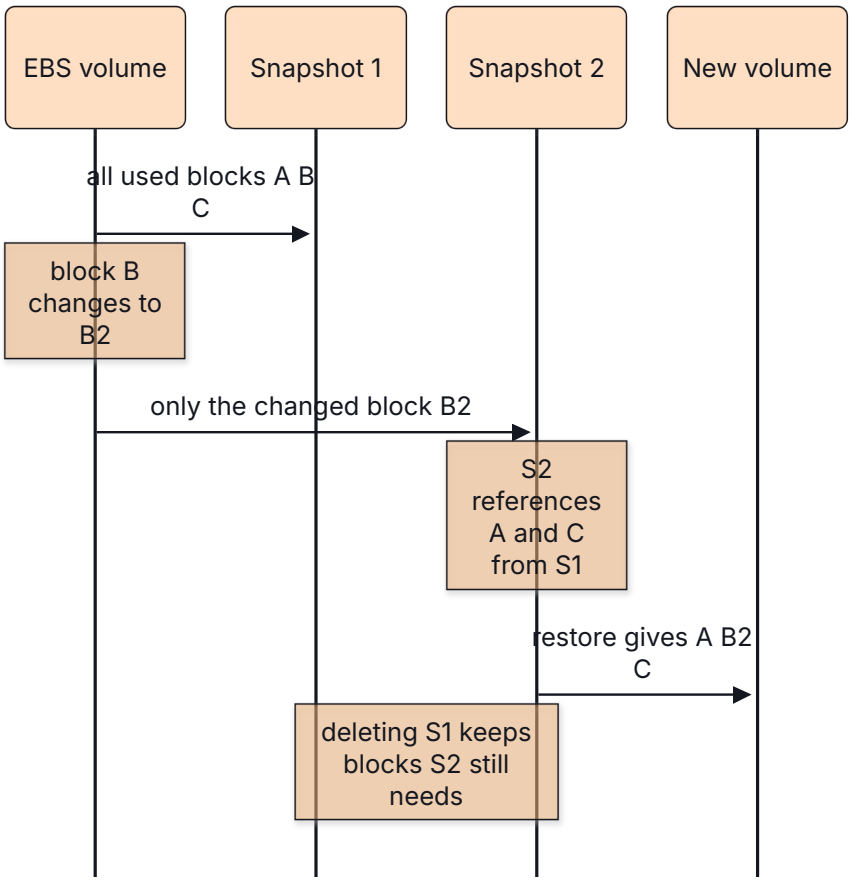


Figure 6.9-2

5 How it works

EBS volume types (check the documentation for current limits and prices):

Type	Medium	Best for	Notes
gp3	SSD	The default for almost everything: boot disks, app data, small and mid databases	Baseline **3,000 IOPS and 125 MiB/s included at any size**; raise IOPS and throughput independently of size at extra cost. About 20% cheaper per GB than gp2
gp2	SSD	Legacy	IOPS tied to size (3 per GB) with burst credits; migrate to gp3 with a live `modify-volume`

Type	Medium	Best for	Notes
io2 (Block Express)	SSD	Latency-sensitive, IO-heavy databases	Provisioned IOPS, highest durability class and performance; costs most; supports Multi-Attach to several Nitro instances in one AZ
io1	SSD	Older provisioned-IOPS	Prefer io2
st1	HDD	Large sequential throughput: logs, data warehouses, Kafka-like streams	Cannot be a boot volume
sc1	HDD	Cold, infrequently read data	Cheapest EBS; cannot be a boot volume

Remember the shape: **IOPS** (operations per second) matters for random reads and writes such as databases; **throughput** (MiB/s) matters for large sequential reads such as analytics. Instance types also cap EBS performance, so a fast volume on a small instance is bottlenecked by the instance (EC2 §6.1).

Other EBS facts

- **Snapshots** are incremental, stored durably by AWS (in S3-managed storage you cannot browse), regional, copyable across regions and accounts, and by default **crash-consistent** (like pulling the power, so freeze or stop writes for application-consistent copies of databases). Automate them with **Amazon Data Lifecycle Manager (DLM)** or AWS Backup. Snapshot storage is billed per GB-month for the stored blocks.
- **Restore is lazy:** a volume created from a snapshot is usable immediately, but blocks are fetched on first read, so the first pass is slow. **Fast Snapshot Restore** or pre-reading with `fio` / `dd` avoids this for time-critical restores.
- **Elastic Volumes:** change size, type, IOPS and throughput while attached. Sizes only grow, never shrink, and after a change you must also grow the partition and filesystem. There is a cooling-off period of about six hours before the next modification.
- **Encryption:** AES-256 with KMS §6.15 keys, no meaningful performance penalty on current instances. Turn on **EBS encryption by default** per region so every new volume and snapshot-copy is encrypted.
- **DeleteOnTermination:** true for the root volume, false for extra volumes by default. Check it for any volume holding data.
- **Instance store** (physical disks on the host in `i` / `d` instance types) is faster but **ephemeral**: it is lost when the instance stops or fails.
- **Multi-AZ:** an EBS volume is not. For cross-AZ protection use snapshots, replication at the application layer, or EFS/S3/RDS.

EFS essentials

Item	Detail
Protocol	NFSv4.1 over TCP **2049**; Linux only (use FSx for Windows or FSx for NetApp ONTAP for Windows and multi-protocol needs)
Availability	A regional file system stores data across multiple AZs; you create one **mount target** (an ENI with an IP) per AZ. **One Zone** file systems are cheaper and live in one AZ
Access control	Security group on the mount target (allow 2049 from your app SG, see Security Groups §6.6), IAM and **access points** (per-application root directory and POSIX identity)
Throughput	**Elastic** (default; scales automatically, pay per use), Provisioned, or Bursting. Performance mode: General Purpose for almost everything
Storage classes	Standard, Infrequent Access and Archive, moved by lifecycle policy; One Zone variants exist
Cost shape	Roughly ten times the per-GB price of EBS gp3 for Standard storage (about $0.30 per GB-month in US East at time of writing), plus elastic throughput charges; IA is far cheaper
Encryption	At rest with KMS (set `--encrypted` on creation) and in transit with the mount helper's `tls` option

EBS vs EFS vs S3 vs instance store

	EBS	EFS	S3	Instance store
Type	Block	File (NFS)	Object (HTTP API)	Block (local)
Shared by many clients	No (Multi-Attach is niche)	**Yes**	**Yes**	No
Scope	One AZ	Region (multi-AZ)	Region	One host
Latency	Lowest of the networked options	Low (ms)	Higher (tens of ms)	Lowest
Capacity	Provisioned, grow only	Elastic	Unlimited	Fixed by instance
Typical use	Databases, boot disks	Shared content, CMS, ML datasets, home dirs	Uploads, backups, logs, static files	Caches, scratch

6 Step-by-step guide

Prerequisites: AWS CLI v2, region `us-east-1`, a running `t3.micro` Amazon Linux 2023 instance in a subnet with the AZ you will use (create one following EC2 §6.1; Session Manager or SSH both work). Set:

```bash
export AWS_DEFAULT_REGION=us-east-1
INSTANCE_ID=i-0123456789abcdef0        # replace with yours
AZ=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --query
  'Reservations[0].Instances[0].Placement.AvailabilityZone' --output text)
```

Cost: an 8 GiB gp3 volume is under a dollar per month (cents for this lab), snapshots about $0.05 per GB-month, EFS a few cents for tiny data, and the `t3.micro` roughly a cent an hour. **Run the cleanup in step 9.** LocalStack mocks the EBS volume and snapshot APIs only (no real block device, so you cannot format or mount); EFS needs LocalStack's paid tier.

1. **Create and attach a gp3 volume** in the same AZ as the instance:

   ```bash
   VOL=$(aws ec2 create-volume --availability-zone "$AZ" --size 8 --volume-type gp3 --
     encrypted \
     --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=shop-api-data}]' --
     query VolumeId --output text)
   aws ec2 wait volume-available --volume-ids "$VOL"
   aws ec2 attach-volume --volume-id "$VOL" --instance-id "$INSTANCE_ID" --device
     /dev/sdf
   ```

   The state goes `creating` then `available`, then `in-use` once attached.

2. **On the instance: find, format and mount it.** On Nitro instances the device appears as an NVMe name, not `/dev/sdf`:

   ```bash
   lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,SERIAL
   ```

   Similar to: `nvme0n1 8G disk` (root, with partitions) and `nvme1n1 8G disk` with serial `vol0123456789abcdef` (your volume ID without the dash). Use that new disk:

   ```bash
   sudo mkfs -t xfs /dev/nvme1n1
   sudo mkdir -p /data && sudo mount /dev/nvme1n1 /data
   echo "snapshot me" | sudo tee /data/hello.txt
   df -h /data
   ```

Always confirm it is the empty disk (`lsblk`, no filesystem) before running `mkfs`; it erases whatever is there.

3. **Make the mount survive reboots, safely.** Use the UUID and `nofail` so a missing volume does not hang boot:

```bash
UUID=$(sudo blkid -s UUID -o value /dev/nvme1n1)
echo "UUID=$UUID /data xfs defaults,nofail 0 2" | sudo tee -a /etc/fstab
sudo umount /data && sudo mount -a && df -h /data
```

`mount -a` succeeding is your proof the fstab line is valid; a bad line can otherwise stop the next boot.

4. **Take a snapshot** (from your laptop):

```bash
SNAP=$(aws ec2 create-snapshot --volume-id "$VOL" --description "shop-api-data lab" \
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=shop-api-data-1}]' \
  --query SnapshotId --output text)
aws ec2 wait snapshot-completed --snapshot-ids "$SNAP"
aws ec2 describe-snapshots --snapshot-ids "$SNAP" --query 'Snapshots[0].
  [State,VolumeSize,Encrypted]'
```

Expect `completed`, `8`, `true`. The snapshot is crash-consistent; for a real database you would flush/freeze first.

5. **Restore into a new volume and mount the clone.** Restores can target any AZ in the region:

```bash
VOL2=$(aws ec2 create-volume --availability-zone "$AZ" --snapshot-id "$SNAP" --volume-
  type gp3 --query VolumeId --output text)
aws ec2 wait volume-available --volume-ids "$VOL2"
aws ec2 attach-volume --volume-id "$VOL2" --instance-id "$INSTANCE_ID" --device
  /dev/sdg
```

On the instance:

```bash
sudo mkdir -p /restore && sudo mount /dev/nvme2n1 /restore       # fails: duplicate
  XFS UUID
sudo mount -o nouuid /dev/nvme2n1 /restore && cat /restore/hello.txt
```

The first mount fails (similar to `wrong fs type, bad option, bad superblock` and a kernel message about duplicate UUID) because XFS refuses two filesystems with the same UUID; `-o nouuid` is the standard workaround for a cloned volume. `cat` prints `snapshot me`.

6. **Grow the live volume.**

```bash
aws ec2 modify-volume --volume-id "$VOL" --size 12
aws ec2 describe-volumes-modifications --volume-ids "$VOL" --query
  'VolumesModifications[0].ModificationState'
```

Wait for `optimizing` (the new size is usable then), and on the instance:

```bash
lsblk /dev/nvme1n1            # device now 12G, filesystem still 8G
sudo xfs_growfs -d /data      # for ext4 use: sudo resize2fs /dev/nvme1n1
df -h /data                   # now about 12G
```

Because this filesystem sits directly on the device there is no partition to grow first; for a partitioned disk run `sudo growpart /dev/nvme0n1 1` before resizing.

7. **Create an EFS file system and mount it.** First allow NFS from the instance's security group to the mount target's:

```bash
SUBNET_ID=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --query
  'Reservations[0].Instances[0].SubnetId' --output text)
VPC_ID=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --query
  'Reservations[0].Instances[0].VpcId' --output text)
INST_SG=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --query
  'Reservations[0].Instances[0].SecurityGroups[0].GroupId' --output text)
EFS_SG=$(aws ec2 create-security-group --group-name shop-api-efs --description "EFS
  NFS" --vpc-id "$VPC_ID" --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id "$EFS_SG" --protocol tcp --port
  2049 --source-group "$INST_SG"
FS=$(aws efs create-file-system --encrypted --performance-mode generalPurpose --
  throughput-mode elastic \
  --tags Key=Name,Value=shop-api-efs --query FileSystemId --output text)
until [ "$(aws efs describe-file-systems --file-system-id "$FS" --query
  'FileSystems[0].LifeCycleState' --output text)" = available ]; do sleep 5; done
aws efs create-mount-target --file-system-id "$FS" --subnet-id "$SUBNET_ID" --
  security-groups "$EFS_SG"
```

Wait until `aws efs describe-mount-targets --file-system-id "$FS" --query 'MountTargets[0].LifeCycleState'` says `available`, then on the instance:

```bash
sudo dnf install -y amazon-efs-utils
sudo mkdir -p /mnt/efs && sudo mount -t efs -o tls "$FS":/ /mnt/efs     # use your fs-... ID literally on the instance
echo "shared by every client" | sudo tee /mnt/efs/readme.txt
df -h /mnt/efs
```

`df` shows a very large (8 EiB-style) capacity because EFS is elastic; you pay only for bytes stored. Mount the same file system on a second instance in another AZ (with its own mount target) and you will read `readme.txt` there too.

8. **Check for waste** (a habit worth forming): unattached volumes and old snapshots appear in these queries.

```bash
aws ec2 describe-volumes --filters Name=status,Values=available --query 'Volumes[].[VolumeId,Size,VolumeType]' --output table
aws ec2 describe-snapshots --owner-ids self --query 'Snapshots[].[SnapshotId,StartTime,VolumeSize]' --output table
```

9. **Clean up.** On the instance: `sudo umount /restore /mnt/efs /data`, then remove the `/data` line from `/etc/fstab`. Then:

```bash
aws ec2 detach-volume --volume-id "$VOL2"; aws ec2 wait volume-available --volume-ids "$VOL2"
aws ec2 delete-volume --volume-id "$VOL2"
aws ec2 detach-volume --volume-id "$VOL";  aws ec2 wait volume-available --volume-ids "$VOL"
aws ec2 delete-volume --volume-id "$VOL"
aws ec2 delete-snapshot --snapshot-id "$SNAP"
MT=$(aws efs describe-mount-targets --file-system-id "$FS" --query 'MountTargets[].MountTargetId' --output text)
for m in $MT; do aws efs delete-mount-target --mount-target-id "$m"; done
until [ -z "$(aws efs describe-mount-targets --file-system-id "$FS" --query 'MountTargets[].MountTargetId' --output text)" ]; do sleep 5; done
aws efs delete-file-system --file-system-id "$FS"
aws ec2 delete-security-group --group-id "$EFS_SG"
```

Terminate the instance if you created it only for this lab. Confirm with step 8 that no volumes or snapshots remain.

7 Real-world scenario

Illustrative scenario, not a documented incident. A self-managed PostgreSQL replica for `shop-api` reports slow queries every night at 02:00. CloudWatch shows `VolumeQueueLength` climbing and `BurstBalance` at zero on the 500 GB **gp2** data volume: the nightly report exhausts the burst credits and the volume falls to its baseline of 1,500 IOPS (3 per GB). Nothing is wrong with the database.

Fix: a live `modify-volume` to **gp3** with 6,000 provisioned IOPS, no downtime and a lower per-GB price, then `iostat -x 1` during the next run confirms the latency drop. **Lesson:** match volume type to the IO pattern, read the volume's own metrics (CloudWatch §6.18) before blaming the application, and remember changes can be made online. A follow-up item: a daily snapshot policy, tested by actually restoring one.

8 Common mistakes and best practices

Don't

- Assume an EBS volume is multi-AZ or that it protects against deletion; it protects against hardware failure only.
- Forget snapshots, or never test a restore.
- Run `mkfs` on a device without checking `lsblk` (formatting the wrong disk erases data).
- Add an `/etc/fstab` line without `nofail` or without testing `mount -a`.
- Use EFS for a single-server database or a high-IOPS workload (use EBS io2/gp3) or leave EFS in Standard class for cold data.
- Leave unattached volumes, old snapshots or idle EFS file systems running.

Do

- Default to **gp3**; migrate gp2 volumes; provision IOPS only when metrics demand it.
- Enable **EBS encryption by default** and encrypt EFS at creation.
- Automate snapshots with DLM or AWS Backup, copy critical ones to another region and test restores (Testing Backup Restoration §15.9).
- Mount by UUID; set `DeleteOnTermination` deliberately; tag volumes with owner and application.
- Allow NFS to EFS only from the application's security group.
- Choose EFS Infrequent Access lifecycle and One Zone where the durability trade-off is acceptable.

9 Tasks

- [ ] **T1 — Attach, format and persist a volume** *(≈40 min, beginner, cents)*
  - **Goal:** add a data disk that survives reboots.
  - **Steps:**
    1. Run guide steps 1-3 on a `t3.micro`.
    2. Reboot the instance (`sudo reboot`), reconnect and check `df -h /data` and the file.
  - **Done when:** `/data/hello.txt` still exists after a reboot with the mount coming from `/etc/fstab`.

- [ ] **T2 — Snapshot and restore into a different AZ** *(≈45 min, intermediate, cents)*
  - **Goal:** prove snapshots are the way to move data between AZs.
  - **Steps:**
    1. Take a snapshot (step 4).
    2. Create a volume from it in a *different* AZ than the instance and attach it to a second instance there.
    3. Mount it (with `-o nouuid` if needed) and read the file.
  - **Done when:** the second instance in another AZ prints `snapshot me`.

- [ ] **T3 — Break it: grow without growing the filesystem** *(≈30 min, intermediate)*
  - **Goal:** understand the three layers (volume, partition, filesystem).
  - **Steps:**
    1. Run `modify-volume` to a larger size and check `lsblk` versus `df -h`.
    2. Notice the filesystem did not change, then run `xfs_growfs`.
    3. Try to reduce the size with `modify-volume` and record the error.
  - **Done when:** `df -h` shows the new size, and you can explain why the shrink request is refused.

- [ ] **T4 — Automate snapshot retention** *(≈60 min, advanced)*
  - **Goal:** a repeatable backup with cleanup.
  - **Steps:**
    1. Write a script that snapshots every volume tagged `Backup=daily`, tags the snapshots with a date, and deletes your own snapshots older than 7 days.
    2. Run it from cron or a scheduled GitHub Actions workflow with a least-privilege role (`ec2:CreateSnapshot`, `ec2:DescribeSnapshots`, `ec2:DeleteSnapshot`, `ec2:CreateTags`; see IAM §6.7).
    3. Read about DLM and say what it would replace.

- **Done when:** two runs create tagged snapshots and a simulated old snapshot is removed.

- [ ] **T5 — Share files with EFS across two AZs** *(≈60 min, advanced, cents)*
  - **Goal:** see shared file storage.
  - **Steps:**
    1. Complete guide step 7 on instance A.
    2. Launch instance B in another AZ, add an EFS mount target in its subnet, and mount the same file system.
    3. Write a file on A and read it on B; append on B and read on A.
  - **Done when:** both instances see each other's writes within seconds; then run the full cleanup.

- [ ] **T6 — Prove it: pick the storage** *(≈30 min, advanced)*
  - **Goal:** justify storage choices.
  - **Steps:**
    1. For each of these: PostgreSQL data on EC2, a WordPress `uploads` folder on 4 web servers, nightly backups, a 10 TB sequential log archive, Kubernetes `ReadWriteMany` volume, state which of EBS gp3, io2, st1, EFS or S3 you pick and why.
    2. Explain what a snapshot is and why the first read after a restore is slow.
  - **Done when:** your six answers are defensible against the comparison table.

10 Self-check questions

1. Why can't two instances in different AZs share one EBS volume, and what do you use instead?
2. What does "incremental snapshot" mean, and what happens when you delete an older snapshot?
3. Why choose gp3 over gp2?
4. After `modify-volume` increases the size, `df -h` is unchanged. Why?
5. When is EFS the right answer, and when is it the wrong one?

ANSWERS

1. A volume lives in a single AZ and is attached to one instance. Use EFS or S3 for shared data, or snapshots and application replication to move or copy data across AZs.

2. After the first full copy, each snapshot stores only changed blocks yet restores a complete volume. Deleting an older snapshot keeps any blocks still needed by later ones.
3. It is cheaper per GB, gives a 3,000 IOPS / 125 MiB/s baseline at any size, and IOPS and throughput can be scaled independently of capacity.
4. The block device is bigger but the filesystem (and maybe partition) was not resized; run `growpart` if partitioned, then `xfs_growfs` or `resize2fs`.
5. Right: many Linux clients need one POSIX shared directory (CMS uploads, shared tooling, `ReadWriteMany`). Wrong: single-server databases and IOPS-hungry workloads (EBS), or object-style storage of assets (S3), due to cost and latency.

11 Further reading

- AWS documentation: *Amazon EBS User Guide* (volume types, snapshots, Elastic Volumes, encryption, Data Lifecycle Manager).
- AWS documentation: *Amazon EFS User Guide* (mounting with the EFS mount helper, throughput modes, lifecycle management).
- AWS pricing pages for EBS and EFS.
- *Systems Performance* by Brendan Gregg, for IOPS, latency and throughput fundamentals.

LAB NOTES What I ran, what I saw, what it means

6.10 RDS and Aurora

Topic 10 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

Amazon RDS (Relational Database Service) runs a relational database for you: AWS provisions the instance, installs the engine (PostgreSQL, MySQL, MariaDB, Oracle, SQL Server or Db2), patches it, takes backups, and can fail over to a standby in another Availability Zone (AZ). You still design schemas, tune queries and size the instance; AWS owns the operating system and the "keep it running" work. You cannot SSH into an RDS instance — you reach it only over the database port.

Amazon Aurora is AWS's own database engine family, available in **MySQL-compatible** and **PostgreSQL-compatible** editions and managed through the RDS API. Instead of each database server owning a disk, Aurora separates compute from a shared, distributed storage layer that keeps copies of your data across three AZs. That changes how replicas, failover and backups behave, which is why Aurora is treated as its own product even though the CLI says `aws rds`.

2 Why it matters in DevOps

- **The database is the part of the system you cannot just redeploy.** A wrong choice (single-AZ, no backups, public endpoint) turns a bad day into data loss.
- **Failover, restore and scaling are your job to design and rehearse**, even when AWS performs the mechanics.
- **Cost:** a forgotten Multi-AZ instance with provisioned storage bills every hour whether or not anything uses it.
- **Security:** RDS is the most common place for customer data; network isolation, encryption (see KMS §6.15) and credential handling (see Secrets Manager §6.16) are decided here.
- Backups are only useful if restores are tested — see DR in Level 0 §0.9.

3 Real-world system example

AWS's own managed-database offering underpins many public-facing products. Aurora was announced by AWS in 2014 and is documented as a "cloud-native" design in which the database layer sends redo log records to a storage service instead of writing full data pages; AWS's published Aurora paper (SIGMOD 2017, *Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases*) describes the six-copy, quorum-based storage. Many SaaS companies publicly describe running PostgreSQL on RDS or Aurora to avoid operating replication and backups themselves, while larger operators often still run their own databases on EC2 or Kubernetes when they need extensions, kernel tuning or parameters RDS does not expose. The recurring pattern: **one writer, several read replicas, automated backups with point-in-time recovery, and a connection pool (for example RDS Proxy or PgBouncer) in front** — exactly the `shop-api` shape used in this roadmap.

4 Diagram

RDS Multi-AZ (instance deployment) versus Aurora:

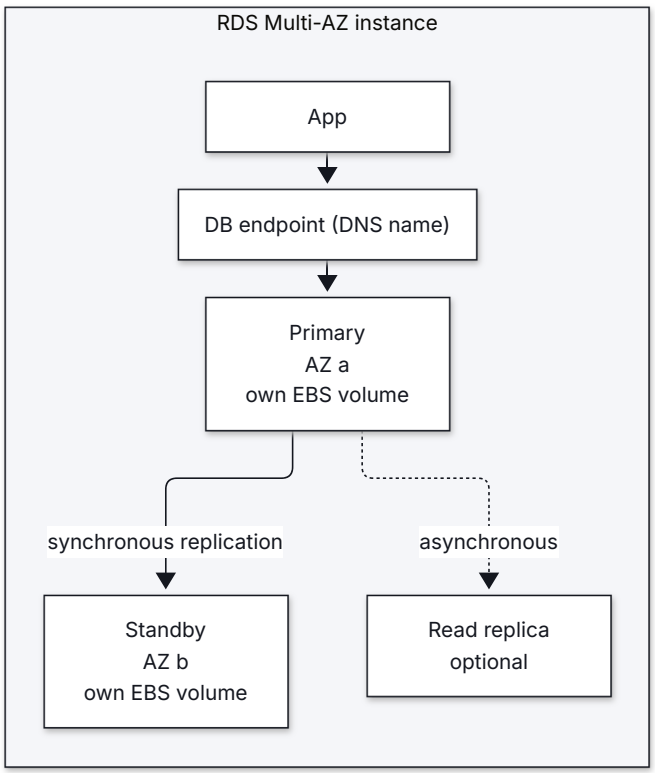


Figure 6.10-1

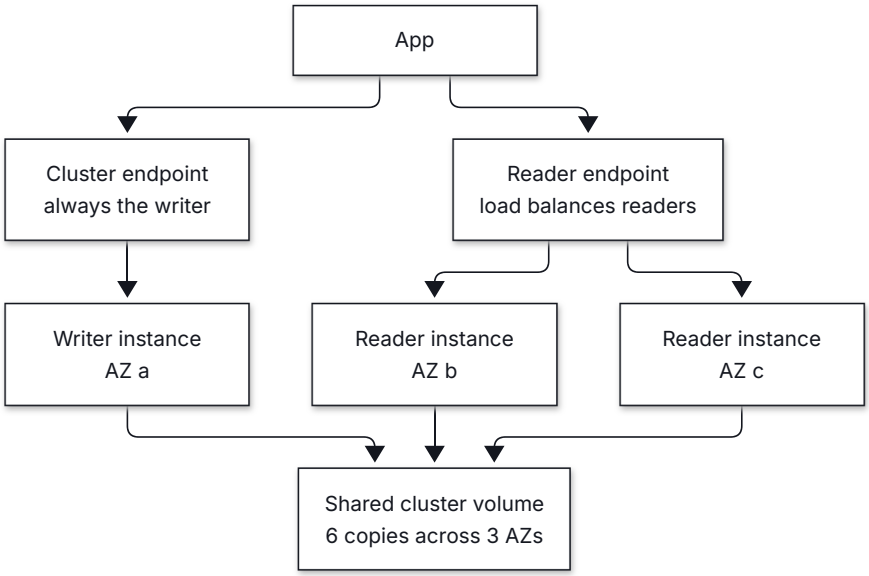


Figure 6.10-2

5 How it works

Key RDS building blocks

Concept	What it means
DB instance class	CPU/RAM size, e.g. `db.t4g.micro` (burstable, Graviton), `db.m7g.large`, `db.r7g.large` (memory optimized)
Storage	EBS-backed: `gp3` (default choice), `io1/io2` (provisioned IOPS). Storage can grow (storage autoscaling) but cannot shrink
DB subnet group	Set of subnets in **at least two AZs** where RDS may place instances. Use private subnets
Parameter group	Engine settings (`max_connections`, `log_min_duration_statement`). Some changes need a reboot
Endpoint	A DNS name. On failover AWS repoints the name, so apps must re-resolve DNS and reconnect
Maintenance window	Weekly window when AWS applies minor patches

Multi-AZ versus read replicas — different problems

	Multi-AZ	Read replica
Purpose	**Availability** (survive an AZ or host failure)	**Read scaling** (and cross-Region copies, migrations)

	Multi-AZ	Read replica
Replication	Synchronous (instance deployment)	Asynchronous — replica can lag
Readable?	Instance deployment: standby is **not** readable. Multi-AZ **DB cluster** (PostgreSQL/MySQL): two readable standbys	Yes, has its own endpoint
Failover	Automatic; the endpoint DNS name moves to the standby. Typically about a minute or two for the instance deployment; the DB-cluster deployment is documented as faster	Manual `promote-read-replica`; the replica becomes a standalone DB
Cost	About 2x the instance	One more instance per replica

Backups and recovery

- **Automated backups:** daily snapshot plus transaction logs, retention **0-35 days** (0 disables them — never do that in production). They enable **point-in-time recovery (PITR)** to any second inside the window, usually up to about the last five minutes. Restore always creates a **new instance** with a new endpoint; it never rewinds the old one.
- **Manual snapshots:** kept until you delete them, survive instance deletion, can be copied across Regions and shared.
- **Encryption** is chosen at creation (see KMS §6.15). You cannot flip an existing unencrypted instance to encrypted: snapshot, copy the snapshot with encryption, restore.

Aurora specifics

- Storage is a **cluster volume** that grows automatically (in 10 GB segments, up to 128 TiB) and is replicated six ways across three AZs; writes need a quorum (4 of 6), reads 3 of 6. You pay for storage actually used plus I/O (or choose **Aurora I/O-Optimized**, which trades higher instance/storage prices for no per-I/O charge).
- One **writer** plus up to **15 readers**, all reading the same volume, so replica lag is typically milliseconds and a reader can be promoted quickly. Use the **cluster endpoint** for writes and the **reader endpoint** for reads.
- **Aurora Serverless v2** scales capacity in fine steps (ACUs) and can scale to zero ACUs with auto-pause (check current behavior for your engine version). **Global Database** replicates to other Regions; **Backtrack** (MySQL edition) rewinds in place.
- Aurora usually costs more per instance-hour than RDS but removes replication-lag and storage-copy chores. Pick plain RDS for small, predictable workloads; Aurora when you need fast failover, many readers or very large storage.

Other features worth knowing: RDS Proxy (connection pooling in front of the DB, helps Lambda and failover times), **Blue/Green Deployments** (staging copy for major upgrades), **Performance Insights**, and **Extended Support** (extra hourly fee when you stay on an engine

version past community end-of-life).

6 Step-by-step guide

Prerequisites: AWS CLI v2 configured (`aws sts get-caller-identity` works), a default or custom VPC in `us-east-1`, a billing alarm (CloudWatch topic §6.18). **Cost warning:** a `db.t4g.micro` with 20 GB `gp3` costs a few cents per hour (a few dollars per month); Multi-AZ roughly doubles it; read replicas and snapshots add more. Free-tier rules changed for accounts created after July 2025 (credit-based "Free plan" instead of 12 free months) — check the Billing console's **Free Tier** page for what applies to you. Keep the session short and run step 9 (cleanup).

1. **Pick networking.** Using the default VPC for a lab (production uses private subnets, see VPC §6.4):

   ```bash
   export AWS_REGION=us-east-1
   VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true --query
     'Vpcs[0].VpcId' --output text)
   SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID --query
     'Subnets[].SubnetId' --output text)
   echo $VPC_ID $SUBNETS
   ```

   Expect a `vpc-...` id and several `subnet-...` ids from different AZs.

2. **Create a subnet group and a security group that only allows your IP.**

   ```bash
   aws rds create-db-subnet-group --db-subnet-group-name shop-lab \
     --db-subnet-group-description "shop-api lab" --subnet-ids $SUBNETS
   SG_ID=$(aws ec2 create-security-group --group-name shop-db-lab --description "lab db"
     \
     --vpc-id $VPC_ID --query GroupId --output text)
   MYIP=$(curl -s https://checkip.amazonaws.com)
   aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 5432
     --cidr $MYIP/32
   ```

   In real systems the source is the *application's security group*, never an IP range (see Security Groups §6.6).

3. **Create a PostgreSQL instance.** `--manage-master-user-password` makes RDS generate the admin password and keep it in Secrets Manager §6.16, so no password appears in your shell

history.

```bash
aws rds create-db-instance --db-instance-identifier shop-lab \
  --engine postgres --db-instance-class db.t4g.micro \
  --allocated-storage 20 --storage-type gp3 --storage-encrypted \
  --master-username shopadmin --manage-master-user-password \
  --db-name shop --db-subnet-group-name shop-lab \
  --vpc-security-group-ids $SG_ID --publicly-accessible \
  --backup-retention-period 7 --no-multi-az
aws rds wait db-instance-available --db-instance-identifier shop-lab   # ~5-10 minutes
```

`--publicly-accessible` is only so you can test from a laptop in this lab; production databases are not public.

4. **Find the endpoint and the secret, then connect.**

```bash
aws rds describe-db-instances --db-instance-identifier shop-lab \
  --query 'DBInstances[0].
  [Endpoint.Address,MasterUserSecret.SecretArn,MultiAZ,DBInstanceStatus]' --output
  text
SECRET=$(aws rds describe-db-instances --db-instance-identifier shop-lab --query
  'DBInstances[0].MasterUserSecret.SecretArn' --output text)
export PGPASSWORD=$(aws secretsmanager get-secret-value --secret-id $SECRET --query
  SecretString --output text | python3 -c 'import sys,json;print(json.load(sys.stdin)
  ["password"])')
HOST=$(aws rds describe-db-instances --db-instance-identifier shop-lab --query
  'DBInstances[0].Endpoint.Address' --output text)
psql "host=$HOST user=shopadmin dbname=shop sslmode=require" -c "select version();"
```

Expect a line starting `PostgreSQL ...`. (RDS PostgreSQL 15+ requires TLS by default, hence `sslmode=require`.)

5. **Turn on Multi-AZ and watch it.**

```bash
aws rds modify-db-instance --db-instance-identifier shop-lab --multi-az --apply-
  immediately
aws rds describe-db-instances --db-instance-identifier shop-lab --query
  'DBInstances[0].[DBInstanceStatus,MultiAZ]' --output text
```

Status goes `modifying` and returns to `available` with `True` — AWS builds the standby from a snapshot. Billing doubles from here.

6. **Force a failover and measure it.**

```bash
aws rds reboot-db-instance --db-instance-identifier shop-lab --force-failover
while true; do date +%T; psql "host=$HOST user=shopadmin dbname=shop sslmode=require
  connect_timeout=2" -tc "select 1" 2>&1 | head -1; sleep 2; done
```

   You will see errors for a minute or two, then success. Check `aws rds describe-events --source-type db-instance --source-identifier shop-lab --duration 30` for "Multi-AZ instance failover started/completed". The takeaway: **apps must retry and re-resolve DNS**.

7. **Create a read replica, then promote it.**

```bash
aws rds create-db-instance-read-replica --db-instance-identifier shop-lab-ro --source-
  db-instance-identifier shop-lab
aws rds wait db-instance-available --db-instance-identifier shop-lab-ro
aws rds promote-read-replica --db-instance-identifier shop-lab-ro   # replica becomes
  an independent database
```

   After promotion it no longer follows the source — this is the manual disaster-recovery move.

8. **Snapshot and point-in-time restore.**

```bash
aws rds create-db-snapshot --db-instance-identifier shop-lab --db-snapshot-identifier
  shop-lab-manual-1
aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier shop-lab
  \
  --target-db-instance-identifier shop-lab-restored --use-latest-restorable-time \
  --db-subnet-group-name shop-lab --db-instance-class db.t4g.micro
```

   The restored copy has a **different endpoint**; you cut the app over by changing config, not by overwriting the original.

9. **Clean up (do not skip).**

```bash
for DB in shop-lab-restored shop-lab-ro shop-lab; do
  aws rds delete-db-instance --db-instance-identifier $DB --skip-final-snapshot --
  delete-automated-backups
done
aws rds delete-db-snapshot --db-snapshot-identifier shop-lab-manual-1
aws rds wait db-instance-deleted --db-instance-identifier shop-lab
aws rds delete-db-subnet-group --db-subnet-group-name shop-lab
aws ec2 delete-security-group --group-id $SG_ID
```

Deleting `shop-lab` also removes its managed secret. Verify with `aws rds describe-db-instances --query 'DBInstances[].DBInstanceIdentifier'` (empty list) and check for leftover snapshots in the console.

Cheaper sandbox: for SQL practice, `docker run --rm -e POSTGRES_PASSWORD=dev -p 5432:5432 postgres:17` behaves like RDS PostgreSQL for queries (not for Multi-AZ, backups or parameter groups). LocalStack can emulate parts of the RDS API, but its coverage depends on edition and has changed over time — check its current coverage page before relying on it.

7 Real-world scenario

Illustrative scenario. On a Monday the `shop-api` team gets paged: checkout latency spikes and CloudWatch shows the RDS instance at 95% CPU and `DatabaseConnections` near `max_connections`. The cause: a new report endpoint runs heavy aggregations on the primary, and each autoscaled app instance opens 20 connections. Short term, they create a read replica and point the report code at its endpoint; they also cap the app's pool size. Replica lag appears in `ReplicaLag` during a bulk import, so the finance team's "real-time" dashboard briefly shows stale numbers — a documented trade-off of asynchronous replication. Next quarter they add RDS Proxy and a Multi-AZ standby after a maintenance-window failover caused 90 seconds of errors because the app did not retry. **Lesson:** replicas scale reads, Multi-AZ gives availability, neither replaces connection pooling and retry logic in the application.

8 Common mistakes and best practices

Don't

- Leave a database `publicly-accessible` with `0.0.0.0/0` on port 5432/3306.
- Set the backup retention to 0, or never test a restore.
- Treat the standby as a read replica (it is not readable in a Multi-AZ instance deployment) or rely on replicas for failover without promoting them.
- Hard-code the master password in code or Terraform state; use managed or rotated secrets.
- Forget that stopped RDS instances restart automatically after seven days, and that snapshots/storage keep billing after a stop.

Do

- Run production in private subnets, Multi-AZ, encrypted, with `deletion protection` on, and allow access only from the app's security group.

- Alert on `CPUUtilization`, `FreeStorageSpace`, `DatabaseConnections`, `ReplicaLag` (CloudWatch §6.18).
- Make applications reconnect with backoff; keep DNS TTL caching short in clients.
- Rehearse a PITR restore and record how long it took (this is your real RTO).
- Use parameter groups and Blue/Green Deployments for upgrades instead of editing a live server by hand.

9 Tasks

Cost note: these tasks create billable resources. Set a billing alarm first, delete everything at the end of each task.

- [ ] **T1 — Launch a managed PostgreSQL** *(≈45 min, beginner)*
  - **Goal:** create, connect to and delete an RDS instance with the CLI.
  - **Steps:**
    1. Follow guide steps 1-4 (use `--no-multi-az`, `db.t4g.micro`).
    2. Create a table `products(id serial primary key, name text)` and insert two rows with `psql`.
    3. Run step 9 for this instance only.
  - **Done when:** `select count(*) from products;` returned `2` and `describe-db-instances` no longer lists the instance.
- [ ] **T2 — Inspect Multi-AZ** *(≈45 min, intermediate)*
  - **Goal:** see what Multi-AZ changes.
  - **Steps:**
    1. Run `aws rds describe-db-instances --query 'DBInstances[0].[AvailabilityZone,SecondaryAvailabilityZone,MultiAZ]'` before and after `modify-db-instance --multi-az`.
    2. Note the cost difference using the AWS Pricing Calculator for both configurations.
  - **Done when:** you can show a populated `SecondaryAvailabilityZone` and state the hourly price difference.
- [ ] **T3 — Break it: measure a failover** *(≈45 min, intermediate)*
  - **Goal:** quantify the downtime a failover causes.
  - **Steps:**
    1. Run the polling loop from step 6 in one terminal.
    2. In another, run `reboot-db-instance --force-failover`.

3. Record the first failing and first succeeding timestamps and the AZ before and after.

- **Done when:** you have a measured outage duration in seconds and the AZ changed.

- [ ] **T4 — Replica lag experiment** *(≈60 min, intermediate)*
  - **Goal:** observe asynchronous replication.
  - **Steps:**
    1. Create a read replica (step 7 without promotion).
    2. Insert 100,000 rows on the primary using `generate_series`.
    3. Query the replica repeatedly and graph `ReplicaLag` in CloudWatch.
  - **Done when:** you saw `ReplicaLag` rise above 0 seconds and fall back, and the replica row count caught up.

- [ ] **T5 — Automate a restore drill** *(≈60 min, advanced)*
  - **Goal:** make restore a script, not a heroic act.
  - **Steps:**
    1. Write `restore-drill.sh` that PITR-restores to a new identifier, waits for availability, runs `select count(*) from products`, prints elapsed seconds and deletes the copy.
    2. Run it twice.
  - **Done when:** the script prints a row count that matches the source and a duration, and leaves no instance behind.

- [ ] **T6 — Prove it: Multi-AZ vs replica vs Aurora** *(≈45 min, advanced)*
  - **Goal:** justify a design choice.
  - **Steps:**
    1. Without the guide, draw the `shop-api` database design for "99.95% availability and a reporting workload", choosing RDS Multi-AZ, replicas, or Aurora.
    2. List failure modes (AZ loss, bad migration, accidental `DELETE`) and the recovery action for each.
  - **Done when:** a teammate can read your one-page design and name the recovery step for all three failures.

10 Self-check questions

1. Which feature protects against an AZ outage and which one scales reads?
2. Why can an application still see errors after a "successful" Multi-AZ failover?

3. How do you restore a database to 10:42 yesterday, and what do you get back?
4. What is different about Aurora's storage compared with an RDS instance's EBS volume?
5. Why must the DB subnet group span two AZs even for a single-AZ instance?
6. How do you encrypt an existing unencrypted RDS instance?

ANSWERS

1. Multi-AZ gives availability (synchronous standby); read replicas scale reads (asynchronous).
2. The endpoint name is repointed to the standby, but existing connections are dropped and clients may cache DNS; apps need reconnect/retry logic.
3. PITR (`restore-db-instance-to-point-in-time`) within the retention window; you get a **new instance** with a new endpoint, and you cut over by configuration.
4. Aurora uses a shared distributed cluster volume replicated six ways across three AZs that grows automatically; readers share it, so lag is low and failover is fast.
5. RDS requires at least two AZs in the group so it can place a standby or recover elsewhere later.
6. Take a snapshot, copy it with encryption enabled (choosing a KMS key), and restore from the encrypted copy.

11 Further reading

- *Amazon RDS User Guide* and *Amazon Aurora User Guide* (AWS documentation) — Multi-AZ, backups, parameter groups.
- *Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases* (SIGMOD 2017).
- AWS Well-Architected Framework — Reliability Pillar.
- *Designing Data-Intensive Applications* by Martin Kleppmann — the replication chapter.
- See also: Secrets Manager §6.16, ElastiCache §6.11 (cache in front of the database).

LAB NOTES What I ran, what I saw, what it means

6.11 ElastiCache

Topic 11 of 18 | ~4 hours | Difficulty: Intermediate

1 What it is

Amazon ElastiCache is a managed in-memory data store. You get a **Valkey**, **Redis OSS** or **Memcached** endpoint that answers in well under a millisecond because the data lives in RAM, and AWS handles node provisioning, patching, replication, failover and monitoring. Valkey is the open-source (Linux Foundation) fork of Redis created after Redis changed its license in 2024; ElastiCache supports it alongside Redis OSS (up to version 7.1), and AWS prices Valkey lower than Redis OSS.

It comes in two shapes: **node-based clusters** (you choose node type, shard and replica counts) and **ElastiCache Serverless** (you name a cache and pay for stored data and compute used; AWS scales it). ElastiCache lives **inside your VPC** and has no public endpoint — you reach it from EC2, containers or Lambda in the same network.

2 Why it matters in DevOps

- **It takes load off the database.** A cache hit costs microseconds and no database CPU; during traffic spikes the cache often decides whether RDS §6.10 survives.
- **Shared state for stateless apps:** sessions, rate-limit counters, distributed locks and job queues must live outside the instances so that Auto Scaling §6.2 can add and remove servers freely.
- **It fails in interesting ways:** evictions silently lower the hit rate, a cold cache after failover can melt the database (a "thundering herd"), and a cache without TTLs serves stale data forever.
- **Cost:** memory is the most expensive resource per GB in AWS; right-sizing matters (see Cost §20.8).

3 Real-world system example

Caching in front of a relational database is a pattern AWS documents in its whitepaper *Database Caching Strategies Using Redis*, which describes **lazy loading (cache-aside)** and **write-through** with TTLs. Large web platforms typically run Redis- or Memcached-compatible fleets for session data, leaderboards, rate limiting and queues; Shopify, for instance, has written publicly about using Redis alongside its primary databases for background jobs and caching. ElastiCache itself is the managed version of the same thing: a primary node, replicas in other AZs, and automatic promotion when the primary dies. For the Redis internals (data types, persistence, eviction) see Redis in Level 13 §13.7.

4 Diagram

Cache-aside read path:

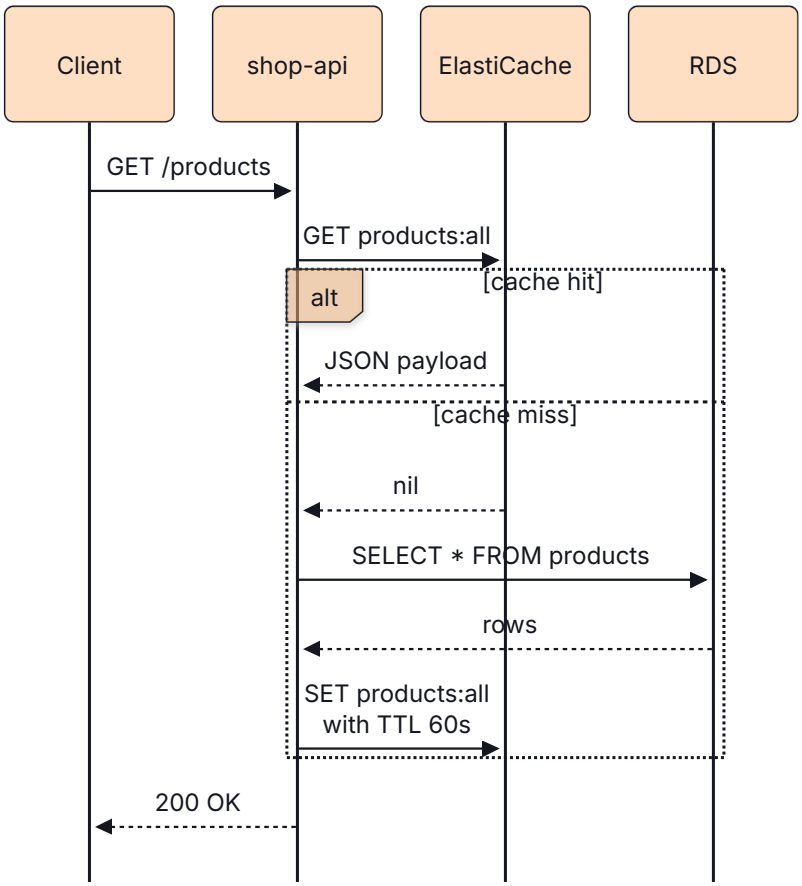


Figure 6.11-1

Replication group with automatic failover (cluster mode disabled):

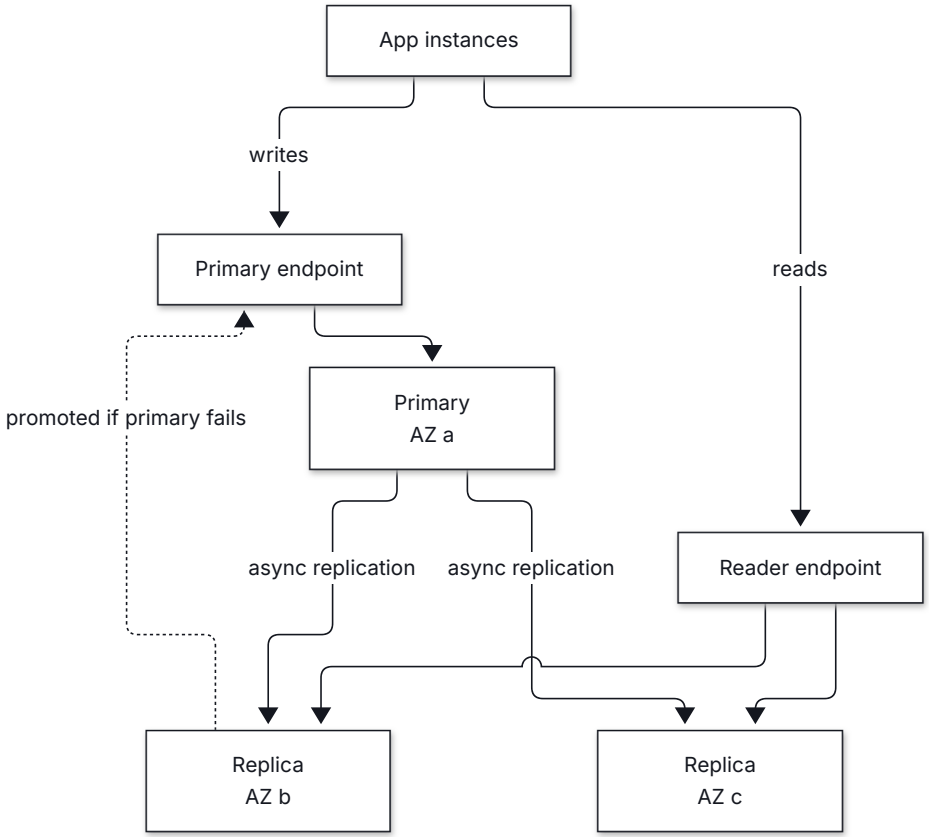


Figure 6.11-2

5 How it works

Engines

	Valkey / Redis OSS	Memcached
Data types	strings, hashes, lists, sets, sorted sets, streams, pub/sub	strings only
Persistence / replication	replication, Multi-AZ failover, snapshots	none — a node loss loses its data
Scaling	add replicas; cluster mode shards data across up to hundreds of shards	add nodes; the **client** hashes keys
Use when	sessions, queues, counters, leaderboards, anything needing failover	simple, multi-threaded, throw-away cache

Topologies (node-based)

- **Cluster mode disabled:** one shard = one primary + up to 5 replicas. Simple; clients use the *primary endpoint* for writes and the *reader endpoint* for reads. Data must fit in one node's memory.
- **Cluster mode enabled:** data is split into 16,384 hash slots over multiple shards, each with its own replicas. Clients use the *configuration endpoint* and must be cluster-aware; multi-key operations need keys in the same slot (hash tags like `{user42}`).
- **Serverless:** no node sizing; you set optional maximum limits. Good for spiky or unknown loads; it has a minimum cost for stored data.

Failover. Enable **Multi-AZ with automatic failover** and at least one replica. When the primary fails, ElastiCache promotes the replica with the least lag and updates DNS of the primary endpoint — typically within tens of seconds to a couple of minutes, and writes made but not yet replicated can be lost (asynchronous replication). You can rehearse it with `test-failover`.

Security. Place it in private subnets with a security group that allows port 6379 only from the app's security group. Turn on **encryption in transit** (clients then use TLS, `rediss://`) and **at rest** (KMS, see KMS §6.15). Authenticate with an **AUTH token** or, better, **RBAC users and user groups**.

Caching patterns

Pattern	How	Trade-off
Cache-aside (lazy loading)	App checks cache, on miss loads from DB and fills cache	Only requested data cached; first request slow; data can go stale
Write-through	App writes DB and cache together	Cache always warm; wasted memory for never-read data
TTL (expiry)	Every key gets `EX` / `EXPIRE`	Bounds staleness; add random jitter to avoid synchronized expiry
Write-behind	Write cache first, flush to DB later	Fast writes, risk of data loss — rarely appropriate

Eviction policy (`maxmemory-policy`) decides what is dropped when memory is full; the ElastiCache default is `volatile-lru` (evict least-recently-used keys *that have a TTL*). Keys without TTL are never evicted, so a full cache of non-expiring keys returns write errors.

6 Step-by-step guide

Prerequisites: AWS CLI v2; a VPC with subnets in two AZs and a security group for the app (use your default VPC for a lab); an EC2 instance in that VPC to connect from (see EC2 §6.1). **Cost warning:** two `cache.t4g.micro` nodes cost a few cents per hour in total (single-digit dollars per month); billing continues until you delete the replication group. Free-tier eligibility for ElastiCache depends on when your account was created — check the Billing console's Free Tier page. Do the cleanup step.

1. **Local practice first (free).**

```bash
docker run -d --name cache -p 6379:6379 valkey/valkey:8      # redis:7 works
  identically for these commands
redis-cli ping                                                  # PONG
redis-cli set products:all '[{"id":1}]' EX 60
redis-cli ttl products:all                                      # a number below 60
```

   LocalStack may emulate the ElastiCache control-plane API, but a local container is the better way to learn commands.

2. **Network objects.**

```bash
export AWS_REGION=us-east-1
VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true --query
  'Vpcs[0].VpcId' --output text)
SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID --query
  'Subnets[].SubnetId' --output text)
aws elasticache create-cache-subnet-group --cache-subnet-group-name shop-cache-lab \
  --cache-subnet-group-description "shop-api cache lab" --subnet-ids $SUBNETS
CACHE_SG=$(aws ec2 create-security-group --group-name shop-cache-lab --description
  "lab cache" \
  --vpc-id $VPC_ID --query GroupId --output text)
```

   For the client, put your EC2 instance's security group as the source:

```bash
APP_SG=sg-0123456789abcdef0     # replace with the security group of your EC2 instance
aws ec2 authorize-security-group-ingress --group-id $CACHE_SG --protocol tcp --port
  6379 --source-group $APP_SG
```

3. **Create a replication group: one primary + one replica across AZs.**

```bash
aws elasticache create-replication-group --replication-group-id shop-cache-lab \
  --replication-group-description "shop-api cache lab" \
  --engine valkey --cache-node-type cache.t4g.micro \
  --num-cache-clusters 2 --automatic-failover-enabled --multi-az-enabled \
  --transit-encryption-enabled --at-rest-encryption-enabled \
  --cache-subnet-group-name shop-cache-lab --security-group-ids $CACHE_SG
aws elasticache wait replication-group-available --replication-group-id shop-cache-lab
  # ~5-15 min
```

If your CLI is older and rejects `valkey`, update AWS CLI v2 or use `--engine redis`.

4. **Read the endpoints.**

```bash
aws elasticache describe-replication-groups --replication-group-id shop-cache-lab \
  --query 'ReplicationGroups[0].NodeGroups[0].
  [PrimaryEndpoint.Address,ReaderEndpoint.Address,Status]' --output text
```

Similar to `shop-cache-lab.abc123.ng.0001.use1.cache.amazonaws.com shop-cache-lab-ro.abc123.ng.0001.use1.cache.amazonaws.com available`.

5. **Connect from the EC2 instance (TLS is on, so use `--tls`).**

```bash
redis-cli -h <PRIMARY_ENDPOINT> --tls -p 6379 ping        # PONG (install with: sudo
  dnf install -y redis6 or valkey)
```

From your laptop this times out — by design the cluster has no public address. For laptop access use an SSM Session Manager port-forward through the EC2 instance.

6. **Implement cache-aside in Node.js** (the pattern `shop-api` uses; `npm i redis`):

```javascript
const { createClient } = require('redis');
const cache = createClient({ url: `rediss://${process.env.CACHE_HOST}:6379` });
cache.on('error', (e) => console.error('cache error', e.message));   // never let
  cache outages crash the app
async function getProducts(db) {
  const hit = await cache.get('products:all').catch(() => null);
  if (hit) return JSON.parse(hit);
  const { rows } = await db.query('select id, name from products');
  await cache.set('products:all', JSON.stringify(rows), { EX: 60 +
  Math.floor(Math.random() * 10) }).catch(() => {});
  return rows;
}
```

Note the TTL jitter and that cache failures degrade to a database read instead of an error.

7. **Rehearse a failover.**

```bash
aws elasticache test-failover --replication-group-id shop-cache-lab --node-group-id
  0001
aws elasticache describe-events --source-type replication-group --duration 30 --query
  'Events[].[Date,Message]' --output text
```

Keep a `redis-cli ... ping` loop running; expect a short window of errors, then recovery, and events mentioning failover.

8. **Look at the metrics that matter.**

```bash
aws cloudwatch get-metric-statistics --namespace AWS/ElastiCache --metric-name
  CacheHitRate \
  --dimensions Name=CacheClusterId,Value=shop-cache-lab-001 --start-time $(date -u -d
  '-1 hour' +%FT%TZ) \
  --end-time $(date -u +%FT%TZ) --period 300 --statistics Average
```

Also watch `EngineCPUUtilization`, `DatabaseMemoryUsagePercentage`, `Evictions`, `CurrConnections`.

9. **Clean up.**

```bash
aws elasticache delete-replication-group --replication-group-id shop-cache-lab
aws elasticache wait replication-group-deleted --replication-group-id shop-cache-lab
aws elasticache delete-cache-subnet-group --cache-subnet-group-name shop-cache-lab
aws ec2 delete-security-group --group-id $CACHE_SG
docker rm -f cache
```

Verify with `aws elasticache describe-replication-groups --query 'ReplicationGroups[].ReplicationGroupId'` (empty).

7 Real-world scenario

Illustrative scenario. Black Friday: `shop-api` serves the product list from Valkey with a 60 s TTL; the RDS CPU sits at 15%. At 02:00 a maintenance event promotes a replica and the primary endpoint briefly refuses connections. The app's cache client throws, and because every request path awaited the cache without a fallback, the API returns 500s for 40 seconds. Worse, the promoted node has the data, but the app also restarted its pool and 300 instances tried to repopulate the same hot key from RDS at once, spiking its CPU to 90%. The fix: treat

the cache as optional (catch errors and read the DB), add jitter and a short single-flight lock on refills, and set alarms on `Evictions` and `CacheHitRate`. **Lesson:** a cache is an optimization, not a dependency — design the failure path as carefully as the happy path.

8 Common mistakes and best practices

Don't

- Expose the cache with a permissive security group, or turn off TLS/AUTH because "it's inside the VPC".
- Store data that exists nowhere else (a cache can be wiped by failover, scaling or maintenance).
- Cache with no TTL or with identical TTLs for millions of keys.
- Use `KEYS *` in production (it blocks the single-threaded engine); use `SCAN`.
- Use cluster mode with a non-cluster-aware client.

Do

- Enable Multi-AZ with automatic failover and a replica for anything serving production traffic.
- Handle cache errors gracefully, use connection pooling and exponential backoff.
- Alert on `Evictions`, `CPU` / `EngineCPUUtilization`, memory use and `ReplicationLag`.
- Size memory with headroom (reserved-memory settings) so a backup/snapshot does not push the node into swap.
- Namespace keys (`shop-api:products:all`) and version them when the cached format changes.

9 Tasks

Cost note: tasks 2-5 create billable ElastiCache nodes. Billing alarm first; delete at the end of each session.

- [ ] **T1 — Cache-aside locally** *(≈40 min, beginner)*
  - **Goal:** implement and observe a hit/miss cycle without any cloud cost.
  - **Steps:**
    1. Run Valkey or Redis in Docker and PostgreSQL in Docker.
    2. Implement the `getProducts` function from step 6 with a `console.log` for hit/miss.
    3. Call it three times within 60 s, then wait for the TTL to expire and call again.

- **Done when:** the log shows `miss, hit, hit, miss` and `redis-cli ttl` counted down between calls.

- [ ] **T2 — Provision a replicated cache** *(≈60 min, intermediate)*
  - **Goal:** build the replication group from the CLI.
  - **Steps:**
    1. Do guide steps 2-4.
    2. Run `describe-replication-groups` and record both endpoints and the AZ of each node.
  - **Done when:** the status is `available`, `MultiAZ` is `enabled` and the nodes sit in two AZs.

- [ ] **T3 — Connect over TLS from EC2** *(≈45 min, intermediate)*
  - **Goal:** prove the cache is private and TLS-only.
  - **Steps:**
    1. From your laptop, run `redis-cli -h <endpoint> --tls ping` and note that it times out.
    2. From an EC2 instance in the app security group, run it again; then try without `--tls`.
  - **Done when:** laptop times out, EC2 + `--tls` returns `PONG`, and EC2 without `--tls` fails.

- [ ] **T4 — Break it: test failover** *(≈45 min, advanced)*
  - **Goal:** measure what failover does to a client.
  - **Steps:**
    1. Run a loop that writes `incr counter` once per second and prints errors.
    2. Execute `test-failover` and note the error window and which node became primary.
    3. Check whether any writes were lost (compare the counter to the number of successes).
  - **Done when:** you report the error window in seconds and whether the counter lost increments.

- [ ] **T5 — Explain a stampede and fix it** *(≈60 min, advanced)*
  - **Goal:** prevent a thundering herd on a hot key.
  - **Steps:**
    1. Using `autocannon` or `ab`, hit an endpoint right after `redis-cli del products:all` and count DB queries.

2. Add a `SET products:lock 1 NX EX 5` guard so only one request refills the cache while others wait or serve stale data.
3. Repeat the load test.

- **Done when:** the DB query count after the delete drops from many to one.

☐ **T6 — Prove it: choose the right engine** *(≈30 min, intermediate)*

- **Goal:** justify engine and topology choices.
- **Steps:**
  1. Without notes, decide Valkey/Redis vs Memcached and cluster mode on/off for: HTTP sessions, a 400 GB product catalog cache, a rate limiter.
  2. State the failure behavior of each choice.
- **Done when:** a teammate agrees with your reasoning on all three cases.

10 Self-check questions

1. Why does ElastiCache refuse connections from your laptop, and how do you reach it for debugging?
2. What is the difference between cache-aside and write-through?
3. What can be lost during a failover, and why?
4. When would you pick Memcached over Valkey/Redis?
5. What happens when `maxmemory` is reached and all keys lack a TTL under `volatile-lru`?

ANSWERS

1. It has no public endpoint and its security group only allows the app; use a bastion or SSM port-forward from a host inside the VPC.
2. Cache-aside fills on read misses; write-through updates the cache on every write.
3. Recent writes not yet replicated to the promoted replica (asynchronous replication), plus in-flight connections.
4. A simple, multi-threaded, disposable cache where persistence, replication and rich data types are unnecessary.
5. Nothing is evictable, so writes fail with an out-of-memory error.

11 Further reading

- *Amazon ElastiCache User Guide* (AWS documentation), including *Best practices* and the *Valkey* sections.

- AWS whitepaper *Database Caching Strategies Using Redis*.
- Valkey and Redis command documentation (valkey.io, redis.io).
- See also: RDS and Aurora §6.10, Auto Scaling §6.2.

LAB NOTES What I ran, what I saw, what it means

LEVEL 6: CLOUD (AWS)

6.12 Route 53

Topic 12 of 18 | ~4 hours | Difficulty: Intermediate

1 What it is

Amazon Route 53 is AWS's **DNS (Domain Name System)** service: it turns names such as `shop.example.com` into the addresses or AWS endpoints that serve them. It can also register domains and run **health checks**, and it can answer differently depending on who is asking or whether a target is healthy (weighted, latency-based, failover and geolocation routing). The "53" is the DNS port number.

The core object is a **hosted zone**: a container of records for one domain. A **public** hosted zone answers queries from the internet; a **private** hosted zone answers only inside the VPCs you associate with it (for names like `db.internal.example.com`). Each hosted zone gets four **name servers** (NS records); you tell your domain registrar to delegate to them. If you want the underlying DNS concepts first, read DNS Resolution §2.5.

2 Why it matters in DevOps

- **DNS is the first hop of every request.** A wrong record, an expired delegation or a long TTL during a migration is a full outage that no load balancer can hide.
- **It is a deployment tool:** weighted records shift traffic between environments (canary), failover records implement active-passive disaster recovery (DR §0.9).
- **It is a dependency of everything else:** ACM certificate validation, CloudFront aliases, ALB names, SES, email (MX) and service discovery all use records.
- **Cost surprises are small but real:** hosted zones bill monthly, queries per million, health checks per check.

3 Real-world system example

The 2016 Dyn attack is the textbook DNS-dependency incident: on 21 October 2016 a Mirai-botnet DDoS against the managed DNS provider Dyn made many popular sites (including Twitter, GitHub and Netflix, per public reporting) unreachable for some users, even though

their own servers were healthy. A widely drawn lesson was to use more than one DNS provider for critical names. In AWS's own incident report for the 19-20 October 2025 `us-east-1` event, AWS described a latent race condition in the automation that manages DNS records for the DynamoDB regional endpoint, which left that endpoint with an empty record and cascaded to dependent services — a reminder that DNS changes are production changes. Route 53 itself is built around anycast name servers on AWS's edge network and is documented with a 100% availability SLA for the DNS data plane. For a deep dive into DNS architecture see DNS Architecture §12.3.

4 Diagram

How a lookup for `shop.example.com` reaches Route 53 and an alias target:

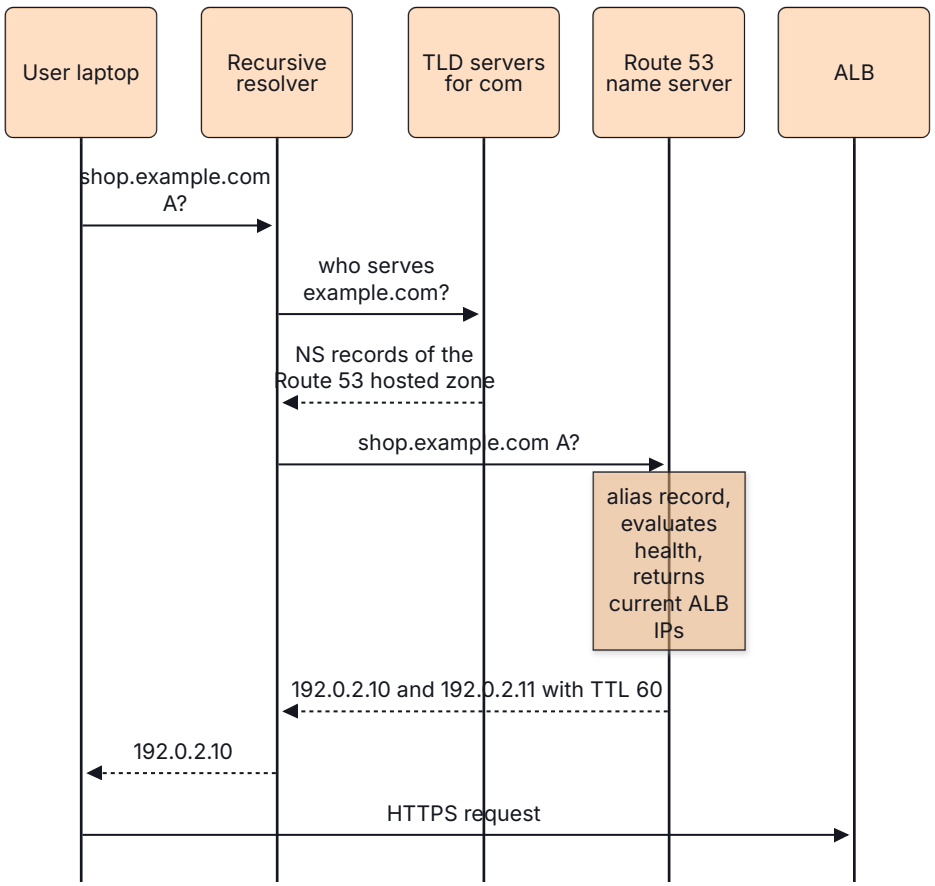


Figure 6.12-1

Failover routing with health checks:

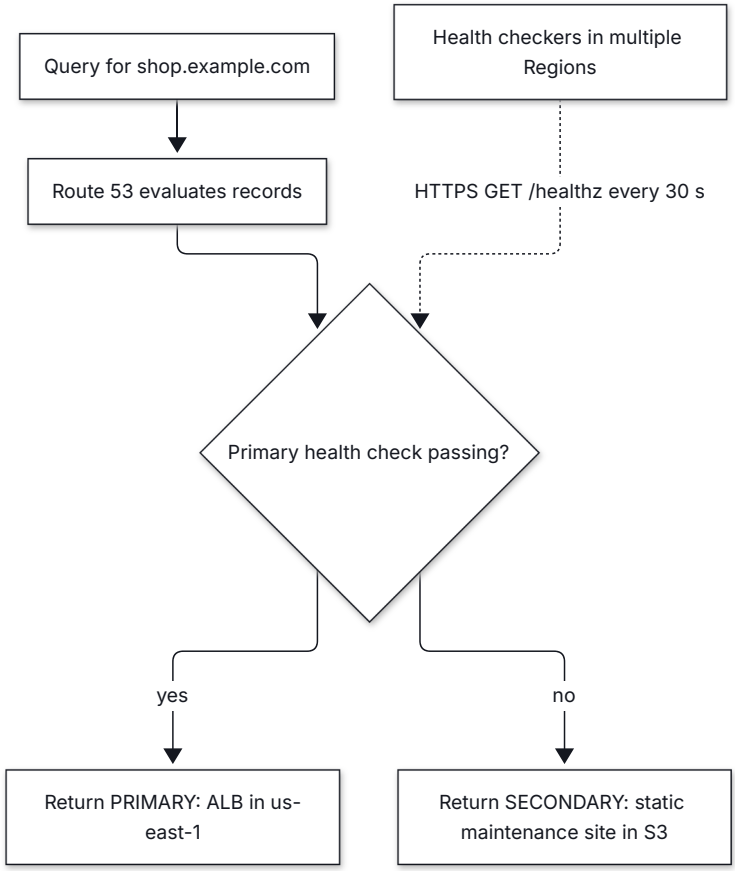


Figure 6.12-2

5 How it works

Record types you will actually use

Type	Maps	Notes
`A` / `AAAA`	name to IPv4 / IPv6	
`CNAME`	name to another name	**Not allowed at the zone apex** (`example.com`), cannot coexist with other records on the same name
Alias (Route 53-specific)	name to an AWS resource (ALB, CloudFront, S3 website, API Gateway, another record)	Shown as `A` / `AAAA` ; works at the apex; no charge for alias queries to AWS resources; follows the target's IP changes automatically

Type	Maps	Notes
`MX` / `TXT`	mail routing / arbitrary text (SPF, DKIM, domain verification)	
`NS` / `SOA`	delegation and zone metadata	created automatically; do not edit the apex NS/SOA
`CAA`	which CAs may issue certificates	Amazon's CA uses `amazon.com`, `amazontrust.com`, `awstrust.com`, `amazonaws.com`
`SRV`, `PTR`, `DS`	service location, reverse lookups, DNSSEC chain	

Routing policies

Policy	Use it for
Simple	One record, no logic
Weighted	Blue/green and canary: 90/10 split (Deployment Strategies §0.7)
Latency	Send users to the AWS Region with the lowest measured latency
Failover	Active-passive: primary until its health check fails
Geolocation / Geoproximity	Content restrictions, data residency, regional sites
IP-based	Route by the client's CIDR (for example a partner's network)
Multivalue answer	Up to 8 healthy records returned (basic client-side load spreading, not a real load balancer)

Health checks are run by checkers in several AWS Regions. Three kinds: an **endpoint** check (HTTP/HTTPS/TCP to an IP or name, every 30 s or every 10 s "fast", with a failure threshold), a **calculated** check (combines other checks with AND/OR), and a check that mirrors a **CloudWatch alarm** (useful for private resources). A record is considered unhealthy after the failure threshold, and Route 53 stops returning it. Remember **TTL**: clients that cached the old answer keep using it until the TTL expires, so low TTLs (30-60 s) matter for failover records. For alias records, `EvaluateTargetHealth` uses the target's own health (for example whether the ALB has healthy targets) without a separate health check.

Pricing shape (check the pricing page for current figures): a hosted zone is billed monthly (about $0.50 for the first 25 zones, and no charge if you delete the zone within 12 hours of creating it), standard queries per million, each health check monthly, and domain registration yearly depending on the TLD. Alias queries to AWS resources are free.

Delegation. After creating the zone, copy its four NS values into the registrar (if the domain is registered in Route 53, this is done automatically for the zone *you set as the domain's name servers*; check them after you delete and recreate a zone because the NS set changes).

6 Step-by-step guide

Prerequisites: AWS CLI v2. **You do not need to own a domain for steps 1-7** — you can create a hosted zone for a name you do not own and query it directly with `test-dns-answer`; it simply is not reachable by the public. With a real domain, see step 8. **Cost warning:** a hosted zone is about $0.50 per month (free if deleted within 12 hours), each health check costs a small monthly fee, and a registered domain costs from a few to tens of dollars per year and is non-refundable. Do step 9.

1. **Create a hosted zone.**

```bash
aws route53 create-hosted-zone --name lab.example.com --caller-reference "lab-$(date
  +%s)"
ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name lab.example.com --query
  'HostedZones[0].Id' --output text | cut -d/ -f3)
aws route53 get-hosted-zone --id $ZONE_ID --query 'DelegationSet.NameServers'
```

   Output: four names like `ns-123.awsdns-45.com`.

2. **Create simple and weighted records.** Write `weighted.json`:

```json
{
  "Changes": [
    {"Action": "UPSERT", "ResourceRecordSet": {"Name": "api.lab.example.com", "Type":
  "A", "SetIdentifier": "blue",
      "Weight": 90, "TTL": 60, "ResourceRecords": [{"Value": "192.0.2.10"}]}},
    {"Action": "UPSERT", "ResourceRecordSet": {"Name": "api.lab.example.com", "Type":
  "A", "SetIdentifier": "green",
      "Weight": 10, "TTL": 60, "ResourceRecords": [{"Value": "192.0.2.20"}]}}
  ]
}
```

```bash
aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch
  file://weighted.json
for i in 1 2 3 4 5 6 7 8 9 10; do
  aws route53 test-dns-answer --hosted-zone-id $ZONE_ID --record-name
  api.lab.example.com --record-type A --query 'RecordData[0]' --output text
done
```

   You should see mostly `192.0.2.10` and occasionally `192.0.2.20`.

3. **Create a health check on a primary endpoint.** (`203.0.113.10` is a documentation ad-

dress, so it will report unhealthy — which is exactly what we want for the demo.)

```bash
cat > hc.json <<'EOF'
{"IPAddress":"203.0.113.10","Port":80,"Type":"HTTP","ResourcePath":"/healthz","Request
  Interval":30,"FailureThreshold":3}
EOF
HC_ID=$(aws route53 create-health-check --caller-reference "hc-$(date +%s)" --health-
  check-config file://hc.json --query 'HealthCheck.Id' --output text)
echo $HC_ID
```

4. **Create failover records.** Write `failover.json` (replace `HC_ID_HERE` with `$HC_ID`):

```json
{
  "Changes": [
    {"Action": "UPSERT", "ResourceRecordSet": {"Name": "www.lab.example.com", "Type":
  "A", "SetIdentifier": "primary",
      "Failover": "PRIMARY", "TTL": 30, "HealthCheckId": "HC_ID_HERE",
  "ResourceRecords": [{"Value": "203.0.113.10"}]}},
    {"Action": "UPSERT", "ResourceRecordSet": {"Name": "www.lab.example.com", "Type":
  "A", "SetIdentifier": "secondary",
      "Failover": "SECONDARY", "TTL": 30, "ResourceRecords": [{"Value":
  "198.51.100.20"}]}}
  ]
}
```

```bash
sed -i "s/HC_ID_HERE/$HC_ID/" failover.json
aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch
  file://failover.json
```

5. **Watch failover happen.**

```bash
aws route53 get-health-check-status --health-check-id $HC_ID --query
  'HealthCheckObservations[].StatusReport.Status' --output text
aws route53 test-dns-answer --hosted-zone-id $ZONE_ID --record-name
  www.lab.example.com --record-type A --query RecordData
```

After one to two minutes the checkers report failures and the answer switches to `["198.51.100.20"]`. (Right after creation, Route 53 may still treat a new check as healthy.)

6. **Add an alias record (needs a real ALB or CloudFront).** Alias to an ALB needs the ALB's DNS name and *its* hosted zone ID (visible in `aws elbv2 describe-load-balancers`, for `us-`

`east-1` it is `Z35SXDOTRQ7X7K`; a CloudFront alias always uses `Z2FDTNDATAQYW2`):

```json
{"Changes":[{"Action":"UPSERT","ResourceRecordSet":
  {"Name":"shop.lab.example.com","Type":"A",
  "AliasTarget":{"HostedZoneId":"Z35SXDOTRQ7X7K","DNSName":"shop-alb-123456.us-east-
  1.elb.amazonaws.com","EvaluateTargetHealth":true}}}]}
```

See ALB §6.14 and CloudFront §6.13.

7. **Inspect everything.**

```bash
aws route53 list-resource-record-sets --hosted-zone-id $ZONE_ID --query
  'ResourceRecordSets[].[Name,Type,SetIdentifier]' --output table
dig +short NS lab.example.com @$(aws route53 get-hosted-zone --id $ZONE_ID --query
  'DelegationSet.NameServers[0]' --output text)
```

`dig` against the zone's own name server works even without a registered domain (the second command queries NS for the apex).

8. **With a real domain you own:** create the zone for it (`example.com` in the docs; use yours), set the four NS values at your registrar, and wait for propagation (`dig +short NS yourdomain.tld` shows the AWS names). Lower the TTL of any existing records a day before a migration.
9. **Clean up.** A hosted zone must be empty (except NS/SOA) before it can be deleted.

```bash
sed 's/UPSERT/DELETE/' weighted.json > del1.json
sed 's/UPSERT/DELETE/' failover.json > del2.json
aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch
  file://del1.json
aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch
  file://del2.json
aws route53 delete-health-check --health-check-id $HC_ID
aws route53 delete-hosted-zone --id $ZONE_ID
```

Delete any alias records you added in step 6 the same way before deleting the zone.

LocalStack: the Route 53 API can be emulated locally to test scripts, but resolution and health-check behavior are not real.

7 Real-world scenario

Illustrative scenario. The `shop-api` team migrates from a single VM to an ALB. They forget that the old record has a 24-hour TTL (86400), so after cutover half of the world keeps hitting the old VM for a day while the other half gets the new stack, and the old VM's database drifts out of sync. Next time they lower the TTL to 60 s a day before the change, use weighted records (5%, 25%, 100%) to shift traffic, and keep the old record for a week for fast rollback. Separately, they add a failover record that points to an S3-hosted maintenance page, with a health check on `/healthz`, so a full ALB failure shows an apology page instead of a browser error. **Lesson:** DNS is cached by people you do not control; plan TTLs and rollback before you change a record.

8 Common mistakes and best practices

Don't

- Put a `CNAME` at the zone apex (use an alias record).
- Leave long TTLs on records you may need to move quickly.
- Delete and recreate a hosted zone casually: the new zone has **different name servers**, so the domain breaks until the registrar is updated.
- Forget that health checks run from the internet: private endpoints need a CloudWatch-alarm-based check.
- Leave dangling records that point to deleted resources (subdomain-takeover risk).

Do

- Use alias records for AWS targets; keep zones in version control (Terraform, see Level 7) and review DNS changes like code.
- Add `CAA` records and enable DNSSEC signing for important zones if your threat model needs it.
- Use `EvaluateTargetHealth` and low TTLs on failover records; test failover regularly.
- Use private hosted zones for internal names; enable DNS hostnames/support on the VPC.
- Consider a second DNS provider for the most critical domains.

9 Tasks

Cost note: a hosted zone and health checks bill monthly. Delete the lab within 12 hours to avoid the zone charge; set a billing alarm.

- [ ] **T1 — Create a zone and records** *(≈30 min, beginner)*

- **Goal:** create a hosted zone, an `A` and a `TXT` record via CLI.
- **Steps:**
  1. Do guide step 1.
  2. UPSERT `hello.lab.example.com A 192.0.2.50` and a `TXT` record with `"owner=devops-roadmap"`.
  3. Verify each with `test-dns-answer`.
- **Done when:** both answers come back with `NOERROR` and the expected data.

☐ **T2 — Weighted traffic split** *(≈30 min, intermediate)*

- **Goal:** model a canary.
- **Steps:**
  1. Create two weighted records 95/5 and run `test-dns-answer` 40 times, counting each answer.
  2. Change to 50/50 and repeat; then set blue to 0.
- **Done when:** your counts roughly match the weights and weight 0 never returns blue.

☐ **T3 — Failover with a health check** *(≈45 min, intermediate)*

- **Goal:** watch DNS fail over automatically.
- **Steps:**
  1. Run `docker run -d -p 8080:80 nginx` on a VPS with a public IP and create a health check against it (port 8080, path `/`) with a primary/secondary pair.
  2. Confirm primary answers, then `docker stop` the container.
  3. Poll `test-dns-answer` until the secondary is returned.
- **Done when:** you recorded the seconds between stopping the container and the answer changing.

☐ **T4 — Break it: the apex CNAME and the dangling record** *(≈30 min, intermediate)*

- **Goal:** hit two classic errors safely.
- **Steps:**
  1. Try to create a `CNAME` for `lab.example.com` (the apex) and read the error.
  2. Replace it with an alias or `A` record.
  3. Create `old.lab.example.com CNAME deleted-bucket.s3.amazonaws.com` and explain why this is a takeover risk.
- **Done when:** you can state the exact error and the safe alternative.

☐ **T5 — Script a DNS audit** *(≈45 min, advanced)*

- **Goal:** automate review.
- **Steps:**
  1. Write a script that lists all records in a zone, flags TTL above 300 s on non-NS/SOA records and flags `CNAME`s whose targets fail `dig`.
  2. Run it against your lab zone.
- **Done when:** the script prints a table with at least one flagged and one clean record.

☐ **T6 — Prove it: plan a zero-downtime DNS migration** *(≈45 min, advanced)*

- **Goal:** explain a safe cutover.
- **Steps:**
  1. Without the guide, write the timeline (T-24h, T-0, T+1h, T+7d) for moving `shop.example.com` from a VM to an ALB with rollback.
  2. Include TTL values, weights and health checks.
- **Done when:** a teammate can execute the plan and knows when to roll back.

10 Self-check questions

1. Why can't you use a CNAME for `example.com` itself, and what do you use?
2. A failover record did not switch for several minutes. Name three reasons.
3. What is the difference between a public and a private hosted zone?
4. What happens to your site if you delete and recreate the hosted zone?
5. How does Route 53 weighted routing differ from an ALB's weighted target groups?

ANSWERS

1. The apex must hold SOA/NS records and a CNAME cannot coexist with other records; use an alias `A` / `AAAA` record.
2. Health-check interval times failure threshold has not elapsed; resolvers cached the old answer for the TTL; the health check was probing the wrong path/port or the record was not linked to a check.
3. Public answers the internet; private answers only from associated VPCs.
4. It gets new name servers, so resolution breaks until the registrar's NS records are updated.
5. DNS weights split *clients* via cached answers (coarse, TTL-delayed); ALB weights split *requests* per connection/request instantly.

11 Further reading

- *Amazon Route 53 Developer Guide* (AWS documentation) — routing policies and health checks.
- RFC 1034 and RFC 1035 (DNS concepts) and RFC 2308 (negative caching).
- *DNS and BIND* by Cricket Liu and Paul Albitz.
- See also: CloudFront §6.13, Disaster Recovery §0.9.

LAB NOTES What I ran, what I saw, what it means

6.13 CloudFront

Topic 13 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

Amazon CloudFront is AWS's **Content Delivery Network (CDN)**: a global fleet of caching servers (called **edge locations** and, behind them, larger **regional edge caches**) that sit between your users and your **origin** — the S3 bucket or load balancer that actually holds the content. A user in Lagos or Sydney is routed (via DNS and anycast-style routing) to a nearby edge location; if that edge already has the object it answers in a few milliseconds, otherwise it fetches it from your origin once, keeps a copy, and serves the next thousand users from memory or disk.

You configure CloudFront as a **distribution**: a name (`d111111abcdef8.cloudfront.net`, or your own domain such as `www.example.com`), a list of **origins**, and one or more **cache behaviors** that say "requests matching this path pattern go to that origin, are cached by this rule, and allow these HTTP methods". Besides speed, a distribution is also a **security and cost layer**: it terminates TLS at the edge, absorbs traffic spikes and DDoS attempts (AWS Shield Standard is included), can attach AWS WAF, and lets you keep your S3 bucket and even your load balancer closed to the public internet.

2 Why it matters in DevOps

- **Latency and load:** static assets (JS, CSS, images, downloads) and cacheable API responses never reach your servers. A cache hit costs you no EC2 §6.1 CPU and no RDS §6.10 query.
- **Cost:** data transferred from AWS origins (S3, ALB, EC2) to CloudFront is free of charge, and CloudFront's own egress is typically cheaper than sending the same bytes straight from S3 or EC2 to the internet. For heavy download workloads this is one of the simplest big savings (see Network Egress Costs §20.7).
- **Security:** with **Origin Access Control (OAC)** the S3 bucket stays fully private (S3 §6.8); with a private ALB or header checks, attackers cannot bypass your WAF by hitting the origin directly.

- **Operations:** most "the site is serving old files" tickets and many "everything is slow after the deploy" incidents are cache mistakes — wrong TTLs, missing cache-busting file names, or an invalidation that was forgotten.
- **Resilience:** CloudFront can fail over between two origins (an **origin group**) and serve stale content if configured to, which can turn an origin outage into a degraded-but-up experience.

3 Real-world system example

Static sites and single-page apps on S3 + CloudFront is the canonical AWS pattern: AWS's own documentation and solution guides describe hosting a static site or SPA by putting a private S3 bucket behind CloudFront with OAC. Media and software-download businesses use CDNs the same way — large streaming platforms typically push video segments through CDN edges so that the origin sees a tiny fraction of viewer requests. CloudFront also fronts dynamic applications: the edge keeps a persistent, optimized connection back to the origin, so even **uncacheable** API calls are often faster than going direct, because TLS and TCP handshakes happen near the user. A common AWS-documented arrangement, and the one used in this roadmap's capstone, is **CloudFront → ALB → EC2** for the dynamic app and **CloudFront → S3** for static assets, both served under one domain using two cache behaviors. For the CDN concept in general see CDNs in Level 12 §12.7.

4 Diagram

Request flow through a cache (hit versus miss):

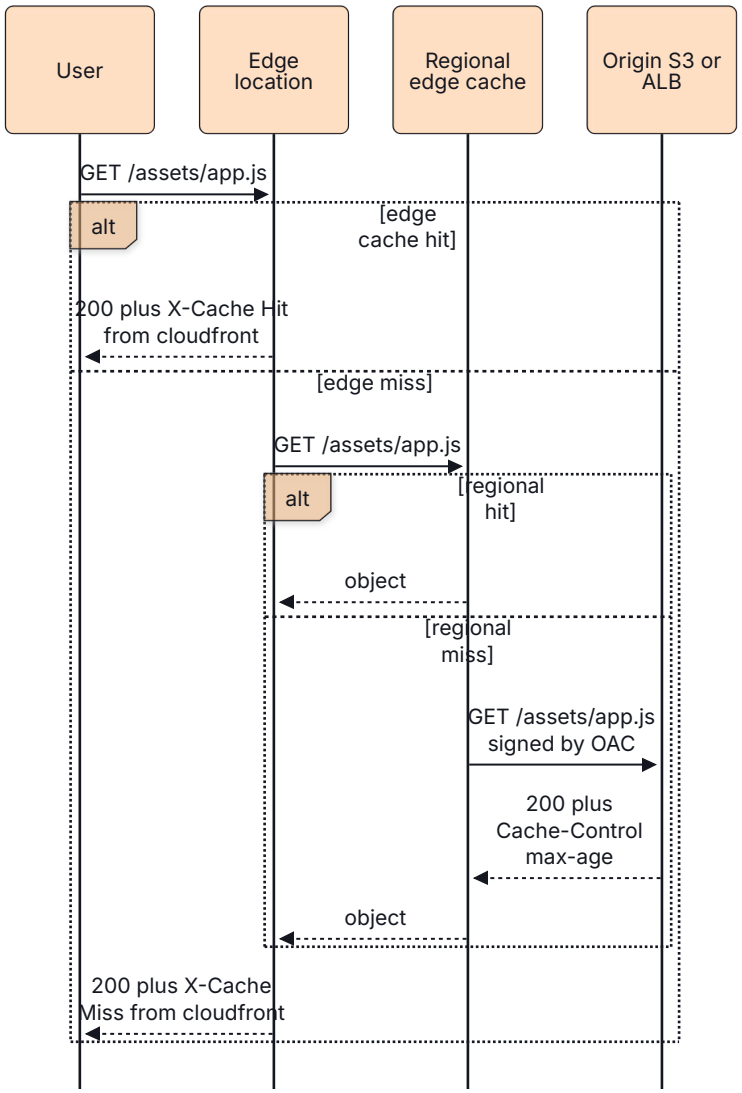


Figure 6.13-1

The `shop-api` distribution with two origins and path-based behaviors:

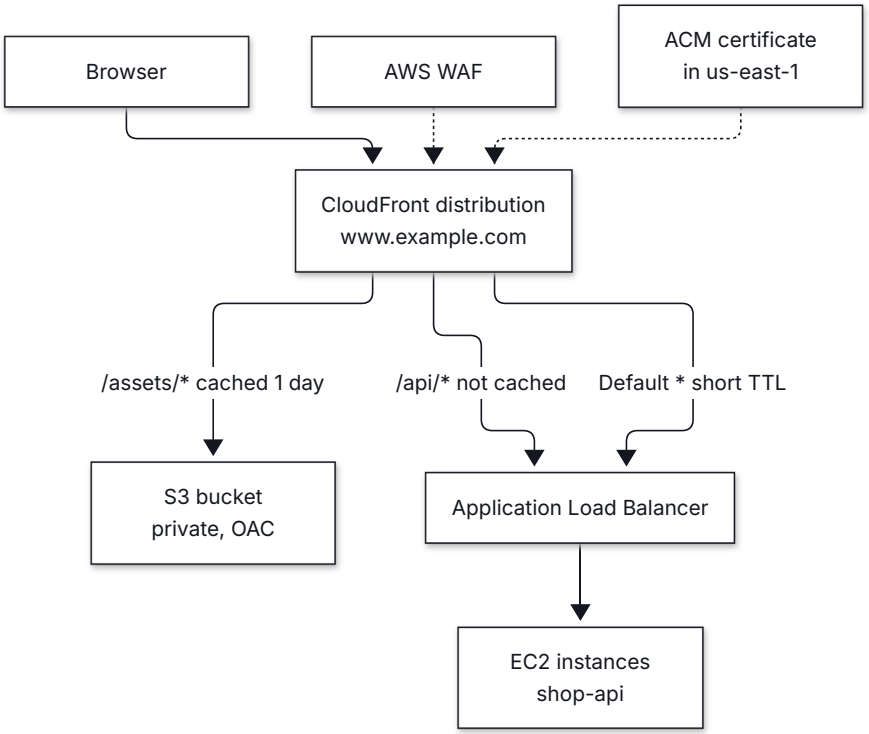


Figure 6.13-2

5 How it works

Core building blocks

Concept	What it is	Notes
Distribution	The CloudFront resource: domain, origins, behaviors, TLS, logging	Gets a `*.cloudfront.net` name; add your domain as an **alternate domain name (CNAME)** plus an ACM certificate
Origin	Where content comes from	S3 bucket, ALB/EC2 or any HTTP(S) server ("custom origin"), API Gateway, MediaPackage; a private ALB/EC2 in a VPC via **VPC origins**
Cache behavior	Path pattern (`/api/*` , `*.jpg` , default `*`) + origin + policies	Evaluated most specific first; the **Default (`*`)** behavior is mandatory
Cache policy	Which parts of the request form the **cache key** (headers, cookies, query strings) and the min/default/max TTL	Managed examples: `CachingOptimized` , `CachingDisabled`

Concept	What it is	Notes
Origin request policy	What extra info is *forwarded* to the origin without becoming part of the cache key	Example: `AllViewerExceptHostHeader` for an ALB
Response headers policy	Adds CORS and security headers (HSTS, X-Frame-Options) at the edge	Avoids code changes in the app
Price class	Which edge regions you pay to use	`PriceClass_100` = cheapest (North America and Europe)

Cache keys and TTLs. The cache key decides when two requests are "the same object". Every extra header, cookie or query string in the key *splits* the cache and lowers the hit ratio, so include only what changes the response. TTL comes from the origin's `Cache-Control: max-age` / `s-maxage` or `Expires`, clamped between the policy's minimum and maximum; if the origin sends nothing, the policy's default TTL (24 hours for `CachingOptimized`) applies. A cached object past its TTL is revalidated with the origin (`If-None-Match` / `If-Modified-Since`) and served from cache again if the origin answers `304`.

Origin Access Control (OAC). OAC lets CloudFront sign its requests to S3 with **Signature Version 4**. The bucket policy then allows only the CloudFront service principal, and only when the request comes from *your* distribution:

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowCloudFrontServicePrincipalReadOnly",
    "Effect": "Allow",
    "Principal": { "Service": "cloudfront.amazonaws.com" },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::BUCKET/*",
    "Condition": { "StringEquals": { "AWS:SourceArn":
   "arn:aws:cloudfront::ACCOUNT_ID:distribution/DIST_ID" } }
  }]
}
```

OAC replaces the older **Origin Access Identity (OAI)**, which AWS now recommends migrating away from; OAC supports all S3 regions, SSE-KMS-encrypted objects (the CloudFront principal also needs `kms:Decrypt` in the key policy — see KMS §6.15) and additional HTTP methods. OAC also exists for Lambda function URLs.

Protecting a dynamic origin (ALB). An ALB origin is reachable on its public DNS name, so anyone can bypass CloudFront. Three layers fix that: (1) **VPC origins** — put the ALB in private subnets and let CloudFront reach it privately; (2) a security group rule that allows only

the AWS-managed prefix list `com.amazonaws.global.cloudfront.origin-facing`; (3) a secret custom header (for example `X-Origin-Verify: <random>`) added by CloudFront and checked by an ALB listener rule. See Application Load Balancer §6.14.

Invalidation. To replace content before its TTL expires you create an **invalidation** for paths (`/index.html`, `/assets/*`, or `/*`). It takes seconds to a few minutes, and the first 1,000 invalidation paths per month are free, then a small charge per path (wildcards count as one path). Better than invalidating: **version your file names** (`app.3f9c2a.js`) and set a very long TTL on them, with a short TTL (or `no-cache`) on the HTML that references them. Then a deploy never needs an invalidation except for `index.html`.

TLS. Viewer HTTPS uses either the default `*.cloudfront.net` certificate or your own certificate from **AWS Certificate Manager (ACM)** — and for CloudFront that certificate **must be requested in** `us-east-1`, whatever region your app lives in. Choose a minimum TLS version via a security policy (for example `TLSv1.2_2021`). The connection *from* CloudFront *to* the origin is set separately (origin protocol policy: `https-only` is the right default).

Edge compute. CloudFront Functions (tiny JavaScript, sub-millisecond, viewer request/response only) are for URL rewrites, redirects and header tweaks. **Lambda@Edge** (full Node.js/Python, origin and viewer triggers, regional) handles heavier logic such as authentication. Both are optional extras for this topic.

Cost model. You pay per GB transferred out to viewers and per 10,000 requests, by region. CloudFront has a long-standing always-free tier of **1 TB of data transfer out and 10 million requests per month**. AWS has also introduced flat-rate pricing plans that bundle CloudFront with related services; check the pricing page for what applies to your account today. WAF, Lambda@Edge, real-time logs and Origin Shield are billed separately.

6 Step-by-step guide

Prerequisites: AWS CLI v2 configured (`aws sts get-caller-identity` works), a **billing alarm already created** (see CloudWatch §6.18), and a private S3 bucket (any name; example below uses `shop-static-<ACCOUNT_ID>`). **Cost warning:** a lab distribution serving a few files costs effectively nothing inside the always-free allowance, but a distribution that is left running with WAF attached or serving large files can cost money. Do the cleanup step. CloudFront is a global service; the CLI talks to it through `us-east-1` regardless of your region setting.

> **No AWS account?** LocalStack's CloudFront support is limited and may require a paid tier, so for a free local experiment simulate the *caching behavior* with Nginx instead (step 9).

1. **Create the bucket and upload two files.**

```bash
export AWS_REGION=us-east-1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET=shop-static-$ACCOUNT_ID
aws s3api create-bucket --bucket $BUCKET          # us-east-1 needs no
  LocationConstraint
aws s3api put-public-access-block --bucket $BUCKET \
  --public-access-block-configuration
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuck
  ets=true
mkdir -p site/assets
echo '<h1>shop-api static site</h1><script src="/assets/app.js"></script>' >
  site/index.html
echo 'console.log("shop-api v1")' > site/assets/app.js
aws s3 sync site/ s3://$BUCKET/
```

   Expected: two `upload:` lines. A direct `curl` to the bucket URL will return `403` — that is what we want.

2. **Create an Origin Access Control.**

```bash
OAC_ID=$(aws cloudfront create-origin-access-control --origin-access-control-config \
  'Name=shop-static-oac,Description=OAC for shop
  static,SigningProtocol=sigv4,SigningBehavior=always,OriginAccessControlOriginType=s3
  ' \
  --query 'OriginAccessControl.Id' --output text)
echo $OAC_ID
```

   Expected: an ID such as `E2QWRUHAPOMQZL`.

3. **Create the distribution** (S3 origin, HTTPS redirect, managed `CachingOptimized` policy, cheapest price class).

```bash
cat > dist.json <<EOF
{
  "CallerReference": "shop-static-lab-1",
  "Comment": "shop-api static lab",
  "Enabled": true,
  "DefaultRootObject": "index.html",
  "PriceClass": "PriceClass_100",
  "Origins": { "Quantity": 1, "Items": [{
    "Id": "s3-static",
    "DomainName": "$BUCKET.s3.us-east-1.amazonaws.com",
    "S3OriginConfig": { "OriginAccessIdentity": "" },
    "OriginAccessControlId": "$OAC_ID"
```

```
  }]},
  "DefaultCacheBehavior": {
    "TargetOriginId": "s3-static",
    "ViewerProtocolPolicy": "redirect-to-https",
    "AllowedMethods": { "Quantity": 2, "Items": ["GET", "HEAD"] },
    "Compress": true,
    "CachePolicyId": "658327ea-f89d-4fab-a63d-7e88639e58f6"
  }
}
EOF
aws cloudfront create-distribution --distribution-config file://dist.json \
  --query 'Distribution.{Id:Id,Domain:DomainName,Status:Status}' --output table
```

`658327ea-f89d-4fab-a63d-7e88639e58f6` is the ID of the AWS-managed `CachingOptimized` policy. Save the `Id` and `Domain`:

```
BASH
DIST_ID=E1ABCDEXAMPLE            # from the output
DOMAIN=d111111abcdef8.cloudfront.net
```

4. **Allow only this distribution to read the bucket.**

```
BASH
cat > policy.json <<EOF
{ "Version": "2012-10-17", "Statement": [{
    "Sid": "AllowCloudFrontServicePrincipalReadOnly", "Effect": "Allow",
    "Principal": { "Service": "cloudfront.amazonaws.com" },
    "Action": "s3:GetObject", "Resource": "arn:aws:s3:::$BUCKET/*",
    "Condition": { "StringEquals": { "AWS:SourceArn":
  "arn:aws:cloudfront::$ACCOUNT_ID:distribution/$DIST_ID" } } }] }
EOF
aws s3api put-bucket-policy --bucket $BUCKET --policy file://policy.json
aws cloudfront wait distribution-deployed --id $DIST_ID      # a few minutes
```

5. **Watch a cache miss become a hit.**

```
BASH
curl -sI https://$DOMAIN/assets/app.js | grep -iE "^(HTTP|x-cache|age|cache-control|x-
  amz-cf-pop)"
curl -sI https://$DOMAIN/assets/app.js | grep -iE "^(HTTP|x-cache|age)"
```

Similar to: first call `X-Cache: Miss from cloudfront`, second call `X-Cache: Hit from cloudfront` and an `Age:` header counting seconds in cache (the very first request can also show `RefreshHit` or `Miss` depending on the edge you hit). Prove the bucket is private: `curl -sI https://$BUCKET.s3.amazonaws.com/index.html` returns `403`.

6. **Change the file and see the stale copy.**

```bash
echo 'console.log("shop-api v2")' > site/assets/app.js
aws s3 cp site/assets/app.js s3://$BUCKET/assets/app.js
curl -s https://$DOMAIN/assets/app.js          # still prints v1, the edge has the old copy
```

7. **Invalidate and verify.**

```bash
aws cloudfront create-invalidation --distribution-id $DIST_ID --paths "/assets/app.js"
aws cloudfront list-invalidations --distribution-id $DIST_ID --query 'InvalidationList.Items[].[Id,Status]' --output table
# once Status is Completed:
curl -s https://$DOMAIN/assets/app.js          # now prints v2
```

8. **Add a second behavior for the dynamic app** (for the capstone). Behaviors live in the distribution config; fetch, edit and push it with the ETag as a concurrency token:

```bash
aws cloudfront get-distribution-config --id $DIST_ID > current.json
ETAG=$(jq -r .ETag current.json)
jq '.DistributionConfig' current.json > config.json
# edit config.json: add an ALB origin and a CacheBehaviors item for "/api/*" using
#   CachePolicyId 4135ea2d-6df8-44a3-9df3-4b5a84be39ad   (managed CachingDisabled)
#   OriginRequestPolicyId b689b0a8-53d0-40ab-baf2-68738e2966ac (managed AllViewerExceptHostHeader)
#   AllowedMethods GET HEAD OPTIONS PUT POST PATCH DELETE
aws cloudfront update-distribution --id $DIST_ID --if-match "$ETAG" --distribution-config file://config.json
```

In the console this is *CloudFront → Distributions → your ID → Behaviors → Create behavior*, which is much easier for a first attempt. The full multi-origin config is built in the capstone Capstone 6.

9. **Free local simulation of cache behavior (Nginx).**

```bash
mkdir -p nginx-cdn && cd nginx-cdn
cat > default.conf <<'EOF'
proxy_cache_path /tmp/cache keys_zone=edge:10m max_size=100m inactive=60m;
server {
  listen 8080;
  location / {
    proxy_pass http://host.docker.internal:9000;
    proxy_cache edge;
```

```
    proxy_cache_valid 200 30s;
    add_header X-Cache $upstream_cache_status;
  }
}
EOF
python3 -m http.server 9000 &                # the "origin"
docker run -d --name edge -p 8080:8080 --add-host=host.docker.internal:host-gateway \
  -v "$PWD/default.conf:/etc/nginx/conf.d/default.conf:ro" nginx:1.27
curl -sI localhost:8080/ | grep -i x-cache   # MISS, then HIT on the second call
```

10. **Cleanup (do not skip).** A distribution must be disabled, fully deployed, then deleted.

```bash
aws cloudfront get-distribution-config --id $DIST_ID > current.json
ETAG=$(jq -r .ETag current.json)
jq '.DistributionConfig | .Enabled=false' current.json > disabled.json
aws cloudfront update-distribution --id $DIST_ID --if-match "$ETAG" --distribution-
  config file://disabled.json
aws cloudfront wait distribution-deployed --id $DIST_ID
ETAG=$(aws cloudfront get-distribution-config --id $DIST_ID --query ETag --output
  text)
aws cloudfront delete-distribution --id $DIST_ID --if-match "$ETAG"
aws cloudfront delete-origin-access-control --id $OAC_ID --if-match "$(aws cloudfront
  get-origin-access-control --id $OAC_ID --query ETag --output text)"
aws s3 rm s3://$BUCKET --recursive && aws s3api delete-bucket --bucket $BUCKET
docker rm -f edge 2>/dev/null; kill %1 2>/dev/null
```

7 Real-world scenario

Illustrative scenario. The `shop-api` team ships a front-end release at 14:00. At 14:10 support reports that half the customers see the new layout with broken buttons while the rest see the old site. **Investigation:** `curl -sI` on `/index.html` shows `X-Cache: Hit` with `Age: 3100` and a `Last-Modified` from yesterday; `/assets/app.js` is the new build. CloudFront edges were holding the old `index.html` (24-hour default TTL) that referenced old bundle names that the deploy script had already deleted — or new HTML pointed at JS not yet cached, depending on the edge. **Fix:** invalidate `/index.html` (and `/` if the default root object is used), which resolves it within a minute. **Lesson:** the pipeline now uploads hashed asset files with `Cache-Control: public, max-age=31536000, immutable`, uploads `index.html` last with `Cache-Control: no-cache`, and runs one invalidation for `/index.html` as the final deploy step. Old assets are retained for a week so that stale HTML never points at a missing file.

8 Common mistakes and best practices

Don't

- Make the S3 bucket public "so CloudFront can read it" — use OAC and block public access.
- Forward all headers, cookies and query strings to the origin by default; every value splits the cache and ruins the hit ratio.
- Cache `/api/*` responses that vary by user (cookies/Authorization) with a shared cache key — users see each other's data.
- Rely on `/*` invalidations for every deploy; use versioned file names instead.
- Request the ACM certificate in your app's region — CloudFront only sees certificates in `us-east-1`.
- Leave an ALB origin reachable directly from the internet, bypassing WAF and caching rules.

Do

- Use the managed cache policies (`CachingOptimized` for static, `CachingDisabled` for dynamic APIs) until you have a reason to write your own.
- Set `Cache-Control` at the origin so TTLs are explicit and visible in the code that produces the content.
- Enable standard access logs (to S3) or real-time logs when you need to investigate hit ratios; watch the `CacheHitRate` and `5xxErrorRate` metrics in CloudWatch §6.18 (these appear in `us-east-1` under the `AWS/CloudFront` namespace).
- Redirect HTTP to HTTPS, set a minimum TLS 1.2 policy, and add a response headers policy for security headers.
- Define distributions as code (Terraform §7.1) so the cache rules are reviewed like any other change.

9 Tasks

- [ ] **T1 — Serve a private bucket through CloudFront** *(≈45 min, beginner)*
  - **Goal:** a working HTTPS site whose bucket is not publicly readable.
  - **Steps:**
    1. Follow steps 1-4 of the guide.
    2. Open `https://$DOMAIN/` in a browser.
    3. Run `curl -sI https://$BUCKET.s3.amazonaws.com/index.html`.
  - **Done when:** the CloudFront URL returns `200` with your page and the direct S3 URL returns `403`.
- [ ] **T2 — Observe the cache** *(≈30 min, beginner)*
  - **Goal:** read cache status from response headers.

- **Steps:**
  1. Request `/assets/app.js` five times with `curl -sI`.
  2. Note `X-Cache`, `Age` and `X-Amz-Cf-Pop` each time.
  3. In the console open *Monitoring* for the distribution and find the request count.
- **Done when:** you can show one `Miss` followed by `Hit`s with a growing `Age`, and name the edge (`X-Amz-Cf-Pop`) that served you.

☐ **T3 — Stale content and invalidation** *(≈30 min, intermediate)*

- **Goal:** reproduce the stale-file problem and fix it two ways.
- **Steps:**
  1. Change `app.js` in S3 and show CloudFront still returns the old body.
  2. Fix it with `create-invalidation` for that path.
  3. Fix it a second way: rename to `app.v3.js`, update `index.html`, upload with `Cache-Control: max-age=31536000, immutable`.
- **Done when:** both fixes work and you can say which one costs nothing per deploy.

☐ **T4 — Add a dynamic behavior** *(≈60 min, intermediate)*

- **Goal:** one domain serving static files from S3 and `/api/*` from an origin with caching disabled.
- **Steps:**
  1. Run any HTTP origin (an EC2 instance, or a public test origin you control) and add it as a second origin.
  2. Create a behavior for `/api/*` using `CachingDisabled` and `AllViewerExceptHostHeader`, allowing all methods.
  3. Request `/api/health` twice and `/assets/app.js` twice.
- **Done when:** `/api/health` never shows `Hit from cloudfront` while `/assets/app.js` does.

☐ **T5 — Break the OAC and fix it** *(≈30 min, intermediate)*

- **Goal:** recognize the symptom of a wrong bucket policy.
- **Steps:**
  1. Change `AWS:SourceArn` in the bucket policy to a wrong distribution ID.
  2. Request a not-yet-cached object (a new file) and note the status code.
  3. Restore the correct ARN.
- **Done when:** you saw `403 AccessDenied` from CloudFront for the broken policy and `200` after fixing it, and can explain why cached objects kept working in between.

- [ ] **T6 — Prove it: design the cache rules** *(≈45 min, advanced)*
  - **Goal:** justify cache settings for a real app without notes.
  - **Steps:**
    1. For `shop-api`, write a table of path pattern, origin, cache policy, TTL, and what is in the cache key for `/`, `/assets/*`, `/api/products` (public, cacheable 60 s) and `/api/cart` (per user).
    2. Explain how a deploy replaces a bad `app.js` with zero invalidations.
    3. Run the cleanup in step 10 and confirm nothing remains with `aws cloudfront list-distributions --query 'DistributionList.Items[].Id'`.
  - **Done when:** the table is internally consistent (nothing user-specific in a shared cache key) and the distribution list no longer shows your lab ID.

10 Self-check questions

1. What is the difference between a cache policy and an origin request policy?
2. Why does CloudFront need its ACM certificate in `us-east-1`?
3. What does OAC do, and why is it better than making the bucket public or using OAI?
4. How can you keep the ALB origin from being reached directly, bypassing CloudFront?
5. Why are versioned file names usually better than invalidations?
6. A user reports seeing another user's data on an API call. Which CloudFront setting is the first suspect?

ANSWERS

1. The cache policy defines the cache key and TTLs (what makes two requests "the same"); the origin request policy defines extra values forwarded to the origin that are *not* part of the key.
2. CloudFront is a global service and reads viewer certificates from ACM in `us-east-1`; a certificate in any other region cannot be attached.
3. OAC makes CloudFront sign origin requests (SigV4) and the bucket policy allows only that distribution's service principal, so the bucket stays private. OAI is the older mechanism with fewer features (for example, limited support for SSE-KMS-encrypted objects, newer regions and some HTTP methods).
4. Use VPC origins with a private ALB, restrict the ALB security group to the CloudFront origin-facing managed prefix list, and/or require a secret custom header checked by an ALB rule.
5. A new name is a new cache key, so there is nothing to purge, rollback is just re-pointing the HTML, and there is no per-path invalidation charge or propagation delay.

6. A cache policy that caches a personalized response without including the user identity (cookie or Authorization header) in the cache key — or caching enabled on a path that should use `CachingDisabled`.

11 Further reading

- *Amazon CloudFront Developer Guide* (docs.aws.amazon.com/cloudfront) — distributions, cache policies, OAC, invalidation.
- *Amazon CloudFront pricing* page — current free allowance, regional rates and flat-rate plans.
- AWS Prescriptive Guidance and Solutions Library material on hosting static websites with S3 and CloudFront.
- *High Performance Browser Networking* by Ilya Grigorik (free online) — why edge termination reduces latency.

6.14 Application Load Balancer

Topic 14 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

An **Application Load Balancer (ALB)** is AWS's managed **Layer 7** (HTTP/HTTPS/gRPC) load balancer. It accepts connections from clients, terminates TLS, reads each HTTP request, decides where it should go, and forwards it to a healthy backend. The backends are grouped into **target groups** — EC2 instances, IP addresses, Lambda functions or even another ALB — and the ALB constantly **health checks** them so that traffic only reaches targets that are answering correctly.

An ALB is made of four things you will configure over and over: **listeners** (a protocol and port to listen on, such as HTTPS:443), **rules** (if the host/path/header matches, do this action), **target groups** (where to send traffic and how to check health) and a **certificate** from **AWS Certificate Manager (ACM)** for TLS. AWS runs the load balancer nodes for you across the subnets (Availability Zones, AZs) you pick, scales them automatically and replaces them when they fail. Its Layer 4 sibling, the Network Load Balancer (NLB), is covered conceptually in L4 versus L7 Load Balancing §2.9.

2 Why it matters in DevOps

- **It is the front door:** every user request to `shop-api` goes through it. A bad listener rule or failing health check is a full outage, even when every server is healthy.
- **It enables everything else in this level:** Auto Scaling §6.2 registers new instances into a target group automatically; deployments (rolling, blue/green, canary — see Blue/Green §5.11) are traffic shifts between target groups.
- **Health checks are your failure detector:** a check that is too shallow keeps broken servers in rotation; one that is too deep (calls the database) can take the whole fleet out of rotation when the database hiccups.
- **Debugging vocabulary:** 502, 503 and 504 from an ALB each point to a different layer of the problem (section 5).

- **Security and cost:** the ALB is where you enforce HTTPS, attach AWS WAF, hide instances in private subnets, and where an idle forgotten ALB quietly bills you every hour.

3 Real-world system example

Amazon ECS and EKS service load balancing. AWS's container services use ALB target groups as the standard way to expose services: ECS registers each task's IP into a target group, and the AWS Load Balancer Controller for Kubernetes creates an ALB from an `Ingress` resource, mapping Ingress path rules to ALB listener rules and pods to `ip` targets (see Ingress §8.5). The same pattern — one ALB, many services, routing by host name or path — is documented by AWS as a way to cut cost, because a single ALB with host- and path-based rules can serve many microservices instead of needing one load balancer per service. Large platforms commonly run many services behind a small number of such shared entry points, and use weighted target-group forwarding for canary releases.

4 Diagram

How one HTTPS request is handled:

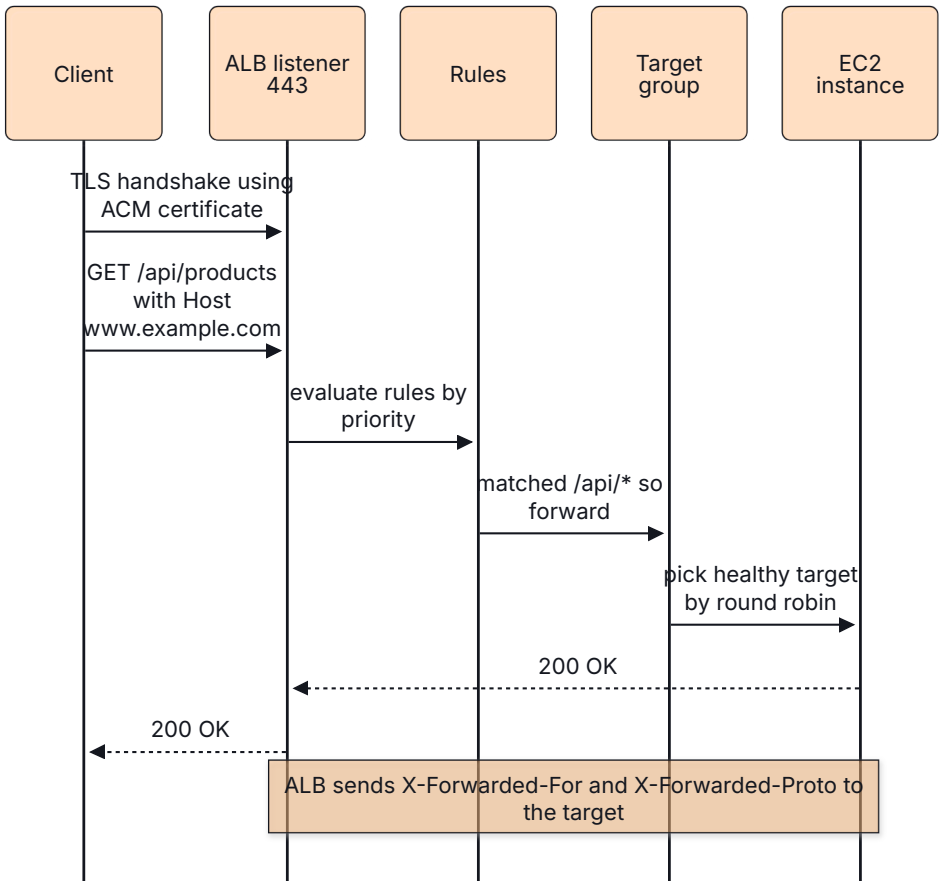


Figure 6.14-1

The `shop-api` entry point:

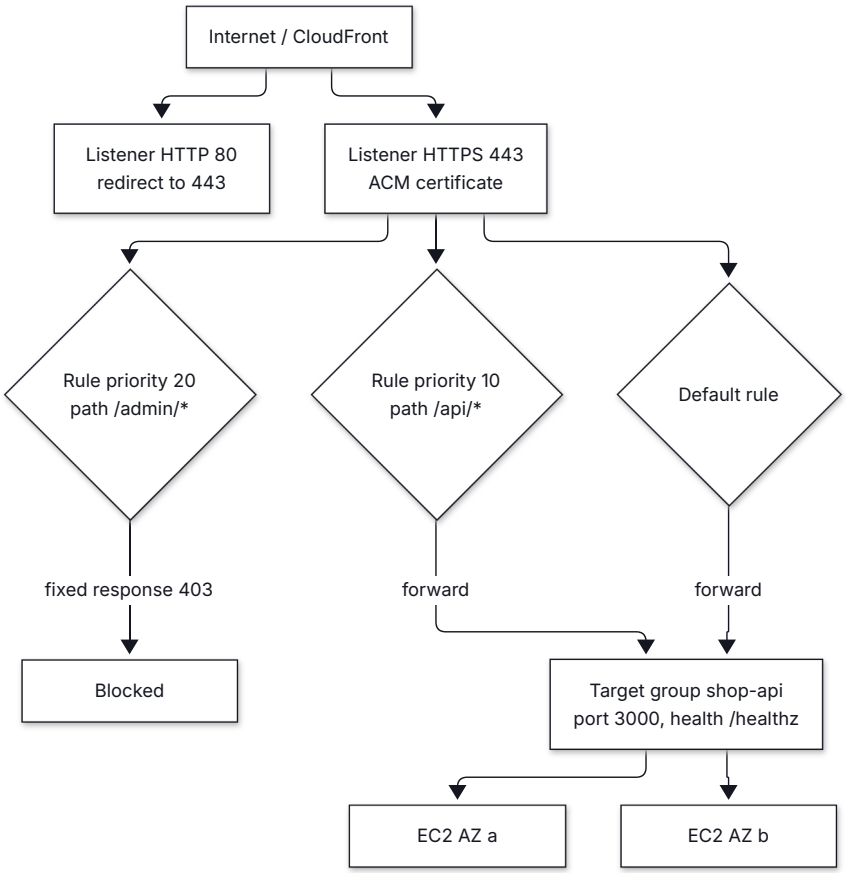


Figure 6.14-2

5 How it works

Components

Component	What it does	Key settings
Load balancer	The entry point; **internet-facing** (public IPs) or **internal** (private IPs only)	needs **at least 2 subnets in 2 different AZs**; a security group; scheme; IP type (IPv4 / dual-stack)
Listener	Checks for connection requests on one protocol and port	`HTTP:80`, `HTTPS:443`; a default action; the TLS certificate and security policy for HTTPS

Component	What it does	Key settings
Rule	Condition(s) + action, evaluated in **priority order** (lowest number first); the default rule is last	conditions: host header, path pattern, HTTP header, query string, source IP, HTTP method
Action	`forward` (to one or several weighted target groups), `redirect`, `fixed-response`, or authenticate (OIDC / Amazon Cognito)	redirect HTTP to HTTPS with a `301`
Target group	A set of targets of one type: `instance`, `ip`, `lambda`, or `alb`	protocol/port, health check, deregistration delay, stickiness, load-balancing algorithm
Target	One backend (instance ID + port, IP + port, a Lambda)	registered manually or by Auto Scaling / ECS / the Kubernetes controller

Health checks are configured on the target group and run from every ALB node:

Setting	Typical default	Meaning
Path	`/`	URL requested; use a dedicated `/healthz`
Interval	30 s	time between checks (5-300 s)
Timeout	5 s	how long to wait for an answer
Healthy threshold	5	consecutive successes to become healthy (2-10)
Unhealthy threshold	2	consecutive failures to become unhealthy (2-10)
Success codes	`200`	accepted codes (ranges such as `200-299` allowed)

A target moves `initial → healthy → unhealthy → draining`. When all targets in a target group are unhealthy, the ALB **fails open** and routes to all of them rather than returning errors for everything — a behavior that surprises people. When targets are removed or deregistered the ALB waits for the **deregistration delay** (default 300 s) to let in-flight requests finish; lower it (for example to 30 s) for fast-responding APIs so deployments are quicker.

Load-balancing algorithms (per target group): `round_robin` (default), `least_outstanding_requests` (better when request durations vary a lot) and `weighted_random` (enables anomaly mitigation). Optional **stickiness** uses a cookie to keep a client on one target — avoid it unless sessions are truly local to the instance (prefer ElastiCache §6.11 for sessions). **Slow start** mode eases a new target into rotation.

TLS with ACM. Request a free public certificate in ACM in the **same region as the ALB** (contrast with CloudFront §6.13, which needs `us-east-1`), prove you own the domain by **DNS validation** (one CNAME record, which Route 53 §6.12 can create for you), and attach it to the HTTPS listener. ACM renews it automatically as long as the validation record stays in place. One listener can carry many certificates, and the ALB chooses between them by **SNI** (Server

Name Indication). Pick a modern security policy such as `ELBSecurityPolicy-TLS13-1-2-2021-06` rather than relying on the default, and redirect port 80 to 443. The ALB **terminates TLS** by default: traffic to the targets is plain HTTP unless you set the target group protocol to HTTPS. For the concept, see TLS Termination §12.11.

Headers your app must understand. The target sees the ALB's IP as the source, so the real client address arrives in `X-Forwarded-For`, the original scheme in `X-Forwarded-Proto` and the port in `X-Forwarded-Port`. Configure the framework to trust the proxy (in Express: `app.set('trust proxy', true)`), or redirects and secure cookies will misbehave.

Reading the error codes

Code	Who generated it	Usual cause
`502 Bad Gateway`	ALB	target closed the connection, sent an invalid response, or its keep-alive timeout is **shorter than the ALB idle timeout (60 s)**
`503 Service Unavailable`	ALB	no registered targets, or the target group has none available
`504 Gateway Timeout`	ALB	target did not respond within the idle timeout, or a security group/NACL blocks the ALB from the target
Any `5xx` produced by your app	target	counted as `HTTPCode_Target_5XX_Count` (not `HTTPCode_ELB_5XX_Count`) — the application, not the ALB

Security groups. The ALB has its own security group (inbound 80/443 from the world or from CloudFront). The instances' security group should allow the app port **only from the ALB's security group** — never from `0.0.0.0/0` (Security Groups §6.6).

Metrics to watch (namespace `AWS/ApplicationELB`): `HTTPCode_Target_5XX_Count`, `HTTPCode_ELB_5XX_Count`, `TargetResponseTime`, `HealthyHostCount`, `UnHealthyHostCount`, `RequestCountPerTarget`, `ActiveConnectionCount`. Turn on **access logs** to an S3 bucket for per-request investigation. **Cost:** an hourly charge per ALB plus Load Balancer Capacity Units (LCUs) for traffic; an ALB with no traffic still costs money every hour. Free-tier terms differ by account age (older accounts had 750 ALB hours a month for 12 months, newer accounts use credit-based plans), so check the Billing console's Free Tier page.

6 Step-by-step guide

Prerequisites: AWS CLI v2; the default VPC (it has public subnets in several AZs); two EC2 instances running a web server on port 3000 (see EC2 §6.1; the quick option is two `t3.micro` instances whose user-data starts `python3 -m http.server 3000` serving a folder with a `healthz`

file). **Cost warning:** an ALB costs a few cents per hour plus LCUs (roughly $16-20 a month if left running); delete it when you finish. Your **billing alarm should exist first** (CloudWatch §6.18).

> **No AWS account?** LocalStack's ELBv2 support is in its paid tiers. The free alternative is to practise the *rule logic* with Nginx (step 8).

1. **Variables and security groups.**

```bash
export AWS_REGION=us-east-1
VPC_ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true --query
  'Vpcs[0].VpcId' --output text)
SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID Name=default-
  for-az,Values=true \
  --query 'Subnets[0:2].SubnetId' --output text)
ALB_SG=$(aws ec2 create-security-group --group-name shop-alb-lab --description "shop
  alb lab" \
  --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $ALB_SG --protocol tcp --port 80
  --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $ALB_SG --protocol tcp --port 443
  --cidr 0.0.0.0/0
# allow the ALB to reach the instances (INSTANCE_SG = security group of your two EC2
  instances)
INSTANCE_SG=sg-0123456789abcdef0
aws ec2 authorize-security-group-ingress --group-id $INSTANCE_SG --protocol tcp --port
  3000 --source-group $ALB_SG
```

2. **Create the target group with a health check.**

```bash
TG_ARN=$(aws elbv2 create-target-group --name shop-api-lab --protocol HTTP --port 3000
  \
  --vpc-id $VPC_ID --target-type instance \
  --health-check-path /healthz --health-check-interval-seconds 10 --health-check-
  timeout-seconds 5 \
  --healthy-threshold-count 2 --unhealthy-threshold-count 2 --matcher HttpCode=200 \
  --query 'TargetGroups[0].TargetGroupArn' --output text)
aws elbv2 modify-target-group-attributes --target-group-arn $TG_ARN \
  --attributes Key=deregistration_delay.timeout_seconds,Value=30
aws elbv2 register-targets --target-group-arn $TG_ARN --targets Id=i-0aaaaaaaaaaaaaaaa
  Id=i-0bbbbbbbbbbbbbbbb
```

3. **Create the load balancer and an HTTP listener.**

```bash
ALB_ARN=$(aws elbv2 create-load-balancer --name shop-alb-lab --type application --
  scheme internet-facing \
  --subnets $SUBNETS --security-groups $ALB_SG --query
  'LoadBalancers[0].LoadBalancerArn' --output text)
ALB_DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB_ARN --query
  'LoadBalancers[0].DNSName' --output text)
aws elbv2 wait load-balancer-available --load-balancer-arns $ALB_ARN
HTTP_L=$(aws elbv2 create-listener --load-balancer-arn $ALB_ARN --protocol HTTP --port
  80 \
  --default-actions Type=forward,TargetGroupArn=$TG_ARN --query
  'Listeners[0].ListenerArn' --output text)
echo $ALB_DNS
```

   Expected: a name like `shop-alb-lab-1234567890.us-east-1.elb.amazonaws.com`.

4. **Check target health, then test.**

```bash
aws elbv2 describe-target-health --target-group-arn $TG_ARN \
  --query 'TargetHealthDescriptions[].
  [Target.Id,TargetHealth.State,TargetHealth.Reason]' --output table
curl -i http://$ALB_DNS/healthz
```

   Expected: both targets `healthy` after about 20-30 s (state `initial` first) and `HTTP/1.1 200 OK`. If a target says `unhealthy` with reason `Target.Timeout`, the instance security group is not allowing the ALB group (step 1).

5. **Add path rules.** A fixed response for `/admin/*` and a header rule:

```bash
aws elbv2 create-rule --listener-arn $HTTP_L --priority 20 \
  --conditions Field=path-pattern,Values='/admin/*' \
  --actions Type=fixed-
  response,FixedResponseConfig='{StatusCode=403,ContentType=text/plain,MessageBody=for
  bidden}'
curl -i http://$ALB_DNS/admin/users          # 403 forbidden, never reaches the
  instances
aws elbv2 describe-rules --listener-arn $HTTP_L --query 'Rules[].
  [Priority,Conditions[0].Values[0]]' --output table
```

6. **TLS with ACM.** With a domain in Route 53 (replace `shop.example.com` with a name you

own):

```bash
CERT_ARN=$(aws acm request-certificate --domain-name shop.example.com --validation-
  method DNS \
  --query CertificateArn --output text)
aws acm describe-certificate --certificate-arn $CERT_ARN \
  --query 'Certificate.DomainValidationOptions[0].ResourceRecord'      # create this
  CNAME (Route 53 console has a one-click button)
aws acm wait certificate-validated --certificate-arn $CERT_ARN
aws elbv2 create-listener --load-balancer-arn $ALB_ARN --protocol HTTPS --port 443 \
  --certificates CertificateArn=$CERT_ARN --ssl-policy ELBSecurityPolicy-TLS13-1-2-
  2021-06 \
  --default-actions Type=forward,TargetGroupArn=$TG_ARN
aws elbv2 modify-listener --listener-arn $HTTP_L \
  --default-actions 'Type=redirect,RedirectConfig=
  {Protocol=HTTPS,Port=443,StatusCode=HTTP_301}'
```

No domain? ACM cannot issue a certificate for the `elb.amazonaws.com` name. For a lab, create a self-signed certificate and import it (browsers will warn, `curl -k` will work):

```bash
openssl req -x509 -newkey rsa:2048 -nodes -days 7 -keyout lab.key -out lab.crt -subj
  "/CN=shop.example.com"
CERT_ARN=$(aws acm import-certificate --certificate fileb://lab.crt --private-key
  fileb://lab.key \
  --query CertificateArn --output text)
```

7. **Break and watch a health check.** Stop the app on one instance and observe:

```bash
aws elbv2 describe-target-health --target-group-arn $TG_ARN \
  --query 'TargetHealthDescriptions[].[Target.Id,TargetHealth.State]' --output text
for i in $(seq 1 6); do curl -s -o /dev/null -w "%{http_code}\n"
  http://$ALB_DNS/healthz; done
```

After about 20 s (2 failures x 10 s) the stopped instance becomes `unhealthy` and every response is `200` from the other one.

8. **Free local simulation (Nginx path routing and upstream health).**

```bash
cat > alb-sim.conf <<'EOF'
upstream shop_api { server host.docker.internal:3001 max_fails=2 fail_timeout=10s;
                    server host.docker.internal:3002 max_fails=2 fail_timeout=10s; }
server {
  listen 8080;
  location /admin/ { return 403 "forbidden\n"; }
  location / { proxy_pass http://shop_api; proxy_set_header X-Forwarded-For
  $remote_addr; }
}
EOF
python3 -m http.server 3001 & python3 -m http.server 3002 &
docker run -d --name albsim -p 8080:8080 --add-host=host.docker.internal:host-gateway
  \
  -v "$PWD/alb-sim.conf:/etc/nginx/conf.d/default.conf:ro" nginx:1.27
curl -i localhost:8080/admin/x            # 403
```

9. **Cleanup (do not skip).** Delete in this order: listeners go with the load balancer, then the target group, then security groups.

```bash
aws elbv2 delete-load-balancer --load-balancer-arn $ALB_ARN
sleep 60                                        # wait for network interfaces to
  release
aws elbv2 delete-target-group --target-group-arn $TG_ARN
aws ec2 delete-security-group --group-id $ALB_SG
aws acm delete-certificate --certificate-arn $CERT_ARN      # only if you created one
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'     #
  must not list shop-alb-lab
docker rm -f albsim 2>/dev/null; kill %1 %2 2>/dev/null
```

Remember to remove the `INSTANCE_SG` rule you added and to terminate the two EC2 instances.

7 Real-world scenario

Illustrative scenario. After a release, `shop-api` shows a trickle of `502` errors — about 1 in 300 requests — only from the ALB, and only on requests sent over reused connections. **Investigation:** `HTTPCode_ELB_502_Count` rises while `HTTPCode_Target_5XX_Count` is flat, so the application never produced these errors. The ALB access log shows `target_status_code` of `-` with an ELB 502. The Node.js server's default HTTP keep-alive timeout is 5 seconds, shorter than the ALB's 60-second idle timeout; the server closes an idle connection just as the ALB sends a request on it. **Fix:** set `server.keepAliveTimeout = 65000` (and `headersTimeout` slight-

ly above it) so the target always outlives the load balancer's idle timer. **Lesson:** a 502 from the ALB is a conversation between the ALB and a target that went wrong; check which side produced the status code before you touch the app.

8 Common mistakes and best practices

Don't

- Use `/` as the health check when that page renders the whole site; use a lightweight `/healthz`.
- Put database or third-party calls in the health check so a database blip removes every target at once.
- Allow the instance security group from `0.0.0.0/0`; allow only the ALB security group.
- Forget that the app sees the proxy's IP; logging or rate limiting by remote address will then punish everyone equally.
- Leave test ALBs running — they bill hourly even with no traffic.
- Keep the 300 s deregistration delay on a fast API; deployments crawl.

Do

- Spread the ALB and its targets across at least two AZs; keep instances in private subnets and the ALB in public ones.
- Redirect HTTP to HTTPS and choose a current TLS policy; let ACM manage renewals (keep the DNS validation record).
- Use one shared ALB with host- and path-based rules for many small services where it makes sense.
- Alarm on `UnHealthyHostCount`, `HTTPCode_Target_5XX_Count` and `TargetResponseTime` in CloudWatch §6.18; enable access logs for investigations.
- Attach the target group to an Auto Scaling group §6.2 and scale on `ALBRequestCountPerTarget`.
- Make the ALB itself code (Terraform §7.1).

9 Tasks

- [ ] **T1 — Put two instances behind an ALB** *(≈60 min, beginner)*
  - **Goal:** a public DNS name that load balances two EC2 instances.
  - **Steps:**
    1. Launch two instances that each return their own instance ID at `/` and `OK` at `/healthz` on port 3000.

2. Complete steps 1-4 of the guide.
3. Run `for i in $(seq 1 10); do curl -s http://$ALB_DNS/; done`.

- **Done when:** the output shows both instance IDs, roughly alternating, and `describe-target-health` shows both `healthy`.

- [ ] **T2 — Add rules and a redirect** *(≈45 min, intermediate)*
  - **Goal:** route by path and block a path.
  - **Steps:**
    1. Create the `/admin/*` fixed-response rule at priority 20.
    2. Add a second rule at priority 10 for `/api/*` that forwards to the target group.
    3. Change the HTTP listener default action to a redirect once you have a certificate (task T3), or redirect only a path pattern in the meantime.
  - **Done when:** `/admin/x` returns `403 forbidden` and `curl -sI` of an HTTP URL shows `301` with a `Location: https://…` header.

- [ ] **T3 — HTTPS with ACM** *(≈60 min, intermediate)*
  - **Goal:** a trusted certificate on the HTTPS listener.
  - **Steps:**
    1. Request an ACM certificate for a hostname you control and validate it via DNS (a Route 53 alias record can then point the name to the ALB).
    2. Create the 443 listener with the TLS 1.3/1.2 policy.
    3. Run `curl -vI https://shop.example.com 2>&1 | grep -E "subject|issuer|TLS"` (use your name). If you have no domain, use the imported self-signed certificate with `curl -k`.
  - **Done when:** the request succeeds over TLS and you can name the certificate issuer, and ACM shows status `Issued` (or `Imported`).

- [ ] **T4 — Break the health check** *(≈45 min, intermediate)*
  - **Goal:** recognize each failure mode from its symptom.
  - **Steps:**
    1. Change the health check path to `/nope`; observe `unhealthy` and the response code you get from the ALB (the fail-open behavior).
    2. Restore it; remove the security group rule that allows the ALB to the instances; note the target reason (`Target.Timeout`) and the user-visible `504`.
    3. Deregister both targets and note the `503`.

- **Done when:** you have a three-row table of change → target health reason → HTTP status seen by the client, verified with `curl -i`.

- [ ] **T5 — Observe and alarm** *(≈45 min, intermediate)*
  - **Goal:** metrics and a log-based view of traffic.
  - **Steps:**
    1. Enable access logs to an S3 bucket (`aws elbv2 modify-load-balancer-attributes --attributes Key=access_logs.s3.enabled,Value=true Key=access_logs.s3.bucket,Value=<bucket>` — the bucket needs the documented ELB log-delivery policy).
    2. Generate 50 requests, including some to a non-existent path.
    3. Create a CloudWatch alarm on `UnHealthyHostCount` greater than 0 for one minute.
  - **Done when:** a log file shows your requests in S3 and the alarm enters `ALARM` when you stop the app on one instance.

- [ ] **T6 — Prove it: blue/green with weighted target groups** *(≈60 min, advanced)*
  - **Goal:** shift traffic between two target groups without downtime, then tear everything down.
  - **Steps:**
    1. Without the guide, create a second target group `shop-api-green` with one instance that returns a different version string.
    2. Change the listener default action to forward 90/10 (`Type=forward,ForwardConfig={TargetGroups=[{TargetGroupArn=…,Weight=90},{TargetGroupArn=…,Weight=10}]}`), measure with 100 requests, then move to 0/100.
    3. Explain how this gives instant rollback, then run the cleanup (step 9).
  - **Done when:** your 100-request sample shows approximately a 90/10 split, the final state serves only the green version, and no load balancers remain.

10 Self-check questions

1. What is the difference between a listener, a rule, and a target group?
2. An ALB returns `504` for some requests and `502` for others. What does each tell you and where do you look first?
3. Why do you need at least two subnets in different AZs, and why is the instance security group tied to the ALB's group?
4. Where must the ACM certificate live for an ALB, and what keeps it renewing?
5. What happens when every target in a target group is unhealthy?

6. How does your app find out the real client IP and whether the client used HTTPS?

ANSWERS

1. A listener is the protocol/port the ALB listens on; rules attached to it match requests (host, path, header…) and choose an action; a target group is the set of backends (with health checks) a forward action sends traffic to.
2. `504`: the target did not answer in time, or the network path (security group/NACL) blocks the ALB; look at `TargetResponseTime`, target health reason and the SG. `502`: the target broke the connection or sent an invalid response, often a keep-alive mismatch or a crash; compare ELB 5XX against Target 5XX and check the access logs.
3. The ALB nodes must exist in multiple AZs to survive an AZ failure; referencing the ALB's security group means only the load balancer, not the internet, can reach the instances.
4. In ACM in the same region as the ALB; auto-renewal works while the DNS validation CNAME remains in place.
5. The ALB fails open and sends traffic to all targets (so a bad health check does not black-hole everything), and `UnHealthyHostCount` alarms should already be firing.
6. From `X-Forwarded-For` and `X-Forwarded-Proto`, which the framework must be told to trust.

11 Further reading

- *Elastic Load Balancing User Guide: Application Load Balancers* (docs.aws.amazon.com/elasticloadbalancing) — listeners, rules, target groups, health checks, access logs.
- *AWS Certificate Manager User Guide* — DNS validation, renewal, regional behavior.
- *AWS Well-Architected Framework: Reliability pillar* — health checks and failure isolation.
- *Designing Data-Intensive Applications* by Martin Kleppmann — why load balancers and health checking matter in distributed systems.

6.15 KMS

Topic 15 of 18 | ~4 hours | Difficulty: Intermediate

1 What it is

AWS Key Management Service (KMS) is the managed service that creates, stores and uses **encryption keys** for you. The defining idea is that the key itself **never leaves KMS in plaintext**: you send data (or a small key) to the KMS API, KMS performs the cryptographic operation inside AWS-operated hardware security modules (HSMs), and returns the result. Who may ask is controlled by IAM policies *and* a **key policy** attached to the key, and every call is recorded in CloudTrail §6.17.

What KMS calls a **KMS key** (formerly "customer master key", CMK) is a *logical* key: a key ID, an ARN, optional **aliases** (friendly names such as `alias/shop-api`), a policy, and the backing key material. Most keys are **symmetric** (AES-256-GCM); KMS also offers asymmetric keys (RSA, ECC) for signing and encryption, and HMAC keys. Because a KMS key can directly encrypt only a few kilobytes (4 KB), real workloads use **envelope encryption**, explained in section 5.

2 Why it matters in DevOps

- **Encryption at rest is a checkbox with consequences:** S3 §6.8, EBS §6.9, RDS §6.10, ElastiCache §6.11, Secrets Manager §6.16, CloudWatch Logs and CloudTrail can all encrypt with KMS. Choose the key wrongly and you cannot share a snapshot, copy an AMI across accounts, or start instances.
- **It is a second permission system:** an engineer with full S3 access still cannot read an object encrypted with a key that their role may not use. That separation is the point — and the most common cause of mysterious `AccessDenied` errors.
- **Deleting a key destroys the data:** once the key is gone, everything encrypted under it is unrecoverable. Understanding the waiting period and "disable instead of delete" prevents career-limiting mistakes.
- **Compliance and audit:** PCI-DSS, HIPAA and SOC 2 reviews ask who can use which key, how it rotates, and where the evidence is.

- **Cost surprise:** a customer managed key costs a monthly fee plus per-request charges; chatty applications calling KMS for every record can rack up cost and hit throttling.

3 Real-world system example

S3 server-side encryption with KMS (SSE-KMS) is the clearest documented example of envelope encryption at scale. For each object S3 asks KMS to generate a unique **data key**, encrypts the object with that data key, and stores the *encrypted* data key alongside the object; to read it, S3 calls KMS to decrypt the data key. AWS documents the feature called **S3 Bucket Keys**, which reduces the number of KMS requests (and cost) by having S3 reuse a bucket-level key for many objects. EBS and RDS follow the same pattern for volumes and database storage. The **AWS Encryption SDK** packages the same envelope technique for application code. For the broader view on how encryption and secrets fit together, see Secrets Management and Encryption §11.4.

4 Diagram

Envelope encryption — the key idea of this topic:

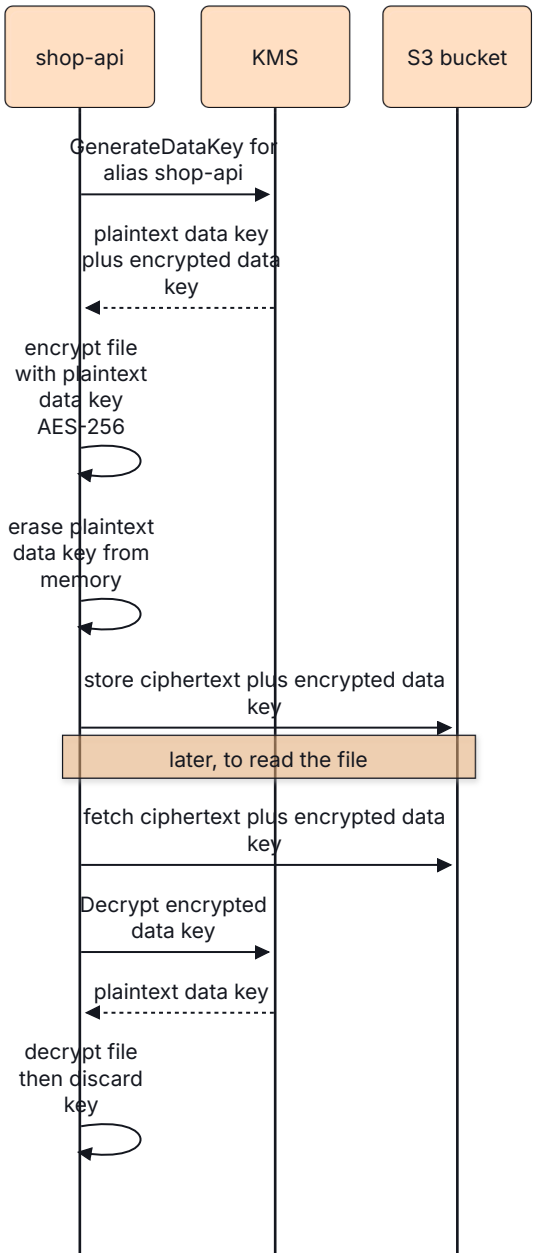


Figure 6.15-1

Who can use a key — both layers must agree:

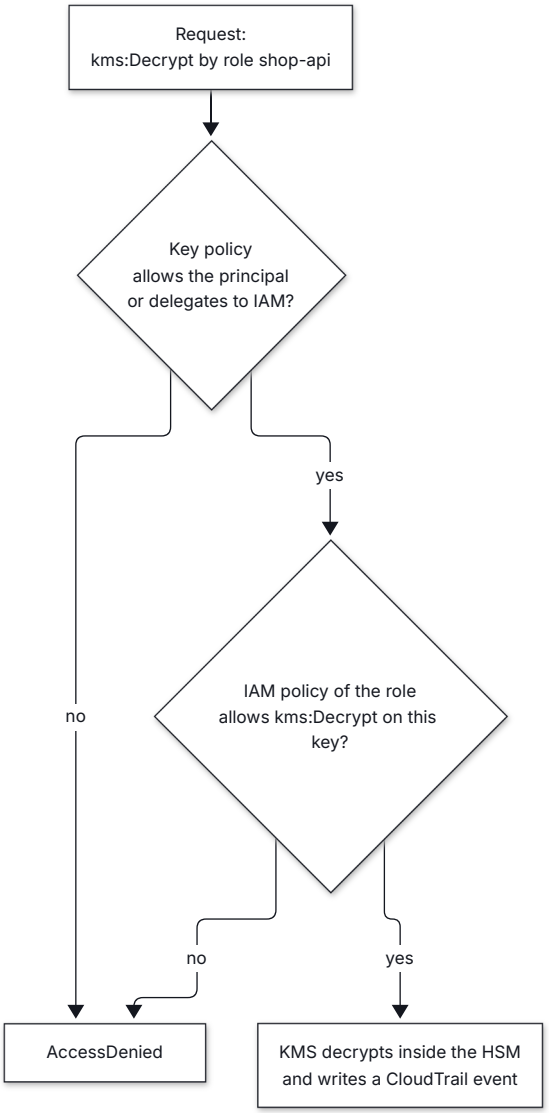


Figure 6.15-2

5 How it works

Key types

Type	Who manages it	Cost	Notes
AWS owned keys	AWS, invisible to you	free	used by default for some services (for example default DynamoDB encryption); no audit trail in your account
AWS managed keys (`aws/s3`, `aws/ebs`, `aws/rds`…)	AWS creates them in your account when you first use a service	no monthly fee; per-request fees may apply	you can view them and their usage in CloudTrail, but you cannot edit the key policy, and rotation is automatic and fixed (yearly)
Customer managed keys	you	monthly fee per key plus per-request charges	custom key policy, aliases, rotation settings, cross-account sharing, disable/delete

Symmetric vs asymmetric. Symmetric keys (default, AES-256-GCM) are what AWS services use for encryption at rest. Asymmetric keys (RSA, ECC) have a public/private pair, where the private half never leaves KMS — used for signing (for example code signing, JWT signing) or for encrypting data for parties outside AWS. HMAC keys generate and verify message authentication codes.

Envelope encryption. KMS encrypts only small payloads directly. For larger data you call `GenerateDataKey`, which returns the data key twice: once in plaintext (use it, then erase it) and once encrypted under your KMS key (store it next to the data). To decrypt, send the encrypted data key to `Decrypt`. Benefits: data never travels to KMS (speed, no 4 KB limit), each object can have its own key, and re-keying only needs the small keys to be re-wrapped.

Key policies and grants. A **key policy** is the primary access control — a KMS key has *no access* unless its key policy allows it. The standard default policy has one statement, "Enable IAM User Permissions", that delegates to IAM for the account:

```json
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "EnableIAMUserPermissions", "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:root" },
      "Action": "kms:*", "Resource": "*" },
    { "Sid": "AllowShopApiRoleToUseKey", "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:role/shop-api-ec2" },
      "Action": ["kms:Decrypt", "kms:GenerateDataKey"], "Resource": "*",
      "Condition": { "StringEquals": { "kms:EncryptionContext:app": "shop-api" } } }
  ]
}
```

Best practice is to separate **key administrators** (can manage but not use the key) from **key users** (can encrypt/decrypt but not delete or change the policy). **Grants** are temporary, programmatic permissions that AWS services (EBS, RDS, Auto Scaling) create on your behalf. The `kms:ViaService` condition can limit a key to use *through* a specific service, such as `s3.us-east-1.amazonaws.com`, so a stolen role cannot call KMS directly.

Encryption context. An optional set of key-value pairs bound to the ciphertext as additional authenticated data. Decryption fails unless the same context is supplied, and the context appears in CloudTrail logs — so never put secrets in it, and use it to bind data to a purpose (`app=shop-api`).

Rotation. For symmetric customer managed keys you can turn on **automatic rotation**: KMS creates new key material on a schedule (default every 365 days, configurable from 90 to 2,560 days), keeps the old material so existing ciphertext still decrypts, and the key ID, ARN and aliases do not change. Applications notice nothing. Keys with imported key material, asymmetric keys and HMAC keys do not rotate automatically; rotating those means creating a new key and repointing the alias. Rotating the material does not re-encrypt existing data; for that, re-encrypt (`ReEncrypt`) or rewrite the objects. Additional charges apply for rotated key versions.

Deletion. `ScheduleKeyDeletion` puts a key in `PendingDeletion` for **7 to 30 days** (30 by default). During that window the key cannot be used and the deletion can be cancelled. When it ends the key and all data encrypted under it are permanently unrecoverable. **Disabling** a key is reversible and is usually what you want.

Other things to know. *Multi-Region keys* (`mrk-…`) have related key material in several regions for cross-region decrypt/DR. *Custom key stores* (CloudHSM / external key store) exist for regulatory needs. KMS has per-account request quotas (shared across keys in a region), and cryptographic operation requests have a modest per-request fee, with a monthly free allowance in the free tier. KMS HSMs are validated under FIPS 140 programs; see AWS documentation for the current level.

6 Step-by-step guide

Prerequisites: AWS CLI v2 (and `base64`, `openssl`). **Cost warning:** a customer managed key costs about **$1 per month** (prorated hourly) from creation until it is *deleted*, which takes at least 7 days of waiting — so the cleanup is "schedule deletion", and you pay a few cents. Request charges are tiny for a lab, and the first 20,000 requests a month are in the free tier. Do the cleanup and keep your billing alarm (CloudWatch §6.18).

> **LocalStack:** KMS is emulated in the free LocalStack. Run `docker run -d -p 4566:4566 localstack/localstack`, then add `--endpoint-url http://localhost:4566` to every command (with `AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_REGION=us-east-1`). Key policies are stored but not strictly enforced locally, so the policy experiments need the real service.

1. **Create a key and an alias.**

```bash
export AWS_REGION=us-east-1
KEY_ID=$(aws kms create-key --description "shop-api lab key" --tags
  TagKey=project,TagValue=shop-api \
  --query KeyMetadata.KeyId --output text)
aws kms create-alias --alias-name alias/shop-api-lab --target-key-id $KEY_ID
aws kms describe-key --key-id alias/shop-api-lab \
  --query 'KeyMetadata.
  {Id:KeyId,State:KeyState,Spec:KeySpec,Usage:KeyUsage,Rotation:KeyRotationStatus}' --
  output table
```

   Expected: `Enabled`, `SYMMETRIC_DEFAULT`, `ENCRYPT_DECRYPT`.

2. **Encrypt and decrypt a small value directly (limit 4 KB).**

```bash
echo -n 'db-password-example' > plain.txt
aws kms encrypt --key-id alias/shop-api-lab --plaintext fileb://plain.txt \
  --encryption-context app=shop-api \
  --query CiphertextBlob --output text | base64 --decode > secret.bin
aws kms decrypt --ciphertext-blob fileb://secret.bin --encryption-context app=shop-api
  \
  --query Plaintext --output text | base64 --decode; echo
```

   Expected: `db-password-example`. Note you did not pass the key ID to `decrypt` — it is embedded in the ciphertext. Run the decrypt again with `--encryption-context app=other` and see `InvalidCiphertextException`.

3. **Envelope-encrypt a file (the real pattern).**

```bash
echo "card=4111-example" > customer.txt
aws kms generate-data-key --key-id alias/shop-api-lab --key-spec AES_256 --encryption-context app=shop-api > dk.json
jq -r .Plaintext dk.json > dk.plain.b64                                    # plaintext data key (base64 text)
jq -r .CiphertextBlob dk.json | base64 --decode > dk.encrypted             # store this next to the data
openssl enc -aes-256-cbc -pbkdf2 -in customer.txt -out customer.enc -pass file:dk.plain.b64
shred -u dk.plain.b64 dk.json 2>/dev/null || rm -f dk.plain.b64 dk.json    # erase the plaintext key
ls -l customer.enc dk.encrypted
```

(`openssl enc` keeps the lab short; production code should use an authenticated mode such as AES-GCM, usually via the AWS Encryption SDK.)

4. **Decrypt it again: ask KMS for the key, then use it.**

```bash
aws kms decrypt --ciphertext-blob fileb://dk.encrypted --encryption-context app=shop-api \
  --query Plaintext --output text > dk.plain.b64
openssl enc -d -aes-256-cbc -pbkdf2 -in customer.enc -pass file:dk.plain.b64
rm -f dk.plain.b64
```

Expected: `card=4111-example`.

5. **Turn on rotation and inspect the policy.**

```bash
aws kms enable-key-rotation --key-id $KEY_ID --rotation-period-in-days 365
aws kms get-key-rotation-status --key-id $KEY_ID
aws kms get-key-policy --key-id $KEY_ID --policy-name default --query Policy --output text | jq .
```

The default policy created by the CLI is the "Enable IAM User Permissions" statement, so the IAM policies on your own user work.

6. **Use the key from S3 (SSE-KMS with a Bucket Key).**

```bash
BUCKET=shop-kms-lab-$(aws sts get-caller-identity --query Account --output text)
aws s3api create-bucket --bucket $BUCKET
aws s3api put-bucket-encryption --bucket $BUCKET --server-side-encryption-
  configuration \
  "{\"Rules\":[{\"ApplyServerSideEncryptionByDefault\":
  {\"SSEAlgorithm\":\"aws:kms\",\"KMSMasterKeyID\":\"$KEY_ID\"},\"BucketKeyEnabled\":t
  rue}]}"
aws s3 cp customer.txt s3://$BUCKET/customer.txt
aws s3api head-object --bucket $BUCKET --key customer.txt --query
  '{SSE:ServerSideEncryption,Key:SSEKMSKeyId,BucketKey:BucketKeyEnabled}'
```

7. **See the audit trail.** KMS calls appear in CloudTrail within about 15 minutes:

```bash
aws cloudtrail lookup-events --lookup-attributes
  AttributeKey=EventSource,AttributeValue=kms.amazonaws.com \
  --max-results 10 --query 'Events[].[EventTime,EventName,Username]' --output table
```

8. **Cleanup (do not skip).**

```bash
aws s3 rm s3://$BUCKET --recursive && aws s3api delete-bucket --bucket $BUCKET
aws kms delete-alias --alias-name alias/shop-api-lab
aws kms schedule-key-deletion --key-id $KEY_ID --pending-window-in-days 7
aws kms describe-key --key-id $KEY_ID --query 'KeyMetadata.[KeyState,DeletionDate]' --
  output text   # PendingDeletion
rm -f plain.txt secret.bin customer.* dk.encrypted
```

Billing for the key stops once it is in `PendingDeletion`. If you change your mind inside the window: `aws kms cancel-key-deletion --key-id $KEY_ID`.

7 Real-world scenario

Illustrative scenario. The `shop-api` team encrypts its AMIs and EBS volumes with a customer managed key and enables an Auto Scaling group §6.2 in a new account. During a traffic spike, scale-out fails: instances launch and terminate within seconds, and the group's activity history reports that the launch failed because of a KMS key permission (`Client.InvalidKMSKey.InvalidState` or an access error on the key). **Investigation:** the key policy lists only the human admins and the application role. EBS volume creation for Auto Scaling is performed with the account's **service-linked role** `AWSServiceRoleForAutoScaling`, which was never allowed to use the key. **Fix:** add the service-linked role to the key policy (or

create a grant for it) with `kms:Decrypt`, `kms:GenerateDataKey*`, `kms:ReEncrypt*`, `kms:CreateGrant` and `kms:DescribeKey`. **Lesson:** with customer managed keys, the *service* acting on your behalf is a principal too; test scaling in a staging account before relying on it, and keep the key policy in version control.

8 Common mistakes and best practices

Don't

- Schedule key deletion to "clean up" without proving nothing depends on the key — every object, snapshot and secret under it becomes unreadable.
- Grant `kms:*` to application roles; they need `Decrypt` and `GenerateDataKey`, nothing else.
- Put secrets or personal data in the **encryption context** — it is logged in plaintext.
- Create one key per object or call KMS per record; use envelope encryption or S3 Bucket Keys to keep request counts and cost down.
- Remove the account-root statement from the key policy without an alternative — you can lock yourself out (and then need AWS Support).
- Copy a snapshot or AMI encrypted with an AWS managed key to another account — it cannot be shared; use a customer managed key.

Do

- Use customer managed keys where you need audit, sharing, or custom policies, and AWS managed keys where defaults suffice.
- Use aliases in code and templates (`alias/shop-api`) so you can swap the underlying key.
- Enable automatic rotation on symmetric customer managed keys; alarm on `ScheduleKeyDeletion` and `DisableKey` events (see CloudTrail §6.17).
- Separate key administrators from key users, and constrain with `kms:ViaService` and `kms:EncryptionContext` conditions.
- Enable default EBS encryption in every region and require encryption in S3 with bucket policies/defaults.
- Define keys and their policies as code (Terraform §7.1).

9 Tasks

- [ ] **T1 — Create a key and round-trip a secret** *(≈30 min, beginner)*
  - **Goal:** use KMS directly.
  - **Steps:**
    1. Create a key with alias `alias/shop-api-lab` (step 1).

2. Encrypt `db-password-example` with `aws kms encrypt` and decrypt it (step 2).
3. Run `describe-key` and note the key state, spec and usage.

- **Done when:** decrypt returns the original text and the ciphertext file is binary garbage when you `xxd` it.

- [ ] **T2 — Envelope encryption by hand** *(≈45 min, intermediate)*
  - **Goal:** understand why the plaintext key is thrown away.
  - **Steps:**
    1. Follow steps 3 and 4 to encrypt and decrypt a 1 MB file (`head -c 1M /dev/urandom > big.bin`) — note that direct `kms encrypt` on it fails at 4 KB.
    2. Show that `dk.encrypted` is safe to store beside the data.
    3. Draw the sequence from section 4 from memory.
  - **Done when:** the decrypted file's `sha256sum` equals the original's, and the direct call fails with a validation error about size.

- [ ] **T3 — Encryption context and policy** *(≈45 min, intermediate)*
  - **Goal:** bind ciphertext to a purpose and see it in logs.
  - **Steps:**
    1. Encrypt with `--encryption-context app=shop-api`; try decrypting with a different context and with none.
    2. Look up the `Decrypt` event in CloudTrail (`lookup-events`) and find the context in the event JSON.
    3. Add a key-policy condition requiring `kms:EncryptionContext:app` and verify a call without it is denied.
  - **Done when:** wrong or missing context returns an error and your CloudTrail event shows `encryptionContext`.

- [ ] **T4 — SSE-KMS on S3 and the cross-service denial** *(≈45 min, intermediate)*
  - **Goal:** see the second permission layer.
  - **Steps:**
    1. Configure the bucket default encryption with your key (step 6) and upload a file.
    2. Create an IAM role (or user) with `s3:GetObject` but no KMS permissions; try to read the object.
    3. Add `kms:Decrypt` for that role on the key and retry.
  - **Done when:** the first read fails with `AccessDenied` mentioning KMS and the second succeeds.

- [ ] **T5 — Rotation and the deletion safety net** *(≈30 min, intermediate)*
  - **Goal:** use rotation and the waiting period correctly.
  - **Steps:**
    1. Enable rotation and confirm with `get-key-rotation-status`; explain why existing ciphertext still decrypts.
    2. Disable the key (`disable-key`) and show `decrypt` failing with `DisabledException`; re-enable it.
    3. Schedule deletion with a 7-day window, then cancel it, then schedule it again at the end of the lab.
  - **Done when:** you demonstrated disable, re-enable, schedule and cancel, and can say why disable is the safe option.

- [ ] **T6 — Prove it: alarm on dangerous key actions** *(≈60 min, advanced)*
  - **Goal:** detect a key being disabled or scheduled for deletion.
  - **Steps:**
    1. Without the guide, write an EventBridge rule (or CloudWatch Logs metric filter on a CloudTrail log group) matching `DisableKey` and `ScheduleKeyDeletion` from `kms.amazonaws.com`, with an SNS email target.
    2. Trigger it by disabling and re-enabling your lab key.
    3. Explain in writing what a key admin can do vs a key user, and run the cleanup (step 8).
  - **Done when:** you received an email for the `DisableKey` event, and the key shows `PendingDeletion`.

10 Self-check questions

1. Why is envelope encryption used instead of encrypting data directly with a KMS key?
2. What is the difference between an AWS managed key and a customer managed key?
3. If a role has `kms:Decrypt` in its IAM policy but still gets `AccessDenied`, where do you look?
4. What does automatic rotation change, and what does it not change?
5. Why is "disable the key" preferred over "delete the key"?
6. Why must you never put sensitive values in an encryption context?

ANSWERS

1. KMS only encrypts up to 4 KB per call and sending all data to KMS would be slow and costly; instead KMS protects a small data key that encrypts the data locally.

2. AWS managed keys are created and controlled by AWS for a service, with a fixed policy and rotation and no monthly fee; customer managed keys have your own policy, aliases, rotation options, cross-account sharing and cost a monthly fee.
3. The key policy (must allow the principal or delegate to IAM), grants, conditions such as `kms:ViaService`/encryption context, SCPs, and whether the key is in the right region/account and enabled.
4. It creates new backing key material on a schedule; the key ID, ARN and alias stay the same and old ciphertext still decrypts. It does not re-encrypt existing data.
5. Disabling is reversible; deletion (after the 7-30 day wait) permanently destroys the key and everything encrypted under it.
6. The encryption context is not secret: it is stored/logged in plaintext, for example in CloudTrail.

11 Further reading

- *AWS Key Management Service Developer Guide* (docs.aws.amazon.com/kms) — key policies, grants, rotation, deletion.
- *AWS KMS Cryptographic Details* whitepaper — how the HSM fleet and key hierarchy work.
- *AWS Encryption SDK* documentation — envelope encryption in application code.
- *Security Engineering* by Ross Anderson (free online) — key management in practice.

LAB NOTES What I ran, what I saw, what it means

6.16 Secrets Manager

Topic 16 of 18 | ~4 hours | Difficulty: Intermediate

1 What it is

AWS Secrets Manager stores, encrypts, distributes and **rotates** secrets: database passwords, API keys, OAuth client secrets, TLS private keys — anything that must not live in source code, an AMI or a plain environment file. A secret is a named object (for example `shop/prod/db`) holding a small value, usually a JSON document, encrypted with a KMS §6.15 key. Every change creates a new **version**, and labels called **staging labels** (`AWSCURRENT`, `AWSPREVIOUS`, `AWSPENDING`) say which version is live.

Applications fetch the secret at start-up (or periodically) with an API call authorized by their IAM §6.7 role, so no human ever copies the password into a config file. Secrets Manager can also **rotate** a secret automatically on a schedule — for Amazon RDS it updates the database user's password and the stored secret together, with no application redeploy.

2 Why it matters in DevOps

- **Leaked credentials are the classic breach.** Passwords committed to Git, baked into container images or printed in CI logs are found by scanners within minutes (see Secrets Scanning §11.11). A central store makes "where are our secrets?" answerable.
- **Rotation is how you shrink the blast radius:** a leaked password that expires in 30 days (or hours) is worth much less than one that never changes.
- **Auditability:** every `GetSecretValue` is logged in CloudTrail §6.17 with the caller, so you can answer "who read the production database password?"
- **Operational glue:** EC2 §6.1 instances, ECS tasks, Lambda functions and RDS §6.10 all integrate natively, so there is a supported path from "secret in the vault" to "environment variable in the process".
- **Cost and limits:** a per-secret monthly fee plus per-call charges means you choose deliberately between Secrets Manager and the cheaper Parameter Store (section 5).

3 Real-world system example

Amazon RDS with a managed master user password. AWS documents an RDS option (`--manage-master-user-password`) in which RDS creates the master credentials *in Secrets Manager* for you, encrypts them with KMS, and rotates them on a schedule, so the database administrator password is never known to a human or written in your infrastructure code. Similarly, **Amazon ECS** and **AWS Lambda** can inject a secret value into a container's environment variable or fetch it through a Secrets Manager extension. On the incident-response side, when a SaaS provider announces a breach (for example the January 2023 CircleCI security incident, after which the company urged customers to rotate their stored secrets), organizations with centralized secrets and rotation can change credentials from one place instead of hunting through repositories and pipelines. For the principles, see Secrets Management and Encryption §11.4 and Secrets and Environment Variables in CI/CD §5.5.

4 Diagram

Application retrieval at start-up:

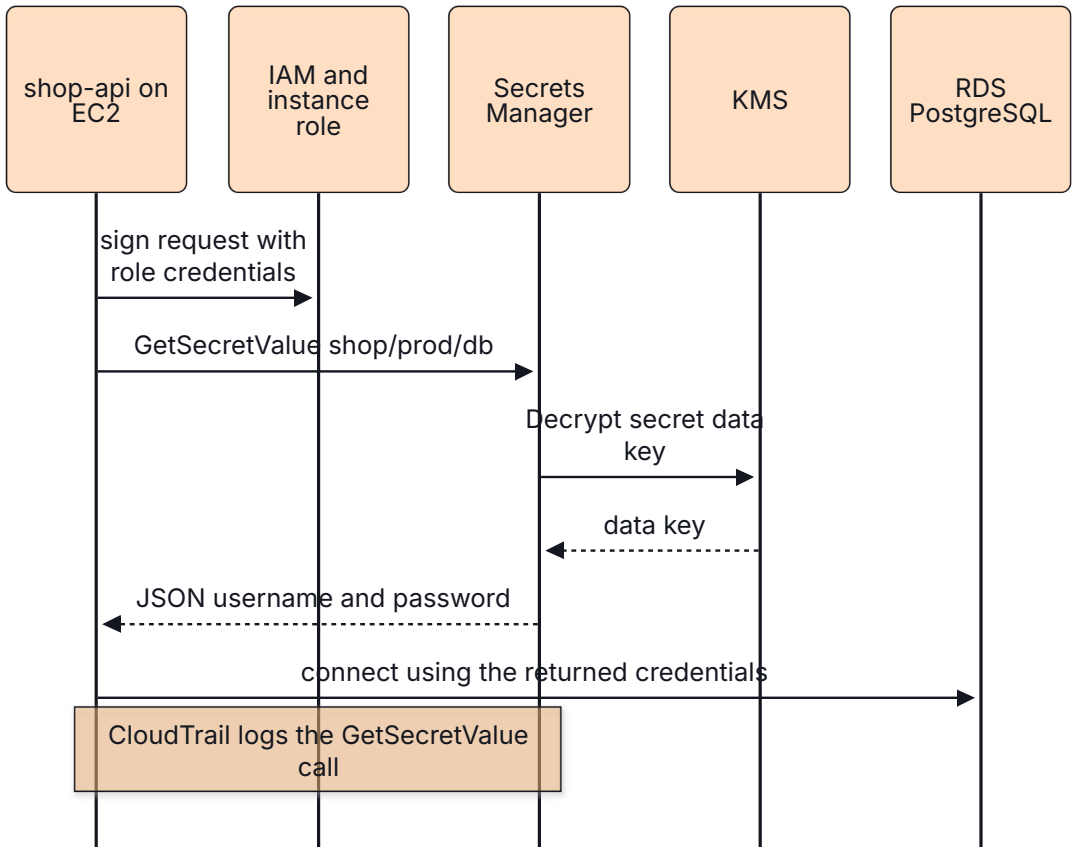


Figure 6.16-1

The rotation state machine (Lambda-driven, alternating through the staging labels):

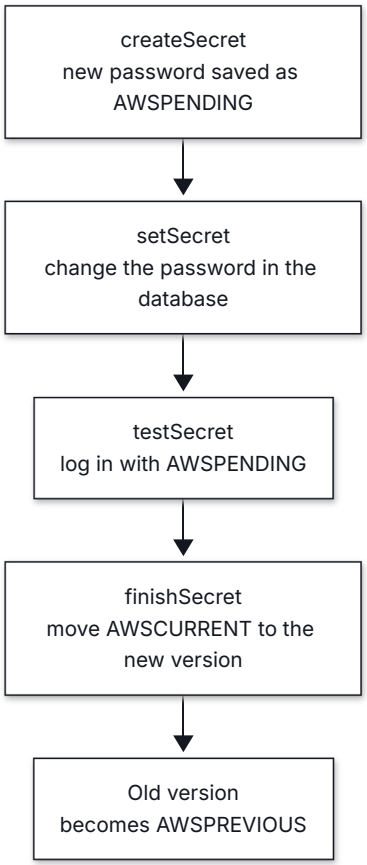


Figure 6.16-2

5 How it works

Secret anatomy. Name (path-style: `shop/prod/db`), ARN (ends in a random six-character suffix such as `shop/prod/db-AbCdEf`), value (`SecretString` up to 64 KB, or `SecretBinary`), the KMS key (default `aws/secretsmanager` or your own customer managed key), resource policy, tags, rotation settings, and optional **multi-region replication**. A typical database secret:

```json
{ "username": "shop_app", "password": "<GENERATED>", "engine": "postgres",
  "host": "shop-db.abc123.us-east-1.rds.amazonaws.com", "port": 5432, "dbname": "shop" }
```

Versions and staging labels. `AWSCURRENT` is what `GetSecretValue` returns by default. During rotation the new value is stored as `AWSPENDING`, tested, then promoted; the old value becomes `AWSPREVIOUS`, which is the rollback path. Versions that carry no label are treated as deprecated and are cleaned up by Secrets Manager over time, so label a version if you need to keep it.

Rotation. Secrets Manager invokes a **rotation Lambda function** with four steps (`createSecret`, `setSecret`, `testSecret`, `finishSecret`). AWS provides templates for RDS, Aurora, Redshift, DocumentDB; for other secrets you write your own function. Two strategies for databases:

Strategy	How	Trade-off
Single-user	Changes the password of the one database user in place	Simple; short window where open connections or caches hold the old password; fine when apps reconnect on auth failure
Alternating (multi-user)	Keeps two users (`shop_app` and `shop_app_clone`), switches which one is current at each rotation	No downtime window; needs a second user and privileges (the superuser secret)

The rotation Lambda must be able to reach both the database (run it in the VPC) and the Secrets Manager API (a NAT gateway or an interface VPC endpoint) — a very common cause of failed rotations. Applications must **fetch again or retry** after an authentication error, or cache with a short TTL; otherwise rotation causes an outage. RDS's managed master password integration removes the Lambda from the picture for the master user.

Retrieving secrets

Method	Notes
SDK / CLI call `GetSecretValue`	Simple; add a caching client (AWS provides caching libraries) to cut cost and latency
ECS task definition `secrets` / `valueFrom`	Injects the value as an environment variable when the task starts; needs the **task execution role** permission
Lambda	Use the AWS Parameters and Secrets Lambda Extension to cache locally
EKS	Secrets Store CSI driver with the AWS provider mounts secrets as files (see Secrets in Kubernetes §8.7)
CloudFormation / Terraform dynamic reference	Resolves at deploy time; take care it does not land in state or logs unencrypted

Permissions. Two-part: an IAM policy on the caller (`secretsmanager:GetSecretValue` on the secret's ARN, plus `kms:Decrypt` if you use a customer managed key) and optionally a **resource policy** on the secret (useful for cross-account access). Because the ARN has a random

suffix, policies use a wildcard: `arn:aws:secretsmanager:us-east-1:111122223333:secret:shop/prod/db-*`.

Deletion. `delete-secret` schedules deletion with a **recovery window of 7-30 days** (default 30), during which `restore-secret` works; `--force-delete-without-recovery` is immediate and irreversible.

Secrets Manager vs SSM Parameter Store

	Secrets Manager	Systems Manager Parameter Store
Purpose	Secrets that benefit from rotation and lifecycle management	Configuration and (cheaply) secrets
Cost	Per secret per month plus per 10,000 API calls (about $0.40 and $0.05 at the time of writing; new secrets get a 30-day free trial)	**Standard** parameters: no charge (up to 10,000 per region, 4 KB each). **Advanced**: small per-parameter fee, 8 KB, parameter policies
Rotation	Built in (Lambda, managed rotation for RDS)	None built in (use EventBridge + your own code)
Encryption	Always KMS	`SecureString` uses KMS; `String` is plaintext
Size	64 KB	4 KB standard / 8 KB advanced
Versioning	Labels (`AWSCURRENT` , ...)	Version numbers and labels
Cross-account / replicas	Resource policy, multi-region replication	Cross-account sharing only for advanced parameters (via RAM)
Typical use	Database credentials, third-party API keys, anything rotated	Feature flags, endpoints, `LOG_LEVEL` , non-rotating tokens, AMI IDs

Rule of thumb: **config goes in Parameter Store, credentials that rotate go in Secrets Manager.** Parameter Store can even read Secrets Manager secrets by the reserved path `/aws/reference/secretsmanager/<name>`.

6 Step-by-step guide

Prerequisites: AWS CLI v2, `jq` , optionally Node.js 20+. **Cost warning:** each secret costs about $0.40 per month after the 30-day free trial; the lab uses one or two secrets for under an hour, so it is effectively free, but secrets scheduled for deletion keep billing until they are deleted — use the force-delete option in cleanup for lab secrets only. A rotation Lambda adds negligible cost.

> **LocalStack:** the free image emulates Secrets Manager well for create/get/put/delete. Start it with `docker run -d -p 4566:4566 localstack/localstack` and use `export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test`

> `AWS_SECRET_ACCESS_KEY=test AWS_REGION=us-east-1` (the AWS CLI v2 honors `AWS_ENDPOINT_URL`). Real rotation against RDS needs the real service.

1. **Generate a password and create the secret** (never type it into the command line, which lands in shell history):

```bash
export AWS_REGION=us-east-1
PW=$(aws secretsmanager get-random-password --password-length 32 --exclude-punctuation
  --query RandomPassword --output text)
jq -n --arg u shop_app --arg p "$PW" --arg h "shop-db.example.internal" \
  '{username:$u,password:$p,engine:"postgres",host:$h,port:5432,dbname:"shop"}' >
  secret.json
aws secretsmanager create-secret --name shop/lab/db --description "shop-api lab
  database credentials" \
  --secret-string file://secret.json --tags Key=project,Value=shop-api
rm secret.json; unset PW
```

   Expected: JSON with `ARN`, `Name` and `VersionId`.

2. **Read it back (and see how reads are logged).**

```bash
aws secretsmanager get-secret-value --secret-id shop/lab/db --query SecretString --
  output text | jq .
aws secretsmanager describe-secret --secret-id shop/lab/db --query
  '{Name:Name,Versions:VersionIdsToStages}'
```

   The versions map shows one version labeled `AWSCURRENT`.

3. **Update the value and watch the labels move.**

```bash
aws secretsmanager put-secret-value --secret-id shop/lab/db \
  --secret-string "$(aws secretsmanager get-secret-value --secret-id shop/lab/db --
  query SecretString --output text | jq -c '.password="changed-by-hand-example"')"
aws secretsmanager describe-secret --secret-id shop/lab/db --query VersionIdsToStages
aws secretsmanager get-secret-value --secret-id shop/lab/db --version-stage
  AWSPREVIOUS --query SecretString --output text | jq -r .password   # the old one
```

   Expected: two versions, one `AWSCURRENT` and one `AWSPREVIOUS`.

4. **Least-privilege access for the app role.** Attach this policy to the EC2 instance role from

IAM §6.7 (replace the account ID):

```json
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": "secretsmanager:GetSecretValue",
    "Resource": "arn:aws:secretsmanager:us-east-1:111122223333:secret:shop/lab/db-*" }
  ] }
```

Add `kms:Decrypt` on your key if the secret uses a customer managed key.

5. **Fetch it from the application** (Node.js, AWS SDK for JavaScript v3):

```bash
mkdir -p app && cd app && npm init -y >/dev/null && npm install @aws-sdk/client-
  secrets-manager
cat > db-config.mjs <<'EOF'
import { SecretsManagerClient, GetSecretValueCommand } from "@aws-sdk/client-secrets-
  manager";
const client = new SecretsManagerClient({ region: process.env.AWS_REGION ?? "us-east-
  1" });
let cache; let fetchedAt = 0;
export async function dbCredentials(ttlMs = 5 * 60 * 1000) {
  if (cache && Date.now() - fetchedAt < ttlMs) return cache;       // short TTL so
  rotation is picked up
  const out = await client.send(new GetSecretValueCommand({ SecretId: "shop/lab/db"
  }));
  cache = JSON.parse(out.SecretString); fetchedAt = Date.now();
  return cache;
}
const c = await dbCredentials();
console.log("user:", c.username, "host:", c.host);                 // never log the
  password
EOF
node db-config.mjs
```

Expected: `user: shop_app host: shop-db.example.internal`. On EC2 the SDK uses the instance role automatically; on a laptop it uses your CLI credentials.

6. **Turn on rotation (concept, against the capstone database).** For an RDS PostgreSQL secret use the AWS-provided template; the console flow *Secrets Manager → secret → Rotation → Edit rotation* creates the Lambda for you. The CLI equivalent once a rotation function

exists:

```bash
aws secretsmanager rotate-secret --secret-id shop/prod/db \
  --rotation-lambda-arn arn:aws:lambda:us-east-
  1:111122223333:function:SecretsManagerRDSPostgreSQLRotationSingleUser \
  --rotation-rules AutomaticallyAfterDays=30
aws secretsmanager describe-secret --secret-id shop/prod/db --query
  '{Enabled:RotationEnabled,Next:NextRotationDate,Last:LastRotatedDate}'
```

To force a rotation now: `aws secretsmanager rotate-secret --secret-id shop/prod/db`. For a brand-new RDS instance, prefer `aws rds create-db-instance ... --manage-master-user-password`, which creates and rotates the master secret for you (RDS §6.10).

7. **Compare with Parameter Store.**

```bash
aws ssm put-parameter --name /shop/lab/log_level --type String --value info
aws ssm put-parameter --name /shop/lab/api_key --type SecureString --value 'example-
  key-not-real'
aws ssm get-parameters-by-path --path /shop/lab --with-decryption --query
  'Parameters[].[Name,Type]' --output table
```

8. **Cleanup (do not skip).**

```bash
aws secretsmanager delete-secret --secret-id shop/lab/db --force-delete-without-
  recovery
aws ssm delete-parameters --names /shop/lab/log_level /shop/lab/api_key
aws secretsmanager list-secrets --query 'SecretList[].Name'      # must not list
  shop/lab/db
rm -rf app
```

Only use `--force-delete-without-recovery` for lab secrets. For real secrets keep the recovery window.

7 Real-world scenario

Illustrative scenario. The `shop-api` database password is rotated automatically every 30 days. At 02:00 on the first rotation after launch, the pager fires with `password authentication failed for user "shop_app"`. **Investigation:** `describe-secret` shows the rotation succeeded at 01:58; the new password is `AWSCURRENT`. The API instances read the secret once at boot and kept the password in a long-lived connection pool; new pool connections used the old password. **Fix:** restart the service for the immediate recovery, then change the data layer to re-

fetch the secret and retry once on an authentication error, with a 5-minute cache TTL; switch to the alternating-user rotation strategy so the previous user stays valid for one more cycle. **Lesson:** rotation is a contract between the vault and the application — the consumer must be built to refresh, and the first rotation should be forced in staging.

8 Common mistakes and best practices

Don't

- Put secrets in environment files committed to Git, user-data scripts (readable via the instance metadata) or AMIs.
- Log the retrieved value, put it in an error message, or print it in CI output.
- Call `GetSecretValue` on every request — it is slow, costs money and may throttle; cache with a short TTL.
- Use `Resource: "*"` in the IAM policy; grant each application only its own secrets.
- Enable rotation without checking that the rotation Lambda can reach both the database and the Secrets Manager endpoint.
- Force-delete a production secret to "fix" a typo; use `put-secret-value`.

Do

- Name secrets hierarchically by environment and app (`shop/prod/db`) and tag them for cost and ownership.
- Use managed rotation (RDS) or AWS's rotation templates, test in staging, and alert on rotation failure (CloudWatch §6.18 alarms or EventBridge rules).
- Use Parameter Store for non-secret configuration and Secrets Manager only where rotation or cross-account/replication features pay for themselves.
- Use a customer managed KMS §6.15 key for high-value secrets so you can control and audit decryption separately.
- Monitor reads: alarm on unusual `GetSecretValue` volume or callers using CloudTrail §6.17.
- Prefer short-lived credentials where possible (IAM roles, IAM database authentication) so there is less to store in the first place.

9 Tasks

- [ ] **T1 — Create and read a secret** *(≈20 min, beginner)*
  - **Goal:** store a generated database credential safely.
  - **Steps:**
    1. Run step 1 to create `shop/lab/db` from a generated password.

2. Read it with the CLI and confirm the password does not appear in your shell history (`history | grep -i password`).
3. Run `describe-secret` and identify the version ID and its staging label.

- **Done when:** `get-secret-value` returns the JSON and your history contains no password.

☐ **T2 — Versions and rollback** *(≈30 min, intermediate)*

- **Goal:** use staging labels to see and restore the previous value.
- **Steps:**
  1. Update the secret with `put-secret-value` (step 3).
  2. Retrieve `AWSPREVIOUS`.
  3. Move `AWSCURRENT` back with `aws secretsmanager update-secret-version-stage --secret-id shop/lab/db --version-stage AWSCURRENT --move-to-version-id <OLD_ID> --remove-from-version-id <NEW_ID>`.
- **Done when:** `get-secret-value` returns the original password again.

☐ **T3 — Application retrieval with a role** *(≈45 min, intermediate)*

- **Goal:** an EC2 instance reads the secret with no stored credentials.
- **Steps:**
  1. Create an instance role with only the least-privilege policy from step 4 and launch a `t3.micro` with it (EC2 §6.1, IAM §6.7).
  2. Run the Node.js script from step 5 on it (or the AWS CLI).
  3. Try to read a different secret and observe the denial.
- **Done when:** the instance prints the user and host for `shop/lab/db`, and `AccessDeniedException` appears for any other secret. Terminate the instance afterward.

☐ **T4 — Rotation experiment** *(≈90 min, advanced)*

- **Goal:** watch a real rotation (optional, requires an RDS instance; costs a few cents if you use a `db.t4g.micro` and delete it immediately).
- **Steps:**
  1. Create an RDS PostgreSQL instance with `--manage-master-user-password` (or store its credentials in a secret and attach the AWS RDS rotation template).
  2. Run `rotate-secret` to force a rotation and watch the version labels with `describe-secret`.
  3. Connect with the new password.

- **Done when:** `LastRotatedDate` is now, `AWSPREVIOUS` exists, and `psql` works with the new `AWSCURRENT` value. Delete the database afterward.

☐ **T5 — Break it: rotation with a stale app** *(≈30 min, intermediate)*

- **Goal:** feel the failure mode from section 7 without a database.
- **Steps:**
  1. Write a script that fetches the secret once and uses the value in a loop that checks it against the current secret every 10 seconds.
  2. Change the secret with `put-secret-value` during the loop.
  3. Add a refresh-on-mismatch with a 60-second TTL.
- **Done when:** the first version of the script reports a mismatch forever and the fixed version recovers within one TTL.

☐ **T6 — Prove it: Secrets Manager or Parameter Store?** *(≈30 min, advanced)*

- **Goal:** justify the choice for every value `shop-api` needs and clean up.
- **Steps:**
  1. Without notes, classify these into Secrets Manager or Parameter Store with a one-line reason each: DB password, Redis auth token, Stripe API key, `LOG_LEVEL`, S3 bucket name, JWT signing key, feature flags.
  2. Estimate the monthly cost of 12 secrets with 2 million API calls and compare with Parameter Store standard parameters.
  3. Run the cleanup (step 8).
- **Done when:** your table names the rotation or cost reason for each choice and `list-secrets` shows nothing left from the lab.

10 Self-check questions

1. What are the staging labels `AWSCURRENT`, `AWSPREVIOUS` and `AWSPENDING` for?
2. What are the four steps of the rotation Lambda, and which network paths must it have?
3. When would you choose Parameter Store over Secrets Manager?
4. Why does a long-lived connection pool break after rotation, and how do you design around it?
5. What permissions does an application need to read a secret encrypted with a customer managed KMS key?
6. What happens when you `delete-secret` without extra flags?

ANSWERS

1. `AWSCURRENT` is the live value returned by default; `AWSPREVIOUS` is the prior value kept for rollback; `AWSPENDING` marks the new value during rotation before it is promoted.
2. `createSecret`, `setSecret`, `testSecret`, `finishSecret`. It needs network access to the database and to the Secrets Manager API (NAT gateway or VPC endpoint).
3. For configuration or non-rotating values, when cost matters (standard parameters are free), or when sizes are small and no automatic rotation is needed.
4. Existing connections keep working but new ones use the cached old password. Re-fetch on authentication failure, cache with a short TTL, and consider alternating-user rotation.
5. `secretsmanager:GetSecretValue` on the secret and `kms:Decrypt` on the key (allowed by both the key policy and the IAM policy).
6. The secret is scheduled for deletion with a 30-day recovery window (7-30 allowed); it can be restored with `restore-secret` until then.

11 Further reading

- *AWS Secrets Manager User Guide* (docs.aws.amazon.com/secretsmanager) — rotation, versions, resource policies, pricing.
- *AWS Systems Manager User Guide: Parameter Store* — standard vs advanced parameters, SecureString.
- *Amazon RDS User Guide*: password management with Secrets Manager.
- OWASP *Secrets Management Cheat Sheet* — vendor-neutral practices.

LAB NOTES What I ran, what I saw, what it means

6.17 CloudTrail

Topic 17 of 18 | ~4 hours | Difficulty: Intermediate

1 What it is

AWS CloudTrail records **who did what, when, from where** in your AWS account. Every call to an AWS API — whether it comes from the console, the AWS CLI, an SDK, Terraform, or another AWS service acting for you — is an **event**: a JSON document naming the caller (`userIdentity`), the action (`eventName`, such as `RunInstances`), the service (`eventSource`), the parameters, the source IP, the user agent, the time and, if it failed, an `errorCode`.

CloudTrail is on by default in a minimal form: **Event history** shows the last **90 days** of *management events* in each region, free, and you can query it from the console or with `aws cloudtrail lookup-events`. For anything longer-lived, or for object-level S3 activity, you create a **trail** that delivers events as compressed JSON files to an S3 §6.8 bucket (and optionally to CloudWatch Logs §6.18), or you use **CloudTrail Lake**, a managed event store you can query with SQL.

2 Why it matters in DevOps

- **Incident response and forensics:** "Who opened port 22 to the world?", "Who deleted the bucket?", "Which access key created these 40 GPU instances?" can only be answered from CloudTrail. Without it, an account compromise has no timeline.
- **Debugging IAM:** an `AccessDenied` event names the exact action and principal that failed — often the fastest way to fix a missing permission in IAM §6.7 (and the base for tools that generate least-privilege policies from observed activity).
- **Compliance:** PCI-DSS, SOC 2, HIPAA and ISO 27001 audits require a tamper-evident record of administrative activity and access to sensitive data.
- **Automation:** CloudTrail events feed EventBridge rules, GuardDuty and AWS Config, which can alert or auto-remediate (for example, close a security group that was opened to `0.0.0.0/0`).
- **Cost control:** it also answers "who created this expensive resource?" when the bill jumps.

3 Real-world system example

AWS's own security services. Amazon GuardDuty analyzes CloudTrail management events (among other sources) to flag suspicious API activity such as credentials used from unusual locations, and IAM Access Analyzer can generate a least-privilege policy by looking at the CloudTrail history of a role — both are documented AWS features. AWS also recommends, in its security best practices and in the AWS Control Tower landing-zone design, an **organization trail** in a dedicated log-archive account whose bucket workload accounts cannot modify. Teams that follow the AWS Well-Architected Security pillar typically combine three things: a multi-region trail, log-file integrity validation, and alerts on high-risk events (root-account use, console login without MFA, changes to trails and security groups). For the SRE and audit context, see DevSecOps §0.12 and Observability §0.10.

4 Diagram

From API call to investigation:

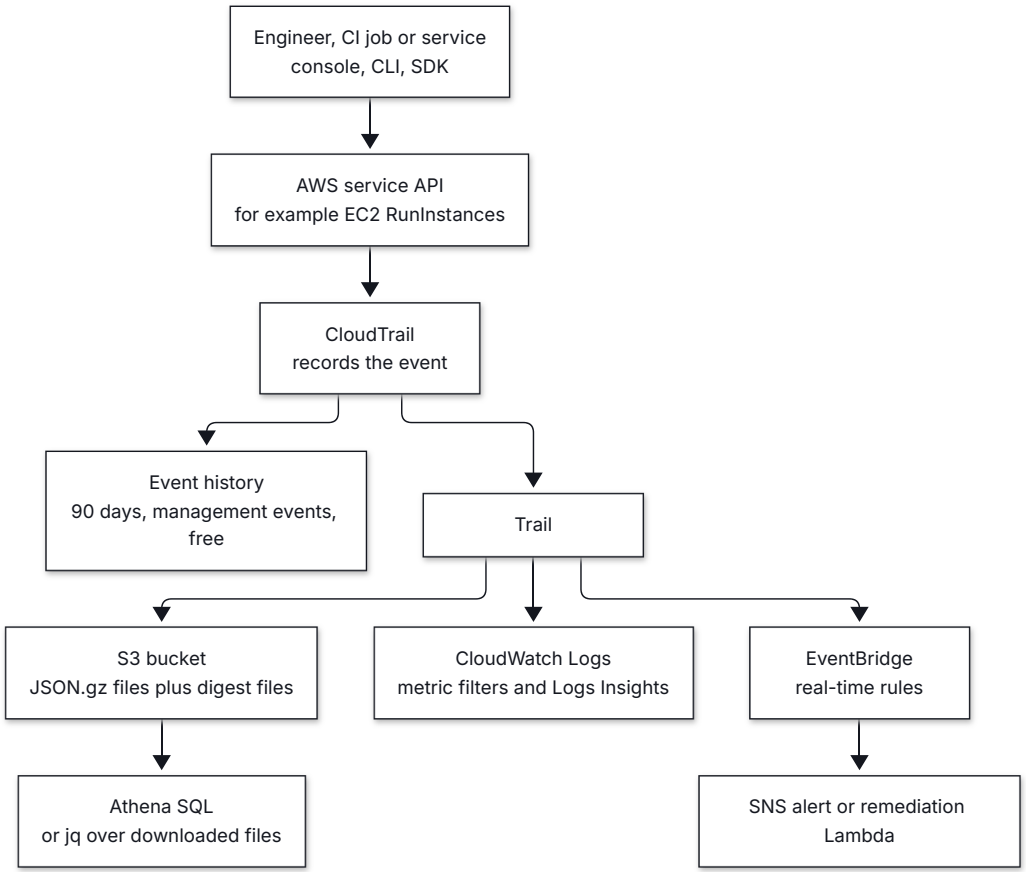


Figure 6.17-1

Anatomy of what you will read most often:

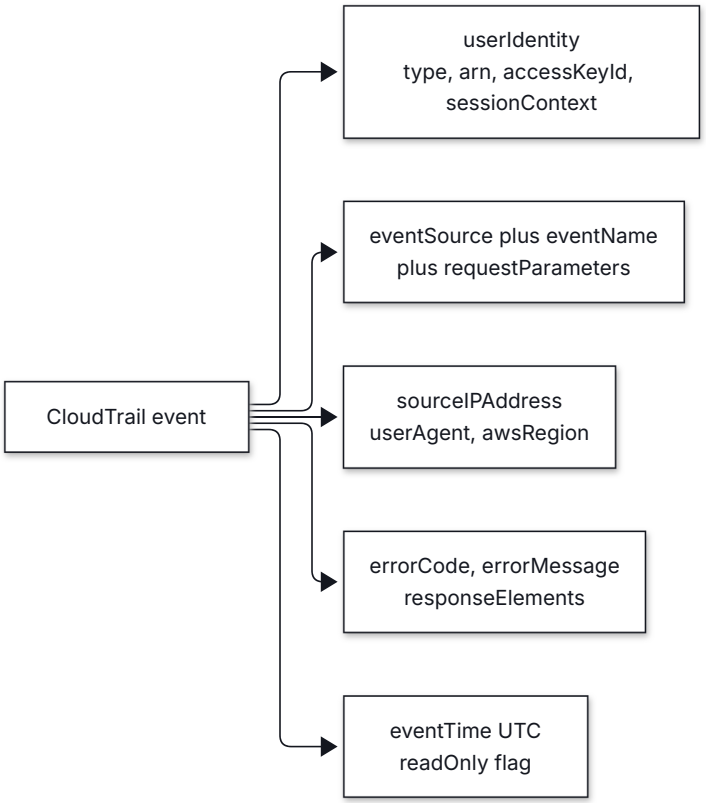


Figure 6.17-2

5 How it works

Event types

Type	What it records	Examples	Default
Management events (control plane)	Operations that create, change or delete resources, plus sign-in and security-config activity	`RunInstances`, `CreateBucket`, `PutBucketPolicy`, `AttachRolePolicy`, `ConsoleLogin`, `AssumeRole`	On: last 90 days in Event history. A trail's first copy of management events per region is free
Data events (data plane)	High-volume operations on the resources' contents	S3 `GetObject` / `PutObject` / `DeleteObject`, Lambda `Invoke`, DynamoDB item operations	**Off**; billed per event once enabled

Type	What it records	Examples	Default
Insights events	Statistically unusual API call volume or error rates	a spike of `RunInstances`	Off; extra charge
Network activity events	VPC endpoint activity (newer feature)	calls through an endpoint	Off; extra charge

Management events split into **read** (`DescribeInstances`, `ListBuckets`) and **write** (`TerminateInstances`) events; a trail can record both or just one.

Where events go

- **Event history:** per-region, 90 days, management events only, free; the console and `lookup-events` (a few requests per second at most). Good for a quick question, not for an audit record.
- **Trail to S3:** continuous delivery of gzipped JSON to `s3://bucket/AWSLogs/<account-id>/CloudTrail/<region>/<yyyy>/<mm>/<dd>/`. Delivery typically lags the API call by around five minutes (up to about fifteen). A **multi-region trail** collects all regions, including the global services (IAM, STS, CloudFront) that report in `us-east-1`; an **organization trail** (from the management account) covers every account in AWS Organizations.
- **Trail to CloudWatch Logs:** lets you use metric filters, alarms and Logs Insights near real time (CloudWatch §6.18).
- **CloudTrail Lake:** an event data store that you query with SQL in the console or API; retention you choose; billed for ingestion and queries. A good alternative to building Athena tables.

Integrity. Turn on **log file validation**: CloudTrail writes hourly **digest files** that are signed (SHA-256 hashes, RSA signature), and `aws cloudtrail validate-logs` proves that log files were not modified or deleted. Encrypt the logs with KMS §6.15 (SSE-KMS) and protect the bucket: block public access, deny deletes except to a break-glass role, optionally enable S3 Object Lock or MFA delete, and keep the bucket in a separate security account.

The anatomy of an event (abridged):

```json
{
  "eventTime": "2026-03-01T02:20:00Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "AuthorizeSecurityGroupIngress",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "203.0.113.9",
  "userAgent": "aws-cli/2.x ...",
  "userIdentity": { "type": "IAMUser", "arn": "arn:aws:iam::111122223333:user/alice",
  "accessKeyId": "AKIAIOSFODNN7EXAMPLE" },
  "requestParameters": { "groupId": "sg-0123456789abcdef0", "ipPermissions": { "items": [
  { "fromPort": 22, "ipRanges": { "items": [ { "cidrIp": "0.0.0.0/0" } ] } } ] } },
  "readOnly": false
}
```

Things to know when reading: `userIdentity.type` is `Root`, `IAMUser`, `AssumedRole`, `AWSService` or `FederatedUser`; for `AssumedRole` the `sessionContext` shows who assumed it; `errorCode` of `AccessDenied`, `UnauthorizedOperation` or `Client.…` means the call failed; times are **UTC**; some secrets in request parameters are redacted (`HIDDEN_DUE_TO_SECURITY_REASONS`).

Cost. Event history is free. For trails, the first copy of management events in a region is free; additional trails and data events are billed (data events on the order of a few cents to $0.10 per 100,000 events — check the pricing page), plus S3 storage and any Athena/CloudWatch Logs/Lake charges. The classic bill shock is enabling S3 data events on a busy bucket — scope them to the buckets that matter with advanced event selectors.

6 Step-by-step guide

Prerequisites: AWS CLI v2, `jq`. **Cost warning:** one multi-region trail with only management events costs nothing for CloudTrail itself; the S3 storage for lab logs is cents. Enabling data events on a bucket is billed per event — keep that part short and turn it off. Keep your billing alarm in place (CloudWatch §6.18).

> **No AWS account / LocalStack:** CloudTrail is not meaningfully available in the free LocalStack tier. Steps 1 and 2 let you practise the *investigation* skills on a sample file with `jq`, and everything else needs a real account (it costs nothing in the lab configuration).

1. **Practise on a sample log first (free, offline).**

```bash
cat > sample-trail.json <<'EOF'
{"Records":[
{"eventTime":"2026-03-
  01T02:14:07Z","eventSource":"signin.amazonaws.com","eventName":"ConsoleLogin","sourc
  eIPAddress":"198.51.100.23","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"},"
  responseElements":{"ConsoleLogin":"Failure"}},
{"eventTime":"2026-03-
  01T02:15:11Z","eventSource":"signin.amazonaws.com","eventName":"ConsoleLogin","sourc
  eIPAddress":"203.0.113.9","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"},"
  responseElements":{"ConsoleLogin":"Success"}},
{"eventTime":"2026-03-
  01T02:20:00Z","eventSource":"ec2.amazonaws.com","eventName":"AuthorizeSecurityGroupI
  ngress","sourceIPAddress":"203.0.113.9","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"},"
  requestParameters":{"groupId":"sg-0123456789abcdef0","ipPermissions":{"items":
  [{"ipProtocol":"tcp","fromPort":22,"toPort":22,"ipRanges":{"items":
  [{"cidrIp":"0.0.0.0/0"}]}}]}}},
{"eventTime":"2026-03-
  01T02:21:00Z","eventSource":"s3.amazonaws.com","eventName":"DeleteBucket","errorCode
  ":"AccessDenied","sourceIPAddress":"203.0.113.9","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"}}
]}
EOF
# Failed console logins
jq -r '.Records[] | select(.eventName=="ConsoleLogin" and
  .responseElements.ConsoleLogin=="Failure") |
  [.eventTime,.userIdentity.userName,.sourceIPAddress] | @tsv' sample-trail.json
# Security groups opened to the world
jq -r '.Records[] | select(.eventName=="AuthorizeSecurityGroupIngress") |
  select(.requestParameters.ipPermissions.items[].ipRanges.items[].cidrIp=="0.0.0.0/0"
  ) | [.eventTime,.userIdentity.arn,.requestParameters.groupId] | @tsv' sample-
  trail.json
# Everything that was denied
jq -r '.Records[] | select(.errorCode != null) | [.eventTime,.eventName,.errorCode] |
  @tsv' sample-trail.json
```

Expected: one failed login from `198.51.100.23`, one world-open SSH rule from `alice` on `sg-0123456789abcdef0`, and one `DeleteBucket AccessDenied`. Together these tell a story: a failed login, then success from a different IP, then an opened firewall and an attempted deletion — the shape of an account takeover.

2. **Query real Event history.**

```bash
export AWS_REGION=us-east-1
aws cloudtrail lookup-events --lookup-attributes
  AttributeKey=EventName,AttributeValue=ConsoleLogin \
  --max-results 5 --query 'Events[].[EventTime,Username,EventName]' --output table
aws cloudtrail lookup-events --lookup-attributes
  AttributeKey=EventSource,AttributeValue=ec2.amazonaws.com \
  --start-time "$(date -u -d '-1 day' +%FT%TZ)" --max-results 20 \
  --query 'Events[].[EventTime,EventName,Username]' --output table
```

(`date -d` is GNU; on macOS use `date -u -v-1d +%FT%TZ`.) `lookup-events` supports one attribute at a time (`EventName`, `Username`, `EventSource`, `ResourceName`, `ResourceType`, `AccessKeyId`, `ReadOnly`, `EventId`). Pull one full event: `... --max-results 1 --query 'Events[0].CloudTrailEvent' --output text | jq .`

3. **Create a log bucket and a multi-region trail with validation.**

```bash
```

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
LOGBUCKET=shop-trail-logs-$ACCOUNT_ID
TRAIL=shop-lab-trail
aws s3api create-bucket --bucket $LOGBUCKET
aws s3api put-public-access-block --bucket $LOGBUCKET --public-access-block-
  configuration \

  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuck
  ets=true
cat > trail-policy.json <<EOF
{ "Version": "2012-10-17", "Statement": [
  { "Sid": "AWSCloudTrailAclCheck", "Effect": "Allow", "Principal": { "Service":
  "cloudtrail.amazonaws.com" },
    "Action": "s3:GetBucketAcl", "Resource": "arn:aws:s3:::$LOGBUCKET",
    "Condition": { "StringEquals": { "aws:SourceArn": "arn:aws:cloudtrail:us-east-
  1:$ACCOUNT_ID:trail/$TRAIL" } } },
  { "Sid": "AWSCloudTrailWrite", "Effect": "Allow", "Principal": { "Service":
  "cloudtrail.amazonaws.com" },
    "Action": "s3:PutObject", "Resource":
  "arn:aws:s3:::$LOGBUCKET/AWSLogs/$ACCOUNT_ID/*",
    "Condition": { "StringEquals": { "s3:x-amz-acl": "bucket-owner-full-control",
                                     "aws:SourceArn": "arn:aws:cloudtrail:us-east-
  1:$ACCOUNT_ID:trail/$TRAIL" } } } ] }
EOF
aws s3api put-bucket-policy --bucket $LOGBUCKET --policy file://trail-policy.json
aws cloudtrail create-trail --name $TRAIL --s3-bucket-name $LOGBUCKET \
  --is-multi-region-trail --include-global-service-events --enable-log-file-validation
aws cloudtrail start-logging --name $TRAIL
aws cloudtrail get-trail-status --name $TRAIL --query
  '{Logging:IsLogging,LastDelivery:LatestDeliveryTime}'
```

Expected: `IsLogging: true`. Delivery to S3 starts within about 5-15 minutes.

4. **Generate some activity, then read the files.**

```
BASH
aws ec2 create-security-group --group-name trail-demo --description "trail demo" >
  /dev/null     # a write event
aws ec2 delete-security-group --group-name trail-demo
sleep 600     # or come back later
aws s3 sync s3://$LOGBUCKET/AWSLogs/$ACCOUNT_ID/CloudTrail/ ./trail-logs/ --quiet
find ./trail-logs -name '*.json.gz' | head -3
find ./trail-logs -name '*.json.gz' -exec gunzip -c {} + \
  | jq -r '.Records[] | select(.eventName | test("SecurityGroup")) |
  [.eventTime,.eventName,.userIdentity.arn,.sourceIPAddress] | @tsv'
```

5. **Validate integrity.**

```bash
aws cloudtrail validate-logs --trail-arn arn:aws:cloudtrail:us-east-
  1:$ACCOUNT_ID:trail/$TRAIL \
  --start-time "$(date -u -d '-2 hours' +%FT%TZ)"
```

Expected: lines saying digest files and log files were valid (log delivery must have occurred and a digest must have been written, which can take an hour).

6. **Investigation queries on a larger scale.**
   - *Athena:* in the console, *CloudTrail → Event history → Create Athena table* (the console fills in the standard table definition for your bucket). Then, for example:

```sql
SELECT eventtime, useridentity.arn, eventname, sourceipaddress, errorcode
FROM cloudtrail_logs
WHERE eventsource = 'ec2.amazonaws.com' AND eventname =
  'AuthorizeSecurityGroupIngress'
  AND eventtime > '2026-03-01T00:00:00Z'
ORDER BY eventtime DESC LIMIT 50;
```

   - *CloudWatch Logs Insights* (if the trail also writes to a log group):

```sql
fields @timestamp, eventName, userIdentity.arn, sourceIPAddress, errorCode
| filter errorCode like /AccessDenied|UnauthorizedOperation/
| stats count(*) as denied by userIdentity.arn, eventName
| sort denied desc | limit 20
```

Useful standing questions: *root account usage* (`userIdentity.type = 'Root'`), *console login without MFA* (`additionalEventData` shows `MFAUsed: "No"`), *who changed IAM* (`eventsource = 'iam.amazonaws.com'` and `readonly = 'false'`), *who touched the trail* (`StopLogging`, `DeleteTrail`, `UpdateTrail`).

7. **Optional: S3 data events for one bucket** (billed per event; do this briefly).

```bash
cat > selectors.json <<EOF
[{ "Name": "S3 objects in the lab log bucket only",
   "FieldSelectors": [ { "Field": "eventCategory", "Equals": ["Data"] },
                       { "Field": "resources.type", "Equals": ["AWS::S3::Object"] },
                       { "Field": "resources.ARN", "StartsWith":
  ["arn:aws:s3:::$LOGBUCKET/"] } ] }]
EOF
aws cloudtrail put-event-selectors --trail-name $TRAIL --advanced-event-selectors
  file://selectors.json
```

   To turn data events off again, switch back to a basic management-only selector: `aws cloudtrail put-event-selectors --trail-name $TRAIL --event-selectors '[{"ReadWriteType":"All","IncludeManagementEvents":true}]'` (or simply delete the trail in step 8).

8. **Cleanup (do not skip).**

```bash
aws cloudtrail stop-logging --name $TRAIL
aws cloudtrail delete-trail --name $TRAIL
aws s3 rm s3://$LOGBUCKET --recursive
aws s3api delete-bucket --bucket $LOGBUCKET
rm -rf trail-logs sample-trail.json trail-policy.json selectors.json
aws cloudtrail describe-trails --query 'trailList[].Name'     # shop-lab-trail must be
  gone
```

   In a real account you keep at least one multi-region management-event trail running permanently; deleting it here is for the lab only.

7 Real-world scenario

Illustrative scenario. On a Saturday the `shop-api` account's bill alarm (CloudWatch §6.18) fires: charges are 10x the normal daily amount. **Investigation:** in Event history, filter `EventName = RunInstances` for the last 24 hours — 60 `p3.16xlarge` instances in `ap-southeast-2`, a region the team never uses, launched using the access key of a CI user, from an IP address that is not the CI runner's. `userAgent` is a Python SDK, not the Terraform user agent the pipeline normally has. The key was leaked in a public repository. **Fix:** deactivate the access key, terminate the instances, check for `CreateUser`, `CreateAccessKey`, `AttachUserPolicy` events from the same key (none, luckily), rotate every secret that CI could read (Secrets Manager §6.16), and file an AWS support request for the charges. **Follow-ups:** a service control policy denying unused regions, EventBridge alerting on `RunInstances` in unexpected re-

gions, and moving CI to short-lived credentials via OIDC. **Lesson:** you can only investigate what was recorded; a multi-region trail with a protected log bucket is the first control every account gets.

8 Common mistakes and best practices

Don't

- Rely on Event history alone: 90 days, management events only, per region, not tamper-evident.
- Create a single-region trail and assume you are covered; attackers use regions you never look at.
- Store the logs in the same account and bucket the attacker can reach with the stolen credentials, with delete permission.
- Enable data events on every bucket and Lambda "just in case" — scope them or the bill and the noise will explode.
- Forget that delivery is delayed by minutes; do not use CloudTrail S3 files for real-time alerting (use EventBridge or CloudWatch Logs).
- Put secrets in request parameters (for example tags or environment variables) and then act surprised when they appear in logs.

Do

- Run an **organization or account multi-region trail** with log file validation, SSE-KMS and a locked-down bucket from day one.
- Send the trail to CloudWatch Logs and create alarms for root usage, console logins without MFA, IAM policy changes, security group and route-table changes, and `StopLogging` / `DeleteTrail`.
- Use consistent user agents / role session names for automation (`RoleSessionName=github-actions-<run-id>`) so events are attributable.
- Keep logs per your retention policy (S3 lifecycle to cheaper storage) and practise Athena or Lake queries before an incident.
- Define the trail as code (Terraform [§7.1]) and review changes to it.

9 Tasks

- [ ] **T1 — Read an event** *(≈30 min, beginner)*
  - **Goal:** get comfortable with the event structure.
  - **Steps:**

1. Do something visible in the console (create a security group), wait a few minutes.
2. Find the event with `lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=CreateSecurityGroup`.
3. Print the full `CloudTrailEvent` JSON and identify who, what, from where, and the request parameters.

- **Done when:** you can point at `userIdentity.arn`, `eventName`, `sourceIPAddress`, `userAgent` and the new group's ID in the response.

- [ ] **T2 — Investigate the sample incident** *(≈30 min, beginner)*
  - **Goal:** reconstruct a timeline with `jq`.
  - **Steps:**
    1. Run the three `jq` queries from step 1 on the sample file.
    2. Add a fourth query listing every event by `sourceIPAddress`, sorted by time.
    3. Write a five-line incident timeline.
  - **Done when:** your timeline lists the failed login, successful login from another IP, the world-open SSH rule and the denied `DeleteBucket`, each with a timestamp.

- [ ] **T3 — Create a protected trail** *(≈45 min, intermediate)*
  - **Goal:** durable, validated, multi-region logging.
  - **Steps:**
    1. Follow step 3 to create the bucket, policy and trail with log file validation.
    2. Generate activity (create and delete a security group) and wait for delivery.
    3. Run `get-trail-status`, list the S3 prefix, and `validate-logs`.
  - **Done when:** `.json.gz` files exist in your bucket and `validate-logs` reports valid digests and files.

- [ ] **T4 — Query it with Athena or Logs Insights** *(≈60 min, intermediate)*
  - **Goal:** answer audit questions in SQL.
  - **Steps:**
    1. Create the Athena table from the console (or send the trail to a CloudWatch Logs group).
    2. Write queries for: all `AccessDenied` events by principal; all IAM write events; any `ConsoleLogin` without MFA.
    3. Save the queries in a `queries.md` file in your repo.
  - **Done when:** each query returns rows (generate a denied call by attempting a forbidden action to be sure) and you can explain each result.

- [ ] **T5 — Alert on dangerous events** *(≈60 min, advanced)*
  - **Goal:** turn the log into a detector.
  - **Steps:**
    1. Create an EventBridge rule on `source: aws.ec2`, `detail-type: AWS API Call via CloudTrail`, `eventName: AuthorizeSecurityGroupIngress` with an SNS email target.
    2. Open a security group port to `0.0.0.0/0` in a test group.
    3. Remove the rule and delete the test group, the rule and the topic.
  - **Done when:** you received an email within a couple of minutes naming the principal and group, and all lab resources are deleted.
- [ ] **T6 — Prove it: tamper test and teardown** *(≈45 min, advanced)*
  - **Goal:** show that integrity validation detects tampering.
  - **Steps:**
    1. Without the guide, pick one delivered log file, download it, change a field and re-upload it to the same key (on your lab bucket only).
    2. Run `validate-logs` for that window and read the output.
    3. Explain why this protects an audit, then run the cleanup (step 8).
  - **Done when:** `validate-logs` reports the modified file as invalid (hash mismatch) and `describe-trails` shows your lab trail is gone.

10 Self-check questions

1. What is the difference between management events and data events, and which are recorded by default?
2. Why isn't Event history enough for a production account?
3. How do you prove that CloudTrail log files were not altered?
4. A user says "something deleted my S3 object last week" — which kind of event do you need, and was it enabled by default?
5. How would you find which access key launched unexpected EC2 instances?
6. Why should the trail bucket live in a different account from the workloads?

ANSWERS

1. Management events are control-plane operations (create, modify, delete resources, sign-ins) and are recorded by default in Event history; data events are high-volume object-level operations (S3 objects, Lambda invokes, DynamoDB items) and must be enabled on a trail.

2. It keeps 90 days, only management events, per region, can't be exported as a tamper-evident record, and is limited in querying.
3. Enable log file validation and run `aws cloudtrail validate-logs`, which checks the signed digest files against the log files.
4. An S3 data event (`DeleteObject`); it is off by default, so unless it was enabled for that bucket you may have no record (S3 server access logs or versioning might still help).
5. `lookup-events` with `EventName=RunInstances` and read `userIdentity.accessKeyId`, `sourceIPAddress` and `userAgent` (or query Athena/Lake for the same).
6. A compromised workload account's credentials then cannot delete or alter the evidence.

11 Further reading

- *AWS CloudTrail User Guide* (docs.aws.amazon.com/awscloudtrail) — event reference, trails, Lake, log file integrity validation.
- *AWS CloudTrail pricing* page — current charges for data events, Insights and Lake.
- *AWS Security Incident Response Guide* — using CloudTrail in investigations.
- *AWS Well-Architected Framework: Security pillar* — detection and logging practices.

LAB NOTES What I ran, what I saw, what it means

6.18 CloudWatch

Topic 18 of 18 | ~5 hours | Difficulty: Intermediate

1 What it is

Amazon CloudWatch is AWS's built-in observability service. It has four parts you will use constantly:

- **Metrics** — numbers over time (CPU utilization, ALB request count, queue depth), published automatically by almost every AWS service or by your own code.
- **Logs** — text and JSON events collected into **log groups** and **log streams**, searchable with **Logs Insights**.
- **Alarms** — rules that watch a metric and change state (`OK`, `ALARM`, `INSUFFICIENT_DATA`), then notify people or trigger actions such as scaling or an EC2 recovery.
- **Dashboards** — customizable pages of graphs and numbers.

CloudWatch is where the other topics of this level report their health: EC2 §6.1 CPU, ALB §6.14 5xx counts, RDS §6.10 connections, ElastiCache §6.11 evictions, Lambda §6.3 errors and CloudTrail §6.17 events. It also hosts the most important alarm in a new AWS account — the **billing alarm**.

2 Why it matters in DevOps

- **You cannot operate what you cannot see.** The first question in an incident is "what changed and what is broken?" — answered by graphs and logs.
- **Autoscaling is driven by it:** Auto Scaling §6.2 policies scale on CloudWatch metrics and alarms.
- **Alerting design is a skill:** noisy alarms train people to ignore pages (see Alert Fatigue Reduction §14.8); missing alarms mean customers tell you about outages first.
- **Money:** a billing alarm catches a runaway resource or a leaked key before the invoice does, and log ingestion itself is a classic hidden cost.
- **It is the default, zero-install option:** even teams that adopt Prometheus and Grafana (Level 10 §10.1) still rely on CloudWatch for AWS-managed services.

3 Real-world system example

AWS Lambda and Auto Scaling. Lambda sends every function's logs to a CloudWatch log group named `/aws/lambda/<function-name>` and publishes invocation, error, duration and throttle metrics to the `AWS/Lambda` namespace without any configuration. Similarly an Auto Scaling group's target-tracking policy creates and manages CloudWatch alarms on your behalf to keep a metric such as average CPU or `ALBRequestCountPerTarget` at a target value. On the practice side, the Amazon Builders' Library article *Instrumenting distributed systems for operational visibility* describes how Amazon teams treat metrics, structured logs and dashboards as part of the product, and AWS's Well-Architected Operational Excellence pillar says the same. The **embedded metric format (EMF)** lets an application write a specially structured JSON log line from which CloudWatch extracts custom metrics automatically — a documented, low-overhead way to publish business metrics such as `OrdersPlaced`. For the three-pillars concepts behind it, see Observability §0.10.

4 Diagram

The CloudWatch data flow:

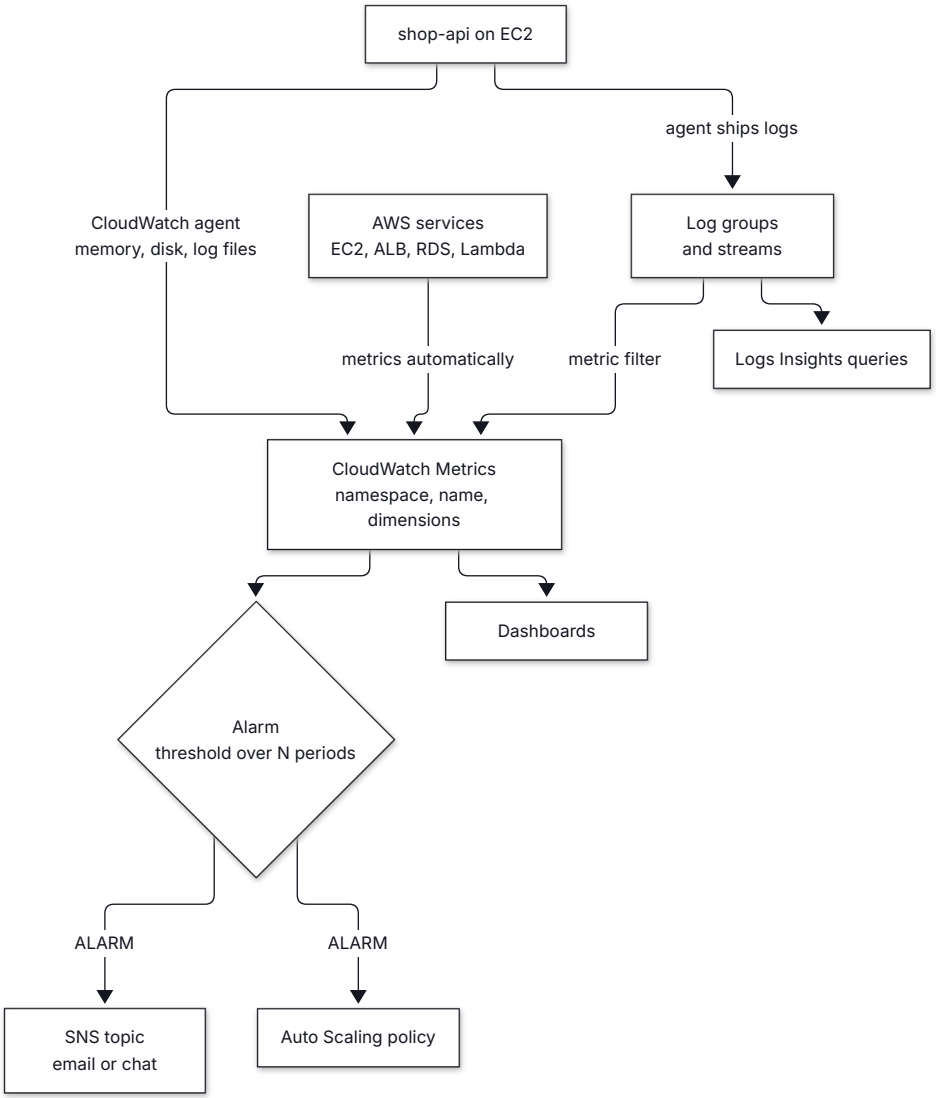


Figure 6.18-1

Alarm state machine:

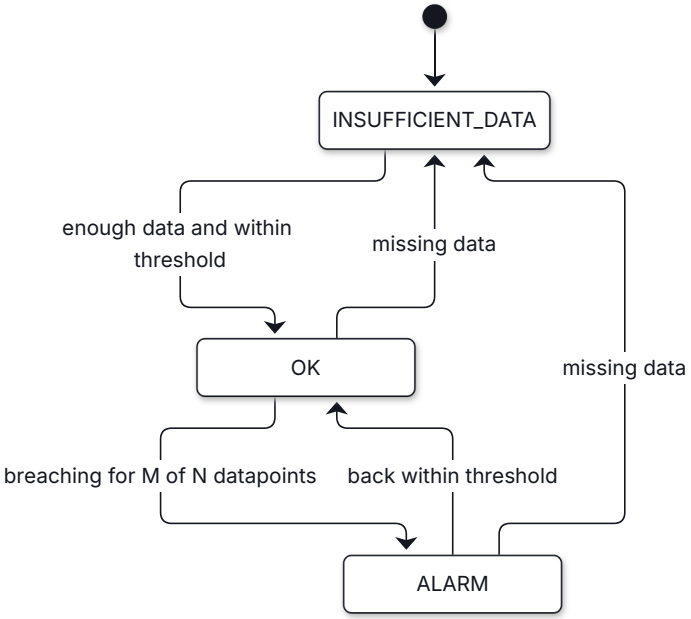


Figure 6.18-2

5 How it works

Metrics. A metric is identified by **namespace** (`AWS/EC2`, `AWS/ApplicationELB`, or your own such as `ShopApi`), **metric name** (`CPUUtilization`) and **dimensions** (name/value pairs such as `InstanceId=i-0abc…`, `LoadBalancer=app/shop-alb/…`). Each unique combination is a separate metric. Data points have a timestamp, value and unit, and are aggregated by **statistic** (`Average`, `Sum`, `Minimum`, `Maximum`, `SampleCount`, percentiles like `p99`) over a **period**.

- EC2 metrics default to **5-minute** granularity (free "basic monitoring"); **detailed monitoring** gives 1 minute for an extra charge. Most other services publish at 1 minute. Custom metrics can be **high-resolution** (down to 1 second).
- **Resolution over time:** CloudWatch rolls data up as it ages — sub-minute data is kept for 3 hours, 1-minute data for 15 days, 5-minute data for 63 days, 1-hour data for 455 days (15 months).
- **What the hypervisor cannot see** — memory use, disk space, processes — is *not* published by default. Install the **CloudWatch agent** to collect it.
- **Metric math** combines metrics: error rate = `100 * errors / requests`. **Anomaly detection** builds an expected-value band instead of a fixed threshold.

- Custom metrics cost money per metric per month (every unique dimension combination counts), so avoid high-cardinality dimensions such as user IDs.

Logs. A **log group** is a collection with shared retention, encryption (KMS §6.15) and access settings; a **log stream** is one source (an instance, a container). Key facts:

- **Retention defaults to "Never expire"** — set it (7, 14, 30, 90 days...) or storage grows forever.
- You pay for **ingestion (per GB)** and storage; ingestion is the large line item, so avoid noisy debug logs in production.
- **Metric filters** turn matching log lines into metrics (for example count lines containing `"level":"error"`), so you can alarm on them.
- **Subscription filters** stream logs in near real time to Kinesis, Lambda or OpenSearch.
- **Log classes** (Standard, Infrequent Access) trade features for a lower ingestion price.
- Structured **JSON logs** make Logs Insights queries easy (see Structured Logging §10.5).

Logs Insights is the query language for logs:

```sql
fields @timestamp, @message
| filter @message like /ERROR/
| stats count(*) as errors by bin(5m)
| sort @timestamp desc
| limit 50
```

Core commands: `fields`, `filter`, `stats` (with `count`, `avg`, `pct`, `sum`), `sort`, `limit`, `parse` (extract fields from text), `dedup`, `display`. It scans the log groups and time range you pick and is billed per GB scanned, so narrow the window first.

Alarms. An alarm evaluates one metric (or a metric-math expression) over **period × evaluation periods**; with **datapoints to alarm** you can require, say, 3 out of 5. The most overlooked setting is **treat missing data** (`missing`, `notBreaching`, `breaching`, `ignore`): a stopped server publishes *nothing*, so a "CPU is high" alarm stays quiet while a host is dead. Alarm actions: publish to an **SNS topic** (email, SMS, HTTPS, chat via AWS Chatbot), run an **Auto Scaling** policy, stop/terminate/reboot/recover an EC2 instance, or create an OpsCenter item. **Composite alarms** combine alarms with AND/OR to cut noise. Each standard alarm has a small monthly cost; there is a free allowance of 10 alarms.

Dashboards. JSON-defined pages of widgets (line, number, gauge, logs table) that can span regions and accounts. A small number of dashboards is included in the free tier; beyond that each costs a monthly fee. Build one per service ("`shop-api` overview": requests, latency p99,

5xx, healthy hosts, DB connections, cache hit rate) following the golden signals (What to Monitor §10.11).

The billing alarm. AWS publishes the `EstimatedCharges` metric (namespace `AWS/Billing`, dimension `Currency=USD`) — **only in** `us-east-1`, updated several times per day, and **only after you enable billing alerts** (*Billing and Cost Management → Billing preferences → Alert preferences → "Receive CloudWatch billing alerts"*; the account root user or an admin with billing permissions must do it; IAM users also need billing access activated). It is cumulative for the month. The alarm uses `Maximum` over a 6-hour (21,600 s) period. Complementary tool: **AWS Budgets** (first budgets are free) sends alerts on actual and forecast spend and also supports per-service and per-tag budgets. Use both: the CloudWatch alarm is simple and always-on, Budgets is richer.

Cost summary. Free tier (always free at the time of writing; verify on the pricing page): 10 custom metrics and alarms, 3 dashboards (up to 50 metrics), 5 GB of log ingestion and storage per month, 1 million API requests. Beyond that: custom metrics, alarms, dashboards, log ingestion/storage/scan and API calls (such as `GetMetricData` from third-party tools) are billed.

6 Step-by-step guide

Prerequisites: AWS CLI v2, an email address, and (for steps 5-6) an EC2 instance with an instance profile that includes the managed policy `CloudWatchAgentServerPolicy` (see IAM §6.7). **Cost warning:** the lab fits inside the always-free tier (a couple of alarms, a few KB of logs); the cost risks are leaving debug logs flowing and creating many custom-metric dimension combinations. Clean up at the end. **Do step 1 first, in every account you will ever use.**

> **LocalStack:** the free image supports CloudWatch metrics, alarms and Logs basics. Use `export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_REGION=us-east-1`. Alarms transition states there but the real `AWS/Billing` data and Logs Insights depth do not exist, so do step 1 on a real account.

1. **Create the billing alarm (first milestone of the capstone).** First enable *Receive*

CloudWatch billing alerts in the Billing console (once per account), then in `us-east-1`:

```bash
export AWS_REGION=us-east-1
TOPIC_ARN=$(aws sns create-topic --name billing-alerts --query TopicArn --output text)
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint
  you@example.com
# open the email AWS sends and click "Confirm subscription"
aws cloudwatch put-metric-alarm --alarm-name billing-over-10-usd \
  --alarm-description "Estimated month-to-date charges exceed 10 USD" \
  --namespace AWS/Billing --metric-name EstimatedCharges --dimensions
  Name=Currency,Value=USD \
  --statistic Maximum --period 21600 --evaluation-periods 1 --threshold 10 \
  --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data
  notBreaching \
  --alarm-actions $TOPIC_ARN
aws cloudwatch describe-alarms --alarm-names billing-over-10-usd \
  --query 'MetricAlarms[].[AlarmName,StateValue,Threshold]' --output table
```

Expected: the alarm exists, state `INSUFFICIENT_DATA` or `OK`. Prove the notification path works without waiting for a bill:

```bash
aws cloudwatch set-alarm-state --alarm-name billing-over-10-usd --state-value ALARM --
  state-reason "test"
```

You should receive an email within a minute; the alarm returns to its real state at the next evaluation. Use a lower threshold (for example 5) if your normal spend is near zero, and add a second alarm at a higher level. (If `put-metric-alarm` succeeds but the alarm stays `INSUFFICIENT_DATA` for a day, billing alerts were probably not enabled.)

2. **Look at the metrics a service gives you for free.**

```bash
aws cloudwatch list-metrics --namespace AWS/EC2 --metric-name CPUUtilization --query
  'Metrics[0:3]'
INSTANCE_ID=i-0123456789abcdef0        # replace with yours
aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name CPUUtilization
  \
  --dimensions Name=InstanceId,Value=$INSTANCE_ID \
  --start-time "$(date -u -d '-1 hour' +%FT%TZ)" --end-time "$(date -u +%FT%TZ)" \
  --period 300 --statistics Average Maximum --output table
```

3. **Publish a custom metric and alarm on it.**

```bash
for i in 1 2 3; do
  aws cloudwatch put-metric-data --namespace ShopApi --metric-name OrdersPlaced --
  value $((RANDOM % 20)) \
    --unit Count --dimensions Env=lab
  sleep 20
done
aws cloudwatch put-metric-alarm --alarm-name shop-no-orders --namespace ShopApi --
  metric-name OrdersPlaced \
  --dimensions Name=Env,Value=lab --statistic Sum --period 60 --evaluation-periods 5 -
  -datapoints-to-alarm 5 \
  --threshold 1 --comparison-operator LessThanThreshold --treat-missing-data breaching
  \
  --alarm-actions $TOPIC_ARN
```

`treat-missing-data breaching` makes "no data at all" count as a failure — the right choice for a "business is silent" alarm.

4. **Logs from the CLI: create a group, set retention, write, query.**

```bash
aws logs create-log-group --log-group-name /shop-api/lab
aws logs put-retention-policy --log-group-name /shop-api/lab --retention-in-days 7
aws logs create-log-stream --log-group-name /shop-api/lab --log-stream-name manual
NOW=$(($(date +%s%N)/1000000))
aws logs put-log-events --log-group-name /shop-api/lab --log-stream-name manual --log-
  events \
  timestamp=$NOW,message='{"level":"info","msg":"order created","ms":42}' \
  timestamp=$((NOW+1)),message='{"level":"error","msg":"payment failed","ms":913}'
```

Query it with Logs Insights (results arrive after a few seconds):

```bash
QID=$(aws logs start-query --log-group-name /shop-api/lab \
  --start-time $(($(date +%s)-3600)) --end-time $(date +%s) \
  --query-string 'fields @timestamp, level, msg, ms | filter level = "error" | sort
  @timestamp desc' \
  --query queryId --output text)
sleep 5; aws logs get-query-results --query-id $QID --query 'results'
```

Live view: `aws logs tail /shop-api/lab --follow`.

5. **Turn a log pattern into a metric and an alarm.**

```bash
aws logs put-metric-filter --log-group-name /shop-api/lab --filter-name errors \
  --filter-pattern '{ $.level = "error" }' \
  --metric-transformations
  metricName=AppErrors,metricNamespace=ShopApi,metricValue=1,defaultValue=0
aws cloudwatch put-metric-alarm --alarm-name shop-app-errors --namespace ShopApi --
  metric-name AppErrors \
  --statistic Sum --period 60 --evaluation-periods 1 --threshold 1 --comparison-
  operator GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching --alarm-actions $TOPIC_ARN
```

Write another error line with `put-log-events` (use a new timestamp, and the sequence token is no longer required) and the alarm goes to `ALARM` within a couple of minutes.

6. **On EC2: CloudWatch agent for memory, disk and app logs** (Amazon Linux 2023):

```bash
sudo dnf install -y amazon-cloudwatch-agent
sudo tee /opt/aws/amazon-cloudwatch-agent/etc/config.json >/dev/null <<'EOF'
{
  "metrics": { "namespace": "ShopApi/EC2",
    "append_dimensions": { "InstanceId": "${aws:InstanceId}" },
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] } } },
  "logs": { "logs_collected": { "files": { "collect_list": [
    { "file_path": "/var/log/shop-api/app.log", "log_group_name": "/shop-api/lab",
   "log_stream_name": "{instance_id}" } ] } } }
}
EOF
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config
  -m ec2 -s \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/config.json
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
```

Expected: `"status": "running"`. After a few minutes `mem_used_percent` appears under namespace `ShopApi/EC2`.

7. **A dashboard as code.**

```bash
cat > dash.json <<'EOF'
{ "widgets": [
  { "type": "metric", "x": 0, "y": 0, "width": 12, "height": 6, "properties": {
      "title": "Orders placed", "region": "us-east-1", "stat": "Sum", "period": 60,
      "metrics": [ [ "ShopApi", "OrdersPlaced", "Env", "lab" ] ] } },
  { "type": "log", "x": 12, "y": 0, "width": 12, "height": 6, "properties": {
      "title": "Recent errors", "region": "us-east-1",
      "query": "SOURCE '/shop-api/lab' | fields @timestamp, msg | filter level =
  'error' | sort @timestamp desc | limit 20" } } ] }
EOF
aws cloudwatch put-dashboard --dashboard-name shop-api-lab --dashboard-body
  file://dash.json
```

   Open *CloudWatch → Dashboards → shop-api-lab* in the console.

8. **Cleanup (do not skip).** Keep the billing alarm and its SNS topic permanently; delete everything else.

```bash
aws cloudwatch delete-alarms --alarm-names shop-no-orders shop-app-errors
aws cloudwatch delete-dashboards --dashboard-names shop-api-lab
aws logs delete-metric-filter --log-group-name /shop-api/lab --filter-name errors
aws logs delete-log-group --log-group-name /shop-api/lab
sudo systemctl stop amazon-cloudwatch-agent 2>/dev/null     # on the EC2 instance
aws cloudwatch describe-alarms --query 'MetricAlarms[].AlarmName'     # only billing-
  over-10-usd should remain
```

   Custom metrics cannot be deleted; they simply stop being billed about two weeks after the last data point.

7 Real-world scenario

Illustrative scenario. A team enables debug logging on `shop-api` to chase a rare bug and forgets to turn it off. Two weeks later the billing alarm fires at 60% of its monthly threshold, much earlier than usual. **Investigation:** Cost Explorer shows CloudWatch as the second-largest line; in the console the `/shop-api/prod` log group's `IncomingBytes` metric jumped from about 2 GB/day to 90 GB/day when the debug flag was enabled. A Logs Insights query (`stats count(*) by msg | sort count(*) desc | limit 10`) shows 80% of lines are one cache-debug message. **Fix:** set the log level to `info`, add a metric filter and alarm on `IncomingBytes` for the group, and set retention to 30 days (the group had been "Never expire"). **Lesson:** observability has its own bill; the billing alarm bought two weeks of reaction time, and an alarm on the cost driver (log volume) would have caught it in a day.

8 Common mistakes and best practices

Don't

- Skip the billing alarm because "I'll stay in the free tier" — a mistake, a leaked key or an orphaned NAT gateway does not care.
- Leave log retention at "Never expire".
- Alarm on CPU only. A web tier fails on latency, errors and saturation long before CPU is the best signal.
- Assume a quiet alarm means a healthy system without setting `treat-missing-data` deliberately.
- Create custom metrics with unbounded dimensions (user ID, request ID, IP).
- Page a human for something that cannot be acted on; send it to a ticket or dashboard instead.

Do

- Create the billing alarm (plus an AWS Budget) before launching anything else in a new account.
- Alarm on symptoms users feel: ALB `HTTPCode_Target_5XX_Count`, `TargetResponseTime` p99, `UnHealthyHostCount`, RDS `FreeStorageSpace` and `DatabaseConnections`, ElastiCache `Evictions`, Lambda `Errors` / `Throttles`.
- Log structured JSON, correlate with a request ID (Correlation IDs §10.7) and keep a handful of saved Logs Insights queries per service.
- Send alarm notifications to SNS topics (one for pages, one for tickets) and test the path with `set-alarm-state`.
- Install the CloudWatch agent through your AMI/launch template so memory and disk are collected from day one.
- Define alarms and dashboards as code (Terraform §7.1) so every environment gets them.

9 Tasks

- [ ] **T1 — Billing alarm first** *(≈30 min, beginner)*
  - **Goal:** a working, tested spending alert in the account.
  - **Steps:**
    1. Enable *Receive CloudWatch billing alerts* in Billing preferences.
    2. Create the SNS topic, confirm the email subscription and create the alarm in `us-east-1` (step 1).

3. Run `set-alarm-state` to `ALARM`.

- **Done when:** you received the alarm email and `describe-alarms` lists `billing-over-10-usd` with your threshold. (Optionally add an AWS Budget with an 80% forecast alert.)

- [ ] **T2 — Metrics, statistics and periods** *(≈30 min, beginner)*
  - **Goal:** read an AWS-provided metric correctly.
  - **Steps:**
    1. Launch (or use) a `t3.micro`; run `yes > /dev/null &` for a few minutes on it.
    2. Pull `CPUUtilization` with `get-metric-statistics` at periods of 300 and 60 seconds.
    3. Explain why the 60-second period shows few or no points unless detailed monitoring is on.
  - **Done when:** your table shows CPU rising and you can state the default EC2 granularity. Kill the process and stop the instance afterward.

- [ ] **T3 — Logs, retention and Insights** *(≈45 min, intermediate)*
  - **Goal:** ship logs and query them.
  - **Steps:**
    1. Create `/shop-api/lab` with 7-day retention and write 20 JSON log lines with varying `ms` and `level`.
    2. Write Logs Insights queries for: errors only; average and p95 of `ms`; count by `level` per minute (`stats count(*) by level, bin(1m)`).
    3. Save one query in the console.
  - **Done when:** each query returns the expected numbers and `describe-log-groups` shows `retentionInDays: 7`.

- [ ] **T4 — Metric filter and alarm** *(≈45 min, intermediate)*
  - **Goal:** alert on a log pattern.
  - **Steps:**
    1. Create the `AppErrors` metric filter and alarm from step 5.
    2. Write an error line; watch the alarm go to `ALARM` and the email arrive.
    3. Stop writing errors and watch it return to `OK`.
  - **Done when:** you captured the alarm's state history with `aws cloudwatch describe-alarm-history --alarm-name shop-app-errors`.

- [ ] **T5 — Break it: missing data** *(≈45 min, advanced)*
  - **Goal:** understand why dead hosts do not trigger "high" alarms.

- **Steps:**
  1. Create an alarm on a custom metric `Heartbeat` that fires when `Sum < 1` for 3 of 3 one-minute periods with `treat-missing-data` set to `notBreaching`.
  2. Stop publishing the metric and show the alarm stays `OK` / `INSUFFICIENT_DATA`.
  3. Change to `breaching` and show it fires.
- **Done when:** you have both behaviors demonstrated and can say which setting suits a heartbeat check.

☐ **T6 — Prove it: a dashboard and alarm set for shop-api, then tear down** *(≈90 min, advanced)*

- **Goal:** operational visibility for the capstone stack.
- **Steps:**
  1. Without the guide, create a dashboard with ALB request count and target 5xx, p99 `TargetResponseTime`, EC2 CPU and `mem_used_percent`, RDS connections and ElastiCache cache hit rate (`CacheHitRate` or `CacheHits` / `CacheMisses` math).
  2. Create alarms for ALB 5xx, unhealthy hosts, RDS free storage and ElastiCache evictions, all notifying one SNS topic.
  3. Explain how you would reduce noise (composite alarm, datapoints to alarm), then run the cleanup (step 8) but keep the billing alarm.
- **Done when:** the dashboard renders data from at least four services and `describe-alarms` shows only the billing alarm after cleanup.

10 Self-check questions

1. What identifies a CloudWatch metric, and why do dimensions matter for cost?
2. Why do you need the CloudWatch agent on EC2, and what does it add?
3. What does `treat-missing-data` do, and which value would you use for a heartbeat alarm?
4. Where must the billing alarm be created, and what must be enabled first?
5. What is the default retention of a log group and why is that a problem?
6. When would you use a metric filter versus a Logs Insights query?

ANSWERS

1. Namespace + metric name + the exact set of dimensions. Every unique combination is a separate (billable) custom metric, so high-cardinality dimensions explode cost.
2. Hypervisor-level metrics do not include memory, disk usage or application logs; the agent collects them from inside the OS and ships log files.

3. It defines how the alarm treats periods with no data (`missing` , `ignore` , `breaching` , `notBreaching`). For a heartbeat, use `breaching` so silence raises the alarm.
4. In `us-east-1` (that is where `AWS/Billing` metrics live), after enabling "Receive CloudWatch billing alerts" in Billing preferences.
5. "Never expire", so logs accumulate and are billed for storage forever; set a retention period.
6. A metric filter runs continuously on incoming logs to create a metric you can alarm on; Logs Insights is for ad-hoc, interactive investigation of past logs.

11 Further reading

- *Amazon CloudWatch User Guide* (docs.aws.amazon.com/cloudwatch) — metrics, alarms, Logs Insights query syntax, agent configuration.
- *Amazon CloudWatch pricing* page — current free tier and per-metric, per-alarm and per-GB rates.
- Amazon Builders' Library: *Instrumenting distributed systems for operational visibility*.
- *Site Reliability Engineering* (Google, free online), chapter on monitoring distributed systems.
- *AWS Budgets* documentation — complementary cost alerts.

LEVEL 6: CLOUD (AWS)

★ shop-api on CloudFront, ALB, EC2, Redis, RDS and S3

Level 6 capstone ~10 hours

1 Goal and outcome

Build, secure, observe and then completely destroy the production-shaped AWS architecture from this level's roadmap: **CloudFront → ALB → EC2 instances (Auto Scaling) → Redis (ElastiCache) → RDS → S3**. The `shop-api` sample app (a small Node.js service) serves `/api/products` from PostgreSQL through a Redis cache-aside layer; static files live in a private S3 bucket and are delivered by CloudFront.

When you finish you will have:

- a **billing alarm** (the first milestone) and a CloudTrail trail watching the account;
- a custom VPC with public and private subnets, an internet gateway, a NAT gateway and least-privilege security groups;
- an encrypted stack (KMS) whose database password lives in Secrets Manager and is never typed or stored by you;
- one HTTPS URL (`https://dxxxx.cloudfront.net`) that returns cached static files, uncached API calls, and shows which instance answered;
- CloudWatch logs, alarms and a dashboard, a Lambda function, and a scripted **teardown that leaves nothing billing**.

Level 6 topic	Where it appears
01 EC2 §6.1, 09 EBS and EFS §6.9	Launch template, encrypted gp3 root volumes (EFS is a stretch goal)
02 Auto Scaling §6.2	ASG with ELB health checks and target tracking
03 Lambda §6.3	Post-deploy CloudFront invalidation function
04 VPC §6.4, 05 routing and NAT §6.5, 06 security groups §6.6	Milestone 2
07 IAM §6.7	Instance role, Lambda role, least privilege
08 S3 §6.8	Private static-assets bucket with OAC

Level 6 topic	Where it appears
10 RDS §6.10, 11 ElastiCache §6.11	Milestone 4 data tier
12 Route 53 §6.12	Optional custom domain alias
13 CloudFront §6.13, 14 ALB §6.14	Milestones 5 and 6
15 KMS §6.15, 16 Secrets Manager §6.16	Customer managed key, managed master password
17 CloudTrail §6.17, 18 CloudWatch §6.18	Audit trail, billing alarm, logs, dashboard, alarms

2 Architecture

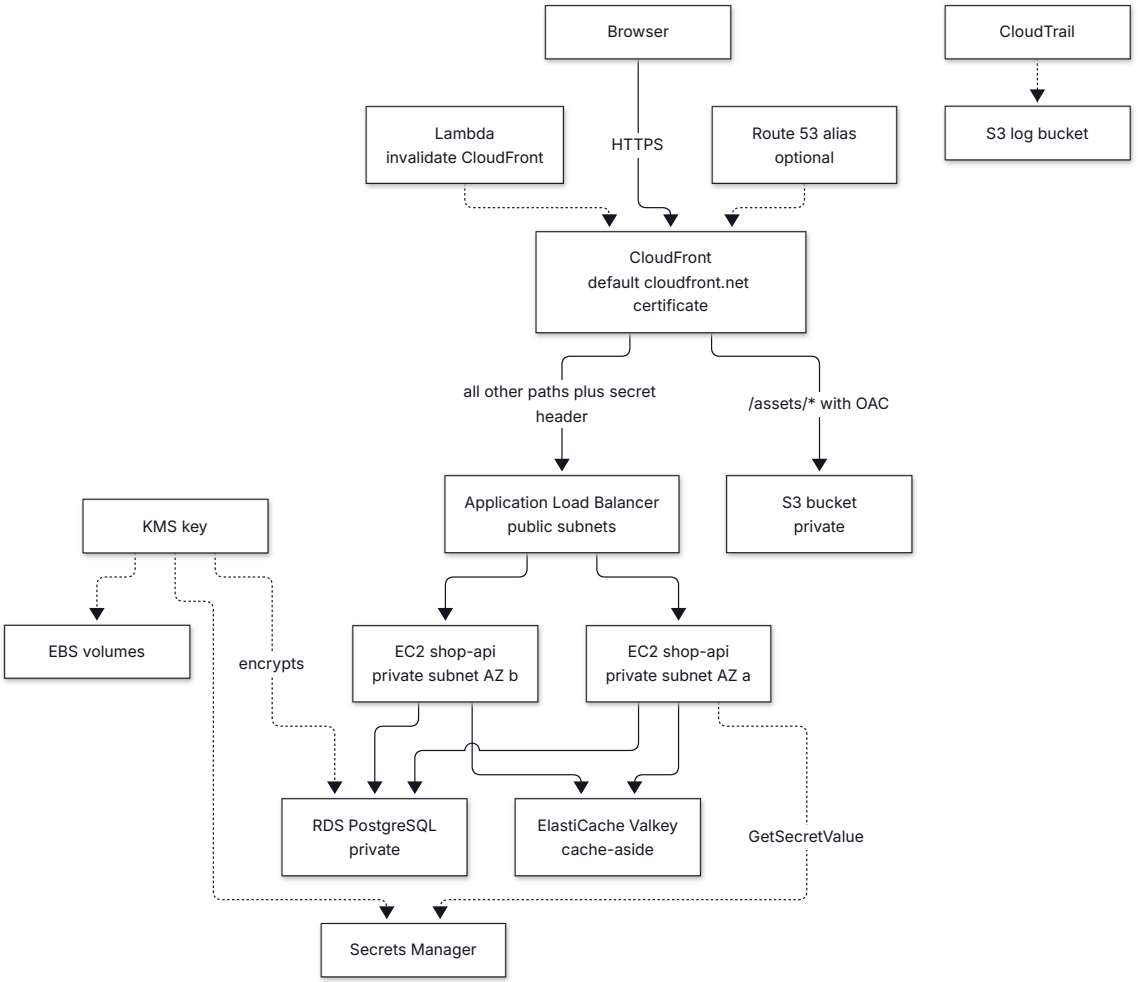


Figure 6-1

Network layout (two Availability Zones, one NAT gateway to keep the lab cheap):

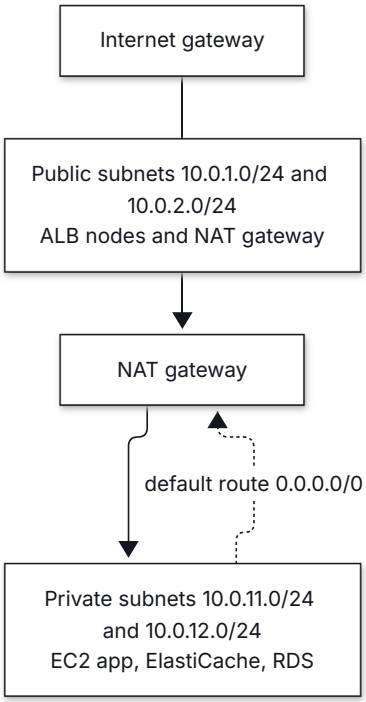


Figure 6-2

3 Prerequisites and cost

- An AWS account you own and AWS CLI v2 (`aws sts get-caller-identity` works with an admin-level IAM Identity Center or IAM user, **not** the root user), plus `jq`, `openssl`, `curl`, `zip`, `python3` and GNU `base64`. Region: `us-east-1`.
- Optional: a domain in Route 53 (about $12 a year plus $0.50 per month for the hosted zone); skip it for a free run.
- **Cost model (approximate, check the pricing pages):** NAT gateway (~$0.045 per hour plus data), ALB (~$0.0225 per hour plus LCUs), two `t3.micro` instances, one `db.t4g.micro`, one `cache.t4g.micro`, a KMS key ($1 a month, prorated) and a Secrets Manager secret ($0.40 a month after a trial). Together roughly **$0.15-0.25 per hour**, so a focused day costs a few dollars; the free tier or credits cover part of it depending on when your account was created.
- **The two rules:** (1) the billing alarm comes first; (2) run the teardown in section 7 **the same day**. Never leave the NAT gateway, ALB, RDS or ElastiCache running overnight.

- Every ID is appended to `lab.env`, so you can resume in a new shell with `source lab.env`.

4 Build it step by step

```bash
mkdir -p ~/shop-capstone && cd ~/shop-capstone
export AWS_REGION=us-east-1 AWS_PAGER=""
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
save() { echo "$1=$2" >> lab.env; }          # usage: save NAME "$VALUE"
save ACCOUNT_ID $ACCOUNT_ID
```

Milestone 0 — Billing alarm and CloudTrail (topics 18, 17)

First enable *Billing and Cost Management → Billing preferences → Receive CloudWatch billing alerts* (console, once per account).

```bash
TOPIC_ARN=$(aws sns create-topic --name shop-capstone-alerts --query TopicArn --output
  text); save TOPIC_ARN $TOPIC_ARN
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint
  you@example.com     # click the confirmation link
aws cloudwatch put-metric-alarm --alarm-name billing-over-10-usd --namespace AWS/Billing \
  --metric-name EstimatedCharges --dimensions Name=Currency,Value=USD --statistic Maximum
  --period 21600 \
  --evaluation-periods 1 --threshold 10 --comparison-operator
  GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching --alarm-actions $TOPIC_ARN
aws cloudwatch set-alarm-state --alarm-name billing-over-10-usd --state-value ALARM --
  state-reason test   # an email must arrive
```

The audit trail (use the bucket policy from CloudTrail §6.17, step 3, with trail name `shop-capstone-trail`):

```bash
LOGBUCKET=shop-capstone-trail-$ACCOUNT_ID; save LOGBUCKET $LOGBUCKET
aws s3api create-bucket --bucket $LOGBUCKET
aws s3api put-public-access-block --bucket $LOGBUCKET --public-access-block-configuration
  \

  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets
  =true
# aws s3api put-bucket-policy --bucket $LOGBUCKET --policy file://trail-policy.json
  (the two CloudTrail statements from topic 17)
aws cloudtrail create-trail --name shop-capstone-trail --s3-bucket-name $LOGBUCKET --is-
  multi-region-trail --enable-log-file-validation
aws cloudtrail start-logging --name shop-capstone-trail
```

Milestone 1 — Encryption key (topic 15)

```bash
KEY_ID=$(aws kms create-key --description "shop-capstone key" --tags
  TagKey=Project,TagValue=shop-capstone --query KeyMetadata.KeyId --output text); save
  KEY_ID $KEY_ID
aws kms create-alias --alias-name alias/shop-capstone --target-key-id $KEY_ID
aws kms enable-key-rotation --key-id $KEY_ID
# Auto Scaling creates encrypted volumes through its service-linked role, which must be
  allowed to use the key
aws iam create-service-linked-role --aws-service-name autoscaling.amazonaws.com
  2>/dev/null || true
aws kms create-grant --key-id $KEY_ID --name asg-grant \
  --grantee-principal arn:aws:iam::$ACCOUNT_ID:role/aws-service-
  role/autoscaling.amazonaws.com/AWSServiceRoleForAutoScaling \
  --operations Decrypt GenerateDataKeyWithoutPlaintext ReEncryptFrom ReEncryptTo
  CreateGrant DescribeKey
```

Milestone 2 — Network (topics 04, 05, 06)

```bash
VPC_ID=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 --query Vpc.VpcId --output text);
  save VPC_ID $VPC_ID
aws ec2 create-tags --resources $VPC_ID --tags Key=Name,Value=shop-capstone
  Key=Project,Value=shop-capstone
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-hostnames
mk_subnet() { aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block $1 --availability-zone
  $2 --query Subnet.SubnetId --output text; }
PUB_A=$(mk_subnet 10.0.1.0/24 us-east-1a);   PUB_B=$(mk_subnet 10.0.2.0/24 us-east-1b)
PRIV_A=$(mk_subnet 10.0.11.0/24 us-east-1a); PRIV_B=$(mk_subnet 10.0.12.0/24 us-east-1b)
for v in PUB_A PUB_B PRIV_A PRIV_B; do save $v ${!v}; done

IGW_ID=$(aws ec2 create-internet-gateway --query InternetGateway.InternetGatewayId --
  output text); save IGW_ID $IGW_ID
aws ec2 attach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID
PUB_RT=$(aws ec2 create-route-table --vpc-id $VPC_ID --query RouteTable.RouteTableId --
  output text); save PUB_RT $PUB_RT
aws ec2 create-route --route-table-id $PUB_RT --destination-cidr-block 0.0.0.0/0 --
  gateway-id $IGW_ID >/dev/null
for s in $PUB_A $PUB_B; do aws ec2 associate-route-table --route-table-id $PUB_RT --
  subnet-id $s >/dev/null; done

EIP_ALLOC=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text);
  save EIP_ALLOC $EIP_ALLOC
NAT_ID=$(aws ec2 create-nat-gateway --subnet-id $PUB_A --allocation-id $EIP_ALLOC --query
  NatGateway.NatGatewayId --output text); save NAT_ID $NAT_ID
aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT_ID
PRIV_RT=$(aws ec2 create-route-table --vpc-id $VPC_ID --query RouteTable.RouteTableId --
  output text); save PRIV_RT $PRIV_RT
aws ec2 create-route --route-table-id $PRIV_RT --destination-cidr-block 0.0.0.0/0 --nat-
  gateway-id $NAT_ID >/dev/null
```

```
for s in $PRIV_A $PRIV_B; do aws ec2 associate-route-table --route-table-id $PRIV_RT --
  subnet-id $s >/dev/null; done
```

Security groups, each tier accepting traffic only from the tier in front of it:

```bash
mk_sg() { aws ec2 create-security-group --group-name $1 --description "$1" --vpc-id
  $VPC_ID --query GroupId --output text; }
ALB_SG=$(mk_sg shop-cap-alb); APP_SG=$(mk_sg shop-cap-app); DB_SG=$(mk_sg shop-cap-db);
  CACHE_SG=$(mk_sg shop-cap-cache)
for v in ALB_SG APP_SG DB_SG CACHE_SG; do save $v ${!v}; done
CF_PL=$(aws ec2 describe-managed-prefix-lists --filters Name=prefix-list-
  name,Values=com.amazonaws.global.cloudfront.origin-facing --query
  'PrefixLists[0].PrefixListId' --output text)
aws ec2 authorize-security-group-ingress --group-id $ALB_SG --ip-permissions
  "IpProtocol=tcp,FromPort=80,ToPort=80,PrefixListIds=[{PrefixListId=$CF_PL}]"
aws ec2 authorize-security-group-ingress --group-id $APP_SG --protocol tcp --port 3000 --
  source-group $ALB_SG
aws ec2 authorize-security-group-ingress --group-id $DB_SG --protocol tcp --port 5432 --
  source-group $APP_SG
aws ec2 authorize-security-group-ingress --group-id $CACHE_SG --protocol tcp --port 6379 -
  -source-group $APP_SG
```

The CloudFront prefix list counts as roughly 55 rules against the security group rule quota; if you hit the quota, request an increase or rely on the secret-header check from Milestone 5 alone.

Milestone 3 — S3 assets bucket and IAM role (topics 08, 07)

```bash
ASSETS=shop-capstone-assets-$ACCOUNT_ID; save ASSETS $ASSETS
aws s3api create-bucket --bucket $ASSETS
aws s3api put-public-access-block --bucket $ASSETS --public-access-block-configuration
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets
  =true
mkdir -p site/assets && echo '<h1>shop-api</h1><p>static file from S3 via CloudFront</p>'
  > site/assets/hello.html
aws s3 sync site/ s3://$ASSETS/

echo '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
  {"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}' > trust-ec2.json
aws iam create-role --role-name shop-capstone-app --assume-role-policy-document
  file://trust-ec2.json >/dev/null
aws iam attach-role-policy --role-name shop-capstone-app --policy-arn
  arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore    # Session Manager instead of
  SSH
aws iam attach-role-policy --role-name shop-capstone-app --policy-arn
  arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
aws iam create-instance-profile --instance-profile-name shop-capstone-app >/dev/null
aws iam add-role-to-instance-profile --instance-profile-name shop-capstone-app --role-name
  shop-capstone-app
```

Milestone 4 — Data tier: RDS with a managed secret, and Redis (topics 10, 11, 16, 15)

```bash
aws rds create-db-subnet-group --db-subnet-group-name shop-cap --db-subnet-group-
  description "shop capstone" --subnet-ids $PRIV_A $PRIV_B >/dev/null
aws elasticache create-cache-subnet-group --cache-subnet-group-name shop-cap --cache-
  subnet-group-description "shop capstone" --subnet-ids $PRIV_A $PRIV_B >/dev/null

aws rds create-db-instance --db-instance-identifier shop-capstone --engine postgres --db-
  instance-class db.t4g.micro \
  --allocated-storage 20 --storage-type gp3 --storage-encrypted --kms-key-id $KEY_ID --db-
  name shop \
  --master-username shopadmin --manage-master-user-password --master-user-secret-kms-key-
  id $KEY_ID \
  --db-subnet-group-name shop-cap --vpc-security-group-ids $DB_SG --no-publicly-accessible
  \
  --backup-retention-period 1 --no-multi-az --tags Key=Project,Value=shop-capstone
  >/dev/null

aws elasticache create-replication-group --replication-group-id shop-cap-cache --
  replication-group-description "shop capstone" \
  --engine valkey --cache-node-type cache.t4g.micro --num-cache-clusters 1 \
  --transit-encryption-enabled --at-rest-encryption-enabled --kms-key-id $KEY_ID \
  --cache-subnet-group-name shop-cap --security-group-ids $CACHE_SG >/dev/null
```

```
aws rds wait db-instance-available --db-instance-identifier shop-capstone
  # about 10 minutes
aws elasticache wait replication-group-available --replication-group-id shop-cap-cache
DB_HOST=$(aws rds describe-db-instances --db-instance-identifier shop-capstone --query
  'DBInstances[0].Endpoint.Address' --output text)
DB_SECRET_ARN=$(aws rds describe-db-instances --db-instance-identifier shop-capstone --
  query 'DBInstances[0].MasterUserSecret.SecretArn' --output text)
REDIS_HOST=$(aws elasticache describe-replication-groups --replication-group-id shop-cap-
  cache --query 'ReplicationGroups[0].NodeGroups[0].PrimaryEndpoint.Address' --output
  text)
for v in DB_HOST DB_SECRET_ARN REDIS_HOST; do save $v ${!v}; done
```

Let the app role read only that one secret and use the key:

BASH

```
cat > app-policy.json <<EOF
{"Version":"2012-10-17","Statement":[
 {"Effect":"Allow","Action":"secretsmanager:GetSecretValue","Resource":"$DB_SECRET_ARN"},
 {"Effect":"Allow","Action":"kms:Decrypt","Resource":"arn:aws:kms:us-east-
  1:$ACCOUNT_ID:key/$KEY_ID"}]}
EOF
aws iam put-role-policy --role-name shop-capstone-app --policy-name read-db-secret --
  policy-document file://app-policy.json
```

The managed secret holds the master user. In a real system you would create a limited application user (stretch goal).

Milestone 5 — App, launch template, ALB and Auto Scaling (topics 01, 09, 14, 02)

The application (cache-aside against Redis, credentials from Secrets Manager). Save as `server.js`:

JAVASCRIPT

```
const http = require('http'), os = require('os'), { Pool } = require('pg'), { createClient
  } = require('redis');
const { SecretsManagerClient, GetSecretValueCommand } = require('@aws-sdk/client-secrets-
  manager');
const E = process.env; let pool, redis;

async function init() {
  const sm = new SecretsManagerClient({ region: E.AWS_REGION });
  const s = JSON.parse((await sm.send(new GetSecretValueCommand({ SecretId:
  E.DB_SECRET_ARN }))).SecretString);
  pool = new Pool({ host: E.DB_HOST, user: s.username, password: s.password, database:
  'shop', ssl: { rejectUnauthorized: false }, max: 5 });
  await pool.query('create table if not exists products(id serial primary key, name text
  not null, price_cents int not null)');
  if ((await pool.query('select count(*) from products')).rows[0].count === '0')
```

```
    await pool.query("insert into products(name, price_cents) values ('keyboard',4900),
  ('monitor',17900),('mouse',1900)");
  redis = createClient({ url: `rediss://${E.REDIS_HOST}:6379` });
  redis.on('error', e => console.error('redis', e.message));
  await redis.connect();
}

const json = { 'Content-Type': 'application/json' };
async function handle(req, res) {
  if (req.url === '/healthz') { res.writeHead(200); return res.end('ok'); }          //
  shallow check: the process is up
  if (req.url === '/api/whoami') { res.writeHead(200, json); return
  res.end(JSON.stringify({ host: os.hostname() })); }
  if (req.url === '/api/products') {
    let body = await redis.get('products:all'), cache = 'hit';
    if (!body) {
      cache = 'miss';
      body = JSON.stringify((await pool.query('select * from products order by
  id')).rows);
      await redis.set('products:all', body, { EX: 60 });
    }
    res.writeHead(200, { ...json, 'X-App-Cache': cache, 'X-App-Host': os.hostname() });
    return res.end(body);
  }
  res.writeHead(404); res.end('not found');
}

init().then(() => http.createServer((req, res) => handle(req, res).catch(e => {
  console.error(e); res.writeHead(500); res.end('error'); }))
  .listen(3000, () => console.log('shop-api ready on 3000'))).catch(e => {
  console.error(e); process.exit(1); });
```

The boot script (user-data). It embeds `server.js` and fills in the three endpoint placeholders. (RDS requires TLS; `rejectUnauthorized: false` keeps the lab short, production loads the RDS CA bundle.)

BASH

```
cat > userdata.sh <<'EOF'
#!/bin/bash
dnf install -y nodejs22 amazon-cloudwatch-agent || dnf install -y nodejs20 amazon-
  cloudwatch-agent    # AWS SDK v3 needs a current Node.js
mkdir -p /opt/shop-api && cd /opt/shop-api
npm init -y >/dev/null && npm install pg redis @aws-sdk/client-secrets-manager     # needs
  the NAT gateway
cat > /opt/shop-api/server.js <<'JS'
__SERVER_JS__
JS
printf 'AWS_REGION=us-east-
  1\nDB_HOST=__DB_HOST__\nDB_SECRET_ARN=__DB_SECRET_ARN__\nREDIS_HOST=__REDIS_HOST__\n' >
  /etc/shop-api.env
cat > /etc/systemd/system/shop-api.service <<'UNIT'
```

```
[Unit]
Description=shop-api
After=network-online.target
[Service]
EnvironmentFile=/etc/shop-api.env
WorkingDirectory=/opt/shop-api
ExecStart=/usr/bin/env node server.js
Restart=always
RestartSec=5
StandardOutput=append:/var/log/shop-api.log
StandardError=append:/var/log/shop-api.log
[Install]
WantedBy=multi-user.target
UNIT
cat > /opt/aws/amazon-cloudwatch-agent/etc/config.json <<'CW'
{"metrics":{"namespace":"ShopApi/EC2","append_dimensions":
  {"AutoScalingGroupName":"${aws:AutoScalingGroupName}"},
  "metrics_collected":{"mem":{"measurement":["mem_used_percent"]},"disk":{"measurement":
  ["used_percent"],"resources":["/"]}}},
 "logs":{"logs_collected":{"files":{"collect_list":[{"file_path":"/var/log/shop-
  api.log","log_group_name":"/shop-api/capstone","log_stream_name":"{instance_id}"}]}}}}
CW
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/config.json
systemctl daemon-reload && systemctl enable --now shop-api
EOF
python3 - <<PY
t = open('userdata.sh').read().replace('__SERVER_JS__', open('server.js').read().rstrip())
for k, v in {'__DB_HOST__': '$DB_HOST', '__DB_SECRET_ARN__': '$DB_SECRET_ARN',
  '__REDIS_HOST__': '$REDIS_HOST'}.items():
    t = t.replace(k, v)
open('userdata.final.sh', 'w').write(t)
PY
grep -c "__[A-Z_]*__" userdata.final.sh || true      # expect 0: every placeholder was
  replaced
```

The unquoted `<<PY` lets the shell substitute the three `$` variables into the Python snippet before it runs.

Log group, launch template, load balancer and target group:

BASH

```
aws logs create-log-group --log-group-name /shop-api/capstone && aws logs put-retention-
  policy --log-group-name /shop-api/capstone --retention-in-days 7
jq -n --arg ud "$(base64 -w0 userdata.final.sh)" --arg sg $APP_SG --arg key $KEY_ID '{
  ImageId:"resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
  x86_64", InstanceType:"t3.micro",
  IamInstanceProfile:{Name:"shop-capstone-app"}, SecurityGroupIds:[$sg], UserData:$ud,
  MetadataOptions:{HttpTokens:"required"},
```

```
  BlockDeviceMappings:[{DeviceName:"/dev/xvda",Ebs:
  {VolumeSize:8,VolumeType:"gp3",Encrypted:true,KmsKeyId:$key,DeleteOnTermination:true}}]
  }' > lt.json
aws iam wait instance-profile-exists --instance-profile-name shop-capstone-app; sleep 10
aws ec2 create-launch-template --launch-template-name shop-capstone --launch-template-data
  file://lt.json >/dev/null

ORIGIN_SECRET=$(openssl rand -hex 16); save ORIGIN_SECRET $ORIGIN_SECRET
ALB_ARN=$(aws elbv2 create-load-balancer --name shop-capstone --type application --subnets
  $PUB_A $PUB_B --security-groups $ALB_SG --query 'LoadBalancers[0].LoadBalancerArn' --
  output text); save ALB_ARN $ALB_ARN
ALB_DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB_ARN --query
  'LoadBalancers[0].DNSName' --output text); save ALB_DNS $ALB_DNS
TG_ARN=$(aws elbv2 create-target-group --name shop-capstone --protocol HTTP --port 3000 --
  vpc-id $VPC_ID --health-check-path /healthz \
  --health-check-interval-seconds 15 --healthy-threshold-count 2 --unhealthy-threshold-
  count 2 --query 'TargetGroups[0].TargetGroupArn' --output text); save TG_ARN $TG_ARN
aws elbv2 modify-target-group-attributes --target-group-arn $TG_ARN --attributes
  Key=deregistration_delay.timeout_seconds,Value=30 >/dev/null
L_ARN=$(aws elbv2 create-listener --load-balancer-arn $ALB_ARN --protocol HTTP --port 80 \
  --default-actions 'Type=fixed-response,FixedResponseConfig=
  {StatusCode=403,ContentType=text/plain,MessageBody=denied}' \
  --query 'Listeners[0].ListenerArn' --output text)
aws elbv2 create-rule --listener-arn $L_ARN --priority 1 \
  --conditions "[{\"Field\":\"http-header\",\"HttpHeaderConfig\":{\"HttpHeaderName\":\"X-
  Origin-Verify\",\"Values\":[\"$ORIGIN_SECRET\"]}}]" \
  --actions "Type=forward,TargetGroupArn=$TG_ARN" >/dev/null
```

Auto Scaling group: two instances across both AZs, replaced if the load balancer reports them unhealthy.

BASH

```
aws autoscaling create-auto-scaling-group --auto-scaling-group-name shop-capstone-asg \
  --launch-template LaunchTemplateName=shop-capstone,Version='$Latest' --min-size 2 --max-
  size 3 --desired-capacity 2 \
  --vpc-zone-identifier "$PRIV_A,$PRIV_B" --target-group-arns $TG_ARN --health-check-type
  ELB --health-check-grace-period 300 \
  --tags Key=Name,Value=shop-capstone,PropagateAtLaunch=true Key=Project,Value=shop-
  capstone,PropagateAtLaunch=true
aws autoscaling put-scaling-policy --auto-scaling-group-name shop-capstone-asg --policy-
  name cpu50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{"PredefinedMetricSpecification":
  {"PredefinedMetricType":"ASGAverageCPUUtilization"},"TargetValue":50}' >/dev/null
# wait for 2 healthy targets (about 5 minutes); repeat this command until both say healthy
aws elbv2 describe-target-health --target-group-arn $TG_ARN --query
  'TargetHealthDescriptions[].[Target.Id,TargetHealth.State]' --output text
```

The ALB security group only admits CloudFront, so to test directly from your laptop temporarily allow your own IP, then remove it again in Milestone 6:

```bash
MYIP=$(curl -s https://checkip.amazonaws.com); save MYIP $MYIP
aws elbv2 wait load-balancer-available --load-balancer-arns $ALB_ARN
aws ec2 authorize-security-group-ingress --group-id $ALB_SG --protocol tcp --port 80 --
  cidr $MYIP/32
curl -s -H "X-Origin-Verify: $ORIGIN_SECRET" http://$ALB_DNS/api/products; echo      #
  JSON rows from PostgreSQL
curl -si http://$ALB_DNS/api/products | head -1                                      #
  403: no secret header
```

If targets stay unhealthy, open Session Manager (`aws ssm start-session --target <instance-id>`) and read `/var/log/cloud-init-output.log` and `/var/log/shop-api.log`.

Milestone 6 — CloudFront in front, with OAC for S3 (topic 13; optional Route 53, topic 12)

```bash
aws ec2 revoke-security-group-ingress --group-id $ALB_SG --protocol tcp --port 80 --cidr
  $MYIP/32      # back to CloudFront-only
OAC_ID=$(aws cloudfront create-origin-access-control --origin-access-control-config \
  'Name=shop-capstone-
  oac,SigningProtocol=sigv4,SigningBehavior=always,OriginAccessControlOriginType=s3' --
  query OriginAccessControl.Id --output text); save OAC_ID $OAC_ID
cat > dist.json <<EOF
{ "CallerReference": "shop-capstone-1", "Comment": "shop-api capstone", "Enabled": true,
  "PriceClass": "PriceClass_100",
  "Origins": { "Quantity": 2, "Items": [
    { "Id": "s3-assets", "DomainName": "$ASSETS.s3.us-east-1.amazonaws.com",
  "S3OriginConfig": { "OriginAccessIdentity": "" }, "OriginAccessControlId": "$OAC_ID" },
    { "Id": "alb-app", "DomainName": "$ALB_DNS",
      "CustomOriginConfig": { "HTTPPort": 80, "HTTPSPort": 443, "OriginProtocolPolicy":
  "http-only" },
      "CustomHeaders": { "Quantity": 1, "Items": [ { "HeaderName": "X-Origin-Verify",
  "HeaderValue": "$ORIGIN_SECRET" } ] } } ] },
  "DefaultCacheBehavior": { "TargetOriginId": "alb-app", "ViewerProtocolPolicy":
  "redirect-to-https", "Compress": true,
    "AllowedMethods": { "Quantity": 7, "Items":
  ["GET","HEAD","OPTIONS","PUT","POST","PATCH","DELETE"] },
    "CachePolicyId": "4135ea2d-6df8-44a3-9df3-4b5a84be39ad", "OriginRequestPolicyId":
  "b689b0a8-53d0-40ab-baf2-68738e2966ac" },
  "CacheBehaviors": { "Quantity": 1, "Items": [ { "PathPattern": "/assets/*",
  "TargetOriginId": "s3-assets",
    "ViewerProtocolPolicy": "redirect-to-https", "Compress": true,
    "AllowedMethods": { "Quantity": 2, "Items": ["GET","HEAD"] }, "CachePolicyId":
  "658327ea-f89d-4fab-a63d-7e88639e58f6" } ] } }
EOF
```

```
read DIST_ID CF_DOMAIN < <(aws cloudfront create-distribution --distribution-config
  file://dist.json --query 'Distribution.[Id,DomainName]' --output text)
save DIST_ID $DIST_ID; save CF_DOMAIN $CF_DOMAIN
cat > bucket-policy.json <<EOF
{"Version":"2012-10-17","Statement":
  [{"Sid":"AllowCloudFront","Effect":"Allow","Principal":
  {"Service":"cloudfront.amazonaws.com"},
 "Action":"s3:GetObject","Resource":"arn:aws:s3:::$ASSETS/*","Condition":{"StringEquals":
  {"AWS:SourceArn":"arn:aws:cloudfront::$ACCOUNT_ID:distribution/$DIST_ID"}}}]}
EOF
aws s3api put-bucket-policy --bucket $ASSETS --policy file://bucket-policy.json
aws cloudfront wait distribution-deployed --id $DIST_ID                          # 5-10
  minutes
curl -sI https://$CF_DOMAIN/assets/hello.html | grep -iE "^(HTTP|x-cache)"      # Miss,
  then Hit on the second call
curl -si https://$CF_DOMAIN/api/products | grep -iE "^(HTTP|x-app-cache|x-app-host)"   #
  app cache miss then hit; hosts vary
```

Optional custom domain (topic 12): request an ACM certificate for `shop.<your-domain>` in `us-east-1`, validate it with DNS, add the name as an alternate domain name with the certificate to the distribution, then create a Route 53 `A` alias record to `$CF_DOMAIN` using the CloudFront alias hosted zone ID `Z2FDTNDATAQYW2`.

Milestone 7 — Lambda: invalidate CloudFront after a deploy (topics 03, 07)

BASH

```
cat > index.py <<'EOF'
import os, time, boto3
def handler(event, context):
    cf = boto3.client("cloudfront")
    r = cf.create_invalidation(DistributionId=os.environ["DIST_ID"], InvalidationBatch={
        "Paths": {"Quantity": 1, "Items": ["/assets/*"]}, "CallerReference":
  str(time.time())})
    return {"invalidation": r["Invalidation"]["Id"]}
EOF
zip -q fn.zip index.py
echo '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
  {"Service":"lambda.amazonaws.com"},"Action":"sts:AssumeRole"}]}' > trust-lambda.json
aws iam create-role --role-name shop-capstone-invalidate --assume-role-policy-document
  file://trust-lambda.json >/dev/null
aws iam attach-role-policy --role-name shop-capstone-invalidate --policy-arn
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam put-role-policy --role-name shop-capstone-invalidate --policy-name invalidate --
  policy-document \
  "{\"Version\":\"2012-10-17\",\"Statement\":
  [{\"Effect\":\"Allow\",\"Action\":\"cloudfront:CreateInvalidation\",\"Resource\":\"arn:
  aws:cloudfront::$ACCOUNT_ID:distribution/$DIST_ID\"}]}"
sleep 15
aws lambda create-function --function-name shop-capstone-invalidate --runtime python3.12 -
  -handler index.handler --zip-file fileb://fn.zip \
```

```
  --role arn:aws:iam::$ACCOUNT_ID:role/shop-capstone-invalidate --environment "Variables=
  {DIST_ID=$DIST_ID}" >/dev/null
aws lambda wait function-active --function-name shop-capstone-invalidate
aws lambda invoke --function-name shop-capstone-invalidate out.json >/dev/null && cat
  out.json; echo
```

Milestone 8 — Observe it (topic 18, and 17 for investigation)

BASH

```
ALB_DIM=${ALB_ARN#*:loadbalancer/}; TG_DIM=${TG_ARN##*:}
aws cloudwatch put-metric-alarm --alarm-name shop-cap-alb-5xx --namespace
  AWS/ApplicationELB --metric-name HTTPCode_Target_5XX_Count \
  --dimensions Name=LoadBalancer,Value=$ALB_DIM --statistic Sum --period 60 --evaluation-
  periods 3 --datapoints-to-alarm 2 \
  --threshold 5 --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data
  notBreaching --alarm-actions $TOPIC_ARN
aws cloudwatch put-metric-alarm --alarm-name shop-cap-unhealthy --namespace
  AWS/ApplicationELB --metric-name UnHealthyHostCount \
  --dimensions Name=LoadBalancer,Value=$ALB_DIM Name=TargetGroup,Value=$TG_DIM --statistic
  Maximum --period 60 --evaluation-periods 2 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data
  notBreaching --alarm-actions $TOPIC_ARN
aws cloudwatch put-metric-alarm --alarm-name shop-cap-rds-cpu --namespace AWS/RDS --
  metric-name CPUUtilization \
  --dimensions Name=DBInstanceIdentifier,Value=shop-capstone --statistic Average --period
  300 --evaluation-periods 2 \
  --threshold 80 --comparison-operator GreaterThanThreshold --alarm-actions $TOPIC_ARN
aws cloudwatch put-metric-alarm --alarm-name shop-cap-redis-evictions --namespace
  AWS/ElastiCache --metric-name Evictions \
  --dimensions Name=CacheClusterId,Value=shop-cap-cache-001 --statistic Sum --period 300 -
  -evaluation-periods 1 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data
  notBreaching --alarm-actions $TOPIC_ARN
for i in $(seq 1 30); do curl -s -o /dev/null https://$CF_DOMAIN/api/products; done
aws logs tail /shop-api/capstone --since 10m | tail -5
```

Build a dashboard named `shop-capstone` from `RequestCount`, `TargetResponseTime` and `HealthyHostCount` (ALB), `CPUUtilization` (RDS), `CacheHitRate` (ElastiCache) and `mem_used_percent` (`ShopApi/EC2`) with `aws cloudwatch put-dashboard` (see CloudWatch §6.18, step 7).

5 Tasks

- [ ] **T1 — Billing alarm and trail first** *(≈45 min, beginner; Milestone 0; topics 18, 17)*
  - **Goal:** a spending alert and an audit log exist before any other resource does.
  - **Steps:**

1. Enable billing alerts; create the SNS topic, the billing alarm and the multi-region trail.
2. Force the alarm with `set-alarm-state`.
3. Run `lookup-events` for `CreateTrail`.

- **Done when:** the alarm email arrived and `aws cloudtrail get-trail-status --name shop-capstone-trail` shows `IsLogging: true`.

- [ ] **T2 — Network and tiered security groups** *(≈90 min, intermediate; Milestone 2; topics 04, 05, 06)*
  - **Goal:** private subnets that reach the internet only through NAT, and a security-group chain.
  - **Steps:**
    1. Build the VPC, subnets, route tables, NAT gateway and groups.
    2. Launch a throwaway `t3.micro` in `PRIV_A` with the app instance profile; open a Session Manager session.
    3. Run `curl -s https://checkip.amazonaws.com`, then `timeout 3 bash -c "</dev/tcp/<some-private-IP>/5432" && echo open || echo closed` against an address your security groups do not allow.
  - **Done when:** the printed IP equals the NAT gateway's Elastic IP (`aws ec2 describe-addresses`), the instance has no public IP, and the blocked port reports `closed`. Terminate the instance.

- [ ] **T3 — Encrypted data tier with a managed secret** *(≈90 min, intermediate; Milestones 1 and 4; topics 15, 16, 10, 11)*
  - **Goal:** RDS and Redis encrypted with your key, with no password in any file.
  - **Steps:**
    1. Create the key, RDS (`--manage-master-user-password`) and ElastiCache.
    2. Check `describe-db-instances` for `StorageEncrypted` and `KmsKeyId`, and `describe-secret` for the secret.
    3. Try reading the secret with a role that lacks `kms:Decrypt`.
  - **Done when:** both services show your key, the secret exists with RDS-managed rotation, and the unprivileged read fails with `AccessDenied`.

- [ ] **T4 — Run the app behind ALB and Auto Scaling** *(≈2 hours, advanced; Milestone 5; topics 01, 09, 14, 02, 07)*
  - **Goal:** two healthy instances behind an ALB that only CloudFront can reach.
  - **Steps:**

1. Build `userdata.final.sh`, the launch template, ALB, target group and ASG.
2. Call `/api/whoami` ten times through the ALB with the secret header (allow your IP temporarily).
3. Terminate one instance and watch the ASG replace it.

- **Done when:** both host names appear, `/api/products` shows `X-App-Cache: miss` then `hit`, a request without the header returns `403`, and the replacement instance is healthy within about five minutes.

- [ ] **T5 — CloudFront with OAC, caching and invalidation** *(≈90 min, intermediate; Milestones 6 and 7; topics 13, 03, 08, 12)*
  - **Goal:** one HTTPS URL for static and dynamic content, with a private bucket.
  - **Steps:**
    1. Create the distribution and bucket policy.
    2. Compare `X-Cache` on `/assets/hello.html` and `/api/products`.
    3. Change `hello.html` in S3, show the stale copy, run the Lambda to invalidate, and request the S3 URL directly.
  - **Done when:** the static file shows `Hit from cloudfront`, the API never does, the new content appears after invalidation, and the direct S3 URL and ALB URL are denied.

- [ ] **T6 — Break it: three failures and an alarm** *(≈90 min, advanced; Milestone 8; topics 14, 18, 11, 17)*
  - **Goal:** detect and diagnose faults using only the telemetry you built.
  - **Steps:**
    1. Stop `shop-api` on one instance with Session Manager; watch `UnHealthyHostCount`, the alarm email and the ASG replacing it.
    2. Remove the app group from the cache security group; read the Redis errors with the Logs Insights query `fields @timestamp, @message | filter @message like /redis/ | sort @timestamp desc`.
    3. Find who changed the security group with `aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=RevokeSecurityGroupIngress`. Restore each fault.
  - **Done when:** each fault produced a visible signal (alarm, log line, audit event) and you wrote a "symptom, evidence, fix" note for each.

- [ ] **T7 — Prove it: explain the request path and tear everything down** *(≈2 hours, advanced; all milestones; all 18 topics)*

- **Goal:** show you understand the request path and the cost model, then leave no trace.
- **Steps:**
  1. Without the guide, draw the path of `GET /api/products` through every component and name the security control at each hop.
  2. List which of the 18 topics each component exercises.
  3. Run the teardown in section 7 and the "nothing left" checks.
- **Done when:** your diagram is correct on review, and every check in section 7 prints an empty list (the KMS key shows `PendingDeletion`).

6 Verification checklist

- [ ] `aws cloudwatch describe-alarms --alarm-names billing-over-10-usd` shows the alarm and you received a test email.
- [ ] `aws cloudtrail get-trail-status --name shop-capstone-trail` reports `IsLogging: true`, and log files exist in the log bucket.
- [ ] `https://$CF_DOMAIN/assets/hello.html` returns 200, and a second request shows `X-Cache: Hit from cloudfront`.
- [ ] `https://$CF_DOMAIN/api/products` returns the three products, with `X-App-Cache` going from `miss` to `hit` within 60 seconds.
- [ ] Direct requests to `http://$ALB_DNS/` time out (security group) and the S3 bucket URL returns 403.
- [ ] Two instances are `InService` in two AZs, none has a public IP, and `UnHealthyHostCount` is 0.
- [ ] RDS and ElastiCache are not publicly accessible; their security groups only allow the app security group.
- [ ] `describe-db-instances` shows `StorageEncrypted: true` with your KMS key, and no password appears in `lab.env`, code or user-data.
- [ ] Logs from `/shop-api/capstone` appear in CloudWatch with 7-day retention, and the four alarms exist.
- [ ] The Lambda returned an invalidation ID, and CloudTrail shows its `CreateInvalidation` call.

7 Teardown

Do this the same day. Order matters (dependencies first). Run it from the working folder.

```bash
source lab.env; export AWS_REGION=us-east-1 AWS_PAGER=""
# 1. Compute and edge
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name shop-capstone-asg --
  force-delete
aws cloudfront get-distribution-config --id $DIST_ID > cur.json
jq '.DistributionConfig | .Enabled=false' cur.json > off.json
aws cloudfront update-distribution --id $DIST_ID --if-match "$(jq -r .ETag cur.json)" --
  distribution-config file://off.json >/dev/null
aws lambda delete-function --function-name shop-capstone-invalidate
aws elbv2 delete-load-balancer --load-balancer-arn $ALB_ARN
aws ec2 delete-launch-template --launch-template-name shop-capstone >/dev/null
# 2. Data tier (no final snapshot: lab only)
aws rds delete-db-instance --db-instance-identifier shop-capstone --skip-final-snapshot --
  delete-automated-backups >/dev/null
aws elasticache delete-replication-group --replication-group-id shop-cap-cache >/dev/null
aws rds wait db-instance-deleted --db-instance-identifier shop-capstone
aws elasticache wait replication-group-deleted --replication-group-id shop-cap-cache
aws rds delete-db-subnet-group --db-subnet-group-name shop-cap; aws elasticache delete-
  cache-subnet-group --cache-subnet-group-name shop-cap
# 3. Finish CloudFront (wait until the disabled config is deployed, then delete)
aws cloudfront wait distribution-deployed --id $DIST_ID
aws cloudfront delete-distribution --id $DIST_ID --if-match "$(aws cloudfront get-
  distribution-config --id $DIST_ID --query ETag --output text)"
aws cloudfront delete-origin-access-control --id $OAC_ID --if-match "$(aws cloudfront get-
  origin-access-control --id $OAC_ID --query ETag --output text)"
# 4. Target group, NAT gateway and the network (the ALB and ASG instances must be gone
  first)
sleep 120; aws elbv2 delete-target-group --target-group-arn $TG_ARN
aws ec2 delete-nat-gateway --nat-gateway-id $NAT_ID
until [ "$(aws ec2 describe-nat-gateways --nat-gateway-ids $NAT_ID --query
  'NatGateways[0].State' --output text)" = deleted ]; do sleep 15; done
aws ec2 release-address --allocation-id $EIP_ALLOC
for sg in $ALB_SG $APP_SG $DB_SG $CACHE_SG; do   # security groups reference each other:
  remove the rules first
  aws ec2 revoke-security-group-ingress --group-id $sg --security-group-rule-ids $(aws ec2
  describe-security-group-rules --filters Name=group-id,Values=$sg --query
  'SecurityGroupRules[?IsEgress==`false`].SecurityGroupRuleId' --output text) >/dev/null
done
for sg in $ALB_SG $APP_SG $DB_SG $CACHE_SG; do aws ec2 delete-security-group --group-id
  $sg; done
for rt in $PUB_RT $PRIV_RT; do
  for a in $(aws ec2 describe-route-tables --route-table-ids $rt --query
  'RouteTables[0].Associations[].RouteTableAssociationId' --output text); do aws ec2
  disassociate-route-table --association-id $a; done
  aws ec2 delete-route-table --route-table-id $rt
done
for s in $PUB_A $PUB_B $PRIV_A $PRIV_B; do aws ec2 delete-subnet --subnet-id $s; done
```

```
aws ec2 detach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID; aws ec2
  delete-internet-gateway --internet-gateway-id $IGW_ID
aws ec2 delete-vpc --vpc-id $VPC_ID
# 5. IAM, logs, buckets, alarms, trail
aws iam remove-role-from-instance-profile --instance-profile-name shop-capstone-app --
  role-name shop-capstone-app; aws iam delete-instance-profile --instance-profile-name
  shop-capstone-app
for p in AmazonSSMManagedInstanceCore CloudWatchAgentServerPolicy; do aws iam detach-role-
  policy --role-name shop-capstone-app --policy-arn arn:aws:iam::aws:policy/$p; done
aws iam delete-role-policy --role-name shop-capstone-app --policy-name read-db-secret; aws
  iam delete-role --role-name shop-capstone-app
aws iam detach-role-policy --role-name shop-capstone-invalidate --policy-arn
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam delete-role-policy --role-name shop-capstone-invalidate --policy-name invalidate;
  aws iam delete-role --role-name shop-capstone-invalidate
aws logs delete-log-group --log-group-name /shop-api/capstone; aws logs delete-log-group -
  -log-group-name /aws/lambda/shop-capstone-invalidate
aws cloudwatch delete-alarms --alarm-names shop-cap-alb-5xx shop-cap-unhealthy shop-cap-
  rds-cpu shop-cap-redis-evictions
aws cloudwatch delete-dashboards --dashboard-names shop-capstone 2>/dev/null
aws s3 rm s3://$ASSETS --recursive && aws s3api delete-bucket --bucket $ASSETS
aws cloudtrail delete-trail --name shop-capstone-trail; aws s3 rm s3://$LOGBUCKET --
  recursive && aws s3api delete-bucket --bucket $LOGBUCKET
# 6. KMS: remove the alias and schedule deletion (7 days is the minimum waiting period)
aws kms delete-alias --alias-name alias/shop-capstone; aws kms schedule-key-deletion --
  key-id $KEY_ID --pending-window-in-days 7
```

The RDS-managed master secret is removed together with the instance. Keep the billing alarm and its SNS topic: they are free and protect your next project.

Nothing-left checks. Each must print an empty list, and the key must show `PendingDeletion`:

BASH

```
aws ec2 describe-nat-gateways --filter Name=state,Values=available,pending --query
  'NatGateways[].NatGatewayId'
aws ec2 describe-addresses --query 'Addresses[].PublicIp'
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'
aws rds describe-db-instances --query 'DBInstances[].DBInstanceIdentifier'
aws elasticache describe-replication-groups --query
  'ReplicationGroups[].ReplicationGroupId'
aws ec2 describe-instances --filters Name=instance-state-name,Values=running,pending --
  query 'Reservations[].Instances[].InstanceId'
aws cloudfront list-distributions --query 'DistributionList.Items[].Id'
aws kms describe-key --key-id $KEY_ID --query KeyMetadata.KeyState --output text
```

Look at the Billing console's Bills page the next day: charges should have stopped growing.

8 Stretch goals and what to learn next

- Replace the hand-run CLI with code: rebuild the whole stack as Terraform modules with remote state in Level 7: Terraform §7.1.
- Add **EFS** for shared uploads, **RDS Multi-AZ** with a read replica and an **ElastiCache replica** with automatic failover; rehearse each failover and measure the downtime.
- Create a least-privilege **application database user**, store it in its own secret and enable alternating-user rotation (Secrets Manager §6.16).
- Put **AWS WAF** on the distribution, switch to **VPC origins** with a private ALB, and use a custom domain with ACM and Route 53.
- Review the bill in Cost Explorer and apply what you find: Level 20: Cost Optimization §20.1.
- Move from instances to containers in Level 8: Kubernetes §8.1, harden it with Level 11: Security §11.3 and instrument it with Level 10: Observability §10.1.
- Critique this design (single NAT gateway, single Redis node, no WAF) against the reference architecture walkthrough §19.10.

LEVEL 7 · PART III

Terraform / Infrastructure as Code

7

Goal: Define, version, review and safely change cloud infrastructure as code.

RECOMMENDED TIME	TOPICS
2-3 weeks	9

PRACTICE / CAPSTONE

Create a complete AWS environment with Terraform: VPC, subnets, security groups, EC2, ALB, RDS and Redis.

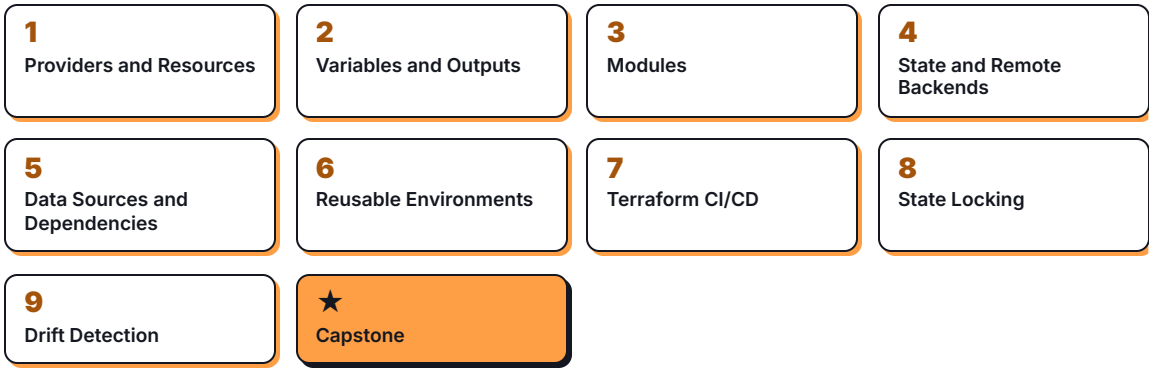
Topics

#	Topic	What you will learn
1	Providers and Resources **p.1405**	HCL, provider config, resource blocks, plan/apply/destroy workflow
2	Variables and Outputs **p.1419**	input types, validation, tfvars, locals, sensitive outputs
3	Modules **p.1431**	module structure, inputs/outputs, registry, versioning, composition
4	State and Remote Backends **p.1443**	what state is, S3 backend, security of state, import, moved blocks
5	Data Sources and Dependencies **p.1457**	data blocks, implicit/explicit depends_on, for_each/count, lifecycle
6	Reusable Environments **p.1471**	dev/stage/prod layout, workspaces vs directories, DRY patterns, Terragrunt mention
7	Terraform CI/CD **p.1485**	plan on PR, apply on merge, OIDC, policy checks, tflint/checkov
8	State Locking **p.1499**	DynamoDB/native S3 locking, stuck locks, force-unlock, concurrency
9	Drift Detection **p.1513**	manual changes, scheduled plan, detecting and reconciling drift
★	Level capstone project **p.1527**	Create a complete AWS environment with Terraform: VPC, subnets, security groups, EC2, ALB, RDS and Redis.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



7.1 Providers and Resources

Topic 1 of 9 | ~4 hours | Difficulty: Beginner

1 What it is

Terraform is a tool that turns text files into real infrastructure. You describe *what should exist* (a network, a server, a DNS record, a GitHub repository) in a declarative language called **HCL** (HashiCorp Configuration Language), and Terraform works out *how* to get there by calling cloud APIs. It remembers what it built in a **state** file, compares that with your code and the real world, and shows you a **plan** before it changes anything.

Two building blocks do most of the work. A **provider** is a plugin that knows how to talk to one API (AWS, Azure, Google Cloud, Kubernetes, Cloudflare, GitHub, even your local disk). A **resource** is one object that provider manages (`aws_vpc`, `aws_instance`, `cloudflare_record`). You write resources; Terraform downloads providers; the providers make the API calls. If this is your first contact with the idea of infrastructure as code, read Infrastructure as Code §0.3 first.

> **A note on names.** Terraform is developed by HashiCorp and has been under the Business Source License since 2023. **OpenTofu** is a community fork hosted by the Linux Foundation that keeps the same HCL language, providers and CLI workflow (the command is `tofu`). Everything in this level works with either tool unless a page says otherwise; where a recent feature differs we call it out.

2 Why it matters in DevOps

- **Reproducibility:** "click-ops" in a console cannot be reviewed, repeated or rolled back. A Terraform repo can rebuild an environment from scratch in minutes.
- **Review and audit:** infrastructure changes become pull requests. The `plan` output is the diff everyone approves.
- **Speed and safety:** `plan` shows exactly what will be created, changed or **destroyed** before you commit. Most "we deleted production" stories are a plan nobody read.

- **One workflow, many clouds:** the same `init / plan / apply` loop manages AWS, Cloudflare DNS, Datadog monitors and GitHub repositories.
- It is the foundation for everything later in this level: modules, state, CI/CD and drift detection.

3 Real-world system example

The Terraform Registry and its providers. The public Terraform Registry hosts thousands of providers. The AWS provider (`hashicorp/aws`) is a large open-source Go project that maps AWS API calls to resources such as `aws_vpc` and `aws_db_instance`. Vendors publish their own providers so customers can manage their products as code: Cloudflare maintains an official provider for DNS, WAF rules and Workers, Datadog for monitors and dashboards, and the open-source `integrations/github` provider lets organisations manage repositories, teams and branch-protection rules from a repo instead of a settings page.

Large organisations commonly layer a pipeline on top (a plan on every pull request, an apply after merge — see L7.7) so that nobody needs standing console access to production. AWS also has its own native IaC service, CloudFormation; Terraform's advantage is a single language across many providers.

4 Diagram

How a single `terraform apply` turns code into API calls:

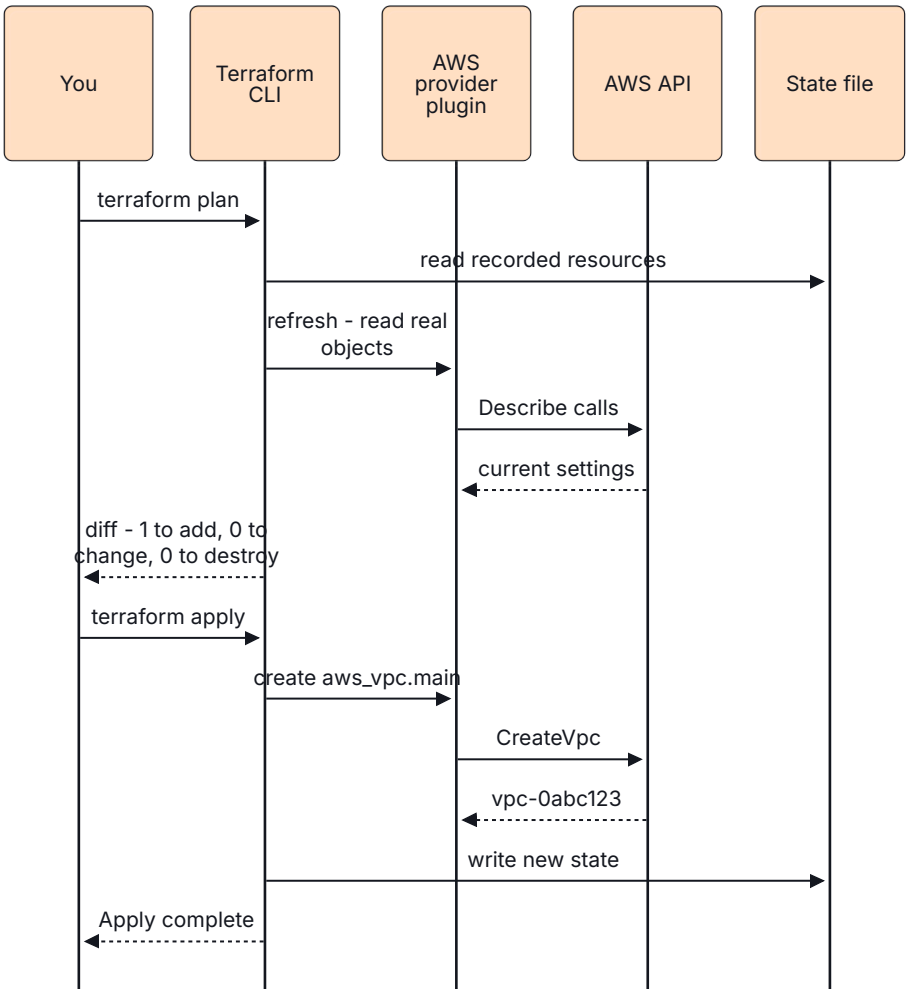


Figure 7.1-1

The core loop as a state machine:

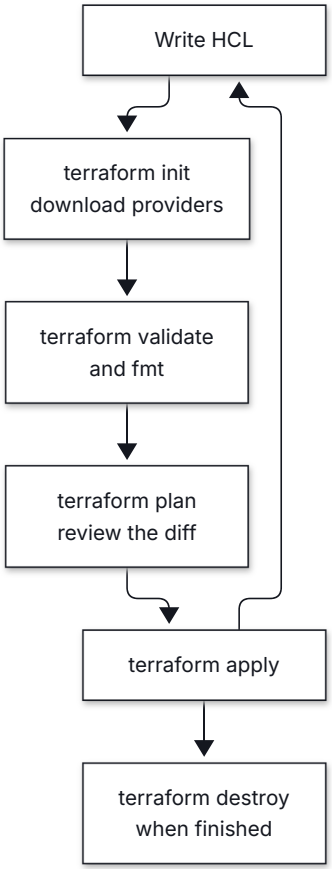


Figure 7.1-2

5 How it works

HCL in five minutes

HCL is made of **blocks** (a type, optional labels, a body in braces) and **arguments** (`name = expression`):

```hcl
resource "aws_s3_bucket" "logs" {     # block type "resource", labels: type + local name
  bucket = "shop-api-logs-example"    # argument
  tags = {                            # map value
    Team = "platform"
  }
}
```

- **Expressions** reference other things: `aws_s3_bucket.logs.id` (a resource attribute), `var.region` (input variable), `local.name` (computed value), `data.aws_caller_identity.me.account_id` (a data source).
- **Types:** string, number, bool, list, map, object. Strings interpolate with `"${var.env}-api"`.
- **Order does not matter** in the file. Terraform builds a dependency graph from the references (see L7.5).
- Files ending in `.tf` in one directory form a single configuration (the **root module**). Typical names: `versions.tf`, `main.tf`, `variables.tf`, `outputs.tf` — conventions only.

The `terraform` block and providers

```hcl
terraform {
  required_version = "≥ 1.9"          # which Terraform CLI versions may run this code

  required_providers {                # where each provider comes from and which versions
  are allowed
    aws = {
      source  = "hashicorp/aws"       # registry address: namespace/type
      version = "~> 5.0"              # any 5.x, never 6.0
    }
  }
}

provider "aws" {
  region = "us-east-1"
  default_tags {                      # applied to every taggable resource
    tags = { Project = "shop-api", ManagedBy = "terraform" }
  }
}
```

- `terraform init` downloads the providers and records exact versions and checksums in `.terraform.lock.hcl`. **Commit the lock file** so everyone (and CI) uses identical provider builds.
- `~> 5.0` means "5.0 or newer but below 6.0". This level's examples use AWS provider 5.x. Version 6.x exists; if you move to it, run `plan` and read the provider's upgrade guide because some arguments were renamed or deprecated.
- **Credentials never go in `.tf` files.** The AWS provider reads the standard chain: environment variables (`AWS_ACCESS_KEY_ID` ...), `AWS_PROFILE`, SSO, or an assumed role/OIDC identity in CI. See IAM §6.7.
- You can configure a provider more than once with `alias` (e.g. a second region) and pick it per resource with `provider = aws.west`.

Resources

```hcl
resource "<TYPE>" "<LOCAL_NAME>" {
  argument = value
}
```

`aws_vpc.main` is the **address** of the resource: it is how you refer to it in code, in the plan and in the state. Resources have **arguments** (what you set) and **attributes** (what you can read, including values only known after creation like `id` or `arn`). Each resource type's documentation page lists both.

The workflow commands

Command	What it does
`terraform init`	Downloads providers/modules, configures the backend. Run after any change to those.
`terraform fmt`	Rewrites files into canonical style. `-check` fails in CI if unformatted.
`terraform validate`	Checks syntax and internal consistency; makes no API calls.
`terraform plan`	Computes the diff between code, state and reality. Changes nothing.
`terraform plan -out=tfplan`	Saves the plan so `apply tfplan` executes exactly what was reviewed.
`terraform apply`	Executes the plan (asks for `yes` unless given a saved plan or `-auto-approve`).
`terraform destroy`	Plans and deletes everything in the state. Same as `apply -destroy`.
`terraform show` / `state list`	Inspect what Terraform currently believes exists.

Plan symbols: `+` create, `-` destroy, `~` update in place, `-/+` **replace** (destroy then create — look for these!).

6 Step-by-step guide

Prerequisites: Terraform (or OpenTofu) installed. No cloud account needed for steps 1-6: we use the `local` and `random` providers, which are free and harmless. Step 8 is an optional AWS step that can cost money; it ends with cleanup.

1. **Install and verify** (any one of these).

```bash
# macOS
brew install hashicorp/tap/terraform
# Linux: follow the official apt/yum repository instructions at
  developer.hashicorp.com/terraform/install
# No install at all - run it in Docker from your project folder:
alias terraform='docker run --rm -it -v "$PWD":/work -w /work
  hashicorp/terraform:latest'
terraform version
```

   Expect `Terraform v1.x.y` and your platform. (OpenTofu users: `tofu version`, then use `tofu` in place of `terraform`.)

2. **Create a project and write the configuration.**

```bash
mkdir tf-hello && cd tf-hello
```

   `main.tf`:

```hcl
terraform {
  required_version = "≥ 1.9"
  required_providers {
    local  = { source = "hashicorp/local",  version = "~> 2.5" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
}

resource "random_pet" "name" {
  length = 2
}

resource "local_file" "greeting" {
  filename = "${path.module}/hello-${random_pet.name.id}.txt"
  content  = "Hello from Terraform, ${random_pet.name.id}!\n"
```

```
}
```

3. **Initialise.**

```bash
terraform init
```

Look for `Terraform has been successfully initialized!` and a new `.terraform/` folder plus `.terraform.lock.hcl`.

4. **Format and validate.**

```bash
terraform fmt
terraform validate
```

Expect `Success! The configuration is valid.`

5. **Plan.**

```bash
terraform plan -out=tfplan
```

Illustrative output (yours will differ in detail):

```
Terraform will perform the following actions:

  # local_file.greeting will be created
  + resource "local_file" "greeting" {
      + content  = (known after apply)
      + filename = (known after apply)
      ...
    }

  # random_pet.name will be created
  + resource "random_pet" "name" {
      + id     = (known after apply)
      + length = 2
    }

Plan: 2 to add, 0 to change, 0 to destroy.
```

Notice `(known after apply)`: the file name depends on the random pet, so Terraform orders them correctly by itself.

6. **Apply the saved plan, then inspect.**

```bash
terraform apply tfplan
ls hello-*.txt && cat hello-*.txt
terraform state list
terraform show
```

   You should see the file, `local_file.greeting` and `random_pet.name` in the state, and a new `terraform.tfstate` file.

7. **Change something and read the diff.** Edit the `content` string, then run `terraform plan`. The plan shows `~ content` (an update) or `-/+` (replace) for `local_file.greeting` and nothing for `random_pet`. Change `length = 3` and see how a replaced `random_pet` forces the file to be replaced too. Apply, then finish with:

```bash
terraform destroy
```

   Type `yes`. The file disappears. Terraform's whole life cycle in one directory.

8. **Optional - first AWS resource (may cost pennies; do the cleanup).** Set a billing alarm first (see CloudWatch §6.18), export credentials for a sandbox account, then:

```hcl
provider "aws" { region = "us-east-1" }

resource "aws_s3_bucket" "demo" {
  bucket_prefix = "shop-api-demo-"     # AWS appends a random suffix, so the name is
  globally unique
  force_destroy = true                 # lets destroy delete the bucket even if it
  holds objects
}
```

   Add the `aws` provider to `required_providers`, run `terraform init && terraform apply`, confirm with `aws s3 ls`, then `terraform destroy`. An empty S3 bucket costs nothing, but build the habit of tearing down.

7 Real-world scenario

(Illustrative.) A startup's staging environment was created by hand over two years. When the engineer who built it left, nobody could say why the load balancer idle timeout was 300 seconds or which security group rule opened port 5432 to the world. A new engineer imports staging into Terraform, runs `terraform plan` and gets a long list of differences between the

code and reality. Working through the diff, the team discovers the open database port, closes it **in a pull request**, and the plan on that PR shows exactly one security-group rule being removed. Three months later the same code builds a production copy in 20 minutes. **Lesson:** the value of Terraform is less the first `apply` and more the permanent, reviewable record of intent that every later change builds on.

8 Common mistakes and best practices

Don't

- Hard-code access keys in `.tf` files or commit `terraform.tfstate`, `*.tfvars` with secrets, or `.terraform/` to Git.
- Run `terraform apply -auto-approve` locally against shared environments, or skip reading the plan.
- Leave `version` constraints off providers; an unpinned `init` on a new laptop can pull a breaking major version.
- Use `-target` as a routine workflow. It skips the dependency graph and leaves code and state out of sync.
- Edit resources in the console "just this once" (this becomes drift; see L7.9).

Do

- Pin `required_version` and provider versions with `~>`, and commit `.terraform.lock.hcl`.
- Run `fmt`, `validate` and `plan` in every pull request.
- Use `default_tags` and a consistent naming scheme so every resource is traceable to its code.
- Read every `-/+` and `-` in a plan twice: those can delete data.
- Use `plan -out` + `apply <planfile>` in automation so what is approved is what is applied.

9 Tasks

- [ ] **T1 — Hello, Terraform** *(≈30 min, beginner)*
  - **Goal:** run the full init/plan/apply/destroy loop with free providers.
  - **Steps:**
    1. Follow steps 1-6 of the guide in a fresh folder.
    2. Run `terraform state list` and `terraform show`.
    3. Run `terraform destroy`.

- **Done when:** the greeting file was created, listed in the state, and then removed by `destroy`.

- [ ] **T2 — Read a plan like a reviewer** *(≈30 min, beginner)*
  - **Goal:** distinguish update-in-place from replace.
  - **Steps:**
    1. Apply the config from T1 again.
    2. Change only the `content` of `local_file.greeting` and run `plan`; note the symbol.
    3. Change `random_pet.name` `length` and run `plan`; note which resources are replaced and why.
  - **Done when:** you can state, for both changes, whether each resource is `~` or `-/+` and what caused it.

- [ ] **T3 — Pin and lock** *(≈30 min, intermediate)*
  - **Goal:** make provider versions reproducible.
  - **Steps:**
    1. Look at `.terraform.lock.hcl` and find the `random` provider's version and `hashes`.
    2. Change the constraint to a range that excludes that version (for example `~> 3.0, < 3.5` if the locked version is newer) and run `terraform init`; read the error.
    3. Run `terraform init -upgrade` after fixing the constraint and watch the lock file change.
  - **Done when:** you can explain the difference between the constraint in `required_providers` and the pinned version in the lock file.

- [ ] **T4 — Break it and fix it** *(≈30 min, intermediate)*
  - **Goal:** recognise common errors quickly.
  - **Steps:**
    1. Misspell an argument (`filenme`), run `validate`, read the message, fix it.
    2. Reference a resource that does not exist (`random_pet.nme.id`) and fix it.
    3. Delete the generated file by hand, then run `plan`. Observe that Terraform notices and proposes to recreate it.
  - **Done when:** all three errors are fixed and a final `plan` reports `No changes` (after re-apply).

- [ ] **T5 — Your first cloud resource** *(≈45 min, intermediate; sandbox AWS account, billing alarm set)*
  - **Goal:** create and destroy a real AWS object with default tags.

- **Steps:**
  1. Configure the `aws` provider with `region = "us-east-1"` and `default_tags`.
  2. Create an `aws_s3_bucket` with `bucket_prefix = "shop-api-"` and apply it.
  3. Confirm with `aws s3api get-bucket-tagging` that the default tags are present.
  4. `terraform destroy`.
- **Done when:** the bucket showed the `Project` and `ManagedBy` tags, and `aws s3 ls` no longer lists it after destroy.

☐ **T6 — Prove it: teach the loop** *(≈30 min, advanced)*

- **Goal:** explain Terraform's mental model without notes.
- **Steps:**
  1. On a blank page, draw the relationship between code, state, plan and real infrastructure.
  2. Explain what `init`, `plan`, `apply` and `destroy` each read and write.
  3. Explain why the lock file is committed but the state file is not.
- **Done when:** a colleague can recreate your diagram after listening to you.

10 Self-check questions

1. What is the difference between a provider and a resource?
2. What does the `~> 5.0` version constraint allow, and where is the exact chosen version recorded?
3. What does a `-/+` in a plan mean, and why is it dangerous for a database?
4. Why use `plan -out=tfplan` followed by `apply tfplan` in automation?
5. Why must credentials never appear in `.tf` files, and how should the AWS provider get them instead?

ANSWERS

1. A provider is a plugin that talks to an API; a resource is one managed object of a type the provider defines.
2. Any 5.x version at or above 5.0 but below 6.0. The exact version and checksums are in `.terraform.lock.hcl`.
3. The resource will be destroyed and recreated. For a stateful resource that can mean data loss or downtime.
4. It guarantees that the reviewed plan is exactly what is executed, even if code or reality changed in between.

5. `.tf` files are committed and widely readable. Use environment variables, shared profiles/SSO or an assumed role (OIDC in CI).

11 Further reading

- Terraform documentation (developer.hashicorp.com/terraform) — Language, CLI and Tutorials sections.
- Terraform Registry (registry.terraform.io) — provider and resource reference pages, including the AWS provider docs.
- OpenTofu documentation (opentofu.org/docs) — if you prefer the community fork.
- *Terraform: Up & Running* by Yevgeniy Brikman — the standard book-length introduction.
- See also: Infrastructure as Code §0.3.

LAB NOTES What I ran, what I saw, what it means

7.2 Variables and Outputs

Topic 2 of 9 | ~4 hours | Difficulty: Beginner

1 What it is

A Terraform configuration is like a function. **Input variables** are its parameters, **local values** (`locals`) are its temporary variables, and **outputs** are its return values. Variables let one piece of code build a small `dev` or a large `prod` without copy-pasting; outputs hand values (a load-balancer DNS name, a database endpoint) to people, to other Terraform configurations and to CI scripts.

Variables have **types** (`string`, `number`, `bool`, `list`, `map`, `object`...), optional **defaults**, and can be **validated** so that bad input fails at `plan` time instead of after a 10-minute RDS creation. Values are supplied from several places with a clear precedence order: command line, `*.tfvars` files, environment variables (`TF_VAR_name`), then defaults.

2 Why it matters in DevOps

- **One codebase, many environments:** `dev`, `stage` and `prod` differ only in variable values (instance size, replica count).
- **Fail early:** validation turns "someone typed `t3.xxlarge`" or "CIDR overlaps" into a one-line error at `plan`.
- **Secrets hygiene:** marking a variable or output `sensitive` keeps passwords out of plan output and CI logs (but see the caveat below: it does **not** keep them out of state).
- **Interfaces between teams:** outputs are the contract between a network team's configuration and the application team that consumes the VPC ID.

3 Real-world system example

Public Terraform modules on the registry. The widely used community module `terraform-aws-modules/vpc/aws` exposes dozens of typed input variables (`cidr`, `azs`, `private_subnets`, `enable_nat_gateway`, `single_nat_gateway`) and many outputs (`vpc_id`, `private_subnets`, `nat_public_ips`). Teams call it with different values per environment — for example one NAT

gateway in dev for cost and one per Availability Zone in prod for resilience. The module's documented inputs and outputs *are* its public API, and that is exactly the pattern you will build in L7.3 and use in the capstone.

4 Diagram

Where a variable's value comes from, lowest to highest precedence, and where outputs go:

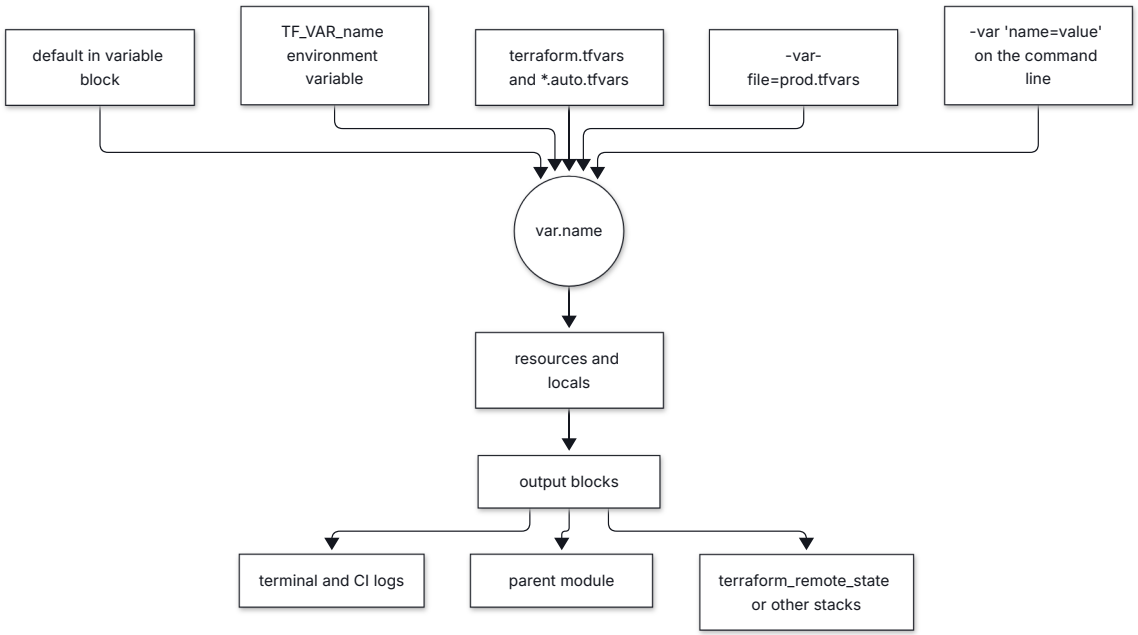


Figure 7.2-1

5 How it works

Input variables

```hcl
variable "environment" {
  description = "Deployment environment name."
  type        = string
  default     = "dev"

  validation {
    condition     = contains(["dev", "stage", "prod"], var.environment)
    error_message = "environment must be one of: dev, stage, prod."
  }
}
```

```hcl
variable "instance_count" {
  type    = number
  default = 2
  validation {
    condition     = var.instance_count >= 1 && var.instance_count <= 10
    error_message = "instance_count must be between 1 and 10."
  }
}

variable "allowed_cidrs" {
  description = "Source ranges allowed to reach the ALB."
  type        = list(string)
  default     = ["0.0.0.0/0"]
  validation {
    condition     = alltrue([for c in var.allowed_cidrs : can(cidrnetmask(c))])
    error_message = "Every entry must be a valid IPv4 CIDR such as 203.0.113.0/24."
  }
}

variable "db" {
  description = "Database settings."
  type = object({
    instance_class = string
    storage_gb     = number
    multi_az       = optional(bool, false)   # optional attribute with a default
  })
}

variable "db_password" {
  type      = string
  sensitive = true      # redacted in plan/apply output
}
```

Type cheat sheet:

Type	Example value	Notes
`string`, `number`, `bool`	`"us-east-1"`, `3`, `true`	primitives
`list(string)`	`["a", "b"]`	ordered, index with `[0]`
`set(string)`	unordered, unique	what `for_each` wants
`map(string)`	`{ Team = "platform" }`	key/value, same value type
`object({...})`	`{ size = "t3.micro", count = 2 }`	fixed named attributes, mixed types
`any`	anything	avoid; it disables type checking

A `validation` block can only reference the variable itself in older versions; Terraform 1.9 and later also allows referencing other variables and data sources. Use `can()` to turn an error into `false`.

Supplying values

Precedence, **highest wins**:

1. `-var "name=value"` and `-var-file=...` on the command line (later flags beat earlier ones)
2. `*.auto.tfvars` / `*.auto.tfvars.json` (alphabetical)
3. `terraform.tfvars` / `terraform.tfvars.json`
4. Environment variables `TF_VAR_name`
5. The variable's `default`

A `prod.tfvars` file is just assignments:

```hcl
environment    = "prod"
instance_count = 4
allowed_cidrs  = ["203.0.113.0/24"]
db = {
  instance_class = "db.t4g.medium"
  storage_gb     = 100
  multi_az       = true
}
```

If a required variable (no default) has no value, `plan` prompts interactively; in CI that hangs, so pass values explicitly or run with `-input=false` so it fails fast.

Locals

`locals` compute reusable values. They are not inputs; callers cannot override them.

```hcl
locals {
  name_prefix = "shop-api-${var.environment}"
  common_tags = {
    Project     = "shop-api"
    Environment = var.environment
  }
  is_prod = var.environment == "prod"
}

resource "aws_db_instance" "main" {
  identifier          = "${local.name_prefix}-pg"
  multi_az            = local.is_prod
```

```
  deletion_protection = local.is_prod
  # ...
}
```

Outputs

```hcl
output "alb_dns_name" {
  description = "Public DNS name of the load balancer."
  value       = aws_lb.app.dns_name
}

output "db_endpoint" {
  value = aws_db_instance.main.endpoint
}

output "db_password" {
  value     = var.db_password
  sensitive = true        # shown as (sensitive value) in the terminal
}
```

- `terraform output` lists all outputs; `terraform output -raw alb_dns_name` prints one without quotes (great in scripts); `terraform output -json` is machine-readable. Sensitive outputs are hidden unless you ask for them by name or use `-json`.
- In a module, outputs are the only way to expose values to the caller: `module.network.vpc_id`.

> **Warning:** `sensitive = true` only hides values in the CLI. **The value is still stored in plain text in the state file**, so state must be encrypted and access-controlled (L7.4). Better still, avoid putting secrets in Terraform at all: let RDS generate and store the master password in Secrets Manager (`manage_master_user_password = true`), or read secrets from Vault/Secrets Manager at runtime. Terraform 1.10 and 1.11 added **ephemeral values** and **write-only arguments** for some resources, which keep secrets out of state entirely; check provider support before relying on them.

6 Step-by-step guide

Prerequisites: Terraform or OpenTofu installed. No cloud account needed (we use `random` and `local` providers).

1. **Create the project.**

```bash
mkdir tf-vars && cd tf-vars
```

`versions.tf`:

```hcl
terraform {
  required_version = "≥ 1.9"
  required_providers {
    random = { source = "hashicorp/random", version = "~> 3.6" }
    local  = { source = "hashicorp/local",  version = "~> 2.5" }
  }
}
```

2. **Declare inputs with validation.** `variables.tf`:

```hcl
variable "environment" {
  type    = string
  default = "dev"
  validation {
    condition     = contains(["dev", "stage", "prod"], var.environment)
    error_message = "environment must be dev, stage or prod."
  }
}

variable "replicas" {
  type    = number
  default = 1
  validation {
    condition     = var.replicas ≥ 1 && var.replicas ≤ 5
    error_message = "replicas must be 1-5."
  }
}

variable "api_token" {
  type      = string
  sensitive = true
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

3. **Use locals, resources and outputs.** `main.tf`:

```hcl
locals {
  name = "shop-api-${var.environment}"
  tags = merge({ Project = "shop-api", Environment = var.environment }, var.tags)
}

resource "random_id" "suffix" {
  count       = var.replicas
  byte_length = 2
}

resource "local_file" "config" {
  filename = "${path.module}/${local.name}.json"
  content = jsonencode({
    name     = local.name
    replicas = [for r in random_id.suffix : "${local.name}-${r.hex}"]
    tags     = local.tags
  })
}
```

`outputs.tf`:

```hcl
output "replica_names" {
  value = [for r in random_id.suffix : "${local.name}-${r.hex}"]
}

output "token_echo" {
  value     = var.api_token
  sensitive = true
}
```

4. **See validation fail first.**

```bash
terraform init
terraform plan -var environment=qa -var api_token=x
```

Expect an error containing `environment must be dev, stage or prod.` Nothing was created.

5. **Supply values three ways.**

```bash
export TF_VAR_api_token="not-a-real-token"      # env var for the secret
terraform apply -var replicas=2 -auto-approve    # command line for the rest
```

Then create `prod.tfvars`:

```hcl
environment = "prod"
replicas    = 3
tags        = { CostCenter = "1234" }
```

Before the next apply, run `cat shop-api-dev.json`. Then run `terraform apply -var-file=prod.tfvars -auto-approve` and read the plan: the `dev` file is **replaced** by a `prod` file, because both runs share one state. That is a preview of why each environment needs its own state (L7.6). `cat shop-api-prod.json` to see the result.

6. **Read outputs.**

```bash
terraform output replica_names
terraform output -json replica_names | python3 -c 'import sys,json;
  print(len(json.load(sys.stdin)))'
terraform output token_echo          # prints the value because you asked by name
terraform output                      # token_echo = <sensitive>
```

7. **Prove sensitive is not secret.**

```bash
grep -n "not-a-real-token" terraform.tfstate
```

The value is there in plain text. This is why the state backend in L7.4 must be encrypted.

8. **Clean up.**

```bash
terraform destroy -var-file=prod.tfvars -auto-approve
```

7 Real-world scenario

(Illustrative.) A team shares one Terraform config for staging and production using `environment.tfvars` files. A developer runs `terraform apply` for staging but accidentally has `prod.tfvars` in the shell history; the plan shows `db_instance_class: "db.t4g.medium" → "db.t4g.micro"` on the production database, and the reviewer catches it. After the near-miss the team adds validation (`instance_class` must be in an allow-list for each environment), makes `deletion_protection` a function of `local.is_prod`, and stops running applies from laptops (see L7.7). **Lesson:** variables multiply power; validation and review gates keep that power safe.

8 Common mistakes and best practices

Don't

- Put real secrets in `terraform.tfvars` that is committed, or in `default` values.
- Use `type = any` everywhere; you lose validation and get confusing errors deep in a resource.
- Assume `sensitive = true` protects the state file. It does not.
- Create dozens of variables for every argument. Group related settings in an `object` and expose only what must vary.
- Rely on interactive prompts in automation.

Do

- Give every variable a `description` and a `type`; add `validation` for allow-lists, ranges and formats.
- Keep per-environment values in `*.tfvars` files and secrets in environment variables or a secrets manager.
- Use `locals` for naming prefixes and tag maps so they are defined once.
- Give every output a `description`; mark sensitive ones `sensitive = true`.
- Run `terraform plan -input=false` in CI so missing variables fail instead of hanging.

9 Tasks

- [ ] **T1 — Build the vars project** *(≈45 min, beginner)*
  - **Goal:** declare typed variables and see two environments produced from one code base.
  - **Steps:**
    1. Follow the guide through step 5.
    2. Save a copy of the dev JSON (`cp shop-api-dev.json /tmp/dev.json`) before applying with `prod.tfvars`, then compare it with the prod file.
  - **Done when:** you have a dev and a prod JSON file with different replica counts and tags, and can explain why the second apply replaced the first file.
- [ ] **T2 — Prove the precedence order** *(≈30 min, intermediate)*
  - **Goal:** learn which value wins.
  - **Steps:**
    1. Set `replicas` as a default (1), in `terraform.tfvars` (2), in `TF_VAR_replicas` (3) and with `-var` (4).

2. After each addition run `terraform plan` and look at the planned number of `random_id` resources.
3. Write the order you observed.

- **Done when:** your observed order matches the precedence list in section 5.

☐ **T3 — Add smarter validation** *(≈30 min, intermediate)*

- **Goal:** reject bad input early.
- **Steps:**
  1. Add a `cidr` variable and validate it with `can(cidrnetmask(var.cidr))`.
  2. Add a validation that `environment == "prod"` requires `replicas >= 2` (Terraform 1.9+ lets a validation reference another variable).
  3. Trigger each error on purpose.
- **Done when:** both bad inputs fail at `plan` with your custom messages and good inputs pass.

☐ **T4 — Break it: leak a secret and fix it** *(≈30 min, intermediate)*

- **Goal:** see exactly what `sensitive` does and does not do.
- **Steps:**
  1. Run step 7 and find the token in `terraform.tfstate`.
  2. List, in writing, three places the state file might end up (laptop, backup, CI cache…) and who could read it.
  3. Change the design so the token is not stored in Terraform at all (for example, the app reads it from a secrets manager at runtime and Terraform only creates the empty secret).
- **Done when:** you can show a `grep` of the state that no longer contains the token value.

☐ **T5 — Outputs for automation** *(≈30 min, intermediate)*

- **Goal:** consume outputs from a script.
- **Steps:**
  1. Write `deploy.sh` that calls `terraform output -json replica_names`, loops over the names with `python3` or `jq` and prints one line per replica.
  2. Make it fail clearly if the output does not exist.
- **Done when:** the script prints one line per replica and exits non-zero when run in an empty directory.

☐ **T6 — Prove it: environment matrix** *(≈45 min, advanced)*

- **Goal:** design the variable interface for a real stack without help.

- **Steps:**
  1. For a `shop-api` stack (VPC CIDR, instance type, DB class, Multi-AZ, allowed CIDRs), write `variables.tf` with types and validations and `dev.tfvars` / `prod.tfvars`.
  2. Explain which values belong in `tfvars`, which in `locals`, which in env vars, and why.
- **Done when:** `terraform validate` passes and `plan` with each tfvars file is accepted, while an invalid value is rejected.

10 Self-check questions

1. List the sources of variable values from highest to lowest precedence.
2. What is the difference between a variable and a local?
3. Does `sensitive = true` encrypt the value in state? What does it actually do?
4. Why is `type = object({...})` with `optional()` attributes better than many loose variables?
5. How do you pass a value to a module's child and read something back?

ANSWERS

1. `-var` / `-var-file` (CLI), `*.auto.tfvars`, `terraform.tfvars`, `TF_VAR_*` environment variables, then the default.
2. A variable is an input the caller can set; a local is an internal computed value that cannot be overridden.
3. No. It only redacts the value in CLI output and plan diffs. The state still holds it in plain text.
4. One cohesive, typed structure with defaults for optional parts is easier to document, validate and keep stable than a sprawl of variables.
5. Pass it as an argument in the `module` block (`module "x" { cidr = "10.0.0.0/16" }`); read it via `module.x.<output_name>`, which must be declared as an `output` in the child.

11 Further reading

- Terraform documentation: *Language → Values and Variables* (input variables, outputs, local values) and *Functions*.
- *Terraform: Up & Running* by Yevgeniy Brikman — chapters on inputs, outputs and secrets.
- Next: L7.3 Modules. Related: Secrets Manager §6.16.

LAB NOTES What I ran, what I saw, what it means

7.3 Modules

Topic 3 of 9 | ~5 hours | Difficulty: Intermediate

1 What it is

A **module** is a directory of `.tf` files that you call from other Terraform code, exactly like a function or a library. Every Terraform configuration is already a module: the directory where you run `terraform apply` is the **root module**. When the root calls another directory (or a package from the registry or Git) with a `module` block, that is a **child module**.

A good module hides a pile of low-level resources behind a small, typed interface. "Give me a VPC with these CIDR ranges" might be 25 resources inside (VPC, subnets, route tables, internet gateway, NAT gateways, associations) but is one block of 8 arguments for the caller. Modules are how you stop copy-pasting the same 300 lines into every environment.

2 Why it matters in DevOps

- **Consistency:** every environment's network or database is built the same way, so prod behaves like staging.
- **Leverage:** one fix to the module (say, turning on encryption) rolls out to every consumer on its next upgrade.
- **Cognitive load:** teams reason about "the `network` module", not 25 individual resources.
- **Governance:** platform teams publish approved modules (encrypted buckets, hardened databases) that app teams consume — a cornerstone of platform engineering §0.13.
- **Risk if misused:** a badly designed module (too generic, too many flags) is harder to understand than the raw resources, and an unpinned module version can change production unexpectedly.

3 Real-world system example

The Terraform Registry's `terraform-aws-modules`**.** The community project `terraform-aws-modules` publishes widely used modules for VPCs, EKS clusters, RDS, security groups, ALBs and more. A team writes something like:

```hcl
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  # inputs ...
}
```

and gets a documented, versioned, tested interface; upgrading is a deliberate change of the `version` line reviewed in a pull request. HashiCorp's registry also gives modules a standard layout and semantic versions, and many companies mirror that model internally with a private registry or Git tags. Whether to use a big public module or write your own thin one is a real trade-off; this topic teaches you to do both.

4 Diagram

How a root module composes child modules, and how data flows between them through outputs:

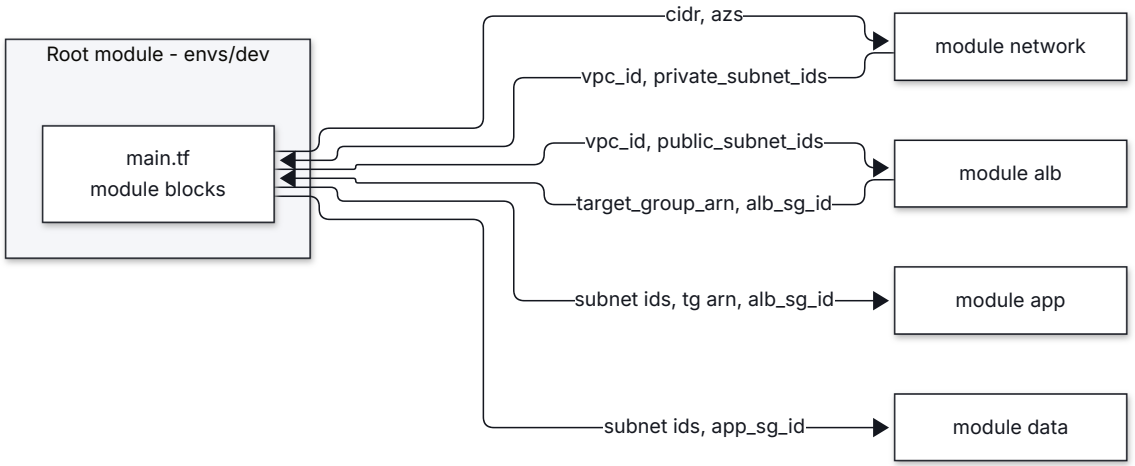


Figure 7.3-1

A child module never reads the parent's variables directly. Everything crosses the boundary as an input or an output.

5 How it works

Anatomy of a module

```
modules/network/
├── main.tf          # resources
├── variables.tf     # inputs (the interface)
├── outputs.tf       # outputs (the interface)
├── versions.tf      # required_providers (no provider blocks!)
└── README.md        # what it does, an example call
```

- **Providers are configured by the root**, not inside reusable modules. A child module declares which providers it needs under `required_providers` and inherits the configuration passed by the caller.
- Treat `variables.tf` and `outputs.tf` as a public API. Renaming a variable is a breaking change.

Calling a module

```hcl
module "network" {
  source = "../../modules/network"   # local path; or a registry address / Git URL

  name     = "shop-api-dev"
  cidr     = "10.10.0.0/16"
  az_count = 2
}

resource "aws_security_group" "app" {
  vpc_id = module.network.vpc_id        # read the child's output
  # ...
}
```

Module `source` options:

Source	Example	Versioning
Local path	`"../modules/network"`	none - same repo, same commit
Terraform Registry	`"terraform-aws-modules/vpc/aws"`	`version = "~> 5.0"`
Git	`"git::https://example.com/org/tf-modules.git//network?ref=v1.4.2"`	pin to a tag or commit with `?ref=`
S3 / HTTP archive	`"https://example.com/network-1.4.2.zip"`	version in the file name

`terraform init` downloads modules into `.terraform/modules/`. Re-run `init` after changing a `source` or `version`.

Writing a good module

```hcl
# modules/network/variables.tf
variable "name"     { type = string }
variable "cidr"     { type = string }
variable "az_count" {
  type    = number
  default = 2
}

# modules/network/main.tf
data "aws_availability_zones" "available" { state = "available" }

locals {
  azs = slice(data.aws_availability_zones.available.names, 0, var.az_count)
}

resource "aws_vpc" "this" {
  cidr_block           = var.cidr
  enable_dns_hostnames = true
  tags                 = { Name = var.name }
}

resource "aws_subnet" "public" {
  count                   = var.az_count
  vpc_id                  = aws_vpc.this.id
  cidr_block              = cidrsubnet(var.cidr, 8, count.index)        # 10.10.0.0/24,
  10.10.1.0/24
  availability_zone       = local.azs[count.index]
  map_public_ip_on_launch = true
  tags                    = { Name = "${var.name}-public-${count.index}" }
}

# modules/network/outputs.tf
output "vpc_id"            { value = aws_vpc.this.id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }
```

(`cidrsubnet(prefix, newbits, netnum)` carves smaller blocks out of a larger one; the subnet maths is covered in IPv4, IPv6, CIDR and Subnets §2.3.)

Module design guidelines

1. **Small, focused interface.** Expose what callers must decide; hard-code safe defaults for the rest.
2. **Opinionated defaults:** encryption on, public access off, deletion protection configurable.

3. **Outputs for everything a caller will need** (IDs, ARNs, endpoints, security group IDs).
4. **Composition over inheritance:** prefer a root that wires `network` + `app` + `data` together over one giant module that builds everything and takes 80 flags. Pass IDs between modules instead of nesting deeply.
5. **Versioning:** tag releases with semantic versions (`v1.4.2`); consumers pin them. Breaking interface change = new major.
6. **Document and example:** a README with a minimal `module` call, and an `examples/` folder you can `plan` against.
7. **Test:** `terraform validate`, `terraform test` (native test framework, `*.tftest.hcl`, Terraform 1.6+; OpenTofu has an equivalent), or Terratest.

Refactoring into modules without recreating anything

If resources already exist at `aws_vpc.main` and you move them into `module.network`, their *address* changes, and Terraform would plan to destroy and recreate. Use a `moved` block (L7.4):

```hcl
moved {
  from = aws_vpc.main
  to   = module.network.aws_vpc.this
}
```

6 Step-by-step guide

Prerequisites: Terraform or OpenTofu. We build a module with the free `random` / `local` providers so no cloud account is needed; the capstone applies the same pattern to AWS.

1. **Layout.**

```bash
mkdir -p tf-modules/modules/service tf-modules/envs/dev tf-modules/envs/prod
cd tf-modules
```

2. **Write the child module** `modules/service/versions.tf`, `variables.tf`, `main.tf`,

`outputs.tf`.

```hcl
# modules/service/versions.tf
terraform {
  required_providers {
    random = { source = "hashicorp/random", version = "~> 3.6" }
    local  = { source = "hashicorp/local",  version = "~> 2.5" }
  }
}
```

```hcl
# modules/service/variables.tf
variable "name" {
  type = string
}
variable "replicas" {
  type    = number
  default = 1
  validation {
    condition     = var.replicas >= 1
    error_message = "replicas must be at least 1."
  }
}
variable "output_dir" {
  type = string
}
```

```hcl
# modules/service/main.tf
resource "random_pet" "instance" {
  count  = var.replicas
  prefix = var.name
  length = 1
}

resource "local_file" "manifest" {
  filename = "${var.output_dir}/${var.name}.manifest"
  content  = join("\n", random_pet.instance[*].id)
}
```

```hcl
# modules/service/outputs.tf
output "instance_names" {
  description = "Generated instance names."
  value       = random_pet.instance[*].id
}
output "manifest_path" {
  value = local_file.manifest.filename
}
```

3. **Call it from two environments.** `envs/dev/main.tf`:

```hcl
terraform {
  required_version = "⩾ 1.9"
}

module "api" {
  source     = "../../modules/service"
  name       = "shop-api"
  replicas   = 1
  output_dir = path.module
}

module "worker" {
  source     = "../../modules/service"   # same module, second instance
  name       = "shop-worker"
  replicas   = 2
  output_dir = path.module
}

output "api_instances"    { value = module.api.instance_names }
output "worker_instances" { value = module.worker.instance_names }
```

`envs/prod/main.tf` is the same with `replicas = 3` and `4`.

4. **Run it.**

```bash
cd envs/dev
terraform init
terraform apply -auto-approve
terraform output
terraform state list
```

`init` prints `Initializing modules...` and lists both module calls. The state addresses look like `module.api.random_pet.instance[0]` and `module.worker.random_pet.instance[1]`: the module path is part of the address.

5. **Change the module once, see it everywhere.** In `modules/service/main.tf` add a trailing newline to `content` (`"${join("\n", random_pet.instance[*].id)}\n"`), then run `terraform plan` in both `envs/dev` and `envs/prod`. Both show the manifest files updating from the single edit.
6. **Version a module with Git.** Put `modules/` in its own repository, commit, `git tag v1.0.0`, and change the call:

   ```hcl
   source = "git::https://github.com/YOUR-ORG/tf-modules.git//service?ref=v1.0.0"
   ```

   Run `terraform init -upgrade` and look in `.terraform/modules/`. Pinning to a tag means later commits cannot surprise you.
7. **Try a registry module** (read-only exploration). Browse `registry.terraform.io/modules/terraform-aws-modules/vpc/aws`, read its Inputs and Outputs tabs, and note how each maps to what you just wrote.
8. **Clean up.**

   ```bash
   terraform destroy -auto-approve   # run in envs/dev, then in envs/prod
   ```

7 Real-world scenario

(Illustrative.) Three teams each had their own copy of "the standard VPC" pasted into their repos. A security finding required enabling VPC flow logs everywhere. The platform team would have had to open three pull requests, each with slightly different code. Instead they extract a `network` module, tag it `v2.0.0` with flow logs enabled by default, and teams upgrade with a one-line `version` / `ref` change. One team pinned nothing and picked up a breaking change during an unrelated apply, causing a surprise subnet replacement. **Lesson:** modules give leverage in both directions. Pin versions, read the plan when you upgrade, and treat the module interface like an API.

8 Common mistakes and best practices

Don't

- Put `provider` blocks inside reusable modules (it blocks `count` / `for_each` and removal of the module).
- Create "god modules" with dozens of boolean flags that toggle half the resources.

- Try to reach into a module's internals (`module.x.aws_vpc.this.id` does not work). Expose an output instead.
- Use unpinned Git `main` or registry modules without a `version`.
- Rename resources inside a published module without a `moved` block; every consumer would plan a destroy/create.

Do

- Pin module versions (`version = "~> 5.0"` or `?ref=v1.4.2`) and update them intentionally in their own PR.
- Keep modules small and composable; wire them together in the root.
- Use `moved` blocks to refactor safely.
- Include a README, typed variables with descriptions, outputs for every ID consumers need, and a tested example.
- Prefer a thin wrapper around a well-maintained public module only when it genuinely saves effort; own the code you depend on for critical paths.

9 Tasks

- [ ] **T1 — Build and call a module** *(≈45 min, beginner)*
  - **Goal:** create the `service` module and call it twice.
  - **Steps:**
    1. Follow steps 1-4 of the guide.
    2. Run `terraform state list` and identify the module path in the addresses.
  - **Done when:** `terraform output` shows instance names for both `api` and `worker` and the state shows `module.api.*` and `module.worker.*`.
- [ ] **T2 — Extend the interface** *(≈30 min, intermediate)*
  - **Goal:** add an input and an output without breaking existing callers.
  - **Steps:**
    1. Add an optional `tags` (`map(string)`, default `{}`) input written into the manifest as JSON.
    2. Add an output `instance_count`.
    3. Run `plan` in both envs; callers that do not set `tags` should still plan cleanly.
  - **Done when:** `dev` still plans with a single in-place change to the manifest content, and `prod` can pass tags.
- [ ] **T3 — Refactor with `moved`** *(≈45 min, intermediate)*

- **Goal:** move a resource into a module without recreating it.
- **Steps:**
  1. In a new folder, create `random_pet.solo` in the root and apply it. Note the pet name.
  2. Move it into a module `pets` (resource renamed to `this`) and add a `moved { from = random_pet.solo to = module.pets.random_pet.this }` block.
  3. Run `plan`.
- **Done when:** the plan shows `random_pet.solo has moved to module.pets.random_pet.this` with `0 to add, 0 to change, 0 to destroy`, and the pet name is unchanged after apply.

☐ **T4 — Break it: unpinned module upgrade** *(≈30 min, intermediate)*

- **Goal:** feel why versions matter.
- **Steps:**
  1. Put the module in a Git repo with tags `v1.0.0` and `v1.1.0`, where `v1.1.0` renames the `name` variable to `service_name`.
  2. Consume `v1.0.0`, then change `ref` to `v1.1.0` and run `plan`; read the error.
  3. Fix the call site and re-plan.
- **Done when:** you can explain why that rename should have been a new major version (`v2.0.0`).

☐ **T5 — Write a test** *(≈45 min, advanced)*

- **Goal:** test a module's behaviour automatically.
- **Steps:**
  1. Create `modules/service/tests/basic.tftest.hcl` with a `run` block that sets `replicas = 3` and asserts `length(output.instance_names) == 3`.
  2. Add a second `run` with `replicas = 0` and `expect_failures = [var.replicas]`.
  3. Run `terraform test` from `modules/service`.
- **Done when:** `terraform test` reports both runs as passing.

☐ **T6 — Prove it: design a `data` module** *(≈45 min, advanced)*

- **Goal:** design a real module interface from scratch.
- **Steps:**
  1. On paper, specify a module that creates an RDS PostgreSQL instance: its variables (with types and defaults), its outputs, and which resources it contains.
  2. Decide what is a variable versus a hard-coded safe default (encryption, backups, public access).

3. Explain how the root would connect it to the `network` and `app` modules.

- **Done when:** a teammate can implement the module from your interface description alone.

10 Self-check questions

1. What is the root module? How does a child module receive and return data?
2. Why should reusable modules not contain `provider` blocks?
3. How do you version a module from the registry and from Git?
4. What does a `moved` block do and when do you need it?
5. When is a thin module around three resources a bad idea?

ANSWERS

1. The root module is the directory you run Terraform in. Children receive inputs through arguments in the `module` block and return data through `output` blocks.
2. Provider configuration belongs to the caller. Provider blocks in a child make it hard to reuse across regions/accounts and block `count`, `for_each` or removing the module.
3. Registry: `version = "~> 5.0"`. Git: a `?ref=<tag or commit>` suffix on the source URL.
4. It tells Terraform that an object has a new address (renamed or moved into a module) so it updates state instead of destroying and recreating it.
5. When it adds an interface to learn without hiding real complexity, or exposes every underlying argument as a variable anyway. Wrapping with no abstraction is overhead.

11 Further reading

- Terraform documentation: *Language → Modules* (module blocks, sources, composition, publishing) and *Testing*.
- Terraform Registry module pages, especially `terraform-aws-modules/vpc/aws`, as examples of interface design.
- *Terraform: Up & Running* by Yevgeniy Brikman — "How to Create Reusable Infrastructure with Terraform Modules".
- See also: Infrastructure Templates §18.6.

LAB NOTES What I ran, what I saw, what it means

7.4 State and Remote Backends

Topic 4 of 9 | ~5 hours | Difficulty: Intermediate

1 What it is

Terraform keeps a **state file** (JSON, `terraform.tfstate` by default) that maps every resource in your code to a real object in the real world: `aws_vpc.main` → `vpc-0a1b2c3d4e5f67890`, together with every attribute the provider returned. On each `plan`, Terraform reads the state, refreshes it from the cloud API, and compares it with your configuration to decide what to create, change or delete. Without state, Terraform would have no idea which resources it owns.

By default the state lives in a file next to your code (the **local backend**). That is fine for a solo experiment and dangerous for a team: it can be lost, overwritten by a colleague, or committed to Git with secrets inside. A **remote backend** stores the state in shared, durable, access-controlled storage, such as an **S3 bucket** on AWS, and supports **locking** so that two people cannot change the same state at once (L7.8).

2 Why it matters in DevOps

- **State is the source of truth for ownership.** Lose it and Terraform will try to create everything again (duplicates and name collisions) or leave orphaned resources you pay for.
- **State contains secrets.** Database passwords, private keys and tokens appear in plain text. Whoever can read the state can often read your secrets, so it needs encryption, versioning and tight IAM.
- **Team workflows depend on it.** CI pipelines, drift detection and multiple engineers all need one shared, consistent state.
- **Refactoring needs it.** Renaming, moving into modules and adopting existing infrastructure are state operations (`moved`, `import`, `removed`).

3 Real-world system example

S3 as a Terraform backend, and HCP Terraform. The `s3` backend is one of the most common remote backends for AWS teams: a versioned, encrypted S3 bucket holds one state object per stack (for example `prod/network/terraform.tfstate`). Until recently locking required a companion DynamoDB table; recent Terraform releases can lock with a lock file in the same bucket (`use_lockfile = true`, introduced in Terraform 1.10 and stable in 1.11; OpenTofu supports it too). HashiCorp's managed service, **HCP Terraform** (formerly Terraform Cloud), and tools such as Spacelift, env0 and Atlantis are other ways to run shared state and runs. The Terraform documentation itself warns that state may contain sensitive data and recommends encrypting it at rest and restricting access.

4 Diagram

How a remote backend is used during a run:

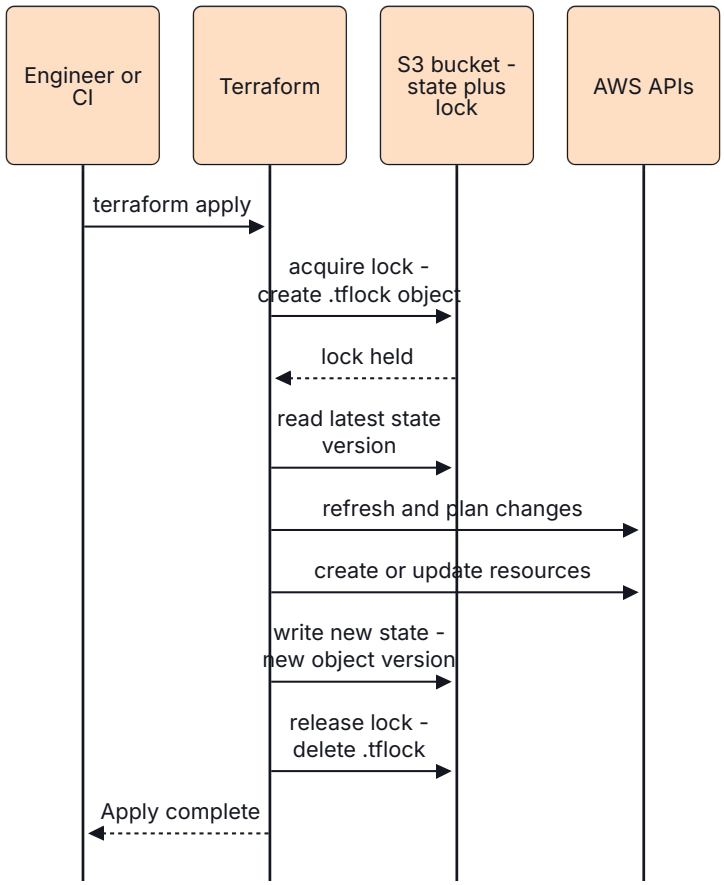


Figure 7.4-1

State is the join between code and reality:

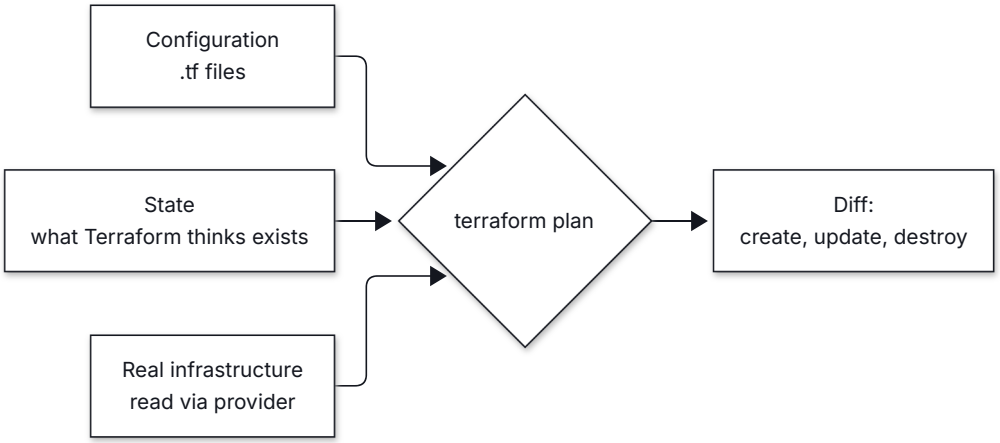


Figure 7.4-2

5 How it works

What is in the state

```bash
terraform state list                      # every resource address
terraform state show aws_vpc.main         # its recorded attributes
terraform show -json | jq '.values'       # entire state as JSON
```

State records resource addresses, provider, dependencies, and all attribute values (including secrets, in plain text). Never edit the JSON by hand; use the `terraform state` subcommands or `moved` / `import` / `removed` blocks.

Configuring the S3 backend

```hcl
terraform {
  backend "s3" {
    bucket       = "shop-api-tfstate-123456789012"
    key          = "dev/network/terraform.tfstate"   # one object per stack
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true      # native S3 locking (Terraform 1.10+/1.11+, OpenTofu 1.10+)
  }
}
```

- Backend blocks **cannot use variables or locals**. Pass differing values with partial configuration at init time: `terraform init -backend-config=backend-dev.hcl`.
- **Native S3 locking** (`use_lockfile`) writes a `<key>.tflock` object using S3 conditional writes. The older approach, still supported but deprecated in current Terraform, is `dynamodb_table = "terraform-locks"` with a DynamoDB table keyed on `LockID`. Existing setups may use either or both while migrating. Details in L7.8.
- Authentication uses the normal AWS credential chain. In CI, assume a role via OIDC (L7.7).
- Changing backends: edit the block and run `terraform init -migrate-state` to copy the state across.

Securing the state bucket

Control	Why
Block Public Access on the bucket	State must never be public.
Versioning on	Recover from corruption or a bad apply by restoring a previous version.
Encryption (SSE-S3 by default, or SSE-KMS with your own key)	Protect at rest; KMS gives you key policy and audit (KMS §6.15).
Bucket policy / IAM least privilege	Only the CI role and a few humans can read; only CI writes. Reading state is as sensitive as reading secrets.
Separate bucket/account for state	Blast-radius isolation; keep it out of the accounts Terraform manages.
TLS-only policy	Deny requests where `aws:SecureTransport` is false.
CloudTrail data events	Audit who read or wrote state (CloudTrail §6.17).
Lifecycle rule expiring old noncurrent versions	Keep history without unbounded growth.

Also: split state into several smaller stacks (network, data, app per environment) so a mistake or a lock affects less, and plans are faster. Never commit `*.tfstate` or `*.tfstate.backup` to Git; add them to `.gitignore`.

Changing state safely

Need	Tool
Adopt an existing resource into Terraform	`import` block (Terraform 1.5+) or `terraform import ADDRESS ID`
Rename a resource / move into a module	`moved` block (1.1+), or `terraform state mv`

Need	Tool
Stop managing a resource **without destroying it**	`removed` block with `lifecycle { destroy = false }` (1.7+), or `terraform state rm`
Inspect a resource	`terraform state show ADDRESS`
Re-read reality into state	`terraform apply -refresh-only`

```hcl
# Adopt an existing bucket
import {
  to = aws_s3_bucket.legacy
  id = "shop-api-legacy-assets"           # the ID format is on the resource's docs page
}

# Rename without replacing
moved {
  from = aws_instance.web
  to   = aws_instance.app
}

# Stop managing, keep the real thing
removed {
  from = aws_instance.old_bastion
  lifecycle { destroy = false }
}
```

`terraform plan -generate-config-out=generated.tf` can write draft configuration for `import` blocks; review it, delete computed or noisy arguments, and merge it into your code.

6 Step-by-step guide

Prerequisites: Terraform or OpenTofu (Terraform 1.11+ for stable `use_lockfile`), AWS CLI with sandbox credentials, a billing alarm. S3 storage for a few KB costs a fraction of a cent; you can leave the bucket for later topics or delete it in the last step.

1. **Bootstrap the state bucket** (the chicken-and-egg step: the bucket must exist before

Terraform can store state in it). We use the CLI once.

```bash
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET="shop-api-tfstate-${ACCOUNT_ID}"
aws s3api create-bucket --bucket "$BUCKET" --region us-east-1
aws s3api put-public-access-block --bucket "$BUCKET" \
  --public-access-block-configuration
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuck
  ets=true
aws s3api put-bucket-versioning --bucket "$BUCKET" --versioning-configuration
  Status=Enabled
aws s3api put-bucket-encryption --bucket "$BUCKET" --server-side-encryption-
  configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
echo "$BUCKET"
```

For regions other than `us-east-1`, add `--create-bucket-configuration LocationConstraint=<region>`.

2. **Start with local state** so you can feel the migration. `main.tf`:

```hcl
terraform {
  required_version = "≥ 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
provider "aws" { region = "us-east-1" }

resource "aws_ssm_parameter" "demo" {
  name  = "/shop-api/dev/demo"
  type  = "String"
  value = "hello"
}
```

```bash
terraform init && terraform apply -auto-approve
ls terraform.tfstate          # local state exists
```

3. **Add the S3 backend and migrate.** Add to the `terraform` block:

```hcl
backend "s3" {
  bucket       = "shop-api-tfstate-123456789012"   # your bucket name from step 1
  key          = "dev/demo/terraform.tfstate"
  region       = "us-east-1"
  encrypt      = true
  use_lockfile = true
}
```

```bash
terraform init -migrate-state      # answer "yes" to copy local state to S3
aws s3 ls "s3://$BUCKET/dev/demo/" # you should see terraform.tfstate
rm -f terraform.tfstate terraform.tfstate.backup
terraform plan                     # "No changes" - the state is now remote
```

4. **Look at versioning and the lock.** Run `terraform apply` with a changed value (`value = "hello2"`), then:

```bash
aws s3api list-object-versions --bucket "$BUCKET" --prefix dev/demo/terraform.tfstate
  --query 'Versions[].VersionId'
```

   Two or more versions exist. During an apply (add `-lock-timeout=0` and watch from a second terminal with `aws s3 ls`), a `terraform.tfstate.tflock` object appears briefly.

5. **Adopt an existing resource with `import`.** Create an unmanaged parameter by hand, then bring it under control:

```bash
aws ssm put-parameter --name /shop-api/dev/manual --type String --value "created by
  hand"
```

Add to code:

```hcl
import {
  to = aws_ssm_parameter.manual
  id = "/shop-api/dev/manual"
}
resource "aws_ssm_parameter" "manual" {
  name  = "/shop-api/dev/manual"
  type  = "String"
  value = "created by hand"
}
```

`terraform plan` should say `1 to import`; `terraform apply` imports it. Remove the `import` block afterwards (it has done its job).

6. **Rename with `moved`.** Rename `aws_ssm_parameter.demo` to `aws_ssm_parameter.greeting` in the code and add:

```hcl
moved {
  from = aws_ssm_parameter.demo
  to   = aws_ssm_parameter.greeting
}
```

`terraform plan` shows the resource has moved with `0 to destroy`. Without the `moved` block it would show a destroy plus a create.

7. **Inspect what is in the state.**

```bash
terraform state list
terraform state show aws_ssm_parameter.greeting | head -20
```

Even if you had used a `SecureString`, the plaintext value would be visible in state: this is why access to the bucket matters.

8. **Clean up.**

```bash
terraform destroy -auto-approve                  # removes the parameters
aws ssm delete-parameter --name /shop-api/dev/manual 2>/dev/null || true
# Keep the bucket for the next topics, or empty and delete it, including all object
  versions, when you are done with the level.
```

Illustrative output for step 3 (yours will be similar):

```
Initializing the backend...
Do you want to copy existing state to the new backend?
  Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
```

7 Real-world scenario

(Illustrative.) Two engineers each had a local `terraform.tfstate` for the same staging stack, because "the state file is in the repo". One applied a change; the other's stale state then proposed to **recreate** the load balancer that already existed, and applying it failed halfway with name collisions, leaving a half-built duplicate. Recovery took an afternoon of `terraform import`. The team moved to an S3 backend with versioning, locking and a CI-only apply role, removed the state files from Git history, and rotated the database password that had been sitting in the committed state. **Lesson:** shared state with locking is not an optimisation; it is the thing that makes a team workflow safe.

8 Common mistakes and best practices

Don't

- Commit state files, or share them via chat, email or a shared drive.
- Hand-edit the JSON, or delete a state file to "fix" a problem (you orphan live infrastructure).
- Use `terraform state rm` or `-target` casually; they create code/state mismatches.
- Put every environment and layer into one giant state: slow plans, huge blast radius, long lock times.
- Give broad developer read access to the state bucket without realising it reveals secrets.

Do

- Use a remote backend from day one on any shared stack: S3 with versioning, encryption, Block Public Access and locking.
- Keep the state bucket in a separate, locked-down account where you can, and restrict writes to the pipeline role.
- Use `moved`, `import` and `removed` blocks so refactors are reviewed in a pull request, not run as ad hoc CLI commands.

- Split state by environment and by layer (network / data / app).
- Back up and test restoring a previous state version before you need it.

9 Tasks

- [ ] **T1 — Bootstrap a secure state bucket** *(≈30 min, beginner; AWS sandbox)*
  - **Goal:** create the bucket with versioning, encryption and Block Public Access.
  - **Steps:**
    1. Run step 1 of the guide.
    2. Verify with `aws s3api get-public-access-block`, `get-bucket-versioning` and `get-bucket-encryption`.
  - **Done when:** all three commands show the protections enabled.
- [ ] **T2 — Migrate local state to S3** *(≈30 min, intermediate)*
  - **Goal:** move a working stack to remote state with no resource changes.
  - **Steps:**
    1. Follow steps 2-3.
    2. Run `terraform plan`.
  - **Done when:** the plan says `No changes`, the state object exists in S3 and the local `terraform.tfstate` is gone.
- [ ] **T3 — Adopt and refactor** *(≈45 min, intermediate)*
  - **Goal:** use `import` and `moved` blocks.
  - **Steps:**
    1. Follow steps 5 and 6.
    2. Try `terraform plan -generate-config-out=generated.tf` for a second hand-made parameter, review the file and merge it.
  - **Done when:** the manual parameter is under Terraform's control, the rename shows 0 destroys, and both changes were done with blocks (no `terraform import` or `state mv` CLI commands).
- [ ] **T4 — Break it: corrupt and recover** *(≈45 min, advanced)*
  - **Goal:** recover state from S3 versioning.
  - **Steps:**
    1. Note the current state object version (`list-object-versions`).

2. Overwrite the state with a bad copy: `echo '{}' | aws s3 cp - s3://$BUCKET/dev/demo/terraform.tfstate`. Run `terraform plan` and observe it wants to create everything again.
3. Restore the previous version with `aws s3api copy-object --copy-source "$BUCKET/dev/demo/terraform.tfstate?versionId=<ID>" ...` (or `get-object --version-id` and `cp`).
4. Run `terraform plan` again.

- **Done when:** the plan returns to `No changes` after the restore.

- [ ] **T5 — Lock down the bucket** *(≈45 min, advanced)*
  - **Goal:** apply least privilege and TLS-only access.
  - **Steps:**
    1. Write a bucket policy that denies all requests with `"aws:SecureTransport": "false"`.
    2. Write an IAM policy for a `terraform-ci` role: `s3:ListBucket` on the bucket, `s3:GetObject` / `PutObject` / `DeleteObject` on `dev/*` (the lock file needs delete).
    3. Validate it with `aws iam simulate-principal-policy` or by assuming the role.
  - **Done when:** the role can run `terraform plan` for `dev/` but receives `AccessDenied` when reading `prod/`.
- [ ] **T6 — Prove it: explain the state** *(≈30 min, advanced)*
  - **Goal:** explain the state file to a teammate and defend your backend design.
  - **Steps:**
    1. Without notes, explain what state stores, why it holds secrets and what happens if two people apply with different states.
    2. List the controls on the state bucket and which threat each addresses.
  - **Done when:** you can answer "can I delete the state file?" and "who may read the bucket?" with specifics.

10 Self-check questions

1. What problem does state solve that the `.tf` files alone cannot?
2. Why are secrets visible in state even if a variable is `sensitive`?
3. Which controls make an S3 state bucket safe, and why is versioning important?
4. What is the difference between `moved`, `import` and `removed` blocks?

5. How do you configure a backend differently for dev and prod, given that backend blocks cannot use variables?

ANSWERS

1. It maps each resource address to the real object ID and its attributes, so Terraform knows what it owns and what changed.
2. `sensitive` only masks CLI output. The state is a plain JSON record of all attribute values.
3. Block Public Access, encryption, least-privilege IAM and TLS-only policy, separate account, audit logging. Versioning lets you restore a previous state after corruption or a bad apply.
4. `moved` changes a resource's address without touching the real object. `import` brings an existing real object under management. `removed` stops managing a resource, optionally leaving the real object in place.
5. Use partial configuration: `terraform init -backend-config=backend-dev.hcl` (or per-environment directories with different backend blocks).

11 Further reading

- Terraform documentation: *State*, *Backends* → *s3*, and the *Import* and *Moved* sections of the language reference.
- AWS documentation: *S3 versioning* and *S3 Block Public Access*.
- *Terraform: Up & Running* by Yevgeniy Brikman — "How to Manage Terraform State".
- See also: S3 §6.8 and State Locking §7.8.

LAB NOTES What I ran, what I saw, what it means

7.5 Data Sources and Dependencies

Topic 5 of 9 | ~5 hours | Difficulty: Intermediate

1 What it is

A **data source** (a `data` block) lets Terraform *read* something that already exists and use its attributes in your code, without managing it: "the latest Amazon Linux AMI", "the list of availability zones in this region", "the VPC somebody else created", "the AWS account ID I am running in". A `resource` block says "make this exist"; a `data` block says "look this up".

A **dependency** is a statement that one object must exist before another. Terraform builds a **dependency graph** (a DAG, a directed acyclic graph) of every resource, data source, module and output, then walks it: independent nodes are created in parallel (10 at a time by default), dependent nodes wait. Most dependencies are **implicit** (you reference another object's attribute, so Terraform infers the order). When the relationship is real but invisible in the code you add an **explicit** `depends_on`. On top of this, `count` and `for_each` create several copies of a block, and the `lifecycle` block tunes how Terraform replaces, protects and validates resources.

2 Why it matters in DevOps

- **No hard-coded IDs.** An AMI ID, subnet ID or account ID copied from the console breaks in the next region or account. Data sources keep the same code working in dev, stage and prod.
- **Ordering bugs are outage bugs.** A security group deleted before the instance that uses it, or an IAM policy attached after the app already started, produce flaky applies that "work on the second run".
- `count` **versus** `for_each` **decides whether a harmless edit destroys a database.** Removing the first item of a `count` list shifts every index after it and Terraform plans to replace them.
- `lifecycle` **is your safety net.** `prevent_destroy` on the production database, `create_before_destroy` for zero-downtime replacement and `ignore_changes` for fields an autoscaler owns are everyday senior-level tools.

- **Cross-stack reads** (the app stack reading the network stack) are how big estates stay split into small, fast states.

3 Real-world system example

The `terraform-aws-modules` family and the AWS provider's own docs. The widely used community modules for VPC, EC2, ALB and RDS publish examples that look up the AMI through a data source (or the public SSM parameter that AWS maintains for the latest Amazon Linux image) instead of pinning an ID, call `aws_availability_zones` to spread subnets across zones, and build IAM policies with `aws_iam_policy_document` rather than hand-written JSON. Terraform itself documents the dependency graph as its core execution model: a plan is a walk over the graph, and `terraform graph` prints it in DOT format. Multi-account setups commonly look up the current account with `aws_caller_identity` (and the organisation with `aws_organizations_organization`) so one codebase can be applied to many accounts.

4 Diagram

How Terraform turns your code into a graph and executes it in waves:

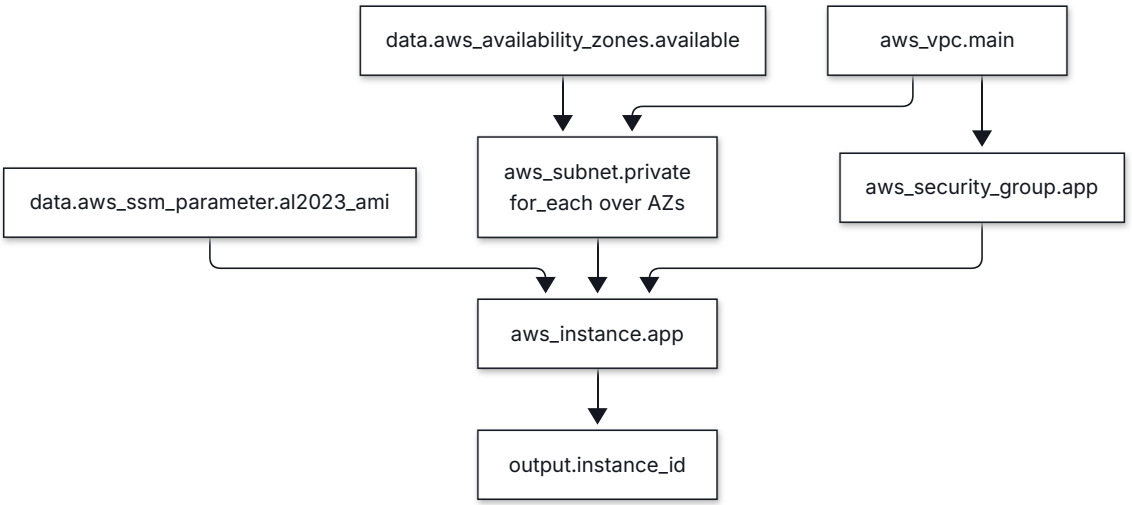


Figure 7.5-1

`aws_vpc.main`, the AZ lookup and the AMI lookup have no parents, so they run first and in parallel. The subnets and the security group run next. The instance waits for all three of its inputs.

When a replacement must not cause downtime, `create_before_destroy` reverses the usual order:

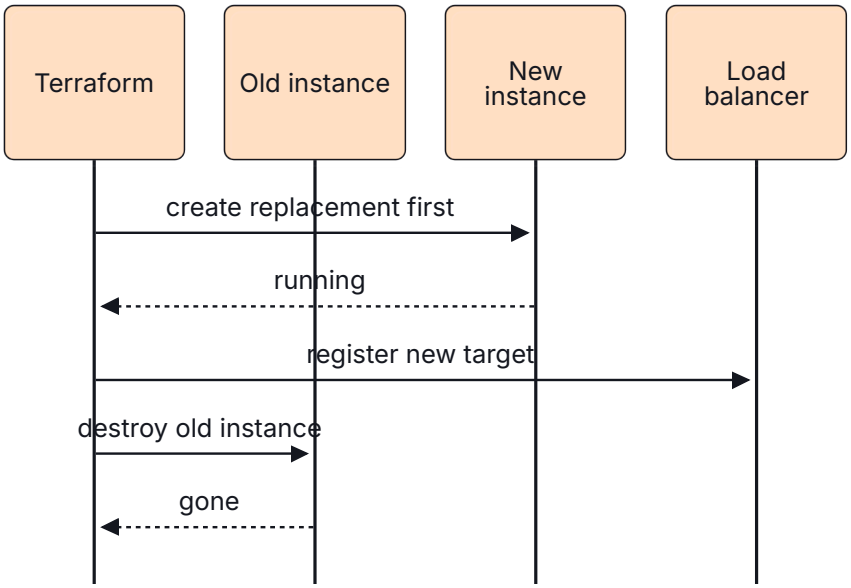


Figure 7.5-2

5 How it works

Data sources

```hcl
data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

data "aws_availability_zones" "available" {
  state = "available"
}

# Latest Amazon Linux 2023 AMI via the public SSM parameter AWS maintains
data "aws_ssm_parameter" "al2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

# Look up an existing VPC by tag (created by another team or another stack)
data "aws_vpc" "shared" {
  tags = { Name = "shared-network" }
}

# Build IAM JSON in HCL instead of a string
```

```
data "aws_iam_policy_document" "read_params" {
  statement {
    actions   = ["ssm:GetParameter", "ssm:GetParametersByPath"]
    resources =
  ["arn:aws:ssm:${data.aws_region.current.name}:${data.aws_caller_identity.current.accoun
  t_id}:parameter/shop-api/*"]
  }
}
```

Reference them with the `data.` prefix: `data.aws_ssm_parameter.al2023_ami.value`. Rules worth knowing:

- Data sources are read during `plan` when their arguments are known; if an argument depends on something not yet created, the read is deferred to apply and the result shows as `(known after apply)`.
- A data source that matches nothing (or more than one object where one is expected) is an error. That is usually what you want: fail early.
- `aws_ami` with `most_recent = true` and a filter also works, but a moving "latest" means a plan next month may want to replace your instance. Decide deliberately: pin the AMI ID per environment for production, use "latest" for dev, and pair it with `lifecycle { ignore_changes = [ami] }` if you roll instances by another mechanism.
- **Cross-stack reads:** `terraform_remote_state` reads another stack's *outputs* from its state file. It needs read access to that whole state (which can contain secrets). A more decoupled alternative is to publish values to SSM Parameter Store or tag resources, and read them with ordinary data sources.

HCL

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "shop-api-tfstate-123456789012"
    key    = "dev/network/terraform.tfstate"
    region = "us-east-1"
  }
}
# data.terraform_remote_state.network.outputs.private_subnet_ids
```

Implicit versus explicit dependencies

Kind	How	When
Implicit	Reference an attribute: `subnet_id = aws_subnet.private["a"].id`	Almost always. Terraform orders for you.

Kind	How	When
Explicit	`depends_on = [aws_iam_role_policy.app]`	The dependency is real but not expressed by any attribute, e.g. an instance's boot script calls S3 and needs the policy attached first.

Prefer implicit references. `depends_on` has a cost: Terraform treats everything in the target as possibly changed, which can make data sources inside the dependent object defer to apply time and produce noisier plans. Use it sparingly and add a comment saying *why*. It works on modules too: `depends_on = [module.network]`.

`count` versus `for_each`

```hcl
# count: copies identified by index 0,1,2 - fragile
resource "aws_subnet" "bad" {
  count             = length(var.azs)
  availability_zone = var.azs[count.index]
  # ...
}

# for_each: copies identified by key - stable
resource "aws_subnet" "private" {
  for_each          = var.private_subnets          # map: az ⇒ cidr
  vpc_id            = aws_vpc.main.id
  availability_zone = each.key
  cidr_block        = each.value
}
```

	`count`	`for_each`
Identity	numeric index `[0]`	key `["us-east-1a"]`
Remove the first item	every later index shifts, so replacements	only that item is destroyed
Input	a number	a map or a set of strings (keys must be known at plan time)
Best for	on/off switch: `count = var.enabled ? 1 : 0`	any collection of similar things

Convert a list to a set with `toset(var.names)`. For lists of objects build a map with a `for` expression: `{ for s in var.services : s.name ⇒ s }`. Reference all copies with a splat or `values()`: `[for s in aws_subnet.private : s.id]`.

The `lifecycle` block

```hcl
resource "aws_db_instance" "main" {
  # ...
  lifecycle {
    prevent_destroy = true            # any plan that destroys this fails
  }
}

resource "aws_launch_template" "app" {
  lifecycle {
    create_before_destroy = true      # build the replacement first
  }
}

resource "aws_autoscaling_group" "app" {
  lifecycle {
    ignore_changes = [desired_capacity]   # autoscaling policies own this field
  }
}

resource "aws_instance" "app" {
  lifecycle {
    replace_triggered_by = [aws_launch_template.app.latest_version]
    precondition {
      condition     = data.aws_vpc.shared.enable_dns_hostnames
      error_message = "The shared VPC must have DNS hostnames enabled."
    }
    postcondition {
      condition     = self.public_ip == "" || self.public_ip == null
      error_message = "App instances must not have public IPs."
    }
  }
}
```

Setting	Effect
`create_before_destroy`	Replacement is created, then the old one destroyed. Names must not collide (use `name_prefix`).
`prevent_destroy`	Plan errors if it would destroy the resource. Not a substitute for IAM or backups; `terraform destroy` fails too, and removing the block from code removes the protection.
`ignore_changes`	Terraform ignores differences in the listed attributes (tags added by other tools, autoscaled capacity).
`replace_triggered_by`	Replace this resource whenever the referenced resource or attribute changes.
`precondition` / `postcondition`	Custom checks before/after the resource is evaluated. Fail the run with a clear message.

6 Step-by-step guide

Prerequisites: Terraform 1.11+ (or OpenTofu), AWS CLI with sandbox credentials, a billing alarm. Everything below uses free or near-free objects (data sources, security groups, SSM parameters). No EC2 instances are created.

1. **Create the project.**

```bash
mkdir -p tf-deps && cd tf-deps
```

`versions.tf`:

```hcl
terraform {
  required_version = "≥ 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
provider "aws" { region = "us-east-1" }
```

2. **Read with data sources.** `main.tf`:

```hcl
data "aws_caller_identity" "current" {}
data "aws_availability_zones" "available" { state = "available" }
data "aws_vpc" "default" { default = true }

data "aws_ssm_parameter" "al2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

output "account_id" {
  value = data.aws_caller_identity.current.account_id
}

output "azs" {
  value = data.aws_availability_zones.available.names
}

output "ami_id" {
  value     = data.aws_ssm_parameter.al2023_ami.value
  sensitive = true
}
```

```bash
terraform init && terraform apply -auto-approve
terraform output azs
terraform output -raw ami_id
```

You should see your 12-digit account ID, a list such as `us-east-1a`, `us-east-1b`, and an `ami-...` ID. (`sensitive = true` is only to hide the long value in the apply summary; it is not a secret.)

3. **Fan out with `for_each` and watch the graph.** Append:

```hcl
locals {
  services = { api = 3000, worker = 3001, metrics = 9100 }
}

resource "aws_security_group" "svc" {
  for_each    = local.services
  name_prefix = "shop-api-${each.key}-"
  description = "shop-api ${each.key}"
  vpc_id      = data.aws_vpc.default.id
  lifecycle { create_before_destroy = true }
}

resource "aws_vpc_security_group_ingress_rule" "svc" {
  for_each          = local.services
  security_group_id = aws_security_group.svc[each.key].id    # implicit dependency
  cidr_ipv4         = "10.0.0.0/16"
  ip_protocol       = "tcp"
  from_port         = each.value
  to_port           = each.value
}
```

```bash
terraform apply -auto-approve
terraform state list | grep svc
terraform graph | head -30        # DOT text; paste into a Graphviz viewer if you like
```

The state shows `aws_security_group.svc["api"]`, `["worker"]`, `["metrics"]`, addressed by key.

4. **See why `for_each` is safer than `count`.** Delete the `api` entry from `local.services` and run `terraform plan`. Only the `api` group and its rule are planned for destruction; `worker` and `metrics` are untouched. Put `api` back.
5. **Add an explicit `depends_on`.** Create a parameter that must be written only after the rules

exist:

```hcl
resource "aws_ssm_parameter" "ready" {
  name  = "/shop-api/dev/network-ready"
  type  = "String"
  value = "true"
  depends_on = [aws_vpc_security_group_ingress_rule.svc]   # real ordering, no
  attribute links them
}
```

Run `terraform apply` and read the output: the parameter is created only after the ingress rules finish. Always add a comment explaining why a `depends_on` exists.

6. **Protect something with `prevent_destroy`.** Add `lifecycle { prevent_destroy = true }` to `aws_ssm_parameter.ready`, run `terraform apply`, then try `terraform destroy`. Expected failure (wording may differ slightly by version):

```
Error: Instance cannot be destroyed
Resource aws_ssm_parameter.ready has lifecycle.prevent_destroy set, but the plan calls
  for it to be destroyed.
```

Remove the `lifecycle` block (a deliberate code change reviewed in a PR is the point) and continue.

7. **Add a `postcondition`.** On `data "aws_vpc" "default"` add:

```hcl
lifecycle {
  postcondition {
    condition     = self.enable_dns_support
    error_message = "VPC must have DNS support enabled."
  }
}
```

Re-run `terraform plan`; it passes. Change the condition to `!self.enable_dns_support` and the plan fails with your message.

8. **Clean up.**

```bash
terraform destroy -auto-approve
```

All resources here are free, but destroy anyway to leave the account tidy.

7 Real-world scenario

(Illustrative.) A team defined five private subnets with `count` over a list of CIDRs, and the RDS subnet group and several instances referenced `aws_subnet.private[2]`. During a re-addressing project someone removed the *first* CIDR from the list. The plan showed four subnets being replaced and the database subnet group being modified. The reviewer almost approved because the diff "was just a removed line". Replacing subnets with live ENIs would have failed midway, and in the worst case dropped connectivity to the database. The fix was to convert to `for_each` keyed by availability zone, using `moved` blocks (`aws_subnet.private[0]` to `aws_subnet.private["us-east-1a"]`) so the refactor itself was a no-op plan, and to add `prevent_destroy` to the database. **Lesson:** a plan that changes more than the diff suggests is a signal to stop, and key-based addressing makes plans match human intent.

8 Common mistakes and best practices

Don't

- Paste AMI IDs, subnet IDs or account IDs into code; they differ per region and account.
- Use `count` over a list that can have items removed from the front or middle.
- Sprinkle `depends_on` everywhere to "make it work". It hides a missing reference and widens plans.
- Treat `prevent_destroy` as a backup; it only blocks Terraform and disappears when the line is deleted.
- Read another team's whole state with `terraform_remote_state` when two outputs would do (it exposes everything in that state).

Do

- Prefer attribute references (implicit dependencies) and use `depends_on` only with a comment explaining the hidden relationship.
- Use `for_each` with stable keys; use `count` only as a 0/1 switch.
- Protect stateful resources with `prevent_destroy`, deletion protection in the provider (`deletion_protection = true` on RDS) *and* backups.
- Use `create_before_destroy` plus `name_prefix` on resources that serve traffic and must be replaced without a gap.
- Use `precondition` / `postcondition` to turn silent assumptions ("this VPC has DNS enabled") into loud failures.
- Explore the graph with `terraform graph` when a plan's ordering surprises you.

9 Tasks

- [ ] **T1 — Replace hard-coded IDs with data sources** *(≈30 min, beginner)*
  - **Goal:** make a config region-independent.
  - **Steps:**
    1. Follow steps 1-2.
    2. Add an output that builds a string like `"<account>/<region>/<first AZ>"` from three data sources.
  - **Done when:** `terraform output` prints your real account ID, region and first AZ without any hard-coded value in the code.

- [ ] **T2 — Fan out with `for_each`** *(≈30 min, intermediate)*
  - **Goal:** create several similar resources with stable addresses.
  - **Steps:**
    1. Follow step 3.
    2. Add a fourth service `admin = 8080` to the map and apply.
  - **Done when:** `terraform state list` shows four security groups addressed by key and the plan only added the new one.

- [ ] **T3 — Compare `count` and `for_each`** *(≈30 min, intermediate)*
  - **Goal:** observe index shifting.
  - **Steps:**
    1. Create three `aws_ssm_parameter` resources with `count` over `["a","b","c"]`; apply.
    2. Remove `"a"` from the list and run `terraform plan`. Note what it wants to replace.
    3. Redo the experiment with `for_each = toset([...])`.
  - **Done when:** you can show that `count` plans changes to `b` and `c` while `for_each` plans only to destroy `a`.

- [ ] **T4 — Break it: an invisible dependency** *(≈45 min, advanced)*
  - **Goal:** hit an ordering failure and fix it with the right tool.
  - **Steps:**
    1. Using two `terraform_data` resources with `local-exec` provisioners, make the first write a parameter (`aws ssm put-parameter`) and the second read it (`aws ssm get-parameter`), with no attribute reference between them.
    2. Apply from scratch and observe the race or failure.
    3. Fix it, first by adding a real reference, then explain when `depends_on` would be the correct fix instead.

- **Done when:** a clean `terraform apply` from an empty state succeeds twice in a row, and you can state in one sentence why you chose reference versus `depends_on`.

☐ **T5 — Lifecycle guards** *(≈45 min, advanced)*

- **Goal:** use `prevent_destroy`, `ignore_changes` and a `postcondition`.
- **Steps:**
  1. Do steps 6-7.
  2. Add `ignore_changes = [value]` to a parameter, edit its value in the console, and show that `terraform plan` reports no changes.
  3. Remove `ignore_changes` and show that the plan now wants to revert the value.
- **Done when:** you have demonstrated all three guards (blocked destroy, a failing condition, ignored drift) with command output.

☐ **T6 — Prove it: read the graph** *(≈30 min, advanced)*

- **Goal:** predict Terraform's order without running it.
- **Steps:**
  1. Draw (on paper or as Mermaid) the dependency graph of your project from step 3 and 5.
  2. Predict which resources are created in parallel.
  3. Verify with `terraform graph` and the timestamps of `terraform apply` output.
- **Done when:** your predicted order matches the apply output, and you can explain in two minutes where implicit and explicit dependencies came from.

10 Self-check questions

1. What is the difference between a `resource` and a `data` block?
2. Why is `for_each` usually safer than `count`?
3. When is `depends_on` the right tool, and what is its downside?
4. What does `create_before_destroy` do and what problem can it cause?
5. What are two ways to share values between separate Terraform stacks, and which is more decoupled?

ANSWERS

1. A resource is created and managed by Terraform. A data source only reads an existing object (or computes a value) and never changes it.
2. `for_each` identifies instances by key, so removing or reordering items affects only those items. `count` uses numeric indexes, so removing an early item shifts and replaces the rest.

3. When a real ordering requirement exists that no attribute reference expresses (e.g. an IAM policy must be attached before a boot script runs). Downside: it widens what Terraform considers changed, defers data reads to apply time and can hide a missing reference.
4. It creates the replacement before destroying the old object, avoiding downtime. It can cause name collisions (unique names), temporary double cost and quota pressure, so use `name_prefix`.
5. `terraform_remote_state` (reads another stack's outputs but requires access to the entire state) or publishing values to SSM Parameter Store, tags or a service catalogue and reading them with data sources. The latter is more decoupled.

11 Further reading

- Terraform documentation: *Data Sources*, *Resource Behavior → Dependencies*, *Meta-Arguments* (`count`, `for_each`, `depends_on`, `lifecycle`) and the `terraform graph` command.
- AWS provider documentation: `aws_availability_zones`, `aws_ssm_parameter`, `aws_caller_identity`, `aws_iam_policy_document`.
- *Terraform: Up & Running* by Yevgeniy Brikman — chapters on loops and conditionals.
- See also: State and Remote Backends §7.4, Modules §7.3 and VPC and Subnets §6.4.

LAB NOTES What I ran, what I saw, what it means

7.6 Reusable Environments

Topic 6 of 9 | ~5 hours | Difficulty: Intermediate

1 What it is

Almost every product runs in several **environments**: `dev` (engineers experiment), `stage` (a production lookalike for final tests) and `prod` (real users). A **reusable environment** setup means you write the infrastructure *once* and instantiate it several times with different inputs (sizes, counts, CIDR ranges, domain names), instead of copy-pasting a folder and watching the copies slowly diverge.

Terraform gives you three building blocks for this: **modules** (the reusable code, L7.3), **separate state per environment** (so a dev mistake cannot touch prod, L7.4), and **input variables** (the differences, L7.2). The design question is how to organise the folders and state around them: one directory per environment, or **workspaces** (several states from one directory), plus helper tools such as **Terragrunt** that remove repeated boilerplate.

2 Why it matters in DevOps

- **"Works in stage, breaks in prod"** is usually environment drift in code: someone fixed stage by hand-editing a copy. One module used by all environments keeps them structurally identical; only the inputs differ.
- **Blast radius.** A typo in dev must never be able to plan a change to the production database. Separate state, separate credentials and often separate AWS accounts make that impossible rather than merely unlikely.
- **Cost.** Dev can run a single `t3.micro` and a small database, prod runs Multi-AZ. The same code with different variables does both without a rewrite.
- **Speed of change.** Promoting a change `dev` then `stage` then `prod` is a review of one diff and a sequence of applies, not three hand-made edits.

3 Real-world system example

Per-environment stacks in AWS-centric organisations, and Terragrunt. AWS's own prescriptive guidance for multi-account setups (the "AWS Well-Architected" multi-account strategy and Control Tower landing zones) recommends separating workloads into different accounts, with production isolated from non-production. Terraform teams mirror that with one root module per environment and one state per environment per layer. Gruntwork, the company behind **Terragrunt**, publishes a reference layout (`infrastructure-live` and `infrastructure-modules` repositories) that separates reusable modules from the "live" per-environment configuration, and Terragrunt itself exists to keep backend and provider blocks DRY across many such directories. Terraform's own documentation notes that workspaces share one backend and configuration and are *not* a strong isolation boundary for production-versus-development credentials, which is why many teams prefer directories (or accounts) for real environments.

4 Diagram

The recommended shape: modules are written once, environments are thin callers with their own state.

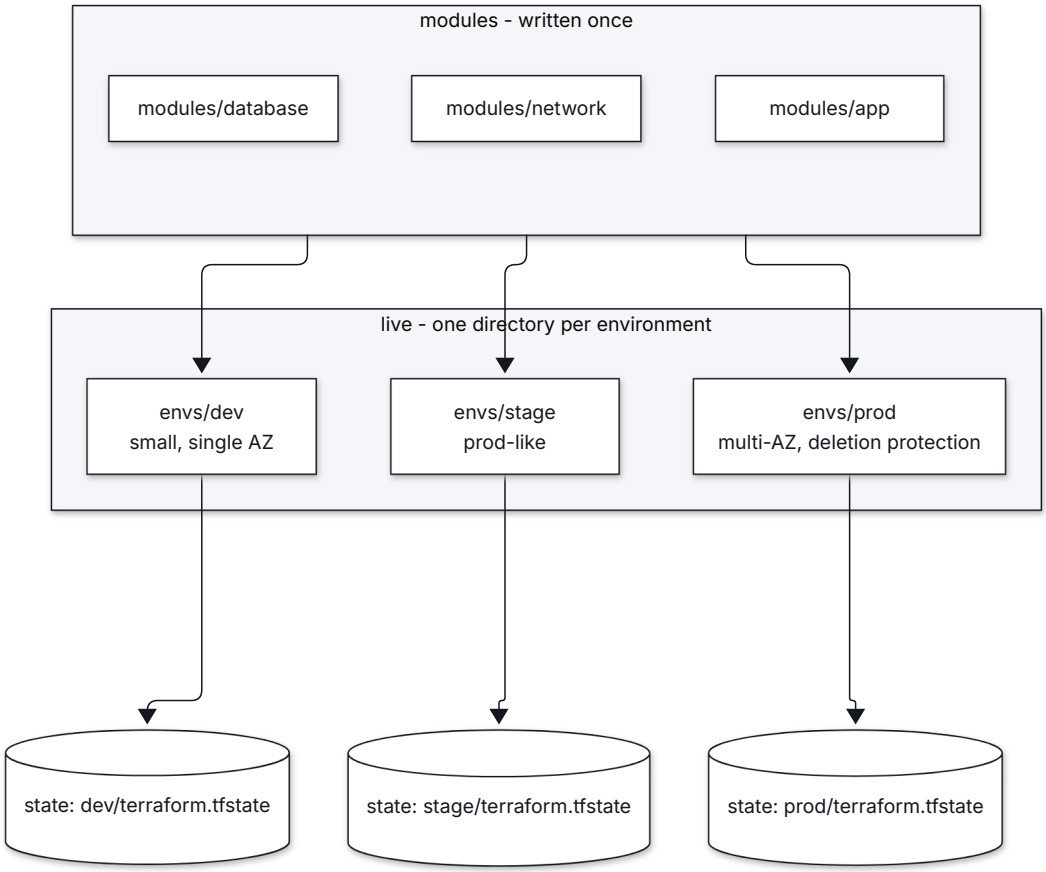


Figure 7.6-1

How a change is promoted:

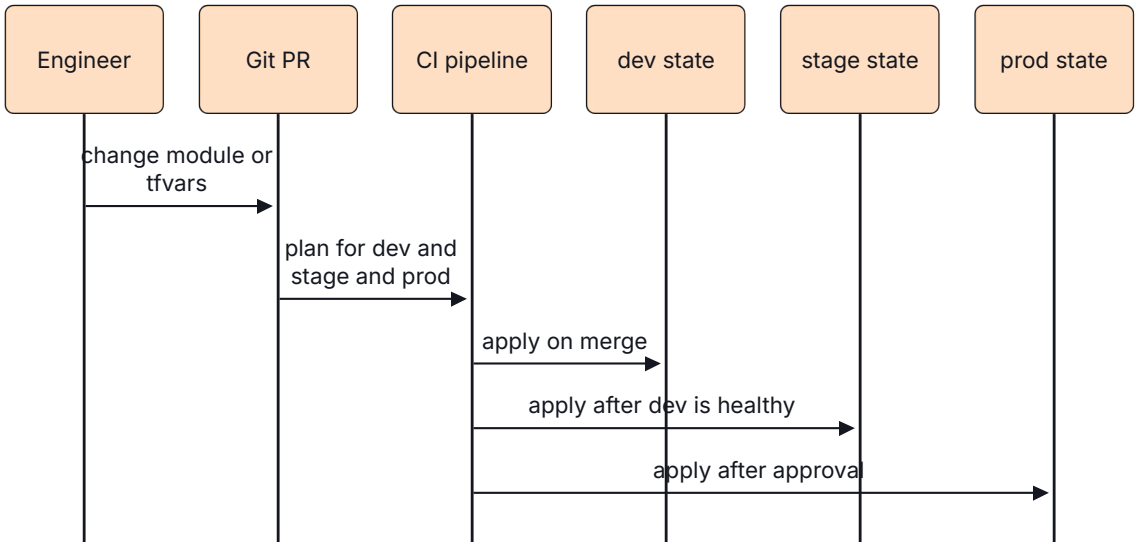


Figure 7.6-2

5 How it works

Option A: a directory per environment (recommended for real environments)

```
infra/
  modules/
    network/   app/   database/
  envs/
    dev/
      backend.tf        # key = "dev/terraform.tfstate"
      main.tf           # module calls
      variables.tf
      dev.auto.tfvars   # the values that make dev different
    stage/
    prod/
```

Each `envs/<name>` is a **root module** with its own backend, its own `terraform init` and its own state. Differences are explicit and reviewable: `diff -r envs/dev envs/prod` shows exactly how they differ. The cost is repetition of small wiring files.

Option B: workspaces (one directory, many states)

```bash
terraform workspace new dev
terraform workspace new prod
terraform workspace select prod
terraform workspace show
```

```hcl
locals {
  settings = {
    dev  = { instance_type = "t3.micro", multi_az = false }
    prod = { instance_type = "t3.large", multi_az = true  }
  }
  env = local.settings[terraform.workspace]
}
```

With the S3 backend each workspace gets its own state object under `env:/<workspace>/<key>`. Workspaces are lightweight, but: they share the same backend, the same provider credentials and the same code. It is easy to run `apply` while the wrong workspace is selected (the prompt does not show it), and a per-environment difference that does not fit a lookup map is awkward. They suit **ephemeral copies** of one stack (a per-PR preview environment, a feature test) better than long-lived prod versus dev.

> Do not confuse Terraform CLI workspaces with HCP Terraform "workspaces", which are separate units with their own variables, state and permissions and behave more like Option A.

	Directories	Workspaces
State isolation	separate backend keys, can be separate buckets/accounts	same backend, separate state objects
Credentials per env	easy: different role per directory/pipeline	hard: one provider config for all
Differences between envs	any code difference possible, visible in review	only what the lookup map or variables express
Risk of applying to wrong env	low (you are in a directory, CI is pinned per path)	higher (hidden CLI state)
Duplication	some boilerplate	minimal
Good for	dev / stage / prod	per-PR preview stacks, short-lived clones

DRY patterns

1. **Modules for logic, thin environment roots for wiring.** Roots contain only module calls and values.
2. **Shared `.tfvars` conventions:** `terraform apply -var-file=../common.tfvars -var-file=prod.tfvars` (later files win).
3. **Partial backend configuration:** a single `backend "s3" {}` block plus `terraform init -backend-config=backend-prod.hcl` to avoid repeating bucket names.
4. **Layered stacks:** `network` then `data` then `app` per environment; upper layers read lower ones through outputs (L7.5). Smaller states mean faster plans and smaller blast radius.
5. **Versioned module sources:** `source = "git::https://github.com/YOUR-ORG/tf-modules.git//app?ref=v1.4.0"` so prod can lag dev by a tag, giving you a real promotion path.
6. **Generated boilerplate:** a wrapper tool writes the backend and provider blocks for you (below).

Terragrunt (mention)

Terragrunt is a thin open-source wrapper around Terraform (and OpenTofu) from Gruntwork. A `terragrunt.hcl` per environment directory can `include` a root file that generates the `backend` and `provider` blocks, sets the state key from the directory path, declares `dependency` blocks between stacks and runs one command across many stacks (`terragrunt run-all plan`, spelled `terragrunt run --all plan` in newer releases). It cuts boilerplate in large estates. It is an extra tool to learn and to keep versions aligned; for a handful of environments plain directories plus modules are enough. Newer Terraform features (`-var-file`, `.auto.tfvars`, `-backend-config`, stacks in HCP Terraform) cover part of the same ground, so adopt Terragrunt for a concrete pain, not by default.

```hcl
# envs/prod/app/terragrunt.hcl  (illustrative)
include "root" {
  path = find_in_parent_folders()        # root.hcl generates backend + provider
}
terraform {
  source = "git::https://github.com/YOUR-ORG/tf-modules.git//app?ref=v1.4.0"
}
inputs = {
  instance_type = "t3.large"
  multi_az      = true
}
```

Environment account strategy

Directories pair naturally with accounts: `dev`, `stage` and `prod` are separate AWS accounts, each with its own role that CI assumes (L7.7). Even the state bucket can be per account. A developer's credentials for dev then cannot touch prod at all. For a small project one account with a naming prefix (`shop-api-dev-...`) is acceptable but weaker.

6 Step-by-step guide

Prerequisites: Terraform 1.11+ (or OpenTofu). The example uses only the `local` and `random` providers, so it costs nothing and needs no cloud account. The pattern is identical for AWS modules.

1. **Create the layout.**

```bash
mkdir -p envs-demo/modules/app envs-demo/envs/dev envs-demo/envs/prod && cd envs-demo
```

2. **Write the reusable module** `modules/app/main.tf`:

```hcl
terraform {
  required_providers {
    local = { source = "hashicorp/local", version = "~> 2.5" }
  }
}

variable "env"           { type = string }
variable "instance_type" { type = string }
variable "replicas" {
  type    = number
  default = 1
}
variable "multi_az" {
  type    = bool
  default = false
}

resource "local_file" "config" {
  filename = "${path.root}/rendered-${var.env}.txt"
  content  = <<-EOT
    env           = ${var.env}
    instance_type = ${var.instance_type}
    replicas      = ${var.replicas}
    multi_az      = ${var.multi_az}
  EOT
}
```

```
output "file" { value = local_file.config.filename }
```

3. **Create the dev root** `envs/dev/main.tf`:

```hcl
terraform {
  required_version = "≥ 1.11"
  # backend "s3" { ... key = "dev/terraform.tfstate" } would go here on AWS; local
   state keeps this demo free
}

module "app" {
  source        = "../../modules/app"
  env           = "dev"
  instance_type = "t3.micro"
}

output "file" { value = module.app.file }
```

4. **Create the prod root** `envs/prod/main.tf` with the same module and different inputs:

```hcl
terraform {
  required_version = "≥ 1.11"
}

module "app" {
  source        = "../../modules/app"
  env           = "prod"
  instance_type = "t3.large"
  replicas      = 3
  multi_az      = true
}

output "file" { value = module.app.file }
```

5. **Apply each environment independently.**

```bash
(cd envs/dev  && terraform init -input=false && terraform apply -auto-approve)
(cd envs/prod && terraform init -input=false && terraform apply -auto-approve)
cat envs/dev/rendered-dev.txt envs/prod/rendered-prod.txt
ls envs/dev/terraform.tfstate envs/prod/terraform.tfstate     # two independent states
```

Output is similar to `instance_type = t3.micro` for dev and `replicas = 3` for prod.

6. **Prove the isolation.** Destroy dev only and confirm prod is unaffected:

```bash
(cd envs/dev && terraform destroy -auto-approve)
ls envs/prod/rendered-prod.txt
```

7. **Promote a module change.** Edit `modules/app/main.tf` (add a line `region = us-east-1` to the content), then run `terraform plan` in both directories. Both show one file to update: one module change, every environment reviewed.
8. **Try the workspace alternative** in a copy (`cp -r envs/dev ws-demo && cd ws-demo`): replace the hard-coded `env` with `terraform.workspace`, then:

```bash
terraform workspace new qa
terraform apply -auto-approve
terraform workspace list
ls terraform.tfstate.d/          # local backend keeps one directory per workspace
```

   Note how nothing in the shell prompt tells you which workspace you are in. That is the risk.
9. **Clean up.**

```bash
(cd envs/prod && terraform destroy -auto-approve)
cd .. && rm -rf envs-demo
```

7 Real-world scenario

(Illustrative.) A team kept stage and prod as two copies of one folder. A hotfix to the load-balancer idle timeout went into prod during an incident and was never copied to stage. Three months later a stage load test showed no timeouts and prod fell over under the same traffic shape, because the two had diverged in eleven small ways nobody had listed. The team extracted a `modules/app` module, reduced each environment to ~30 lines of inputs, pinned prod to a module tag one release behind dev and stage, and added a CI job that fails when `envs/stage` and `envs/prod` pass different *structural* arguments to a module. **Lesson:** environments should differ only in values, never in structure, and the diff between them should fit on one screen.

8 Common mistakes and best practices

Don't

- Copy a whole environment folder and edit it; copies drift.
- Use CLI workspaces to separate prod from dev when they need different credentials or approval rules.
- Share one state (or one giant root module) across all environments and layers.
- Put environment-specific `if var.env == "prod"` branches all over a module; prefer explicit variables (`multi_az`, `deletion_protection`).
- Let dev have the ability to read or write the prod state bucket.

Do

- Keep modules generic and environment roots thin; the roots should read like configuration, not code.
- Give each environment its own state key (ideally its own account and bucket) and its own CI role.
- Pin module versions per environment to promote changes dev, stage, prod.
- Keep prod safety on by variable: `deletion_protection`, `prevent_destroy`, `multi_az`, backup retention.
- Use workspaces for short-lived copies (PR previews) and destroy them automatically.
- Adopt Terragrunt (or generation scripts) only when repeated backend/provider boilerplate becomes a real maintenance cost.

9 Tasks

- [ ] **T1 — Build two environments from one module** *(≈45 min, beginner)*
  - **Goal:** instantiate a module twice with different inputs.
  - **Steps:**
    1. Follow steps 1-5.
    2. Run `diff envs/dev/main.tf envs/prod/main.tf`.
  - **Done when:** both rendered files exist with different values and the diff shows only input differences.
- [ ] **T2 — Prove state isolation** *(≈20 min, beginner)*
  - **Goal:** show that environments cannot affect each other.
  - **Steps:**

1. Follow step 6.
2. Run `terraform state list` in each directory.

- **Done when:** destroying dev leaves prod's file and state untouched.

- [ ] **T3 — Add a third environment** *(≈30 min, intermediate)*
  - **Goal:** measure how cheap a new environment is.
  - **Steps:**
    1. Create `envs/stage` with values between dev and prod.
    2. Add a new variable `log_level` to the module with a default and set it only in prod.
  - **Done when:** stage applies cleanly, and dev and stage needed no edits for the new variable.

- [ ] **T4 — Workspaces versus directories** *(≈45 min, intermediate)*
  - **Goal:** feel the trade-off.
  - **Steps:**
    1. Do step 8 and create two workspaces.
    2. Deliberately run `terraform destroy` while on the wrong workspace in a sandbox and note how easily it happens.
    3. Write three sentences on when you would pick each approach.
  - **Done when:** `terraform workspace list` shows your workspaces and your written comparison cites isolation, credentials and visibility.

- [ ] **T5 — Break it: configuration drift between copies** *(≈45 min, advanced)*
  - **Goal:** see why copy-paste fails and fix it.
  - **Steps:**
    1. Copy `envs/prod` to `envs/prod-copy`, change one argument in the copy only and apply both.
    2. Find the difference using only `diff -r`.
    3. Refactor so both call the module with a single shared `common.tfvars` plus a small environment file.
  - **Done when:** `diff -r` between environments shows only values, and a module change appears in the plan of every environment.

- [ ] **T6 — Prove it: design an environment strategy** *(≈45 min, advanced)*
  - **Goal:** defend a layout for `shop-api` (VPC, ALB, EC2, RDS, Redis).
  - **Steps:**

1. Without notes, sketch the repository tree, state keys, AWS accounts and CI roles for dev, stage and prod.
2. Explain what is shared (modules) and what differs (values), and how a module change is promoted.
3. State one situation where you would add Terragrunt and one where you would not.

- **Done when:** a teammate can read your sketch and answer "what stops a dev engineer from changing prod?" with a specific control.

10 Self-check questions

1. Why prefer directories over CLI workspaces for dev, stage and prod?
2. What should differ between environments and what must never differ?
3. How does a module version pin help you promote a change?
4. What problem does Terragrunt solve, and what does it cost you?
5. Where does each environment's state live and how is it protected?

ANSWERS

1. Directories give separate backends, credentials and review paths, and the target environment is explicit in the path. Workspaces share one backend, one provider configuration and one code path, and the active workspace is hidden CLI state.
2. Values (sizes, counts, CIDRs, names, feature flags like `multi_az`) may differ. Structure (which resources exist and how they are wired) should not, otherwise stage stops predicting prod.
3. Prod can reference an older tag while dev and stage move to a newer one. After the new version proves itself you bump the prod pin in a reviewed PR.
4. It removes repeated backend/provider boilerplate, wires dependencies between stacks and runs many stacks at once. The cost is another tool, another configuration language layer and version alignment.
5. In its own state object (ideally its own bucket and account), with versioning, encryption and IAM so only that environment's CI role can write it.

11 Further reading

- Terraform documentation: *Workspaces*, *Backend configuration (partial configuration)* and *Input variables → variable definition files*.
- Terragrunt documentation (terragrunt.gruntwork.io): *Keep your Terraform code DRY* and the `run-all` command.

- AWS whitepaper: *Organizing Your AWS Environment Using Multiple Accounts*.
- *Terraform: Up & Running* by Yevgeniy Brikman — "How to Manage Multiple Environments".
- See also: Modules §7.3, Variables and Outputs §7.2 and Immutable Infrastructure §0.5.

LAB NOTES What I ran, what I saw, what it means

7.7 Terraform CI/CD

Topic 7 of 9 | ~6 hours | Difficulty: Advanced

1 What it is

Terraform CI/CD (continuous integration and continuous delivery) means infrastructure changes travel through the same pipeline as application code: a developer opens a **pull request (PR)**, automation formats, validates, lints, scans and runs `terraform plan`, a human reviews the plan, and only after **merge** does a pipeline run `terraform apply` with the plan that was reviewed. Nobody runs `apply` from a laptop against shared environments.

Two supporting ideas make this safe. First, the pipeline authenticates to AWS with **OIDC** (OpenID Connect), a short-lived, federated identity issued per workflow run, instead of long-lived access keys stored as secrets. Second, **policy checks** (static analysis with `tflint`, `checkov` or `trivy config`, and policy engines such as Open Policy Agent via `conftest`) block dangerous changes (a public S3 bucket, an open security group) before they reach the cloud.

2 Why it matters in DevOps

- **Review the plan, not just the code.** The same diff can produce a harmless or a destructive plan depending on current state. Posting the plan on the PR makes the real effect reviewable.
- **Credentials.** A laptop with admin keys that can change prod is a breach waiting to happen. A pipeline role that only a protected branch can assume is auditable and revocable.
- **Consistency.** CI uses a pinned Terraform version and provider lock file, so "works on my machine" differences disappear.
- **Auditability.** Who changed the firewall, when, why and who approved it is answered by Git history plus run logs.
- **Compliance.** Policy-as-code runs the same rules on every change instead of relying on a reviewer remembering them (DevSecOps §0.12).

3 Real-world system example

Atlantis, HCP Terraform and GitHub Actions. Three publicly documented patterns dominate. **Atlantis** (an open-source project) is a self-hosted service that listens to PR webhooks, runs `terraform plan` when you open or update a PR, comments the plan on the PR, and applies when a reviewer comments `atlantis apply`. **HCP Terraform** (formerly Terraform Cloud) provides remote runs, state, run tasks and Sentinel / OPA policy checks as a managed service, with VCS-driven speculative plans on PRs. Many teams instead write the pipeline themselves in **GitHub Actions** using `hashicorp/setup-terraform`, and authenticate with GitHub's OIDC provider: AWS documents trusting `token.actions.githubusercontent.com` and scoping the role to a repository and branch via the token's `sub` claim. GitHub's own documentation describes this OIDC flow as the way to avoid storing long-lived cloud secrets.

4 Diagram

Pull request to production:

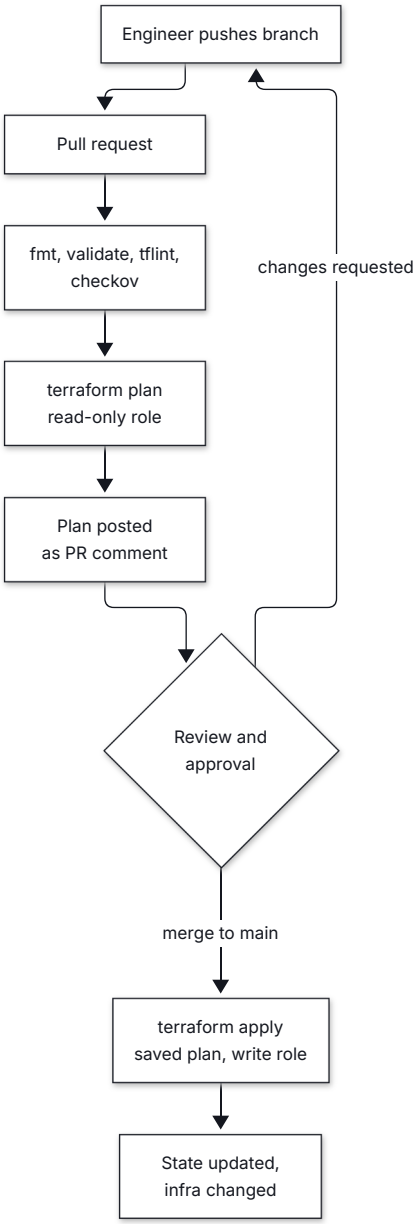


Figure 7.7-1

How the pipeline gets AWS credentials without a stored secret:

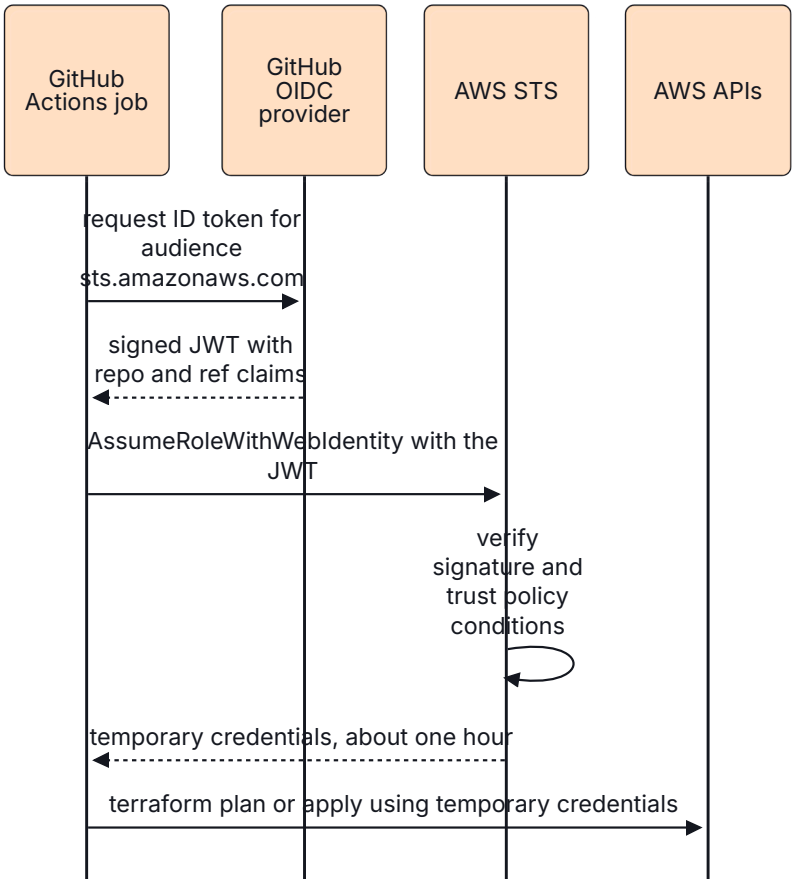


Figure 7.7-2

5 How it works

Pipeline stages

Stage	Command	Purpose	Fails when
Format	`terraform fmt -check -recursive`	consistent style	file is not formatted
Validate	`terraform init -backend=false && terraform validate`	syntax and type errors	invalid configuration
Lint	`tflint --init && tflint`	provider-aware mistakes (invalid instance type, deprecated syntax, unused variables)	rule violations

Stage	Command	Purpose	Fails when
Security scan	`checkov -d .` or `trivy config .`	misconfigurations (public buckets, unencrypted volumes, `0.0.0.0/0` on SSH)	high-severity finding
Policy	`conftest test tfplan.json`	custom org rules on the *plan* (required tags, allowed regions)	rego `deny`
Plan	`terraform plan -out=tfplan`	what will change	any error; PR comment shows the diff
Apply	`terraform apply tfplan`	make the change	only on `main`, after approval

Key rules:

- **Apply the plan you reviewed.** `terraform plan -out=tfplan` then `terraform apply tfplan` applies exactly that plan and errors if state has changed since. Plan files can contain secrets in plain text, so treat the artifact like state: short retention, private.
- **Lock the toolchain.** Pin `required_version`, commit `.terraform.lock.hcl`, and pin the Terraform version in CI.
- **Two roles.** A **plan role** with read-only access (and read/write on the state lock object only) for PR runs. An **apply role** with write permissions that only `main` (or a protected GitHub Environment) can assume. Untrusted forks never get either.
- **Approval gates.** Use GitHub Environments with required reviewers for `prod` (Environments and Secrets §3.12).
- **Concurrency.** One run per stack at a time (`concurrency:` in Actions plus state locking, L7.8).
- **One pipeline per stack** (path filters): a change in `envs/dev` should not plan `envs/prod`.

OIDC trust: the AWS side

The role trusts GitHub's OIDC provider and only tokens from your repository and branch (or environment):

```hcl
resource "aws_iam_openid_connect_provider" "github" {
  url            = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
  # AWS verifies GitHub's certificate chain itself; thumbprint_list is no longer required
}

data "aws_iam_policy_document" "trust" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type        = "Federated"
```

```
      identifiers = [aws_iam_openid_connect_provider.github.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:aud"
      values   = ["sts.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:sub"
      values   = ["repo:YOUR-ORG/shop-api-infra:ref:refs/heads/main"]   # apply role: main
  only
    }
  }
}

resource "aws_iam_role" "terraform_apply" {
  name               = "terraform-apply-dev"
  assume_role_policy = data.aws_iam_policy_document.trust.json
  max_session_duration = 3600
}
```

For a plan role on PRs use `values = ["repo:YOUR-ORG/shop-api-infra:pull_request"]` with `StringLike` / `StringEquals` as appropriate. For environments the subject is `repo:ORG/REPO:environment:prod`. **Never use a wildcard** `sub` such as `repo:*`: any repository on GitHub could then assume your role.

The workflow

YAML

```yaml
# .github/workflows/terraform.yml
name: terraform
on:
  pull_request:
    paths: ["infra/envs/dev/**", "infra/modules/**"]
  push:
    branches: [main]
    paths: ["infra/envs/dev/**", "infra/modules/**"]

permissions:
  id-token: write        # request the OIDC token
  contents: read
  pull-requests: write   # post the plan comment

concurrency:
  group: tf-dev
  cancel-in-progress: false   # never cancel a running apply

defaults:
  run:
```

```
    working-directory: infra/envs/dev

jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.11.4" }     # pin; use a version you have tested
      - run: terraform fmt -check -recursive
      - run: terraform init -backend=false && terraform validate
      - uses: terraform-linters/setup-tflint@v4
      - run: tflint --init && tflint --recursive
      - uses: bridgecrewio/checkov-action@v12
        with: { directory: infra, framework: terraform, soft_fail: false }

  plan:
    needs: checks
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.11.4" }
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/terraform-plan-dev
          aws-region: us-east-1
      - run: terraform init -input=false
      - run: terraform plan -input=false -lock-timeout=5m -out=tfplan
      - run: terraform show -no-color tfplan > plan.txt
      - uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const plan = fs.readFileSync('infra/envs/dev/plan.txt', 'utf8').slice(0,
  60000);
            const fence = '`'.repeat(3);   // a markdown code fence
            github.rest.issues.createComment({
              owner: context.repo.owner, repo: context.repo.repo, issue_number:
  context.issue.number,
              body: '#### Terraform plan\n' + fence + '\n' + plan + '\n' + fence
            });

  apply:
    needs: checks
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: dev          # add required reviewers on the prod environment
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.11.4" }
```

```
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/terraform-apply-dev
          aws-region: us-east-1
      - run: terraform init -input=false
      - run: terraform plan -input=false -lock-timeout=5m -out=tfplan
      - run: terraform apply -input=false tfplan
```

Notes: the apply job plans again on `main` (the merged code), then applies that fresh saved plan. A stricter design uploads the PR plan as an artifact and applies it after merge, failing if state moved; that gives the strongest "applied what was reviewed" guarantee but needs fast merges. Pin third-party actions to a full commit SHA in security-sensitive repositories, because tags can move. (Action versions above are examples; check the current major versions.)

Policy checks

- **tflint** understands the AWS provider (`tflint --init` installs the AWS ruleset from `.tflint.hcl`): it catches invalid instance types or deprecated arguments that `validate` cannot.
- **checkov** and **trivy config** scan the HCL (and plans) against hundreds of built-in rules (CIS-style: encryption on, no public access, logging enabled). Suppress a rule only with an inline comment and a reason, e.g. `#checkov:skip=CKV_AWS_18: access logs live in the central bucket`.
- **OPA / conftest** checks the JSON plan against your own rules:

```bash
terraform show -json tfplan > tfplan.json
conftest test tfplan.json --policy policy/
```

```rego
# policy/tags.rego
package main

deny contains msg if {
  some rc in input.resource_changes
  rc.type == "aws_instance"
  rc.change.actions[_] != "delete"
  not rc.change.after.tags.Owner
  msg := sprintf("%s must have an Owner tag", [rc.address])
}
```

Cost estimation (Infracost) and drift checks (L7.9) are common extra stages.

6 Step-by-step guide

Prerequisites: a GitHub repository you own, Terraform 1.11+, Docker or local installs of `tflint` and `checkov` for local runs, and (for the OIDC part) an AWS sandbox account with billing alarm. The local tool steps cost nothing; the AWS OIDC role and SSM parameter cost effectively nothing.

1. **Create the repo layout.**

```bash
mkdir -p shop-api-infra/infra/envs/dev && cd shop-api-infra
git init -q
```

`infra/envs/dev/main.tf`:

```hcl
terraform {
  required_version = "≥ 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
provider "aws" { region = "us-east-1" }

resource "aws_ssm_parameter" "greeting" {
  name  = "/shop-api/dev/greeting"
  type  = "String"
  value = "hello from CI"
  tags  = { Owner = "platform-team" }
}
```

Add a `backend "s3"` block using the bucket from L7.4. CI needs remote state.

2. **Run the checks locally first.**

```bash
cd infra/envs/dev
terraform fmt -check -recursive
terraform init -backend=false && terraform validate
```

Expected: `Success! The configuration is valid.` If `fmt -check` prints a filename, run `terraform fmt` to fix it.

3. **Add linting and scanning.** Create `infra/envs/dev/.tflint.hcl`:

```hcl
plugin "aws" {
  enabled = true
  version = "0.38.0"     # example; use the current ruleset release
  source  = "github.com/terraform-linters/tflint-ruleset-aws"
}
```

```bash
tflint --init && tflint
pip install checkov && checkov -d . --framework terraform
```

   Checkov prints passed/failed check counts. Treat the output as similar to, not identical to, any example: rules change between releases.

4. **Create the AWS OIDC provider and roles** with the Terraform from section 5 (apply it once from your laptop in a separate `bootstrap/` directory; this is the only manual apply). Attach a read-only policy (`ReadOnlyAccess` plus S3 access to the state key) to the plan role and a narrowly scoped policy to the apply role. Note the role ARNs.
5. **Add the workflow** from section 5 at `.github/workflows/terraform.yml`, replacing the role ARNs, Terraform version and bucket. Commit and push to `main` to apply the initial state.
6. **Open a pull request** that changes the parameter value to `hello from a PR`. In the Actions tab expect `checks` and `plan` to run; the PR gets a comment ending with a summary similar to:

```
Plan: 0 to add, 1 to change, 0 to destroy.
```

7. **Merge and watch the apply.** After merging, the `apply` job runs and `aws ssm get-parameter --name /shop-api/dev/greeting --query Parameter.Value --output text` returns the new value.
8. **Protect `main`.** In repository settings require the `checks` and `plan` jobs to pass and at least one review before merge (Protected Branches and Pull Requests §3.8).
9. **Clean up.** Run `terraform destroy` (from a protected, manually triggered workflow or your sandbox shell), delete the roles and OIDC provider if you no longer need them, and delete the repository if it was only for practice.

7 Real-world scenario

(Illustrative.) An engineer, debugging at 2 a.m., ran `terraform apply` from their laptop against prod with a stale branch checked out. It "fixed" the incident and silently reverted two security-group rules added by a colleague the day before. The next morning a scan showed SSH open to the internet. The team's response: remove standing credentials from laptops, move prod applies to a pipeline whose role is assumable only from `main` via OIDC, require a reviewed plan on every PR, add `checkov` so a rule exposing port 22 to `0.0.0.0/0` fails the build, and keep a documented, audited break-glass role for real emergencies. **Lesson:** the pipeline is not bureaucracy; it is the only place where "what did we review" and "what did we apply" are guaranteed to be the same thing.

8 Common mistakes and best practices

Don't

- Store `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` as long-lived repository secrets when OIDC is available.
- Use a wildcard `sub` condition in the trust policy, or let pull requests from forks assume a write role.
- Use `-auto-approve` on a fresh plan that nobody saw; apply the saved, reviewed plan.
- Print plan output with secrets into public logs, or upload plan files as long-retention public artifacts.
- Give the PR plan role write permissions "to make it easier".
- Silence scanner findings globally; suppress individual checks with a comment and a reason.

Do

- Use two roles (plan = read-only, apply = write, `main`/environment only) with `max_session_duration` of an hour.
- Pin the Terraform version, provider versions and `.terraform.lock.hcl`; update them via Dependabot or Renovate PRs.
- Run `fmt`, `validate`, `tflint` and a security scanner on every PR; fail the build on high severity.
- Require manual approval (GitHub Environment reviewers) for `prod`; serialise runs per stack with `concurrency` and state locking.
- Keep a break-glass procedure that is logged and reviewed, and reconcile with a follow-up PR.

- Treat the pipeline's own code (workflow files, IAM roles) as protected: CODEOWNERS review for `.github/workflows/`.

9 Tasks

- [ ] **T1 — Local quality gates** *(≈30 min, beginner)*
  - **Goal:** run the same checks CI will run.
  - **Steps:**
    1. Follow steps 1-3.
    2. Introduce an unformatted file and an invalid reference; observe `fmt -check` and `validate` fail, then fix both.
  - **Done when:** `fmt -check`, `validate`, `tflint` and `checkov` all pass, and you have seen each fail once on purpose.
- [ ] **T2 — Plan on pull request** *(≈60 min, intermediate; AWS sandbox)*
  - **Goal:** automate the plan with a PR comment.
  - **Steps:**
    1. Complete steps 4-6 (OIDC provider, plan role, workflow).
    2. Open a PR changing one value.
  - **Done when:** the PR shows a green `plan` job and a comment with `Plan: 0 to add, 1 to change, 0 to destroy.`
- [ ] **T3 — Apply on merge with an approval gate** *(≈45 min, intermediate)*
  - **Goal:** only reviewed code reaches the cloud.
  - **Steps:**
    1. Finish steps 7-8.
    2. Create a GitHub Environment named `prod` with a required reviewer and point a second job at it.
  - **Done when:** merging triggers the dev apply automatically and the prod job waits for approval.
- [ ] **T4 — Break it: tighten and test the trust policy** *(≈45 min, advanced)*
  - **Goal:** prove OIDC scoping works.
  - **Steps:**
    1. Change the apply role's `sub` to a different branch name and re-run the apply job on `main`.

2. Read the error (`Not authorized to perform sts:AssumeRoleWithWebIdentity`) in the job log.
3. Restore the correct condition. Then try to run the plan job from a branch in the same repo with the apply role ARN and note it is denied.

- **Done when:** you can show one job denied by the trust policy and one allowed, and explain the `sub` claim for each.

- [ ] **T5 — Policy as code** *(≈60 min, advanced)*
  - **Goal:** enforce an organisation rule on the plan.
  - **Steps:**
    1. Add the Rego rule requiring an `Owner` tag and run `conftest` against `tfplan.json`.
    2. Remove the tag and confirm the check fails; add a `checkov` custom or built-in check for an open SSH security group and trigger it.
    3. Wire both into the `checks` job.
  - **Done when:** a PR that removes the `Owner` tag or opens port 22 to the world fails CI with a clear message.

- [ ] **T6 — Prove it: design the pipeline** *(≈45 min, advanced)*
  - **Goal:** explain a safe infrastructure pipeline from memory.
  - **Steps:**
    1. Without notes, draw the PR-to-prod flow including the roles, the OIDC exchange, the saved plan and the approval gate.
    2. List what each stage protects against and what remains unprotected.
  - **Done when:** you can answer "what stops an engineer or a compromised fork from changing prod?" with specific controls.

10 Self-check questions

1. Why apply a saved plan rather than running `terraform apply -auto-approve`?
2. How does OIDC remove the need for stored AWS keys, and what makes it safe?
3. Why use separate plan and apply roles?
4. What do `tflint`, `checkov` and `conftest` each check that the others do not?
5. What happens if two pipeline runs target the same state at once?

ANSWERS

1. A saved plan applies exactly what was reviewed and fails if state changed in the meantime. `-auto-approve` on a fresh plan can apply something nobody reviewed.
2. The workflow requests a short-lived signed token from GitHub; AWS STS verifies it against the trust policy (audience, repository, branch or environment) and returns temporary credentials. Nothing long-lived is stored, and the scope is controlled by the `sub` condition.
3. PRs from branches run untrusted code; they only need read access. Write access is limited to a role assumable only from `main` or a protected environment.
4. `tflint` finds provider-specific errors and deprecated usage. `checkov` or `trivy config` scan for security misconfigurations against built-in rules. `conftest` enforces your own custom rules on the plan JSON.
5. State locking makes the second run wait or fail with a lock error (L7.8); `concurrency` groups in the CI system prevent it earlier.

11 Further reading

- Terraform documentation: *Running Terraform in automation* and the `terraform plan -out` / `apply` documentation.
- GitHub documentation: *Security hardening with OpenID Connect* and *Configuring OpenID Connect in Amazon Web Services*.
- Documentation of `aws-actions/configure-aws-credentials`, `hashicorp/setup-terraform`, TFLint, Checkov, Trivy (config scanning) and Conftest / Open Policy Agent.
- Atlantis documentation (runatlantis.io) for the PR-comment workflow.
- See also: GitHub Actions for CI/CD §5.2, IAM §6.7 and Policy as Code §18.7.

7.8 State Locking

Topic 8 of 9 | ~4 hours | Difficulty: Advanced

1 What it is

State locking is the mechanism that stops two Terraform runs from modifying the same state at the same time. Before any operation that could write state (`plan`, `apply`, `destroy`, `import`, `state mv` ...), Terraform asks the backend for an exclusive **lock**. If someone else holds it, Terraform waits (up to `-lock-timeout`) or fails with an error that names the holder. When the run finishes, the lock is released.

Without locking, two concurrent applies each read the same old state, each make changes, and the second write overwrites the first. The result is a state that no longer matches reality: orphaned resources, duplicate creates, or a state file that corrupts. Locking is built into the backend you choose. With the `s3` backend there are two ways: the **native S3 lock file** (`use_lockfile = true`, the current approach) and the older **DynamoDB table** (`dynamodb_table`).

2 Why it matters in DevOps

- **CI plus humans plus schedulers** all touch the same stack: the PR pipeline, the merge pipeline, the nightly drift check (L7.9) and an engineer debugging. Collisions are normal, not rare.
- **A stuck lock blocks every deploy** for that stack. Knowing when `force-unlock` is safe (and when it is data-corrupting) is an on-call skill.
- **Cost of a mistake is high.** `-lock=false` and careless `force-unlock` are among the few Terraform commands that can destroy the guarantees you rely on.
- **Platform choice.** Moving off DynamoDB removes a table, its IAM permissions and its capacity settings from every environment.

3 Real-world system example

The S3 backend itself. The documented `s3` backend originally required a DynamoDB table with a string partition key named `LockID` for locking: Terraform wrote an item per state path with the holder's details. Following Amazon S3's introduction of **conditional writes** (an `If-None-Match` header so an object is only created if it does not already exist), the backend gained `use_lockfile`, which creates a `<state key>.tflock` object next to the state; this arrived as an experimental feature in Terraform 1.10 and is stable in 1.11. The backend documentation now marks `dynamodb_table` as deprecated and expected to be removed in a future minor version. OpenTofu implemented the same lock-file option. **HCP Terraform** (formerly Terraform Cloud) sidesteps the problem by queueing runs per workspace, so only one run is active at a time.

4 Diagram

Two runs racing for the same state with native S3 locking:

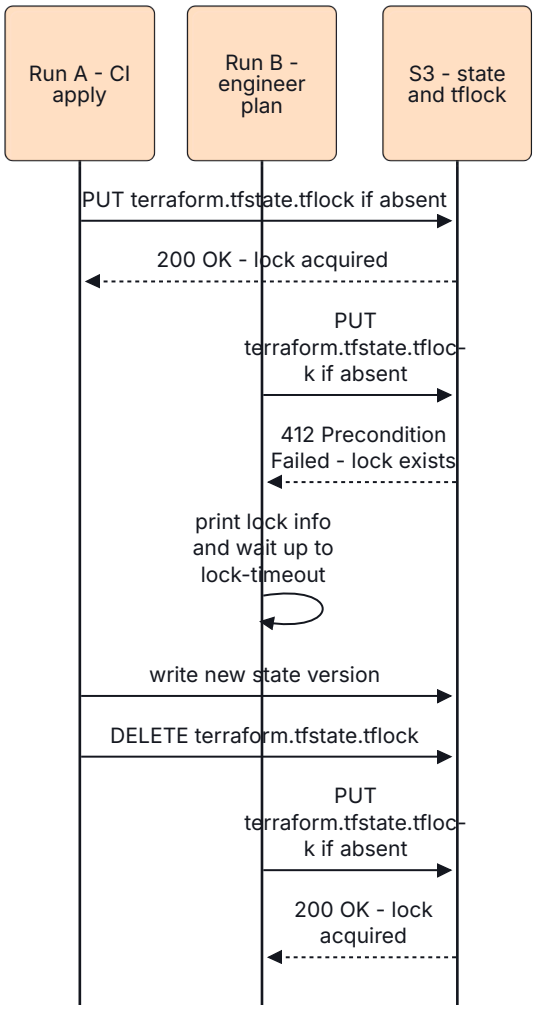


Figure 7.8-1

Lock life cycle, including the failure path:

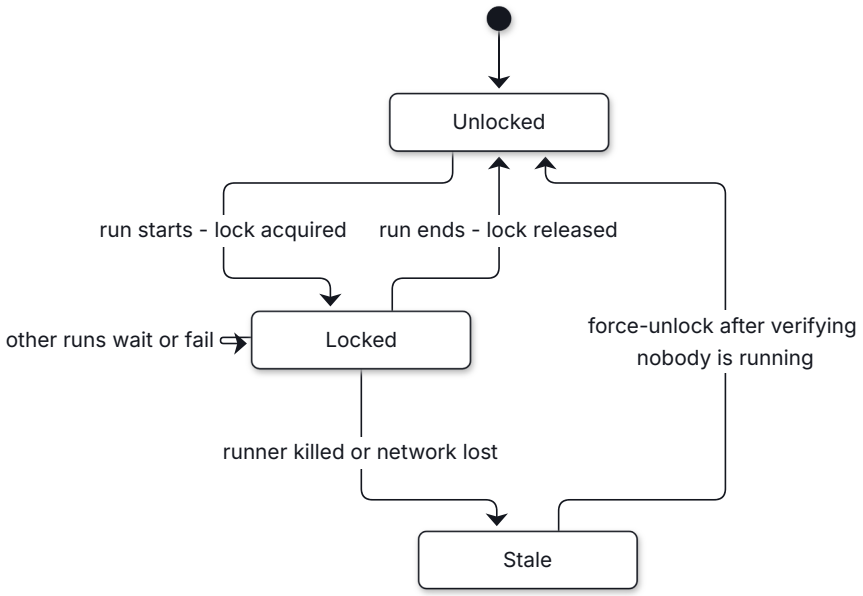


Figure 7.8-2

5 How it works

Native S3 locking (current)

```hcl
terraform {
  backend "s3" {
    bucket       = "shop-api-tfstate-123456789012"
    key          = "prod/app/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true        # creates prod/app/terraform.tfstate.tflock
  }
}
```

- Needs Terraform 1.10+ (stable in 1.11+) or a recent OpenTofu, and an S3 endpoint that supports conditional writes (AWS S3 does; check S3-compatible stores before relying on it).
- The identity needs `s3:GetObject`, `s3:PutObject` and `s3:DeleteObject` on `<key>.tflock` in addition to the state object, and `s3:ListBucket` on the bucket.
- Enable bucket versioning as before; the lock object is small and short-lived.

DynamoDB locking (older approach)

```hcl
backend "s3" {
  bucket         = "shop-api-tfstate-123456789012"
  key            = "prod/app/terraform.tfstate"
  region         = "us-east-1"
  dynamodb_table = "terraform-locks"     # deprecated in current Terraform
  encrypt        = true
}

resource "aws_dynamodb_table" "locks" {
  name         = "terraform-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

The identity needs `dynamodb:GetItem`, `PutItem` and `DeleteItem` on the table. You will still meet this in older repositories.

	Native S3 (`use_lockfile`)	DynamoDB (`dynamodb_table`)
Extra infrastructure	none	a table per account/region
Extra IAM	object permissions on `.tflock`	table permissions
Status	recommended	deprecated, still works
Min versions	Terraform 1.10 (experimental), 1.11 stable	all versions
Cost	negligible (a few S3 requests)	negligible, but one more resource to manage

Migrating from DynamoDB to native locking

1. Upgrade to Terraform 1.11+ everywhere that uses the stack (laptops, CI images, pre-commit hooks).
2. Set **both** `use_lockfile = true` and `dynamodb_table = "terraform-locks"` and run `terraform init -reconfigure`. Terraform acquires both locks, so old and new clients still exclude each other during the transition.
3. When every client is updated, remove `dynamodb_table`, run `terraform init -reconfigure` again, and finally destroy the table.

Lock-related flags

Flag	Meaning
`-lock-timeout=5m`	Wait up to 5 minutes for a held lock instead of failing immediately. Use in CI.
`-lock=false`	Do not lock at all. Only for read-only experiments you fully understand; never for `apply`.
`terraform force-unlock <LOCK_ID>`	Remove a lock you are certain is stale. Does not touch state.

Reading a lock error

```
Error: Error acquiring the state lock

Error message: operation error S3: PutObject, https response error StatusCode: 412 ...
Lock Info:
  ID:        6f0f2a64-0c3e-3c1a-8a2b-5f2d4f7c9b10
  Path:      shop-api-tfstate-123456789012/prod/app/terraform.tfstate
  Operation: OperationTypeApply
  Who:       runner@fv-az123-456
  Version:   1.11.4
  Created:   2026-03-02 09:14:07.123456789 +0000 UTC
  Info:
```

(Illustrative; the exact wording depends on the backend and Terraform version.) The useful fields are **Who** and **Created**: they tell you whether the holder is a live CI job, a colleague's laptop or a process that died an hour ago.

Stuck locks: a decision procedure

1. Read **Who** and **Operation** in the error. Is that run still going? Check the CI job list and ask the person.
2. If the job is running, **wait**. Do not unlock a live apply: two writers will corrupt state.
3. If the job was cancelled or the runner died, check for partial changes. The next `plan` shows what remains.
4. Unlock with the ID from the error: `terraform force-unlock 6f0f2a64-0c3e-3c1a-8a2b-5f2d4f7c9b10`. (With native S3 locks you could also delete the `.tflock` object; prefer `force-unlock`.)
5. Run `terraform plan` and reconcile. If a state write was interrupted, restore the previous state version from S3 versioning (L7.4).

Concurrency in CI

Locking is the last line of defence; avoid contention earlier:

- GitHub Actions `concurrency: { group: tf-prod-app, cancel-in-progress: false }` queues runs per stack and never kills an in-flight apply.
- Prefer **small states** (network / data / app per environment) so unrelated changes never contend.
- Scheduled jobs (drift checks, L7.9) must still lock: use `-lock-timeout` and retry later if the lock is held. Never bypass it with `-lock=false`.
- Cancelling a CI job sends a signal; Terraform tries to release the lock on interrupt, but a hard kill leaves it behind, which is why `cancel-in-progress: false` matters for applies.

6 Step-by-step guide

Prerequisites: Terraform 1.11+ (or OpenTofu). Part A uses only the local backend and costs nothing. Part B uses the S3 bucket from L7.4 and costs a fraction of a cent.

Part A: see locking locally

1. **Create a slow stack.**

```bash
mkdir -p lock-demo && cd lock-demo
```

`main.tf`:

```hcl
terraform {
  required_version = "≥ 1.11"
}

resource "terraform_data" "slow" {
  provisioner "local-exec" {
    command = "sleep 40"
  }
}
```

```bash
terraform init
```

2. **Start an apply in one terminal.**

```bash
terraform apply -auto-approve
```

3. **In a second terminal, try to plan the same directory.**

```bash
cd lock-demo && terraform plan
```

You should see `Error: Error acquiring the state lock` with a `Lock Info` block (ID, Operation, Who). That is Terraform refusing to race the first run.

4. **Wait instead of failing.**

```bash
terraform plan -lock-timeout=60s
```

Run it during the first apply. It prints `Acquiring state lock. This may take a few moments...` and succeeds when the first run finishes.

Part B: native S3 locking and a stuck lock

5. **Configure the backend.** In a new directory:

```hcl
terraform {
  required_version = "≥ 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket       = "shop-api-tfstate-123456789012"   # your bucket from L7.4
    key          = "dev/lock-demo/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
provider "aws" { region = "us-east-1" }

resource "aws_ssm_parameter" "demo" {
  name  = "/shop-api/dev/lock-demo"
  type  = "String"
  value = "v1"
}

resource "terraform_data" "slow" {
  triggers_replace = timestamp()      # forces a 40 second run every time
  provisioner "local-exec" {
    command = "sleep 40"
  }
}
```

```bash
export BUCKET=shop-api-tfstate-123456789012
terraform init && terraform apply -auto-approve
```

6. **Watch the lock object appear.** Run `terraform apply -auto-approve` again. While it sleeps, in another terminal:

```bash
aws s3 ls "s3://$BUCKET/dev/lock-demo/"
```

   You see `terraform.tfstate` and, only during the run, `terraform.tfstate.tflock`.

7. **Create a stuck lock.** Start the apply and kill it hard:

```bash
terraform apply -auto-approve &
sleep 8 && kill -9 %1
terraform plan
```

   The plan fails with `Error acquiring the state lock` and shows the dead run's lock ID.

8. **Recover safely.** Confirm no run is active (`ps`, CI), then:

```bash
terraform force-unlock <LOCK_ID_FROM_THE_ERROR>      # answer yes
terraform plan
```

   The plan works again. Verify the `.tflock` object is gone with `aws s3 ls`.

9. **Clean up.**

```bash
terraform destroy -auto-approve
rm -rf lock-demo
```

7 Real-world scenario

(Illustrative.) A Friday-evening deploy hung when the CI runner was pre-empted mid-apply. Monday morning every pipeline for the `prod/app` stack failed with a lock error from the weekend; the on-call engineer saw `Who: runner@...`, `Created: Friday 18:42` and no matching running job. They checked S3 versioning to confirm the last state write was complete, ran `terraform plan` to see two resources left unchanged, then `terraform force-unlock <ID>`, re-ran the pipeline and everything converged. Later the same quarter a colleague "fixed" a

different lock error with `-lock=false` while a drift job was running; the two writes collided and the state lost a security group, which took an hour of `terraform import` to repair. **Lesson:** verify the holder first; unlock only dead locks, and never switch locking off to get unblocked.

8 Common mistakes and best practices

Don't

- Use `-lock=false` for `apply` or `destroy`, ever.
- Run `force-unlock` without proving the owner is gone.
- Share one state and one lock across unrelated teams and layers: one long apply blocks everyone.
- Delete the lock object, table or state "to reset things" without checking what the run was doing.
- Cancel an in-flight apply in CI with hard kills (`cancel-in-progress: true` on apply jobs).

Do

- Enable locking from the first day: `use_lockfile = true` on new S3 backends; plan a migration for old DynamoDB tables.
- Add `-lock-timeout=5m` (or similar) to CI plans and applies so transient contention resolves itself.
- Serialise runs per stack with CI `concurrency` groups, and keep states small.
- Give the pipeline role permissions on the `.tflock` object (or the DynamoDB table) and test it.
- Document the stuck-lock runbook and keep S3 versioning on so an interrupted write can be rolled back.
- Alert when a lock is older than your longest normal apply.

9 Tasks

- [ ] **T1 — Provoke and read a lock error** *(≈30 min, beginner)*
  - **Goal:** see locking protect a state.
  - **Steps:**
    1. Do Part A steps 1-3.
    2. Copy the `Lock Info` block into your notes and label each field.

- **Done when:** the second `plan` fails with a lock error and you can say who holds the lock and since when.

- [ ] **T2 — Wait with `-lock-timeout`** *(≈20 min, beginner)*
  - **Goal:** make contention self-healing.
  - **Steps:**
    1. Do Part A step 4.
    2. Repeat with `-lock-timeout=5s` and observe the failure after 5 seconds.
  - **Done when:** the long timeout succeeds after waiting and the short one fails, both with output you can show.

- [ ] **T3 — Native S3 lock in action** *(≈45 min, intermediate; AWS sandbox)*
  - **Goal:** observe the `.tflock` object.
  - **Steps:**
    1. Do Part B steps 5-6.
    2. Check the lock object's contents with `aws s3 cp s3://$BUCKET/dev/lock-demo/terraform.tfstate.tflock -` during the run.
  - **Done when:** you captured the lock object's JSON (ID, Who, Operation) while a run was active and showed it vanishes afterwards.

- [ ] **T4 — Break it: stuck lock and recovery** *(≈45 min, advanced)*
  - **Goal:** practise the decision procedure.
  - **Steps:**
    1. Do Part B steps 7-8.
    2. Write the checklist you followed before running `force-unlock`.
  - **Done when:** the stuck lock is cleared, `terraform plan` succeeds and your checklist has at least three verification steps.

- [ ] **T5 — Migrate from DynamoDB to native locking** *(≈60 min, advanced)*
  - **Goal:** perform the transition safely.
  - **Steps:**
    1. Create a stack that uses `dynamodb_table` locking (create the table with `PAY_PER_REQUEST`).
    2. Add `use_lockfile = true`, run `terraform init -reconfigure`, and confirm both a DynamoDB item and a `.tflock` object appear during a run.
    3. Remove `dynamodb_table`, `init -reconfigure`, and destroy the table.

- **Done when:** a run now locks only through the `.tflock` object and the table no longer exists.

☐ **T6 — Prove it: concurrency design** *(≈45 min, advanced)*

- **Goal:** design locking and queueing for a team.
- **Steps:**
  1. Without notes, describe what happens when a PR plan, a merge apply and a nightly drift check start within the same minute for one stack.
  2. List the three layers that handle it (CI concurrency group, backend lock, `-lock-timeout`) and what each does.
  3. Write the on-call runbook for "state lock held for hours".
- **Done when:** a teammate can follow your runbook and knows exactly when `force-unlock` is forbidden.

10 Self-check questions

1. What exactly does a state lock protect against?
2. How does native S3 locking work, and why did it become possible?
3. When is `terraform force-unlock` safe, and what does it actually change?
4. Why is `-lock=false` dangerous?
5. How do you migrate from DynamoDB locking to native S3 locking without a window where both old and new clients can run together?

ANSWERS

1. Two concurrent operations reading the same state and overwriting each other's writes, which corrupts the mapping between code and real infrastructure.
2. Terraform creates a `<key>.tflock` object using an S3 conditional write (create only if absent), so only one client succeeds; it deletes the object when finished. S3 added conditional writes, so a separate DynamoDB table is no longer needed.
3. Only when you have verified the holder is dead (job cancelled, runner gone, laptop closed). It removes the lock record only; it does not modify or repair state.
4. It skips the exclusion entirely, so concurrent applies can overwrite each other's state and leave orphaned or duplicated resources.
5. Enable `use_lockfile = true` while keeping `dynamodb_table` so both locks are taken, update all clients to a supporting version, then remove `dynamodb_table` and delete the table.

Further reading

11

- Terraform documentation: *Backend type: s3* (locking, `use_lockfile`, deprecation of `dynamodb_table`), *State Locking* and the `force-unlock` command.
- AWS documentation: *Amazon S3 conditional writes*.
- OpenTofu documentation: *S3 backend* for its lock-file option.
- GitHub documentation: *Control the concurrency of workflows and jobs*.
- See also: State and Remote Backends §7.4, S3 §6.8 and Terraform CI/CD §7.7.

LAB NOTES What I ran, what I saw, what it means

7.9 Drift Detection

Topic 9 of 9 | ~4 hours | Difficulty: Advanced

1 What it is

Drift is any difference between what your Terraform code and state say exists and what actually exists in the cloud. It appears when someone changes a resource outside Terraform: an engineer opens a port in the console during an incident, an autoscaler or another tool edits a tag, a vendor agent modifies a setting, or a resource is deleted by hand. The code still says one thing; reality says another.

Drift detection is the habit of finding those differences on purpose and early, instead of discovering them in the middle of the next `apply`. Terraform already contains the mechanism: on every `plan` it *refreshes* (reads the real objects from the provider) and compares them with your configuration. A scheduled, read-only plan whose result you act on is drift detection. **Reconciling** drift means deciding, per difference, whether reality or the code is right, then making them match.

2 Why it matters in DevOps

- **Security.** The classic drift is "temporarily" opening SSH to `0.0.0.0/0` and forgetting. Code review never saw it, so only a scan of reality finds it.
- **Surprise applies.** The next unrelated PR plan suddenly proposes to revert a hotfix (maybe re-breaking production) or to destroy something a colleague created by hand.
- **Loss of trust in the repo.** If the code is not what runs, runbooks, audits and disaster recovery built on the code are wrong (Disaster Recovery §0.9).
- **Compliance.** Auditors want evidence that production matches approved configuration.

3 Real-world system example

HCP Terraform health assessments, AWS CloudTrail and AWS Config. HCP Terraform (formerly Terraform Cloud) offers *health assessments* that run drift detection (a refresh-only style check) on a schedule for workspaces and report resources that no longer match the configu-

ration. Teams that run open-source Terraform build the same thing with a scheduled CI job around `terraform plan -detailed-exitcode`, which Terraform documents for exactly this kind of automation. To find out *who* made a manual change, AWS teams look in **CloudTrail** event history (CloudTrail §6.17); **AWS Config** records configuration changes per resource and can run rules (for example "security groups must not allow 0.0.0.0/0 on port 22") that alert regardless of how the change was made. Kubernetes GitOps tools such as Argo CD apply the same idea continuously to cluster objects (Continuous Reconciliation §17.3).

4 Diagram

A scheduled drift check:

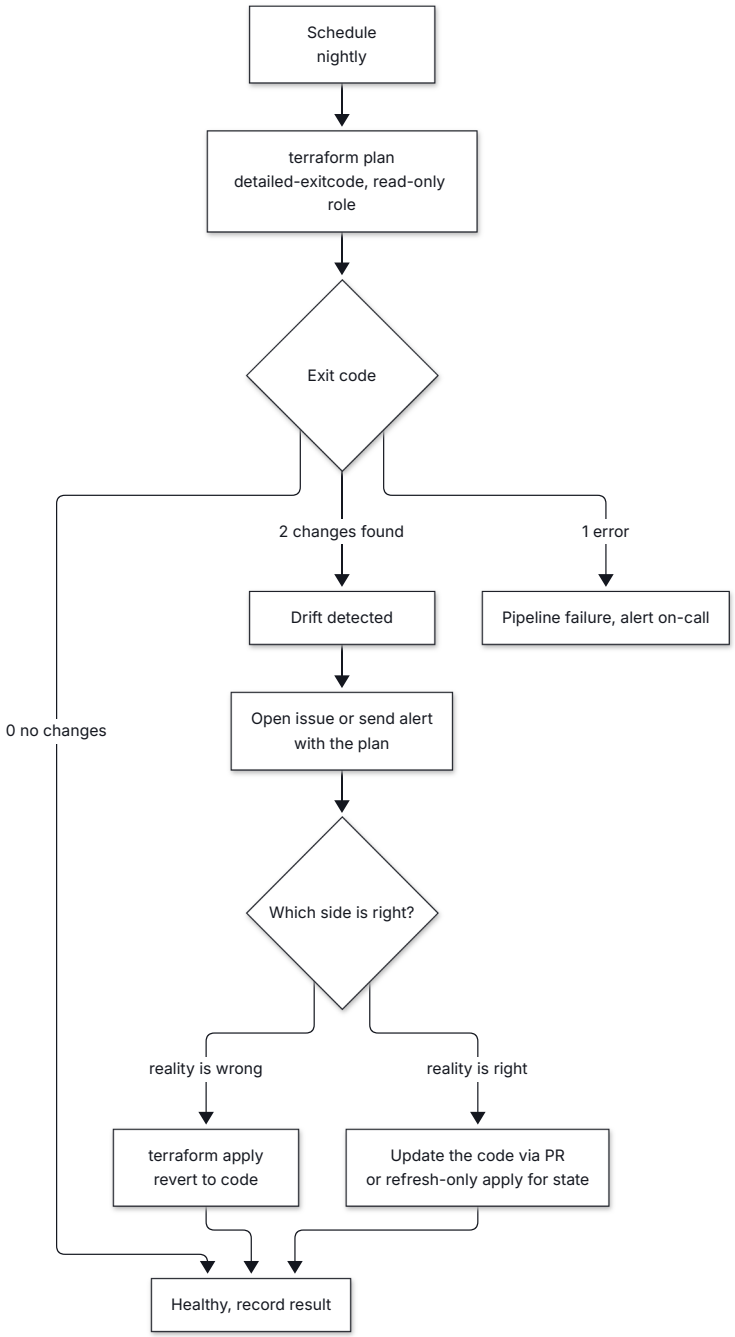

Schedule
nightly
terraform plan
detailed-exitcode, read-only
role
Exit code
0 no changes
2 changes found
1 error
Drift detected
Pipeline failure, alert on-call
Open issue or send alert
with the plan
Which side is right?
reality is wrong
reality is right
terraform apply
revert to code
Update the code via PR
or refresh-only apply for state
Healthy, record result

Figure 7.9-1

How drift is created and found:

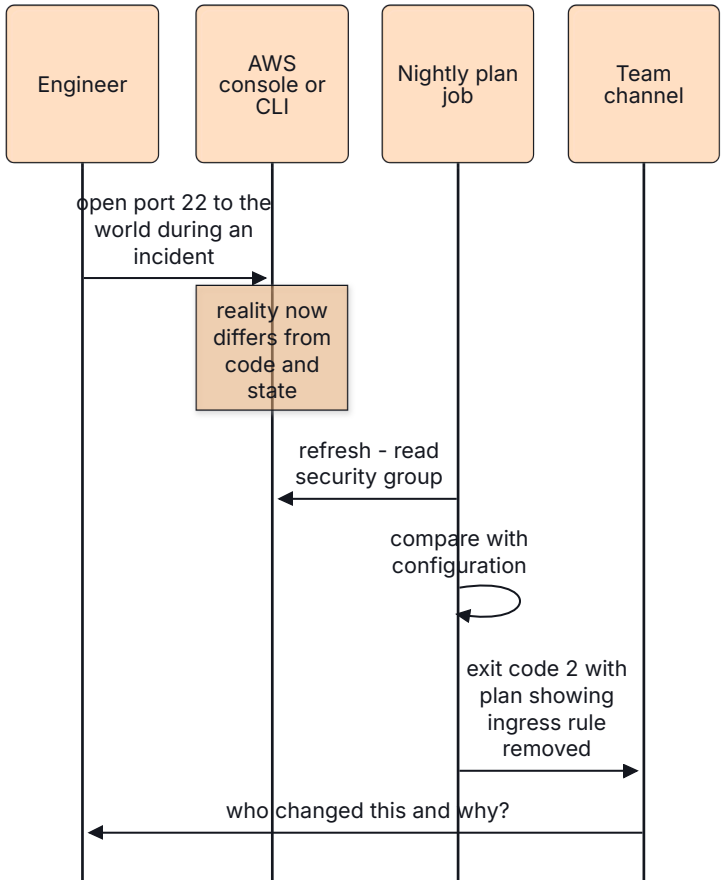


Figure 7.9-2

5 How it works

What Terraform compares

Source	Meaning
Configuration	what you want
State	what Terraform last recorded
Real infrastructure	what the provider API returns during refresh

`plan` refreshes state from reality first, then diffs configuration against the refreshed state. Changes made outside Terraform appear in two forms in recent Terraform versions: a section titled **"Objects have changed outside of Terraform"** (the refresh found differences) and then the normal list of actions needed to bring reality back to the configuration.

Commands

```bash
# Is there any difference? Exit code: 0 = none, 1 = error, 2 = changes present
terraform plan -detailed-exitcode -lock-timeout=5m -input=false
echo $?

# Show only what changed in the real world, and do not propose config changes
terraform plan -refresh-only

# Accept reality into the state file (never changes infrastructure)
terraform apply -refresh-only
```

`-refresh-only` is the explicit, safe form of "refresh": it updates **state** to match reality and touches no infrastructure. Use it when reality is right and you also will update the code to match, or for attributes the provider reports differently after a provider upgrade. The old standalone `terraform refresh` command is deprecated in favour of it.

Reconciling: decide which side wins

Situation	Action
Manual change was wrong or unauthorised	`terraform apply`: Terraform reverts reality to the code. Review the plan first.
Manual change was right (a hotfix that should stay)	Edit the code in a PR to include it, then `plan` until it says no changes. Optionally `apply -refresh-only` to record the observed values.
Resource was deleted by hand	`apply` recreates it (check data loss for stateful resources), or remove it from code with a PR.
Resource was created by hand and should be managed	Add it to code with an `import` block (L7.4).
Another system legitimately owns an attribute (autoscaler capacity, tags added by a cost tool)	`lifecycle { ignore_changes = [...] }` (L7.5) so it stops appearing as drift.
Provider returns a different but equivalent value after an upgrade	Pin the provider, read its changelog, and use `apply -refresh-only` to settle state.

Preventing drift

Detection is the safety net; prevention is cheaper:

- Remove write access to production from humans: changes go through the pipeline (L7.7). Keep a logged, time-boxed break-glass role.
- Use **AWS Config** or service control policies to block or alert on classes of manual change.
- Tag Terraform-managed resources (`ManagedBy = "terraform"`) so people know not to touch them.
- Use `default_tags` in the AWS provider so tags are consistent and the tags agent-tools add stand out.
- Prefer immutable replacement over in-place edits (Immutable Infrastructure §0.5).

A scheduled drift job (GitHub Actions)

YAML

```yaml
# .github/workflows/drift.yml
name: drift-check
on:
  schedule:
    - cron: "17 5 * * *"      # daily, off the hour to avoid peak load
  workflow_dispatch: {}

permissions:
  id-token: write
  contents: read
  issues: write

jobs:
  drift:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: infra/envs/prod
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.11.4"
          terraform_wrapper: false          # so the real exit code is visible
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/terraform-plan-prod   # read-only
  role
          aws-region: us-east-1
      - run: terraform init -input=false
      - id: plan
        run: |
          set +e
```

```
        terraform plan -detailed-exitcode -lock-timeout=5m -input=false -no-color >
plan.txt
        echo "exitcode=$?" >> "$GITHUB_OUTPUT"
    - if: steps.plan.outputs.exitcode == '1'
      run: echo "Plan failed" && exit 1
    - if: steps.plan.outputs.exitcode == '2'
      uses: actions/github-script@v7
      with:
        script: |
          const fs = require('fs');
          const plan = fs.readFileSync('infra/envs/prod/plan.txt', 'utf8').slice(0,
60000);
          const fence = '`'.repeat(3);
          await github.rest.issues.create({
            owner: context.repo.owner, repo: context.repo.repo,
            title: 'Drift detected in prod',
            labels: ['drift'],
            body: 'Nightly plan found differences.\n\n' + fence + '\n' + plan + '\n' +
fence
          });
```

Use a **read-only** role: detection never needs to write. Avoid opening a new issue every night: check for an existing open `drift` issue first, or update it. Because the plan text may contain sensitive values, post it only to private repositories or post a summary.

6 Step-by-step guide

Prerequisites: Terraform 1.11+ (or OpenTofu), AWS CLI with sandbox credentials and the state bucket from L7.4. The resources (an SSM parameter and a security group) are free.

1. **Create a small managed stack.** `main.tf`:

```hcl
terraform {
  required_version = "≥ 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket       = "shop-api-tfstate-123456789012"
    key          = "dev/drift-demo/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}

provider "aws" {
  region = "us-east-1"
```

```
  default_tags {
    tags = { ManagedBy = "terraform", Project = "shop-api" }
  }
}

data "aws_vpc" "default" { default = true }

resource "aws_security_group" "web" {
  name        = "shop-api-drift-demo"
  description = "Drift demo"
  vpc_id      = data.aws_vpc.default.id
}

resource "aws_vpc_security_group_ingress_rule" "https" {
  security_group_id = aws_security_group.web.id
  cidr_ipv4         = "10.0.0.0/16"
  ip_protocol       = "tcp"
  from_port         = 443
  to_port           = 443
}

resource "aws_ssm_parameter" "feature" {
  name  = "/shop-api/dev/feature-flag"
  type  = "String"
  value = "off"
}

output "sg_id" {
  value = aws_security_group.web.id
}
```

BASH

```
terraform init && terraform apply -auto-approve
terraform plan -detailed-exitcode; echo "exit=$?"
```

Expect `No changes.` and `exit=0`.

2. **Introduce drift by hand** (pretend to be the 2 a.m. engineer):

BASH

```
SG=$(terraform output -raw sg_id)
aws ec2 authorize-security-group-ingress --group-id "$SG" --protocol tcp --port 22 --
  cidr 0.0.0.0/0
aws ssm put-parameter --name /shop-api/dev/feature-flag --type String --value on --
  overwrite
aws ec2 create-tags --resources "$SG" --tags Key=Owner,Value=somebody-else
```

3. **Detect it.**

```bash
terraform plan -detailed-exitcode; echo "exit=$?"
```

The exit code is `2`. The output is similar to:

```
Note: Objects have changed outside of Terraform
...
  # aws_ssm_parameter.feature has changed
  ~ value = (sensitive value)
...
Plan: 0 to add, 1 to change, 0 to destroy.
```

Terraform reports the parameter (and, depending on provider version, the extra tag) that it will revert. Note the extra SSH rule is a separate `aws_vpc_security_group_ingress_rule` resource, so it is **not** managed and Terraform does not see it: that is the limit of plan-based detection. Section 8 covers how AWS Config closes the gap.

4. **See the refresh-only view.**

```bash
terraform plan -refresh-only
```

This lists only the real-world changes and proposes updating state, not infrastructure.

5. **Reconcile in the "reality is wrong" direction.**

```bash
terraform apply -auto-approve
aws ssm get-parameter --name /shop-api/dev/feature-flag --query Parameter.Value --
  output text   # off again
```

Then remove the rogue SSH rule by hand (`aws ec2 revoke-security-group-ingress ...`) or by managing the whole group inline with `ingress` blocks, which makes unknown rules visible as drift.

6. **Reconcile in the "reality is right" direction.** Set the flag by hand again, decide the change should stay, change the code to `value = "on"` in a PR-like edit and run `terraform plan`. It says `No changes`. If you only wanted to record the observed value without editing code first, `terraform apply -refresh-only` updates state.

7. **Ignore an attribute owned by someone else.** Add `lifecycle { ignore_changes = [value] }` to the parameter, change the value by hand, and show `terraform plan` stays at `No changes`. Document why in a comment.

8. **Schedule it.** Add the drift workflow from section 5 to a repository (or run the plan command from cron in your sandbox). Trigger it manually with `workflow_dispatch` and confirm it opens an issue when exit code 2 occurs.
9. **Clean up.**

```bash
terraform destroy -auto-approve
aws ec2 revoke-security-group-ingress --group-id "$SG" --protocol tcp --port 22 --cidr
  0.0.0.0/0 2>/dev/null || true
```

7 Real-world scenario

(Illustrative.) During an incident an engineer added an ingress rule to a production security group from their home IP, resolved the issue and forgot. Two weeks later a routine PR changing an ALB timeout produced a plan that also removed an unrelated rule because the group's rules were managed inline, which confused the reviewer. Investigation through CloudTrail showed the manual `AuthorizeSecurityGroupIngress` call. The team introduced a nightly read-only plan job that opens an issue on exit code 2, added an AWS Config rule for world-open management ports, removed standing console write access to prod, and agreed a rule: any manual change during an incident gets a follow-up PR within 24 hours or is reverted by the next apply. **Lesson:** drift is inevitable under pressure; the goal is to make it visible within a day and cheap to reconcile.

8 Common mistakes and best practices

Don't

- Run `terraform apply` on a drift plan without reading it: you may revert a deliberate hotfix, or recreate a deleted database empty.
- Assume `plan` sees everything. It only covers resources and attributes that Terraform manages; unmanaged resources and unmodelled settings are invisible to it.
- Hide noisy drift with blanket `ignore_changes = all`; ignore specific attributes with a reason.
- Give the drift job write credentials.
- Let drift issues pile up unread; a check nobody acts on is worse than none because it creates false confidence.

Do

- Run a scheduled, read-only `plan -detailed-exitcode` for every important stack and alert on exit code 2 (and 1).
- Decide explicitly per difference: revert, adopt in code, or ignore, and record why in the PR.
- Use `terraform apply -refresh-only` to adopt observed values into state, never hand-edit state.
- Complement Terraform with AWS Config rules and CloudTrail to catch unmanaged resources and answer "who changed it?".
- Tag managed resources with `default_tags`, and reduce the human write paths that create drift in the first place.
- Re-run the check after every reconciliation until the exit code is 0.

9 Tasks

- [ ] **T1 — Create and detect drift** *(≈45 min, beginner; AWS sandbox)*
  - **Goal:** produce a detectable difference and read the output.
  - **Steps:**
    1. Follow steps 1-3.
    2. Save the plan output and underline the "Objects have changed outside of Terraform" section.
  - **Done when:** `terraform plan -detailed-exitcode` returns `2` and you can name the changed attribute and its old and new values.
- [ ] **T2 — Refresh-only versus apply** *(≈30 min, intermediate)*
  - **Goal:** understand the two reconciliation directions.
  - **Steps:**
    1. Do step 4, then step 5 for one resource and step 6 for another.
    2. Explain in a sentence what `apply -refresh-only` changed and what it did not.
  - **Done when:** one resource is reverted to the code, one is adopted, and `plan` returns exit code 0.
- [ ] **T3 — Ignore attributes properly** *(≈30 min, intermediate)*
  - **Goal:** model an attribute owned by another system.
  - **Steps:**
    1. Do step 7.
    2. Then remove `ignore_changes` and show the drift reappears.

- **Done when:** the plan is clean with `ignore_changes` and shows a diff without it.

- [ ] **T4 — Automate nightly detection** *(≈60 min, advanced)*
  - **Goal:** a scheduled job that alerts a human.
  - **Steps:**
    1. Add the drift workflow, with a read-only plan role (see L7.7).
    2. Create drift and trigger it with `workflow_dispatch`.
    3. Make it update an existing open issue rather than creating a duplicate.
  - **Done when:** a labelled issue with the plan appears once, and after reconciliation the next run closes it or leaves it untouched.

- [ ] **T5 — Break it: find the unmanaged change** *(≈45 min, advanced)*
  - **Goal:** discover what plan cannot see, and who did it.
  - **Steps:**
    1. Add the world-open SSH rule by hand (step 2) and confirm `plan` ignores it.
    2. Find the call in CloudTrail event history (`aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=AuthorizeSecurityGroupIngress`).
    3. Change the security group to declare its rules inline so the extra rule becomes visible, or add an AWS Config rule that flags it.
  - **Done when:** you can show the CloudTrail event with the caller identity and a mechanism that now flags the rule.

- [ ] **T6 — Prove it: write the drift policy** *(≈45 min, advanced)*
  - **Goal:** explain drift handling to a team.
  - **Steps:**
    1. Without notes, write a one-page policy: how often drift is checked, who gets alerted, how each kind of difference is resolved, and how break-glass changes are recorded.
    2. Walk a teammate through reconciling one real drift from this topic.
  - **Done when:** the teammate can answer "the nightly job found drift: what do I do first?" using only your page.

10 Self-check questions

1. What is drift and what are three common causes?
2. What do the exit codes 0, 1 and 2 of `terraform plan -detailed-exitcode` mean?

3. How does `terraform apply -refresh-only` differ from `terraform apply`?
4. Name two kinds of change that a Terraform plan cannot detect.
5. How do you decide whether to revert drift or adopt it?

ANSWERS

1. A difference between declared configuration/state and the real infrastructure. Causes: console or CLI hotfixes, other tools or autoscalers editing resources, manual deletions, provider or API default changes.
2. 0: succeeded, no changes. 1: error. 2: succeeded, changes present.
3. `-refresh-only` updates only the state to match reality and changes no infrastructure. A normal apply changes infrastructure to match the configuration.
4. Resources not managed by Terraform at all (created by hand) and attributes the provider or configuration does not model, such as separately managed rules or settings in another tool.
5. Ask whether the manual change was authorised and intended. If yes, update the code through a PR; if no, apply to revert it. Either way, record the decision and re-check for a clean plan.

11 Further reading

- Terraform documentation: `terraform plan` (`-detailed-exitcode`, `-refresh-only`) and *Manage resource drift* tutorial.
- HCP Terraform documentation: *Health assessments* (drift detection).
- AWS documentation: *AWS Config* (managed rules, configuration history) and *CloudTrail event history*.
- *Terraform: Up & Running* by Yevgeniy Brikman — chapters on teamwork and automation.
- See also: Infrastructure as Code §0.3, Terraform CI/CD §7.7 and State Locking §7.8.

★ shop-api on AWS with Terraform

Level 7 capstone ~14 hours

1 Goal and outcome

Build a complete, reviewable, repeatable AWS environment for the `shop-api` sample app using only Terraform: a **VPC** with public, private and data **subnets**, layered **security groups**, **EC2** application servers behind an **Application Load Balancer (ALB)**, a **PostgreSQL RDS** database and a **Redis (ElastiCache)** cache. You then promote it to a second environment, put it behind a pull-request pipeline, and practise the operational skills of the level: locking, drift detection and safe change.

When you finish you will have:

- one codebase (`modules/` plus thin `envs/dev` and `envs/prod` roots) that stands up the whole stack with `terraform apply` and removes it with `terraform destroy`;
- remote, locked, versioned state per environment;
- a CI workflow that plans on pull requests and applies on merge using OIDC (no stored AWS keys);
- a drift check that catches a manual change.

2 Architecture

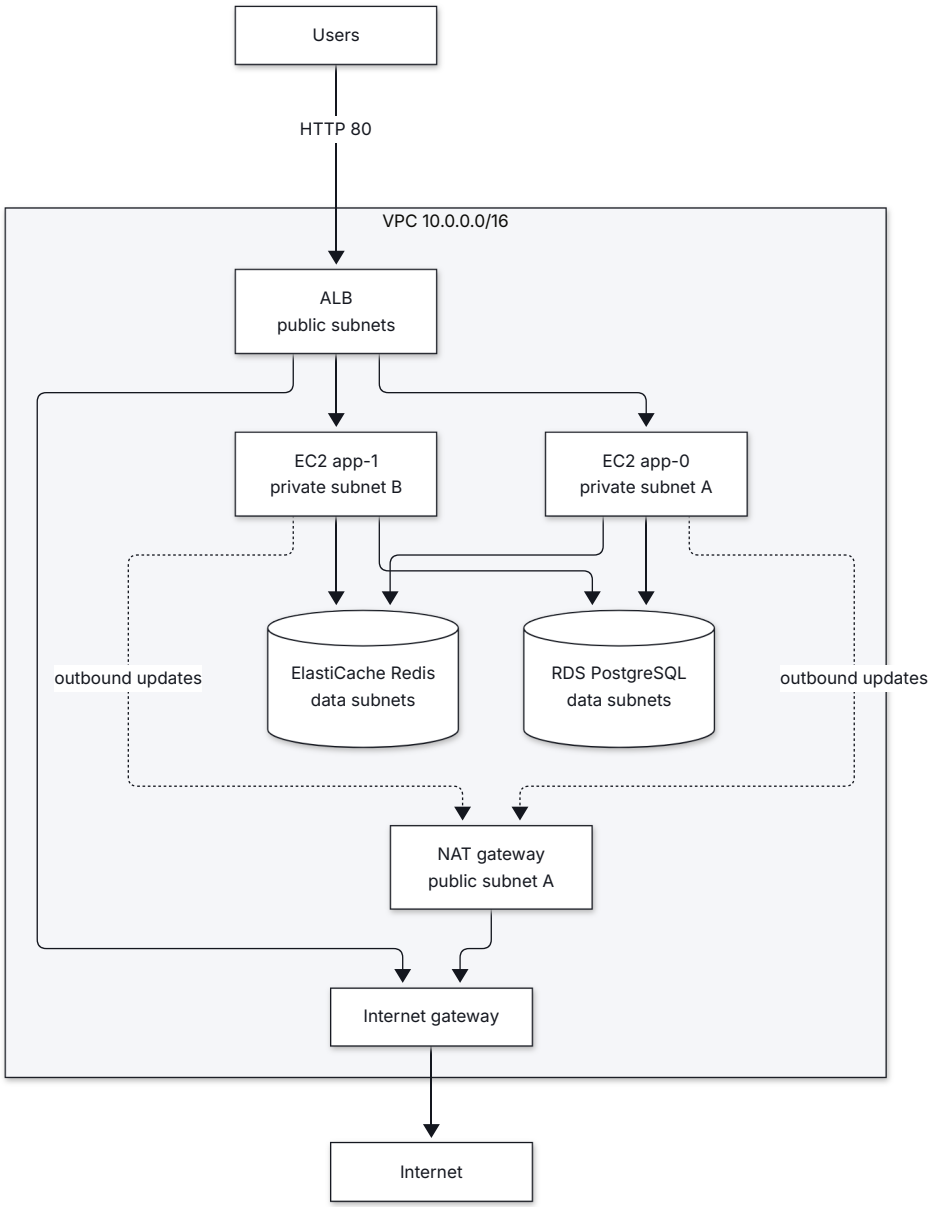


Figure 7-1

Traffic is allowed only along the arrows, enforced by security groups that reference each other rather than CIDR ranges:

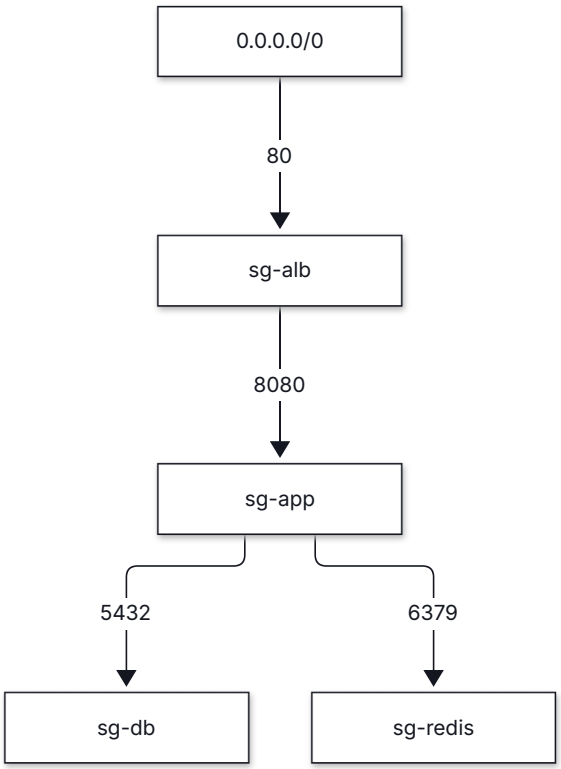


Figure 7-2

Repository layout (state per environment, modules written once):

```
shop-api-infra/
  bootstrap/            # state bucket (one-off, CLI or its own tiny stack)
  modules/
    network/            # VPC, subnets, IGW, NAT, route tables
    data/               # RDS PostgreSQL + ElastiCache Redis
    app/                # IAM profile, ALB, target group, EC2 instances
  envs/
    dev/                # backend key dev/terraform.tfstate, small sizes
    prod/               # backend key prod/terraform.tfstate, bigger, protected
  .github/workflows/    # terraform.yml, drift.yml
```

3 Prerequisites and cost

- Terraform 1.11 or newer (OpenTofu 1.10+ also works for everything here), AWS CLI v2, `git`, a GitHub repository for the CI milestone.
- An **AWS sandbox account** (never production), a **billing alarm**, and credentials with permissions to create the resources above.
- Region `us-east-1`. Everything is tagged `Project = shop-api`.
- **This costs real money while it runs.** The NAT gateway, ALB, RDS instance, ElastiCache node and EC2 instances are all billed per hour; the whole dev stack is typically a few US dollars per day (check current regional pricing). Build, test and **destroy the same day**, and confirm teardown in section 7. Do not leave `prod` running.
- Free-tier eligibility varies by account age and region; do not assume the stack is free.

4 Build it step by step

Milestone 0: remote, locked state (topics 4 and 8)

Create the state bucket once, as in L7.4 (versioning, encryption, Block Public Access), then every root uses it with native locking:

```hcl
# envs/dev/versions.tf
terraform {
  required_version = "≥ 1.11"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket       = "shop-api-tfstate-123456789012"   # replace with your bucket
    key          = "dev/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true          # native S3 locking; older setups used dynamodb_table
  }
}

provider "aws" {
  region = "us-east-1"
  default_tags {
    tags = { Project = "shop-api", Environment = "dev", ManagedBy = "terraform" }
  }
}
```

Milestone 1: network module (topics 1, 2, 3, 5)

HCL

```hcl
# modules/network/variables.tf
variable "name"     { type = string }
variable "vpc_cidr" {
  type    = string
  default = "10.0.0.0/16"
}
variable "az_count" {
  type    = number
  default = 2
}

# modules/network/main.tf
data "aws_availability_zones" "available" { state = "available" }

locals {
  azs = slice(data.aws_availability_zones.available.names, 0, var.az_count)
  idx = { for i, az in local.azs : az => i }
}

resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags                 = { Name = "${var.name}-vpc" }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
}

resource "aws_subnet" "public" {
  for_each          = local.idx
  vpc_id            = aws_vpc.this.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, each.value)         # 10.0.0.0/24,
  10.0.1.0/24
  tags              = { Name = "${var.name}-public-${each.key}" }
}

resource "aws_subnet" "private" {
  for_each          = local.idx
  vpc_id            = aws_vpc.this.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, each.value + 10)    # 10.0.10.0/24, ...
  tags              = { Name = "${var.name}-private-${each.key}" }
}

resource "aws_subnet" "data" {
  for_each          = local.idx
```

```
  vpc_id            = aws_vpc.this.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, each.value + 20)    # 10.0.20.0/24, ...
  tags              = { Name = "${var.name}-data-${each.key}" }
}

resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "this" {                  # one NAT to save cost; use one per
  AZ for real HA
  allocation_id = aws_eip.nat.id
  subnet_id     = values(aws_subnet.public)[0].id
  depends_on    = [aws_internet_gateway.this]        # explicit: NAT needs the IGW to
  exist first
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.this.id
  }
}

resource "aws_route_table" "data" {                  # no default route: the data tier has
  no internet path
  vpc_id = aws_vpc.this.id
}

resource "aws_route_table_association" "public" {
  for_each       = aws_subnet.public
  subnet_id      = each.value.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  for_each       = aws_subnet.private
  subnet_id      = each.value.id
  route_table_id = aws_route_table.private.id
}

resource "aws_route_table_association" "data" {
  for_each       = aws_subnet.data
```

```
  subnet_id      = each.value.id
  route_table_id = aws_route_table.data.id
}

# modules/network/outputs.tf
output "vpc_id"             { value = aws_vpc.this.id }
output "public_subnet_ids"  { value = [for s in aws_subnet.public : s.id] }
output "private_subnet_ids" { value = [for s in aws_subnet.private : s.id] }
output "data_subnet_ids"    { value = [for s in aws_subnet.data : s.id] }
```

Call it from `envs/dev/main.tf` and run `terraform init && terraform plan`:

```hcl
module "network" {
  source = "../../modules/network"
  name   = "shop-api-dev"
}
```

Expected plan (similar to): `Plan: 19 to add, 0 to change, 0 to destroy.` (the exact count varies with `az_count`).

Milestone 2: security groups (topics 1, 5)

```hcl
# envs/dev/security.tf
resource "aws_security_group" "alb" {
  name_prefix = "shop-api-dev-alb-"
  description = "Public ALB"
  vpc_id      = module.network.vpc_id
  lifecycle { create_before_destroy = true }
}

resource "aws_security_group" "app" {
  name_prefix = "shop-api-dev-app-"
  description = "App servers"
  vpc_id      = module.network.vpc_id
  lifecycle { create_before_destroy = true }
}

resource "aws_security_group" "db" {
  name_prefix = "shop-api-dev-db-"
  description = "PostgreSQL"
  vpc_id      = module.network.vpc_id
  lifecycle { create_before_destroy = true }
}

resource "aws_security_group" "redis" {
  name_prefix = "shop-api-dev-redis-"
  description = "Redis"
```

```
  vpc_id      = module.network.vpc_id
  lifecycle { create_before_destroy = true }
}

resource "aws_vpc_security_group_ingress_rule" "alb_http" {
  security_group_id = aws_security_group.alb.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 80
  to_port           = 80
}

locals {
  # source SG ⇒ destination SG and port
  tiers = {
    app_from_alb    = { to = aws_security_group.app.id,   from =
  aws_security_group.alb.id, port = 8080 }
    db_from_app     = { to = aws_security_group.db.id,    from =
  aws_security_group.app.id, port = 5432 }
    redis_from_app  = { to = aws_security_group.redis.id, from =
  aws_security_group.app.id, port = 6379 }
  }
}

resource "aws_vpc_security_group_ingress_rule" "tier" {
  for_each                     = local.tiers
  security_group_id            = each.value.to
  referenced_security_group_id = each.value.from
  ip_protocol                  = "tcp"
  from_port                    = each.value.port
  to_port                      = each.value.port
}

resource "aws_vpc_security_group_egress_rule" "all" {
  for_each          = { alb = aws_security_group.alb.id, app = aws_security_group.app.id }
  security_group_id = each.value
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
}
```

Milestone 3: data tier, RDS and Redis (topics 1, 2, 3, 5)

HCL

```
# modules/data/variables.tf
variable "name"                { type = string }
variable "subnet_ids"          { type = list(string) }
variable "db_sg_id"            { type = string }
variable "redis_sg_id"         { type = string }
variable "db_instance_class" {
  type    = string
  default = "db.t4g.micro"
```

```
}
variable "redis_node_type" {
  type    = string
  default = "cache.t4g.micro"
}
variable "multi_az" {
  type    = bool
  default = false
}
variable "deletion_protection" {
  type    = bool
  default = false
}

# modules/data/main.tf
resource "aws_db_subnet_group" "this" {
  name       = "${var.name}-db"
  subnet_ids = var.subnet_ids
}

resource "aws_db_instance" "this" {
  identifier                  = "${var.name}-pg"
  engine                      = "postgres"
  engine_version              = "16"
  instance_class              = var.db_instance_class
  allocated_storage           = 20
  storage_type                = "gp3"
  storage_encrypted           = true
  db_name                     = "shop"
  username                    = "shopadmin"
  manage_master_user_password = true          # AWS creates and rotates the secret in
  Secrets Manager: no password in code
  db_subnet_group_name        = aws_db_subnet_group.this.name
  vpc_security_group_ids      = [var.db_sg_id]
  publicly_accessible         = false
  multi_az                    = var.multi_az
  backup_retention_period     = 1
  deletion_protection         = var.deletion_protection
  skip_final_snapshot         = !var.deletion_protection
  final_snapshot_identifier   = var.deletion_protection ? "${var.name}-final" : null

  lifecycle {
    ignore_changes = [engine_version]         # AWS applies minor upgrades; avoid
  perpetual drift
  }
}

resource "aws_elasticache_subnet_group" "this" {
  name       = "${var.name}-redis"
  subnet_ids = var.subnet_ids
}

resource "aws_elasticache_replication_group" "this" {
```

```
  replication_group_id       = "${var.name}-redis"
  description                = "${var.name} cache"
  engine                     = "redis"
  engine_version             = "7.1"
  node_type                  = var.redis_node_type
  num_cache_clusters         = var.multi_az ? 2 : 1
  automatic_failover_enabled = var.multi_az
  multi_az_enabled           = var.multi_az
  subnet_group_name          = aws_elasticache_subnet_group.this.name
  security_group_ids         = [var.redis_sg_id]
  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  port                       = 6379
}

# modules/data/outputs.tf
output "db_endpoint" { value = aws_db_instance.this.address }
output "db_secret_arn" { value = aws_db_instance.this.master_user_secret[0].secret_arn }
output "redis_endpoint" { value =
  aws_elasticache_replication_group.this.primary_endpoint_address }
```

Wire it in `envs/dev/main.tf`:

```hcl
module "data" {
  source      = "../../modules/data"
  name        = "shop-api-dev"
  subnet_ids  = module.network.data_subnet_ids
  db_sg_id    = aws_security_group.db.id
  redis_sg_id = aws_security_group.redis.id
}
```

RDS and ElastiCache take roughly 5-15 minutes to create. (Redis is the default engine here; AWS also offers Valkey, a Redis-compatible engine, in the same resource.)

Milestone 4: compute and load balancer (topics 1, 3, 5)

```hcl
# modules/app/variables.tf  (abbreviated: name, vpc_id, public_subnet_ids,
  private_subnet_ids,
# alb_sg_id, app_sg_id, db_endpoint, redis_endpoint, instance_type, instance_count)

# modules/app/main.tf
data "aws_ssm_parameter" "ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

data "aws_iam_policy_document" "ec2_assume" {
  statement {
```

```
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "app" {
  name               = "${var.name}-app"
  assume_role_policy = data.aws_iam_policy_document.ec2_assume.json
}

resource "aws_iam_role_policy_attachment" "ssm" {        # Session Manager access instead
  of SSH
  role       = aws_iam_role.app.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}

resource "aws_iam_instance_profile" "app" {
  name = "${var.name}-app"
  role = aws_iam_role.app.name
}

locals {
  placements = { for i in range(var.instance_count) : tostring(i) ⇒
  var.private_subnet_ids[i % length(var.private_subnet_ids)] }
  server_py  = <<-PY
    import json, socket
    from http.server import BaseHTTPRequestHandler, HTTPServer
    class H(BaseHTTPRequestHandler):
        def do_GET(self):
            body = json.dumps({"service": "shop-api", "host": socket.gethostname(),
                               "db": "${var.db_endpoint}", "cache":
  "${var.redis_endpoint}"}).encode()
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.end_headers()
            self.wfile.write(body)
    HTTPServer(("0.0.0.0", 8080), H).serve_forever()
  PY
}

resource "aws_instance" "app" {
  for_each               = local.placements
  ami                    = data.aws_ssm_parameter.ami.value
  instance_type          = var.instance_type
  subnet_id              = each.value
  vpc_security_group_ids = [var.app_sg_id]
  iam_instance_profile   = aws_iam_instance_profile.app.name

  metadata_options { http_tokens = "required" }        # IMDSv2 only
  root_block_device { encrypted = true }
```

```
  user_data_replace_on_change = true
  user_data = <<-EOT
    #!/bin/bash
    cat > /opt/server.py <<'PYEOF'
    ${indent(4, local.server_py)}
    PYEOF
    nohup python3 /opt/server.py >/var/log/shop-api.log 2>&1 &
  EOT

  tags = { Name = "${var.name}-app-${each.key}" }

  lifecycle {
    precondition {
      condition     = var.instance_count >= 1
      error_message = "instance_count must be at least 1."
    }
  }
}

resource "aws_lb" "this" {
  name               = "${var.name}-alb"
  load_balancer_type = "application"
  subnets            = var.public_subnet_ids
  security_groups    = [var.alb_sg_id]
}

resource "aws_lb_target_group" "app" {
  name     = "${var.name}-tg"
  port     = 8080
  protocol = "HTTP"
  vpc_id   = var.vpc_id
  health_check {
    path    = "/"
    matcher = "200"
  }
}

resource "aws_lb_target_group_attachment" "app" {
  for_each         = aws_instance.app
  target_group_arn = aws_lb_target_group.app.arn
  target_id        = each.value.id
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.this.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

```
output "alb_dns_name" { value = aws_lb.this.dns_name }
```

The user data starts a tiny Python server as a stand-in for the Node.js `shop-api` image; it proves routing and shows the database and cache endpoints the app would use. (ALB and target group names are limited to 32 characters. A production app would use HTTPS with an ACM certificate and an auto scaling group, see Auto Scaling §6.2.)

Wire the module in `envs/dev/main.tf` and add outputs:

HCL

```hcl
module "app" {
  source             = "../../modules/app"
  name               = "shop-api-dev"
  vpc_id             = module.network.vpc_id
  public_subnet_ids  = module.network.public_subnet_ids
  private_subnet_ids = module.network.private_subnet_ids
  alb_sg_id          = aws_security_group.alb.id
  app_sg_id          = aws_security_group.app.id
  db_endpoint        = module.data.db_endpoint
  redis_endpoint     = module.data.redis_endpoint
  instance_type      = "t3.micro"
  instance_count     = 2
}

output "url" {
  value = "http://${module.app.alb_dns_name}"
}
```

BASH

```bash
cd envs/dev && terraform init && terraform apply
curl -s "$(terraform output -raw url)"       # repeat several times
```

Expected after a few minutes: JSON containing `"service": "shop-api"` and a `host` that changes between requests as the ALB spreads load across the two instances.

Milestone 5: a second environment (topic 6)

Copy `envs/dev` to `envs/prod`, change the backend key to `prod/terraform.tfstate`, the names to `shop-api-prod`, and the values:

Setting	dev	prod
`vpc_cidr`	`10.0.0.0/16`	`10.1.0.0/16`
`instance_type` / `instance_count`	`t3.micro` / 2	`t3.small` / 2 (spread over 2 AZs)
`multi_az` (RDS and Redis)	`false`	`true`

Setting	dev	prod
`deletion_protection`	`false`	`true`

Run `terraform plan` in both directories. Do **not** apply `prod` unless you accept the extra cost and will destroy it right away (`deletion_protection` must be set to `false` and applied before `destroy` works, see section 7).

Milestone 6: pipeline with OIDC (topic 7)

Create the GitHub OIDC provider and plan/apply roles as in L7.7, then add `.github/workflows/terraform.yml` with `fmt` , `validate` , `tflint` , `checkov` , `plan` (PR comment) and `apply` (on `main` , `environment: dev`). Require reviews and status checks on `main` .

Milestone 7: operations drills (topics 8 and 9)

- **Locking:** start an apply, run a plan in a second terminal and read the lock error; then simulate a stuck lock and use `force-unlock` (L7.8).
- **Drift:** change an instance tag and add an ingress rule by hand, then run `terraform plan -detailed-exitcode` and add `drift.yml` (L7.9).

5 Tasks

- [ ] **T1 — Remote state and a clean network** *(≈90 min, intermediate; Milestones 0-1; topics 4, 8, 1, 3)*
  - **Goal:** build the VPC layer with locked remote state.
  - **Steps:**
    1. Create the state bucket and the `envs/dev` root with the S3 backend and `use_lockfile = true` .
    2. Write and apply `modules/network` .
    3. Check `aws s3 ls s3://<bucket>/dev/` and `aws ec2 describe-subnets --filters Name=tag:Project,Values=shop-api --query 'Subnets[].CidrBlock'` .
  - **Done when:** six subnets (3 tiers x 2 AZs) exist, `terraform plan` shows `No changes` , and the state object is in S3.
- [ ] **T2 — Layered security groups** *(≈60 min, intermediate; Milestone 2; topics 1, 5)*
  - **Goal:** allow traffic only along ALB, app, database and cache paths.
  - **Steps:**
    1. Add `security.tf` and apply.

2. List the rules with `aws ec2 describe-security-group-rules --filters Name=group-id,Values=<app-sg-id>`.
3. Try to add a fifth rule with `for_each` (for example SSM or a metrics port) without copying a block.

- **Done when:** the database and Redis groups have no CIDR-based rules, only references to the app group.

- [ ] **T3 — Data tier** *(≈90 min, advanced; Milestone 3; topics 2, 3, 5)*
  - **Goal:** private PostgreSQL and Redis with no secrets in code or state outputs.
  - **Steps:**
    1. Create `modules/data` and apply.
    2. Fetch the generated credentials with `aws secretsmanager get-secret-value --secret-id <db_secret_arn>`.
    3. Confirm `publicly_accessible` is false and that neither endpoint resolves to a public address from outside the VPC.
  - **Done when:** `terraform output db_endpoint redis_endpoint` shows private endpoints and no password appears in your `.tf` files.

- [ ] **T4 — Serve traffic through the ALB** *(≈90 min, intermediate; Milestone 4; topics 1, 3, 5)*
  - **Goal:** an end-to-end working request path.
  - **Steps:**
    1. Create `modules/app` and wire it in.
    2. `curl` the `url` output 10 times and note the changing `host`.
    3. Stop one instance (`aws ec2 stop-instances ...`) and watch the target group health with `aws elbv2 describe-target-health`.
  - **Done when:** the ALB keeps answering with 200 while one instance is stopped, and both hosts appear again after restart.

- [ ] **T5 — Promote to a second environment** *(≈60 min, intermediate; Milestone 5; topic 6)*
  - **Goal:** prove the code is reusable and states are isolated.
  - **Steps:**
    1. Create `envs/prod` with the differences from the table.
    2. Run `terraform plan` in `prod` and `diff -r envs/dev envs/prod`.
    3. Show that `terraform state list` in `dev` never contains prod addresses.
  - **Done when:** the plan for `prod` succeeds with different sizes and keys, and the `diff` shows only values.

- [ ] **T6 — Break it: concurrency and a stuck lock** *(≈45 min, advanced; Milestone 7; topic 8)*
  - **Goal:** recover safely from a lock.
  - **Steps:**
    1. Start `terraform apply` with a trivial change and run `terraform plan` in a second terminal.
    2. Kill the first run with `kill -9`, then clear the lock with `terraform force-unlock`.
    3. Run `terraform plan` to confirm state is consistent.
  - **Done when:** you captured the lock error, unlocked after verifying nobody was running, and the next plan is clean.
- [ ] **T7 — Detect and reconcile drift** *(≈60 min, advanced; Milestone 7; topic 9)*
  - **Goal:** find a manual change and resolve it deliberately.
  - **Steps:**
    1. Change the `Name` tag of one instance and the target group health-check path by hand in the console or CLI.
    2. Run `terraform plan -detailed-exitcode` and `terraform plan -refresh-only`.
    3. Revert one change with `apply` and adopt the other into code in a reviewed edit.
  - **Done when:** the exit code returns to `0` and you can explain for each change why you reverted or adopted it.
- [ ] **T8 — Prove it: PR to production, from memory** *(≈2 hours, advanced; Milestone 6; topics 5, 6, 7)*
  - **Goal:** operate the pipeline and explain it.
  - **Steps:**
    1. Without the guide, add a CloudWatch alarm (or a tag) via a pull request: confirm `fmt`, `validate`, `tflint`, `checkov` and `plan` pass and the plan comment appears.
    2. Merge, and watch `apply` assume the apply role through OIDC.
    3. Explain aloud: why no AWS keys are stored, how the plan role differs from the apply role, and where each of the nine topics appears in this project.
  - **Done when:** the change is live via the pipeline, and a teammate accepts your explanation of the OIDC flow and the state, lock and drift safeguards.

6 Verification checklist

- [ ] `terraform fmt -check -recursive` and `terraform validate` pass in every root.

- [ ] `terraform plan` shows `No changes` immediately after apply (`echo $?` is `0` with `-detailed-exitcode`).
- [ ] State for each environment lives in S3 at its own key, versioning is on, and `use_lockfile = true` is set.
- [ ] `curl http://<alb-dns>` returns JSON from at least two different hosts.
- [ ] From inside an instance (SSM Session Manager) `nc -zv <db_endpoint> 5432` succeeds and from your laptop it does not.
- [ ] No security group allows `0.0.0.0/0` except the ALB on port 80 (and egress).
- [ ] No password, key or token appears in `.tf` files, `terraform output`, or the repository (`git log -p | grep -i password` is empty).
- [ ] `checkov` and `tflint` pass, or every suppression has a written reason.
- [ ] A PR shows a plan comment and `apply` only runs from `main`.
- [ ] A deliberate manual change is detected by a drift check and resolved.

7 Teardown

This is the step that stops the billing. Do it the same day.

```bash
# prod only, if you created it: set deletion_protection = false in envs/prod/main.tf, then
cd envs/prod  && terraform apply && terraform destroy
cd ../dev     && terraform destroy
```

Destroy takes about 10-15 minutes (RDS and ElastiCache are slowest). Then verify nothing is left:

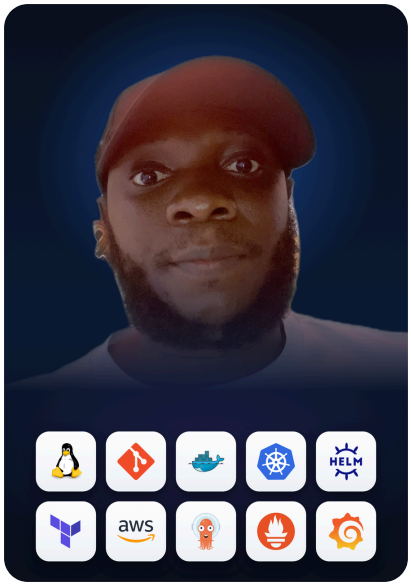
```bash
aws ec2 describe-instances --filters Name=tag:Project,Values=shop-api Name=instance-state-
  name,Values=running --query 'Reservations[].Instances[].InstanceId'
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'
aws rds describe-db-instances --query 'DBInstances[].DBInstanceIdentifier'
aws elasticache describe-replication-groups --query
  'ReplicationGroups[].ReplicationGroupId'
aws ec2 describe-nat-gateways --filter Name=state,Values=available --query
  'NatGateways[].NatGatewayId'
aws ec2 describe-addresses --query 'Addresses[].PublicIp'
```

All should be empty. Also delete the OIDC roles and provider if they were only for this exercise, delete any final RDS snapshot you created, and empty and delete the state bucket (including all object versions) when you finish the level. Check the Billing console the next day.

8 Stretch goals and what to learn next

- Replace `aws_instance` with a launch template and **auto scaling group**, and add HTTPS with ACM and a Route 53 record (Route 53 §6.12).
- Swap the hand-written network module for `terraform-aws-modules/vpc/aws` and compare interfaces.
- Add `terraform test` cases for the modules (L7.3) and an Infracost step to the PR comment.
- Store application config in SSM Parameter Store and read it from the instances with the instance profile.
- Add AWS Config rules and CloudTrail alerts for manual changes (see Drift Detection §7.9).
- Put prod in its own AWS account with its own state bucket and role.
- Next level: run the application on Kubernetes (Level 8 — Kubernetes §8.1) and package it with Helm, then manage clusters and workloads declaratively with GitOps (Level 17 §17.1).

About the Author



Ubong Eduok

Author and instructor, DevOps Roadmap: Beginner to Senior

Ubong Eduok created the DevOps Roadmap, the open-source, hands-on curriculum that this book sets in print, along with its companion **devops-deploy-toolkit** for deploying real applications with an AI pair-deployer.

The roadmap is free to read and released under the MIT License, so that anyone can use it, including in their own courses and projects.

GitHub: github.com/mascot200
Roadmap: github.com/mascot200/devops-roadmap
Toolkit: github.com/mascot200/devops-deploy-toolkit

COLOPHON

This book is set in Source Serif 4 for the text, Inter for headings, tables and diagrams, and JetBrains Mono for code, with DejaVu Sans and Noto Emoji supplying a few symbols. All are open-licensed typefaces. Trim size is 7.5 by 9.25 inches.

It was typeset directly from the Markdown of the repository at commit faf55fa, so the book and the repository can be kept in step. The 570 diagrams were drawn from their Mermaid sources as vector art. The staircase motif has one step for each of the twenty-one levels. The typesetting toolchain is in the `book/` folder of the repository, and you can rebuild the whole book from it with a single command.